BX26-1500-02
File No. S370-40

Systems

# BRICKS Transaction Server Application Programming Guide and Reference

**BRICKS Version 3.5.4**

m o s h i x

# Edition Notice

**Second Edition (October 2026)**

This edition replaces BX26-1500-01. It applies to Version 3.5.4 of the BRICKS Transaction Server (bricks) and to all subsequent versions and modifications until otherwise indicated in new editions. Changes are made periodically to the information herein; before using this publication to write or maintain application programs, consult the release notices distributed with the version of bricks installed at your installation.

This edition has been reorganized throughout. The command reference is now arranged by function in IBM order; the supplied transactions (CSSN, CEMT, CEDA, CECI, CEDF, ISPF and IDCAMS) are described in a part of their own; and the COBOL, PL/I and REXX language material is collected in one part for each language. Descriptions have been checked against the behavior of Version 3.5.4. Where the first edition and the program disagreed, this edition describes the program.

Information in this publication is subject to change. Any such change will be reported in a new edition, or in the release notes that accompany the program.

Screens reproduced in this publication were captured with `s3270` from a running bricks region, on a Model 2, a Model 3, or a Model 4 display station; bricks supports all three. The exact contents of the screens at a given installation depend on the resources the installation has defined, on the security group of the signed-on user, and on the activity of the region.

Requests for copies of this publication should be made to the administrator of your bricks region. A form for readers' comments is provided at the back of this publication.

**Copyright 2025, 2026 by moshix. All rights reserved.**

bricks is the community name of the BRICKS Transaction Server. Other product and company names mentioned herein, including IBM, CICS, MVS, z/OS, ISPF, VSAM, DB2 and Enterprise PL/I, are used for descriptive purposes only. bricks is not a product of, and is not endorsed by, IBM.

**Preface**

# About This Publication

This publication is a guide and reference for programmers who write application programs for the BRICKS Transaction Server (bricks). It describes the facilities that bricks provides to a transaction, the `EXEC CICS` and `EXEC SQL` commands through which a program uses them, the COBOL, PL/I and REXX languages as bricks implements them, and the supplied transactions with which a programmer defines, tests and diagnoses programs at a display station.

The installation, configuration and operation of a bricks region are described only where a programmer needs them, and are not described in detail. They are the subject of the operator documentation listed under "Related Publications".

## Who Should Read This Publication

This publication is intended for application programmers who design, write, test and maintain transactions that run under bricks, and for system programmers who support them. It assumes that the reader can program in at least one of COBOL, PL/I or REXX. Experience of IBM CICS is helpful but is not required: every command is described in full. A programmer who already knows CICS should read Part 1 for the bricks environment and then use Part 3 as a reference, paying attention to the notes headed "Differences from IBM CICS".

## How This Publication Is Organized

This publication is divided into nine parts and four appendixes.

**Part 1, "Concepts and Facilities"**, introduces bricks, its transactions, programs and region, describes the environment in which an `EXEC CICS` command runs, and defines the map language with which screens are described:


- Chapter 1, "Introduction to bricks"
- Chapter 2, "The EXEC CICS Command Environment"
- Chapter 3, "Defining Screens: The Map Language"


**Part 2, "Supplied Transactions"**, describes the transactions supplied with bricks with which a programmer signs on, controls the region, defines resources, issues commands interactively, traces a transaction, edits source and maintains files:

- Chapter 4, "Signing On and Off: CSSN and CSSF"
- Chapter 5, "CEMT: The Master Terminal Transaction"
- Chapter 6, "CEDA: Resource Definition Online"
- Chapter 7, "CECI: The Command-Level Interpreter"
- Chapter 8, "CEDF and CEDG: The Execution Diagnostic Facility"
- Chapter 9, "ISPF: The Source Editor"
- Chapter 10, "IDCAMS: Access Method Services"


The abend analyser CAID, which reads transaction dumps, is described with them in Chapter 16.

**Part 3, "EXEC CICS Command Reference"**, describes each command by function, then the Execute Interface Block, the response codes and conditions, and the commands that bricks does not implement:


- Chapter 11, "Terminal Control Commands"
- Chapter 12, "Program Control Commands"
- Chapter 13, "System Services Commands"
- Chapter 14, "File Control Commands"
- Chapter 15, "Temporary Storage and Transient Data Commands"
- Chapter 16, "Recovery and Condition Handling"
- Chapter 17, "The Execute Interface Block"
- Chapter 18, "Response Codes and Conditions"
- Chapter 19, "Commands Not Implemented"


**Part 4, "Web Services"**, describes the commands for programs that act as an HTTP server or client:


- Chapter 20, "Web Services: EXEC CICS WEB and DOCUMENT"


**Part 5, "Database Access"**, describes embedded SQL:


- Chapter 21, "Embedded SQL"


**Part 6, "COBOL"**, describes COBOL as bricks implements it:

- Chapter 22, "COBOL Source Format"
- Chapter 23, "COBOL DATA DIVISION"
- Chapter 24, "COBOL PROCEDURE DIVISION"
- Chapter 25, "The EIB and EXEC CICS in COBOL"
- Chapter 26, "COBOL Copybooks"
- Chapter 27, "COBOL Restrictions"


**Part 7, "PL/I"**, describes PL/I as bricks implements it:


- Chapter 28, "PL/I Program Structure"
- Chapter 29, "PL/I Data Declarations"
- Chapter 30, "PL/I Expressions, Conversions and Decimal Arithmetic"
- Chapter 31, "PL/I Statements and Control Flow"
- Chapter 32, "PL/I Built-in Functions"
- Chapter 33, "EXEC CICS in PL/I: Runtime Abends, the Unit of Work and %INCLUDE"
- Chapter 34, "PL/I Restrictions and Deviations"


**Part 8, "REXX"**, describes REXX as bricks implements it:


- Chapter 35, "REXX Program Structure"
- Chapter 36, "REXX Variables and Stems"
- Chapter 37, "REXX Control Flow"
- Chapter 38, "REXX Parsing"
- Chapter 39, "REXX Conditions and Condition Traps"
- Chapter 40, "REXX Operators and Built-in Functions"


**Part 9, "Sample Programs"**, describes the sample transactions supplied with bricks and works through complete examples:


- Chapter 41, "Supplied Sample Transactions"
- Chapter 42, "Worked Examples"


**The appendixes** give reference material:


- Appendix A, "Pitfalls and Idioms"
- Appendix B, "Quick Reference"
- Appendix C, "Abend Codes"
- Appendix D, "Configuration for the Application Programmer"

### Conventions Used in This Publication

- Keys are shown as **F3**, **ENTER**, **PA1**, and **CLEAR**. On most 3270 emulators, the keys **F13** through **F24** are produced by holding **Shift** while pressing **F1** through **F12**. Some screens label the program function keys **PF1**, **PF3** and so on; this publication refers to them as **F1**, **F3** and so on. In program text the keys keep the names that the language defines for them, such as `DFHPF3`.
- Commands, options, program text, data that the user types, and text that the system displays are shown in `THIS TYPEFACE`. Program listings are set in the same typeface, indented under a rule at the left.
- Screens captured from a running region are shown as numbered figures. Each caption indicates whether the screen was taken on a Model 2 (24 x 80), a Model 3 (32 x 80), or a Model 4 (43 x 80) display station, and names the file in which the capture is kept. Tables are numbered in the same sequence as the screens.
- The format of each command and statement is shown in a syntax block set between two rules. In a syntax block:
    - Words in upper case are keywords, and are coded as shown.
    - Words in lower case stand for a value that the program supplies; for example, `name`, `data-area` or `data-value`.
    - An item in brackets, `[ ]`, is optional.
    - Items in braces, `{ }`, and separated by a vertical bar, `|`, are alternatives, of which exactly one is coded.
    - An ellipsis, **`...`**, means that the item before it may be repeated.
- bricks implements its commands to match IBM CICS. Where the behavior of bricks differs from that of IBM CICS, the difference is stated in a note headed **Differences from IBM CICS** at the end of the description of the command concerned.

### Related Publications

The following documents are distributed with bricks, in the top-level directory of the distribution (`https://github.com/moshix/BRICKS_TS`), beside this publication, `programming_reference.pdf`:

- *README* (`README.md`): the operator's guide. Installation, starting and stopping a region, the `bricks.cnf` configuration file, security and users, and the operation of the supplied transactions.
- *Bricks ISPF Editor User Manual* (`ISPF_editor.md`): the complete description of the ISPF editor, its line commands, primary commands and function keys.
- *SQL in BRICKS* (`SQL_guide.md`): a tutorial on SQL support, from configuring a database through the CEDA DATABASE panel to monitoring SQL activity at runtime.
- *Multi-Region Operation* (`MRO.md`): how several bricks regions cooperate, and how a transaction is routed to the region that owns it.

# Contents

**Edition Notice** ..... **i**

**About This Publication** ..... **ii**

Who Should Read This Publication ..... ii

How This Publication Is Organized ..... ii

Conventions Used in This Publication ..... v

Related Publications ..... v

**Part 1. Concepts and Facilities** ..... **1-1**

**Chapter 1. Introduction to bricks** ..... **1-1**

Transactions and Programs ..... 1-1

Programming Languages ..... 1-3

Tasks and the Pseudo-conversational Model ..... 1-4

Terminals ..... 1-5

Security Groups ..... 1-6

The runtime Directory ..... 1-9

Where Definitions Live ..... 1-10

**Chapter 2. The EXEC CICS Command Environment** ..... **2-1**

Command Forms ..... 2-1

Arguments ..... 2-2

Responses and Conditions ..... 2-2

Length Options ..... 2-4

Character Data ..... 2-4

Summary of the Programming Model ..... 2-7

**Chapter 3. Defining Screens: The Map Language** ..... **3-1**

Map Files ..... 3-1

Rows, Columns and Field Layout ..... 3-1

Statements ..... 3-2

Attributes ..... 3-4

Character Set ..... 3-5

Example ..... 3-5

Changing a Field's Color at Runtime ..... 3-6

Coloring Single Characters ... 3-7
The Map Catalog ... 3-8
Selecting a Map by Mapset ... 3-8
Converting BMS Map Sources ... 3-9
Adapting to the Terminal Size ... 3-9

**Part 2. Supplied Transactions ... 4-1**

**Chapter 4. Signing On and Off: CSSN and CSSF ... 4-1**
Connecting to the Region ... 4-1
Signing On: CSSN ... 4-4
The Blank Screen ... 4-6
Signing Off: CSSF ... 4-10
Transaction Aliases ... 4-12

**Chapter 5. CEMT: The Master Terminal Transaction ... 5-1**
The CEMT Menu ... 5-1
INQUIRE ... 5-4
INQUIRE CONTROLBLOCKS ... 5-13
MONITOR ... 5-17
PERFORM ... 5-21

**Chapter 6. CEDA: Resource Definition Online ... 6-1**
The CEDA Menu ... 6-1
USER ... 6-3
TRANSACTION ... 6-6
PROGRAM ... 6-8
DATABASE ... 6-9
VSAM ... 6-12
URIMAP ... 6-13
WAPI and WEBSERVER ... 6-15
MRO ... 6-15
QUEUES ... 6-16

**Chapter 7. CECI: The Command-Level Interpreter ... 7-1**
The CECI Screen ... 7-1

Executing Commands ... 7-3
Units of Work in CECI ... 7-8
Commands Not Permitted ... 7-8

**Chapter 8. CEDF and CEDG: The Execution Diagnostic Facility ... 8-1**
Switching EDF On and Off ... 8-1
What EDF Intercepts ... 8-4
The EDF Display ... 8-5
Before and After a Command ... 8-8
Abends and Task Termination ... 8-16
Function Keys ... 8-18
The Auxiliary Displays ... 8-20
Stop Conditions ... 8-27
Changing the Program with CEDF ... 8-29

**Chapter 9. ISPF: The Source Editor ... 9-1**
The ISPF Menu ... 9-1
The File Browser ... 9-2
The Editor ... 9-4
Saving ... 9-9
Editing Several Files ... 9-10
Edit Locks ... 9-10

**Chapter 10. IDCAMS: Access Method Services ... 10-1**
The IDCAMS Screen ... 10-1
Control Statements ... 10-5
Condition Codes ... 10-7
Example ... 10-8
The idcams Command ... 10-8

**Part 3. EXEC CICS Command Reference ... 11-1**

**Chapter 11. Terminal Control Commands ... 11-1**
How bricks Uses the Terminal ... 11-1
SEND MAP ... 11-3
RECEIVE MAP ... 11-9

SEND TEXT ..... 11-11
SEND CONTROL ..... 11-13
RECEIVE ..... 11-15
CONVERSE ..... 11-16

**Chapter 12. Program Control Commands ..... 12-1**
Programs, Levels and the Unit of Work ..... 12-1
RETURN ..... 12-2
XCTL ..... 12-5
LINK ..... 12-8
ABEND ..... 12-10
START ..... 12-12
RETRIEVE ..... 12-16
CANCEL ..... 12-18

**Chapter 13. System Services Commands ..... 13-1**
ASSIGN ..... 13-1
ASKTIME ..... 13-3
FORMATTIME ..... 13-4
DELAY ..... 13-7
ADDRESS ..... 13-9
GETMAIN and FREEMAIN ..... 13-11
ENQ and DEQ ..... 13-13
QUERY SECURITY ..... 13-14
VERIFY PASSWORD ..... 13-16
INQUIRE SYSTEM ..... 13-18
INQUIRE and SET for Resources ..... 13-19
WRITE OPERATOR ..... 13-22

**Chapter 14. File Control Commands ..... 14-1**
Key-Sequenced Files in bricks ..... 14-1
Summary of the Commands ..... 14-2
READ ..... 14-2
WRITE ..... 14-4
REWRITE ..... 14-5

DELETE ..... 14-7
UNLOCK ..... 14-8
Record Locks ..... 14-9
Browsing a File ..... 14-10
STARTBR ..... 14-11
READNEXT ..... 14-12
READPREV ..... 14-13
RESETBR ..... 14-14
ENDBR ..... 14-14
Sequential Files in tmp_dir ..... 14-15

**Chapter 15. Temporary Storage and Transient Data Commands ..... 15-1**
The Two Kinds of Queue ..... 15-1
READQ TS ..... 15-2
WRITEQ TS ..... 15-3
DELETEQ TS ..... 15-5
READQ TD ..... 15-5
WRITEQ TD ..... 15-7
DELETEQ TD ..... 15-8
The tmp_dir Directory ..... 15-9

**Chapter 16. Recovery and Condition Handling ..... 16-1**
Units of Work ..... 16-1
Commit Points ..... 16-2
Backout Points ..... 16-3
Locks and the Unit of Work ..... 16-3
XCTL and the Unit of Work ..... 16-4
SYNCPOINT ..... 16-4
SYNCPOINT ROLLBACK ..... 16-6
Conditions ..... 16-7
Unhandled Conditions ..... 16-8
HANDLE CONDITION ..... 16-9
IGNORE CONDITION ..... 16-11
HANDLE AID ..... 16-12

HANDLE ABEND ..... 16-13
Abends ..... 16-17
How an Abend Ends a Task ..... 16-17
Transaction Dumps ..... 16-20
Runaway Tasks ..... 16-28
CAID: The Abend Analyser ..... 16-29

**Chapter 17. The Execute Interface Block ..... 17-1**
How Each Language Sees the EIB ..... 17-1
EIB Fields ..... 17-2
The EIB in COBOL ..... 17-3
The EIB in PL/I ..... 17-4
The EIB in REXX ..... 17-5
Differences from IBM CICS ..... 17-6

**Chapter 18. Response Codes and Conditions ..... 18-1**
Testing the Response ..... 18-1
Responses ..... 18-2
Responses Defined but Not Returned ..... 18-4
Secondary Responses ..... 18-5

**Chapter 19. Commands Not Implemented ..... 19-1**
What Happens When a Program Uses One ..... 19-1
Commands Not Implemented ..... 19-2
Unsupported Forms of Supported Commands ..... 19-3
Options Accepted Without Effect ..... 19-3
Commands Described Elsewhere ..... 19-4

**Part 4. Web Services ..... 20-1**

**Chapter 20. Web Services: EXEC CICS WEB and DOCUMENT ..... 20-1**
How an HTTP Request Becomes a Task ..... 20-1
Defining Routes: web_routes.conf ..... 20-4
Static Files on the Web Listener ..... 20-7
Authentication and Authorization ..... 20-7
Code Page Conversion of Bodies ..... 20-9

Server Command Reference ..... 20-11
WEB EXTRACT ..... 20-12
WEB EXTRACT CERTIFICATE ..... 20-14
WEB READ HTTPHEADER, QUERYPARM and FORMFIELD ..... 20-15
WEB STARTBROWSE, READNEXT and ENDBROWSE ..... 20-17
WEB RECEIVE (server) ..... 20-18
WEB RETRIEVE ..... 20-20
WEB WRITE HTTPHEADER (server) ..... 20-20
WEB SEND (server) ..... 20-20
WEB PARSE URL ..... 20-22
WEB CONVERTTIME ..... 20-22
The HTTP Client ..... 20-23
URIMAP Definitions ..... 20-24
WEB OPEN ..... 20-25
WEB CONVERSE ..... 20-26
WEB SEND and WEB RECEIVE (client) ..... 20-30
WEB SEND URIMAP ..... 20-30
WEB CLOSE ..... 20-31
Client Request and Response Headers ..... 20-31
WEB EXTRACT SESSTOKEN ..... 20-33
WEB EXTRACT URIMAP ..... 20-33
The DOCUMENT Commands ..... 20-34
DOCUMENT CREATE ..... 20-34
DOCUMENT INSERT ..... 20-35
DOCUMENT SET ..... 20-36
DOCUMENT RETRIEVE ..... 20-36
DOCUMENT DELETE ..... 20-36
Copybooks for Web Programs ..... 20-37
Sample Web Transactions ..... 20-39
Summary of Differences from IBM CICS ..... 20-40

**Part 5. Database Access ..... 21-1**

**Chapter 21. Embedded SQL ..... 21-1**

Connecting a Transaction to a Database ... 21-1
Coding EXEC SQL Statements ... 21-2
Host Variables ... 21-3
Null Indicators ... 21-5
SELECT INTO ... 21-5
INSERT, UPDATE and DELETE ... 21-6
Cursors ... 21-7
WHENEVER ... 21-11
CONNECT TO ... 21-12
Units of Work and Commit Points ... 21-13
COMMIT and ROLLBACK ... 21-14
Refused Commits ... 21-15
A Failed Statement Spoils the Transaction ... 21-16
Time Limits and Automatic Retry ... 21-17
Database Abends ... 21-18
The SQL Communication Area ... 21-19
SQLCODE Values ... 21-20
Examples in Each Language ... 21-22
Restrictions ... 21-24
Summary of Differences from DB2 under CICS ... 21-25

**Part 6. COBOL ... 22-1**

**Chapter 22. COBOL Source Format ... 22-1**
Program Structure ... 22-2
Reference Format ... 22-3
Literals ... 22-4
Character Set ... 22-6
Compile-Time Diagnostics ... 22-8
Differences from IBM Enterprise COBOL ... 22-8

**Chapter 23. COBOL DATA DIVISION ... 23-1**
Data Description Entry ... 23-1
Level Numbers ... 23-2

PICTURE Clause ..... 23-2
USAGE Clause ..... 23-5
Storage of Signed DISPLAY Items ..... 23-7
VALUE Clause ..... 23-7
OCCURS Clause ..... 23-7
FILLER ..... 23-8
REDEFINES Clause ..... 23-8
Condition-Names (Level 88) ..... 23-10
Qualification ..... 23-11
LINKAGE SECTION ..... 23-11

**Chapter 24. COBOL PROCEDURE DIVISION ..... 24-1**
Sentences and Scope Terminators ..... 24-1
Statement Summary ..... 24-1
Runtime Errors ..... 24-2
Arithmetic Statements ..... 24-2
COMPUTE ..... 24-3
ROUNDED and ON SIZE ERROR ..... 24-3
CONTINUE ..... 24-4
DISPLAY ..... 24-4
EVALUATE ..... 24-5
EXIT ..... 24-6
GO TO ..... 24-7
GOBACK and STOP RUN ..... 24-7
IF ..... 24-7
INSPECT ..... 24-8
JSON GENERATE ..... 24-9
JSON PARSE ..... 24-11
MOVE ..... 24-15
PERFORM ..... 24-16
SET (Pointers) ..... 24-17
STRING ..... 24-19
UNSTRING ..... 24-20

Conditions ..... 24-21
Reference Modification ..... 24-24
Intrinsic Functions ..... 24-24
Differences from IBM Enterprise COBOL ..... 24-26

**Chapter 25. The EIB and EXEC CICS in COBOL ..... 25-1**
The EXEC CICS Statement ..... 25-1
Condition Handling ..... 25-3
The EXEC Interface Block ..... 25-5
Automatically Declared Items ..... 25-7
The COMMAREA ..... 25-7
Ending the Program ..... 25-8
Differences from IBM CICS ..... 25-9

**Chapter 26. COBOL Copybooks ..... 26-1**
COPY Statement ..... 26-1
The Dual-Name Convention ..... 26-2
Supplied Copybooks ..... 26-2
DFHAID ..... 26-3
DFHRESP ..... 26-3
SQLCA ..... 26-5
DFHCOLOR ..... 26-6
DFHVALUE ..... 26-7
Web and Document Copybooks ..... 26-8
Using Named Constants ..... 26-9
Differences from IBM Enterprise COBOL ..... 26-10

**Chapter 27. COBOL Restrictions ..... 27-1**
Program Structure ..... 27-1
DATA DIVISION ..... 27-2
PROCEDURE DIVISION ..... 27-3
Limits ..... 27-5
Summary of Behavioral Differences ..... 27-5

**Part 7. PL/I ..... 28-1**

**Chapter 28. PL/I Program Structure** ..... **28-1**
How to Read the Syntax Diagrams ..... 28-1
The PL/I Program ..... 28-2
Source Rules ..... 28-3
The PL/I Character Set ..... 28-5
Procedures ..... 28-6
Where Programs Are Kept ..... 28-7
Checking and Editing Programs ..... 28-8
Program Caching and Task Storage ..... 28-9

**Chapter 29. PL/I Data Declarations** ..... **29-1**
DECLARE ..... 29-1
Data Types ..... 29-2
Picture Characters ..... 29-3
Structures and Arrays ..... 29-4
INIT, VALUE and Default Values ..... 29-5
Storage Classes ..... 29-5
Storage Layout and COMMAREA Exchange with COBOL ..... 29-6
Differences from IBM Enterprise PL/I ..... 29-8

**Chapter 30. PL/I Expressions, Conversions and Decimal Arithmetic** ..... **30-1**
Operators and Precedence ..... 30-1
Fixed-Decimal Precision ..... 30-2
Division ..... 30-4
Mixed Binary and Decimal Operands ..... 30-4
Truncation and ROUND ..... 30-4
Conversions ..... 30-5
Validating Data Before Conversion ..... 30-6
Differences from IBM Enterprise PL/I ..... 30-7

**Chapter 31. PL/I Statements and Control Flow** ..... **31-1**
Assignment ..... 31-1
CALL ..... 31-2
DO ..... 31-3

External Entries ... 31-3
IF ... 31-4
LEAVE and ITERATE ... 31-5
The Null Statement ... 31-5
PUT LIST ... 31-6
RETURN ... 31-6
SELECT ... 31-7
Differences from IBM Enterprise PL/I ... 31-7

**Chapter 32. PL/I Built-in Functions ... 32-1**
String Functions ... 32-2
Arithmetic Functions ... 32-3
Conversion and Validation Functions ... 32-4
Aggregate and Storage Functions ... 32-5
Date and Time Functions ... 32-5
CICS Constants and the ABEND Subroutine ... 32-6
Differences from IBM Enterprise PL/I ... 32-7

**Chapter 33. EXEC CICS in PL/I: Runtime Abends, the Unit of Work and %INCLUDE ... 33-1**
Runtime Abends ... 33-1
CALL ABEND ... 33-3
Errors Detected at Compile Time ... 33-4
The Unit of Work ... 33-5
HANDLE ABEND in PL/I ... 33-6
Preventing Conditions Instead of Handling Them ... 33-7
EXEC CICS ... 33-7
Options Are PL/I Expressions ... 33-8
RESP, RESP2 and DFHRESP ... 33-9
Unhandled Conditions ... 33-10
The EIB and Other Predeclared Names ... 33-11
Maps as Structures ... 33-13
COMMAREA and DFHCOMMAREA ... 33-14
LINK, XCTL and RETURN Across Languages ... 33-15

START and RETRIEVE ... 33-16
HANDLE CONDITION, HANDLE AID and HANDLE ABEND ... 33-17
Commands That Return Addresses ... 33-18
EXEC SQL in PL/I ... 33-18
%INCLUDE ... 33-19
Supplied Members: DFHAID and DFHRESP ... 33-20

**Chapter 34. PL/I Restrictions and Deviations ... 34-1**
Deviations from Enterprise PL/I ... 34-1
Features Not Supported ... 34-2
Errors Detected at Compile Time ... 34-3
Limits ... 34-3

**Part 8. REXX ... 35-1**

**Chapter 35. REXX Program Structure ... 35-1**
Form of a Program ... 35-1
Clauses, Labels and Continuation ... 35-2
Comments ... 35-2
Literal Strings ... 35-2
Character Set ... 35-3
Keyword Instructions ... 35-5
Internal Routines ... 35-6
ADDRESS and Commands ... 35-8
Special Variables ... 35-9
The EXEC CICS and EXEC SQL Preprocessor ... 35-9

**Chapter 36. REXX Variables and Stems ... 36-1**
Symbols ... 36-1
Simple Variables and NOVALUE ... 36-1
Stems ... 36-1
Derived Names of Compound Symbols ... 36-2
DROP ... 36-3
PROCEDURE EXPOSE ... 36-3

**Chapter 37. REXX Control Flow ... 37-1**

IF ... 37-1
SELECT ... 37-1
DO ... 37-2
LEAVE and ITERATE ... 37-4
CALL ... 37-4
RETURN and EXIT ... 37-4
SIGNAL ... 37-5
INTERPRET ... 37-5
NUMERIC ... 37-6
NOP and Commands ... 37-6

**Chapter 38. REXX Parsing ... 38-1**
PARSE ... 38-1
Parsing into Words ... 38-2
Examples ... 38-3

**Chapter 39. REXX Conditions and Condition Traps ... 39-1**
SIGNAL ON and SIGNAL OFF ... 39-1
Conditions ... 39-2
Unhandled CICS Conditions ... 39-3
Abends That End a REXX Task ... 39-4

**Chapter 40. REXX Operators and Built-in Functions ... 40-1**
Operators ... 40-1
Concatenation ... 40-2
Calling a Function ... 40-2
String Functions ... 40-4
Word Functions ... 40-6
Conversion and Bit Functions ... 40-7
Arithmetic Functions ... 40-8
Information Functions ... 40-9
Stream Functions ... 40-10
JSON Functions (bricks Extension) ... 40-12

**Part 9. Sample Programs ... 41-1**

**Chapter 41. Supplied Sample Transactions** **41-1**
Sample Definitions 41-1
REXX Samples 41-2
COBOL Samples 41-6
PL/I Samples 41-9
The Abend Samples 41-9
Sample Screens 41-10

**Chapter 42. Worked Examples** **42-1**
Example 1. Producer and Consumer on a TS Queue 42-1
Example 2. SEND TEXT and RECEIVE Without a Map 42-2
Example 3. Browsing a File 42-3
Example 4. LINK With a COMMAREA 42-4
Example 5. Sequential Import with READQ TD and WRITE 42-4
Example 6. Menu Actions Controlled by QUERY SECURITY 42-6
Example 7. Porting CUST to PL/I 42-7

**Appendix A. Pitfalls and Idioms** **A-1**
All Languages A-1
REXX A-1
COBOL and PL/I A-3
COBOL A-4
PL/I A-4

**Appendix B. Quick Reference** **B-1**
Terminal Control (Chapter 11) B-1
Program Control (Chapter 12) B-2
System Services (Chapter 13) B-3
File Control (Chapter 14) B-4
Queues (Chapter 15) B-4
Recovery and Condition Handling (Chapter 16) B-5
EXEC SQL (Chapter 21) B-6
Web Services (Chapter 20) B-7
EIB Fields (Chapter 17) B-8

Copybooks and Include Members (Chapters 26 and 33) ..... B-8
REXX (Chapters 35 to 40) ..... B-9
PL/I (Chapters 28 to 34) ..... B-10

**Appendix C. Abend Codes ..... C-1**
Condition Abends ..... C-1
Condition Abend Codes ..... C-1
Conditions That Never Abend ..... C-3
When Condition Abends Occur ..... C-4
Preventing a Condition Abend ..... C-5
Catching a Condition Abend ..... C-5
Other Transaction Abend Codes ..... C-6
When Each Code Occurs ..... C-8
Preventing the Other Abends ..... C-13
Catching the Other Abends ..... C-14
Codes bricks Does Not Raise ..... C-14

**Appendix D. Configuration for the Application Programmer ..... D-1**
The runtime Directory ..... D-1
The bricks.cnf File ..... D-3
The Resource Definition Files ..... D-7
The transactions.conf File ..... D-8
The users.conf File ..... D-9
The aliases.conf File ..... D-9
The databases.conf File ..... D-9
The web_routes.conf File ..... D-10
The mro.conf File ..... D-10

# Figures


Figure 1-1. Fields of a transactions.conf definition ..... 1-2
Figure 1-2. Supported programming languages ..... 1-3
Figure 1-3. 3270 display station models ..... 1-6
Figure 1-4. Transaction access-control lists ..... 1-7
Figure 1-5. Built-in transactions ..... 1-9
Figure 1-6. Directories used by applications ..... 1-9
Figure 1-7. Resource definition files ..... 1-10
Figure 2-1. The graphic plane of bricks SBCS ..... 2-4
Figure 2-2. Where character data is converted ..... 2-6
Figure 2-3. The bricks programming model ..... 2-7
Figure 3-1. Map field attributes ..... 3-4
Figure 3-2. Map name suffixes by terminal model ..... 3-9
Figure 3-3. HELO on a Model 2 terminal: map HELO1 ..... 3-11
Figure 3-4. HELO on a Model 4 terminal: map HELO1L ..... 3-12
Figure 3-5. Sized map families supplied with bricks ..... 3-13
Figure 4-1. The supplied transactions and their group requirements ..... 4-1
Figure 4-2. The logo splash painted when a terminal connects ..... 4-2
Figure 4-3. The sign-on prompt of a terminal that is not signed on ..... 4-3
Figure 4-4. The CSSN sign-on panel ..... 4-4
Figure 4-5. CSSN sign-on messages ..... 4-5
Figure 4-6. Sign-on complete ..... 4-6
Figure 4-7. The blank screen ..... 4-7
Figure 4-8. A transaction identifier and argument typed on the blank screen ..... 4-7
Figure 4-9. An undefined transaction identifier ..... 4-9
Figure 4-10. A supplied transaction refused for want of a group ..... 4-10
Figure 4-11. CSSF without an argument ..... 4-11
Figure 4-12. The splash after CSSF LOGOFF ..... 4-12
Figure 5-1. The CEMT menu displayed to an administrator ..... 5-1
Figure 5-2. The CEMT menu displayed to a user who is not an administrator ..... 5-3
Figure 5-3. The INQUIRE menu ..... 5-4
Figure 5-4. Keys on the CEMT list panels ..... 5-5
Figure 5-5. INQUIRE TS ..... 5-5

Figure 5-6. INQUIRE FILE ..... 5-6
Figure 5-7. INQUIRE USER ..... 5-7
Figure 5-8. INQUIRE TERMINAL ..... 5-8
Figure 5-9. INQUIRE TRANSACTION ..... 5-9
Figure 5-10. INQUIRE TRANSACTION on a Model 4 screen ..... 5-10
Figure 5-11. INQUIRE MRO in a single region ..... 5-11
Figure 5-12. INQUIRE MRO status values ..... 5-12
Figure 5-13. INQUIRE ENQ with no enqueues held ..... 5-12
Figure 5-14. The CONTROLBLOCKS menu ..... 5-14
Figure 5-15. Terminal control blocks ..... 5-14
Figure 5-16. User control blocks ..... 5-15
Figure 5-17. Transaction control blocks ..... 5-16
Figure 5-18. File control blocks ..... 5-16
Figure 5-19. MONITOR: process metrics ..... 5-17
Figure 5-20. The DB Transaction Monitor ..... 5-19
Figure 5-21. The VSAM File Monitor ..... 5-21
Figure 5-22. The PERFORM menu ..... 5-22
Figure 5-23. PERFORM MAPCACHE ..... 5-23
Figure 5-24. The PERFORM RESCAN menu ..... 5-24
Figure 5-25. PERFORM RESCAN TRANS ..... 5-25
Figure 5-26. PERFORM RESCAN MAP ..... 5-26
Figure 5-27. PERFORM RESCAN PROGRAMS ..... 5-27
Figure 5-28. PERFORM PROGRAMCACHE ..... 5-28
Figure 5-29. PERFORM METRICS with the endpoint disabled ..... 5-29
Figure 6-1. CEDA panels and the files they maintain ..... 6-1
Figure 6-2. The CEDA menu ..... 6-2
Figure 6-3. CEDA USER ..... 6-4
Figure 6-4. CEDA DEFINE USER ..... 6-5
Figure 6-5. CEDA TRANSACTION ..... 6-6
Figure 6-6. CEDA DEFINE TRANSACTION ..... 6-7
Figure 6-7. CEDA ALTER TRANSACTION ..... 6-8
Figure 6-8. CEDA PROGRAM ..... 6-9
Figure 6-9. CEDA DATABASE ..... 6-10

Figure 6-10. CEDA DATABASE actions ..... 6-11
Figure 6-11. The tables of a database ..... 6-12
Figure 6-12. CEDA VSAM ..... 6-13
Figure 6-13. CEDA URIMAP ..... 6-14
Figure 6-14. CEDA DEFINE URIMAP ..... 6-15
Figure 6-15. CEDA MRO ..... 6-16
Figure 6-16. CEDA QUEUES ..... 6-17
Figure 6-17. Confirming the deletion of a queue ..... 6-18
Figure 7-1. The CECI screen before a command is entered ..... 7-1
Figure 7-2. CECI function keys ..... 7-2
Figure 7-3. ASKTIME executed in CECI ..... 7-3
Figure 7-4. ASSIGN executed in CECI ..... 7-4
Figure 7-5. READ FILE executed in CECI ..... 7-5
Figure 7-6. READQ TS executed in CECI ..... 7-5
Figure 7-7. A command that raises NOTFND ..... 7-6
Figure 7-8. A command with a required option missing ..... 7-7
Figure 7-9. CECI responses other than conditions ..... 7-7
Figure 7-10. Commands that CECI refuses ..... 7-9
Figure 8-1. EDF mode switched on with CEDF ..... 8-2
Figure 8-2. EDF mode switched on with CEDG ..... 8-2
Figure 8-3. Forms of the CEDF and CEDG commands ..... 8-3
Figure 8-4. EDF mode switched off ..... 8-3
Figure 8-5. EDF interception points ..... 8-4
Figure 8-6. PROGRAM INITIATION ..... 8-5
Figure 8-7. PROGRAM INITIATION on a Model 4 screen ..... 8-7
Figure 8-8. Colours of the EDF displays ..... 8-8
Figure 8-9. ABOUT TO EXECUTE COMMAND ..... 8-9
Figure 8-10. ABOUT TO EXECUTE COMMAND for SEND MAP ..... 8-10
Figure 8-11. COMMAND EXECUTION COMPLETE ..... 8-10
Figure 8-12. Forecasts on the RESPONSE row ..... 8-11
Figure 8-13. A condition that will abend the task ..... 8-12
Figure 8-14. ABOUT TO EXECUTE COMMAND on a Model 4 screen ..... 8-13
Figure 8-15. COMMAND EXECUTION COMPLETE on a Model 4 screen ..... 8-14

Figure 8-16. ABOUT TO EXECUTE COMMAND under CEDG, REXX program ..... 8-15
Figure 8-17. COMMAND EXECUTION COMPLETE under CEDG ..... 8-15
Figure 8-18. AN ABEND HAS OCCURRED ..... 8-16
Figure 8-19. ABNORMAL TASK TERMINATION ..... 8-17
Figure 8-20. TASK TERMINATION ..... 8-18
Figure 8-21. EDF function keys ..... 8-20
Figure 8-22. WORKING STORAGE ..... 8-21
Figure 8-23. WORKING STORAGE in hexadecimal ..... 8-22
Figure 8-24. Working storage by language ..... 8-22
Figure 8-25. WORKING STORAGE on a Model 4 screen ..... 8-23
Figure 8-26. EIB DISPLAY ..... 8-24
Figure 8-27. COMMAND EXECUTION COMPLETE in hexadecimal ..... 8-25
Figure 8-28. SOURCE VIEW ..... 8-26
Figure 8-29. SOURCE VIEW with comment lines hidden ..... 8-27
Figure 8-30. STOP CONDITIONS ..... 8-28
Figure 8-31. EDF stop conditions ..... 8-29
Figure 8-32. Fields that CEDF can change ..... 8-30
Figure 8-33. An abend requested with F12 ..... 8-31
Figure 9-1. The ISPF EDIT UTILITY menu ..... 9-1
Figure 9-2. ISPF areas ..... 9-2
Figure 9-3. The FILE BROWSER for the COBOL area ..... 9-2
Figure 9-4. The ISPF NEW FILE panel ..... 9-4
Figure 9-5. EDIT of a COBOL program ..... 9-5
Figure 9-6. EDIT on a Model 4 screen ..... 9-6
Figure 9-7. ISPF editor function keys ..... 9-7
Figure 9-8. ISPF editor primary commands ..... 9-8
Figure 9-9. ISPF editor line commands ..... 9-9
Figure 10-1. The IDCAMS screen ..... 10-1
Figure 10-2. IDCAMS function keys ..... 10-2
Figure 10-3. Control statements typed in SYSIN ..... 10-3
Figure 10-4. The listing after LISTCAT and PRINT ..... 10-4
Figure 10-5. The PRINT output, scrolled forward ..... 10-4
Figure 10-6. IDCAMS condition codes ..... 10-7

Figure 11-1. SEND MAP options ..... 11-5
Figure 11-2. The three ways of painting a map ..... 11-6
Figure 11-3. SEND MAP conditions ..... 11-6
Figure 11-4. HELC: SEND MAP of map HELO1 from a COBOL program ..... 11-8
Figure 11-5. RECEIVE MAP options ..... 11-10
Figure 11-6. RECEIVE MAP conditions ..... 11-10
Figure 11-7. SEND TEXT options ..... 11-12
Figure 11-8. SEND TEXT conditions ..... 11-13
Figure 11-9. SEND CONTROL options ..... 11-14
Figure 11-10. SEND CONTROL conditions ..... 11-14
Figure 11-11. RECEIVE options ..... 11-15
Figure 11-12. RECEIVE conditions ..... 11-16
Figure 11-13. CONVERSE options ..... 11-17
Figure 12-1. Program control at each logical level ..... 12-1
Figure 12-2. RETURN options ..... 12-2
Figure 12-3. RETURN conditions ..... 12-4
Figure 12-4. XCTL options ..... 12-5
Figure 12-5. XCTL conditions ..... 12-7
Figure 12-6. LINK options ..... 12-8
Figure 12-7. LINK conditions ..... 12-10
Figure 12-8. ABEND options ..... 12-11
Figure 12-9. START options ..... 12-14
Figure 12-10. When a started transaction runs ..... 12-14
Figure 12-11. START conditions ..... 12-16
Figure 12-12. RETRIEVE options ..... 12-17
Figure 12-13. RETRIEVE conditions ..... 12-17
Figure 12-14. CANCEL options ..... 12-18
Figure 12-15. CANCEL conditions ..... 12-19
Figure 13-1. System services commands ..... 13-1
Figure 13-2. ASSIGN options ..... 13-3
Figure 13-3. ASSIGN conditions ..... 13-3
Figure 13-4. ASKTIME options ..... 13-4
Figure 13-5. FORMATTIME options ..... 13-6

Figure 13-6. FORMATTIME conditions ..... 13-7
Figure 13-7. DELAY options ..... 13-8
Figure 13-8. DELAY conditions ..... 13-8
Figure 13-9. ADDRESS options ..... 13-9
Figure 13-10. ADDRESS conditions ..... 13-10
Figure 13-11. GETMAIN and FREEMAIN options ..... 13-11
Figure 13-12. GETMAIN and FREEMAIN conditions ..... 13-12
Figure 13-13. ENQ and DEQ options ..... 13-13
Figure 13-14. ENQ and DEQ conditions ..... 13-14
Figure 13-15. QUERY SECURITY options ..... 13-15
Figure 13-16. How QUERY SECURITY derives access ..... 13-15
Figure 13-17. QUERY SECURITY conditions ..... 13-16
Figure 13-18. VERIFY PASSWORD options ..... 13-17
Figure 13-19. VERIFY PASSWORD conditions ..... 13-18
Figure 13-20. INQUIRE SYSTEM options ..... 13-19
Figure 13-21. INQUIRE SYSTEM conditions ..... 13-19
Figure 13-22. Resource attributes returned by INQUIRE ..... 13-21
Figure 13-23. INQUIRE and SET conditions ..... 13-21
Figure 13-24. WRITE OPERATOR options ..... 13-23
Figure 13-25. WRITE OPERATOR conditions ..... 13-23
Figure 14-1. File control commands ..... 14-2
Figure 14-2. READ conditions ..... 14-3
Figure 14-3. WRITE conditions ..... 14-5
Figure 14-4. REWRITE conditions ..... 14-6
Figure 14-5. DELETE conditions ..... 14-8
Figure 14-6. How long a record lock is held ..... 14-10
Figure 14-7. READNEXT and READPREV conditions ..... 14-13
Figure 14-8. File control commands on sequential files ..... 14-16
Figure 14-9. Conditions on sequential files ..... 14-17
Figure 15-1. Temporary storage and transient data compared ..... 15-1
Figure 15-2. READQ TS conditions ..... 15-3
Figure 15-3. WRITEQ TS conditions ..... 15-4
Figure 15-4. READQ TD conditions ..... 15-6

Figure 15-5. WRITEQ TD conditions ..... 15-8
Figure 16-1. Commit points ..... 16-2
Figure 16-2. Backout points ..... 16-3
Figure 16-3. How SYNCPOINT ROLLBACK undoes each change ..... 16-6
Figure 16-4. SYNCPOINT ROLLBACK conditions ..... 16-7
Figure 16-5. Order in which condition handling applies ..... 16-8
Figure 16-6. HANDLE AID conditions ..... 16-13
Figure 16-7. ABNH: a HANDLE ABEND LABEL exit catches abend AEIM ..... 16-16
Figure 16-8. DFHAC2206 abend message: ABNC ends with abend AEIM ..... 16-19
Figure 16-9. Abends that take a transaction dump ..... 16-21
Figure 16-10. Records of a transaction dump ..... 16-25
Figure 16-11. CAID refused to a user who is not an administrator ..... 16-30
Figure 16-12. The CAID input screen with the most recent dumps ..... 16-31
Figure 16-13. A dump selected in the list of recent dumps ..... 16-32
Figure 16-14. CAID analysis of a ZERODIVIDE abend (ABZP) ..... 16-33
Figure 16-15. CAID analysis of an unhandled NOTFND (ABNC, abend AEIM) ..... 16-35
Figure 16-16. CAID cannot name the cause of a user abend code (ABUR, abend USR1) ..... 16-36
Figure 16-17. The dump of ABZP displayed by CAID ..... 16-37
Figure 16-18. CAID function keys ..... 16-37
Figure 16-19. CAID: no dump for the task number typed ..... 16-38
Figure 16-20. CAID messages ..... 16-39
Figure 16-21. Abends that CAID diagnoses ..... 16-41
Figure 17-1. EIB fields that bricks sets ..... 17-3
Figure 18-1. Responses that bricks returns ..... 18-4
Figure 18-2. Responses defined for compatibility ..... 18-5
Figure 18-3. Secondary responses ..... 18-5
Figure 19-1. IBM CICS commands not implemented ..... 19-2
Figure 19-2. Unsupported forms and options ..... 19-3
Figure 19-3. Options accepted without effect ..... 19-4
Figure 19-4. Commands formerly listed as not implemented ..... 19-4
Figure 20-1. How the listener builds the HTTP response ..... 20-3
Figure 20-2. Responses produced by the web listener ..... 20-4

Figure 20-3. Fields of a route row in web_routes.conf ..... 20-5
Figure 20-4. Sign-on at a display station and on the web listener ..... 20-8
Figure 20-5. Conversion of a body received by the program ..... 20-10
Figure 20-6. Options of WEB EXTRACT ..... 20-13
Figure 20-7. Conditions of WEB EXTRACT ..... 20-13
Figure 20-8. Options of WEB EXTRACT CERTIFICATE ..... 20-14
Figure 20-9. Conditions of WEB EXTRACT CERTIFICATE ..... 20-15
Figure 20-10. Conditions of WEB READ ..... 20-16
Figure 20-11. Conditions of the WEB browse commands ..... 20-17
Figure 20-12. Conditions of WEB RECEIVE (server) ..... 20-19
Figure 20-13. Conditions of WEB SEND (server) ..... 20-21
Figure 20-14. Fields of a URIMAP row ..... 20-25
Figure 20-15. Conditions of WEB OPEN ..... 20-26
Figure 20-16. Conditions of WEB CONVERSE, WEB RECEIVE (client) and WEB SEND URIMAP ..... 20-29
Figure 20-17. Conditions of the client header commands ..... 20-32
Figure 20-18. Conditions of DOCUMENT INSERT ..... 20-35
Figure 20-19. Copybooks for web programs ..... 20-38
Figure 20-20. Supplied web sample transactions ..... 20-39
Figure 20-21. Web support: differences from IBM CICS ..... 20-40
Figure 21-1. Form of an EXEC SQL statement in each language ..... 21-2
Figure 21-2. Supported SQL statements ..... 21-3
Figure 21-3. Adjustment of column values ..... 21-4
Figure 21-4. Results of SELECT INTO ..... 21-6
Figure 21-5. Cursor errors ..... 21-9
Figure 21-6. WHENEVER conditions ..... 21-11
Figure 21-7. Abends for a refused commit ..... 21-16
Figure 21-8. Database abend codes ..... 21-18
Figure 21-9. Fields of the SQLCA ..... 21-19
Figure 21-10. SQLCODE values returned by bricks ..... 21-21
Figure 21-11. Embedded SQL restrictions ..... 21-24
Figure 21-12. Embedded SQL: differences from DB2 under CICS ..... 21-25
Figure 22-1. COBOL reference format rules ..... 22-4

Figure 22-2. Figurative constants ..... 22-5
Figure 22-3. The bricks graphic plane, X'80' to X'9F' ..... 22-6
Figure 23-1. Level numbers ..... 23-2
Figure 23-2. Unedited PICTURE strings ..... 23-3
Figure 23-3. Editing symbols ..... 23-4
Figure 23-4. Numeric editing examples ..... 23-5
Figure 23-5. USAGE clause values ..... 23-5
Figure 23-6. Length of binary items ..... 23-6
Figure 24-1. Supported statements ..... 24-1
Figure 24-2. JSON PARSE status codes ..... 24-13
Figure 24-3. JSON PARSE exception codes ..... 24-14
Figure 24-4. MOVE rules ..... 24-15
Figure 24-5. Relational operators ..... 24-22
Figure 24-6. Abbreviated combined relation conditions ..... 24-23
Figure 24-7. Intrinsic functions ..... 24-25
Figure 25-1. Fields of DFHEIB ..... 25-6
Figure 25-2. Automatically declared items ..... 25-7
Figure 26-1. Copybooks supplied in runtime/cobolcopy ..... 26-2
Figure 26-2. DFHAID constants ..... 26-3
Figure 26-3. DFHRESP constants ..... 26-4
Figure 26-4. SQLCA SQLCODE constants ..... 26-5
Figure 26-5. DFHCOLOR constants ..... 26-6
Figure 26-6. Web and document copybooks ..... 26-9
Figure 27-1. Unsupported program structure ..... 27-1
Figure 27-2. Unsupported DATA DIVISION elements ..... 27-2
Figure 27-3. Unsupported PROCEDURE DIVISION elements ..... 27-4
Figure 27-4. COBOL limits ..... 27-5
Figure 28-1. Syntax notation used in the PL/I chapters ..... 28-2
Figure 28-2. PL/I source rules ..... 28-4
Figure 28-3. Where PL/I programs and members are kept ..... 28-7
Figure 29-1. PL/I data types ..... 29-2
Figure 29-2. Picture characters ..... 29-3
Figure 29-3. Initial values of variables declared without INIT ..... 29-5

Figure 29-4. PL/I storage classes ..... 29-5
Figure 29-5. Storage layout of PL/I data types ..... 29-7
Figure 30-1. PL/I operator precedence ..... 30-1
Figure 30-2. Precision of fixed-decimal results ..... 30-2
Figure 30-3. Conversion rules ..... 30-5
Figure 30-4. Expression differences from Enterprise PL/I ..... 30-7
Figure 32-1. String built-in functions ..... 32-2
Figure 32-2. Arithmetic built-in functions ..... 32-3
Figure 32-3. Conversion and validation built-in functions ..... 32-4
Figure 32-4. Aggregate and storage built-in functions ..... 32-5
Figure 32-5. Date and time built-in functions ..... 32-5
Figure 32-6. CICS constant functions and the ABEND subroutine ..... 32-6
Figure 33-1. Abend codes of a PL/I task ..... 33-3
Figure 33-2. Errors that bricks detects at compile time ..... 33-5
Figure 33-3. How the unit of work of a PL/I task is settled ..... 33-6
Figure 33-4. Replacements for ON-units ..... 33-7
Figure 33-5. EIB fields declared in a PL/I program ..... 33-12
Figure 33-6. Supplied PL/I include members ..... 33-21
Figure 34-1. Deviations from Enterprise PL/I ..... 34-1
Figure 34-2. PL/I features not supported ..... 34-2
Figure 34-3. Limits of bricks PL/I ..... 34-3
Figure 35-1. Hexadecimal and binary strings ..... 35-3
Figure 35-2. Characters allowed in REXX source ..... 35-4
Figure 35-3. Supported keyword instructions ..... 35-5
Figure 35-4. IBM instructions not supported ..... 35-6
Figure 35-5. Special variables ..... 35-9
Figure 36-1. Derivation of compound names ..... 36-2
Figure 37-1. Forms of the DO instruction ..... 37-3
Figure 37-2. NUMERIC settings ..... 37-6
Figure 38-1. PARSE sources ..... 38-1
Figure 38-2. Template elements ..... 38-2
Figure 39-1. REXX conditions ..... 39-2
Figure 40-1. Operators in priority order ..... 40-1

Figure 40-2. Forms of concatenation ..... 40-2
Figure 40-3. String functions ..... 40-5
Figure 40-4. Word functions ..... 40-6
Figure 40-5. Conversion and bit functions ..... 40-7
Figure 40-6. Arithmetic functions ..... 40-8
Figure 40-7. Information functions ..... 40-9
Figure 40-8. DATE and TIME options ..... 40-10
Figure 40-9. Stream functions ..... 40-11
Figure 40-10. JSON functions ..... 40-12
Figure 41-1. REXX sample transactions ..... 41-5
Figure 41-2. COBOL sample transactions ..... 41-8
Figure 41-3. PL/I sample transactions ..... 41-9
Figure 41-4. Abend samples ..... 41-10
Figure 41-5. The CUST menu (REXX) ..... 41-11
Figure 41-6. The GUST menu (COBOL) ..... 41-11
Figure 41-7. The CUSP menu (PL/I) ..... 41-12
Figure 41-8. GUST customer detail in QUERY mode ..... 41-12
Figure 41-9. CUSL customer list, page 1 ..... 41-13
Figure 41-10. GUSL customer list, page 1 ..... 41-13
Figure 41-11. ABMR ends with abend ABM0 ..... 41-14
Figure 41-12. SPDB ends with abend ASP3 ..... 41-14
Figure 42-1. REXX and PL/I idioms compared ..... 42-9
Figure B-1. Terminal control commands ..... B-1
Figure B-2. Program control commands ..... B-2
Figure B-3. System service commands ..... B-3
Figure B-4. File control commands ..... B-4
Figure B-5. Queue commands ..... B-4
Figure B-6. Recovery commands ..... B-5
Figure B-7. EXEC SQL statements ..... B-6
Figure B-8. SQLCA fields ..... B-6
Figure B-9. Web and document commands ..... B-7
Figure B-10. EIB fields ..... B-8
Figure B-11. Copybooks and include members ..... B-8

Figure B-12. REXX forms ..... B-9
Figure B-13. PL/I forms ..... B-10
Figure C-1. Condition abend codes AEIA through AEI9 ..... C-2
Figure C-2. Condition abend codes AEYB through AEXK ..... C-3
Figure C-3. Common causes of condition abends ..... C-4
Figure C-4. ABNP (PL/I) ends with abend AEIM ..... C-5
Figure C-5. Other transaction abend codes ..... C-8
Figure C-6. ABMP ends with abend ABMB ..... C-9
Figure C-7. APCH ends with abend APCT ..... C-10
Figure C-8. SPLC ends with abend ASP7 ..... C-11
Figure C-9. IBM abend codes that bricks does not raise ..... C-14
Figure D-1. Default layout of the runtime directory ..... D-2
Figure D-2. bricks.cnf knobs: terminals, sign-on and the region ..... D-4
Figure D-3. bricks.cnf knobs: database ..... D-5
Figure D-4. bricks.cnf knobs: web listener ..... D-6
Figure D-5. bricks.cnf knobs: HTTP client ..... D-7
Figure D-6. Fields of a transactions.conf row ..... D-8

Part 1

# Concepts and Facilities

This part introduces bricks: its transactions, programs and languages, the region and its terminals, and where resources are defined. It then describes the environment in which a program issues `EXEC CICS` commands, and the map language with which the screens of a transaction are defined.

**Chapter 1. Introduction to bricks** ..... **1-1**

**Chapter 2. The EXEC CICS Command Environment** ..... **2-1**

**Chapter 3. Defining Screens: The Map Language** ..... **3-1**

Chapter 1

# Introduction to bricks

bricks is a transaction server for 3270 display stations. It runs application programs written in REXX, COBOL and PL/I under a command interface modeled on IBM CICS: programs issue `EXEC CICS` commands to paint screens, read and update files and queues, call one another, and schedule further work. This chapter introduces the resources that make up a bricks application, the way a transaction is started and ended, and the places where the definitions of those resources are kept.

The chapter is an overview. The `EXEC CICS` command environment is described in Chapter 2, the map language in Chapter 3, and each command in Part 3.

## Transactions and Programs

A **transaction** is a unit of work that a terminal operator, another program, or the scheduler can start. It is known by a four-character **transaction identifier** (TRANSID) of the letters A through Z and the digits 0 through 9. Each transaction names one **program**: a source file in one of the three supported languages. When the transaction is started, bricks runs its program as a new **task**.

Transactions are defined in the file `runtime/transactions.conf`, one definition per line. The CEDA transaction (see "TRANSACTION" in Chapter 6) maintains the file; it may also be edited by hand. A line has the following form:

```
TRANSID:LANG:PROGRAM[:GROUPS[:DATABASE[:TWASIZE]]][:[REGION]]
```

| Field | Meaning |
|---|---|
| TRANSID | Exactly four characters, A-Z and 0-9. The name of a supplied transaction (CEMT, CEDA, CECI, CEDF, CEDG, CAID, CSSN, CSSF, ISPF, IDCA) cannot be redefined; such a line is skipped. |
| LANG | `rexx`, `cobol` or `pli`. |
| PROGRAM | The source file, relative to the directory for the language (for example `hello.rexx` or `billing/invoice.cob`). |
| GROUPS | Optional access-control list: a comma-separated list of security groups. See "Security Groups". |
| DATABASE | Optional name of the database (a row of `runtime/databases.conf`) that the program's `EXEC SQL` statements use. When omitted, the first database defined is used. See "Connecting a Transaction to a Database" in Chapter 21. |
| TWASIZE | Optional size in bytes (0 through 32767) of the Transaction Work Area that each task of the transaction receives. Default 0 (no TWA). See "ADDRESS" in Chapter 13. |
| [REGION] | Optional last field in square brackets. The transaction is routed to the named peer region (multiregion operation) instead of being run locally. |

**Figure 1-1. Fields of a transactions.conf definition**

The fields are positional: to code a later field, code the earlier ones, empty if necessary (`RPT1:cobol:rpt1.cob:::512`). A line with more than six fields, a malformed TRANSID, an unknown language, an empty PROGRAM, a bad group name or a TWASIZE out of range is rejected with a message in the log, and the transaction is not defined. When the same TRANSID is defined twice, the first definition is used and the later one is skipped. A `#` preceded by a blank starts a comment. Examples:

```
HELO:rexx:hello.rexx:PUBLIC
QAGE:rexx:qage.rexx:PUBLIC,USERS,ADMIN
GUST:cobol:gust.cob:PUBLIC
CUSP:pli:cusp.pli:PUBLIC
BANK:cobol:bank.cob:PUBLIC,USERS,ADMIN:bank
CSGM:rexx:csgm.rexx:PUBLIC,USERS,ADMIN:[DBREG1]
```

In bricks a program is known by the TRANSID of the transaction that runs it. `EXEC CICS LINK PROGRAM(name)`, `XCTL PROGRAM(name)` and `INQUIRE PROGRAM(name)` all name a TRANSID defined in `transactions.conf`; there is no separate program definition.

### Programming Languages

bricks accepts three languages. All three issue `EXEC CICS` commands through the same command processor, so every command in Part 3 is available, with the same syntax and the same results, from each of them. Programs in different languages may `LINK` and `XCTL` to one another freely.

| Language | Directory | Described in |
|---|---|---|
| REXX | `runtime/rexx/` | Part 8, Chapters 35 through 40 |
| COBOL | `runtime/cobol/` | Part 6, Chapters 22 through 27 (copybooks in `runtime/cobolcopy/`) |
| PL/I | `runtime/pli/` | Part 7, Chapters 28 through 34 (`%INCLUDE` members in `runtime/pliinc/`) |

**Figure 1-2. Supported programming languages**

There is no separate compile step. A program is parsed the first time its transaction is started, and the parsed form is kept in the **program cache** (two tiers, sized by the `program_cache` keyword of `bricks.cnf`). Later tasks use the cached form. An edited source file is detected by its modification time and is parsed again on the next start; for PL/I, an edit to any `%INCLUDE` member the program used has the same effect. Each task has its own storage: no variable is shared between two tasks that run the same program at the same time.

The `brickscompile` utility checks programs without running them. It accepts a single file or a directory; for a directory it checks every program in it and its subdirectories and ends with return code 0 only if every file is free of errors. For a PL/I program it also reports a misspelled `EXEC CICS` verb.

***Organizing programs in subdirectories***

Programs may be grouped in subdirectories of the language directory, to any reasonable depth. The PROGRAM field then gives the path relative to the language directory, with forward slashes on every operating system:

```
INVC:cobol:billing/invoice.cob:PUBLIC
AUTH:cobol:cards/visa/authz.cob:ADMIN
USRP:rexx:admin/userprov.rexx:ADMIN
```

The CEDA PROGRAM display and `brickscompile` walk the directories recursively, to a depth of eight, and skip hidden entries (names that begin with a period). CEMT INQUIRE TRANSACTION shortens a long path in the middle (`billing/.../authz.cob`) so that the application directory and the file name stay visible. CEDA TRANSACTION rejects a path that contains `..`, an empty segment (`a//b.rexx`), a colon, a line end, or a blank followed by `#`; a Windows drive path such as `C:\app\prog.cob` is therefore refused. An absolute path is accepted unchanged.

### Tasks and the Pseudo-conversational Model

A task is one run of one transaction. A program interacts with three kinds of resource:

- the **3270 terminal**, through `SEND MAP`, `RECEIVE MAP`, `SEND TEXT` and the other terminal control commands (Chapter 11), using screens defined in the map language (Chapter 3);
- **recoverable data**: key-sequenced files (Chapter 14), temporary storage and transient data queues (Chapter 15), and SQL databases (Chapter 21); files and queues are kept in an embedded store, `data/files.boltdb`;
- **other programs**, through `LINK` (a call that returns), `XCTL` (a transfer of control that does not return) and `RETURN TRANSID` (which names the transaction to run on the operator's next input) ("RETURN" in Chapter 12).

The Execute Interface Block (EIB) gives the program the state of the task and the result of the last command (Chapter 17).

Most bricks transactions, like most CICS transactions, are **pseudo-conversational**. A task paints a screen, names the transaction that is to process the operator's reply, and ends; it holds no resources while the operator reads and types. When the operator presses ENTER or a function key, the named transaction starts as a new task and reads the screen. Data that must survive from one task to the next travels in the **communication area** (COMMAREA): the ending task passes it on `RETURN`, and the next task finds it in `DFHCOMMAREA`, with its length in `EIBCALEN`. A first task, started from the blank screen, has an `EIBCALEN` of zero.

```
ADDRESS CICS
IF EIBCALEN = 0 THEN DO                 /* first time: show the panel  */
  EXEC CICS SEND MAP('CUST1') FROM(SCR.) ERASE END-EXEC
END
ELSE DO                                 /* the operator has replied    */
  EXEC CICS RECEIVE MAP('CUST1') INTO(IN.) END-EXEC
  /* ... validate, read and update files, build SCR. ...               */
  EXEC CICS SEND MAP('CUST1') FROM(SCR.) ERASE END-EXEC
END
EXEC CICS RETURN TRANSID('CUST') COMMAREA(STATE) END-EXEC
```

Each task runs in a **unit of work**. Its changes to recoverable resources are committed when the task ends normally (or at an explicit `SYNCPOINT`) and are backed out when it ends abnormally. Units of work, condition handling and abends are described in Chapter 16.

A program may also be **conversational**: it paints a screen, waits for the reply within the same task and loops. bricks supports both styles. In bricks every `SEND MAP` and `SEND TEXT` waits for the operator's next key before it returns (see "SEND MAP" in Chapter 11).

### Terminals

bricks accepts TN3270 connections, with or without TLS. The size of the screen is negotiated when the terminal connects; the common sizes are the following.

| Model | Rows x columns | Buffer positions |
|---|---|---|
| Model 2 | 24 x 80 | 0–1919 |
| Model 3 | 32 x 80 | 0–2559 |
| Model 4 | 43 x 80 | 0–3439 |
| Model 5 | 27 x 132 | 0–3563 |

**Figure 1-3. 3270 display station models**

A program learns the size of its terminal with `EXEC CICS ASSIGN SCREENHT(...) SCREENWD(...)` and chooses a map of the right size; see "Adapting to the Terminal Size" in Chapter 3. Buffer positions, as used by the `CURSOR` option and by `EIBCPOSN`, are counted from zero.

Each connection receives a terminal identifier of the form `T` followed by four digits (`T0001`, ...). It is returned by `ASSIGN TERMID` and in `EIBTRMID`. COBOL and PL/I programs should receive it in an eight-character field.

When a terminal connects, the region paints its logo splash (Figure 4-2 in Chapter 4), and the operator signs on with CSSN. When no task is running, the terminal shows the blank screen (Figure 4-7 in Chapter 4). The operator starts a transaction by typing its TRANSID, optionally followed by data, and pressing **`ENTER`**. A REXX program receives the data after the TRANSID as `ARG(1)`; any program can read the whole line, TRANSID included, with `EXEC CICS RECEIVE INTO`. An installation may also define longer names for transactions in `runtime/aliases.conf`; see "Transaction Aliases" in Chapter 4.

The **`PA1`** key is the operator's escape. When PA1 answers a `SEND MAP` or `SEND TEXT`, the running task ends at once: its unit of work is backed out, no next transaction is started, and the terminal returns to the blank screen.

### Security Groups

Operators sign on with the CSSN transaction ("Signing On: CSSN" in Chapter 4). Each user, defined in `runtime/users.conf`, belongs to one or more **groups**. The GROUPS field of a transaction definition is an access-control list that the dispatcher checks every time the transaction is started, whether from the terminal, by `LINK`, by `XCTL` or by `START`.

| GROUPS field | Who may run the transaction |
|---|---|
| (empty) | Everyone, signed on or not. |
| `PUBLIC` | Everyone, signed on or not. |
| `*` | Any signed-on user. |
| one or more group names | A signed-on user who belongs to at least one of the groups. Group names are compared without regard to case. |

**Figure 1-4. Transaction access-control lists**

A list that names neither `PUBLIC` nor `*` admits only members of the groups named. A group that no user holds therefore denies the transaction to everyone; a misspelled group name shows up as "access denied", not as a definition error.

The built-in transactions are part of bricks itself, not of `transactions.conf`. Their access rules are fixed and cannot be changed with CEDA. The dispatcher checks them in the same way: at the terminal, by `LINK` and by `XCTL`. No alias in `aliases.conf` can take one of these names.

| Transaction | Purpose | Who may run it |
|---|---|---|
| CSSN | Sign on. CESN is accepted as a synonym. See "Signing On: CSSN". | Everyone, from the terminal only. A LINK or XCTL to CSSN is always refused. |
| CSSF | Sign off (CSSF LOGOFF), or sign off and disconnect (CSSF DISC, DISCONNECT, GOODNIGHT). CESF is accepted as a synonym. See "Signing Off: CSSF". | Any signed-on user. |
| CEMT | Master terminal: inquire on and monitor resources, tasks, files, databases and MRO connections. See Chapter 5. | Any signed-on user. The CONTROLBLOCKS and PERFORM sub-menus require ADMIN and are not shown to other users. |
| CEDA | Define, alter and delete resources: users, transactions, programs, databases, VSAM files, URIMAPs and web services, MRO connections and queues. See Chapter 6. | ADMIN. |
| CECI | Command-level interpreter: enter and run single EXEC CICS commands. See Chapter 7. | DEV. |
| CEDF | Execution diagnostic facility: step through a transaction command by command, with its source, EIB and variables. See Chapter 8. | DEV or ADMIN. |
| CEDG | As CEDF, but display only: nothing the task does can be changed. See Chapter 8. | DEV or ADMIN. |
| ISPF | Browse and edit the program and map sources of the region (REXX, COBOL and PL/I programs, map files). See Chapter 9. | DEV. |
| IDCA | Access method services: define, list and maintain KSDS files. IDCAMS is accepted as a synonym. See Chapter 10. | ADMIN. |

| Transaction | Purpose | Who may run it |
|---|---|---|
| CAID | Abend analyser: find the transaction dump of an abended task in abend.log and diagnose the cause. See "CAID: The Abend Analyser". | ADMIN. Dumps contain program data. |

**Figure 1-5. Built-in transactions**

Group names are compared without regard to case. A user who is refused receives TRANSID "CAID": access denied -- requires admin group, naming the transaction and the groups it requires (requires dev, admin group for CEDF and CEDG), in the frame shown in Figure 4-10 in Chapter 4. A program can ask about the current user's access with QUERY SECURITY and can check a password again with VERIFY PASSWORD ("QUERY SECURITY" in Chapter 13).

### The runtime Directory

Everything an application consists of lives under one directory, by default runtime/ in the bricks working directory. The runtime_dir keyword of bricks.cnf moves it; each subdirectory can also be moved on its own.

| Path | bricks.cnf keyword | Contents |
|---|---|---|
| runtime/rexx/ | rexx_dir | REXX programs |
| runtime/cobol/ | cobol_dir | COBOL programs |
| runtime/cobolcopy/ | copybook_dir | COBOL copybooks (DFHAID, DFHRESP, DFHVALUE, DFHCOLOR, ...) |
| runtime/pli/ | pli_dir | PL/I programs |
| runtime/pliinc/ | pli_include_dir | PL/I %INCLUDE members |
| runtime/map/ | maps_dir | Map definitions (*.map) |
| runtime/tmp/ | tmp_dir | Sandbox for transient data queues and sequential files |
| runtime/web/ | webserver_dir | Static files served by the web listener |
| data/ | data_dir | files.boltdb: files and temporary storage (not under runtime/) |

**Figure 1-6. Directories used by applications**

### Where Definitions Live

Resources are defined in plain text files under `runtime/`. Most can also be maintained online with CEDA (Chapter 6). `transactions.conf` and the map files are read again when they change, so a new or changed transaction or map is used without restarting the region.

| File | Defines | See |
|---|---|---|
| `transactions.conf` | Transactions, their programs, access lists, databases and TWA sizes | This chapter |
| `users.conf` | Users, password hashes and groups | Chapter 4, 6 |
| `databases.conf` | The SQL databases programs may use | Chapter 21 |
| `aliases.conf` | Long names an operator may type for a TRANSID | Chapter 4 |
| `web_routes.conf` | HTTP routes served by transactions | Chapter 20 |
| `mro.conf` | Peer regions for multiregion operation | Appendix D |
| `map/*.map` | Screen layouts | Chapter 3 |

**Figure 1-7. Resource definition files**

Region-wide settings, such as port numbers, directory locations, cache sizes, the time zone and the size of the Common Work Area, are kept in `bricks.cnf`. "The bricks.cnf File" in Appendix D lists the keywords that concern the application programmer.

Chapter 2

# The EXEC CICS Command Environment

A program requests services from bricks by issuing `EXEC CICS` commands. This chapter describes how a command is written in each language, how its arguments are passed, how its outcome is reported, and how character data crosses the boundary between the program and the outside world. The rules apply to every command in Part 3 and to the `EXEC CICS WEB` and `DOCUMENT` commands of Part 4.

## Command Forms

A command consists of a **verb** (`SEND`, `READ`, `RETURN`, ...), sometimes a second keyword (`SEND MAP`, `READQ TS`), and **options**. An option is either a keyword alone (`ERASE`) or a keyword followed by an argument in parentheses (`MAP('HELO1')`). Options may be written in any order. bricks accepts two forms of the command.

### *The EXEC CICS form*

The IBM form is available in all three languages. In COBOL and REXX the command ends with `END-EXEC`; in PL/I it ends with a semicolon (see "EXEC CICS" in Chapter 33):

```
EXEC CICS SEND MAP('HELO1') FROM(SCR.) ERASE END-EXEC      (REXX)
EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE END-EXEC.      (COBOL)
EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE;               (PL/I)
```

A command may span several source lines. In COBOL the compiler collects the words between `EXEC CICS` and `END-EXEC`. In REXX a preprocessor turns each `EXEC CICS ... END-EXEC` block into one command string before the program is parsed; the lines are joined with blanks and the line numbers of later statements are unchanged. Do not end a line inside the block with the REXX continuation comma: the comma becomes part of the command, and the command is rejected with `unexpected token ","`. Comments and string literals inside the block are left alone.

### *The command-string form (REXX only)*

Under `ADDRESS CICS`, a REXX program may also pass the command as a string expression. The two forms are processed in the same way.

```
ADDRESS CICS
"ASSIGN USERID(USR) TERMID(TRM)"
"SEND MAP('HELO1') FROM(SCR.) ERASE"
"RETURN"
```

The PL/I compiler checks the verb of every command, so a misspelled verb in a PL/I program is a compile-time error. In COBOL and REXX it is found when the command is run: the command fails with `unsupported EXEC CICS verb`, which is not a condition and ends the task. The verbs bricks supports are listed in Appendix B; those it does not support are listed in Chapter 19.

### Arguments

Inside the parentheses of an option:

- a **name** (`LENGTH(LEN)`, `INTO(WS-REC)`, `FROM(SCR.)`) refers to a variable or data area. bricks reads its value when the option is an input, and stores the result in it when the option is an output;
- a **quoted string** (`'...'` or `"..."`, with the quote doubled to include it) or a **number** (`LENGTH(80)`) is a literal. bricks reads it but never stores into it.

Some options are both input and output. `LENGTH(LEN)` on `READQ TS` passes the largest length the program will accept and returns the length actually delivered; `LENGTH(80)` only passes the limit. When an output option is given a literal, most commands do not store the result (`ASSIGN`, `QUERY SECURITY`, `RESP`); a few, whose whole purpose is the output (`INTO` on `RECEIVE` and `RETRIEVE`), reject it with `INVREQ`.

A COBOL data area passed as `COMMAREA` or `FROM` travels with its full declared length, trailing blanks and binary fields included, as on z/OS. A REXX variable travels as its value.

### Responses and Conditions

Every command ends with a **response**. bricks stores the response code in `EIBRESP`, the secondary code in `EIBRESP2`, and (in REXX) the response code again in `RC`. A response other than `NORMAL` is a **condition**, such as `NOTFND`, `MAPFAIL` or `PGMIDERR`. "Responses" in Chapter 18 lists every condition with its number.

Every command accepts the options `RESP(var)` and `RESP2(var)`, which store the same two codes in variables of the program's choice, and `NOHANDLE`. The symbolic names of the codes are available as `DFHRESP(name)` in COBOL and PL/I. The compiler replaces `DFHRESP(NOTFND)` with the number 13; an unknown name is a compile-time error. Both languages take the numbers from one table, so they are the same everywhere.

```
EXEC CICS READ FILE('CUST') INTO(REC) RIDFLD(KEY)
               RESP(WS-RC) END-EXEC.
EVALUATE WS-RC
  WHEN DFHRESP(NORMAL)  PERFORM USE-RECORD
  WHEN DFHRESP(NOTFND)  PERFORM CREATE-RECORD
  WHEN OTHER            PERFORM REPORT-ERROR
END-EVALUATE.
```

***What happens to a condition***

The response determines what happens next, in this order:

1. If the command codes `RESP` or `NOHANDLE`, the program has taken charge of the outcome. The condition never ends the task, in any language; `HANDLE CONDITION` and `HANDLE AID` are not acted on for this command, and the program continues with the next statement.
2. Otherwise, if the program has an active `HANDLE CONDITION` for the condition (or for `ERROR`), control passes to the label named; if it has issued `IGNORE CONDITION` for it, the program continues with `EIBRESP` set to 0 (`NORMAL`).
3. Otherwise the condition is **unhandled**. A COBOL or PL/I task ends abnormally with the IBM abend code of the condition (`NOTFND` gives `AEIM`, `MAPFAIL` gives `AEI9`, `PGMIDERR` gives `AEI0`). A REXX task, as in IBM CICS REXX, does not end: `RC` and `EIBRESP` carry the response and the program continues; `SIGNAL ON ERROR`, if enabled, is raised with the real response in `RC`. The complete rules are given in "Unhandled Conditions" in Chapter 16.

A few conditions never cause an abend because IBM's default action for them is to continue: among them `EOC`, `NOSTG` and `ENQBUSY`. `HANDLE CONDITION`, `IGNORE CONDITION` and the abend codes are described in "Conditions" in Chapter 16 and in "Condition Abends" in Appendix C.

***Outcomes that are not conditions***

Some outcomes are not conditions. `RESP` and `NOHANDLE` do not absorb them, and they end the task (or transfer control) whatever the language:

- RETURN, XCTL and ABEND, which end the program by design;
- the abends that a command raises itself: ATNI when the terminal is lost during a terminal command, ABMB for a CURSOR position beyond the screen, ABM0 for RECEIVE MAP of a map that the named mapset does not contain, AEXZ for an internal failure in a command, and the syncpoint abends ASP3, ASP7 and ASPR;
- an abend or runtime failure in a program called with LINK, which becomes an abend of the calling program ("LINK" in Chapter 12);
- a request for a facility bricks does not provide, such as CHANNEL on LINK or XCTL, the REPLY option of WRITE OPERATOR, or a waiting SEND MAP in a task started in the background by START.

### Length Options

On commands that deliver data to the program or pass it on, LENGTH(n) is the IBM input-output parameter: the value the program supplies limits the number of bytes delivered or passed, and after the command a LENGTH variable holds the number actually delivered. This applies to READQ TS, RECEIVE INTO, RETRIEVE, LINK COMMAREA, XCTL COMMAREA and RETURN COMMAREA. LENGTH(80) on a READQ TS of a 200-byte item delivers 80 bytes and reports 80.

On READ, READNEXT and READPREV of a file, LENGTH is an output only: the record decides the length, and the value the program supplies is ignored. See "READ" in Chapter 14.

### Character Data

Everything a program can address holds **one byte per character**, as on a mainframe: variables, the COMMAREA, START and RETRIEVE data, queue items, file keys and records, DOCUMENT buffers, ENQ resource names and the text of EXEC CICS and EXEC SQL commands. The code page, **bricks SBCS**, is ISO-8859-1 (Latin-1) with a plane of graphic characters at X'80' through X'9F' in place of the C1 control characters:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| X'80' | € | ▉ | ▀ | ▄ | ▌ | ▐ | ▪ | ─ | │ | ┌ | ┐ | └ | ┘ | ├ | ┤ | ┬ |
| X'90' | ┴ | ┼ | ═ | ‖ | · | ← | → | ↑ | ↓ | ≤ | ≥ | ≠ | ○ | ◊ | ¯ | ≡ |

**Figure 2-1. The graphic plane of bricks SBCS**

Every length (`LENGTH`, `FLENGTH`, `EIBCALEN`, `KEYLENGTH`, `MAXLENGTH`) therefore counts characters, and all 256 byte values pass through the program interface unchanged: an AID byte, binary and packed fields inside a COMMAREA, `LOW-VALUES` and `HIGH-VALUES`.

Outside the program, text is UTF-8. bricks converts once, where data crosses an edge, as shown in the following table. A character that has no SBCS byte (a CJK character, an em dash, an emoji, a C1 control character) becomes `?`. The substitution is silent except for SQL results.

| Edge | Into the program | Out of the program |
|---|---|---|
| Terminal | `RECEIVE MAP`, `RECEIVE INTO` and `CONVERSE` convert the 3270 input to SBCS. | `SEND MAP` and `SEND TEXT` paint each byte as its character; the graphic plane is sent as 3270 graphic-escape pairs (the euro sign is native on the euro code pages). |
| REXX `ARG(1)` | The data typed after the TRANSID is converted. | - |
| Files, temporary storage, `START`/`RETRIEVE`, `ENQ`, COMMAREA, multiregion | No conversion: bytes are bytes. | No conversion. |
| `tmp_dir` and `runtime_dir` files (`READQ TD`, sequential `READNEXT`, REXX `LINEIN`, IDCAMS `REPRO`) | Each UTF-8 line becomes SBCS; a line that is not valid UTF-8 is taken as Latin-1. | Stored as UTF-8. |
| SQL (`EXEC SQL`) | Results are converted; `BYTEA` is raw. A substituted `?` sets `SQLWARN8` and `SQLWARN0` to `'W'`; `SQLCODE` stays 0. `SQLERRMC` is SBCS, 70 characters. | Statement text and host variables are sent as UTF-8. |
| HTTP (`EXEC CICS WEB`) | See Chapter 20. | |
| `DOCUMENT RETRIEVE` | SBCS by default; `CHARACTERSET('UTF-8')` delivers UTF-8. | - |
| `VERIFY PASSWORD` | - | The password is checked as UTF-8, as it was typed at sign-on. |
| `INQUIRE SYSTEM GMMTEXT` | The `bricks.cnf` text is converted. | - |
| `WRITE OPERATOR`, the log, CEMT INQUIRE ENQ, the CECI result area, IDCAMS listings | - | Shown as UTF-8; control bytes as `.` or `?`. |

**Figure 2-2. Where character data is converted**

The write path for `tmp_dir` and `runtime_dir` files accepts TAB, LF, `X'20'` through `X'7E'` and `X'80'` through `X'FF'`. NUL, the other C0 controls, DEL and CR are rejected with `INVREQ` and a message such as `byte 0x7F at offset 3 is not printable`. Error messages that quote program data are plain ASCII and name a non-ASCII character by its code point (`"K\u00c4SE"`).

**Differences from IBM CICS:**

The code page is Latin-1 with the graphic plane, not an EBCDIC CCSID such as 037 or 1140. Data collates in byte order. EBCDIC hexadecimal constants, other than the AID bytes, must be translated. Temporary storage queue names and file names must be ASCII. The default character set for outbound HTTP is UTF-8, not ISO-8859-1.

## Summary of the Programming Model

| Concept | In bricks |
|---|---|
| Task | One run of one transaction. |
| Program | A REXX, COBOL or PL/I source file, parsed once and cached; named by its TRANSID. |
| Pseudo-conversation | `RETURN TRANSID(t) COMMAREA(data)`. |
| Conversation | A loop in one task; each `SEND MAP` waits for the operator. |
| Call another program | `LINK PROGRAM(name) COMMAREA(area)`. |
| Transfer to another program | `XCTL PROGRAM(name) COMMAREA(area)`. |
| Data shared between tasks | Files, temporary storage and transient data queues, SQL tables, the Common Work Area. |
| Data private to a task | REXX variables, COBOL WORKING-STORAGE, PL/I storage, the Transaction Work Area, `GETMAIN` storage. |
| Normal end of task | `EXEC CICS RETURN`; also the end of the program, COBOL `GOBACK` or `STOP RUN`, or the end of the PL/I main procedure. The unit of work is committed. |
| Abnormal end of task | `EXEC CICS ABEND`, an unhandled condition (COBOL, PL/I), or an abend raised by bricks. The unit of work is backed out. |

**Figure 2-3. The bricks programming model**

Chapter 3

# Defining Screens: The Map Language

A **map** describes one 3270 screen: where its constant text appears, where the operator may type, and where the cursor is placed. Programs send and receive maps with the terminal control commands of Chapter 11. In IBM CICS maps are written as BMS assembler macros; bricks uses a smaller line-oriented language with the same concepts. This chapter describes that language, the map catalog, and the way a program adapts its maps to the size of the terminal.

## Map Files

Maps are kept in files with the suffix `.map` in the map directory (`runtime/map/`, or the directory named by the `maps_dir` keyword of `bricks.cnf`). A file holds one or more maps, each enclosed in `MAP` and `ENDMAP`, and all the maps of one file belong to one **mapset**. The structure of a file is:

```
[MAPSET name]
MAP name SIZE rowsxcols [FREEKB]
  FIELD [name] AT row,col LEN n [attributes] ["text"]
  INPUT name   AT row,col LEN n [attributes] [DEFAULT "text"]
  STOP         AT row,col
  CURSOR AT {row,col | name}
ENDMAP
[MAP name SIZE rowsxcols [FREEKB]
  ...
ENDMAP]
```

Each statement occupies one line. Keywords, map names, field names and attributes may be written in any case; names are stored in upper case. An asterisk (`*`) outside a quoted string starts a comment that runs to the end of the line. A quoted string may be enclosed in double or single quotes.

## Rows, Columns and Field Layout

Rows and columns are counted **from zero**. On a 24 x 80 map the rows are 0 through 23 and the columns 0 through 79; `AT 0,0` is the top left corner. A statement whose row or column lies outside the map's `SIZE` is rejected.

As on any 3270, each field begins with an **attribute byte** that occupies one screen position and shows as a blank. A field declared `AT r,c` has its attribute byte at column `c`; its text starts at column `c+1` and occupies `LEN` positions. `LEN` counts screen positions (characters), not bytes. So that the text stays on its row, `LEN` should not exceed *cols* - 1 - `c`. The parser rejects a field whose column plus `LEN` exceeds the width of the map; a field that reaches exactly to the width is accepted and its last character is dropped when the map is painted.

A value supplied for a field is padded to `LEN` or cut to it. Text fields and protected input fields are padded with blanks; unprotected input fields are padded with nulls, as BMS does, so that the terminal's Insert key works in them.

### Statements

#### *MAPSET name*

Optional. If present, it must be the first statement of the file. It names the mapset to which every map in the file belongs. Without it the mapset is the file name in upper case without its suffix: the maps in `help1.map` belong to mapset `HELP1`. The catalog knows each map by its qualified name `MAPSET.MAP`, so two files may each contain a map called `MAPA`.

#### *MAP name SIZE rowsxcols [FREEKB]*

Begins a map. The name must be unique within its mapset. The size gives the rows and columns the map is laid out for, for example `24x80` or `43x80`. A file may contain any number of maps one after another, as a BMS mapset can contain several `DFHMDI` macros.

`FREEKB`, the counterpart of BMS `CTRL=FREEKB`, makes every `SEND MAP` of the map unlock the keyboard, as if `FREEKB` were coded on the command. It matters for `SEND MAP ... DATAONLY`, which otherwise leaves the keyboard locked ("The keyboard lock and FREEKB" in Chapter 11). It is the only keyword allowed after `SIZE`, and may be written once:

```
MAP CLOCK1 SIZE 24x80 FREEKB
```

#### *FIELD [name] AT row,col LEN n [attributes] ["text"]*

A display field. Without a name it is constant text: a label, a heading or a line of instructions. With a name it is a display field that the program can fill: the quoted text is the default, and `SEND MAP FROM` replaces it with the value of the matching variable. A `FIELD` is always protected; the operator cannot type into it.

***INPUT name AT row,col LEN n [attributes] [DEFAULT "text"]***

A named field that the program can fill and the operator can type into. The name connects the field to the program: `SEND MAP FROM(SCR.)` paints the field from the variable `SCR.name`, and `RECEIVE MAP INTO(IN.)` stores what the operator typed in `IN.name`. `DEFAULT` gives the text shown when the program supplies none. With the `PROT` attribute the field is protected but keeps its name, which is the usual way to define an output field that the program fills, such as a message line. The name of an `INPUT` field must be unique within the map.

***STOP AT row,col***

Places a protected, autoskip attribute byte at the given position. It ends the preceding input field, so that the operator cannot type past its end and the cursor skips to the next input field.

***CURSOR AT row,col and CURSOR AT name***

The position of the cursor when the map is sent. The named form places the cursor on the first character of the named field, one position to the right of its attribute byte: `CURSOR AT YEAR` for a field declared `AT 6,12` places the cursor at row 6, column 13. The name may be that of an `INPUT` or a named `FIELD`, and may refer to a field declared later in the map. The numeric form gives the position directly, counted from zero. Without a `CURSOR` statement the cursor is placed on the first character of the first `INPUT` field, or at row 0, column 0 if the map has none. Only one `CURSOR` statement is allowed in a map. The `CURSOR` option of `SEND MAP` overrides it.

***ENDMAP***

Ends the map.

### Attributes

| Attribute | Effect |
|---|---|
| PROT | On an INPUT field: the field is protected. A FIELD is always protected. |
| UNPROT | Accepted. An INPUT field without PROT is unprotected already. |
| NUMERIC | On an unprotected INPUT field: the terminal accepts only numeric input. |
| BRIGHT | High intensity. |
| HIDDEN | Nondisplay: the field's contents, and what the operator types into it, are not shown. Use it for passwords. |
| UNDERLINE | Underscore highlighting. UNDERSCORE is accepted as a synonym. |
| BLINK | Blinking highlighting. |
| REVERSE | Reverse-video highlighting. |
| COLOR=c | Color of the field: BLUE, RED, PINK, GREEN, TURQUOISE (or CYAN), YELLOW, WHITE, DEFAULT or NEUTRAL. Any other value gives the terminal's default color. |
| MARK=x:c | Shows every occurrence of the character *x* in the field in color *c*; the rest of the field keeps its own color. A bricks extension. See "Coloring Single Characters". |
| DIM | Accepted; no effect. |
| MDT | Accepted; no effect. |

**Figure 3-1. Map field attributes**

A field has one kind of highlighting. When more than one is coded, UNDERLINE takes precedence over BLINK, and BLINK over REVERSE. An unknown attribute is a syntax error.

### Character Set

Map files are UTF-8. Every character in a quoted string occupies one screen position, so `LEN` counts characters: a string of three accented letters fits `LEN 3`. Outside quotes the language is ASCII. A quoted string may contain:

- printable ASCII;
- the Latin-1 letters and symbols U+00A0 through U+00FF, except the currency sign U+00A4, which the euro code pages cannot show;
- the characters the 3270 sends as graphic-escape pairs: block and box-drawing characters, the bullet, arrows, the comparison signs, the APL symbols, and the euro sign. A terminal without graphic-escape support shows a substitute character instead.

Control characters (TAB, CR, NUL, DEL, C1), invalid UTF-8, typographic dashes and quotes, emoji, CJK and the letters of other alphabets are rejected. Comments may contain anything, and a UTF-8 byte order mark at the start of the file is ignored. Although the brackets, braces and similar ASCII characters are accepted, they are not printable on every EBCDIC code page; screens that must display correctly under code page 037 should avoid `[ ] { } ~ \ ` | ^`.

A rejected character makes the whole file fail to load. The message names the line, the code point and the column:

```
line 3: U+2014 not displayable on 3270 (col 27)
line 4: U+0009 control char in literal (col 30)
line 5: invalid UTF-8 byte X'E9' (col 28)
line 6: U+00DC outside quotes (col 33)
```

A string longer than its `LEN` is not an error. The map loads with a warning, counted by CEMT PERFORM RESCAN MAP, and the text is cut on the screen.

### Example

The following map, supplied as `runtime/map/qage1.map`, asks for a date of birth. It shows constant text, a protected named output field (`TERMID`, `MSG`), three numeric input fields each closed by a `STOP`, and a numeric `CURSOR` that places the cursor on the first character of the `YEAR` field.

```
* QAGE -- birthdate input. Pseudo-conversational; chains to QAGR
* via EXEC CICS RETURN TRANSID('QAGR'). PF3 returns to bare prompt.
MAP QAGE1 SIZE 24x80
  FIELD AT 0,2  LEN 30 PROT BRIGHT COLOR=BLUE  "Query Your Age"
  FIELD AT 0,60 LEN 5  PROT COLOR=BLUE         "TERM="
  INPUT TERMID  AT 0,65 LEN 8  PROT COLOR=BLUE
  FIELD AT 4,4  LEN 30 PROT COLOR=BLUE         "Enter your birth date below."
  FIELD AT 6,4  LEN 6  PROT                    "Year:"
  INPUT YEAR    AT 6,12 LEN 4  UNDERSCORE COLOR=GREEN NUMERIC
  STOP          AT 6,17
  FIELD AT 6,20 LEN 6  PROT                    "Month:"
  INPUT MONTH   AT 6,28 LEN 2  UNDERSCORE COLOR=GREEN NUMERIC
  STOP          AT 6,31
  FIELD AT 6,34 LEN 4  PROT                    "Day:"
  INPUT DAY     AT 6,40 LEN 2  UNDERSCORE COLOR=GREEN NUMERIC
  STOP          AT 6,43
  INPUT MSG     AT 18,4 LEN 70 PROT BRIGHT COLOR=RED
  FIELD AT 22,2 LEN 50 PROT COLOR=BLUE         "ENTER=Submit  PF3=Cancel"
  CURSOR        AT 6,13
ENDMAP
```

The `YEAR` field has its attribute byte at column 12 and its four characters at columns 13 through 16; the `STOP` at column 17 ends it. `CURSOR AT YEAR` would have the same effect as `CURSOR AT 6,13`. A REXX program fills and reads the map like this:

```
SCR. = ''
SCR.TERMID = TRM
EXEC CICS SEND MAP('QAGE1') FROM(SCR.) ERASE END-EXEC
EXEC CICS RECEIVE MAP('QAGE1') INTO(IN.) END-EXEC
SAY 'Born' IN.YEAR'-'IN.MONTH'-'IN.DAY
```

### Changing a Field's Color at Runtime

The `COLOR=` attribute is the default color of a field. A program can override it for any named field when it sends the map, by setting a companion variable before the `SEND MAP`: in COBOL an item named `field-C` in the `FROM` structure, in REXX the stem tail `field_C`. A blank or unset value keeps the map's color.

```
COPY DFHCOLOR.
01 SCR.
   05 STATUS    PIC X(10).
   05 STATUS-C  PIC X(9) VALUE SPACES.
...
IF BALANCE < 0 MOVE DFHRED TO STATUS-C ELSE MOVE DFHGREEN TO STATUS-C.
EXEC CICS SEND MAP('BALC') FROM(SCR) ERASE END-EXEC.
```

```
IF BALANCE < 0 THEN SCR.'STATUS_C' = 'RED'
               ELSE SCR.'STATUS_C' = 'GREEN'
EXEC CICS SEND MAP('BALC') FROM(SCR.) ERASE END-EXEC
```

The value is one of the color names listed under "Attributes". The copybook DFHCOLOR declares each color twice, under its plain name (RED) and its IBM name (DFHRED, DFHTURQ, DFHYELLO, DFHNEUTR, DFHDFT, ...). An unknown value gives the terminal's default color. In REXX, a stem tail is replaced by the value of a variable of the same name; quote the tail, as above, if the program also uses a variable called STATUS_C. The supplied programs runtime/cobol/balc.cob and runtime/rexx/balc.rexx are complete examples.

### Coloring Single Characters

COLOR= colors a whole field. The MARK= attribute colors one character inside a field: every occurrence of the character *x* in the field's contents is shown in color *c*, and every other character keeps the field's color.

```
FIELD BRD2  AT 1,0  LEN 23 PROT BRIGHT COLOR=BLUE MARK=Q:YELLOW
```

With this field, a program that sends !Q! ! ! gets a blue grid with a yellow Q. The rules are:

- *x* is exactly one letter or digit (A-Z, a-z, 0-9). Case is kept: MARK=q:YELLOW marks only lowercase q.
- *c* is BLUE, RED, PINK, GREEN, TURQUOISE (or CYAN), YELLOW or WHITE. Any other value is a syntax error.
- MARK= is allowed on a FIELD and on a PROT INPUT. On an unprotected INPUT field it is a syntax error. A field may have only one MARK=.
- A HIDDEN field is never marked.
- The field's contents are unchanged: LEN, the data the program sends and receives, and the column limits are the same as without MARK=. The program needs no change.

bricks sends a Set Attribute order before each run of the marked character and resets the color after it, so the characters use no extra screen positions. A terminal without extended color support ignores the orders and shows the character in the field's color. The supplied map runtime/map/nque2.map (transaction NQUE) is an example.

**Differences from IBM CICS:**

BMS has no equivalent. A `DFHMDF` macro colors a whole field, and `bricksconvert` never generates `MARK=`. A map that uses it cannot be assembled for IBM CICS.

### The Map Catalog

bricks reads every `.map` file in the map directory when it starts and builds a catalog of the maps by mapset and name. Parsed maps are kept in a cache whose size is set by the `map_cache` keyword of `bricks.cnf`. The catalog checks the directory and the files at most every quarter of a second while maps are being used: a new, changed or deleted file is noticed within that time, and the next `SEND MAP` uses the new definition. No restart and no command is needed.

A file that fails to parse is left out of the catalog with a message in the log; the other files are loaded. A `SEND MAP` of a map in the failing file gets `MAPFAIL` with the reason, for example `SEND MAP: map "X" has bad char U+2014 at line 3` or `... has syntax error at line 7`. When the file is corrected it is loaded again. CEMT PERFORM RESCAN MAP checks every map file and lists those that fail (see "PERFORM" in Chapter 5).

### Selecting a Map by Mapset

`SEND MAP('NAME') MAPSET('SET')` looks up the qualified name `SET.NAME`, so maps of the same name in different files are kept apart:

```
EXEC CICS SEND MAP('MAPA') MAPSET('AM01') FROM(AM01) ERASE END-EXEC.
EXEC CICS SEND MAP('MAPA') MAPSET('AM16') FROM(AP16) ERASE END-EXEC.
```

Without `MAPSET`, bricks looks the name up alone. If the name exists in more than one mapset the command fails with `MAPFAIL` and the message `SEND MAP: map "MAPA" is ambiguous; specify MAPSET`; it never chooses one at random. Programs whose map names are unique, as those supplied with bricks are, need not code `MAPSET`. `RECEIVE MAP` accepts `MAPSET` with the same meaning.

### Converting BMS Map Sources

The utility `bricksconvert` converts IBM BMS map source (`DFHMSD`, `DFHMDI` and `DFHMDF` macros) into the bricks map language, one `MAP` block per `DFHMDI`. It writes no `MAPSET` statement, so the mapset is the name of the file the output is saved in; name the file after the BMS mapset. BMS features that have no equivalent (such as `PICIN`, `PICOUT`, `OCCURS` and `GRPNAME`) are reported as `* WARNING:` comment lines in the output. `CTRL=FREEKB` on the `DFHMDI`, or on the `DFHMSD` for every map that does not code its own `CTRL`, becomes `FREEKB` on the `MAP` statement. It is the recommended way to bring the screens of an existing CICS application to bricks:

```
bricksconvert --out=runtime/map/am01.map am01.bms
```

### Adapting to the Terminal Size

A 3270 connection negotiates its screen size when it connects (see "Terminals" in Chapter 1). When it erases the screen, bricks uses the Erase/Write Alternate command, so the terminal uses and clears its whole screen; but a 24-row map painted on a 43-row screen leaves rows 24 through 42 blank. To use the larger screen, a program sends a map designed for it. It learns the size with `ASSIGN`:

```
EXEC CICS ASSIGN SCREENHT(SCRH) SCREENWD(SCRW) END-EXEC
```

`ALTSCRNHT` and `ALTSCRNWD` return the same values; bricks does not distinguish a default and an alternate size.

#### *Naming convention*

The convention used by the supplied programs is one map per model. The Model 2 map keeps the plain name; maps for larger models add a one-letter suffix.

| Model | Size | Suffix | Example |
|---|---|---|---|
| Model 2 | 24 x 80 | (none) | HEL01, CUST1 |
| Model 3 | 32 x 80 | M | HEL01M |
| Model 4 | 43 x 80 | L | HEL01L, CUST1L |
| Model 5 | 27 x 132 | W | BRDS1W |

**Figure 3-2. Map name suffixes by terminal model**

The HELO transaction (`runtime/rexx/hello.rexx`) chooses the map like this:

```
SUFFIX = ''
IF SCRH >= 43 THEN SUFFIX = 'L'
ELSE IF SCRH >= 32 THEN SUFFIX = 'M'
ELSE IF SCRW >= 132 THEN SUFFIX = 'W'
MAPNAME = 'HELO1' || SUFFIX
EXEC CICS SEND MAP(MAPNAME) FROM(SCR.) ERASE END-EXEC
IF EIBRESP = 36 THEN DO      /* MAPFAIL: no map for this size     */
  EXEC CICS SEND MAP('HELO1') FROM(SCR.) ERASE END-EXEC
END
```

The tests run from the largest screen down, so that a Model 3 is not taken for a Model 4. There is no `HELO1W`, so on a Model 5 the first `SEND MAP` gets `MAPFAIL` and the program falls back to `HELO1`. A REXX program can test `EIBRESP` without coding `RESP`, because an unhandled condition does not end a REXX task; a COBOL or PL/I program must code `RESP` on the first `SEND MAP`, or the `MAPFAIL` ends the task with abend `AEI9`. The PL/I sample `runtime/pli/cusp.pli` does this:

```
EXEC CICS ASSIGN USERID(UID) TERMID(TID) SCREENHT(SCRH) RESP(RC);
IF SCRH >= 43 THEN SUFFIX = 'L';
M_MENU   = 'CUST1' || SUFFIX;
M_DETAIL = 'CUST2' || SUFFIX;
```

and falls back when `RC = DFHRESP(MAPFAIL) & M_MENU <> 'CUST1'`.

Figure 3-3 and Figure 3-4 show the result. On a Model 2 terminal HELO sends `HELO1`: the information line (field `INFOLINE`) on row 1, the greeting (`GREETING`), and the key legend (`FOOTER`) on row 24. On a Model 4 terminal it sends `HELO1L`, which carries the same three fields in the same places and uses the extra rows for the System information and Recent activity panes. The program fills both maps from the same `SCR.` stem.

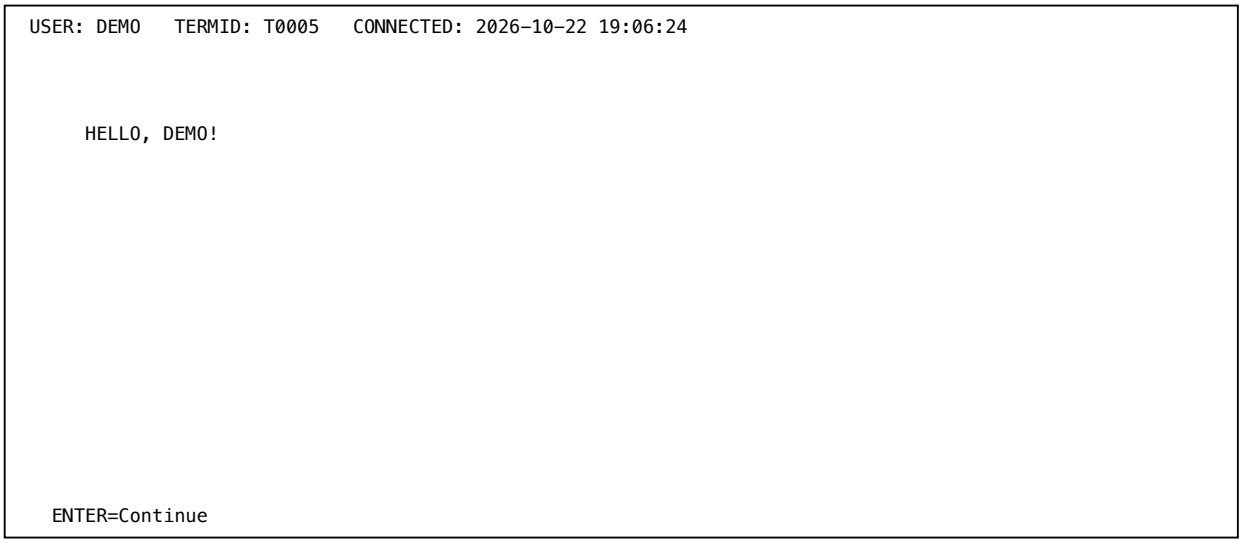



**Figure 3-3. HELO on a Model 2 terminal: map HELO1** (Model 2, 24 x 80; capture screens/helo.txt)

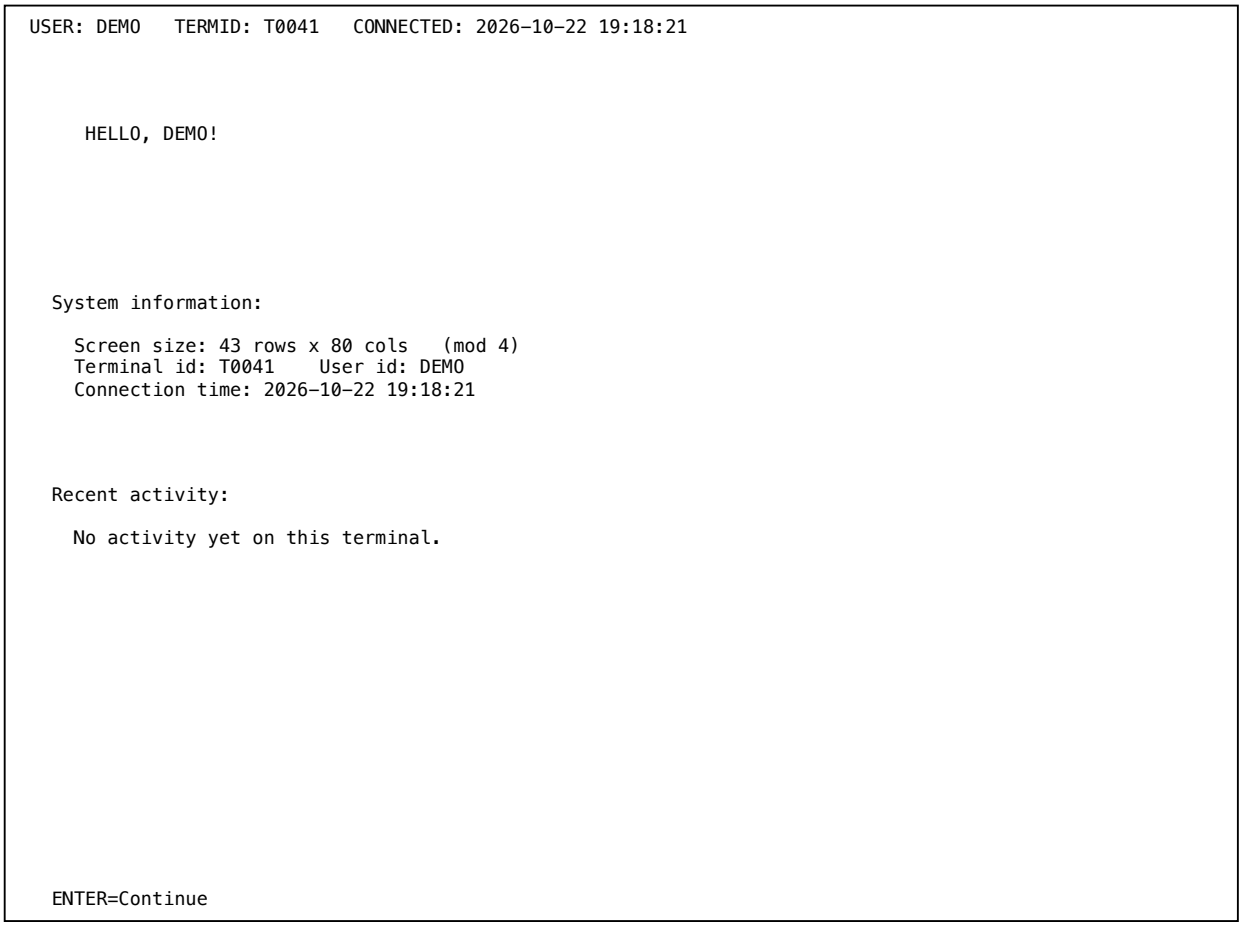



**Figure 3-4. HELO on a Model 4 terminal: map HELO1L** (Model 4, 43 x 80; capture screens/helo-mod4.txt)

*Why the convention works*

1. **The maps of a family use the same field names.** `helo1.map`, `helo1m.map` and `helo1l.map` all declare `INFOLINE`, `GREETING` and `FOOTER`, so one `SCR.` stem fills any of them. Values for fields that only the larger maps have (`INFO1`, `ACT1`, ...) are ignored by a map that does not declare them.
2. **A missing map is not fatal.** If the sized map does not exist, the program falls back to the Model 2 map, so deleting `helo1l.map` does not break Model 4 terminals; they see the 24 x 80 layout.
3. **Paging adapts at runtime.** The list program `runtime/rexx/cusl.rexx` shows 35 rows per page on a Model 4 and 15 on a Model 2, choosing `CUSTLL` or `CUSTL`; `cusp.pli` does the same. A PL/I program declares the list structure with the largest row count of the family (`2 ROW(35)`); the smaller map ignores the extra elements.

bricks supplies sized maps for the demonstration transactions:

| Map file | Size | Contents |
|---|---|---|
| `helo1.map` | `24 x 80` | HELO greeting |
| `helo1m.map` | `32 x 80` | HELO greeting with the system-information and recent-activity panes packed into 32 rows |
| `helo1l.map` | `43 x 80` | HELO greeting with system-information and recent-activity panes |
| `cust1.map` | `24 x 80` | Customer menu |
| `cust1l.map` | `43 x 80` | Customer menu with recent-activity history |
| `cust2.map` | `24 x 80` | Customer detail |
| `cust2l.map` | `43 x 80` | Customer detail with an audit-log pane |
| `custl.map` | `24 x 80` | Customer list, 15 rows |
| `custll.map` | `43 x 80` | Customer list, 35 rows |
| `brds1.map, brds1m.map, brds1l.map, brds1w.map` | `24x80 to 27x132` | BRDS, one map for each of the four models |

**Figure 3-5. Sized map families supplied with bricks**

To see the difference, connect once with `c3270 -model 2` and once with `c3270 -model 4`, sign on, and run `HELO`, `CUST` or `CUSP`.

Part 2

# Supplied Transactions

This part describes the transactions that are supplied with bricks and that a programmer uses at a display station: to sign on and off, to inquire on and control the region, to define resources, to issue commands interactively, to trace a transaction as it runs, to edit source, and to define and maintain files. The abend analyser CAID is described in Chapter 16, "Recovery and Condition Handling", together with the transaction dumps it reads.

**Chapter 4. Signing On and Off: CSSN and CSSF** ..... **4-1**
**Chapter 5. CEMT: The Master Terminal Transaction** ..... **5-1**
**Chapter 6. CEDA: Resource Definition Online** ..... **6-1**
**Chapter 7. CECI: The Command-Level Interpreter** ..... **7-1**
**Chapter 8. CEDF and CEDG: The Execution Diagnostic Facility** ..... **8-1**
**Chapter 9. ISPF: The Source Editor** ..... **9-1**
**Chapter 10. IDCAMS: Access Method Services** ..... **10-1**

Chapter 4

# Signing On and Off: CSSN and CSSF

This chapter describes what the terminal operator sees from the moment a 3270 session connects to a bricks region: the logo splash, the sign-on prompt, the `CSSN` sign-on transaction, the blank screen on which transaction identifiers are typed, and the `CSSF` sign-off transaction. It also describes the optional table of long transaction aliases, `runtime/aliases.conf`.

The supplied transactions described in this part of the publication (`CSSN`, `CSSF`, `CEMT`, `CEDA`, `CECI`, `CEDF`, `CEDG`, `ISPF` and `IDCA`), and the abend analyser `CAID` described in "CAID: The Abend Analyser" in Chapter 16, are built into the region. They have no entry in `runtime/transactions.conf`, and a definition of the same name there is never dispatched. Each is protected by a fixed group requirement, shown in the following table.

| Transid | Function | Required group | Chapter |
|---|---|---|---|
| `CSSN` | Sign on (alias `CESN`) | None; typed at the prompt only | 4 |
| `CSSF` | Sign off (alias `CESF`) | None | 4 |
| `CEMT` | Master terminal | None; CONTROLBLOCKS and PERFORM need `admin` | 5 |
| `CEDA` | Resource definition | `admin` | 6 |
| `CECI` | Command-level interpreter | `dev` | 7 |
| `CEDF, CEDG` | Execution diagnostic facility | `dev` or `admin` | 8 |
| `ISPF` | Source editor | `dev` | 9 |
| `IDCA, IDCAMS` | Access method services | `admin` | 10 |
| `CAID` | Abend analyser | `admin` | 16 |

**Figure 4-1. The supplied transactions and their group requirements**

Groups are assigned to each user in `runtime/users.conf` ("USER" in Chapter 6 describes the CEDA USER panel that maintains it). The same requirement applies when a program issues `EXEC CICS LINK` or `XCTL` to a supplied transaction: a refused `LINK` returns `PGMIDERR`, and a refused `XCTL` raises `NOTAUTH` at the `XCTL`, before the issuing program gives up control.

## Connecting to the Region

When a 3270 session connects, the region negotiates the terminal type and screen size, assigns a terminal identifier (`T0001`, `T0002`, ...) and paints the logo splash shown in Figure 4-2.

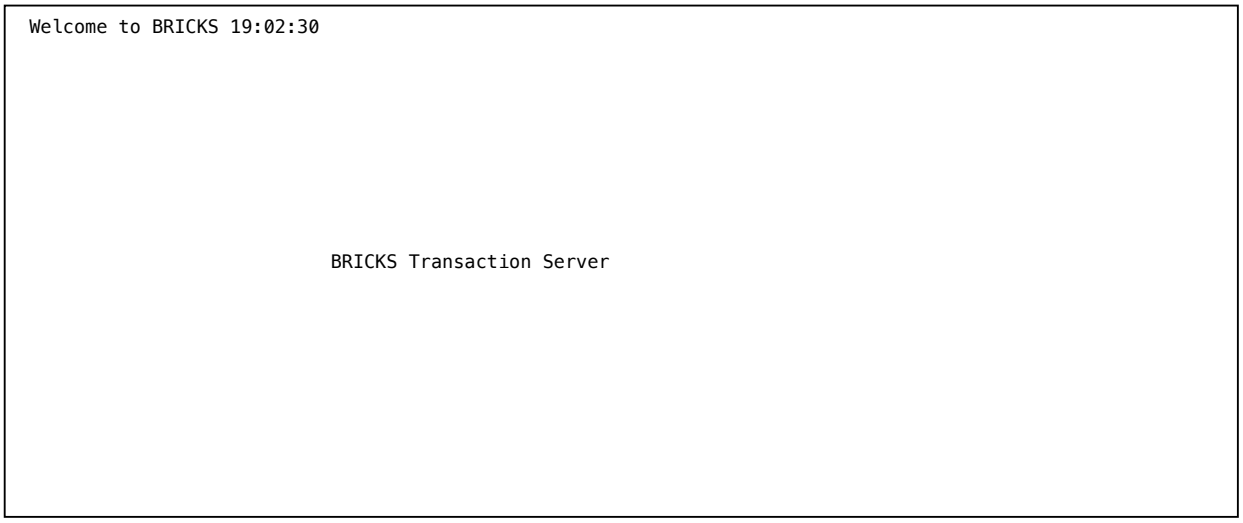



**Figure 4-2. The logo splash painted when a terminal connects** (Model 2, 24 x 80; capture screens/region-splash.txt)

Row 1 carries the good-morning text: the value of the `gmtext=` parameter in `bricks.cnf`, followed by the time of day. The BRICKS Transaction Server logo is centered on the screen. The logo is painted in color graphics, which a text capture cannot show; only its caption line appears in the figure. The splash has no input fields: any attention key leaves it. Setting `show_welcome=no` omits the splash, and the terminal goes straight to the prompt.

When `enforce_secure_login=yes` is in effect and the terminal is not yet signed on, the region next displays the sign-on prompt shown in Figure 4-3.

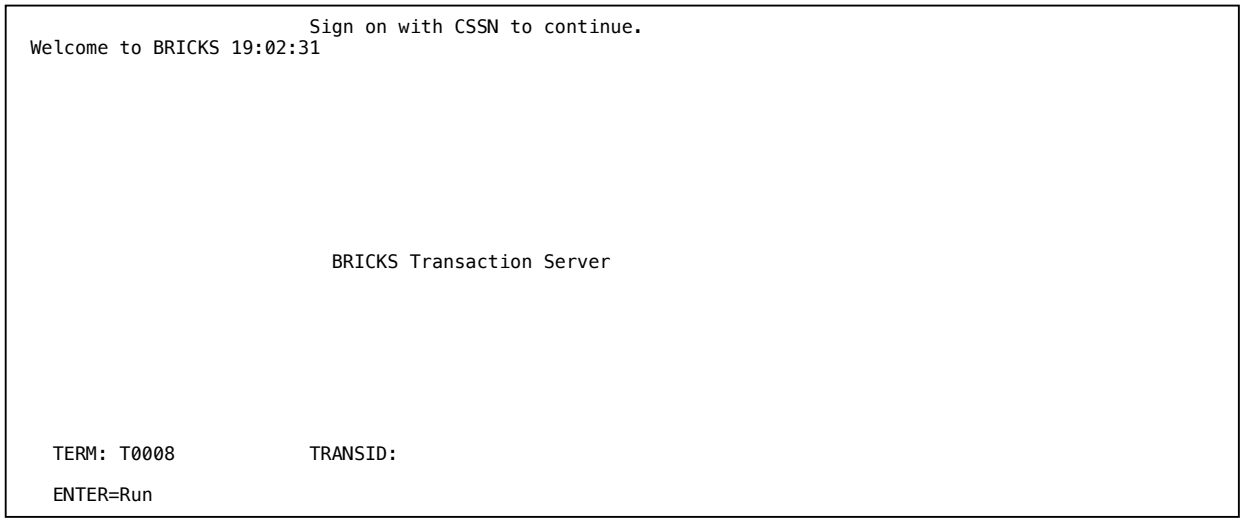



**Figure 4-3. The sign-on prompt of a terminal that is not signed on** (Model 2, 24 x 80; capture screens/region-prompt.txt)

The lines of the sign-on prompt are:

**`Sign on with CSSN to continue.`**

The notice on row 1. It names the logon transaction, which is `CSSN` unless `bricks.cnf` names another (`logon_transid=`).

**`Welcome to BRICKS 19:02:31`**

The good-morning text and the time, as on the splash.

**`TERM: T0008`**

The terminal identifier assigned to this session. It is the value of `EIBTRMID` in every task the terminal runs.

**`TRANSID:`**

An input field of eight characters. Type `CSSN` (or `CESN`) and press **`ENTER`**. A long alias from `aliases.conf` may also be typed here. Arguments typed after the transaction identifier are ignored on this prompt.

**`ENTER=Run`**

The key legend. **`F3`**, **`CLEAR`** and the PA keys redisplay the prompt; they do not disconnect the terminal.

Any transaction identifier other than the logon transaction typed on this prompt is refused with `Not signed on. Run CSSN first.` A terminal that does not sign on within `idle_timeout_secs` seconds (default 900) is disconnected. Once the operator has signed on, no time limit applies.

When `enforce_secure_login=no` is in effect, the terminal goes from the splash directly to the blank screen ("The Blank Screen"), and transactions that require no group may be run without signing on.

### Signing On: CSSN

The `CSSN` transaction identifies the operator to the region. It displays the panel shown in Figure 4-4; the operator has typed the user identifier `DEMO` and has not yet typed the password.

```
                            CSSN SIGN-ON

TERMID:    T0008
Userid:    DEMO
Password:




Sign on with ENTER.
```

**Figure 4-4. The CSSN sign-on panel** (Model 2, 24 x 80; capture screens/cssn-signon.txt)

The fields of the sign-on panel are:

**TERMID**

The terminal identifier. Protected.

**Userid**

The user identifier, up to eight characters, as defined in `runtime/users.conf`. The cursor is placed in this field when the panel is displayed.

**Password**

The password, up to 32 characters. The field is not displayed as it is typed. The password is compared exactly as typed: upper and lower case are different characters, and only the spelling with which the password was enrolled is accepted.

**Sign on with ENTER.**

The legend, on the last row of the screen whatever the terminal model. A message line is painted two rows above it when an attempt fails.

Only **ENTER** submits the panel. Every other attention key (**F3**, **CLEAR**, the PA keys and the other PF keys) redisplays it, keeping the user identifier and clearing the password. The operator leaves CSSN only by signing on successfully or by exhausting the permitted attempts.

The following messages may appear on the message line:

| Message | Meaning |
|---|---|
| `Userid required.` | ENTER was pressed with the Userid field empty. This does not count as a failed attempt. |
| `Sign-on failed (n of 3). Check userid and password.` | The user identifier is unknown or the password is wrong. The message does not say which. |

**Figure 4-5. CSSN sign-on messages**

After the third consecutive failed attempt on one connection, the region writes the line `term=Tnnnn disconnecting after 3 failed sign-on attempts` to its log and drops the connection. The count starts again when the terminal reconnects.

When the user identifier and password are accepted, the region attaches the user and the user's groups to the terminal and displays the confirmation shown in Figure 4-6. Pressing **ENTER** clears the screen and leaves the terminal at the blank screen.

```
BRICKS Transaction Server                                TERM T0008


CSSN sign-on successful as DEMO. Type a transaction id and press ENTER.









Press ENTER to continue.
```

**Figure 4-6. Sign-on complete** (Model 2, 24 x 80; capture screens/cssn-signed-on.txt)

The confirmation is the map `CSSNOK` (or *xxxx*`OK`, where *xxxx* is a logon transaction other than CSSN, when such a map exists). If `start_transaction=` names a transaction in `bricks.cnf`, that transaction is started at once instead, as if the operator had typed it; the confirmation is not shown.

**Note:**

CSSN can be started only by typing it at the prompt. A program that issues `EXEC CICS LINK` or `XCTL` to `CSSN` receives the message `CSSN cannot be invoked from a transaction; sign on at the blank prompt.`

### The Blank Screen

A signed-on terminal that is not running a task shows the blank screen: a cleared screen with one unprotected input field that starts in the top left-hand corner, as shown in Figure 4-7. The cursor is at row 1, column 2.

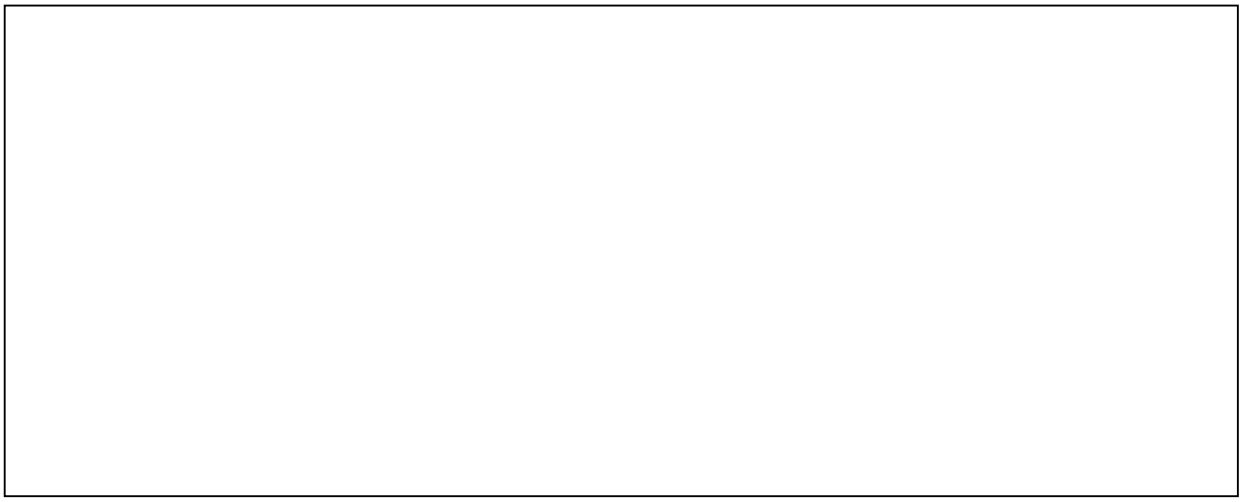


**Figure 4-7. The blank screen** (Model 2, 24 x 80; capture screens/blank-screen.txt)

The operator types a transaction identifier, optionally followed by a blank and arguments, and presses **ENTER**. Figure 4-8 shows the command `CSSF LOGOFF` typed before **ENTER**.

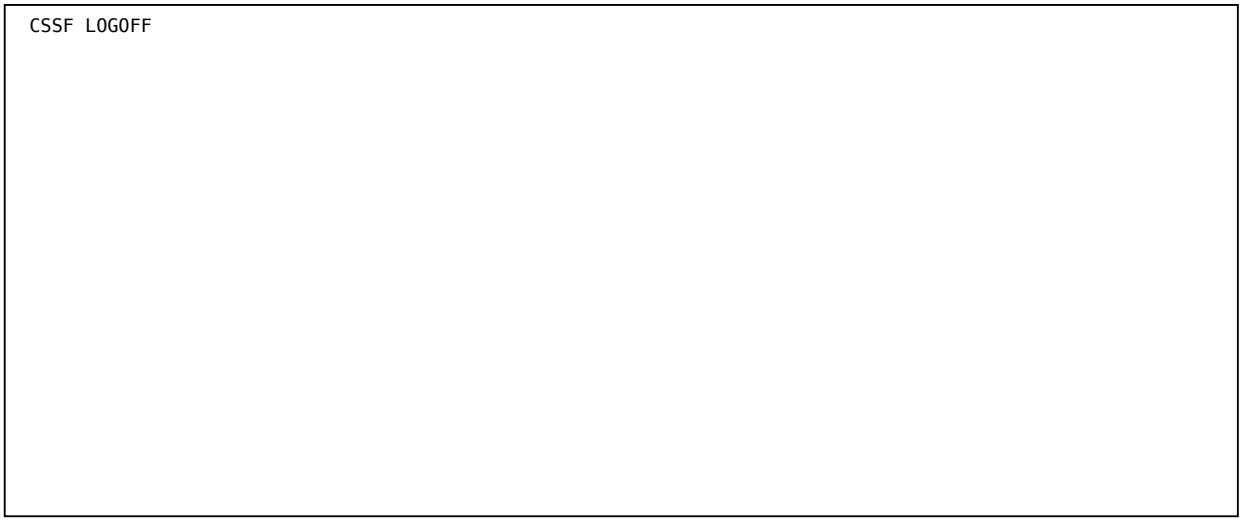



**Figure 4-8. A transaction identifier and argument typed on the blank screen** (Model 2, 24 x 80; capture screens/blank-screen-typed.txt)

The region interprets the line as follows:

- The first word is the transaction identifier. It is translated to upper case. If it is a long alias defined in `aliases.conf`, it is replaced by its target (see "Transaction Aliases"). `CESN` and `CESF` are replaced by `CSSN` and `CSSF`.
- The rest of the line, after the first blank, is the argument string. Its case is preserved. A REXX program receives it through `ARG` or `PARSE ARG`; a COBOL or PL/I program that issues `EXEC CICS RECEIVE INTO` as its first terminal command receives the whole line, transaction identifier included, as in IBM CICS.
- For `CEDF` and `CEDG` only, a comma also ends the transaction identifier, so that `CEDF,OFF` is read as `CEDF` with the argument `,OFF`.
- `EIBAID` is set to `X'7D'` (ENTER) for the task that starts.

**`ENTER`** on an empty line, **`F3`**, **`CLEAR`** and the PA keys redisplay the blank screen. None of them disconnects the terminal; a signed-on terminal stays connected until it signs off with `CSSF DISC` or the session is closed by the emulator.

***Errors at the blank screen***

A transaction identifier that is neither supplied nor defined in `runtime/transactions.conf` produces the message frame shown in Figure 4-9.

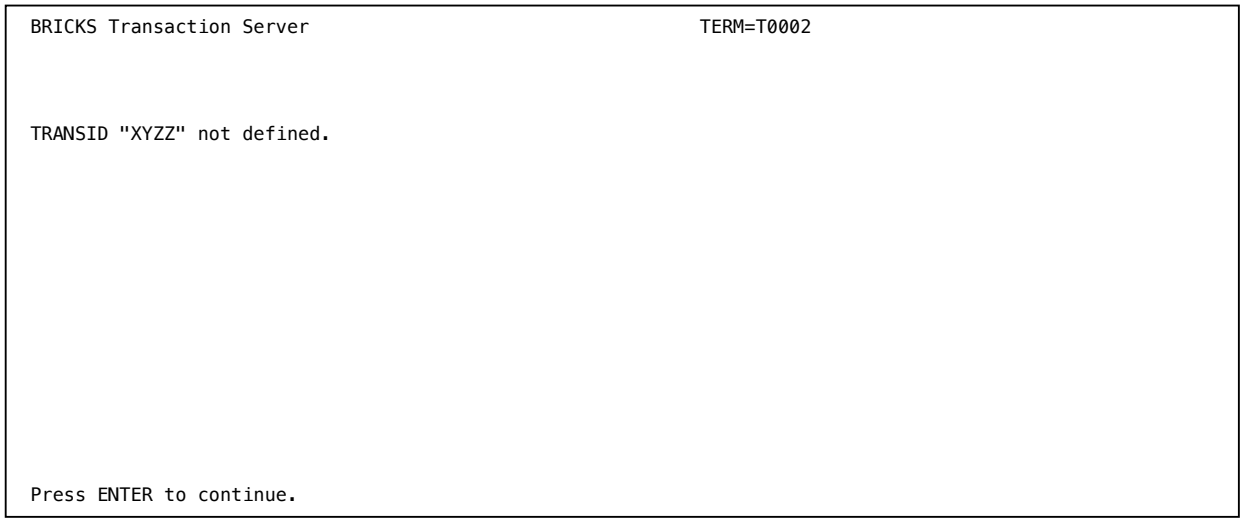



**Figure 4-9. An undefined transaction identifier** (Model 2, 24 x 80; capture screens/unknown-transid.txt)

Row 1 of every message frame carries the region banner and the terminal identifier; the message is on row 6, and `Press ENTER to continue.` is on row 23. Pressing **`ENTER`** returns to the blank screen.

A transaction that the operator's groups do not permit is refused in the same frame. Figure 4-10 shows the user `CLERK`, who belongs only to the group `users`, typing `CECI`.

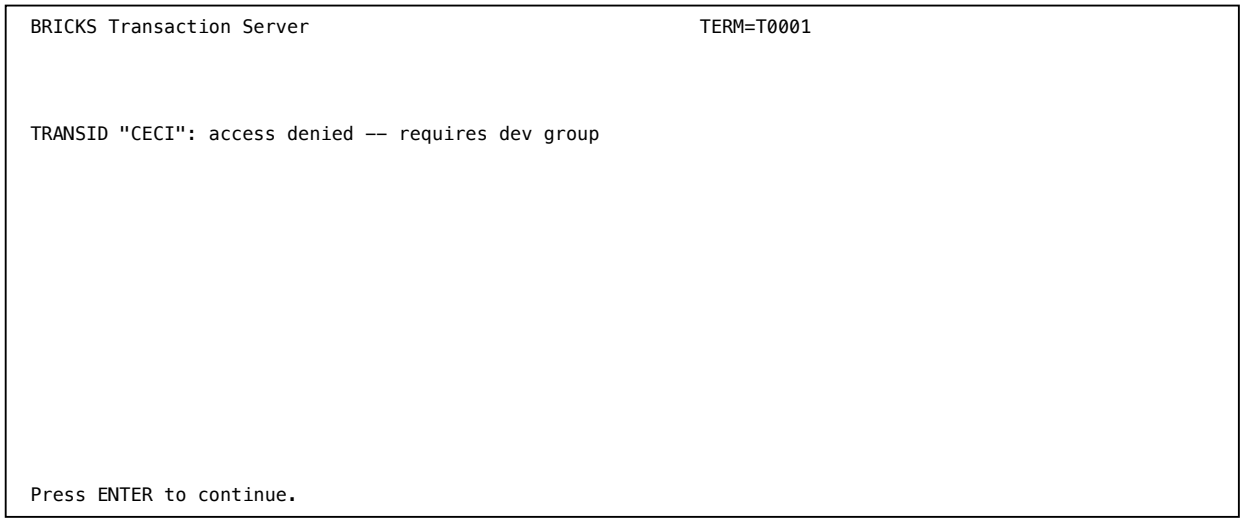



**Figure 4-10. A supplied transaction refused for want of a group** (Model 2, 24 x 80; capture screens/builtin-access-denied.txt)

The text after `--` names the group that is required. The refusal is also written to the region log with the user, the user's groups and the required groups. Transactions defined in `transactions.conf` are refused the same way when the user belongs to none of the groups in the definition.

### Signing Off: CSSF

The `CSSF` transaction (alias `CESF`) signs the operator off. It requires one argument. Figure 4-11 shows the response to `CSSF` typed alone, or with an argument it does not recognize.

```
BRICKS Transaction Server                                TERM=T0008


Usage: CSSF LOGOFF | DISC | GOODNIGHT









Press ENTER to continue.
```

**Figure 4-11. CSSF without an argument** (Model 2, 24 x 80; capture screens/cssf-usage.txt)

The arguments are:

**`CSSF LOGOFF`**

Detaches the user from the terminal and clears the terminal's user identifier and groups. The connection stays open: the region paints the logo splash with the message `DFHCE3590 Sign-off is complete.` on the last row, as shown in Figure 4-12, and then returns to the sign-on prompt. The idle time limit applies again from this point.

**`CSSF DISC`**, **`CSSF DISCONNECT`**, **`CSSF GOODNIGHT`**

Signs the user off as for LOGOFF, displays `CSSF DISC: `*`user`*` signed off. Disconnecting in 1 second.`, and closes the connection one second later.

```
Welcome to BRICKS 19:02:46




                          BRICKS Transaction Server




DFHCE3590 Sign-off is complete.
```

**Figure 4-12. The splash after CSSF LOGOFF** (Model 2, 24 x 80; capture screens/cssf-logoff.txt)

The arguments may be typed in any case. Signing off also switches off EDF mode for the terminal ("Switching EDF On and Off" in Chapter 8). When the last terminal of a user signs off, the user's control block is discarded, so the user no longer appears among the signed-on users of CEMT INQUIRE USER.

### Transaction Aliases

The optional file `runtime/aliases.conf` (named by `aliases_file=` in `bricks.cnf`) defines long, easily remembered names for installed transactions. An operator who types an alias at the blank screen or on the sign-on prompt runs the transaction it names, with the same arguments: `sabre JFK LAX` runs exactly as `SABR JFK LAX` does. The file looks like this:

```
#aliases for transactions
# Format: alias:TRANSID  (alias 2..8 alphanumeric, target 4 chars)
sabre:SABR
customers:CUST
custpli:CUSP
```

The rules are:

- Each line is *`alias`*`:`*`TRANSID`*. The alias is two to eight letters or digits and is matched in any case; the target is exactly four letters or digits.
- A line that begins with `#`, and the rest of a line after a blank followed by `#`, are comments. A `#` with no blank before it is data, so `sab#re:BOOK` is an invalid alias and is rejected with `aliases.conf:N: alias "SAB#RE": need 2..8 alnum`.
- If an alias is defined twice, the first definition is used and the later one is skipped with a log line.
- A real name always wins. An alias named after a supplied transaction, `CESN`, `CESF`, the logon transaction, or a transaction defined in `transactions.conf` has no effect.
- An alias is resolved once; its target is not looked up as an alias again.
- Aliases apply only to what the operator types. They are never used for `RETURN TRANSID`, `XCTL`, `START`, web routes, MRO or `start_transaction`.
- The file is optional; when it is missing there are no aliases. It is reread when it changes, so an edit takes effect without a restart.

There is no CEDA panel for aliases; the file is maintained with an editor.

Chapter 5

# CEMT: The Master Terminal Transaction

The master terminal transaction, `CEMT`, displays the resources of the running region and the performance of the process, and lets an administrator rescan definitions and resize the caches. It is the bricks counterpart of the IBM CICS master terminal transaction, with a menu-driven surface in place of IBM's command syntax.

Any signed-on user may run CEMT. The INQUIRE functions (except CONTROLBLOCKS) and the MONITOR functions are open to every user. The CONTROLBLOCKS sub-tree of INQUIRE and the whole PERFORM tree require the `admin` group, because they expose internal control blocks and change the running region. Definitions are changed with CEDA (Chapter 6), not with CEMT.

## The CEMT Menu

Type `CEMT` at the blank screen and press **`ENTER`**. Figure 5-1 shows the menu displayed to a member of the `admin` group.

```
CEMT                                                  TERM=T0010  19:02:58
3.5.4

Pick an option and press ENTER. PF3 to back out.


  I  INQUIRE  -- resources, CICS-style
  M  MONITOR  -- process metrics
  P  PERFORM  -- rescans + caches (admin)
  Q  QUIT

Choice:









ENTER=Select  PF3=Back
```

**Figure 5-1. The CEMT menu displayed to an administrator** (Model 2, 24 x 80; capture screens/cemt-menu.txt)

The lines of the menu are:

**Row 1**

The title (here `CEMT`), the terminal identifier and the time. On the panels below the menu the title is the path of the panel, for example `INQUIRE CONTROLBLOCKS TCBS`.

**Row 2**

The level of bricks that is running (`3.5.4`). Shown on the top menu only.

**Option lines**

One line per choice: the letter to type, the name of the choice and, after `--`, a short description or a live summary.

**`Choice:`**

The input field in which the letter is typed.

**Row 24**

The key legend: **`ENTER`** selects, **`F3`** returns to the previous menu, and from the top menu ends CEMT.

The choices are `I` INQUIRE, `M` MONITOR, `P` PERFORM and `Q` QUIT. `Q` has the same effect as **`F3`**. A choice that the user is not entitled to is not shown at all: Figure 5-2 shows the menu displayed to `CLERK`, who belongs only to the group `users`; the PERFORM line is absent. If such a user types a path into an administrator function directly, CEMT displays an access-denied message and returns to the blank screen.

```
CEMT                                                    TERM=T0001  19:20:33
3.5.4

Pick an option and press ENTER. PF3 to back out.


  I  INQUIRE  -- resources, CICS-style
  M  MONITOR  -- process metrics
  Q  QUIT

Choice:









ENTER=Select  PF3=Back
```

**Figure 5-2. The CEMT menu displayed to a user who is not an administrator** (Model 2, 24 x 80; capture screens/cemt-menu-nonadmin.txt)

### *Typing a path*

Every choice may be typed in full or abbreviated to its letter or to any unambiguous beginning of its name. The words may also be typed on the blank screen after `CEMT`, so that CEMT goes straight to the panel without displaying the menus between. For example, each of the following displays the transaction list:

```
CEMT I R
CEMT INQ TRANS
CEMT INQUIRE TRANSACTION
```

A letter is matched first; otherwise a word must be the beginning of exactly one name. An unknown word is answered with `unknown "X" -- try S, F, U, ...`, and an ambiguous one with `ambiguous "X" -- matches ...`. A word after the name of a panel that has no further choices is answered with `takes no arguments`.

When the user leaves a panel that was reached by a typed path, the transaction ends and the terminal returns to the blank screen. When the panel was reached through the menus, **`F3`** returns to the menu above it, and **`F3`** from the top menu ends the transaction.

## INQUIRE

The INQUIRE menu, shown in Figure 5-3, lists the resource displays. All are read-only.

```
INQUIRE                                                TERM=T0010  19:02:59


Pick an option and press ENTER. PF3 to back out.


  S  TS
  F  FILE
  U  USER
  T  TERMINAL
  R  TRANSACTION
  W  WEB
  M  URIMAP
  O  MRO  -- (single region)
  E  ENQ  -- 0 held, 0 waiting
  C  CONTROLBLOCKS  -- TCBs / UCBs / TXCBs / FCBs (admin)

Choice:




ENTER=Select  PF3=Back
```

**Figure 5-3. The INQUIRE menu** (Model 2, 24 x 80; capture screens/cemt-inquire-menu.txt)

The summaries show live counts: `MRO -- (single region)` when no peer regions are defined, and the number of enqueues held and waiting. The `C CONTROLBLOCKS` line appears only for administrators.

### *Layout of the list panels*

The INQUIRE panels share one layout. Row 1 is the title line; row 2 holds the column headings; the rows below list one resource each, sorted by the first column; the second-last row reads `Page` *p* `of` *n* `rows` *a*–*b* `of` *t*; the last row is the key legend. Columns are sized to the widest value, and a value too long for the screen is shortened. The keys are:

| Key | Function |
|---|---|
| ENTER | Refreshes the panel with current values. |
| F7 | Displays the previous page. |
| F8 | Displays the next page. |
| F3 | Returns to the previous menu. CLEAR has the same effect. |

**Figure 5-4. Keys on the CEMT list panels**

A Model 4 screen shows more rows on each page. Figure 5-10 on a later page shows the transaction list on a 43-row screen.

### *INQUIRE TS*

INQUIRE TS (`CEMT I S`) lists the temporary-storage queues in the region, as shown in Figure 5-5.

```
INQUIRE TS                                                TERM=T0012  19:22:16
QUEUE     ITEMS  READS  WRITES  REWRT  LASTACC       STATUS
AUDITLOG  1      0      2       0      19:22:11.557  ENABLED
ORDERS    2      2      4       0      19:22:12.162  ENABLED









Page 1 of 1   rows 1–2 of 2
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-5. INQUIRE TS** (Model 2, 24 x 80; capture screens/cemt-inquire-ts.txt)

The columns are `QUEUE`, the queue name; `ITEMS`, the number of items in the queue; `READS`, `WRITES` and `REWRT`, the number of `READQ TS`, `WRITEQ TS` and `WRITEQ TS REWRITE` commands since the region started; `LASTACC`, the time of the last access; and `STATUS`. Queues are deleted with CEDA QUEUES ("QUEUES" in Chapter 6).

### *INQUIRE FILE*

INQUIRE FILE (`CEMT I F`) lists the files of the file store, as shown in Figure 5-6.

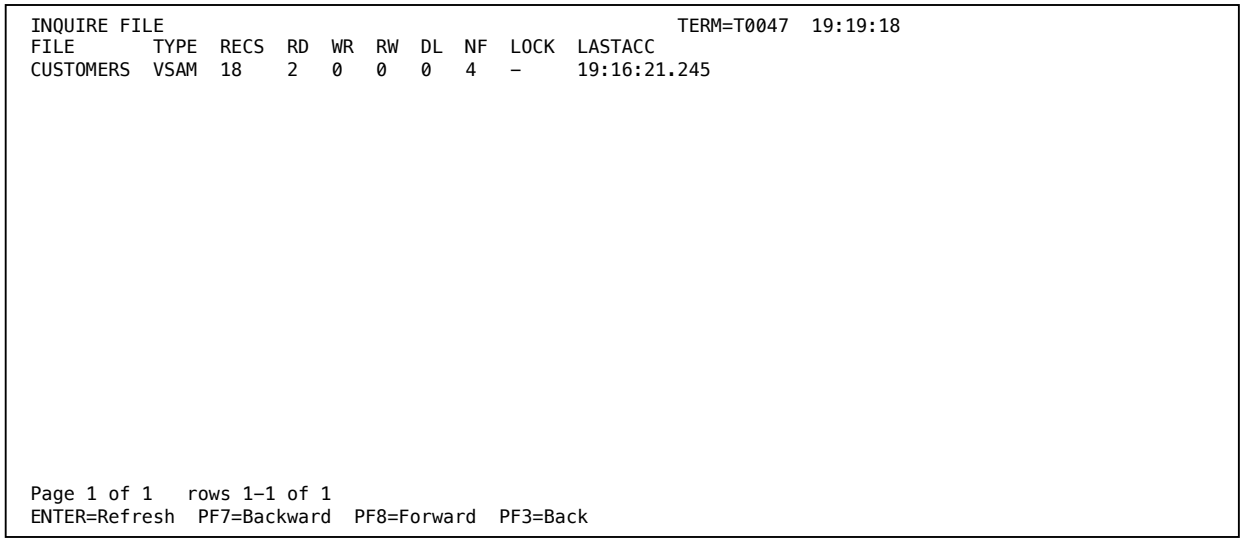



**Figure 5-6. INQUIRE FILE** (Model 2, 24 x 80; capture screens/cemt-inquire-files.txt)

The columns are `FILE`; `TYPE`; `RECS`, the number of records; `RD`, `WR`, `RW` and `DL`, the number of reads, writes, rewrites and deletes; `NF`, the number of requests that ended `NOTFND`; `LOCK`, the terminal that holds a record lock in the file, or `-`; and `LASTACC`, the time of the last access.

### *INQUIRE USER*

INQUIRE USER (`CEMT I U`) lists the users, as shown in Figure 5-7.

```
INQUIRE USER                                              TERM=T0048  19:19:22
USERID  GROUPS           LOGINS  TXNRUN  LASTLOGIN  TERMS
DEMO    admin,users,dev  3       0       19:19:21   T0046,T0048
admin   admin,users,dev  0       0       -          -









Page 1 of 1   rows 1-2 of 2
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-7. INQUIRE USER** (Model 2, 24 x 80; capture screens/cemt-inquire-users.txt)

The columns are `USERID`; `GROUPS`; `LOGINS`, the number of sign-ons since the region started; `TXNRUN`, the number of transactions run; `LASTLOGIN`; and `TERMS`, the terminals at which the user is signed on. A user signed on at two terminals has one row, with both terminals listed. Each user appears once, in one of four groups of rows, distinguished on a color terminal by color:

- users who are connected and signed on (red);
- terminals that are connected but not signed on, one row per terminal with the user identifier `(none)` (pink);
- users who signed on earlier in the life of the region and have since disconnected, the most recent first (turquoise);
- users defined in `users.conf` who have not signed on since the region started (yellow), at the end of the list.

The record of earlier sign-ons is kept in storage only; it is lost when the region stops.

### *INQUIRE TERMINAL*

INQUIRE TERMINAL (`CEMT I T`) lists the connected terminals, as shown in Figure 5-8.

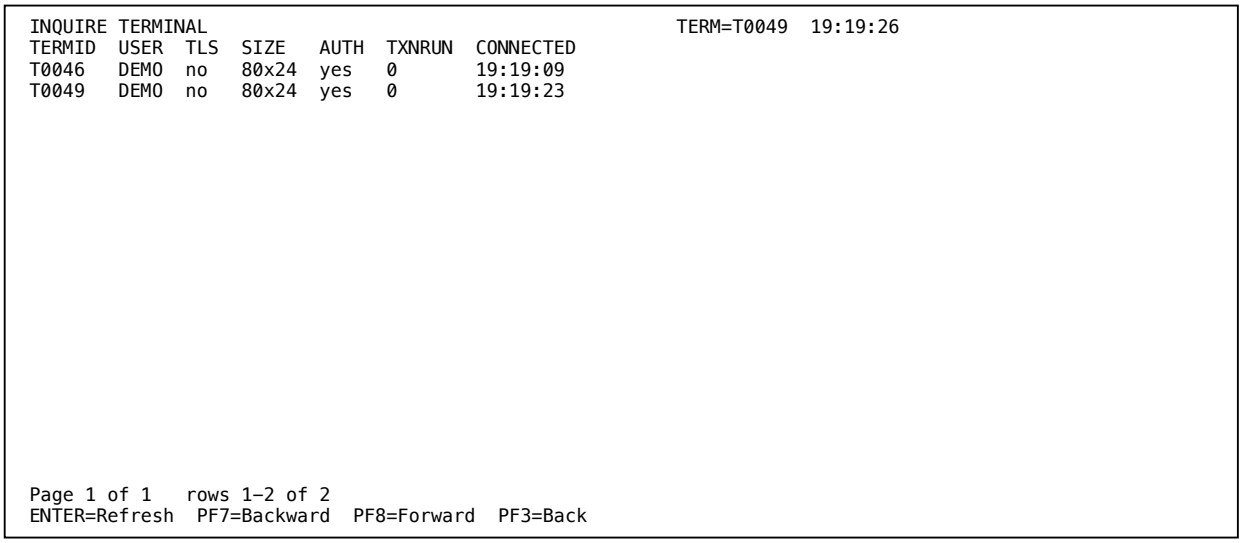



**Figure 5-8. INQUIRE TERMINAL** (Model 2, 24 x 80; capture screens/cemt-inquire-terminals.txt)

The columns are `TERMID`; `USER`; `TLS`, whether the connection is encrypted; `SIZE`, the screen size in columns and rows; `AUTH`, whether the terminal is signed on; `TXNRUN`, the number of transactions run at the terminal; and `CONNECTED`, the time the terminal connected.

### *INQUIRE TRANSACTION*

INQUIRE TRANSACTION (`CEMT I R`) lists the transactions defined in `runtime/transactions.conf`, as shown in Figure 5-9.

```
INQUIRE TRANSACTION                                          TERM=T0020  19:04:04
TRANSID  LANG   PROGRAM     INVOKED  CACHED  CACHE%  RES  GROUPS
ABMP     PLI    abmp.pli    0        0       -       NOT  PUBLIC
ABMR     REXX   abmr.rexx   0        0       -       NOT  PUBLIC
ABNC     COBOL  abnc.cob    0        0       -       NOT  PUBLIC
ABNH     COBOL  abnh.cob    0        0       -       NOT  PUBLIC
ABNP     PLI    abnp.pli    0        0       -       NOT  PUBLIC
APCH     COBOL  apch.cob    0        0       -       NOT  PUBLIC
BALC     COBOL  balc.cob    0        0       -       NOT  PUBLIC
BALR     REXX   balc.rexx   0        0       -       NOT  PUBLIC
BANK     COBOL  bank.cob    0        0       -       NOT  PUBLIC,USERS,ADMIN
BOOK     REXX   book.rexx   0        0       -       NOT  PUBLIC
BRDS     REXX   brds.rexx   0        0       -       NOT  USERS
CHAT     REXX   chat.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
CHEL     COBOL  help.cob    0        0       -       NOT  PUBLIC
CONS     REXX   cons.rexx   0        0       -       NOT  USERS,ADMIN
CSGM     REXX   csgm.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
CSSR     REXX   cssr.rexx   0        0       -       NOT  USERS,ADMIN
CUSL     REXX   cusl.rexx   0        0       -       NOT  PUBLIC
CUSP     PLI    cusp.pli    0        0       -       NOT  PUBLIC
CUST     REXX   cust.rexx   0        0       -       NOT  PUBLIC
CUSV     REXX   cusv.rexx   0        0       -       NOT  PUBLIC
Page 1 of 3   rows 1-20 of 60
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-9. INQUIRE TRANSACTION** (Model 2, 24 x 80; capture screens/cemt-inquire-transactions.txt)

The columns are:

**`TRANSID`, `LANG`, `PROGRAM`**

The transaction identifier, the language (`REXX`, `COBOL` or `PLI`) and the program file.

**`INVOKED`**

The number of times the transaction has been started.

**`CACHED`, `CACHE%`**

The number of starts that took the program from the program cache instead of reading and parsing the file, and that number as a percentage of `INVOKED` (`-` before the first start).

**`RES`**

`RES` when the program is resident in the program cache now, so that the next start skips the parse; `NOT` when the next start reads and parses the file.

**`GROUPS`**

The groups permitted to run the transaction, from the definition (`-` for a definition without a group list).

```
INQUIRE TRANSACTION                                          TERM=T0038  19:18:12
TRANSID  LANG   PROGRAM     INVOKED  CACHED  CACHE%  RES  GROUPS
ABMP     PLI    abmp.pli    0        0       -       NOT  PUBLIC
ABMR     REXX   abmr.rexx   0        0       -       NOT  PUBLIC
ABNC     COBOL  abnc.cob    2        1       50%     RES  PUBLIC
ABNH     COBOL  abnh.cob    0        0       -       NOT  PUBLIC
ABNP     PLI    abnp.pli    0        0       -       NOT  PUBLIC
APCH     COBOL  apch.cob    0        0       -       NOT  PUBLIC
BALC     COBOL  balc.cob    0        0       -       NOT  PUBLIC
BALR     REXX   balc.rexx   0        0       -       NOT  PUBLIC
BANK     COBOL  bank.cob    0        0       -       NOT  PUBLIC,USERS,ADMIN
BOOK     REXX   book.rexx   0        0       -       NOT  PUBLIC
BRDS     REXX   brds.rexx   0        0       -       NOT  USERS
CHAT     REXX   chat.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
CHEL     COBOL  help.cob    0        0       -       NOT  PUBLIC
CONS     REXX   cons.rexx   0        0       -       NOT  USERS,ADMIN
CSGM     REXX   csgm.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
CSSR     REXX   cssr.rexx   0        0       -       NOT  USERS,ADMIN
CUSL     REXX   cusl.rexx   0        0       -       NOT  PUBLIC
CUSP     PLI    cusp.pli    0        0       -       NOT  PUBLIC
CUST     REXX   cust.rexx   0        0       -       NOT  PUBLIC
CUSV     REXX   cusv.rexx   0        0       -       NOT  PUBLIC
ESDC     COBOL  esdc.cob    0        0       -       NOT  PUBLIC,USERS,ADMIN
ESDR     REXX   esdr.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
ESPF     REXX   espf.rexx   0        0       -       NOT  ADMIN
EXAM     COBOL  exam.cob    0        0       -       NOT  PUBLIC
EXAR     REXX   exam.rexx   0        0       -       NOT  PUBLIC
FSPF     REXX   fspf.rexx   0        0       -       NOT  PUBLIC
FUNC     REXX   func.rexx   0        0       -       NOT  PUBLIC
GETC     REXX   getc.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
GMPT     COBOL  gmpt.cob    0        0       -       NOT  PUBLIC
GUSL     COBOL  gusl.cob    0        0       -       NOT  PUBLIC
GUST     COBOL  gust.cob    8        7       88%     RES  PUBLIC
GUSV     COBOL  gusv.cob    0        0       -       NOT  PUBLIC
HELC     COBOL  hello.cob   3        2       67%     RES  PUBLIC
HELO     REXX   hello.rexx  1        0       0%      RES  PUBLIC
HELP     REXX   help.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
INQR     REXX   inqr.rexx   0        0       -       NOT  PUBLIC
MNDL     REXX   mndl.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
MNDU     REXX   mndu.rexx   0        0       -       NOT  PUBLIC,USERS,ADMIN
ORDR     COBOL  ordr.cob    0        0       -       NOT  PUBLIC
Page 1 of 2   rows 1-39 of 60
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-10. INQUIRE TRANSACTION on a Model 4 screen** (Model 4, 43 x 80; capture screens/cemt-inquire-transactions-mod4.txt)

Figure 5-10 shows the same panel on a Model 4 screen after a few transactions have run: 39 rows fit on a page, and the cache columns show the effect of the program cache.

### *INQUIRE WEB and INQUIRE URIMAP*

INQUIRE WEB (`CEMT I W`) lists the inbound web routes, with the columns `METHOD`, `PATTERN`, `TRANSID`, `HITS`, `ERR`, `LAST` and `AVG`. INQUIRE URIMAP (`CEMT I M`) lists the outbound URIMAP definitions, with the columns `NAME`, `SCHEME`, `HOST`, `PORT` and `PATH-PREFIX`. In a region without the web interface they display `INQUIRE WEB: WAPI not configured` and `INQUIRE URIMAP: WAPI / web_routes.conf not configured`. Web services are described in Chapter 20.

### *INQUIRE MRO*

INQUIRE MRO (`CEMT I O`) lists the peer regions defined in `runtime/mro.conf`, with the columns `REGION`, `HOST`, `PORT`, `STATUS`, `LASTOK` (the time of the last successful probe) and `RTT` (the round-trip time of the probe). In a region with no peers it displays `Single Region Operation`, as shown in Figure 5-11.

```
INQUIRE MRO                                          TERM=T0023  19:04:16
REGION    HOST                    PORT   STATUS       LASTOK    RTT

                           Single Region Operation









PF3=End
```

**Figure 5-11. INQUIRE MRO in a single region** (Model 2, 24 x 80; capture screens/cemt-inquire-mro.txt)

The values of `STATUS` are:

| Status | Meaning |
|---|---|
| ACK | The peer responds and lists this region in its own `mro.conf`: the link works in both directions. |
| REACHABLE | The peer responds but does not list this region. |
| UNREACHABLE | The peer was probed and did not respond. |
| Unknown | The peer has not been probed since the region started. |

**Figure 5-12. INQUIRE MRO status values**

### *INQUIRE ENQ*

INQUIRE ENQ (`CEMT I E`) shows who holds each resource named on an `EXEC CICS ENQ`, and which tasks are waiting for it. It is the panel to use when a terminal has stopped responding: an `ENQ` without `NOSUSPEND` suspends the task, and nothing else in the region shows it. Figure 5-13 shows the panel when no resource is held.

```
INQUIRE ENQ                                          TERM=T0024  19:04:20
RESOURCE                 OWNER       TRANID  TERM    STATE    HOLD  AGE

                              No enqueues held.









PF3=End
```

**Figure 5-13. INQUIRE ENQ with no enqueues held** (Model 2, 24 x 80; capture screens/cemt-inquire-enq.txt)

The columns are:

**RESOURCE**
: The resource name, as coded on `ENQ RESOURCE(...)`, shortened to 24 characters. Unprintable bytes are shown as `?`.

**OWNER, TRANID, TERM**
: The task that holds or waits for the resource, its transaction identifier and its terminal. Dashes are shown when the task has already ended.

**STATE**
: `OWNED` or `WAITING`. The IBM state `RETAINED` cannot occur in bricks.

**HOLD**
: `YES` when the enqueue was acquired so as to survive a syncpoint, `NO` when it is released at syncpoint, and `–` on a waiting row.

**AGE**
: How long the resource has been held or waited for.

On a color terminal waiting rows are yellow and held rows that survive a syncpoint are pink. The IBM columns ENQTYPE, ENQSCOPE, NETUOWID, ENQFAILS and RELATION are not shown, because bricks has no data for them.

## INQUIRE CONTROLBLOCKS

CONTROLBLOCKS (`CEMT I C`) displays the region's own control blocks. It requires the `admin` group. Figure 5-14 shows its menu, with the current number of blocks of each kind.

```
INQUIRE CONTROLBLOCKS                                    TERM=T0050  19:19:31


Pick an option and press ENTER. PF3 to back out.


  T  TCBS  -- (2 active terminals)
  U  UCBS  -- (1 signed-on users)
  X  TXCBS  -- (1 running transactions)
  F  FCBS  -- (1 known files)

Choice:









ENTER=Select  PF3=Back
```

**Figure 5-14. The CONTROLBLOCKS menu** (Model 2, 24 x 80; capture screens/cemt-inquire-controlblocks.txt)

TCBS (`CEMT I C T`), shown in Figure 5-15, lists the terminal control blocks, one per connected terminal, with the same columns as INQUIRE TERMINAL.

```
INQUIRE CONTROLBLOCKS TCBS                               TERM=T0051  19:19:35
TERMID  USER  TLS  SIZE   AUTH  TXNRUN  CONNECTED
T0046   DEMO  no   80x24  yes   0       19:19:09
T0051   DEMO  no   80x24  yes   0       19:19:31
```

**Figure 5-15. Terminal control blocks** (Model 2, 24 x 80; capture screens/cemt-inquire-tcbs.txt)

UCBS (`CEMT I C U`), shown in Figure 5-16, lists the user control blocks, one per signed-on user: `USERID`, `GROUPS`, `TERMS` (the number of terminals), `LOGINS`, `TXNRUN` and `LASTLOGIN`.

```
INQUIRE CONTROLBLOCKS UCBS                                  TERM=T0052  19:19:39
USERID  GROUPS           TERMS  LOGINS  TXNRUN  LASTLOGIN
DEMO    admin,users,dev  2      7       0       19:19:38









Page 1 of 1   rows 1-1 of 1
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-16. User control blocks** (Model 2, 24 x 80; capture screens/cemt-inquire-ucbs.txt)

TXCBS (`CEMT I C X`), shown in Figure 5-17, lists the transaction control blocks, one per running task: `ID`, the task identifier; `TRANSID`; `PROGRAM`; `TERM`; `USER`; `AGE`, the time the task has been running; and `FCBS`, the number of files the task has used. In the figure a second terminal is running GUST.

```
INQUIRE CONTROLBLOCKS TXCBS                                  TERM=T0053  19:19:43
ID        TRANSID  PROGRAM   TERM   USER  AGE      FCBS
X0000022  GUST     gust.cob  T0046  DEMO  30.814s  0









Page 1 of 1   rows 1-1 of 1
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-17. Transaction control blocks** (Model 2, 24 x 80; capture screens/cemt-inquire-txcbs.txt)

FCBS (`CEMT I C F`), shown in Figure 5-18, lists the file control blocks: `FILE`, `READS`, `WRITES`, `REWRT`, `DELS`, `NOTFND` and `LOCKEDBY`, the terminal holding a record lock.

```
INQUIRE CONTROLBLOCKS FCBS                                   TERM=T0054  19:19:47
FILE       READS  WRITES  REWRT  DELS  NOTFND  LOCKEDBY
CUSTOMERS  2      0       0      0     4       -









Page 1 of 1   rows 1-1 of 1
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-18. File control blocks** (Model 2, 24 x 80; capture screens/cemt-inquire-fcbs.txt)

### MONITOR

MONITOR (`CEMT M`) displays the performance of the region process, as shown in Figure 5-19. It is open to every user.

```
MONITOR                                              TERM=T0055  19:19:51
Process                              Activity
Memory (heap)      6.7 MB            Transactions (total)   4
Memory (sys)       27.6 MB             txn/s (interval)     –
Heap objects       11719             EXEC CICS (total)      55
GC last pause      0.03 ms             EXEC CICS/s (intvl)  –
CPU% (interval)    –                 EXEC SQL (total)       2
Goroutines         10                  EXEC SQL/s (intvl)   –
Uptime             10m 1s            Time since last ENTER  (first refresh)
Sessions                             VSAM accesses          8
Active terminals   2                 EXEC CICS WEB          0
Signed-on users    1                 Inter-region (MRO)     0
Active transaction 1                 Caches
Known files        1                 L1 hits/miss   13/8 (–)
MRO                                  L1 entries     4/128
Single Region                        L2 hits/miss   0/4 (–)
                                     L2 usage       0.1/4 MB (2%)
                                     Map hits/miss  181/5 (–)
                                     Map slots      5/128 (4%)
                                     Web req/err    0/0 (0 in flight)
                                     Webserver      –
                                     VSAM hit/miss  1/5 (17%)

ENTER=Refresh  PF3=Back  PF10=Database  PF11=VSAM
```

**Figure 5-19. MONITOR: process metrics** (Model 2, 24 x 80; capture screens/cemt-monitor.txt)

The panel has four groups of figures:

**Process**

Storage in use by the program's heap and obtained from the operating system, the number of heap objects, the last garbage-collection pause, the processor use over the last refresh interval, the number of goroutines (the threads of the region) and the time since the region started.

**Sessions**

The number of connected terminals, signed-on users, running transactions and known files. Under them, `MRO` shows `Single Region` or the peer regions.

**Activity**

Transactions, `EXEC CICS` commands and `EXEC SQL` statements since the region started, with the rate per second over the last interval (– on the first display), the time since the last ENTER, VSAM accesses, `EXEC CICS WEB` requests and inter-region requests.

**Caches**

Hits, misses and occupancy of the level-1 and level-2 program caches and the map cache, web requests and errors, the static web server, and the VSAM record cache hit ratio since the region started.

**`ENTER`** refreshes the panel; the interval rates are computed between two refreshes. **`F10`** displays the database monitor and **`F11`** the VSAM monitor. **`F3`** returns.

***The database and VSAM monitors***

The DB Transaction Monitor (`CEMT M D`, or **`F10`** from MONITOR) and the VSAM File Monitor (`CEMT M V`, or **`F11`**) are live displays that refresh themselves. Each draws four vertical bar charts with their scales; on a color terminal the bars are colored cells, which a text capture cannot show, so the bars are blank in Figure 5-20 and Figure 5-21.

```
Auto Update: ON            DB Transaction Monitor              19:05:00
Refresh:     5s      CPU%                                Uptime:  00:04:30
                     0 . . 20 . . 40 . . 60 . . 80 . . 100
CPU:   0.0%
    TxRead/m      TxWrit/m     TotTx/min       DB ms       Online: 1
      7000          500           8000          3.0

                                                         Reads:    0
      5600          400           6400          2.4       Writes:   0

                                                         Avg ms:   0.00
      4200          300           4800          1.8       dbTx/h:   0

                                                         Read/m:   0.0
      2800          200           3200          1.2       Writ/m:   0.0
                                                         Total/m:  0.0

      1400          100           1600          0.6       Max Seen:
                                                         TxRead/m: 0
      467            33           533           0.2       TxWrit/m: 0
  TxRead/m      TxWrit/m      TotTx/min     DB ms         TotTx/m:  0
       0             0             0           000.00     DB msec:  0.00

F3=Return  F4=History  F9=Slower  F10=Faster
```

**Figure 5-20. The DB Transaction Monitor** (Model 2, 24 x 80; capture screens/cemt-monitor-database.txt)

The elements of the monitors are:

**`Auto Update:`**

An input field. `ON` (the default) refreshes the panel automatically; type `OFF` and press **`ENTER`** to freeze it.

**`Refresh:`**

The refresh interval. It starts at 5 seconds. For an administrator **`F9`** lengthens it and **`F10`** shortens it in steps of 0.5 second between 0.5 and 5 seconds; for other users the steps are 1 second between 3 and 10 seconds.

**`CPU%`**

A horizontal bar of the process's processor use on a scale of 0 to 100, with the value on the `CPU:` line.

**Bar charts**

On the database monitor: database read and write transactions per minute, total transactions per minute, and the average statement time in milliseconds. On the VSAM monitor: keyed reads and updates per minute, the average record-operation latency, and the record-cache hit percentage. The scale is printed beside each chart, the column titles are repeated under the charts, and the current values are on the bottom line.

**Side panel**

The number of terminals online and running totals. On the database monitor: reads, writes, average milliseconds, database transactions per hour, the per-minute rates and the maximum values seen. On the VSAM monitor: reads, writes, average milliseconds, operations per hour, the per-minute rates, and the record cache (hit percentage, storage used against its limit, entries, hits and misses).

```
Auto Update: ON                VSAM File Monitor               19:05:04
Refresh:     5s      CPU%                                Uptime:  00:04:35
                     0 . . 20 . . 40 . . 60 . . 80 . . 100
CPU:   0.0%
    TxRead/m      TxWrit/m     Latency ms    CacheHit%     Online: 1
      7000          2000          5.0           100
                                                           Reads:    0
                                                           Writes:   18
      5600          1600          4.0            80       Avg ms:   8.894
                                                           Ops/h:    355

      4200          1200          3.0            60       Read/m:   0.0
                                                           Writ/m:   0.0

      2800          800           2.0            40       Cache:
                                                           Hit %:    0.0
                                                           Used MB:  0.0/16
      1400          400           1.0            20       Entries:  0
                                                           Hits:     0
      467           133           0.3            7        Misses:   0
  TxRead/m      TxWrit/m      Latency       CacheHit%
       0             0            8.89           0

F3=Return  F4=History  F9=Slower  F10=Faster
```

**Figure 5-21. The VSAM File Monitor** (Model 2, 24 x 80; capture screens/cemt-monitor-vsam.txt)

The keys are **F3** (return; **CLEAR** also returns), **F9** and **F10** (slower and faster), and **F4**, which on a Model 2 or 3 screen opens a history window of the last 16 samples; in the window **F5** changes the measurement shown and **F3** closes it. A Model 4 screen shows the history beneath the charts at all times, and its legend offers **F5**=Cycle Metric in place of **F4**.

## PERFORM

PERFORM (`CEMT P`) groups the functions that rescan the definitions on disk and adjust the caches of the running region. It requires the `admin` group. Figure 5-22 shows its menu.

```
PERFORM                                                  TERM=T0033  19:05:08


Pick an option and press ENTER. PF3 to back out.


  M  MAPCACHE  -- (4/128 entries)
  R  RESCAN  -- trans / maps / programs
  C  PROGRAMCACHE  -- (L1 2/128, L2 4 MB)
  E  METRICS  -- (disabled)

Choice:









ENTER=Select  PF3=Back
```

**Figure 5-22. The PERFORM menu** (Model 2, 24 x 80; capture screens/cemt-perform-menu.txt)

### *PERFORM MAPCACHE*

MAPCACHE (`CEMT P M`), shown in Figure 5-23, displays the cache of parsed maps, whose initial size is set by `map_cache=` in `bricks.cnf`.

```
PERFORM MAPCACHE                                          TERM=T0034  19:05:12
Map LRU (decoded *Map)
**********************************
Capacity       128 entries
Occupancy      4/128
Evictions      0

                Hits        Misses      Hit ratio
Lookups         173         4           97.7%


New capacity:        entries  (1..1028)









ENTER=Refresh  PF5=Apply  PF3=Back
```

**Figure 5-23. PERFORM MAPCACHE** (Model 2, 24 x 80; capture screens/cemt-perform-mapcache.txt)

The panel shows the capacity in entries, the occupancy, the number of evictions, and the hits, misses and hit ratio of the lookups. To change the capacity, type a number from 1 to 1028 in `New capacity` and press **F5**. A smaller capacity evicts the least recently used maps at once; the counters are kept. **ENTER** refreshes and **F3** returns. A full cache with a steady stream of evictions shows that the capacity should be raised.

### *PERFORM RESCAN*

RESCAN (`CEMT P R`) checks the definitions and source files on disk. Its menu, shown in Figure 5-24, summarizes each check.

```
PERFORM RESCAN                                          TERM=T0035  19:05:16


Pick an option and press ENTER. PF3 to back out.


  T  TRANS  -- (60 transactions, 0 missing)
  M  MAP  -- (53 maps, 0 syntax errors)
  P  PROGRAMS  -- (62 programs, 3 orphans)
  O  MRO  -- (single region)

Choice:









ENTER=Select  PF3=Back
```

**Figure 5-24. The PERFORM RESCAN menu** (Model 2, 24 x 80; capture screens/cemt-perform-rescan-menu.txt)

RESCAN TRANS (`CEMT P R T`), shown in Figure 5-25, checks that the program file of every transaction in `transactions.conf` exists. The columns are `TRANSID`, `LANG`, `PROGRAM`, `STATUS` and `PATH`, the resolved file name. `STATUS` is `OK`, `MISSING`, or `ERROR:` followed by the reason. Each **`ENTER`** repeats the check, so the operator can correct a definition or install a file and watch the row change.

```
PERFORM RESCAN TRANS                                    TERM=T0001  19:06:04
TRANSID  LANG   PROGRAM     STATUS  PATH
ABMP     PLI    abmp.pli    OK      runtime/pli/abmp.pli
ABMR     REXX   abmr.rexx   OK      runtime/rexx/abmr.rexx
ABNC     COBOL  abnc.cob    OK      runtime/cobol/abnc.cob
ABNH     COBOL  abnh.cob    OK      runtime/cobol/abnh.cob
ABNP     PLI    abnp.pli    OK      runtime/pli/abnp.pli
APCH     COBOL  apch.cob    OK      runtime/cobol/apch.cob
BALC     COBOL  balc.cob    OK      runtime/cobol/balc.cob
BALR     REXX   balc.rexx   OK      runtime/rexx/balc.rexx
BANK     COBOL  bank.cob    OK      runtime/cobol/bank.cob
BOOK     REXX   book.rexx   OK      runtime/rexx/book.rexx
BRDS     REXX   brds.rexx   OK      runtime/rexx/brds.rexx
CHAT     REXX   chat.rexx   OK      runtime/rexx/chat.rexx
CHEL     COBOL  help.cob    OK      runtime/cobol/help.cob
CONS     REXX   cons.rexx   OK      runtime/rexx/cons.rexx
CSGM     REXX   csgm.rexx   OK      runtime/rexx/csgm.rexx
CSSR     REXX   cssr.rexx   OK      runtime/rexx/cssr.rexx
CUSL     REXX   cusl.rexx   OK      runtime/rexx/cusl.rexx
CUSP     PLI    cusp.pli    OK      runtime/pli/cusp.pli
CUST     REXX   cust.rexx   OK      runtime/rexx/cust.rexx
CUSV     REXX   cusv.rexx   OK      runtime/rexx/cusv.rexx
Page 1 of 3   rows 1-20 of 60
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-25. PERFORM RESCAN TRANS** (Model 2, 24 x 80; capture screens/cemt-perform-rescan-trans.txt)

RESCAN MAP (`CEMT P R M`), shown in Figure 5-26, parses every `.map` file in the map directory. The columns are `FILE`, `NAME` (the map name), `STATUS` (whether the file can be read) and `SYNTAX`: `Pass` when the map is valid, `Pass, n long` when it is valid but *n* of its literals are longer than the `LEN` of their field and are truncated when the map is displayed, the parser's error message when it is not valid, and `-` when the file cannot be read. When a map fails to parse, the region keeps the copy it loaded earlier; this panel is the way to find out which file is in error.

```
PERFORM RESCAN MAP                                          TERM=T0037  19:05:25
FILE         NAME      STATUS  SYNTAX
balc.map     BALC      OK      Pass
bank1.map    BANK1     OK      Pass
bank2.map    BANK2     OK      Pass, 1 long
bank3.map    BANK3     OK      Pass
bank4.map    BANK4     OK      Pass
bank5.map    BANK5     OK      Pass
bookm2.map   BOOKM2    OK      Pass
bookm3.map   BOOKM3    OK      Pass
bookm4.map   BOOKM4    OK      Pass
brds1.map    BRDS1     OK      Pass
brds1l.map   BRDS1L    OK      Pass
brds1m.map   BRDS1M    OK      Pass
brds1w.map   BRDS1W    OK      Pass
chatm2.map   CHATM2    OK      Pass
chatm4.map   CHATM4    OK      Pass
cons1.map    CONS1     OK      Pass, 2 long
cssn.map     CSSN      OK      Pass
cssn_ok.map  CSSNOK    OK      Pass
cssrpw.map   CSSRPW    OK      Pass
cust1.map    CUST1     OK      Pass
Page 1 of 3   rows 1-20 of 53
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-26. PERFORM RESCAN MAP** (Model 2, 24 x 80; capture screens/cemt-perform-rescan-map.txt)

RESCAN PROGRAMS (`CEMT P R P`), shown in Figure 5-27, lists every file in the REXX, COBOL and PL/I program directories, with its language, the transactions that use it, and the result of a syntax check. A program that no transaction uses shows **–** as its transaction; such files are counted as orphans on the RESCAN menu. PL/I include members are not programs and are not listed.

```
PERFORM RESCAN PROGRAMS                                    TERM=T0038  19:05:29
PATH                        LANG   TRANSID    SYNTAX
cobol/abnc.cob              COBOL  ABNC       OK
cobol/abnh.cob              COBOL  ABNH       OK
cobol/apch.cob              COBOL  APCH       OK
cobol/balc.cob              COBOL  BALC       OK
cobol/bank.cob              COBOL  BANK       OK
cobol/compt.cob             COBOL  -          OK
cobol/esdc.cob              COBOL  ESDC       OK
cobol/exam.cob              COBOL  EXAM       OK
cobol/gmpt.cob              COBOL  GMPT       OK
cobol/gusl.cob              COBOL  GUSL       OK
cobol/gust.cob              COBOL  GUST       OK
cobol/gusv.cob              COBOL  GUSV       OK
cobol/hello.cob             COBOL  HELC       OK
cobol/help.cob              COBOL  CHEL       OK
cobol/ordr.cob              COBOL  ORDR       OK
cobol/qagc.cob              COBOL  QAGC       OK
cobol/splc.cob              COBOL  SPLC       OK
cobol/sqld.cob              COBOL  SQLD       OK
cobol/star.cob              COBOL  STAR       OK
cobol/star_original.cob     COBOL  -          OK
Page 1 of 4   rows 1-20 of 62
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 5-27. PERFORM RESCAN PROGRAMS** (Model 2, 24 x 80; capture screens/cemt-perform-rescan-programs.txt)

RESCAN MRO (`CEMT P R O`) rereads `runtime/mro.conf` and reports `RESCAN MRO: n peer region(s) loaded from mro.conf`, or `RESCAN MRO: single region (MRO not configured)`.

### *PERFORM PROGRAMCACHE*

PROGRAMCACHE (`CEMT P C`), shown in Figure 5-28, displays the two-level program cache. Level 1 holds decoded programs, counted in entries; level 2 holds encoded programs in a storage arena, measured in megabytes.

```
PERFORM PROGRAMCACHE                                    TERM=T0039  19:05:33
L1 cache (decoded ASTs)                 L2 cache (gob-encoded arena)
*********************************       *********************************
Capacity       128 entries              Capacity       4 MB
Occupancy      2/128                    Occupancy      0.1/4 MB (2%)

                Hits        Misses      Hit ratio
Since start  L1 21          4           84.0%
Since resize L1 21          4           84.0%
Since start  L2 0           2           0.0%
Since resize L2 0           2           0.0%


New L1 size:         entries  (32..512)
New L2 size:         MB       (1..16384)









ENTER=Refresh  PF5=Apply  PF3=Back
```

**Figure 5-28. PERFORM PROGRAMCACHE** (Model 2, 24 x 80; capture screens/cemt-perform-programcache.txt)

For each level the panel shows the capacity and occupancy, and the hits, misses and hit ratio both since the region started and since the last resize. To resize, type a new level-1 size (32 to 512 entries), a new level-2 size (1 to 16384 megabytes), or both, and press **`F5`**. **`ENTER`** refreshes and **`F3`** returns.

### *PERFORM METRICS*

METRICS (`CEMT P E`), shown in Figure 5-29, controls the `/metrics` endpoint from which a monitoring system reads the region's figures in JSON form.

```
PERFORM METRICS                                          TERM=T0040  19:05:37

Status        DISABLED    Endpoint  (no listener bound)
Uptime        5m07s

Counters since process start

Scrapes       0                       In flight     0
Errors        0                       Bytes served  0 B
Last status   -                       Last bytes    0 B
Last scrape   -                       Last duration -
Rate          0.00 /sec               Avg / scrape  -


Toggle:           Y=Enable  N=Disable
Port:                 current 9100









ENTER=Refresh  PF5=Apply    PF3=Back
```

**Figure 5-29. PERFORM METRICS with the endpoint disabled** (Model 2, 24 x 80; capture screens/cemt-perform-metrics.txt)

The top of the panel shows the status (`ENABLED` or `DISABLED`), the endpoint address and the time since the region started. The counters since the region started are the number of scrapes, errors, scrapes in progress, bytes served, the status, size, time and duration of the last scrape, the rate per second, and the average size of a scrape. To change the endpoint, type `Y` (enable) or `N` (disable) in `Toggle`, optionally a new port number in `Port`, and press **`F5`**; the port is changed first, so that an endpoint being enabled listens on the new port.

While the endpoint is disabled, every request is answered `503 Service Unavailable` at once and the counters continue to advance, so the panel shows whether a monitoring system is still calling. The change lasts until the region stops; it does not alter `start_metrics=` in `bricks.cnf`.

Chapter 6

# CEDA: Resource Definition Online

The resource definition transaction, CEDA, defines, alters and deletes the resources of the region while it runs: users, transactions, databases, files, outbound URIMAPs, peer regions and temporary-storage queues. Each change is written to the configuration file that holds the resource, and the region uses the new definition at once; no restart and no separate install step is needed.

CEDA requires the admin group throughout. A user who is not an administrator receives an access-denied message. The IBM CICS concepts of groups, lists, INSTALL, COPY and CHECK do not exist in bricks: a definition is a line in a text file, and the region rereads the file when it changes.

The following table shows where each CEDA panel keeps its resources.

| Panel | Letter | Resource kept in | bricks.cnf parameter |
|---|---|---|---|
| USER | U | runtime/users.conf | users_file= |
| TRANSACTION | T | runtime/transactions.conf | transactions_file= |
| PROGRAM | P | The program directories (read-only) | rexx_dir=, cobol_dir=, pli_dir= |
| DATABASE | D | runtime/databases.conf and the PostgreSQL server | databases_file= |
| VSAM | V | files.boltdb in the data directory | data_dir= |
| URIMAP | M | runtime/web_routes.conf (URIMAP lines) | web_routes_file= |
| WAPI | W | The running web interface only | None changed |
| WEBSERVER | B | The running static web server only | None changed |
| MRO | O | runtime/mro.conf | mro_file= |
| QUEUES | Q | Temporary-storage queues in the data store | data_dir= |

**Figure 6-1. CEDA panels and the files they maintain**

## The CEDA Menu

Type CEDA at the blank screen and press **ENTER**. The menu is shown in Figure 6-2.

```
CEDA                                                    TERM=T0013  19:22:21


Pick an option and press ENTER. PF3 to back out.


  U  USER  -- (3 users)
  T  TRANSACTION  -- (60 transactions)
  P  PROGRAM  -- (33 REXX, 26 COBOL, 3 PL/I on disk)
  D  DATABASE  -- (2 databases, default=bricks)
  V  VSAM  -- (1 VSAM files)
  M  URIMAP  -- (1 URIMAP)
  W  WAPI  -- (not configured)
  B  WEBSERVER  -- (not configured)
  O  MRO  -- (1 regions)
  Q  QUEUES  -- (2 queues -- type P to purge)

Choice:




ENTER=Select  PF3=Back
```

**Figure 6-2. The CEDA menu** (Model 2, 24 x 80; capture screens/ceda-menu.txt)

Each line shows the letter of a panel, its name and a live summary: the number of users, transactions and files, the number of programs on disk in each language, the default database, and whether the web interface and the static web server are configured. CEDA uses the same layout, keys and path rules as CEMT ("The CEMT Menu" in Chapter 5): a panel may be chosen from the menu or typed on the blank screen, in full or abbreviated, as in `CEDA U`, `CEDA TRANS` or `CEDA VS`. CEDA has no QUIT choice: `Q` selects QUEUES, and **`F3`** leaves the menu.

***Conventions of the CEDA panels***

- A list panel has a one-character selection field (headed `SEL` or `S`) before each row. The operator types an action letter in one or more selection fields and presses **`F5`** to apply them. **`ENTER`** never changes anything: it refreshes the list.
- **`F6`** opens a blank form to define a new resource. The forms are full screens; the title on row 1 names the action (`CEDA DEFINE USER`, `CEDA ALTER TRANSACTION`). On a form **`F5`** applies and **`F3`** cancels; the legend is on the last row of the screen for every terminal model.
- **`F7`** and **`F8`** page through a long list; **`F3`** or **`CLEAR`** leaves the panel.
- The result of an action, or the reason it was refused, is shown on the status line.
- Every change is written atomically (a new file is written and renamed over the old one). If the file was changed by someone else after CEDA read it, for example by an editor, the change is refused with a message asking the operator to press **`ENTER`** and try again; CEDA reread the file first, so that the retry works on the current contents.
- Every change is written to the region log as one audit line, for example `ceda=USER op=DEFINE target=test1 detail="USERS" term=T0001 user=admin`.

### USER

The USER panel (`CEDA U`) lists the users defined in `users.conf`, as shown in Figure 6-3.

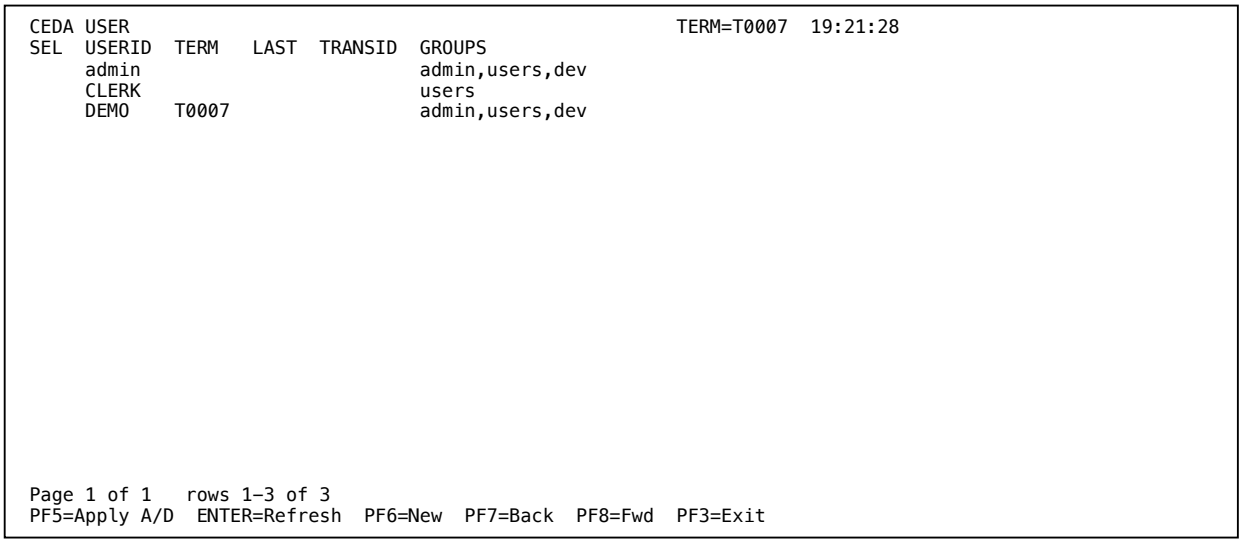



**Figure 6-3. CEDA USER** (Model 2, 24 x 80; capture screens/ceda-user.txt)

The columns are `SEL`; `USERID`; `TERM`, the terminals at which the user is signed on now; `LAST`, the time of the user's last activity; `TRANSID`, the last transaction the user ran; and `GROUPS`. `TERM`, `LAST` and `TRANSID` come from the running region and are blank for a user who has not signed on; they are never written to the file. The actions are `A` (alter) and `D` (delete). Deleting one's own user identifier is refused, so that an administrator cannot lock themselves out.

**`F6`**, or `A` beside a user, opens the form shown in Figure 6-4.

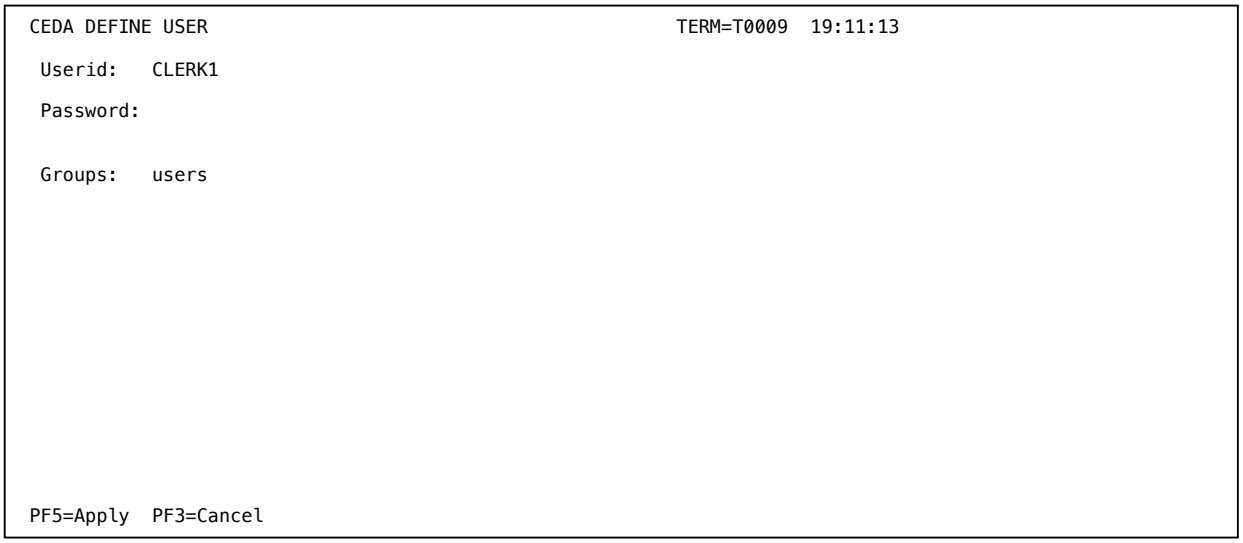



**Figure 6-4. CEDA DEFINE USER** (Model 2, 24 x 80; capture screens/ceda-user-new.txt)

The fields are:

**`Userid`**

The user identifier, up to eight characters.

**`Password`**

The password, up to 32 characters, not displayed. It is required for a new user. On ALTER, a blank password keeps the existing one. The password is stored as a bcrypt hash computed by CEDA, and is verified exactly as typed. A password longer than 72 bytes is refused, because bcrypt would ignore the excess.

**`Groups`**

The groups, separated by commas. Names may be typed in lower case. The groups `admin` and `dev` have the meanings described in "Security Groups" in Chapter 1; any other group is defined by the installation and used in the group lists of `transactions.conf`.

CEDA rewrites `users.conf` sorted by user identifier; comments in the file are not kept. The next sign-on sees the change.

## TRANSACTION

The TRANSACTION panel (`CEDA T`) lists the definitions in `transactions.conf`, as shown in Figure 6-5.

```
CEDA TRANSACTION                                          TERM=T0002  19:10:15
SEL  TRANSID  LANG   PROGRAM     LAST  CACHE  GROUPS
     ABMP     PLI    abmp.pli                PUBLIC
     ABMR     REXX   abmr.rexx               PUBLIC
     ABNC     COBOL  abnc.cob                PUBLIC
     ABNH     COBOL  abnh.cob                PUBLIC
     ABNP     PLI    abnp.pli                PUBLIC
     APCH     COBOL  apch.cob                PUBLIC
     BALC     COBOL  balc.cob                PUBLIC
     BALR     REXX   balc.rexx               PUBLIC
     BANK     COBOL  bank.cob                PUBLIC,USERS,ADMIN
     BOOK     REXX   book.rexx               PUBLIC
     BRDS     REXX   brds.rexx               USERS
     CHAT     REXX   chat.rexx               PUBLIC,USERS,ADMIN
     CHEL     COBOL  help.cob                PUBLIC
     CONS     REXX   cons.rexx               USERS,ADMIN
     CSGM     REXX   csgm.rexx               PUBLIC,USERS,ADMIN
     CSSR     REXX   cssr.rexx               USERS,ADMIN
     CUSL     REXX   cusl.rexx               PUBLIC
     CUSP     PLI    cusp.pli                PUBLIC
     CUST     REXX   cust.rexx               PUBLIC
     CUSV     REXX   cusv.rexx               PUBLIC
Page 1 of 3   rows 1-20 of 60
PF5=Apply A/D  ENTER=Refresh  PF6=New  PF7=Back  PF8=Fwd  PF3=Exit
```

**Figure 6-5. CEDA TRANSACTION** (Model 2, 24 x 80; capture screens/ceda-transaction.txt)

The columns are `SEL`, `TRANSID`, `LANG`, `PROGRAM`, `LAST` (the time the transaction last ran), `CACHE` (whether its program is in the program cache) and `GROUPS`. The actions are `A` and `D`. Figure 6-6 shows the form opened by **`F6`**, filled in for a new transaction `ORDX`.

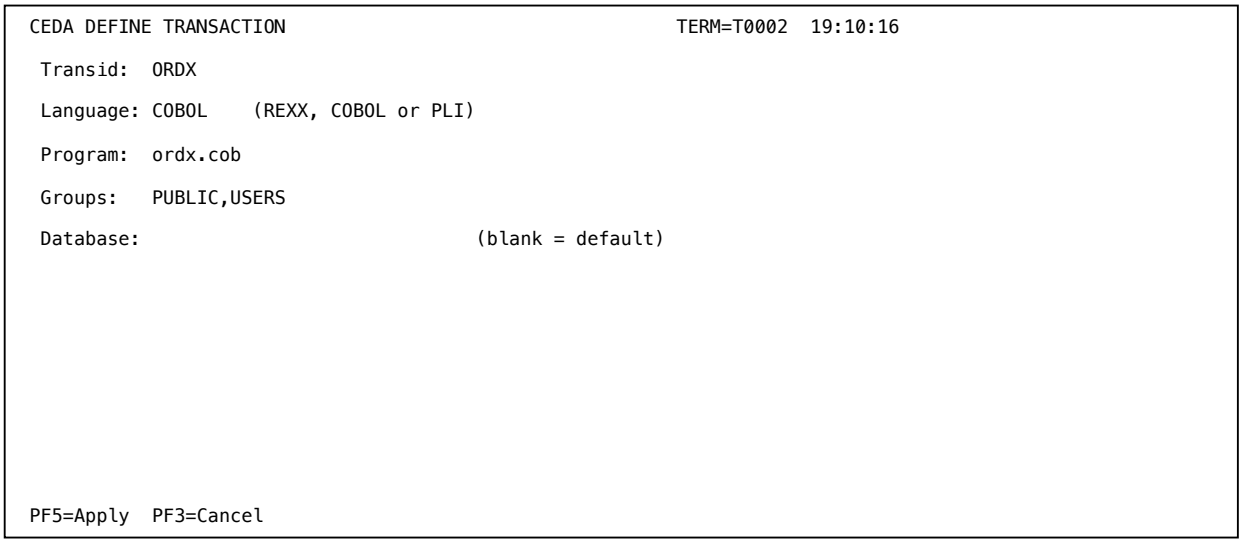



**Figure 6-6. CEDA DEFINE TRANSACTION** (Model 2, 24 x 80; capture screens/ceda-transaction-new.txt)

The fields are:

**`Transid`**

Four characters, translated to upper case. The name of a supplied transaction (`CEMT`, `CEDA`, `CECI`, `IDCA`, `ISPF` ...) is refused: the supplied transaction would always be run instead of the definition.

**`Language`**

`REXX`, `COBOL` or `PLI`. The spellings `PL/I` and `PL1` are refused.

**`Program`**

The program file, relative to the program directory of the language. A name that leaves the directory is refused.

**`Groups`**

The groups permitted to run the transaction, separated by commas.

**`Database`**

The database the transaction's `EXEC SQL` statements use. Blank means the default database.

A colon, a line end, or a blank followed by # in the Program or Database field is refused, because it would change the meaning of the line in the file. Figure 6-7 shows the form opened by A beside ABNC. The counters that CEMT INQUIRE TRANSACTION shows are kept when a definition is altered.

```
CEDA ALTER TRANSACTION                                  TERM=T0002  19:10:17

 Transid:  ABNC

 Language: COBOL    (REXX, COBOL or PLI)

 Program:  abnc.cob

 Groups:   PUBLIC

 Database:                              (blank = default)









PF5=Apply  PF3=Cancel
```

**Figure 6-7. CEDA ALTER TRANSACTION** (Model 2, 24 x 80; capture screens/ceda-transaction-alter.txt)

## PROGRAM

The PROGRAM panel (CEDA P), shown in Figure 6-8, lists the files in the REXX, COBOL and PL/I program directories. It is read-only.

```
CEDA PROGRAM                                          TERM=T0026  19:09:06
LANG   FILE                 SIZE    MODIFIED             TRANSID    SYNTAX
COBOL  abnc.cob             967B    2026-10-22 19:05:52  ABNC       OK
COBOL  abnh.cob             1.4K    2026-10-22 19:05:52  ABNH       OK
COBOL  apch.cob             905B    2026-10-22 19:05:52  APCH       OK
COBOL  balc.cob             1.4K    2026-10-22 19:05:52  BALC       OK
COBOL  bank.cob             40.3K   2026-10-22 19:05:52  BANK       OK
COBOL  compt.cob            1020B   2026-10-22 19:05:52  -          OK
COBOL  esdc.cob             19.2K   2026-10-22 19:05:52  ESDC       OK
COBOL  exam.cob             2.8K    2026-10-22 19:05:52  EXAM       OK
COBOL  gmpt.cob             4.0K    2026-10-22 19:05:52  GMPT       OK
COBOL  gusl.cob             18.4K   2026-10-22 19:05:52  GUSL       OK
COBOL  gust.cob             16.0K   2026-10-22 19:05:52  GUST       OK
COBOL  gusv.cob             885B    2026-10-22 19:05:52  GUSV       OK
COBOL  hello.cob            1.2K    2026-10-22 19:05:52  HELC       OK
COBOL  help.cob             656B    2026-10-22 19:05:52  CHEL       OK
COBOL  ordr.cob             5.5K    2026-10-22 19:05:52  ORDR       OK
COBOL  qagc.cob             5.4K    2026-10-22 19:05:52  QAGC       OK
COBOL  splc.cob             1.6K    2026-10-22 19:05:52  SPLC       OK
COBOL  sqld.cob             6.3K    2026-10-22 19:05:52  SQLD       OK
COBOL  star.cob             38.9K   2026-10-22 19:05:52  STAR       OK
COBOL  star_original.cob    45.4K   2026-10-22 19:05:52  -          OK
Page 1 of 4   rows 1-20 of 62
ENTER=Refresh  PF7=Backward  PF8=Forward  PF3=Back
```

**Figure 6-8. CEDA PROGRAM** (Model 2, 24 x 80; capture screens/ceda-program.txt)

The columns are `LANG`, `FILE`, `SIZE`, `MODIFIED`, `TRANSID` (the transactions that use the file, or `-`) and `SYNTAX`: `OK` when the file parses, or `ERR` and the first error. Only the page on the screen is parsed at each refresh. The panel is used to check a newly installed program before a transaction is defined for it.

### DATABASE

The DATABASE panel (`CEDA D`), shown in Figure 6-9, lists the databases of `databases.conf` and their state on the PostgreSQL server.

```
CEDA DATABASE                                          TERM=T0008  19:11:07

SERVER    localhost:5432
USER      bricks                SSLMODE   disable
S  NAME         DESCRIPTION        TABLES  INDEXES  SIZE       STATE
   bricks       default applicat+  3       3        8.2 MB     ONLINE   (def)
   bank         retail banking d+  5       11       41.8 MB    ONLINE









2 databases   default=bricks
PF5=Apply A=Add C=Create D=Delete X=Drop U=Alter R=Retest L=Tables PF3=Exit
```

**Figure 6-9. CEDA DATABASE** (Model 2, 24 x 80; capture screens/ceda-database.txt)

The heading shows the server, the database user and the SSL mode, from `bricks.cnf`. The columns are `S`; `NAME`; `DESCRIPTION` (shortened with `+`); `TABLES`, `INDEXES` and `SIZE`; and `STATE`, which is `ONLINE`, `OFFLINE` or `NEVER`. The default database is marked `(def)`. For an `OFFLINE` database, the reason reported by the server (`connection refused`, `password authentication failed`, ...) is shown in place of the counts. The actions, applied with **`F5`**, are:

| Action | Function |
|---|---|
| A | Adds a database to `databases.conf` (a form for the name and description). **F6** opens the same form. |
| C | Creates the database on the server (`CREATE DATABASE`) for a row of the file, and tests the connection again. |
| D | Deletes the row from the file. The default database cannot be deleted. |
| X | Drops the database on the server, after the operator types its name on a confirmation form. The region's connections to it are closed first. |
| U | Alters the description. |
| R | Tests the connection again. |
| L | Lists the tables of the database. |

**Figure 6-10. CEDA DATABASE actions**

`A` followed by `C` adds a database; `X` followed by `D` removes one. Figure 6-11 shows the result of `L`: the schema, name, kind, number of rows (`?` when the server has no estimate) and number of indexes of each table.

```
CEDA DATABASE                                          TERM=T0008  19:11:08

Tables in database bricks

SCHEMA          TABLE                     KIND              ROWS    INDEXES
public          bricks_pgtest_t           BASE TABLE        ?       1
public          customers_sql             BASE TABLE        ?       1
public          spdb_demo                 BASE TABLE        0       1









3 tables
ENTER=Refresh  PF3=Back
```

**Figure 6-11. The tables of a database** (Model 2, 24 x 80; capture screens/ceda-database-tables.txt)

### VSAM

The VSAM panel (`CEDA V`), shown in Figure 6-12, lists the files of the file store: the same files that `EXEC CICS WRITE` creates and that IDCAMS maintains (Chapter 10).

```
CEDA VSAM                                              TERM=T0028  19:09:14


S  NAME              RECORDS    KEYMAX RECMAX LAST-MOD       LOCK
   CUSTOMERS         18         8      95     22Oct26 19:08  -









1 VSAM files
PF5=Apply  ENTER=Refresh  P=Purge  PF3=Exit
```

**Figure 6-12. CEDA VSAM** (Model 2, 24 x 80; capture screens/ceda-vsam.txt)

The columns are `S`, `NAME`, `RECORDS`, `KEYMAX` and `RECMAX` (the longest key and record allowed), `LAST-MOD`, and `LOCK`, the terminal holding a `READ UPDATE` lock in the file, or `-`. The only action is `P` (purge). After **`F5`**, a confirmation screen lists each file selected beside an input field; a file is purged only if its name is typed there exactly. **`F3`** cancels the whole purge. A file that is locked is skipped and reported, and never reaches the confirmation screen. Purging removes the file and its catalog entry; a later `WRITE FILE` creates it again, empty.

### URIMAP

The URIMAP panel (`CEDA M`), shown in Figure 6-13, maintains the named outbound endpoints that `EXEC CICS WEB OPEN URIMAP(`*`name`*`)` uses ("URIMAP Definitions" in Chapter 20).

```
CEDA URIMAP                                              TERM=T0010  19:11:17
S  NAME       SCHEME  HOST                          PORT   PATH
   GITHUB     https   api.github.com                443    -









A=Alter D=Delete  PF5=Apply  ENTER=Refresh  PF6=Define  PF3=Back
```

**Figure 6-13. CEDA URIMAP** (Model 2, 24 x 80; capture screens/ceda-urimap.txt)

The columns are `S`, `NAME`, `SCHEME`, `HOST`, `PORT` and `PATH`. The actions are `A`, which opens the form with the endpoint filled in, and `D`, which asks for `Y` to confirm. **`F6`** opens the form for a new URIMAP, shown in Figure 6-14: the name, and the endpoint as one URL, which CEDA splits into scheme, host, port and path.

```
CEDA DEFINE URIMAP

NAME    :  WEATHER

ENDPOINT:  https://api.weather.example.com/v1

Example: https://api.github.com  or  http://internal.svc:8080/v2









PF5=Apply  PF3=Cancel
```

**Figure 6-14. CEDA DEFINE URIMAP** (Model 2, 24 x 80; capture screens/ceda-urimap-define.txt)

### WAPI and WEBSERVER

The WAPI panel (`CEDA W`) controls the web interface that runs transactions for inbound HTTP requests, and the WEBSERVER panel (`CEDA B`) controls the static web server. Each shows the state and counters of its server and has an `Action` field: `E` (enable) or `D` (disable), and for the web server also `R` (restart). WAPI has a `Port` field and WEBSERVER a `Dir` field (the directory served). **`F5`** applies the action and the field together; **`ENTER`** refreshes. These panels change only the running region: `bricks.cnf` is not altered, and a restart returns each server to its configured state. In a region without the server, the menu line reads `(not configured)`.

### MRO

The MRO panel (`CEDA O`), shown in Figure 6-15, maintains the peer regions of `mro.conf`.

```
CEDA MRO                                                TERM=T0006  19:10:48
Peer regions this region can reach (mro.conf).
 S  NAME     HOST                        PORT   STATUS       RTT
    DOR1     dor1.example.com            9997   UNREACHABLE  -









Page 1 of 1   rows 1-1 of 1   Type A to Alter, D to Delete
PF5=Apply A/D  ENTER=Refresh  PF6=New  PF7=Back  PF8=Fwd  PF3=End
```

**Figure 6-15. CEDA MRO** (Model 2, 24 x 80; capture screens/ceda-mro.txt)

The columns are `S`, `NAME`, `HOST`, `PORT`, `STATUS` (as in CEMT INQUIRE MRO, "INQUIRE" in Chapter 5) and `RTT`. The actions are `A` and `D`; **`F6`** defines a new peer. The form has the fields `Peer name`, `Host`, `Port` and `Token`, the shared secret of eight to 24 characters that the two regions exchange. The token is treated like a password: it is masked in the list and not displayed on the form, and a blank token on ALTER keeps the existing one. **`F5`** saves and **`F3`** cancels.

### QUEUES

The QUEUES panel (`CEDA Q`), shown in Figure 6-16, lists the temporary-storage queues with the same counters as CEMT INQUIRE TS, and deletes them.

```
CEDA QUEUES                                              TERM=T0014  19:22:25
S  QUEUE           ITEMS    READS    WRITES   REWRT  LASTACC       STATUS
   AUDITLOG        1        0        2        0      19:22:11.557  ENABLED
   ORDERS          2        2        4        0      19:22:12.162  ENABLED









Type P next to a queue to purge.  PF5=submit  PF3=back
```

**Figure 6-16. CEDA QUEUES** (Model 2, 24 x 80; capture screens/ceda-queues.txt)

To delete a queue, type `P` (or `D`) beside it and press **`F5`**. Only one queue may be marked at a time; marking more is refused with `Type P on only one queue at a time.` CEDA then displays the confirmation window shown in Figure 6-17. Type `Y` and press **`ENTER`** to delete the queue, as `EXEC CICS DELETEQ TS` would; any other reply, or **`F3`**, cancels.

```
CEDA QUEUES                                              TERM=T0015  19:22:29
S  QUEUE          ITEMS    READS    WRITES   REWRT  LASTACC       STATUS
   AUDITLOG       1        0        2        0      19:22:11.557  ENABLED
   ORDERS         2        2        4        0      19:22:12.162  ENABLED


                 +----------------------------------------+
                 | PURGE TS QUEUE                         |
                 | 'AUDITLOG'                             |
                 |                                        |
                 | Type Y to confirm:                     |
                 |                                        |
                 | ENTER=submit  PF3=cancel               |
                 |                                        |
                 +----------------------------------------+
```

**Figure 6-17. Confirming the deletion of a queue** (Model 2, 24 x 80; capture screens/ceda-queues-purge-confirm.txt)

Chapter 7

# CECI: The Command-Level Interpreter

The command-level interpreter, `CECI`, executes a single `EXEC CICS`, `EXEC WEB` or `EXEC SQL` command typed at the terminal and displays the response, the values the command set, and the time it took. It is used to try a command before coding it in a program, to inspect a file or queue, and to repair data by hand.

CECI requires the `dev` group. A user without it who types `CECI` receives the refusal shown in Figure 4-10 in Chapter 4. The same requirement applies to programs: an `EXEC CICS LINK PROGRAM('CECI')` issued for such a user returns `PGMIDERR`, and an `EXEC CICS XCTL PROGRAM('CECI')` raises `NOTAUTH` at the `XCTL`; nothing is painted on the terminal.

## The CECI Screen

Type `CECI` at the blank screen and press **`ENTER`**. CECI displays the screen shown in Figure 7-1.

```
CECI                                                 TERM=T0016  19:12:33
>
>
>
>
>
RESPONSE:  ......           RESP2: ...... LEN: ......   ELAPSED: ......









PF 3 END   5 PROCESS   7 SCROLL UP   8 SCROLL DOWN   PA1 BREAK
```

**Figure 7-1. The CECI screen before a command is entered** (Model 2, 24 x 80; capture screens/ceci-idle.txt)

The screen has four parts:

**Row 1**

The title `CECI`, the terminal identifier and the time.

**Input lines (rows 2 to 6)**

Five input lines, each marked `>`. A command may be typed across as many lines as it needs. A line that ends in `(` or `,` is joined to the next line with no blank between, so that a word or an argument may be split; any other line is joined to the next with one blank. A leading `EXEC CICS`, `EXEC WEB` or `EXEC SQL` may be typed or omitted. What is typed stays on the screen after the command runs, so it can be corrected and run again.

**Response line (row 7)**

`RESPONSE:` followed by the condition name and its `EIBRESP` value in parentheses; `RESP2:`, the `EIBRESP2` value; `LEN:`, the number of bytes of output in the result area; and `ELAPSED:`, the time the command took. Dots are shown before the first command.

**Result area (rows 8 to 23) and legend (row 24)**

The variables the command set, any data it returned, and error details on lines that begin with `!`. The area is cleared each time a command runs.

The keys are:

| Key | Function |
|---|---|
| F5 | Executes the command in the input lines. |
| F7 | Scrolls the result area to the next page of output. |
| F8 | Scrolls the result area back to the previous page; it has no effect on the first page. |
| F3 | Ends CECI. CLEAR has the same effect. |
| PA1 | Backs out any uncommitted recoverable work and ends CECI. |
| ENTER | Redisplays the screen without running the command (the clock is brought up to date). |

**Figure 7-2. CECI function keys**

**Note:**

The legend reads `7 SCROLL UP 8 SCROLL DOWN`: the output moves up under **`F7`**, which shows later lines, and down under **`F8`**. This is the reverse of the IBM CECI convention, in which F7 pages back.

### Executing Commands

Figure 7-3 shows the result of `ASKTIME`.

```
CECI                                                    TERM=T0012  19:11:44
> ASKTIME
>
>
>
>
RESPONSE:  NORMAL(0)         RESP2: 0      LEN: 29       ELAPSED: 0.025msec
EIBDATE=0126295
EIBTIME=171144









PF 3 END   5 PROCESS   7 SCROLL UP   8 SCROLL DOWN   PA1 BREAK
```

**Figure 7-3. ASKTIME executed in CECI** (Model 2, 24 x 80; capture screens/ceci-asktime.txt)

The response is `NORMAL(0)`, and the result area lists the two EIB fields the command set, `EIBDATE` (`0CYYDDD`) and `EIBTIME` (`HHMMSS`). On a color terminal a successful response is white, an exceptional condition red, and a successful command that changed recoverable data yellow (see "Units of Work in CECI").

A command that names a variable in a receiving option creates the variable. Figure 7-4 shows `ASSIGN USERID(UID) APPLID(APPL)`; the result area lists each variable the command set, in the form *`name=value`*.

```
CECI                                                        TERM=T0013  19:12:04
> ASSIGN USERID(UID) APPLID(APPL)
>
>
>
>
RESPONSE:  NORMAL(0)         RESP2: 0      LEN: 21       ELAPSED: 0.040msec
APPL=BRICKS01
UID=DEMO









PF 3 END   5 PROCESS   7 SCROLL UP   8 SCROLL DOWN   PA1 BREAK
```

**Figure 7-4. ASSIGN executed in CECI** (Model 2, 24 x 80; capture screens/ceci-assign-userid.txt)

The variables last for the whole CECI session, so a later command can use them as arguments. They are discarded when CECI ends.

Figure 7-5 shows a keyed read of the `customers` file. The record is returned in the variable `REC`; a value longer than a line continues on the next line of the result area.

```
CECI                                                    TERM=T0013  19:12:05
> READ FILE('customers') INTO(REC) RIDFLD('C1004')
>
>
>
>
RESPONSE:  NORMAL(0)         RESP2: 0      LEN: 99       ELAPSED: 0.039msec
REC=Chen Wei                        |18 Elm Street                   |Riversid
e           |555-0118









PF 3 END   5 PROCESS   7 SCROLL UP   8 SCROLL DOWN   PA1 BREAK
```

**Figure 7-5. READ FILE executed in CECI** (Model 2, 24 x 80; capture screens/ceci-read-file.txt)

Figure 7-6 shows `READQ TS` reading item 2 of the queue `ORDERS` into `ITEM`.

```
CECI                                                    TERM=T0011  19:22:12
> READQ TS QUEUE('ORDERS') INTO(ITEM) ITEM(2)
>
>
>
>
RESPONSE:  NORMAL(0)         RESP2: 0      LEN: 24       ELAPSED: 0.041msec
ITEM=ORDER 1002 GADGET 3









PF 3 END   5 PROCESS   7 SCROLL UP   8 SCROLL DOWN   PA1 BREAK
```

**Figure 7-6. READQ TS executed in CECI** (Model 2, 24 x 80; capture screens/ceci-readq-ts.txt)

***Exceptional conditions***

CECI never abends on a condition: it displays the condition, whatever the language default would be. Figure 7-7 shows a read for a key that does not exist.

```
CECI                                                    TERM=T0013  19:12:06
> READ FILE('customers') INTO(REC) RIDFLD('C9999')
>
>
>
>
RESPONSE:  NOTFND(13)        RESP2: 0      LEN: 0        ELAPSED: 0.054msec









PF 3 END   5 PROCESS   7 SCROLL UP   8 SCROLL DOWN   PA1 BREAK
```

**Figure 7-7. A command that raises NOTFND** (Model 2, 24 x 80; capture screens/ceci-read-notfnd.txt)

A command whose options are incomplete or inconsistent returns `INVREQ`, and the reason is given on a `!` line, as in Figure 7-8, where the `RIDFLD` option of `READ` was omitted.

```
CECI                                                        TERM=T0013  19:12:06
> READ FILE('customers') INTO(REC)
>
>
>
>
RESPONSE:  INVREQ(16)        RESP2: 0      LEN: 33       ELAPSED: 0.048msec
! READ FILE: RIDFLD(...) required









PF 3 END   5 PROCESS   7 SCROLL UP   8 SCROLL DOWN   PA1 BREAK
```

**Figure 7-8. A command with a required option missing** (Model 2, 24 x 80; capture screens/ceci-invreq.txt)

In place of a condition name, the response line may show one of the following words, in red, when the command was not executed or did not finish:

| Response | Meaning |
|---|---|
| SYNTAX | The command could not be parsed. The ! line gives the reason. |
| DENIED | The command is not permitted in CECI (see "Commands Not Permitted"). |
| TIMEOUT | The command ran for more than 7 seconds. See the note below. |
| BADCHAR | An input line contains a character that a 3270 cannot display in code page 037: [ ] { } ~ \, the grave accent, \| or ^. |
| NOSQL | An EXEC SQL statement was entered in a region with no database configured. |

**Figure 7-9. CECI responses other than conditions**

**Note:**

Every command is limited to 7 seconds of elapsed time. When the limit is reached CECI displays `TIMEOUT` and abandons the command. The session cannot then be used safely: every later **`F5`** also returns `TIMEOUT`, and the operator must leave CECI with **`F3`**.

### Units of Work in CECI

CECI runs as one task for as long as the operator stays in it. Each **`F5`** is a unit of work: when the command succeeds, CECI takes a syncpoint and its changes are committed; when it fails, the changes are backed out. A command that changes recoverable data (for example `WRITE`, `REWRITE`, `DELETE`, `WRITEQ TS` or `WRITEQ TD`) and succeeds shows `NORMAL(0)` in yellow, as a reminder that the change has been committed.

Positions are not reset by the syncpoint: the read position of a temporary-storage queue, an open browse, an open transient-data queue, an outbound web session and a document survive from one **`F5`** to the next. Record locks are released, so a `READ UPDATE` and its `REWRITE` cannot be entered on separate **`F5`** presses. To update a record, type both commands in the input lines and run them together, or protect the record with `ENQ ...` and `DEQ`. Everything still open is closed, and every lock released, when CECI ends.

An `ENQ` is always executed as if `NOSUSPEND` had been coded, so that a resource held elsewhere returns `ENQBUSY` at once instead of suspending the session.

### Commands Not Permitted

CECI refuses commands that would end or take over its own task, with the response `DENIED` and a message such as `LINK not permitted in CECI`:

| Commands | Reason |
|---|---|
| RETURN, XCTL, LINK, ABEND | They would end or transfer control from the CECI task. |
| SEND MAP, SEND TEXT, SEND CONTROL, RECEIVE MAP, CONVERSE | They would overwrite or read the CECI screen. RECEIVE INTO is permitted. |
| START, RETRIEVE, CANCEL, DELAY | They need a task that is not CECI. |
| HANDLE CONDITION, HANDLE AID, HANDLE ABEND, IGNORE | A trap set by one command would not apply to the next. |
| SYNCPOINT ROLLBACK | Each command is already its own unit of work. A plain SYNCPOINT is accepted and has no effect. |
| VERIFY PASSWORD | The password would be displayed on the screen. |

**Figure 7-10. Commands that CECI refuses**

Every other command runs as it would in a program, including the WEB and DOCUMENT commands and every EXEC SQL statement.

Chapter 8

# CEDF and CEDG: The Execution Diagnostic Facility

The execution diagnostic facility (EDF) lets a programmer follow a COBOL, REXX or PL/I transaction through its `EXEC CICS` commands. The task stops at program initiation, before and after every command, when it abends and when it ends; at each stop EDF displays the command, its arguments, the response, the EIB and the program source around the statement. Two transactions are supplied:

- `CEDF`: display and modify. The programmer may change argument values, the response, working storage and some EIB fields, skip a command, or abend the task.
- `CEDG`: display only. Nothing that would change the program can be altered.

Both require the `dev` or `admin` group; the same requirement applies to `LINK` and `XCTL` to them. The facility is modeled on IBM CICS EDF. Where IBM shows the offset of the command in the load module, bricks shows the source member and line, since an interpreted program has no load module.

EDF follows a task while it runs. To find out afterwards why a task abended, use the abend analyser CAID, which reads the task's transaction dump ("CAID: The Abend Analyser" in Chapter 16).

### Switching EDF On and Off

EDF mode belongs to the terminal. Type `CEDF` (or `CEDG`) at the blank screen and press **`ENTER`**. The region confirms as shown in Figure 8-1.

```
THIS TERMINAL: EDF MODE ON
MODE: CEDF – DISPLAY AND MODIFY  TERMINAL: T002 USER: DEMO
CLEAR, THEN TYPE A TRANSID.  CEDF ,OFF ENDS EDF.
```

**Figure 8-1. EDF mode switched on with CEDF** (Model 2, 24 x 80; capture screens/cedf-mode-on.txt)

Row 2 shows the mode, the terminal and the user; row 3 tells the operator what to do next. Press **CLEAR**, type the transaction to be tested, and press **ENTER**. Figure 8-2 shows the same confirmation for CEDG.

```
THIS TERMINAL: EDF MODE ON
MODE: CEDG – DISPLAY ONLY        TERMINAL: T003 USER: DEMO
CLEAR, THEN TYPE A TRANSID.  CEDG ,OFF ENDS EDF.
```

**Figure 8-2. EDF mode switched on with CEDG** (Model 2, 24 x 80; capture screens/cedg-mode-on.txt)

The forms of the command are:

| Typed | Effect |
|---|---|
| `CEDF` | EDF on for this terminal, display and modify. `CEDF ON` and `CEDF ,ON` are the same. |
| `CEDG` | EDF on for this terminal, display only. |
| `CEDF ,OFF` | EDF off. `CEDF,OFF`, `CEDF OFF` and the same forms of `CEDG` are accepted. |
| `CEDF` *`tttt`* | Refused with `CEDF: other terminals not supported`. EDF can be used only on the terminal that runs the transaction. |

**Figure 8-3. Forms of the CEDF and CEDG commands**

Figure 8-4 shows the response to `CEDF ,OFF`. EDF mode also ends when the operator presses **`F3`** on any EDF display, replies other than `YES` at task termination, or signs off.

```
THIS TERMINAL: EDF MODE OFF

CLEAR, THEN TYPE A TRANSID.
```

**Figure 8-4. EDF mode switched off** (Model 2, 24 x 80; capture screens/cedf-mode-off.txt)

### What EDF Intercepts

Every task the operator starts on a terminal in EDF mode runs under EDF, including every program it links to, at that program's link level. The task stops at the following points:

| Event | When | Display |
|---|---|---|
| Program initiation | The program has been loaded and its EIB set, before its first statement. | PROGRAM INITIATION |
| Before a command | Each EXEC CICS command, after it is parsed and before it runs. | ABOUT TO EXECUTE COMMAND |
| After a command | Each command that did not end the program, before HANDLE CONDITION and IGNORE CONDITION are applied. | COMMAND EXECUTION COMPLETE |
| Abend | EXEC CICS ABEND, an abend requested with F12, a runaway task (AICA), a program check (ASRA), or an unhandled condition in COBOL or PL/I, before HANDLE ABEND is searched. | AN ABEND HAS OCCURRED |
| Program termination | A linked-to program returns. | PROGRAM TERMINATION |
| Task termination | The top-level program ends. | TASK TERMINATION or ABNORMAL TASK TERMINATION |

**Figure 8-5. EDF interception points**

RETURN and XCTL have no COMMAND EXECUTION COMPLETE display: the termination display that follows names them (ENDED BY: EXEC CICS RETURN NEXT TRANSID: GUST COMMAREA: 120, or ENDED BY: EXEC CICS XCTL NEXT PROGRAM: X). The next task of a pseudo-conversation and the target of an XCTL stay under EDF, and keep its stop conditions and previous displays.

EDF does not intercept tasks started with START to run in the background, tasks without a terminal, inbound web and MRO requests, or the supplied transactions.

While a display waits for the operator, the task is paused, and the time is not counted against its runaway limit (ICVR). The program's own screen is saved at each SEND; F6 displays it, and EDF puts it back on the terminal before a SEND or CONVERSE without ERASE, at task termination and when EDF ends, so the program never writes over an EDF display or reads its input from one.

### The EDF Display

Every EDF display has the same frame. Figure 8-6 shows the program-initiation display of the COBOL transaction `GUST`.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  PROGRAM INITIATION                                          CEDF
EIBTIME = 171511          EIBDATE = 0126295         EIBTRNID= 'GUST'
EIBTASKN= 14              EIBTRMID= 'T002'          EIBCPOSN= 500
EIBCALEN= 0               EIBAID  = X'7D' ENTER     EIBFN   = X'0000'
EIBRCODE= X'000000000000' EIBRESP = 0               EIBRESP2= 0
COMMAREA= (NONE)
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00062  *> Initialise the menu group ONCE outside the loop. SCR.MSG is set
00063  *> by each action and carries through to the next iteration's SEND
00064  *> so the operator sees the prior result. Resetting at loop top
00065  *> would erase the message before it could be painted.
00066       MOVE SPACES TO SCR.
00067  *> Match the REXX cust.rexx INFOLINE layout: "User: USR" then a
00068  *> 28-space gutter then "Term: TRM" so the second column lines up
00069  *> in the same screen position the REXX twin produces.
ENTER:  CONTINUE         SOURCE: gust.cob     LINE: 00060  EIBFN=X'0000'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-6. PROGRAM INITIATION** (Model 2, 24 x 80; capture screens/cedf-initiation.txt)

The parts of the frame are:

**Row 1**

`TRANSACTION:` the transaction identifier; `PROGRAM:` the program; `TASK:` the task number; `APPLID:` the region's application identifier; `DISPLAY: 00` for the current display, or `-1` to `-9` for a previous one.

**Row 2**

`STATUS:` the kind of display; a message area, which also shows `LINK LEVEL` *`n`* `FROM` *`program`* for a linked-to program; and the badge `CEDF` or `CEDG` at the right.

**Body**

The contents of the display, described below, followed by the source pane: the program text around the statement, with line numbers. The current statement is marked with `>` after its line number on every line it occupies (all the lines of a multi-line `EXEC CICS`). COBOL source is shown without columns 1 to 6.

**Row 22**

`ENTER:` what ENTER does on this display (`CONTINUE`, or `CURRENT DISPLAY` on an auxiliary display), then `SOURCE:` the source member (a copybook or `%INCLUDE` member when the statement is in one), `LINE:` the line number, and the `EIBFN` of the command.

**Rows 23 and 24**

The key legend, **`F1`** to **`F13`**. On a CEDG display **`F12`** is not shown.

The body of the program-initiation display is a compact EIB, four rows of three fields: `EIBTIME`, `EIBDATE`, `EIBTRNID`, `EIBTASKN`, `EIBTRMID`, `EIBCPOSN`, `EIBCALEN`, `EIBAID` (with the key name), `EIBFN`, `EIBRCODE`, `EIBRESP` and `EIBRESP2`. Under them, `COMMAREA=` shows the communication area, or `(NONE)`. In CEDF the COMMAREA may be overtyped here.

Models 3 and 4 show more of the source. Figure 8-7 shows the same display on a 43-row screen, where the initiation and termination displays also show the full EIB.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000021 APPLID: BRICKS01 DISPLAY: 00
STATUS:  PROGRAM INITIATION                                          CEDF
EIBTIME  = 171829                      EIBRECV  = X'00'
EIBDATE  = 0126295                     EIBSEND  = X'00'
EIBTRNID = 'GUST'                      EIBATT   = X'00'
EIBTASKN = 21                          EIBEOC   = X'00'
EIBTRMID = 'T004'                      EIBFMH   = X'00'
EIBCPOSN = 500                         EIBCOMPL = X'00'
EIBCALEN = 0                           EIBSIG   = X'00'
EIBAID   = X'7D' ENTER                 EIBCONF  = X'00'
EIBFN    = X'0000'                     EIBERR   = X'00'
EIBRCODE = X'000000000000'             EIBERRCD = X'00000000'
EIBDS    = '        '                  EIBSYNRB = X'00'
EIBREQID = '        '                  EIBNODAT = X'00'
EIBRSRCE = '        '                  EIBRESP  = 0 NORMAL
EIBSYNC  = X'00'                       EIBRESP2 = 0
EIBFREE  = X'00'                       EIBRLDBK = X'00'
COMMAREA= (NONE)
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00062  *> Initialise the menu group ONCE outside the loop. SCR.MSG is set
00063  *> by each action and carries through to the next iteration's SEND
00064  *> so the operator sees the prior result. Resetting at loop top
00065  *> would erase the message before it could be painted.
00066       MOVE SPACES TO SCR.
00067  *> Match the REXX cust.rexx INFOLINE layout: "User: USR" then a
00068  *> 28-space gutter then "Term: TRM" so the second column lines up
00069  *> in the same screen position the REXX twin produces.
00070       STRING 'User: ' DELIMITED BY SIZE
00071              USR DELIMITED BY SIZE
00072              '                            Term: ' DELIMITED BY SIZE
00073              TRM DELIMITED BY SIZE
00074          INTO INFOLINE
00075       END-STRING.
00076
00077       PERFORM MENU-LOOP UNTIL EXIT-FLAG = 'Y'.
ENTER:  CONTINUE         SOURCE: gust.cob     LINE: 00060  EIBFN=X'0000'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-7. PROGRAM INITIATION on a Model 4 screen** (Model 4, 43 x 80; capture screens/cedf-initiation-mod4.txt)

### *Colors*

On a color terminal EDF uses color consistently. The elements are:

| Element | Color |
|---|---|
| Labels (STATUS:, ENTER:, RESPONSE:, REPLY:, LOCATE:, FIND:, ABEND CODE:, ENDED BY:), AT X'..', offsets, the SOURCE segment, +, source line numbers, LINK LEVEL, and the key name (F1 ... F13) in the legend | Turquoise |
| Header values, status text, the CEDF badge | White |
| EXEC CICS verb, ENDED BY verb | Red |
| Options and values, all EIB fields, the ENTER action, and the description of each key in the legend | Green |
| A field that may be overtyped (CEDF, current display) | Green, underscored (the verb: red, underscored) |
| A value the command has just set | White |
| An exceptional response, an abend code, NOT HANDLED – ABEND *code* | Red |
| NOT HANDLED – RC SET, NOT HANDLED – EIBRESP SET, RESP CODED – NO TRAP, NO HANDLE ACTIVE | Green |
| A HANDLE or IGNORE forecast | Pink |
| Source text, the PICTURE column | Blue |
| The current statement | Yellow nnnnn> and yellow reverse video |
| The statement that abended | Red nnnnn> and red reverse video |
| Messages, the CEDG badge | Yellow (errors: red) |

**Figure 8-8. Colours of the EDF displays**

EIB fields that the language does not maintain (REXX variables that bricks never sets, fields missing from the PL/I EIB) are supplied from what the region knows and shown in green like every other EIB field; they are never writable.

### Before and After a Command

Figure 8-9 shows GUST about to execute its first command, `EXEC CICS ASSIGN USERID(USR) TERMID(TRM)`.

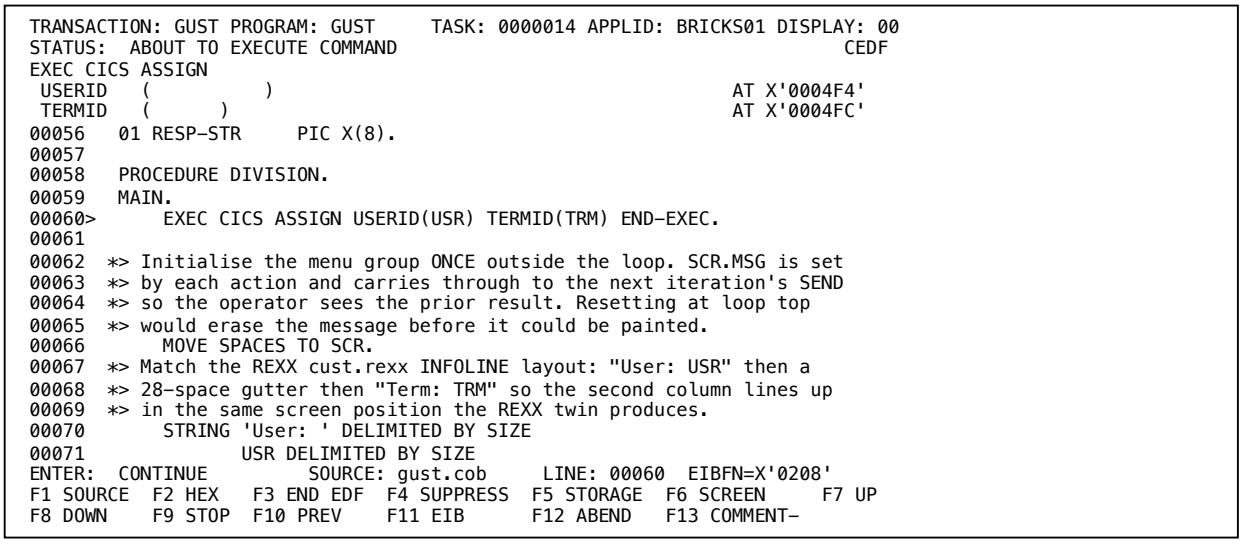



**Figure 8-9. ABOUT TO EXECUTE COMMAND** (Model 2, 24 x 80; capture screens/cedf-before-command.txt)

Row 3 shows `EXEC CICS` and the verb. Each option follows on its own row: the option name, the value in parentheses, and, for an argument in storage, `AT X'`*`offset`*`'`, its offset in working storage (for REXX, `AT` *`name`*, the variable). A value too long for the row ends in `+` instead of `)`, as with the `FROM` area in Figure 8-10. When there are more options than rows, **`F7`** and **`F8`** scroll them.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  ABOUT TO EXECUTE COMMAND                                       CEDF
EXEC CICS SEND
 MAP      ( CUST1 )
 FROM     ( User: DEMO                              Term: +  AT X'000509'
 ERASE
00079       EXEC CICS RETURN END-EXEC.
00080       GOBACK.
00081
00082   MENU-LOOP.
00083>      EXEC CICS SEND MAP('CUST1') FROM(SCR) ERASE END-EXEC.
00084       EXEC CICS RECEIVE MAP('CUST1') INTO(SCR) END-EXEC.
00085
00086  *> PF12 from the menu screen exits the transaction immediately --
00087  *> matching cust.rexx's `IF C2X(EIBAID) = '7C'`.
00088       IF EIBAID = PF12 THEN
00089           MOVE 'Y' TO EXIT-FLAG
00090       END-IF.
00091
00092       IF EXIT-FLAG NOT = 'Y' THEN
00093           MOVE FUNCTION UPPER-CASE(ACTION) TO ACTION
ENTER:  CONTINUE         SOURCE: gust.cob     LINE: 00083  EIBFN=X'1804'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-10. ABOUT TO EXECUTE COMMAND for SEND MAP** (Model 2, 24 x 80; capture screens/cedf-before-send-map.txt)

Figure 8-11 shows the same ASSIGN after it has run. The receivers now hold the values the command set, shown in white. A `RESPONSE:` row has been added.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  COMMAND EXECUTION COMPLETE                                     CEDF
EXEC CICS ASSIGN
 USERID   ( DEMO     )                                    AT X'0004F4'
 TERMID   ( T002 )                                        AT X'0004FC'
RESPONSE: NORMAL       EIBRESP=0 EIBRESP2=0   NO HANDLE ACTIVE
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00062  *> Initialise the menu group ONCE outside the loop. SCR.MSG is set
00063  *> by each action and carries through to the next iteration's SEND
00064  *> so the operator sees the prior result. Resetting at loop top
00065  *> would erase the message before it could be painted.
00066       MOVE SPACES TO SCR.
00067  *> Match the REXX cust.rexx INFOLINE layout: "User: USR" then a
00068  *> 28-space gutter then "Term: TRM" so the second column lines up
00069  *> in the same screen position the REXX twin produces.
00070       STRING 'User: ' DELIMITED BY SIZE
ENTER:  CONTINUE         SOURCE: gust.cob     LINE: 00060  EIBFN=X'0208'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-11. COMMAND EXECUTION COMPLETE** (Model 2, 24 x 80; capture screens/cedf-after-command.txt)

The RESPONSE: row shows the condition name, the values of EIBRESP and EIBRESP2, and a forecast of what the program will do with the response. The forecast is computed from the program's HANDLE CONDITION, HANDLE AID and IGNORE CONDITION settings and the RESP option of the command, before any of them is applied. Its values are:

| Forecast | Meaning |
|---|---|
| NO HANDLE ACTIVE | The response is NORMAL and no trap applies. |
| RESP CODED – NO TRAP | The command has a RESP option, so no condition is raised; the program tests the response itself. |
| HANDLE –> *label* | A HANDLE CONDITION for this condition will pass control to the label. |
| HANDLE ERROR –> *label* | The HANDLE CONDITION ERROR exit will be taken. |
| HANDLE AID *key* –> *label* | A HANDLE AID for the key the operator pressed will be taken. |
| IGNORED: WAS *condition* | An IGNORE CONDITION applies; the program continues. |
| NOT HANDLED – ABEND *code* | COBOL or PL/I: nothing handles the condition, and the task will abend with the IBM code for it (for example AEIM for NOTFND). The AN ABEND HAS OCCURRED display follows. |
| NOT HANDLED – EIBRESP SET | COBOL or PL/I: nothing handles the condition, but it is one whose default action is to continue (for example EOC or ENQBUSY); only EIBRESP is set. |
| NOT HANDLED – RC SET | REXX: a condition never abends a REXX program. RC and EIBRESP are set, and SIGNAL ON ERROR sees the real RC. |

**Figure 8-12. Forecasts on the RESPONSE row**

Figure 8-13 shows the COBOL sample ABNC after a READ for a key that does not exist. The response is NOTFND, EIBRESP is 13, and the forecast is NOT HANDLED – ABEND AEIM.

```
TRANSACTION: ABNC PROGRAM: ABNC     TASK: 0000018 APPLID: BRICKS01 DISPLAY: 00
STATUS:  COMMAND EXECUTION COMPLETE                                      CEDF
EXEC CICS READ
 FILE     ( customers )
 INTO     (                                              +  AT X'000000'
 RIDFLD   ( NOSUCHKEY )
RESPONSE: NOTFND       EIBRESP=13 EIBRESP2=0  NOT HANDLED - ABEND AEIM
00013   01 SCR     PIC X(79).
00014
00015   PROCEDURE DIVISION.
00016   MAIN.
00017>      EXEC CICS READ FILE('customers') INTO(REC)
00018>                RIDFLD('NOSUCHKEY') END-EXEC.
00019  *> Never reached: the READ above abends the task AEIM.
00020       MOVE 'ABNC: READ RETURNED - NOT EXPECTED' TO SCR.
00021       EXEC CICS SEND TEXT FROM(SCR) ERASE END-EXEC.
00022       EXEC CICS RETURN END-EXEC.
00023       GOBACK.



ENTER:  CONTINUE         SOURCE: abnc.cob     LINE: 00017  EIBFN=X'0602'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-13. A condition that will abend the task** (Model 2, 24 x 80; capture screens/cedf-not-handled.txt)

Models 3 and 4 give the command block more rows. Figure 8-14 and Figure 8-15 show the ASSIGN on a Model 4 screen.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000021 APPLID: BRICKS01 DISPLAY: 00
STATUS:  ABOUT TO EXECUTE COMMAND                                       CEDF
EXEC CICS ASSIGN
 USERID   (          )                                      AT X'0004F4'
 TERMID   (      )                                          AT X'0004FC'
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00062  *> Initialise the menu group ONCE outside the loop. SCR.MSG is set
00063  *> by each action and carries through to the next iteration's SEND
00064  *> so the operator sees the prior result. Resetting at loop top
00065  *> would erase the message before it could be painted.
00066       MOVE SPACES TO SCR.
00067  *> Match the REXX cust.rexx INFOLINE layout: "User: USR" then a
00068  *> 28-space gutter then "Term: TRM" so the second column lines up
00069  *> in the same screen position the REXX twin produces.
00070       STRING 'User: ' DELIMITED BY SIZE
00071              USR DELIMITED BY SIZE
00072              '                            Term: ' DELIMITED BY SIZE
00073              TRM DELIMITED BY SIZE
00074           INTO INFOLINE
00075       END-STRING.
00076
00077       PERFORM MENU-LOOP UNTIL EXIT-FLAG = 'Y'.
00078
00079       EXEC CICS RETURN END-EXEC.
00080       GOBACK.
00081
00082   MENU-LOOP.
00083       EXEC CICS SEND MAP('CUST1') FROM(SCR) ERASE END-EXEC.
00084       EXEC CICS RECEIVE MAP('CUST1') INTO(SCR) END-EXEC.
00085
00086  *> PF12 from the menu screen exits the transaction immediately --
00087  *> matching cust.rexx's `IF C2X(EIBAID) = '7C'`.
00088       IF EIBAID = PF12 THEN
00089           MOVE 'Y' TO EXIT-FLAG
00090       END-IF.
ENTER:  CONTINUE         SOURCE: gust.cob     LINE: 00060  EIBFN=X'0208'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-14. ABOUT TO EXECUTE COMMAND on a Model 4 screen** (Model 4, 43 x 80; capture screens/cedf-before-command-mod4.txt)

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000021 APPLID: BRICKS01 DISPLAY: 00
STATUS:  COMMAND EXECUTION COMPLETE                                      CEDF
EXEC CICS ASSIGN
 USERID   ( DEMO     )                                        AT X'0004F4'
 TERMID   ( T004 )                                            AT X'0004FC'
RESPONSE: NORMAL       EIBRESP=0 EIBRESP2=0   NO HANDLE ACTIVE
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00062  *> Initialise the menu group ONCE outside the loop. SCR.MSG is set
00063  *> by each action and carries through to the next iteration's SEND
00064  *> so the operator sees the prior result. Resetting at loop top
00065  *> would erase the message before it could be painted.
00066       MOVE SPACES TO SCR.
00067  *> Match the REXX cust.rexx INFOLINE layout: "User: USR" then a
00068  *> 28-space gutter then "Term: TRM" so the second column lines up
00069  *> in the same screen position the REXX twin produces.
00070       STRING 'User: ' DELIMITED BY SIZE
00071              USR DELIMITED BY SIZE
00072              '                            Term: ' DELIMITED BY SIZE
00073              TRM DELIMITED BY SIZE
00074           INTO INFOLINE
00075       END-STRING.
00076
00077       PERFORM MENU-LOOP UNTIL EXIT-FLAG = 'Y'.
00078
00079       EXEC CICS RETURN END-EXEC.
00080       GOBACK.
00081
00082   MENU-LOOP.
00083       EXEC CICS SEND MAP('CUST1') FROM(SCR) ERASE END-EXEC.
00084       EXEC CICS RECEIVE MAP('CUST1') INTO(SCR) END-EXEC.
00085
00086  *> PF12 from the menu screen exits the transaction immediately --
00087  *> matching cust.rexx's `IF C2X(EIBAID) = '7C'`.
00088       IF EIBAID = PF12 THEN
00089           MOVE 'Y' TO EXIT-FLAG
ENTER:  CONTINUE         SOURCE: gust.cob     LINE: 00060  EIBFN=X'0208'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-15. COMMAND EXECUTION COMPLETE on a Model 4 screen** (Model 4, 43 x 80; capture screens/cedf-after-command-mod4.txt)

### *REXX programs and CEDG*

Figure 8-16 shows the REXX transaction `HELO` under CEDG. The arguments are REXX variables, so each is shown with `AT` *`name`* instead of an offset. The badge reads `CEDG`, no field is underscored, and the legend has no **`F12`**.

```
TRANSACTION: HELO PROGRAM: HELO     TASK: 0000019 APPLID: BRICKS01 DISPLAY: 00
STATUS:  ABOUT TO EXECUTE COMMAND                                      CEDG
EXEC CICS ASSIGN
 USERID   (          )                                    AT USR
 TERMID   (          )                                    AT TRM
 CONNECTE (          )                                    AT CT
 SCREENHT (          )                                    AT SCRH
 SCREENWD (          )                                    AT SCRW
00014  /* sized variant falls back to the unsuffixed mod-2 map.          */
00015
00016  ADDRESS CICS
00017
00018> EXEC CICS ASSIGN USERID(USR) TERMID(TRM) CONNECTED(CT)
00019>                  SCREENHT(SCRH) SCREENWD(SCRW) END-EXEC
00020
00021  /* Classify the terminal model up front so we can reuse the label  */
00022  /* in the info pane below. Largest-first so a mod-3 connection     */
00023  /* isn't misclassified as mod-4.                                   */
00024  IF SCRH >= 43 THEN MDL = 'mod 4'
00025  ELSE IF SCRH >= 32 THEN MDL = 'mod 3'
00026  ELSE IF SCRW >= 132 THEN MDL = 'mod 5'
ENTER:  CONTINUE         SOURCE: hello.rexx   LINE: 00018  EIBFN=X'0208'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB                 F13 COMMENT-
```

**Figure 8-16. ABOUT TO EXECUTE COMMAND under CEDG, REXX program** (Model 2, 24 x 80; capture screens/cedg-before-command.txt)

Figure 8-17 shows the command after it has run. The forecast for an unhandled condition in a REXX program would be `NOT HANDLED — RC SET`.

```
TRANSACTION: HELO PROGRAM: HELO     TASK: 0000019 APPLID: BRICKS01 DISPLAY: 00
STATUS:  COMMAND EXECUTION COMPLETE                                    CEDG
EXEC CICS ASSIGN
 USERID   ( DEMO     )                                    AT USR
 TERMID   ( T0030    )                                    AT TRM
 CONNECTE ( 2026-10-22 19:16:29 )                         AT CT
 SCREENHT ( 24       )                                    AT SCRH
 SCREENWD ( 80       )                                    AT SCRW
RESPONSE: NORMAL       EIBRESP=0 EIBRESP2=0   NO HANDLE ACTIVE
00014  /* sized variant falls back to the unsuffixed mod-2 map.          */
00015
00016  ADDRESS CICS
00017
00018> EXEC CICS ASSIGN USERID(USR) TERMID(TRM) CONNECTED(CT)
00019>                  SCREENHT(SCRH) SCREENWD(SCRW) END-EXEC
00020
00021  /* Classify the terminal model up front so we can reuse the label  */
00022  /* in the info pane below. Largest-first so a mod-3 connection     */
00023  /* isn't misclassified as mod-4.                                   */
00024  IF SCRH >= 43 THEN MDL = 'mod 4'
00025  ELSE IF SCRH >= 32 THEN MDL = 'mod 3'
ENTER:  CONTINUE         SOURCE: hello.rexx   LINE: 00018  EIBFN=X'0208'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB                 F13 COMMENT-
```

**Figure 8-17. COMMAND EXECUTION COMPLETE under CEDG** (Model 2, 24 x 80; capture screens/cedg-after-command.txt)

### Abends and Task Termination

When the task abends, EDF displays AN ABEND HAS OCCURRED before HANDLE ABEND is searched. Figure 8-18 follows the unhandled `NOTFND` of Figure 8-13.

```
TRANSACTION: ABNC PROGRAM: ABNC     TASK: 0000018 APPLID: BRICKS01 DISPLAY: 00
STATUS:  AN ABEND HAS OCCURRED                                        CEDF
ABEND CODE: AEIM  READ FILE('customers') NOTFND  HANDLE ABEND: NONE
EIBTIME = 171620          EIBDATE = 0126295         EIBTRNID= 'ABNC'
EIBTASKN= 18              EIBTRMID= 'T002'          EIBCPOSN= 500
EIBCALEN= 0               EIBAID  = X'7D' ENTER     EIBFN   = X'0602'
EIBRCODE= X'000000000000' EIBRESP = 13              EIBRESP2= 0
00013   01 SCR     PIC X(79).
00014
00015   PROCEDURE DIVISION.
00016   MAIN.
00017>      EXEC CICS READ FILE('customers') INTO(REC)
00018>                 RIDFLD('NOSUCHKEY') END-EXEC.
00019  *> Never reached: the READ above abends the task AEIM.
00020       MOVE 'ABNC: READ RETURNED - NOT EXPECTED' TO SCR.
00021       EXEC CICS SEND TEXT FROM(SCR) ERASE END-EXEC.
00022       EXEC CICS RETURN END-EXEC.
00023       GOBACK.



ENTER:  CONTINUE         SOURCE: abnc.cob     LINE: 00017  EIBFN=X'0602'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-18. AN ABEND HAS OCCURRED** (Model 2, 24 x 80; capture screens/cedf-abend.txt)

Row 3 shows `ABEND CODE:`, the command that failed and its condition, and `HANDLE ABEND:`, which is `NONE` or `-> label`, the exit that will be given control. The compact EIB follows, and the failing statement is marked in red. When the task ends, Figure 8-19 is displayed.

```
TRANSACTION: ABNC PROGRAM: ABNC     TASK: 0000018 APPLID: BRICKS01 DISPLAY: 00
STATUS:  ABNORMAL TASK TERMINATION                                      CEDF
TO CONTINUE EDF SESSION REPLY YES       REPLY: YES
ABEND CODE: AEIM  BACKED OUT: YES  READ FILE('customers') NOTFND
EIBTIME = 171620          EIBDATE = 0126295         EIBTRNID= 'ABNC'
EIBTASKN= 18              EIBTRMID= 'T002'          EIBCPOSN= 500
EIBCALEN= 0               EIBAID  = X'7D' ENTER     EIBFN   = X'0602'
EIBRCODE= X'000000000000' EIBRESP = 13              EIBRESP2= 0
00013   01 SCR     PIC X(79).
00014
00015   PROCEDURE DIVISION.
00016   MAIN.
00017>      EXEC CICS READ FILE('customers') INTO(REC)
00018>                 RIDFLD('NOSUCHKEY') END-EXEC.
00019  *> Never reached: the READ above abends the task AEIM.
00020       MOVE 'ABNC: READ RETURNED - NOT EXPECTED' TO SCR.
00021       EXEC CICS SEND TEXT FROM(SCR) ERASE END-EXEC.
00022       EXEC CICS RETURN END-EXEC.
00023       GOBACK.


ENTER:  CONTINUE         SOURCE: abnc.cob     LINE: 00017  EIBFN=X'0602'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-19. ABNORMAL TASK TERMINATION** (Model 2, 24 x 80; capture screens/cedf-abnormal-termination.txt)

Row 4 repeats the abend code and adds `BACKED OUT: YES`: the task's unit of work has been backed out. Row 3 asks the operator whether EDF is to continue for the next task of this terminal.

A task that ends normally shows TASK TERMINATION, as in Figure 8-20 for the COBOL transaction HELC. Row 4 shows `ENDED BY:` the command that ended the task, `NEXT TRANSID:` the transaction named on `RETURN TRANSID` (or `NONE`), and `COMMAREA:` the length of the communication area passed to it.

```
TRANSACTION: HELC PROGRAM: HELC     TASK: 0000016 APPLID: BRICKS01 DISPLAY: 00
STATUS:  TASK TERMINATION                                            CEDF
TO CONTINUE EDF SESSION REPLY YES       REPLY: YES
ENDED BY: EXEC CICS RETURN  NEXT TRANSID: NONE  COMMAREA: 0
EIBTIME = 171556          EIBDATE = 0126295         EIBTRNID= 'HELC'
EIBTASKN= 16              EIBTRMID= 'T002'          EIBCPOSN= 0
EIBCALEN= 0               EIBAID  = X'7D' ENTER     EIBFN   = X'0E08'
EIBRCODE= X'000000000000' EIBRESP = 0               EIBRESP2= 0
00030       MOVE 'ENTER=Continue'       TO FOOTER.
00031
00032       EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE END-EXEC.
00033
00034>      EXEC CICS RETURN END-EXEC.
00035       GOBACK.









ENTER:  CONTINUE         SOURCE: hello.cob    LINE: 00034  EIBFN=X'0E08'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-20. TASK TERMINATION** (Model 2, 24 x 80; capture screens/cedf-task-termination.txt)

On both termination displays the `REPLY:` field contains `YES`. Press **`ENTER`** to keep EDF on for the next task, or overtype the reply with anything else (for example `NO`) to switch EDF off. The reply works in CEDG too, because it changes EDF, not the program. A linked-to program that returns shows PROGRAM TERMINATION, with no reply.

### Function Keys

The keys are the same on every display. They are named F1 to F13; on a PC keyboard F13 is usually Shift+F1.

| Key | Function |
|---|---|
| ENTER | On the current display: continues the task, unless a field has been overtyped (see "Changing the Program with CEDF") or the cursor is on an argument value, in which case the working storage display opens at that variable. On an auxiliary display (labeled ENTER: CURRENT DISPLAY): applies what was typed there, or returns to the current display. |
| F1 SOURCE | Displays the program source on the whole screen. **F7** and **F8** page; FIND: accepts a line number or a text. |
| F2 HEX | Switches between character and hexadecimal display of values. In hexadecimal, values are shown as the native bytes the interpreter holds. |
| F3 END EDF | Ends EDF for the terminal. The program's screen is put back and the task continues without EDF. |
| F4 SUPPRESS | Continues the task without displays until a stop condition is met (see "Stop Conditions"). |
| F5 STORAGE | Displays working storage. Pressed again (the legend then reads F5 CURRENT), it returns to the current display. |
| F6 SCREEN | Displays the program's last screen. Any key returns. |
| F7 UP, F8 DOWN | Scroll the option list, the working storage or the source. |
| F9 STOP | Displays the stop conditions. |

| Key | Function |
|---|---|
| F10 PREV | Displays the previous display. Up to nine earlier displays are kept (DISPLAY: −1 to −9), including suppressed ones; they are read-only. NO EARLIER DISPLAY is shown at the oldest. ENTER or CLEAR returns to the current display. |
| F11 EIB | Displays the whole EIB. |
| F12 ABEND | CEDF only: abends the task. See "Changing the Program with CEDF". In CEDG it is refused with CEDG: DISPLAY ONLY. |
| F13 COMMENT−, COMMENT+ | Hides or shows comment-only source lines in the source pane and the F1 view. The legend shows COMMENT− when comments are shown (press to hide them) and COMMENT+ when they are hidden (press to show them). The current statement is never hidden, and line numbers keep their gaps. The setting lasts for the EDF session. |
| CLEAR | Returns to the current display. |

**Figure 8-21. EDF function keys**

## The Auxiliary Displays

### *Working storage (F5)*

Figure 8-22 shows the working storage of GUST, positioned at USR by typing its name in LOCATE:.

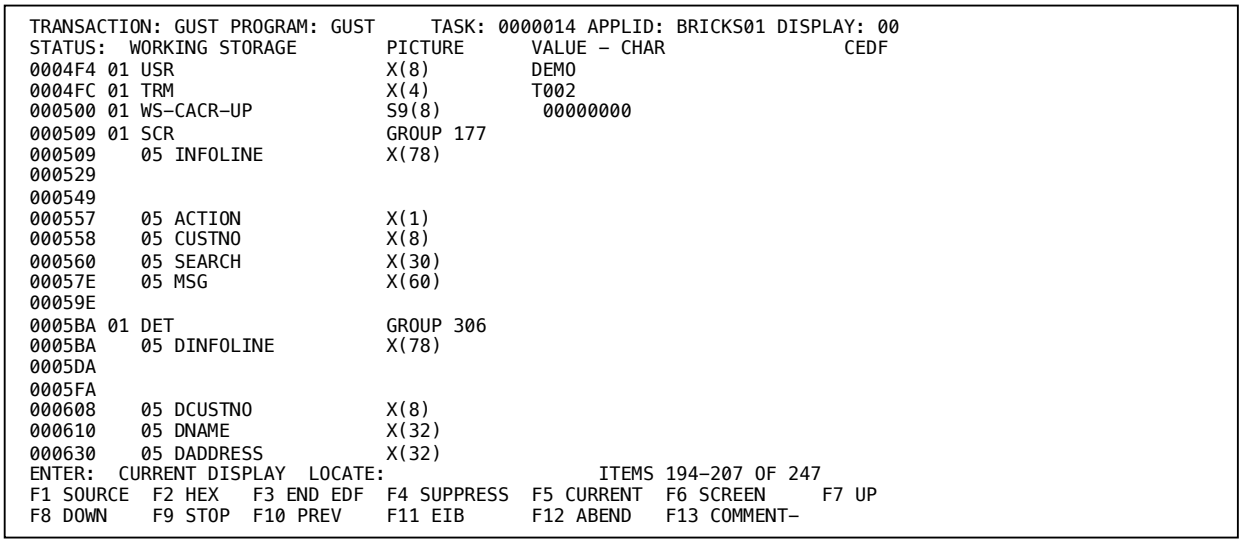



**Figure 8-22. WORKING STORAGE** (Model 2, 24 x 80; capture screens/cedf-storage.txt)

Each row shows the offset, the level and name, the declaration (headed `PICTURE` for COBOL, `TYPE` for REXX and `ATTRIBUTES` for PL/I) and the value: up to 32 characters, or 16 bytes in hexadecimal, per row, on up to four rows, then `+`. Row 22 shows the `LOCATE:` field and which items are on the screen (`ITEMS 194-207 OF 247`). Figure 8-23 shows the same items with **`F2`**: `VALUE - HEX`.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  WORKING STORAGE        PICTURE      VALUE - HEX                CEDF
0004F4 01 USR                   X(8)         44454D4F20202020
0004FC 01 TRM                   X(4)         54303032
000500 01 WS-CACR-UP            S9(8)        203030303030303030
000509 01 SCR                   GROUP 177
000509    05 INFOLINE           X(78)        202020202020202020202020202020202020
000519                                       202020202020202020202020202020202020
000529                                       202020202020202020202020202020202020
000539                                       202020202020202020202020202020202020  +
000557    05 ACTION              X(1)         20
000558    05 CUSTNO              X(8)         2020202020202020
000560    05 SEARCH              X(30)        202020202020202020202020202020202020
000570                                       20202020202020202020202020202020
00057E    05 MSG                 X(60)        202020202020202020202020202020202020
00058E                                       202020202020202020202020202020202020
00059E                                       202020202020202020202020202020202020
0005AE                                       20202020202020202020202020
0005BA 01 DET                   GROUP 306


ENTER:  CURRENT DISPLAY  LOCATE:                    ITEMS 194-203 OF 247
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 CURRENT  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-23. WORKING STORAGE in hexadecimal** (Model 2, 24 x 80; capture screens/cedf-storage-hex.txt)

The display differs by language:

| Language | Storage display |
|---|---|
| COBOL | Declaration order, with the offset in working storage, the level, and the PICTURE (`X(8)`, `S9(7) C3` for COMP-3, `S9(4) C` for COMP), `GROUP` *n* for a group, OCCURS expanded as `NAME(1)` ..., and REDEFINES marked `R`. COMP and COMP-3 values are decoded. |
| REXX | The variables of the current procedure (its own and those exposed to it), in alphabetical order, with the TYPE `CHAR` *n*, `NUM` *n* or `STEM` *n*; stem tails are indented under the stem. There are no offsets. |
| PL/I | AUTOMATIC storage of the current procedure and its parents, then STATIC, with the level, the ATTRIBUTES (`CHAR(8) VAR`, `BIN(15)`, `DEC(7,2)`), structures and arrays expanded, and the offset in the storage block. |

**Figure 8-24. Working storage by language**

In hexadecimal, COBOL and PL/I show the storage bytes (COMP-3 nibbles, big-endian COMP), and REXX shows the bytes of the variable. Figure 8-25 shows the display on a Model 4 screen.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000021 APPLID: BRICKS01 DISPLAY: 00
STATUS:  WORKING STORAGE        PICTURE      VALUE - CHAR                CEDF
0004F4 01 USR                   X(8)         DEMO
0004FC 01 TRM                   X(4)         T004
000500 01 WS-CACR-UP            S9(8)         00000000
000509 01 SCR                   GROUP 177
000509    05 INFOLINE           X(78)
000529
000549
000557    05 ACTION              X(1)
000558    05 CUSTNO              X(8)
000560    05 SEARCH              X(30)
00057E    05 MSG                 X(60)
00059E
0005BA 01 DET                   GROUP 306
0005BA    05 DINFOLINE          X(78)
0005DA
0005FA
000608    05 DCUSTNO             X(8)
000610    05 DNAME               X(32)
000630    05 DADDRESS            X(32)
000650    05 DCITY               X(20)
000664    05 DPHONE              X(16)
000674    05 DMODE               X(60)
000694
0006B0    05 DMSG                X(60)
0006D0
0006EC 01 KEY-IN                X(8)
0006F4 01 SRCH                  X(30)
000712 01 SAVED-SEARCH          X(30)
000730 01 REC                   X(120)
000750
000770
000790
0007A8 01 QNAME                 X(32)
0007C8 01 QADDR                 X(32)
0007E8 01 QCITY                 X(20)
0007FC 01 QPHONE                X(16)
00080C 01 EXIT-FLAG              X(1)         N
00080D 01 VALID-ACTION          X(1)         N
ENTER:  CURRENT DISPLAY  LOCATE:                    ITEMS 194-221 OF 247
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 CURRENT  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-25. WORKING STORAGE on a Model 4 screen** (Model 4, 43 x 80; capture screens/cedf-storage-mod4.txt)

### *The EIB (F11)*

Figure 8-26 shows the whole EIB (DFHEIBLK) in two columns. `EIBFN` is decoded with the command name, `EIBAID` with the key name, and `EIBRESP` with the condition name. bricks sets `EIBFN` to IBM's published function code of every command (`X'0602'` READ, `X'1804'` SEND MAP, `X'0E08'` RETURN ...), and `EIBDS`, `EIBRSRCE` and `EIBREQID` from the command's FILE, QUEUE, PROGRAM, MAP, TRANSID and REQID options.

```
 TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
 STATUS:  EIB DISPLAY                                                  CEDF
 EIBTIME  = 171511                      EIBRECV  = X'00'
 EIBDATE  = 0126295                     EIBSEND  = X'00'
 EIBTRNID = 'GUST'                      EIBATT   = X'00'
 EIBTASKN = 14                          EIBEOC   = X'00'
 EIBTRMID = 'T002'                      EIBFMH   = X'00'
 EIBCPOSN = 500                         EIBCOMPL = X'00'
 EIBCALEN = 0                           EIBSIG   = X'00'
 EIBAID   = X'7D' ENTER                 EIBCONF  = X'00'
 EIBFN    = X'0208' ASSIGN              EIBERR   = X'00'
 EIBRCODE = X'000000000000'             EIBERRCD = X'00000000'
 EIBDS    = '        '                  EIBSYNRB = X'00'
 EIBREQID = '        '                  EIBNODAT = X'00'
 EIBRSRCE = '        '                  EIBRESP  = 0 NORMAL
 EIBSYNC  = X'00'                       EIBRESP2 = 0
 EIBFREE  = X'00'                       EIBRLDBK = X'00'
 COMMAREA = (NONE)
 EIBABCODE= '    '


 ENTER:  CURRENT DISPLAY  SOURCE: gust.cob     LINE: 00060  EIBFN=X'0208'
 F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
 F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-26. EIB DISPLAY** (Model 2, 24 x 80; capture screens/cedf-eib.txt)

### *Hexadecimal values (F2)*

Figure 8-27 shows the after-command display with **`F2`** in effect: `USERID` is shown as `44454D4F20202020`, the bytes of `DEMO` followed by four blanks as the interpreter holds them.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  COMMAND EXECUTION COMPLETE                                      CEDF
EXEC CICS ASSIGN
 USERID   ( 44454D4F20202020 )                                 AT X'0004F4'
 TERMID   ( 54303032 )                                         AT X'0004FC'
RESPONSE: NORMAL       EIBRESP=0 EIBRESP2=0   NO HANDLE ACTIVE
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00062  *> Initialise the menu group ONCE outside the loop. SCR.MSG is set
00063  *> by each action and carries through to the next iteration's SEND
00064  *> so the operator sees the prior result. Resetting at loop top
00065  *> would erase the message before it could be painted.
00066       MOVE SPACES TO SCR.
00067  *> Match the REXX cust.rexx INFOLINE layout: "User: USR" then a
00068  *> 28-space gutter then "Term: TRM" so the second column lines up
00069  *> in the same screen position the REXX twin produces.
00070       STRING 'User: ' DELIMITED BY SIZE
ENTER:  CONTINUE         SOURCE: gust.cob     LINE: 00060  EIBFN=X'0208'
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-27. COMMAND EXECUTION COMPLETE in hexadecimal** (Model 2, 24 x 80; capture screens/cedf-after-command-hex.txt)

### *Source (F1) and comments (F13)*

Figure 8-28 shows the source view. Row 2 shows the member and the range of lines on the screen (`51-69 OF 407`); row 22 holds the `FIND:` field. On a Model 2 screen the view shows 19 lines; on a Model 3, 27; on a Model 4, 38.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  SOURCE VIEW                gust.cob  51-69 OF 407              CEDF
00051   01 QPHONE   PIC X(16).
00052
00053   01 EXIT-FLAG    PIC X(1) VALUE 'N'.
00054   01 VALID-ACTION PIC X(1) VALUE 'N'.
00055   01 CMSG         PIC X(30).
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00062  *> Initialise the menu group ONCE outside the loop. SCR.MSG is set
00063  *> by each action and carries through to the next iteration's SEND
00064  *> so the operator sees the prior result. Resetting at loop top
00065  *> would erase the message before it could be painted.
00066       MOVE SPACES TO SCR.
00067  *> Match the REXX cust.rexx INFOLINE layout: "User: USR" then a
00068  *> 28-space gutter then "Term: TRM" so the second column lines up
00069  *> in the same screen position the REXX twin produces.
ENTER:  CURRENT DISPLAY  FIND:                       LINE NUMBER OR TEXT
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-28. SOURCE VIEW** (Model 2, 24 x 80; capture screens/cedf-source.txt)

Figure 8-29 shows the same view after **F13**. Comment-only lines (COBOL lines with ∗ or / in the indicator column and ∗> lines, REXX and PL/I /∗ ∗/ blocks; REXX comments nest) are hidden, so lines 51 to 76 fit; the line numbers show where lines were removed. The legend now offers F13 COMMENT+.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  SOURCE VIEW                gust.cob  51-76 OF 407              CEDF
00051   01 QPHONE   PIC X(16).
00052
00053   01 EXIT-FLAG    PIC X(1) VALUE 'N'.
00054   01 VALID-ACTION PIC X(1) VALUE 'N'.
00055   01 CMSG         PIC X(30).
00056   01 RESP-STR     PIC X(8).
00057
00058   PROCEDURE DIVISION.
00059   MAIN.
00060>      EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.
00061
00066       MOVE SPACES TO SCR.
00070       STRING 'User: ' DELIMITED BY SIZE
00071              USR DELIMITED BY SIZE
00072              '                            Term: ' DELIMITED BY SIZE
00073              TRM DELIMITED BY SIZE
00074          INTO INFOLINE
00075       END-STRING.
00076
ENTER:  CURRENT DISPLAY  FIND:                       LINE NUMBER OR TEXT
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT+
```

**Figure 8-29. SOURCE VIEW with comment lines hidden** (Model 2, 24 x 80; capture screens/cedf-source-nocomments.txt)

### Stop Conditions

**F9** displays the stop conditions, shown in Figure 8-30. **F4** then runs the task without displays until any one of them is met; the display at which it stops says `STOP CONDITION MET:` and names the condition. Displays that were suppressed are still recorded, and **F10** shows them.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000001 APPLID: BRICKS01 DISPLAY: 00
STATUS:  STOP CONDITIONS                                            CEDF
DISPLAY ON CONDITION:
  COMMAND:                    EXEC CICS
  RESOURCE (FILE/QUEUE/PGM):
  PROGRAM INITIATION:
  SOURCE LINE:
  CICS EXCEPTIONAL CONDITION:
  ANY CICS CONDITION:          NO
  TRANSACTION ABEND:           YES
  NORMAL TASK TERMINATION:     YES
  ABNORMAL TASK TERMINATION:   YES
  WORKING STORAGE CHANGE:

F4 SUPPRESS RUNS WITHOUT DISPLAYS UNTIL ANY CONDITION ABOVE IS MET.
SUPPRESSED DISPLAYS ARE STILL RECORDED FOR F10.




ENTER:  CURRENT DISPLAY
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-30. STOP CONDITIONS** (Model 2, 24 x 80; capture screens/cedf-stop-conditions.txt)

Type values in the fields and press **ENTER**. The stop conditions may be set in CEDG as well as in CEDF, because they change EDF, not the program. The conditions are:

| Condition | Met by |
|---|---|
| COMMAND | ABOUT TO EXECUTE of a command whose verb begins with the text typed after EXEC CICS (READ, SEND MAP). |
| RESOURCE | ABOUT TO EXECUTE of a command that names this file, queue, program, map or transaction. |
| PROGRAM INITIATION | Initiation of this program, including a linked-to one. |
| SOURCE LINE | Any display at this source line. |
| CICS EXCEPTIONAL CONDITION | COMMAND EXECUTION COMPLETE with this condition. |
| ANY CICS CONDITION | COMMAND EXECUTION COMPLETE with any response other than NORMAL. Default NO. |
| TRANSACTION ABEND | Any abend. Default YES. |
| NORMAL TASK TERMINATION, ABNORMAL TASK TERMINATION | The task ends. Default YES. |
| WORKING STORAGE CHANGE | The named item's bytes have changed since suppression began. |

**Figure 8-31. EDF stop conditions**

Under suppression the task is not paused, so a task that loops still abends AICA when it exceeds its runaway limit.

## Changing the Program with CEDF

In CEDF, the fields shown underscored on the current display may be overtyped. Press **ENTER** to apply the changes: EDF shows the display again with the result, and the next **ENTER** continues the task. ENTER continues at once when nothing has changed: the terminal sends back every writable field, but a field whose text is the same as what EDF painted does not count as a change. Errors are shown on the STATUS row, for example VERB CAN ONLY BE CHANGED TO NOP or UNKNOWN CONDITION X. On the after-command display the cursor starts on the RESPONSE field, so that a plain ENTER continues.

| Field | Rule |
|---|---|
| Verb (before) | NOP or NOOP skips the command: it completes NORMAL and the next display says COMMAND WAS SKIPPED (NOP). Any other verb is refused. |
| Argument (before) | Every argument. A literal is replaced in the command; a variable is assigned with the language's own rules (COBOL MOVE, PL/I conversion, REXX assignment). |
| Argument (after) | Receivers only. Inputs such as FILE, FROM, MAP, QUEUE and PROGRAM, and literals, are protected. |
| RESPONSE (after) | A condition name or number. It replaces the command's response before the traps are applied, so the program's HANDLE CONDITION, IGNORE CONDITION, RESP option and EIBRESP tests all see the new value. The forecast is recomputed (RESPONSE CHANGED TO NOTFND). |
| COMMAREA (initiation) | Assigned to the communication area. |
| Working storage | In character mode, assigned with the language's rules; in hexadecimal, the native bytes, left-justified. A value containing unprintable bytes (shown as .) can be changed only in hexadecimal. |
| EIB | EIBCPOSN, EIBRESP, EIBRESP2 and EIBAID. EIBAID accepts X'7D', the bare hexadecimal 7D, or a key name: ENTER, CLEAR, PF3, PA1 and so on. |

**Figure 8-32. Fields that CEDF can change**

With the cursor on an argument value and nothing overtyped, **ENTER** does not continue: it opens the working storage display at the variable named by the argument.

### *Abending the task (F12)*

**F12** is accepted on the current program-initiation, before-command and after-command displays; elsewhere it is refused with ABEND NOT POSSIBLE HERE. The first **F12** asks for the code, as in Figure 8-33: the STATUS row reads ENTER CODE, PRESS F12 AGAIN and an ABEND CODE: field appears on row 22. Type a code of up to four characters and press **F12** again. The task abends with that code (AEDF if none is typed) exactly as if the program had issued EXEC CICS ABEND ABCODE(*code*): HANDLE ABEND is honored. Unlike that command, an abend requested with **F12** takes no transaction dump. **ENTER** instead of the second **F12** cancels the request.

```
TRANSACTION: GUST PROGRAM: GUST     TASK: 0000014 APPLID: BRICKS01 DISPLAY: 00
STATUS:  ABOUT TO EXECUTE COMMAND    ENTER CODE, PRESS F12 AGAIN         CEDF
EXEC CICS SEND
 MAP      ( CUST1 )
 FROM     ( User: DEMO                                Term: +  AT X'000509'
 ERASE
00079       EXEC CICS RETURN END-EXEC.
00080       GOBACK.
00081
00082   MENU-LOOP.
00083>      EXEC CICS SEND MAP('CUST1') FROM(SCR) ERASE END-EXEC.
00084       EXEC CICS RECEIVE MAP('CUST1') INTO(SCR) END-EXEC.
00085
00086  *> PF12 from the menu screen exits the transaction immediately --
00087  *> matching cust.rexx's `IF C2X(EIBAID) = '7C'`.
00088       IF EIBAID = PF12 THEN
00089           MOVE 'Y' TO EXIT-FLAG
00090       END-IF.
00091
00092       IF EXIT-FLAG NOT = 'Y' THEN
00093           MOVE FUNCTION UPPER-CASE(ACTION) TO ACTION
ABEND CODE:
F1 SOURCE  F2 HEX   F3 END EDF  F4 SUPPRESS  F5 STORAGE  F6 SCREEN     F7 UP
F8 DOWN    F9 STOP  F10 PREV    F11 EIB      F12 ABEND   F13 COMMENT-
```

**Figure 8-33. An abend requested with F12** (Model 2, 24 x 80; capture screens/cedf-abend-request.txt)

### *CEDG*

In CEDG no field that would change the program is writable, **F12** is refused with CEDG: DISPLAY ONLY, and any field a terminal sends back is ignored. The stop conditions, the end-of-task reply and the other keys work as in CEDF.

Chapter 9

# ISPF: The Source Editor

The `ISPF` transaction browses and edits the program and map sources of the region from a 3270 terminal: REXX, COBOL and PL/I programs, and map files. Its menu, file list and editor follow the look and the commands of the IBM ISPF editor. A program saved with ISPF is used by the next task that runs it; no other step is needed.

ISPF requires the `dev` group. A user without it who types `ISPF` receives `TRANSID "ISPF": access denied -- requires dev group`, in the frame shown in Figure 4-10 in Chapter 4.

## The ISPF Menu

Type `ISPF` at the blank screen and press **`ENTER`**. ISPF displays the menu shown in Figure 9-1.

```
------------------------ ISPF EDIT UTILITY -----------------------------------
OPTION  ===>

   1   -  Edit REXX programs        (runtime/rexx)
   2   -  Edit COBOL programs       (runtime/cobol)
   3   -  Edit BMS map files        (runtime/map)
   4   -  Edit PL/I programs        (runtime/pli)

Press ENTER after typing the option number, or F3 to exit ISPF.

Files in each area are shown in a browser; ENTER opens for edit,
F6 creates a new file. While a file is open it is locked to other
ISPF users (you will see Locked by USER123 since HH:MM on the
browser status line).

Save validation:
   REXX / COBOL / PL/I -- parse before save, PF12 saves anyway on errors
   MAPS                -- parse before save, no bypass (must be valid)




F3=Exit
```

**Figure 9-1. The ISPF EDIT UTILITY menu** (Model 2, 24 x 80; capture screens/ispf-menu.txt)

Type the number of an area in the `OPTION ===>` field and press **`ENTER`**; **`F3`** ends ISPF. The areas and the directories they edit, which are taken from `bricks.cnf`, are:

| Option | Area | Parameter | Default | Extension |
|---|---|---|---|---|
| 1 | REXX programs | `rexx_dir` | `runtime/rexx` | `.rexx` |
| 2 | COBOL programs | `cobol_dir` | `runtime/cobol` | `.cob` |
| 3 | Map files | `maps_dir` | `runtime/map` | `.map` |
| 4 | PL/I programs | `pli_dir` | `runtime/pli` | `.pli` |

**Figure 9-2. ISPF areas**

The lower part of the menu summarizes two rules described later in this chapter: a file open for editing is locked against other ISPF users, and a file is checked before it is saved. PL/I include members (`.inc`, in `pli_include_dir`) are not listed in area 4, but the save-time check of a PL/I program resolves its `%INCLUDE` statements from that directory, as the region does.

## The File Browser

Choosing an area displays its files in a two-column list. Figure 9-3 shows the COBOL area.

```
------------------------ FILE BROWSER -- COBOL -- Sort: NAME -- Page 1/1 -----
Command ===>                           Sort: N=Name D=Date
S Filename           Size Date  Syntax  S Filename           Size Date  Syntax
  abnc.cob             967 22Oct OK       timc.cob            4938 22Oct OK
  abnh.cob            1461 22Oct OK       todo.cob           89600 22Oct OK
  apch.cob             905 22Oct OK       topx.cob           51254 22Oct OK
  balc.cob            1436 22Oct OK       uppr.cob            3279 22Oct OK
  bank.cob           41303 22Oct OK       whdr.cob            8552 22Oct OK
  compt.cob           1020 22Oct OK       wzen.cob            4721 22Oct OK
  esdc.cob           19664 22Oct OK
  exam.cob            2851 22Oct OK
  gmpt.cob            4140 22Oct OK
  gusl.cob           18833 22Oct OK
  gust.cob           16403 22Oct OK
  gusv.cob             885 22Oct OK
  hello.cob           1213 22Oct OK
  help.cob             656 22Oct OK
  ordr.cob            5629 22Oct OK
  qagc.cob            5506 22Oct OK
  splc.cob            1589 22Oct OK
  sqld.cob            6446 22Oct OK
  star.cob           39792 22Oct OK
  star_original.cob  46456 22Oct OK
F3=Menu  F6=New  F7=Up  F8=Down  any char=Open  D=Delete
```

**Figure 9-3. The FILE BROWSER for the COBOL area** (Model 2, 24 x 80; capture screens/ispf-browser-cobol.txt)

The parts of the browser are:

**Row 1**

The title, the area, the sort order and the page.

**`Command ===>`**

Type `N` or `NAME` to sort by file name, or `D` or `DATE` to sort by the time of the last change, newest first.

**File rows**

Each file has a one-character selection field (`S`), its name, size in bytes, date of the last change and `Syntax` status: `OK` when the file parses, `Err:`*n* when the parser reports an error at line *n*, `Err` when no line is known, and `edit` when another user has the file open. The name of a file another user is editing is shown in red.

**Row 24**

The key legend.

Type any character except `D` beside a file and press **`ENTER`** to edit it. Type `D` to delete it: a confirmation window appears, and **`F9`** confirms; any other key cancels. A file another user is editing can be neither opened nor deleted; the status line shows `Locked by` *`user`* `since` *`hh:mm`*. **`F7`** and **`F8`** page, **`F3`** returns to the menu, and **`F6`** creates a new file.

***Creating a file***

**`F6`** displays the panel shown in Figure 9-4.

```
------------------------ ISPF NEW FILE ---------------------------------------

Area:      COBOL
Directory: runtime/cobol

FILENAME  ===

Enter a filename, then ENTER to create and edit.
Extension .cob will be appended automatically if you omit it.









                                              ENTER=Create   F3=Cancel
```

**Figure 9-4. The ISPF NEW FILE panel** (Model 2, 24 x 80; capture screens/ispf-new-file.txt)

Type the file name after `FILENAME ===` and press **`ENTER`**; **`F3`** cancels. The area extension is added when the name has no period. The name must not be empty, must contain only printable ASCII characters, must not contain `/`, `\` or `..`, must not lie outside the area directory (subdirectories are not supported), and must not name a file that exists. ISPF creates the empty file, locks it, and opens it in the editor. If two users create the same name at the same time, the second sees `Locked by` and returns to the browser.

### The Editor

Figure 9-5 shows the COBOL program `hello.cob` open in the editor.

```
EDIT  cobol/hello.cob                              Columns 00001 00071       1
Command ===>                                            Scroll ===> PAGE
****** ***************************** Top of Data ******************************
''''''       *> HELC -- COBOL twin of the REXX HELO transaction. Greets the
''''''       *> signed-on user using the helo1 map. Demonstrates the bricks
''''''       *> COBOL subset: WORKING-STORAGE group items routed to map fields
''''''       *> EXEC CICS ASSIGN populating WORKING-STORAGE, MOVE, STRING-styl
''''''       *> concatenation via successive MOVE into a group item, and the
''''''       *> mandatory EXEC CICS RETURN to give the task back to bricks.
''''''        IDENTIFICATION DIVISION.
''''''        PROGRAM-ID. HELC.
''''''
''''''        DATA DIVISION.
''''''        WORKING-STORAGE SECTION.
''''''        01 USR     PIC X(8).
''''''        01 TRM     PIC X(4).
''''''
''''''        01 SCR.
''''''           05 INFOLINE PIC X(78).
''''''           05 GREETING PIC X(20).
''''''           05 FOOTER   PIC X(60).
''''''
''''''        PROCEDURE DIVISION.
```

**Figure 9-5. EDIT of a COBOL program** (Model 2, 24 x 80; capture screens/ispf-edit.txt)

The rows of the editor are:

**Row 1**

`EDIT` and the file; `Columns` *`nnnnn nnnnn`*, the columns of the file on the screen; and, in column 79, the number (1 to 9) of the file among the files open. Messages are also shown on this row.

**Row 2**

`Command ===>`, the command line, and `Scroll ===>`, the scroll amount. The amount is `PAGE` when the editor opens.

**Data rows**

Each line of the file has a six-character prefix area, shown as `''''''` (or the line number, with `NUMBER ON`), followed by the text. The text may be overtyped. The prefix is where line commands are typed. The `Top of Data` and `Bottom of Data` rows mark the ends of the file; their `******` prefix also accepts `I`, `A` and `B`.

On a color terminal the prefix is red for an unchanged line, yellow for the line the cursor is on, and turquoise for a line changed by the last interaction. A larger screen shows more lines: Figure 9-6 shows `gust.cob` on a Model 4 screen after one **`F8`**.

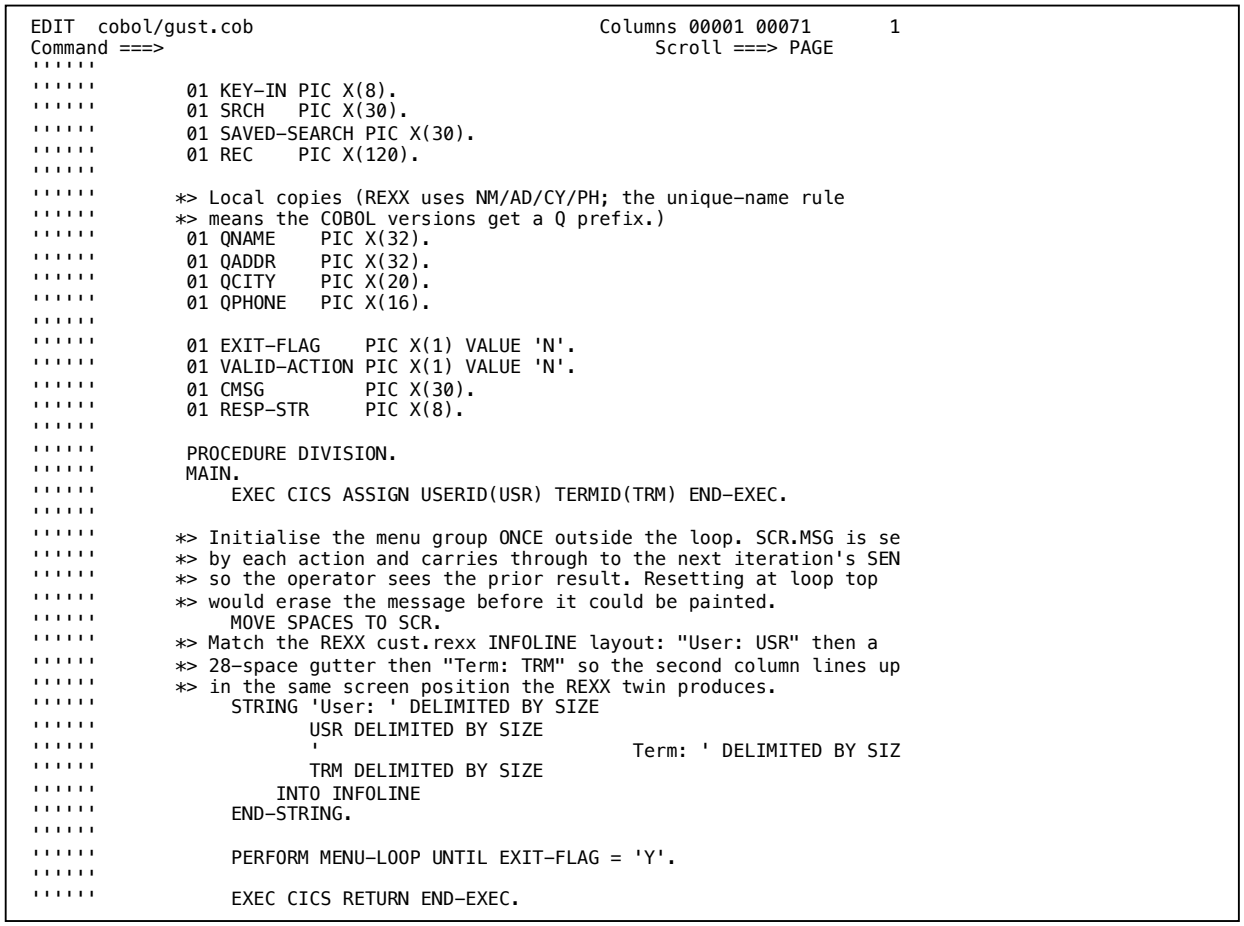



**Figure 9-6. EDIT on a Model 4 screen** (Model 4, 43 x 80; capture screens/ispf-edit-mod4.txt)

### *Function keys*

| Key | Function |
|---|---|
| ENTER | Applies the changes typed on the screen, the line commands, and the command on the command line. |
| F1 | Displays help. Any key removes it. |
| F2 | Displays the file browser without closing the file, so that another file may be opened beside it. |
| F3 | Closes the current file. If it has been changed, a window asks for confirmation, and **F9** discards the changes. |
| F4 | Opens the editor's help text as a further file. |
| F5 | Repeats the last FIND. |
| F7, F8 | Scroll up and down by the scroll amount. |
| F9 | Switches to the next open file. |
| F10, F11 | Scroll left and right by 8 columns. |
| F12 | Saves the current file and closes it. See "Saving". |

**Figure 9-7. ISPF editor function keys**

The scroll amount may be `PAGE` (a screenful), `HALF` (half a screenful), `DATA` (a screenful less one line) or `CSR` (one line). It may be abbreviated or misspelt slightly: `P` and `PGE` mean `PAGE`, and `CURSOR` means `CSR`.

### *Primary commands*

Commands are typed on the command line and run with **`ENTER`**. They may be typed in either case; a quoted argument keeps its case and may contain blanks, as in `FIND 'no records'`.

| Command | Function |
|---|---|
| `FIND text, F text` | Finds the next occurrence of the text, ignoring case, wrapping at the end of the file. Every occurrence is highlighted, and the status line shows `Found n matches`. |
| `F, RFIND` | Repeats the last FIND, as **F5** does. |
| `CHANGE old new [ALL], C` | Changes the first occurrence, or with `ALL` every occurrence, and reports `Changed n`. |
| `LOCATE n, LOC, L` | Displays line *n* at the top. |
| `TOP, T; BOTTOM, BOT` | Displays the start or the end of the file. |
| `SAVE` | Saves the file and keeps it open. |
| `CANCEL, CAN` | Closes the file without saving and without asking. |
| `UNDO` | Restores the file as it was before the last change made with line commands. |
| `NUMBER ON \| OFF, NUM` | Shows or hides line numbers in the prefix area; without an operand, switches. |
| `COLS, COL` | Shows or hides a column ruler. |
| `HI ON \| OFF, HI` | Switches syntax highlighting. It is off when the editor opens. |
| `RESET, RES` | Removes FIND highlighting, the ruler, exclusions and a pending block command. |
| `HELP` | As **F1**. |
| `EDIT` | As **F2**. |

**Figure 9-8. ISPF editor primary commands**

Anything else is answered with `Invalid command`. FIND and CHANGE work on plain text; there are no regular expressions.

### *Line commands*

Line commands are typed over the prefix area of a line and run with **ENTER**. Most accept a count: `D3` deletes three lines.

| Command | Function |
|---|---|
| `D, Dn` | Deletes the line, or *n* lines. |
| `I, In` | Inserts one or *n* blank lines after the line. |
| `R, Rn` | Repeats the line once or *n* times. |
| `C, Cn` | Copies the line, or *n* lines, for a later A, B or O. |
| `M, Mn` | Moves the line, or *n* lines: as C, but the lines are removed. |
| `A, B` | Places the copied or moved lines after or before this line. |
| `O` | Overlays the copied or moved lines on this line and the lines below it. |
| `U, L` | Translates the line to upper or lower case. |
| `) n, ( n` | Shifts the line right or left by *n* columns (default 1). A left shift stops at the first non-blank character. |
| `X, Xn` | Excludes the line, or *n* lines, from the display. They are replaced by a line `- - - n Line(s) excluded`; RESET shows them again. |

**Figure 9-9. ISPF editor line commands**

The block forms `DD`, `CC`, `MM`, `UU`, `LL`, `))`, `((` and `XX` act on a range of lines: type the command on the first line of the range and press **ENTER** (the prefix keeps the pending command), then type the same command on the last line and press **ENTER** again. The two lines may be on different screens. RESET discards a pending first half.

## Saving

Before writing a file, ISPF parses it with the parser of its area. A file that parses is saved at once, and `Saved` is shown. For REXX, COBOL and PL/I, a file with an error is not saved the first time: the parser's message is shown on row 1. A second **F12** or `SAVE` saves the file in spite of the error. A map file must be valid; there is no second-press bypass for maps. If the file system refuses the write, `Save error` is shown.

### Editing Several Files

Up to nine files may be open at once; a tenth is refused with `Maximum files open (9)`. A file is added by opening it from the browser reached with **`F2`**, or by **`F4`**. Each file keeps its own position, changes and undo copy, and **`F9`** moves from one to the next. If a file that is already open is chosen again in the browser, the open copy is used; when the file has changed on disk in the meantime, row 1 shows `External changes – buffer kept`.

**`F3`**, **`F12`** and `CANCEL` close only the current file; the editor then shows the next open file. It returns to the browser only when the last file is closed. Each changed file asks for its own confirmation, so no change is discarded without the operator's knowledge.

### Edit Locks

The file being edited is locked against every other ISPF user. With several files open, only the current file is locked: **`F9`** releases the lock of the file left and takes the lock of the file shown. If another user has meanwhile locked a file, the operator sees `Locked by` *`user`* and the file is dropped from the operator's list. Locks are released when the file is closed, when the operator moves to another file, when the session ends or is lost, and if the region fails during the session.

Chapter 10

# IDCAMS: Access Method Services

The `IDCA` transaction is the bricks counterpart of the IBM access method services program, IDCAMS. It defines, loads, unloads, lists, prints and deletes the keyed (KSDS) files of the file store without an application program. The files are the same files that `EXEC CICS WRITE` creates ("Key-Sequenced Files in bricks" in Chapter 14), that CEMT INQUIRE FILE lists, and that CEDA VSAM purges ("VSAM" in Chapter 6).

The transaction may be typed as `IDCA` or `IDCAMS`. It requires the `admin` group, and every statement it runs is written to the audit log.

## The IDCAMS Screen

Type `IDCA` at the blank screen and press **`ENTER`**. The screen shown in Figure 10-1 is displayed.

```
IDCAMS                                                  TERM=T0037  19:17:57

Access Method Services -- DEFINE / REPRO / DELETE / LISTCAT / PRINT

SYSIN:









SYSPRINT:









Type IDCAMS control statements and press PF12 to run.
PF12=Run  PF3=Exit  PF5=Clear  PF7/8=Scroll
```

**Figure 10-1. The IDCAMS screen** (Model 2, 24 x 80; capture screens/idcams-menu.txt)

The parts of the screen are:

**Row 1**

The title, the terminal identifier and the time. Row 3 lists the functions.

**`SYSIN:`**

Eight input lines of 72 characters for the control statements.

**`SYSPRINT:`**

The listing produced by the last run, with the range of lines on the screen (`rows 1–7 of 16`).

**Row 23**

Messages.

**Row 24**

The key legend.

The keys are:

| Key | Function |
|---|---|
| F12 | Runs the statements in SYSIN. |
| F5 | Clears SYSIN and the listing. |
| F7, F8 | Scroll the listing back and forward. |
| F3 | Ends IDCA. CLEAR and the PA keys have the same effect. |
| ENTER | Redisplays the screen. It does not run the statements. |

**Figure 10-2. IDCAMS function keys**

**Note:**

The statements are run by **`F12`**, not by **`ENTER`**. An operator typing several lines of statements often presses ENTER to reach the next line; making ENTER a run key would start an incomplete deck that may define, delete or load real files.

Figure 10-3 shows two statements typed in SYSIN.

```
IDCAMS                                                  TERM=T0037  19:17:57

Access Method Services -- DEFINE / REPRO / DELETE / LISTCAT / PRINT

SYSIN:
LISTCAT
PRINT INFILE(CUSTOMERS) COUNT(3)









SYSPRINT:









Type IDCAMS control statements and press PF12 to run.
PF12=Run  PF3=Exit  PF5=Clear  PF7/8=Scroll
```

**Figure 10-3. Control statements typed in SYSIN** (Model 2, 24 x 80; capture screens/idcams-sysin.txt)

After **F12**, the listing in Figure 10-4 is displayed. Each statement is echoed, indented, followed by its output and its `IDC0001I` completion message. Row 23 reports the highest condition code of the run, and SYSIN is cleared so that the next **F12** runs only what is typed next.

```
IDCAMS                                                  TERM=T0037  19:17:58

Access Method Services -- DEFINE / REPRO / DELETE / LISTCAT / PRINT

SYSIN:









SYSPRINT: (rows 1-7 of 16)
   LISTCAT
CATALOG LISTING -- 1 ENTRY(S)
  NAME                 RECORDS  KEYMAX  RECMAX  LAST-MOD
  CUSTOMERS                18       8      95  22Oct26 19:08
IDC0001I FUNCTION COMPLETED, CONDITION CODE IS 0
   PRINT INFILE(CUSTOMERS) COUNT(3)
LISTING FROM CLUSTER CUSTOMERS
Run complete. Cond code: 0 (OK). SYSIN cleared - type next command.
PF12=Run  PF3=Exit  PF5=Clear  PF7/8=Scroll
```

**Figure 10-4. The listing after LISTCAT and PRINT** (Model 2, 24 x 80; capture screens/idcams-listcat-print.txt)

Figure 10-5 shows the listing after **F8**: the output of PRINT, with the key and the data of each record.

```
IDCAMS                                                  TERM=T0037  19:17:58

Access Method Services -- DEFINE / REPRO / DELETE / LISTCAT / PRINT

SYSIN:









SYSPRINT: (rows 7-13 of 16)
LISTING FROM CLUSTER CUSTOMERS
KEY OF RECORD - C1001
  Alice Smith                    |1 Main St                      |Springfiel
KEY OF RECORD - C1002
  Robert Jones                   |42 Oak Avenue                  |Shelbyvill
KEY OF RECORD - C1003
  Maria Garcia                   |7 Harbor Road                  |Portsmouth
Run complete. Cond code: 0 (OK). SYSIN cleared - type next command.
PF12=Run  PF3=Exit  PF5=Clear  PF7/8=Scroll
```

**Figure 10-5. The PRINT output, scrolled forward** (Model 2, 24 x 80; capture screens/idcams-print-scrolled.txt)

### Control Statements

Keywords may be typed in either case; values in parentheses (file names, keys) are used as typed. A line that ends in − continues on the next line. A line that begins with ∗, and text between /∗ and ∗/, are comments.

```
DEFINE CLUSTER (NAME(name) [RECORDSIZE(avg max)] [KEYS(len off)]
                [INDEXED] [REPLACE])
REPRO INFILE(seqfile) OUTFILE(cluster) [KEYS(len off)] [SKIP(n)]
      [COUNT(n)]
REPRO INFILE(cluster) OUTFILE(seqfile) [COUNT(n)]
DELETE name [CLUSTER]
LISTCAT [LEVEL(prefix)] [NAMES | ALL]
PRINT INFILE(cluster) [CHARACTER | HEX | DUMP] [COUNT(n)]
      [FROMKEY(key)] [TOKEY(key)]
```

**DEFINE CLUSTER**

Defines a file. KEYS gives the key length and its offset, length first, as in IBM IDCAMS; the key length becomes the file's KEYMAX. RECORDSIZE gives the average and maximum record length; the maximum becomes RECMAX. Defining a file that exists fails with condition code 8 unless REPLACE is coded. The response is IDC0508I CLUSTER *name* DEFINED (KEYMAX=*k*, RECMAX=*r*).

**REPRO**

Copies records. A name containing a period is a sequential file; a name without one is a cluster, so the direction is known from the names. Loading reads one record per line of the sequential file; KEYS locates the key in the line (by default the cluster's KEYMAX at offset 0), SKIP skips the first *n* lines, and COUNT limits the number of records. Unloading writes one line per record. The responses include IDC0005I NUMBER OF RECORDS PROCESSED WAS *n*, IDC0006I RECORDS SKIPPED VIA SKIP() WAS *n* and IDC0007I DUPLICATE RECORDS BYPASSED WAS *n*; duplicate keys give condition code 4.

**DELETE**

Deletes the file and its catalog entry: IDC0550I ENTRY *name* DELETED.

**LISTCAT**

Lists the catalog: name, record count, KEYMAX, RECMAX and the time of the last change. LEVEL limits the list to names that begin with the prefix; NAMES lists the names only. An empty result gives IDC0061I NO ENTRIES FOUND and condition code 4.

**PRINT**

Lists records in key order, each preceded by KEY OF RECORD – *key*, in character form (the default) or, with HEX or DUMP, in hexadecimal. FROMKEY and TOKEY limit the key range, and COUNT the number of records.

### *Sequential files*

The sequential files named on REPRO are relative to the runtime directory (runtime_dir=, by default runtime/), and may be in subdirectories:

```
REPRO INFILE(tmp/orders.in)          OUTFILE(ORDERS)    KEYS(7 0)
REPRO INFILE(loads/jan/customers.in) OUTFILE(CUSTOMERS) KEYS(7 0)
REPRO INFILE(ORDERS)                 OUTFILE(dumps/orders.dat)
```

A directory is not created: if `runtime/dumps/` does not exist, the last statement fails. A name may contain only letters, digits, `_`, `-` and `.` in each part; an absolute name, a name containing `..` or a backslash, a part beginning with a period, and a symbolic link that leads outside the runtime directory are all refused with `IDC3009I` and condition code 8, before any file is opened.

### Condition Codes

Each statement ends with a condition code on the IBM scale, and the run ends with `IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS` *n* (or, when SYSIN was empty, with `IDC0002I (no control statements submitted)` before it):

| Code | Meaning |
|---|---|
| 0 | The statement completed. |
| 4 | Warning: for example, duplicate records were bypassed on REPRO, or LISTCAT found no entries. |
| 8 | Error: the file does not exist, is locked by another terminal, or already exists on a DEFINE without REPLACE; or a file name was refused. |
| 12 | Severe error: the statement could not be parsed, or an input or output operation failed. |

**Figure 10-6. IDCAMS condition codes**

`DELETE` and a loading `REPRO` check whether a task at another terminal holds a `READ UPDATE` lock in the file. If one does, the statement is refused with `IDC3009I ** name LOCKED BY term=Tnnnn, NOT DELETED` (or `REPRO NOT PERFORMED`) and condition code 8. A lock held by the operator's own terminal does not prevent the statement.

Every statement writes one line to the audit log, for example:

```
idcams=DEFINE target=CUSTOMERS term=T0001 user=admin status=OK
idcams=DELETE target=CUSTOMERS term=T0001 user=admin status=OK
```

### Example

The following deck defines the customer file, loads it from a sequential file, lists it and prints its first five records:

```
DEFINE CLUSTER (NAME(CUSTOMERS) RECORDSIZE(80 80) KEYS(7 0))
REPRO INFILE(tmp/customers.in) OUTFILE(CUSTOMERS) KEYS(7 0)
LISTCAT LEVEL(CUST)
PRINT INFILE(CUSTOMERS) CHARACTER COUNT(5)
```

### The idcams Command

The same statements can be run from the operating system shell with the `idcams` command, while the region is stopped, in the manner of a batch IDCAMS job. It opens the file store directly, so it refuses to run (exit code 16, with a message suggesting IDCA) while a region holds the store. Its exit code is the highest condition code.

```
idcams [-c bricks.cnf] [-d datadir] [-r runtimedir] -sysin deck
idcams [-c bricks.cnf] -e "statement"
idcams                          (statements read from standard input)
```

`-c` names the configuration file (by default `./bricks.cnf`, then `../bricks.cnf`); `-d` and `-r` override its data and runtime directories; `-logdir` names the audit-log directory; and `-q` suppresses the echo of the statements.

Part 3

# EXEC CICS Command Reference

This part describes each `EXEC CICS` command that bricks implements, grouped by function. For each command it gives the function, the syntax, the options, the conditions that can be raised, and examples. It then describes the Execute Interface Block, the response codes and conditions, and the commands that bricks does not implement.


**Chapter 11. Terminal Control Commands** ..... **11-1**

**Chapter 12. Program Control Commands** ..... **12-1**

**Chapter 13. System Services Commands** ..... **13-1**

**Chapter 14. File Control Commands** ..... **14-1**

**Chapter 15. Temporary Storage and Transient Data Commands** ..... **15-1**

**Chapter 16. Recovery and Condition Handling** ..... **16-1**

**Chapter 17. The Execute Interface Block** ..... **17-1**

**Chapter 18. Response Codes and Conditions** ..... **18-1**

**Chapter 19. Commands Not Implemented** ..... **19-1**

Chapter 11

# Terminal Control Commands

This chapter describes the commands that write to and read from the 3270 terminal of a task: `SEND MAP`, `RECEIVE MAP`, `SEND TEXT`, `SEND CONTROL`, `RECEIVE` and `CONVERSE`. Maps are defined in the map language of Chapter 3. `HANDLE AID`, which routes control on the key the operator pressed, is described in "HANDLE AID" in Chapter 16.

## How bricks Uses the Terminal

In IBM CICS a `SEND MAP` writes the screen and returns at once; the operator's reply is read later, by a `RECEIVE MAP` in the same task or in the next task of a pseudo-conversation. bricks differs in one respect that affects every program: **a `SEND MAP` or `SEND TEXT` writes the screen and then waits for the operator to press an attention key** (ENTER, a PF key, a PA key or CLEAR). The key and the contents of the screen are saved, `EIBAID` and `EIBCPOSN` are set, and a later `RECEIVE MAP` takes the field values from the saved reply without reading the terminal again.

The usual CICS sequences work unchanged:

- `SEND MAP` followed by `RECEIVE MAP` in the same task (a conversational program) reads the reply to that screen;
- `SEND MAP` followed by `RETURN TRANSID(t)` starts `t` at once with the key the operator pressed in `EIBAID`, and `t`'s `RECEIVE MAP` reads the fields. To the operator this is exactly what IBM CICS does ("RETURN" in Chapter 12).

Two forms do not wait. `SEND MAP ... DATAONLY` writes the fields and returns at once; it is used to refresh part of a screen, for example a clock, while another task on the same terminal waits for the operator. `SEND CONTROL` never waits.

### *The keyboard lock and FREEKB*

When the operator presses an attention key, the 3270 locks its keyboard (the operator information area shows `X SYSTEM`) until the program writes to the terminal with the keyboard-restore bit set. As in IBM CICS, a write sets that bit only when it is asked to:

- A write that is followed by a wait for the operator always unlocks the keyboard: `SEND MAP` without `DATAONLY`, `SEND TEXT` and `CONVERSE`. Otherwise the operator could not answer.
- A write that does not wait (`SEND MAP ... DATAONLY`, `SEND CONTROL`, and the cursor-only write of either with `CURSOR`) leaves the keyboard locked, unless `FREEKB` is coded on the command or, for `SEND MAP`, the map was defined with `FREEKB` ("Statements" in Chapter 3).
- A write by a task started in the background always unlocks the keyboard, because the operator is in the middle of answering the terminal's own task.

A program can therefore paint a progress message with `SEND MAP ... DATAONLY` before a long computation, and the operator cannot type into the screen until the result is sent. The PL/I sample NQUE (`runtime/pli/nqueen.pli`) paints its `Solving ..., please wait.` message this way before it searches. To let the operator type while the task continues, code `FREEKB` on the `DATAONLY` write, or issue `SEND CONTROL FREEKB`, which sends nothing but the unlock.

**Differences from IBM CICS:**

A task that ends after a write that left the keyboard locked leaves it locked in IBM CICS until the operator presses RESET. bricks unlocks it when it next waits for the operator: at the blank screen after the task, or when the transaction named by `RETURN TRANSID` is waited for. And because `SEND MAP` without `DATAONLY` waits for the operator in bricks, it unlocks the keyboard even without `FREEKB`, where in IBM CICS it would not.

### *The PA1 key*

PA1 is the operator's escape from any program. When PA1 answers a `SEND MAP` or `SEND TEXT`, the task ends at once: its unit of work is backed out, no next transaction is started, and the terminal returns to the blank screen. The program does not see the key. A program therefore cannot use PA1 for its own purposes.

### *A lost terminal*

If the connection fails while a terminal command writes or waits, the task ends abnormally with abend code `ATNI`. This is not a condition; `RESP` and `NOHANDLE` do not absorb it. A `HANDLE ABEND` exit may receive the first `ATNI`, but any terminal command issued after it ends the task at once with a final `ATNI`, without driving an exit again.

***Tasks started in the background***

A task started by START while the terminal's own task is waiting for the operator runs in the background ("START" in Chapter 12). The operator's keys belong to the waiting task, so the background task may not wait for input: a SEND MAP without DATAONLY, or a CONVERSE, fails with INVREQ, which ends the task. SEND TEXT in a background task writes the screen and returns without waiting. SEND MAP DATAONLY and SEND CONTROL work as usual.

## SEND MAP

***Function***

Writes a map to the terminal and, unless DATAONLY is coded, waits for the operator's reply.

***Syntax***

```
EXEC CICS SEND MAP(name)
     [MAPSET(name)]
     [FROM(area)]
     [MAPONLY]
     [ERASE | ERASEAUP | DATAONLY]
     [CURSOR(data-value)]
     [TIMEOUT(milliseconds)]
     [FREEKB] [ALARM] [FRSET]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| MAP(name) | Required. The name of the map, in any case. |
| MAPSET(name) | The mapset that contains the map. Needed only when the map name exists in more than one mapset (see "Selecting a Map by Mapset" in Chapter 3). |
| FROM(area) | The data for the named fields of the map. In REXX, a stem: the field NAME is filled from stem.NAME; the trailing period may be omitted (FROM(SCR) and FROM(SCR.) are the same). In COBOL and PL/I, a structure whose items have the names of the fields. A literal fills every named field with the literal. Without FROM the map is painted with its own text and DEFAULT values. |
| MAPONLY | Paint only the map's own text and DEFAULT values; any FROM data is ignored. |
| ERASE | Clear the screen and paint every field of the map. Use it for the first paint of a screen and for a full refresh. |
| ERASEAUP | Treated as ERASE. |
| DATAONLY | Paint without clearing, as without ERASE, and return at once without waiting for the operator. |

| Option | Description |
|---|---|
| CURSOR(n) | Place the cursor at buffer position *n*, counted from zero: *n* = (row - 1) x columns + (column - 1), with rows and columns numbered from 1. CURSOR(0) is row 1, column 1; on an 80-column screen CURSOR(80) is row 2, column 1. Overrides the map's CURSOR statement. Works with or without ERASE and with DATAONLY. The valid range is 0 through rows x columns - 1 (0-1919 on a Model 2); see Conditions. |
| TIMEOUT(ms) | bricks extension. Wait at most *ms* milliseconds for the operator. If no key is pressed in that time the command ends normally with EIBAID set to X'00', a value no key produces. Ignored with DATAONLY. |
| FREEKB | Unlock the keyboard. It matters only with DATAONLY: a SEND MAP that waits for the operator always unlocks it. A map defined with FREEKB has the same effect. See "The keyboard lock and FREEKB". |
| FRSET | Accepted. Every paint resets the modified-data tags already. |
| ALARM | Accepted; the alarm is not sounded. |

**Figure 11-1. SEND MAP options**

### *Which fields are sent*

With ERASE, the screen is cleared and every field of the map is sent: named fields with the program's value, or the map's default when the program supplies none. Without ERASE, bricks sends a **partial repaint**: the unnamed fields (constant text), the STOP fields, and only those named fields for which the program supplied a value. Named fields the program did not supply are not sent, and what the operator has typed into them stays on the screen.

| Option | Screen cleared | Named fields sent | Waits for the operator |
|---|---|---|---|
| ERASE | Yes | All | Yes |
| (neither) | No | Those supplied | Yes |
| DATAONLY | No | Those supplied | No |

**Figure 11-2. The three ways of painting a map**

In REXX a field counts as supplied when its stem tail has been assigned (after `SCR. = ''`, every tail has been). In COBOL and PL/I every field that has an item of the same name in the `FROM` structure is supplied. Each value is padded or cut to the field's `LEN`, so a shorter value always overwrites the whole of a longer one.

### *The operator's reply*

Unless `DATAONLY` is coded, the command waits for an attention key. It then sets `EIBAID` to the key and `EIBCPOSN` to the cursor position (zero-based, on the same scale as `CURSOR`), and saves the field values for `RECEIVE MAP`. If PA1 was pressed, the task ends (see "How bricks Uses the Terminal"). `SEND ... CURSOR(EIBCPOSN)` puts the cursor back where the operator left it. `DATAONLY` changes neither `EIBAID` nor `EIBCPOSN`; it only records the map, so that a later `RECEIVE MAP` of it is accepted.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The map was sent and, unless DATAONLY, the reply received. |
| MAPFAIL | 36 | The map is not in the catalog; the name exists in more than one mapset and MAPSET was not coded; or the file that defines it failed to parse (the message gives the line). |
| INVREQ | 16 | ERASE, ERASEAUP and DATAONLY combined; or the CURSOR value is not a number. In a background task, a SEND MAP without DATAONLY is also refused with INVREQ; that refusal is not a condition and ends the task. |

**Figure 11-3. SEND MAP conditions**

The command can also end the task abnormally, whether or not RESP is coded: ABMB when the CURSOR position is at or beyond the end of the screen (1920 or more on a 24 x 80 screen, 3440 or more on 43 x 80; nothing is sent), ATNI when the terminal is lost, and AEXZ when bricks fails internally while building the screen. See "Other Transaction Abend Codes" in Appendix C.

***Examples***

```
SCR. = ''
SCR.GREETING = 'Hello,' USR
EXEC CICS SEND MAP('HELO1') FROM(SCR.) ERASE END-EXEC
```

```
MOVE 'Customer not found' TO MSG OF CUSTMAP.
EXEC CICS SEND MAP('CUST1') FROM(CUSTMAP) CURSOR(EIBCPOSN)
          RESP(WS-RESP) END-EXEC.
```

Figure 11-4 shows the screen that the COBOL sample HELC (`runtime/cobol/hello.cob`) sends with `SEND MAP('HELO1') FROM(SCR) ERASE`. The map defines only the position, length and attributes of its three protected fields, INFOLINE on row 1, GREETING and FOOTER on row 24; the program supplies every value, by moving the text to the items of the group SCR before the command. Because ERASE is coded, the rest of the screen is cleared. The REXX sample HELO sends the same map from the stem SCR. (see Figure 3-3 in Chapter 3).

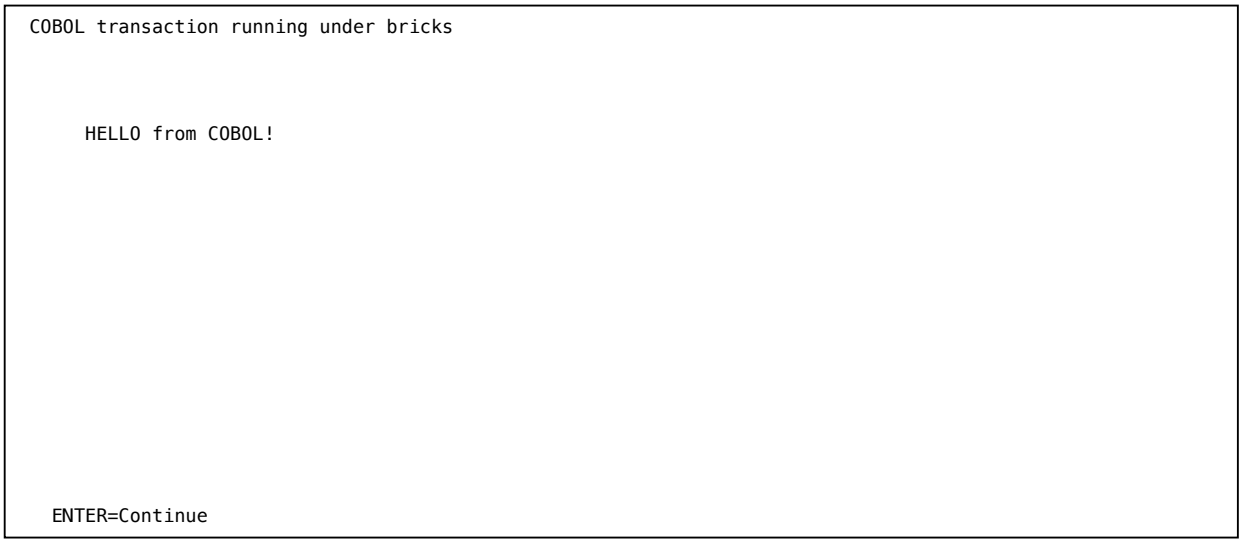



**Figure 11-4. HELC: SEND MAP of map HELO1 from a COBOL program** (Model 2, 24 x 80; capture screens/helc.txt)

### *Refreshing a screen in the background*

`DATAONLY` together with `START` lets a transaction refresh part of its screen, such as a clock or a message count, while its main task waits for the operator. The task schedules itself:

```
EXEC CICS START TRANSID('CHAT') INTERVAL(000002) FROM('TICK') END-EXEC
```

When the interval expires and the terminal's task is waiting in a `SEND MAP`, bricks runs the started task in the background. It recognizes itself by the data it retrieves (`RETRIEVE` returns `TICK`), supplies only the fields it refreshes, so the operator's input field is never repainted, issues `SEND MAP ... DATAONLY`, schedules the next tick and returns. The main task stays in its `SEND MAP` throughout. `runtime/rexx/chat.rexx` is a complete example. The rules for background tasks are given under "START" in Chapter 12.

***Notes***

- The colors of named fields can be changed for one `SEND MAP`; see "Changing a Field's Color at Runtime" in Chapter 3.
- A bare `CURSOR` keyword without a value (BMS symbolic cursor positioning) is ignored.
- A program that issues no `SEND` after reading a key, and returns with `RETURN TRANSID`, makes the next transaction wait for the operator's next key.

***Differences from IBM CICS***

- `SEND MAP` without `DATAONLY` waits for the operator's reply.
- `DATAONLY` means "write the supplied fields, do not wait". The IBM handling of attribute bytes supplied by the application in the symbolic map is not implemented.
- `ERASEAUP` clears the whole screen, as `ERASE`.
- `ALARM` is not sounded. `TIMEOUT` is a bricks extension.

## RECEIVE MAP

***Function***

Moves the field values of the operator's last reply to a map into the program.

***Syntax***

```
EXEC CICS RECEIVE MAP(name)
     [MAPSET(name)]
     [INTO(area)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| `MAP(name)` | Required. The map on the screen: the map of the most recent `SEND MAP`. |
| `MAPSET(name)` | The mapset of the map. When coded, it must be the mapset the map was sent from. |
| `INTO(area)` | Where the values go. In REXX a stem (default `MAP.`): the value of field `NAME` is stored in `stem.NAME`. In COBOL and PL/I a structure with items named after the fields. |

**Figure 11-5. RECEIVE MAP options**

### *Description*

`RECEIVE MAP` does not read the terminal. It takes the reply saved by the most recent `SEND MAP` of this task, or the reply that started the task when the previous task ended with `RETURN TRANSID` (see "How bricks Uses the Terminal"). The values are converted to bricks SBCS. It also sets `EIBAID` and `EIBCPOSN` to the key and cursor position of that reply. A `HANDLE AID` is acted on at most once for each reply, whether the reply is collected by the `SEND MAP` or the `RECEIVE MAP`.

A program called with `LINK` may send and receive its own maps. When it returns, the caller's last map and reply are restored, so the caller's `RECEIVE MAP` still reads the caller's screen.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| `NORMAL` | `0` | The values were moved. |
| `MAPFAIL` | `36` | There is no saved reply; the reply is for a different map, or a different mapset; the map name is ambiguous and `MAPSET` was not coded; or the mapset named does not exist. |

**Figure 11-6. RECEIVE MAP conditions**

When MAPSET names a mapset that exists but does not contain the map, the task ends abnormally with abend code ABM0, with the message RECEIVE MAP: map "NOPE" not in mapset "HELO1". RESP does not absorb it.

***Example***

```
EXEC CICS RECEIVE MAP('CUST1') INTO(IN.) END-EXEC
IF C2X(EIBAID) = 'F3' THEN SIGNAL QUIT     /* PF3 */
ACTION = IN.ACTION
CKEY   = IN.CUSTNO
```

***Differences from IBM CICS***

IBM CICS reads the terminal at RECEIVE MAP; bricks reads it at SEND MAP and returns the saved reply. bricks does not set the length (-L) and flag (-F) fields of a symbolic map, and it does not raise MAPFAIL merely because no field was modified.

## SEND TEXT

***Function***

Writes unformatted text to the terminal, without a map, and waits for the operator's reply.

***Syntax***

```
EXEC CICS SEND TEXT FROM(area)
     [LENGTH(data-value)]
     [ERASE]
     [CURSOR(data-value)]
     [FREEKB] [ALARM] [FRSET]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| FROM(area) | Required. The text: a variable, a COBOL group item or a literal. |
| LENGTH(n) | Use only the first *n* characters of the text. |
| ERASE | Clear the screen first. Without it, what is on the screen outside the painted rows stays. |
| CURSOR(n) | Place the cursor at buffer position *n*, counted from zero, as for "SEND MAP". Without it the cursor goes to row 1, column 1 on ERASE and otherwise stays where it was. |
| FREEKB | Accepted. SEND TEXT always unlocks the keyboard, because it waits for the operator (or, in a background task, is written for a terminal whose operator is answering another task). |
| FRSET, ALARM | Accepted, as for SEND MAP. |

**Figure 11-7. SEND TEXT options**

### *Description*

The 3270 has no line-feed or carriage-return character. bricks lays the text out as one flat row-major buffer: each run of *cols* characters (80 on most terminals) fills one row, starting at the top row. Each row is painted as a field whose attribute byte occupies column 1, so only the first *cols* - 1 characters of each run are shown; the last character of the run is dropped, not carried to the next row. Text beyond rows x *cols* characters is dropped. To display separate lines, pad each line to the width of the screen and concatenate the lines: `LEFT(line, 80)` in REXX, a group of `PIC X(80)` items in COBOL. Text sent this way is shown in turquoise. Control bytes are shown as `?`.

The command then waits for an attention key and sets `EIBAID` and `EIBCPOSN`, as `SEND MAP` does, so the operator has time to read the screen before the program continues. PA1 ends the task. In a task started in the background, the text is written and the command returns without waiting.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The text was painted and the reply received. |
| INVREQ | 16 | FROM is missing, or the CURSOR value is not a number. |

**Figure 11-8. SEND TEXT conditions**

A CURSOR position beyond the screen ends the task with ABMB; a lost terminal with ATNI.

### *Example*

```
MSG = LEFT('Customer' CKEY, 80) || LEFT('Balance:' BAL, 80)
EXEC CICS SEND TEXT FROM(MSG) ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

The GETC program in "Example 2. SEND TEXT and RECEIVE Without a Map" in Chapter 42 shows a longer multi-row display.

### *Differences from IBM CICS*

IBM CICS formats SEND TEXT data into pages, honors new-line characters, and writes the pages without waiting. bricks paints the text at once, line breaks only at multiples of the screen width, and waits for the operator's reply. The ACCUM, PAGING, HEADER and TRAILER options are not supported.

## SEND CONTROL

### *Function*

Sends 3270 control functions without data: clears the screen, moves the cursor, or both.

### *Syntax*

```
EXEC CICS SEND CONTROL
     [ERASE | ERASEAUP]
     [CURSOR(data-value)]
     [FREEKB] [ALARM] [FRSET]
     [PRINT] [LDC(name)] [STRFIELD]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| ERASE, ERASEAUP | Clear the screen. |
| CURSOR(n) | Move the cursor to buffer position *n*, counted from zero, as for "SEND MAP". Without ERASE, the contents of the screen are left exactly as they are and only the cursor moves. |
| FREEKB | Unlock the keyboard. Without it, a SEND CONTROL leaves the keyboard locked (except in a background task). On its own, SEND CONTROL FREEKB sends a write that only unlocks the keyboard. |
| FRSET | Accepted. Every paint resets the modified-data tags already. |
| ALARM, PRINT, LDC, STRFIELD | Accepted; no effect. |

**Figure 11-9. SEND CONTROL options**

### *Description*

SEND CONTROL never waits for the operator and never changes EIBAID or EIBCPOSN. With none of ERASE, CURSOR and FREEKB it sends nothing and ends normally. It may be used in a task started in the background.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The control functions were sent (or there was nothing to send). |
| INVREQ | 16 | The CURSOR value is not a number. |

**Figure 11-10. SEND CONTROL conditions**

A CURSOR position beyond the screen ends the task with ABMB and nothing is sent; a lost terminal with ATNI.

***Example***

```
EXEC CICS SEND CONTROL CURSOR(EIBCPOSN) END-EXEC   /* cursor only */
EXEC CICS SEND CONTROL ERASE END-EXEC              /* clear       */
EXEC CICS SEND CONTROL FREEKB END-EXEC             /* unlock      */
```

## RECEIVE

***Function***

Returns the line the operator typed at the blank screen to start the task.

***Syntax***

```
EXEC CICS RECEIVE INTO(data-area)
     [LENGTH(data-area)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

***Options***

| Option | Description |
|---|---|
| `INTO(data-area)` | Required. The variable that receives the line. It must be a variable, not a literal. |
| `LENGTH(data-area)` | On input, if positive, the largest number of characters to deliver; the line is cut to it. On output, when the option names a variable, the number of characters delivered. |

**Figure 11-11. RECEIVE options**

***Description***

The operator starts a transaction by typing its TRANSID at the blank screen, optionally followed by data. `RECEIVE` delivers that whole line, TRANSID included (for example `EXAM 1 2 3`), converted to SBCS. It is the usual way for a COBOL or PL/I program to pick up arguments; a REXX program can also use `ARG(1)`.

The line can be received once. A second `RECEIVE` in the same task, and any `RECEIVE` in a task that was not started from the blank screen (by `RETURN TRANSID`, `XCTL` or `START`), returns `EOC` and leaves `INTO` unchanged. A COBOL receiving field is filled by the rules of `MOVE`: a shorter line is padded, a longer one cut. Compare the returned length with the size of the field to detect a cut.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The line was delivered. |
| EOC | 6 | There is no line to deliver: it has been received already, or the task was not started from the blank screen. The default action is to continue; an unhandled EOC does not end the task. |
| INVREQ | 16 | INTO is a literal. |

**Figure 11-12. RECEIVE conditions**

### *Examples*

```
EXEC CICS RECEIVE INTO(BUF) LENGTH(LEN) END-EXEC   /* "GETC 100" */
PARSE VAR BUF TID CKEY .
```

```
EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC.
UNSTRING WS-INPUT DELIMITED BY ' '
    INTO WS-TID WS-A WS-B WS-C
END-UNSTRING.
```

### *Differences from IBM CICS*

RECEIVE does not read the terminal; it returns the input that started the task. LENGERR is not raised when the line is longer than the receiving field; the line is cut. MAXLENGTH, NOTRUNCATE, SET and BUFFER are not supported.

## CONVERSE

### *Function*

Sends output to the terminal and receives the reply in one command.

***Syntax***

```
EXEC CICS CONVERSE MAP(name) [MAPSET(name)]
     [FROM(area) | FROMMAP(area)]
     [INTO(area)]
     [ERASE] [CURSOR(data-value)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC

EXEC CICS CONVERSE FROM(area) [FROMLENGTH(data-value)]
     INTO(data-area) [TOLENGTH(data-area) | MAXLENGTH(data-value)]
     [STRFIELD] [DEFRESP]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

***Options***

| Option | Description |
|---|---|
| MAP(name) | Selects the mapped form. The map is sent and then received, as by SEND MAP and RECEIVE MAP. |
| FROM(area), FROMMAP(area) | Mapped form: the data for the SEND MAP; the two spellings are equivalent. Text form: the text for the SEND TEXT. |
| INTO(area) | Mapped form: the target of the RECEIVE MAP. Text form: the target of the RECEIVE. |
| MAPSET, ERASE, CURSOR | Mapped form: as for SEND MAP. |
| FROMLENGTH(n) | Text form: the number of characters of FROM to send. |
| TOLENGTH(v), MAXLENGTH(n) | Text form: the length limit and result for INTO, as LENGTH on RECEIVE. |
| STRFIELD, DEFRESP | Accepted; no effect. |

**Figure 11-13. CONVERSE options**

***Description***

The **mapped form** is a SEND MAP followed by a RECEIVE MAP of the same map. If the send does not end normally, the receive is not attempted and the response of the send is returned. Older CICS programs use this form, and it lets them be moved to bricks without rewriting.

The **text form** (no MAP; both FROM and INTO required) is a SEND TEXT followed by a RECEIVE. Because bricks' RECEIVE returns the line that started the task (see "RECEIVE"), the text form does not return what the operator typed in reply to the text: INTO receives the starting line, or the command ends with EOC if it has been received already. The reply's key and cursor position are in EIBAID and EIBCPOSN.

CONVERSE WEB is a synonym of WEB CONVERSE; see "WEB CONVERSE" in Chapter 20.

***Conditions***

Those of SEND MAP and RECEIVE MAP (mapped form) or SEND TEXT and RECEIVE (text form). INVREQ (16) if neither MAP nor both FROM and INTO are coded.

***Example***

```
EXEC CICS CONVERSE MAP('TIM1') FROM(SCR) INTO(SCR) ERASE END-EXEC.
IF EIBAID = DFHPF3
    EXEC CICS RETURN END-EXEC
END-IF.
```

This replaces SEND MAP('TIM1') FROM(SCR) ERASE followed by RECEIVE MAP('TIM1') INTO(SCR). runtime/cobol/timc.cob uses CONVERSE for both of its screens.

Chapter 12

# Program Control Commands

This chapter describes the commands that end a program, pass control between programs, and schedule further work: RETURN, XCTL, LINK, ABEND, START, RETRIEVE and CANCEL. HANDLE ABEND, SYNCPOINT and the unit of work are described in Chapter 16.

## Programs, Levels and the Unit of Work

A task begins with one program, its **top-level** program. A program can call another with LINK; the called program runs at the next **logical level** and, when it ends, control returns to the statement after the LINK. A program can transfer control with XCTL; the target replaces it at the same level and control does not come back. A program in bricks is named by the TRANSID of the transaction that runs it ("Transactions and Programs" in Chapter 1).

All the programs of a task, at every level, run in one **unit of work**: one database transaction, one journal of file and temporary storage updates, and one set of locks (ENQ resources and file records held for update). Neither LINK, nor XCTL, nor a RETURN from a linked program is a syncpoint. The unit of work is committed when the task ends normally, or at an explicit SYNCPOINT, and backed out when the task ends abnormally.

| Command | Issued at the top level | Issued at a LINK level |
|---|---|---|
| RETURN | Ends the task and commits the unit of work. May name the next transaction and pass a COMMAREA. | Returns to the program that issued the LINK. No commit. TRANSID is recorded for the end of the task. |
| XCTL | The target replaces the program in the same task and unit of work. | The target replaces the program at the same level; its RETURN goes back to the LINKer. |
| LINK | The target runs at the next level, in the same task and unit of work, and returns. | |
| ABEND | Ends the task abnormally and backs out the unit of work, unless a HANDLE ABEND at this level or a higher one catches the abend. | |

**Figure 12-1. Program control at each logical level**

## RETURN

### *Function*

Returns control from a program: to the program that linked to it, or, at the top level, to bricks, ending the task. At the top level it can name the transaction that is to process the operator's next input, and pass it a COMMAREA.

### *Syntax*

```
EXEC CICS RETURN
     [TRANSID(name) [IMMEDIATE]]
     [COMMAREA(data-area) [LENGTH(data-value)]]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| `TRANSID(name)` | The transaction to run next on this terminal. At a LINK level it is recorded for the end of the task. Not allowed in a task that has no terminal. |
| `IMMEDIATE` | Start the next transaction at once, without waiting for terminal input. Top level only. |
| `COMMAREA(data-area)` | Data to pass to the next transaction, which finds it in `DFHCOMMAREA` with its length in `EIBCALEN`. Top level only. |
| `LENGTH(n)` | Pass at most *n* bytes of the COMMAREA. |

**Figure 12-2. RETURN options**

### *At the top level*

`RETURN` ends the task. bricks takes the implicit syncpoint first: the unit of work is committed. If the database refuses the commit, the task ends abnormally with `ASP3`, `ASP7` or `ASPR` instead, and nothing is handed on (see "Commit Points" in Chapter 16).

Without `TRANSID` the terminal returns to the blank screen. With `TRANSID(x)`, transaction `x` processes the terminal's next input, as in IBM CICS: `RETURN` sends nothing and leaves the screen as it is, and `x` starts when the operator next presses ENTER, a PF or PA key, or CLEAR. That input belongs to `x`, whatever was typed: the first characters are not taken as a TRANSID, `EIBAID` is the key pressed (so `IF EIBAID = DFHCLEAR` works on entry), and a map still on the screen can be read with `RECEIVE MAP`.

Because a bricks `SEND MAP` or `SEND TEXT` already waits for the operator's key, a task that has read a key that way and then returns with `TRANSID(x)` has the next input already: `x` starts at once, with that key in `EIBAID`. Only a task that read no input (it painted with `SEND MAP DATAONLY`, or painted nothing) makes `x` wait. While the terminal waits, it behaves as at the blank screen: a terminal that is not signed on is still subject to the idle timeout, and a `START` that becomes due runs (in the background) without disturbing `x` or its COMMAREA.

The COMMAREA reaches the next transaction only when `RETURN` codes the `COMMAREA` option; bricks never passes on the task's own `DFHCOMMAREA` by itself. Without the option the next transaction starts with `EIBCALEN` 0. A COBOL data area travels with its full declared length. A COBOL program that uses the default `DFHCOMMAREA` (one that declares none receives a 2000-byte area) passes on only the `EIBCALEN` bytes it received, so `RETURN COMMAREA(DFHCOMMAREA)` hands on the same length; to pass more, declare an area of the needed size in WORKING-STORAGE or LINKAGE SECTION. On a first start (`EIBCALEN` 0), what was moved into the default area travels with its trailing blanks removed.

***At a LINK level***

In a program called with `LINK`, `RETURN` goes back to the caller, which continues after its `LINK`. It is not a syncpoint; the unit of work is settled when the task ends. As in IBM CICS:

- `TRANSID` is allowed. It is recorded as the task's next transaction and takes effect when the task ends normally, unless the top-level program's own `RETURN TRANSID` names a different one. An abend discards it, and so does an error on the final `RETURN`.
- `COMMAREA`, `CHANNEL` and `IMMEDIATE` belong to the highest logical level. Coded in a linked program they raise `INVREQ` with RESP2 2. A linked program returns data by updating the COMMAREA it was passed.

***Conditions***

| Condition | RESP | RESP2 | Meaning |
|---|---|---|---|
| INVREQ | 16 | 1 | TRANSID in a task that has no terminal (one started by START without TERMID), at any level. |
| INVREQ | 16 | 1 | IMMEDIATE at the top level with no transaction to start (IBM documents no RESP2 for this case). |
| INVREQ | 16 | 2 | COMMAREA, CHANNEL or IMMEDIATE in a linked program. |

**Figure 12-3. RETURN conditions**

***Examples***

```
EXEC CICS RETURN TRANSID('CUST') COMMAREA(SAVEAREA) END-EXEC
```

```
EXEC CICS RETURN TRANSID('QAGR') COMMAREA(WS-STATE)
          LENGTH(LENGTH OF WS-STATE) END-EXEC.
GOBACK.
```

***Notes***

A COBOL program may also end with GOBACK, STOP RUN, EXIT PROGRAM or by running off the end of the PROCEDURE DIVISION. bricks treats each as a normal end and takes the implicit syncpoint, so SQL updates are not lost. None of them can pass a TRANSID or COMMAREA, and STOP RUN is poor style in CICS code; end with EXEC CICS RETURN and put GOBACK after it.

***Differences from IBM CICS***

- While the terminal waits for the input that starts the next transaction, a START that becomes due runs in the background and the pending TRANSID is kept. In IBM CICS an automatically initiated task cancels the pending TRANSID.
- In a task routed to another region (multiregion operation), RETURN TRANSID starts the next transaction at once.
- Text typed on an unformatted screen during the wait is not passed to the next transaction; the fields of a map on the screen are.
- CHANNEL has no effect at the top level; bricks has no channels.

## XCTL

### *Function*

Transfers control to another program at the same logical level. The issuing program does not regain control.

### *Syntax*

```
EXEC CICS XCTL PROGRAM(name)
     [COMMAREA(data-area) [LENGTH(data-value)]]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| PROGRAM(name) | Required. The TRANSID of the target, defined in `transactions.conf`, or a supplied transaction such as CEMT. |
| COMMAREA(data-area) | Data for the target, which receives it as DFHCOMMAREA. Without it the target's EIBCALEN is 0. |
| LENGTH(n) | Pass at most *n* bytes of the COMMAREA. |

**Figure 12-4. XCTL options**

### *Checking the target*

The target is checked at the XCTL, before control leaves the issuing program, at every level. A target that is not defined, or whose language is not supported, raises PGMIDERR; a target the user may not run, including a supplied transaction whose group the user lacks, raises NOTAUTH. Both are ordinary conditions in the issuing program: RESP and HANDLE CONDITION apply, an unhandled PGMIDERR in COBOL or PL/I is abend AEI0, and a REXX program continues with RC 27. Nothing is handed on or backed out.

A target that passes the check but cannot be loaded (its source file is missing or fails to compile, or its definition changed after the XCTL) fails when it starts. The task has failed: the unit of work is backed out, and the log shows `XCTL A->B: target not started, unit of work backed out`.

***The unit of work crosses XCTL***

`XCTL` is not a syncpoint. The target runs in the same task and the same unit of work, in any language and over any number of `XCTL`s, so everything the issuing program has not committed is carried to the target:

- its open SQL transaction, on the same database connection;
- its pending updates to recoverable files and temporary storage;
- its locks: every `ENQ` (with or without `HOLD`), a record still held by `READ ... UPDATE` (the target can `REWRITE` or `DELETE` it) and every record the task has changed. No other task can take them before the target commits or backs out.

The carried work is committed when the target (or a program it transfers to) issues `RETURN` or `SYNCPOINT` or ends normally, and backed out, including the issuing program's updates, when the target abends, issues `SYNCPOINT ROLLBACK`, or has a commit refused (`ASP3`, `ASP7`, `ASPR`).

***Target on another database***

One unit of work cannot span two databases. When the target is bound to a different database from the one the open SQL transaction uses (the DATABASE field of its definition, or after `CONNECT TO`), or the target is a supplied transaction or a transaction routed to another region, bricks takes an **implicit syncpoint at the XCTL**: the issuing program's database, file and temporary storage updates are committed before control passes, and the target starts a new unit of work. For a target on another database the task goes on, so its `ENQ`s and a record still held for update are carried to the target. A supplied or routed target runs outside the task, so they are released at the `XCTL`. If the database refuses that commit, the issuing program ends abnormally with `ASP3`, `ASP7` or `ASPR` (message `XCTL: ...`), its file and temporary storage updates are backed out, and the target is not started.

***XCTL at a LINK level***

In a program that was called with `LINK`, `XCTL` transfers control at the same level, as in IBM CICS. The target replaces the issuing program, and when the target returns or ends, control goes back to the program that issued the `LINK`. Chains work: if A links to B, B transfers to C and C to D, then D's `RETURN` resumes A, in any mix of languages.

- The target runs in the same task and unit of work as the rest of the chain, at the same depth; it does not count as another level, and CEDF shows it at the level of the program it replaced. Nothing is committed or carried at such an XCTL.
- The target receives the COMMAREA coded on the XCTL (EIBCALEN 0 without one). The linker sees changes to its own COMMAREA only while every program in the chain works on the linker's area: the linked program does, and so does each target that is passed COMMAREA(DFHCOMMAREA). Once an XCTL in the chain passes another area, or none, the linker's area is returned unchanged.
- An abend in the target is an abend of the LINK: the linker's HANDLE ABEND catches it; if unhandled, the task ends with it and the whole unit of work is backed out.
- RETURN in the target is a LINK-level RETURN: no syncpoint, and COMMAREA or IMMEDIATE raise INVREQ.
- A target routed to another region raises INVREQ; bricks routes LINK, never XCTL, to a peer region. A supplied transaction runs at the level as a LINK to it would, and ends the level.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| PGMIDERR | 27 | The target is not defined, or its language is not supported. Raised in the issuing program, at any level. |
| NOTAUTH | 70 | The user may not run the target. |
| INVREQ | 16 | PROGRAM is missing; or, at a LINK level, the target is routed to another region. |

**Figure 12-5. XCTL conditions**

CHANNEL is refused with INVREQ and the message CHANNEL not supported in bricks (use COMMAREA); the refusal is not a condition and ends the task.

***Example***

```
EXEC CICS XCTL PROGRAM('CUSV') COMMAREA(SCR) RESP(RC) END-EXEC
IF RC = 27 THEN SAY 'CUSV is not defined'
```

### *Differences from IBM CICS*

- Open SQL cursors are closed at the `XCTL`. The target could not address them anyway; the difference shows only when a program transfers back to itself and expects a cursor to be open.
- A target on another database is reached through an implicit syncpoint; IBM CICS would keep both databases in one unit of work with two-phase commit.

## LINK

### *Function*

Calls another program at the next logical level. When the called program returns, control comes back to the statement after the `LINK`.

### *Syntax*

```
EXEC CICS LINK PROGRAM(name)
     [COMMAREA(data-area) [LENGTH(data-value)]]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| `PROGRAM(name)` | Required. The TRANSID of the program to call, or a supplied transaction. |
| `COMMAREA(data-area)` | The communication area. A variable is passed to the called program and updated from it on return; a literal is passed but nothing comes back. |
| `LENGTH(n)` | Pass at most *n* bytes. Only those bytes come back; the caller's bytes beyond *n* are not touched. |

**Figure 12-6. LINK options**

***Description***

The called program runs in the caller's task and unit of work, with `DFHCOMMAREA` set from the caller's area and `EIBCALEN` its length. Programs in any of the three languages may link to one another; the COMMAREA is passed as bytes, so a PL/I structure and a COBOL group with the same layout exchange packed and binary fields unchanged. The user's access to the called program is checked, so a program cannot reach an administrative program the user could not start directly.

The called program cannot change the length of the COMMAREA: the caller always gets back exactly the number of bytes it passed. A COBOL called program works on those bytes; fields it does not touch keep the caller's data. In a REXX called program, assigning to `DFHCOMMAREA` replaces the whole area, as a COBOL `MOVE` to the whole 01 does: a shorter value is padded with blanks (`DFHCOMMAREA = 'Q'` over `ABCDEFGH` gives `Q` and seven blanks), a longer one is cut. Use `OVERLAY` or `SUBSTR` to change part of it. A supplied transaction returns no COMMAREA; the caller's area is unchanged.

bricks saves the caller's terminal state before the `LINK` and restores it afterwards: the next TRANSID and COMMAREA it has set, and its last map and reply, so that its own `RECEIVE MAP` still reads its own screen. The one exception is a `RETURN TRANSID` in the called program, which is recorded for the end of the task (see "RETURN"). If the called program transfers with `XCTL`, the target takes over the level (see "XCTL at a LINK level").

***Abends in the called program***

An abend in the called program, whether from `EXEC CICS ABEND`, an unhandled condition, a database abend such as `ADBU`, or the runaway-task abend `AICA`, is an abend of the caller. It is not a condition: `RESP` does not absorb it and `HANDLE CONDITION` does not see it. `EIBABCODE` takes the abend code and the caller's `HANDLE ABEND`, if any, receives control; otherwise the caller does not continue after the `LINK`, the abend passes to the next level up, and at the top the task ends and the whole unit of work is backed out, including what the caller changed before the `LINK`. A called program that fails in some other way while it runs also ends the caller's task.

### *Nesting limit*

At most 100 LINK levels may be active in a task at one time; a program that links to itself, directly or through others, counts every level. The LINK that would open level 101 raises INVREQ, which a program with RESP coded can test to stop recursing. The interpreters' own nesting limits are shared by all the programs of a task, not restarted at each level: 1000 nested REXX CALLs, 1024 nested COBOL PERFORMs and 100 nested PL/I calls per task.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| PGMIDERR | 27 | The program is not defined, its language is not supported, it cannot be loaded, or the user may not run it. |
| INVREQ | 16 | The LINK would exceed 100 active levels. |

**Figure 12-7. LINK conditions**

CHANNEL is refused with INVREQ; the refusal is not a condition and ends the task.

### *Example*

```
SAV = 'CHK' CKEY
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(SAV) RESP(RC) END-EXEC
IF RC \= 0 THEN SIGNAL NOPROG
PARSE VAR SAV STATUS '|' MSG
```

### *Differences from IBM CICS*

- A program the user may not run raises PGMIDERR, not NOTAUTH.
- IBM CICS sets no fixed limit on LINK nesting; bricks runs every level on one stack and stops at 100 so that a runaway recursion cannot exhaust the region.
- The COMMAREA is passed by value and copied back on return; the effect is that of IBM's pass by address.

## ABEND

### *Function*

Ends the task abnormally.

### *Syntax*

```
EXEC CICS ABEND
     [ABCODE(name)]
     [CANCEL]
     [NODUMP]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| ABCODE(name) | A four-character abend code. Without it the code is AAAA and no dump is taken. |
| CANCEL | Ignore every HANDLE ABEND, at this level and at every level above; the task ends. |
| NODUMP | Take no transaction dump. |

**Figure 12-8. ABEND options**

### *Description*

The abend code is stored in EIBABCODE (and is returned by ASSIGN ABCODE). Unless a HANDLE ABEND at this level, or at a level above, catches the abend ("HANDLE ABEND" in Chapter 16; CANCEL prevents this), the task ends: its unit of work is backed out, message DFHAC2236 is written to the log, and the terminal shows DFHAC2206. No next TRANSID is honored after an abend, so a pseudo-conversation is broken: the next transaction the operator starts has EIBCALEN 0. When a HANDLE ABEND catches the abend, nothing is backed out; the exit decides whether to issue SYNCPOINT ROLLBACK.

With ABCODE and without NODUMP, a transaction dump is written to abend.log before any HANDLE ABEND is searched, so it is taken even when an exit catches the abend, and message DFHDU0203I is logged ("Transaction Dumps" in Chapter 16). The command appears in the dump's CMD record. CAID ("CAID: The Abend Analyser") has no rule for a code that the application chose, and lists the typical causes of such an abend instead; the REXX sample ABUR shows this.

### *Conditions*

None. ABEND does not return to the program.

***Example***

```
IF EIBRESP \= 0 THEN
   EXEC CICS ABEND ABCODE('FIO1') END-EXEC
```

## START

***Function***

Schedules a transaction to start on this terminal after an interval or at a time of day, optionally passing it data that it reads with RETRIEVE.

***Syntax***

```
EXEC CICS START TRANSID(name)
     [INTERVAL(hhmmss) | TIME(hhmmss)
      | [AFTER] [HOURS(n)] [MINUTES(n)] [SECONDS(n)] [MILLISECS(n)]
      | AT [HOURS(n)] [MINUTES(n)] [SECONDS(n)] [MILLISECS(n)]]
     [FROM(data-area) [LENGTH(data-value)]]
     [TERMID(name)]
     [REQID(name)]
     [PROTECT]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

***Options***

| Option | Description |
|---|---|
| TRANSID(name) | Required. The transaction to start. It is not checked when the START is issued; the definition and the user's access are checked when it is started. |
| INTERVAL(hhmmss) | Start after this interval: one to six digits, read as hhmmss with leading zeros supplied. INTERVAL(30) is 30 seconds, INTERVAL(000130) one minute 30 seconds. Minutes and seconds must be 0 through 59. |
| TIME(hhmmss) | Start at this time of day, in the time zone of bricks.cnf. A time that has passed (or is now) means the same time tomorrow. |
| AFTER HOURS MINUTES SECONDS MILLISECS | Start after the sum of the components, any of which may be omitted. They are counts, not clock digits: AFTER MINUTES(90) is an hour and a half. Limits: HOURS 0-99, MINUTES 0-5999, SECONDS 0-359999, MILLISECS 0-360000000, total at most 100 hours. Components without AFTER or AT are read as AFTER. |
| AT HOURS MINUTES SECONDS MILLISECS | Start at the next time of day equal to the sum, which must be under 24 hours; rolls to tomorrow as for TIME. |
| FROM(data-area) | Data for the started task. It is copied when the START is issued, so the program may change the area afterwards. A COBOL area travels with its full declared length. |
| LENGTH(n) | Pass at most *n* bytes of FROM. |

| Option | Description |
|---|---|
| TERMID(name) | Must be this terminal's identifier. With it the started task is a terminal task; without it, a task with no terminal (see below). |
| REQID(name) | A name for the request, by which CANCEL can find it. |
| PROTECT | Accepted; no effect. |

**Figure 12-9. START options**

Without a time option the transaction is due at once. MILLISECS is a bricks extension, and may also be coded alone (START TRANSID('X') MILLISECS(100)). INTERVAL and TIME exclude each other and the component forms; AFTER excludes AT.

### *When the transaction runs*

A due START never interrupts a task that is running. What happens depends on what the terminal is doing:

| The terminal is | The started transaction |
|---|---|
| at the blank screen, or waiting for the input that starts a RETURN TRANSID transaction | runs at once as an ordinary task on the terminal, in any language. A pending RETURN TRANSID and its COMMAREA are kept. |
| in a task that is waiting for the operator (in a SEND MAP or SEND TEXT) | runs at once as a **background task**, alongside the waiting task. |
| in a task that is running (computing, in SQL or file I/O, in a DELAY) | waits. It runs in the background when that task next waits for the operator, or as an ordinary task when it ends. |

**Figure 12-10. When a started transaction runs**

***Background tasks***

A background task runs alongside the terminal's waiting task, on its own thread of control: a slow or looping background task never delays the scheduler, other terminals, or later `START`s for other terminals. Background tasks for the same terminal run one at a time, because they share the terminal. The rules are:

- The transaction must be a REXX or PL/I program. A COBOL transaction is not run; the failure is logged.
- The task may not wait for the operator: `SEND MAP` without `DATAONLY` and `CONVERSE` fail with `INVREQ`, and `SEND TEXT` writes without waiting (see "How bricks Uses the Terminal" in Chapter 11). It is meant for `SEND MAP DATAONLY`, file and queue commands, `ASSIGN` and `START`. It has no SQL connection.
- It has its own task state: no COMMAREA, its own `RETRIEVE` data, `EIBAID`, next TRANSID and map state. Nothing it does changes the waiting task's state.
- It commits its unit of work when it ends normally and backs it out when it abends. An `XCTL` in it runs the target (REXX or PL/I) in the same task and unit of work.
- A background task that loops without issuing commands is ended by the runaway-task check with abend `AICA` after two seconds ("Runaway Tasks" in Chapter 16).
- It has no screen of its own: an abend is logged as `DFHAC2236`, and for a `START` that named `TERMID` the `DFHAC2206` screen is shown when the terminal is next free. Other failures are logged.

***Tasks without a terminal***

A `START` without `TERMID` starts a task that, as in IBM CICS, has no terminal: its abend goes to the log only, and `RETURN TRANSID` in it raises `INVREQ` RESP2 1. bricks nevertheless runs it on the issuing terminal, under the rules above.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The request was queued. |
| INVREQ | 16 | TRANSID missing or blank; TERMID names another terminal; a malformed or out-of-range time; a contradictory combination of time options; a LENGTH that is not a non-negative number. |

**Figure 12-11. START conditions**

SYSID and QUEUE are refused with INVREQ and a message that names the option; the refusal is not a condition and ends the task.

***Example***

```
EXEC CICS START TRANSID('TIMC') INTERVAL(000030) FROM(MSG)
          TERMID(EIBTRMID) REQID('TIMC1') END-EXEC.
```

TIMC starts on this terminal in 30 seconds and retrieves MSG. runtime/cobol/timc.cob and runtime/rexx/timr.rexx are complete examples.

***Differences from IBM CICS***

- A transaction can be started only on the issuing terminal; a TERMID of another terminal is refused.
- Pending requests are held in storage. They are lost when bricks stops, and dropped when the terminal disconnects.
- Started tasks that find the terminal busy run in the background, with the restrictions above, instead of waiting for the terminal.
- PROTECT has no effect; SYSID and QUEUE are not supported. MILLISECS is a bricks extension.

## RETRIEVE

***Function***

Returns the data passed by the START that started this task.

### *Syntax*

```
EXEC CICS RETRIEVE INTO(data-area)
     [LENGTH(data-area)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| INTO(data-area) | Required. The variable that receives the data. It must be a variable, not a literal. |
| LENGTH(data-area) | On input, if positive, the largest number of bytes to deliver. On output, when the option names a variable, the number delivered. |

**Figure 12-12. RETRIEVE options**

### *Description*

A task started by a START with FROM can retrieve the data once. ENDDATA is returned, and INTO left unchanged, when there is no data: the task was started from the terminal, the START passed no FROM data, or the data has been retrieved already. A program that can be started both ways issues RETRIEVE first and branches on the response. In COBOL and PL/I, code RESP: an unhandled ENDDATA ends the task with abend AEI2.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The data was delivered. |
| ENDDATA | 29 | There is no data to retrieve. |
| INVREQ | 16 | INTO is missing or is a literal. |

**Figure 12-13. RETRIEVE conditions**

***Example***

```
MOVE SPACES TO BUF.
EXEC CICS RETRIEVE INTO(BUF) LENGTH(BUFLEN) RESP(WS-RESP) END-EXEC.
IF WS-RESP = DFHRESP(NORMAL)
    PERFORM SHOW-REMINDER         *> started by START
    EXEC CICS RETURN END-EXEC
END-IF.
PERFORM PROMPT-AND-SCHEDULE.      *> started from the terminal
```

***Differences from IBM CICS***

Only the data of the one START that started the task can be retrieved; requests for the same transaction are not accumulated. RTRANSID, RTERMID, QUEUE and WAIT are not supported.

## CANCEL

***Function***

Cancels a START that has not yet run.

***Syntax***

```
EXEC CICS CANCEL
     {REQID(name) | TRANSID(name) [TERMID(name)]}
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

***Options***

| Option | Description |
|---|---|
| REQID(name) | Cancel the pending request that was given this REQID. Takes precedence when both REQID and TRANSID are coded. |
| TRANSID(name) | Cancel the pending request for this transaction that is due first. |
| TERMID(name) | With TRANSID: the terminal whose request is canceled. Default: this terminal. |

**Figure 12-14. CANCEL options**

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | A request was canceled. |
| NOTFND | 13 | No pending request matches. After bricks is restarted there are no pending requests. |
| INVREQ | 16 | Neither REQID nor TRANSID is coded. |

**Figure 12-15. CANCEL conditions**

### *Example*

```
EXEC CICS CANCEL REQID('TIMC1') RESP(RC) END-EXEC
IF RC = 13 THEN SAY 'The reminder has already run.'
```

### *Differences from IBM CICS*

CANCEL applies only to START requests; it does not cancel DELAY or POST requests. SYSID is not supported.

Chapter 13

# System Services Commands

This chapter describes the commands that give a program information about its environment, the date and the time, storage, serialization and security:

| Command | Purpose |
|---|---|
| ASSIGN | Return values about the task, the user and the terminal |
| ASKTIME, FORMATTIME | Read and format the date and time |
| DELAY | Suspend the task for an interval or until a time |
| ADDRESS | Obtain the address of a CICS storage area (COBOL) |
| GETMAIN, FREEMAIN | Obtain and release task storage (COBOL) |
| ENQ, DEQ | Serialize access to a named resource |
| QUERY SECURITY | Ask what access the user has to a transaction |
| VERIFY PASSWORD | Check a user's password again |
| INQUIRE SYSTEM | Return region settings |
| INQUIRE, SET, STARTBROWSE, ENDBROWSE | Inquire on files, tasks, transactions, terminals and programs |
| WRITE OPERATOR | Write a message to the console |

**Figure 13-1. System services commands**

## ASSIGN

### *Function*

Returns values about the task, the user, the terminal and the region.

### *Syntax*

```
EXEC CICS ASSIGN option(data-area) [option(data-area) ...]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Description*

Any number of the options in the following table may be coded in one command. Each stores its value in the variable named. An option whose argument is a literal is skipped. An unknown option ends the command with INVREQ.

***Options***

| Option | Value returned |
|---|---|
| USERID | The signed-on user; blank before sign-on. |
| TERMID, EIBTRMID | The terminal identifier, T and four digits. Use an eight-character field in COBOL and PL/I. |
| EIBAID | The attention identifier of the operator's last reply (one byte); blank if there has been none. |
| EIBCPOSN | The cursor position of the last reply, counted from zero. |
| EIBCALEN | The length of the COMMAREA passed to the program. |
| SCREENHT, SCREENWD | The rows and columns of the terminal. |
| ALTSCRNHT, ALTSCRNWD | The same values as SCREENHT and SCREENWD. |
| CWALENG | The length of the Common Work Area (wrkarea in bricks.cnf); 0 if there is none. |
| TWALENG | The length of the Transaction Work Area (TWASIZE of the transaction); 0 if there is none. |
| TCTUALENG | Always 0. |
| ABCODE | The current abend code; blank if none. |
| EIBTASKN | The task number. |
| STARTCODE | Always TO. |
| SYSID | BRKS. |
| APPLID | BRICKS01. |
| NETID | TCPIP. |
| CONNECTED, CONNTIME | bricks: when the terminal connected, YYYY-MM-DD HH:MM:SS. |
| TLS | bricks: yes if the terminal is connected with TLS, otherwise no. |
| DATE | bricks: today, YYYYMMDD. |

| Option | Value returned |
|---|---|
| TIME | bricks: now, HHMMSS. |
| TODAYYR, TODAYMO, TODAYDY | bricks: today's year (4 digits), month and day (2 digits each). |
| DAYCOUNT | bricks: the number of days since 1 January 1900, the base of ABSTIME. The difference of two values is a number of days. |

**Figure 13-2. ASSIGN options**

The date and time options use the region's clock, corrected by NTP when that is configured, in the time zone set by `time_zone` in `bricks.cnf`. They let a COBOL program do date arithmetic without intrinsic functions; `runtime/cobol/qagc.cob` computes an age from a date of birth this way.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | All values were returned. |
| INVREQ | 16 | An option is not known. |

**Figure 13-3. ASSIGN conditions**

### *Example*

```
EXEC CICS ASSIGN USERID(USR) TERMID(TRM)
                 SCREENHT(SCRH) SCREENWD(SCRW) END-EXEC
```

### *Differences from IBM CICS*

`STARTCODE` is always `TO`, also for a task started by `START`. `SYSID`, `APPLID` and `NETID` are fixed values. The options marked bricks are extensions. bricks has no `EIBUSER` field; obtain the user with `ASSIGN USERID`.

## ASKTIME

### *Function*

Updates `EIBDATE` and `EIBTIME` and optionally returns the current time as an absolute time.

### *Syntax*

```
EXEC CICS ASKTIME [ABSTIME(data-area)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| `ABSTIME(data-area)` | Receives the current time as a 15-digit decimal number of milliseconds since 00:00:00.000 on 1 January 1900, UTC. |

**Figure 13-4. ASKTIME options**

### *Description*

`EIBDATE` and `EIBTIME` are set when a task starts. `ASKTIME` sets them again from the region's clock, in the time zone of `bricks.cnf`:

- `EIBDATE` has the IBM form `0CYYDDD`, where `C` is the century minus 19, `YY` the year in the century and `DDD` the day of the year. 12 May 2026, day 132, is `0126132`.
- `EIBTIME` has the form `HHMMSS`.

`ASKTIME` is the only way to obtain an ABSTIME; pass it to `FORMATTIME` to obtain a formatted date or time.

### *Conditions*

`NORMAL` only.

### *Example*

```
EXEC CICS ASKTIME ABSTIME(NOW) END-EXEC
EXEC CICS FORMATTIME ABSTIME(NOW) YYYYMMDD(TODAY) DATESEP('-')
          TIME(CLOCK) TIMESEP(':') END-EXEC
SAY 'Stamped at' TODAY CLOCK        /* 2026-05-12 10:30:45 */
```

### *Differences from IBM CICS*

`EIBDATE`, `EIBTIME` and ABSTIME are character strings of decimal digits, not packed decimal fields.

## FORMATTIME

***Function***

Converts an absolute time into a date and time in the formats requested.

***Syntax***

```
EXEC CICS FORMATTIME ABSTIME(data-value)
     [DATE(data-area)] [YYYYMMDD(data-area)] [MMDDYYYY(data-area)]
     [DDMMYYYY(data-area)] [MMDDYY(data-area)] [DDMMYY(data-area)]
     [YYYYDDD(data-area)] [YYDDD(data-area)]
     [DATESEP(data-value)]
     [TIME(data-area)] [TIMESEP(data-value)]
     [YEAR(data-area)] [MONTHOFYEAR(data-area)] [DAYOFMONTH(data-area)]
     [DAYOFWEEK(data-area)] [DAYCOUNT(data-area)]
     [TIMEZONE(data-value)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

***Options***

| Option | Value returned |
|---|---|
| ABSTIME(v) | Required. The time to convert: a decimal number of milliseconds since 1 January 1900, as returned by ASKTIME. Leading zeros may be omitted. |
| YYYYMMDD, DATE | Year, month, day. |
| MMDDYYYY | Month, day, year. |
| DDMMYYYY | Day, month, year. |
| MMDDYY, DDMMYY | As above with a two-digit year. |
| YYYYDDD, YYDDD | Year and day of the year (three digits). |
| DATESEP(c) | The separator placed between the parts of every date option. Without DATESEP(c) there is no separator; an empty value (DATESEP('')) gives /. |
| TIME | Hours, minutes, seconds. |
| TIMESEP(c) | The separator for TIME. Without TIMESEP(c) there is none; an empty value gives :. |
| YEAR | The year, four digits. |
| MONTHOFYEAR | The month, two digits (01-12). |
| DAYOFMONTH | The day, two digits (01-31). |
| DAYOFWEEK | 0 (Sunday) through 6 (Saturday). |
| DAYCOUNT | Days since 1 January 1900. |
| TIMEZONE(name) | Convert in this time zone instead of the region's: an IANA name such as Europe/Paris, or UTC. |

**Figure 13-5. FORMATTIME options**

Any combination of output options may be coded; an output option whose argument is a literal is skipped. The conversion uses the time zone of time_zone in bricks.cnf unless TIMEZONE is coded.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The values were returned. |
| INVREQ | 16 | ABSTIME is missing, blank or not a number; or TIMEZONE names an unknown zone. |

**Figure 13-6. FORMATTIME conditions**

### *Example*

```
EXEC CICS ASKTIME ABSTIME(WS-NOW) END-EXEC.
EXEC CICS FORMATTIME ABSTIME(WS-NOW)
          DDMMYYYY(WS-DATE) DATESEP('/')
          TIME(WS-TIME) TIMESEP(':')
          DAYOFWEEK(WS-DOW) END-EXEC.
*  WS-DATE = "12/05/2026"  WS-TIME = "10:30:45"  WS-DOW = "2"
```

### *Differences from IBM CICS*

DATEFORM is not supported: DATE always returns year, month and day, in that order. Use the option of the format wanted (DDMMYYYY, MMDDYYYY, ...) instead. DATESTRING, STRINGFORMAT, MILLISECONDS and FULLDATE are not supported. TIMEZONE takes a zone name. DATESEP and TIMESEP coded without parentheses give no separator; code the separator character explicitly.

## DELAY

### *Function*

Suspends the task for an interval or until a time of day.

### *Syntax*

```
EXEC CICS DELAY
     [INTERVAL(hhmmss) | TIME(hhmmss)
      | [FOR] [HOURS(n)] [MINUTES(n)] [SECONDS(n)] [MILLISECS(n)]
      | UNTIL [HOURS(n)] [MINUTES(n)] [SECONDS(n)] [MILLISECS(n)]]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| INTERVAL(hhmmss) | Wait this long: one to six digits read as hhmmss, as for "START" in Chapter 12. Minutes and seconds must be 0 through 59. |
| TIME(hhmmss) | Wait until this time of day; a time that has passed means the same time tomorrow. |
| FOR HOURS MINUTES SECONDS MILLISECS | Wait for the sum of the components. FOR may be omitted. |
| UNTIL HOURS MINUTES SECONDS MILLISECS | Wait until the time of day given by the sum of the components, rolling to tomorrow when it has passed. |

**Figure 13-7. DELAY options**

### *Description*

The task waits, then continues. A DELAY without a time, or with components that are all zero, returns at once. The time spent waiting does not count toward the runaway-task limit. A START that becomes due during the DELAY waits until the task next waits for the operator or ends ("START" in Chapter 12).

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The delay has expired. |
| INVREQ | 16 | The value of INTERVAL or TIME is not a valid hhmmss. |

**Figure 13-8. DELAY conditions**

### *Example*

```
EXEC CICS DELAY FOR SECONDS(5) END-EXEC
EXEC CICS DELAY INTERVAL(000130) END-EXEC      /* 1 min 30 s */
EXEC CICS DELAY FOR MILLISECS(250) END-EXEC
```

### *Differences from IBM CICS*

A component of FOR or UNTIL that is not a non-negative number counts as zero; no INVREQ is raised for it, and the component ranges are not checked. REQID is not supported, so a DELAY cannot be canceled.

## ADDRESS

### *Function*

Obtains the address of a storage area owned by CICS: the COMMAREA, the EIB, the Common Work Area or the Transaction Work Area.

### *Syntax*

```
EXEC CICS ADDRESS
     [COMMAREA(ptr-ref)] [EIB(ptr-ref)]
     [CWA(ptr-ref)] [TWA(ptr-ref)]
     [TCTUA(ptr-ref)] [ACEE(ptr-ref)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Area addressed |
|---|---|
| COMMAREA | DFHCOMMAREA. Never null; test EIBCALEN to learn whether a COMMAREA was passed. |
| EIB | The EIB (DFHEIB). Never null. |
| CWA | The Common Work Area: one area for the whole region, of the size set by wrkarea in bricks.cnf (0 through 3584 bytes). Null when wrkarea is 0, the default. Its length is returned by ASSIGN CWALENG. |
| TWA | The Transaction Work Area: a new area, set to binary zeros, for each task, of the TWASIZE of the transaction. Null when TWASIZE is 0. Its length is returned by ASSIGN TWALENG. |
| TCTUA, ACEE | Always null. |

**Figure 13-9. ADDRESS options**

### *Description*

In COBOL, each *ptr-ref* is a `USAGE POINTER` item. `ADDRESS` stores in it a handle for the area, and the program maps a LINKAGE SECTION item onto the area with `SET ADDRESS OF item TO ptr` (see "SET (Pointers)" in Chapter 24). The handle is not a machine address and cannot be displayed meaningfully, but `SET ptr UP BY n` and `DOWN BY n` move it within the area it addresses.

The CWA is shared by every task of the region, and bricks does not serialize access to it (nor does IBM CICS). Protect updates with `ENQ` and `DEQ`.

In REXX, which has no pointers, `ADDRESS` sets every *ptr-ref* to 0 and ends normally; the COMMAREA and EIB fields are already available as variables. The PL/I compiler rejects `ADDRESS`.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The handles were stored. A null handle for an area that does not exist is a normal result. |

**Figure 13-10. ADDRESS conditions**

Any other option, such as `CSA` or `TCTTE`, is refused with `INVREQ` and a message that names the option; the refusal is not a condition and ends the task.

### *Example*

```
01 WS-CPTR  USAGE POINTER.
...
EXEC CICS ADDRESS COMMAREA(WS-CPTR) END-EXEC.
SET ADDRESS OF DFHCOMMAREA TO WS-CPTR.
```

`runtime/cobol/gmpt.cob` (transaction `GMPT`) uses `ADDRESS COMMAREA` and `ADDRESS EIB` together with `GETMAIN`.

### *Differences from IBM CICS*

`TCTUA` and `ACEE` are always null; a program that uses either gets a null-pointer error, not data. The handles are not machine addresses. `ADDRESS` is not available in PL/I.

## GETMAIN and FREEMAIN

### *Function*

GETMAIN obtains storage for the task and returns its address; FREEMAIN releases it.

### *Syntax*

```
EXEC CICS GETMAIN SET(ptr-ref)
     {FLENGTH(data-value) | LENGTH(data-value)}
     [INITIMG(data-value)] [SHARED] [NOSUSPEND]
     [USERDATAKEY | CICSDATAKEY]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC

EXEC CICS FREEMAIN {DATA(data-area) | DATAPOINTER(ptr-value)}
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| SET(ptr-ref) | Required. Receives the address of the storage. |
| FLENGTH(n), LENGTH(n) | Required. The number of bytes. FLENGTH is used if both are coded. |
| INITIMG(c) | Set every byte to the first byte of the value. Default X'00'. |
| SHARED | Accepted; the storage is still released at the end of the task. |
| NOSUSPEND, USERDATAKEY, CICSDATAKEY | Accepted; no effect. |
| DATAPOINTER(ptr) | FREEMAIN: a pointer that holds the address obtained by GETMAIN. |
| DATA(area) | FREEMAIN: a based data area whose current address is the storage obtained by GETMAIN. |

**Figure 13-11. GETMAIN and FREEMAIN options**

### *Description*

In COBOL, the program maps a LINKAGE SECTION item onto the storage with `SET ADDRESS OF` and reads and writes it through that item. `SET ptr UP BY n` and `DOWN BY n` move a pointer within one area; they do not reach into another. All storage that the task has not released is released when the task ends.

In REXX both commands are accepted for compatibility: `GETMAIN` checks the length and stores a dummy non-zero value in `SET`, and `FREEMAIN` ends normally. The PL/I compiler rejects both.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The storage was obtained or released. |
| LENGERR | 22 | GETMAIN: the length is zero or negative. |
| NOSTG | 42 | GETMAIN: the length exceeds 16 MiB. No storage is obtained. The default action is to continue. |
| INVREQ | 16 | GETMAIN: SET missing, or the length missing or not a number. FREEMAIN: no operand, or the address was not obtained by GETMAIN (including a null address). |

**Figure 13-12. GETMAIN and FREEMAIN conditions**

### *Example*

```
01 WS-PTR  USAGE POINTER.
LINKAGE SECTION.
01 GM-AREA.
   05 GM-TEXT PIC X(16).
PROCEDURE DIVISION.
    EXEC CICS GETMAIN SET(WS-PTR) FLENGTH(16) INITIMG(' ') END-EXEC.
    SET ADDRESS OF GM-AREA TO WS-PTR.
    MOVE 'POINTER-OK' TO GM-TEXT.
    EXEC CICS FREEMAIN DATAPOINTER(WS-PTR) END-EXEC.
```

### *Differences from IBM CICS*

`SHARED` storage does not outlive the task. Use a temporary storage queue or the CWA for data that must be shared between tasks. `GETMAIN` and `FREEMAIN` are not available in PL/I.

## ENQ and DEQ

### *Function*

ENQ obtains exclusive use of a named resource; DEQ releases it.

### *Syntax*

```
EXEC CICS ENQ RESOURCE(data-area) [LENGTH(data-value)]
     [HOLD] [NOSUSPEND]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC

EXEC CICS DEQ RESOURCE(data-area) [LENGTH(data-value)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| RESOURCE(name) | Required. The name of the resource. Leading and trailing blanks are removed and the name is folded to upper case; the result is the key. |
| LENGTH(n) | Accepted; the whole name is used. |
| HOLD | Keep the resource across SYNCPOINT, until DEQ or the end of the task. |
| NOSUSPEND | If another task holds the resource, return ENQBUSY at once instead of waiting. |

**Figure 13-13. ENQ and DEQ options**

### *Description*

Resources are known throughout the region. An ENQ for a resource that another task holds waits until it is released, unless NOSUSPEND is coded. An ENQ by the task that already holds the resource ends normally. A resource obtained without HOLD is released at the next SYNCPOINT or SYNCPOINT ROLLBACK; one obtained with HOLD is kept until DEQ. Every resource still held is released when the task ends, normally or not.

A resource belongs to the task, not to the program: it is kept across LINK and XCTL, and any program of the task can release it. DEQ of a resource the task does not hold ends normally. A task started in the background holds its own resources, separate from those of the terminal's main task.

An ENQ that has waited five minutes returns ENQBUSY instead of waiting longer. The time spent waiting does not count toward the runaway-task limit.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The resource was obtained (or released). |
| ENQBUSY | 55 | ENQ: another task holds the resource and NOSUSPEND was coded, or the wait reached five minutes. The default action is to continue. |
| INVREQ | 16 | RESOURCE is missing or blank. |

**Figure 13-14. ENQ and DEQ conditions**

### *Example*

```
EXEC CICS ENQ RESOURCE('CWA-COUNTER') END-EXEC.
ADD 1 TO CWA-COUNT.
EXEC CICS DEQ RESOURCE('CWA-COUNTER') END-EXEC.
```

CEMT INQUIRE ENQ shows every resource held and every task waiting for one, with the owning task, whether HOLD was coded, and for how long ("INQUIRE" in Chapter 5).

### *Differences from IBM CICS*

LENGTH is ignored and the name is folded to upper case, so the resource is always identified by its text. The five-minute limit on waiting is a bricks safeguard. ENQSCOPE, LUW, TASK and UOW are not supported.

## QUERY SECURITY

### *Function*

Asks what access the signed-on user has to a transaction, so that a program can, for example, hide a menu choice the user may not use.

### *Syntax*

```
EXEC CICS QUERY SECURITY RESOURCE(data-value)
     [RESCLASS(data-value)]
     [READ(cvda)] [UPDATE(cvda)] [CONTROL(cvda)] [ALTER(cvda)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| RESOURCE(name) | Required. The TRANSID to check. |
| RESCLASS(name) | TCICSTRN (the default) or PCICSPSB. Both check the transaction definition, which in bricks is also the program definition. |
| READ(v) | Receives READABLE (50) or NOTREADABLE (51). |
| UPDATE(v) | Receives UPDATABLE (52) or NOTUPDATABLE (53). |
| CONTROL(v) | Receives CTRLABLE (54) or NOTCTRLABLE (55). |
| ALTER(v) | Receives ALTERABLE (56) or NOTALTERABLE (57). |

**Figure 13-15. QUERY SECURITY options**

### *Description*

bricks' access control is a single yes or no for each transaction ("Security Groups" in Chapter 1), so the four kinds of access are derived as follows:

| Access | Granted when |
|---|---|
| READ | The user may run the transaction: the same rule the dispatcher applies (an empty group list, PUBLIC, *, or a group the user belongs to). |
| UPDATE, CONTROL, ALTER | The user belongs to the ADMIN group. |

**Figure 13-16. How QUERY SECURITY derives access**

The command answers for the signed-on user. On a terminal that is not signed on, every kind of access is denied, even for a PUBLIC transaction. A transaction that is not defined also gives "denied" on every axis, with response NORMAL; there is no NOTFND. The command changes nothing and writes nothing to the log. An output option whose argument is a literal is skipped.

The values are declared in the copybook DFHVALUE, each under two names: UPDATABLE and DFHVALUE-UPDATABLE, and so on. They are declared PIC S9(8) (display). Declare the receiving fields the same way: a COMP field compared with a display constant never matches.

***Conditions***

| Condition | RESP | RESP2 | Meaning |
|---|---|---|---|
| NORMAL | 0 | 0 | The values were returned. |
| INVREQ | 16 | 1 | RESOURCE is missing or blank. |
| INVREQ | 16 | 2 | RESCLASS is neither TCICSTRN nor PCICSPSB. |
| INVREQ | 16 | 3 | The region has no security checker (a configuration error). |

**Figure 13-17. QUERY SECURITY conditions**

***Examples***

```
COPY DFHVALUE.
01 WS-UPD PIC S9(8) VALUE 0.
...
IF ACTION = 'D'
    EXEC CICS QUERY SECURITY RESOURCE('GUST') UPDATE(WS-UPD) END-EXEC
    IF WS-UPD NOT = DFHVALUE-UPDATABLE
        MOVE 'Delete not permitted for this user.' TO MSG
        MOVE 'N' TO VALID-ACTION
    END-IF
END-IF.
```

```
EXEC CICS QUERY SECURITY RESOURCE('GUST') READ(RD) UPDATE(UP) END-EXEC
SHOW_DELETE = (UP = 52)
```

The supplied program runtime/cobol/gust.cob guards its Delete action this way.

***Differences from IBM CICS***

The four kinds of access are derived from the transaction's group list and membership of ADMIN, not from an external security manager. The administrator group is always named ADMIN. Only the transaction classes are supported; RESTYPE, LOGMESSAGE and USERID are not.

## VERIFY PASSWORD

### *Function*

Checks a password against the user's definition, for example to ask the operator to type the password again before a sensitive action.

### *Syntax*

```
EXEC CICS VERIFY PASSWORD(data-value) USERID(data-value)
     [CHANGETIME(data-area)] [EXPIRYTIME(data-area)]
     [DAYSLEFT(data-area)] [INVALIDCOUNT(data-area)]
     [LASTUSETIME(data-area)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| `USERID(name)` | Required. The user. Surrounding blanks are removed; the userid is compared without regard to case. |
| `PASSWORD(value)` | The password, compared exactly: case and blanks count. A missing or empty password is wrong. |
| `CHANGETIME, EXPIRYTIME, DAYSLEFT, INVALIDCOUNT, LASTUSETIME` | Accepted. bricks keeps no password history, so the variables are not changed. |

**Figure 13-18. VERIFY PASSWORD options**

### *Description*

The password is checked against the definition in `users.conf`, as at sign-on, as UTF-8. Every kind of failure (unknown userid, wrong password, empty password) gives the same response, `NOTAUTH` with RESP2 0, and takes the same time, so that the command cannot be used to discover which userids exist. The command changes nothing. CECI refuses to run it, because the password would be shown on its screen; a program that needs the check runs as a transaction of its own and reads the password from a field with the `HIDDEN` attribute.

***Conditions***

| Condition | RESP | RESP2 | Meaning |
|---|---|---|---|
| NORMAL | 0 | 0 | The password is correct. |
| NOTAUTH | 70 | 0 | The userid or password is wrong. In COBOL and PL/I code RESP; an unhandled NOTAUTH is abend AEY7. |
| INVREQ | 16 | 1 | USERID is missing or blank. |
| INVREQ | 16 | 3 | The region has no password verifier (a configuration error). |
| IOERR | 17 | 0 | The verifier failed (for example, the user file could not be read). |

**Figure 13-19. VERIFY PASSWORD conditions**

***Example***

```
EXEC CICS ASSIGN USERID(WS-USR) END-EXEC.
EXEC CICS VERIFY PASSWORD(WS-PWD) USERID(WS-USR)
          RESP(WS-RC) END-EXEC.
EVALUATE WS-RC
  WHEN DFHRESP(NORMAL)  EXEC CICS LINK PROGRAM('ADMN') END-EXEC
  WHEN DFHRESP(NOTAUTH) MOVE 'Password does not match.' TO MSG
  WHEN OTHER            MOVE 'Password check failed.' TO MSG
END-EVALUATE.
```

The supplied transaction CSSR (`runtime/rexx/cssr.rexx`) asks for the password on the CSSRPW map, whose password field is hidden, and verifies it.

***Differences from IBM CICS***

IBM CICS distinguishes failures by RESP2; bricks deliberately does not. The password-history options are not set.

## INQUIRE SYSTEM

***Function***

Returns region settings. bricks supports one: the sign-on greeting.

***Syntax***

```
EXEC CICS INQUIRE SYSTEM
     [GMMTEXT(data-area)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| GMMTEXT(data-area) | Receives the greeting set by gmtext in bricks.cnf (up to 256 characters; default Welcome to bricks), in full. |

**Figure 13-20. INQUIRE SYSTEM options**

### *Conditions*

| Condition | RESP | RESP2 | Meaning |
|---|---|---|---|
| NORMAL | 0 | 0 | The value was returned. INQUIRE SYSTEM with no option is also normal. |
| INVREQ | 16 | 1 | An option other than GMMTEXT; the message names it. |
| INVREQ | 16 | 2 | The region has no system information provider (a configuration error). |

**Figure 13-21. INQUIRE SYSTEM conditions**

### *Example*

```
MSG = ''
EXEC CICS INQUIRE SYSTEM GMMTEXT(MSG) RESP(RC) END-EXEC
IF RC \= 0 THEN MSG = 'INQUIRE SYSTEM failed, RESP2=' EIBRESP2
EXEC CICS SEND TEXT FROM(MSG) ERASE END-EXEC
```

This is the supplied transaction GMTQ (runtime/rexx/gmtq.rexx).

### *Differences from IBM CICS*

Only GMMTEXT is supported. Any other option is refused, so that a program that depends on it fails visibly instead of receiving an empty value.

## INQUIRE and SET for Resources

### *Function*

Returns the attributes of a file, task, transaction, terminal or program, one by name or all in turn.

***Syntax***

```
EXEC CICS INQUIRE class(name) attribute(data-area) ...
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC

EXEC CICS STARTBROWSE class END-EXEC
EXEC CICS INQUIRE class(data-area) NEXT attribute(data-area) ...
     [RESP(data-area)] END-EXEC
EXEC CICS ENDBROWSE class END-EXEC

EXEC CICS SET class(name) [attribute(data-value) ...]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

*class* is `FILE`, `TASK`, `TRANSACTION`, `TERMINAL` or `PROGRAM`.

***Description***

The **direct** form returns the attributes of the resource named. The **browse** form returns every resource of the class in turn: `STARTBROWSE` takes a list of the names, in alphabetical order; each `INQUIRE ... NEXT` stores the next name in the class option's variable and the resource's attributes in the attribute variables; after the last resource it returns `END`. `ENDBROWSE` ends the browse. A task can have one browse open for each class; a second `STARTBROWSE` for the class starts again from the beginning. Names are compared without regard to case. A task is named by its number (`EIBTASKN`); a program, as always in bricks, by its TRANSID.

bricks keeps no enabled, open or in-service state for these resources. The status attributes therefore return IBM's normal values; the others return real values. `SET` checks that the resource exists and ends normally, but changes nothing.

| Class | Real values | Fixed values |
|---|---|---|
| FILE | RECORDSIZE, KEYLENGTH, BASEDSNAME (the file name) | ENABLESTATUS ENABLED, OPENSTATUS OPEN, KEYPOSITION 0, RECORDFORMAT VARIABLE, ACCESSMETHOD VSAM |
| TASK | TRANSACTION, FACILITY (the terminal), USERID | RUNSTATUS RUNNING, PRIORITY 1 |
| TRANSACTION | PROGRAM (the source file), TWASIZE | STATUS ENABLED, PRIORITY 1, PROFILE DFHCICST |
| TERMINAL | USERID, NETNAME (the terminal identifier) | ACQSTATUS ACQUIRED, SERVSTATUS INSERVICE |
| PROGRAM | LANGUAGE (COBOL, REXX or PLI), USECOUNT | STATUS ENABLED, PROGTYPE PROGRAM, RESCOUNT 0, LENGTH 0 |

**Figure 13-22. Resource attributes returned by INQUIRE**

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NORMAL | 0 | The command succeeded. |
| END | 83 | INQUIRE ... NEXT after the last resource. In COBOL and PL/I code RESP: an unhandled END is abend AEXK. |
| NOTFND | 13 | The file, task or terminal named does not exist. |
| PGMIDERR | 27 | The program named does not exist (RESP2 1). |
| TRANSIDERR | 28 | The transaction named does not exist (RESP2 1). |
| ILLOGIC | 21 | INQUIRE ... NEXT without a STARTBROWSE for the class. |
| INVREQ | 16 | An unknown class; a direct INQUIRE or a SET without a name; ENDBROWSE with no browse open. |

**Figure 13-23. INQUIRE and SET conditions**

An attribute that the class does not have is refused with INVREQ and a message naming it; the refusal is not a condition and ends the task.

***Example***

```
EXEC CICS ASSIGN EIBTASKN(MYTASK) END-EXEC
EXEC CICS INQUIRE TASK(MYTASK) TRANSACTION(TR) RESP(RC) END-EXEC

EXEC CICS STARTBROWSE FILE END-EXEC
RC = 0
DO WHILE RC = 0
  FN = ''
  EXEC CICS INQUIRE FILE(FN) NEXT RECORDSIZE(RZ) KEYLENGTH(KL) RESP(RC) END-EXEC
  IF RC = 0 THEN SAY FN RZ KL
END
EXEC CICS ENDBROWSE FILE END-EXEC
```

The supplied transaction INQR (`runtime/rexx/inqr.rexx`) runs this sequence and shows the result.

***Differences from IBM CICS***

- Status attributes return fixed values; a program cannot use INQUIRE to learn that a resource is closed or disabled, because bricks has no such state.
- SET changes nothing. It does not pretend to make a change that a following INQUIRE would contradict.
- Only the five classes and the attributes in the table above are supported.

## WRITE OPERATOR

***Function***

Writes a message to the bricks console log.

***Syntax***

```
EXEC CICS WRITE OPERATOR TEXT(data-value) [LENGTH(data-value)]
     [ROUTECODES(data-value) NUMROUTES(data-value)]
     [EVENTUAL | ACTION | CRITICAL | IMMEDIATE]
     [CONSNAME(data-value)]
     [RESP(data-area)] [RESP2(data-area)] [NOHANDLE]
END-EXEC
```

### *Options*

| Option | Description |
|---|---|
| `TEXT(text)` | Required. The message. |
| `LENGTH(n)` | Write only the first *n* characters. |
| `ROUTECODES, NUMROUTES, CONSNAME, EVENTUAL, ACTION, CRITICAL, IMMEDIATE` | Accepted. There is one console; every message goes to it. |

**Figure 13-24. WRITE OPERATOR options**

### *Description*

The message is written to the console log with the `SYS` tag, preceded by the terminal identifier of the task, so the operator can tell which task wrote it: `operator (T0003): text`.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| `NORMAL` | `0` | The message was written. |
| `INVREQ` | 16 | `TEXT` is missing. |

**Figure 13-25. WRITE OPERATOR conditions**

The `REPLY` option is refused with `INVREQ`; the refusal is not a condition and ends the task. bricks has no console input from which a reply could come, and waiting for one would hang the task. To obtain an operator's answer, send a map to a terminal.

### *Example*

```
EXEC CICS WRITE OPERATOR TEXT('Nightly report complete') END-EXEC
```

Chapter 14

# File Control Commands

This chapter describes the file control commands. They read, add, replace and delete single records of a key-sequenced file, browse a file in key order, and browse the sequential files of the `tmp_dir` directory by relative byte address. The chapter also describes the record locks that file control takes and how long they are held.

### Key-Sequenced Files in bricks

A bricks file corresponds to a VSAM key-sequenced data set (KSDS). Each file is a B+tree bucket in the region's embedded database, `data/files.boltdb`. A record is identified by its key (`RIDFLD`); the record itself is an opaque string of bytes whose layout the application chooses. The supplied samples store fields separated by the character `|`, but bricks attaches no meaning to the content.

A file need not be defined before it is used. The first `WRITE` to a file name that does not exist creates the file. A file can also be defined, loaded, listed, printed and deleted with the IDCA transaction or the offline `idcams` command; see Chapter 10. A file created by `WRITE` and a file defined by IDCAMS are identical.

File names are 1 through 64 characters long and consist of the letters `A-Z` and `a-z`, the digits, the underscore and the hyphen. Names are folded to uppercase, so `FILE('customers')` and `FILE('CUSTOMERS')` name the same file. A name that breaks these rules gives `INVREQ`.

Each successful `WRITE`, `REWRITE` and `DELETE` is written to the database at once, and an undo entry is added to the task's unit of work. The change can therefore be backed out until the next syncpoint. See "Units of Work" in Chapter 16 for the unit of work.

## Summary of the Commands

| Command | Function |
|---|---|
| READ | Read a record by key, optionally for update. |
| WRITE | Add a new record. |
| REWRITE | Replace the record read for update. |
| DELETE | Delete one record, or every record with a generic key. |
| UNLOCK | Release the record read for update without changing it. |
| STARTBR | Start a browse at a key. |
| READNEXT | Read the next record of a browse. |
| READPREV | Read the previous record of a browse. |
| RESETBR | Reposition a browse. |
| ENDBR | End a browse. |

**Figure 14-1. File control commands**

## READ

### *Function*

Read a record from a file by its key. With UPDATE, the record is also locked so that the program can change it with REWRITE or DELETE.

### *Syntax*

```
EXEC CICS READ FILE(name)
          {INTO(area) | SET(area)}
          RIDFLD(key)
          [UPDATE]
          [LENGTH(len)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**`FILE(name)`**

The file to read. Required.

**`INTO(area)` or `SET(area)`**

The variable that receives the record. One of the two is required; bricks treats them alike.

**`RIDFLD(key)`**

The key of the record. Required; an empty key gives `INVREQ`.

**`UPDATE`**

Lock the record for update. The lock is taken before the record is returned. See "Record Locks".

**`LENGTH(len)`**

When *len* is a variable, it receives the length of the record. The value it holds on input is not used.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | No record has this key, or the file does not exist. |
| INVREQ | 16 | A required option is missing, the key is empty, or the file name is not valid. |
| IOERR | 17 | The database reported an error. |
| ENQBUSY | 55 | UPDATE: another task holds the lock on this record. |

**Figure 14-2. READ conditions**

### *Examples*

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
IF RC = 13 THEN MSG = 'Customer not found'
```

### *Notes*

The REXX example tests `RC` after the command. A COBOL or PL/I program that tests the response must code `RESP`; without it, `NOTFND` abends the task `AEIM` before the test runs (see "Unhandled Conditions" in Chapter 16).

***Differences from IBM CICS***

- `ENQBUSY` is returned at once when the record is locked by another task. IBM CICS suspends the task until the lock is released.
- `LENGTH` is an output only. In IBM CICS it also gives the length of the receiving area on input.

## WRITE

***Function***

Add a new record to a file. The file is created if it does not yet exist. The new record stays locked until the unit of work ends.

***Syntax***

```
EXEC CICS WRITE FILE(name)
          FROM(data)
          RIDFLD(key)
          [LENGTH(n)]
          [MASSINSERT]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

***Options***

**`FILE(name)`**

The file. Required. The file is created on its first `WRITE`.

**`FROM(data)`**

The record to write. Required.

**`RIDFLD(key)`**

The key of the new record. Required.

**`LENGTH(n)`**

The maximum number of bytes of `FROM` to write. A longer value is truncated to *n* bytes.

**`MASSINSERT`**

Accepted for compatibility; it has no effect.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| DUPREC | 14 | A record with this key already exists. |
| INVREQ | 16 | A required option is missing, the file name is not valid, or the name is a sequential file in `tmp_dir` (use `WRITEQ TD`). |
| IOERR | 17 | The database reported an error. |
| ENQBUSY | 55 | Another task holds the lock on this key: it deleted the record and has not yet reached a syncpoint. |

**Figure 14-3. WRITE conditions**

### *Examples*

```
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS WRITE FILE('CUSTOMERS') FROM(REC) RIDFLD(CKEY) END-EXEC
```

### *Notes*

The record is written to the database before the command returns. A later `SYNCPOINT ROLLBACK` or an abend that is not handled deletes it again.

## REWRITE

### *Function*

Replace the record that the most recent `READ ... UPDATE` of the same file returned.

### *Syntax*

```
EXEC CICS REWRITE FILE(name)
          FROM(data)
          [LENGTH(n)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**`FILE(name)`**

The file. Required. The key is the one the `READ ... UPDATE` named; `REWRITE` takes no `RIDFLD`.

**`FROM(data)`**

The new content of the record. Required.

**`LENGTH(n)`**

The maximum number of bytes of `FROM` to write.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The record no longer exists. |
| INVREQ | 16 | No `READ ... UPDATE` of this file is outstanding (none was issued, or it was ended by an earlier `REWRITE`, `DELETE` or `UNLOCK`), `FROM` is missing, or the name is a sequential file in `tmp_dir`. |
| IOERR | 17 | The database reported an error. |

**Figure 14-4. REWRITE conditions**

### *Examples*

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) UPDATE END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
PH = NEWPHONE
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS REWRITE FILE('CUSTOMERS') FROM(REC) END-EXEC
```

### *Notes*

`REWRITE` ends the `READ ... UPDATE`: a second `REWRITE` needs a new `READ ... UPDATE`. It does not release the record lock. The changed record stays locked until the unit of work ends (see "Record Locks").

A `READ ... UPDATE` followed by `REWRITE` may span a terminal conversation (`SEND MAP` and `RECEIVE MAP` in between); the lock is held across the operator's think time.

CECI ends each press of **F5** with an implicit syncpoint, so a `READ ... UPDATE` and a `REWRITE` entered on two separate presses do not form a pair. See "Units of Work in CECI" in Chapter 7.

## DELETE

### *Function*

Delete a record from a file: the record with a given key, the record read for update, or every record whose key begins with a generic key.

### *Syntax*

```
EXEC CICS DELETE FILE(name)
          [RIDFLD(key) [KEYLENGTH(n) GENERIC [NUMREC(count)]]]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**`FILE(name)`**

The file. Required.

**`RIDFLD(key)`**

The key of the record to delete. When `RIDFLD` is omitted, the record of the outstanding `READ ... UPDATE` of this file is deleted.

**`KEYLENGTH(n) GENERIC`**

Delete every record whose key begins with the first *n* bytes of `RIDFLD`. *n* must be a positive integer no greater than the length of `RIDFLD`.

**`NUMREC(count)`**

With `GENERIC`, receives the number of records deleted.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | No record has this key; with GENERIC, no record matches. |
| INVREQ | 16 | No RIDFLD and no outstanding READ ... UPDATE; GENERIC without a valid KEYLENGTH; the file name is not valid; or the name is a sequential file in tmp_dir (use DELETEQ TD). |
| IOERR | 17 | The database reported an error. |
| ENQBUSY | 55 | RIDFLD: another task holds the lock on this record. |

**Figure 14-5. DELETE conditions**

### *Examples*

```
EXEC CICS DELETE FILE('CUSTOMERS') RIDFLD(CKEY) END-EXEC

EXEC CICS DELETE FILE('ORDERS') RIDFLD('NY-') KEYLENGTH(3)
          GENERIC NUMREC(N) END-EXEC
```

### *Notes*

A deleted record stays locked until the unit of work ends, because a backout re-creates it. A DELETE ... GENERIC takes no record locks.

## UNLOCK

### *Function*

End a READ ... UPDATE without changing the record, and release its lock.

### *Syntax*

```
EXEC CICS UNLOCK FILE(name)
          [TOKEN(token)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**FILE(name)**

The file. Required.

**TOKEN(token)**

Accepted for compatibility; it has no effect.

***Conditions***

`INVREQ` (16) when `FILE` is missing. An `UNLOCK` with no outstanding `READ ... UPDATE` is `NORMAL`.

***Notes***

`UNLOCK` does not release a record that this unit of work has already changed. On a sequential file in `tmp_dir` it does nothing and returns `NORMAL`.

## Record Locks

File control locks individual records, not files. A lock belongs to a task and to a record (file and key). It is taken as follows:

- `READ ... UPDATE` locks the record it reads.
- `WRITE` locks the key of the new record.
- `DELETE` with `RIDFLD` locks the record it deletes.

A lock held by another task makes `READ ... UPDATE`, `WRITE` and `DELETE ... RIDFLD` of the same record return `ENQBUSY` (55) at once. A `READ` without `UPDATE` is never blocked: it reads the record as it is in the file, including changes another task has not yet committed.

***How long a lock is held***

A lock taken by `READ ... UPDATE` for a record that the task does not change is released by the first of these:

- `UNLOCK` of the file;
- a `READ ... UPDATE` of another key in the same file (the task remembers one update record per file);
- the end of the task.

A record that the task has changed with `WRITE`, `REWRITE` or `DELETE` stays locked until the unit of work ends: at `SYNCPOINT`, `SYNCPOINT ROLLBACK`, the backout of an abend, or the end of the task. `REWRITE`, `DELETE` and `UNLOCK` do not release it. This is what makes a backout safe: no other task can change a record whose before-image the backout may still restore.

The locks belong to the task, not to the program. They are kept across `LINK` and carried across `XCTL`; see "Locks and the Unit of Work" in Chapter 16.

| Lock taken by | Released by |
|---|---|
| READ ... UPDATE, record not changed | UNLOCK; a READ ... UPDATE of another key in the same file; task end |
| READ ... UPDATE then REWRITE or DELETE | End of the unit of work |
| WRITE | End of the unit of work |
| DELETE ... RIDFLD | End of the unit of work |
| DELETE ... GENERIC | No lock is taken |

**Figure 14-6. How long a record lock is held**

***Differences from IBM CICS***

- IBM CICS suspends a task that asks for a locked record; bricks returns ENQBUSY instead, so two tasks can never deadlock on records (there is no abend AFCF). ENQBUSY never abends, so a program can test it and retry in every language.
- All bricks files are recoverable. IBM CICS holds a changed record to the syncpoint only for a recoverable file.

## Browsing a File

A browse reads the records of a file in key order, forward with READNEXT or backward with READPREV. It is started by STARTBR, can be repositioned by RESETBR, and is ended by ENDBR.

A browse sees the file as it was at STARTBR: records that other tasks add, change or delete after that point are not seen until the next STARTBR. A browse takes no record locks.

A browse belongs to the task. A program may have browses open on several files at once. To have more than one browse of the same file, give each a different request identifier with REQID and name it on every command of that browse. A second STARTBR with the same file and REQID replaces the first browse. Every browse still open at the end of the task is ended.

A browse is not ended by SYNCPOINT; see the differences under "ENDBR".

## STARTBR

### *Function*

Start a browse of a file at a given key.

### *Syntax*

```
EXEC CICS STARTBR FILE(name)
          [RIDFLD(key)]
          [GTEQ | EQUAL]
          [GENERIC KEYLENGTH(n)]
          [REQID(id)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**`RIDFLD(key)`**

The starting key. Without it, the browse starts at the first record (or, for `READPREV`, the last).

**`GTEQ`**

Start at the first record whose key is equal to or greater than `RIDFLD`. This is the default.

**`EQUAL`**

Start only at a record whose key equals `RIDFLD`.

**`GENERIC KEYLENGTH(n)`**

Browse only the records whose key begins with the first *n* bytes of `RIDFLD`. `GENERIC` without `KEYLENGTH` has no effect.

**`REQID(id)`**

Identifies this browse among several of the same file.

### *Conditions*

`INVREQ` (16) when `FILE` is missing or not valid; `IOERR` (17) when the database reports an error.

### *Examples*

```
EXEC CICS STARTBR FILE('CUSTOMERS') RIDFLD('NY-')
          GENERIC KEYLENGTH(3) END-EXEC
```

### *Notes*

`STARTBR` positions the browse but reads nothing. When no record qualifies (an `EQUAL` key that is not on file, a generic key that matches nothing, a file that does not exist) `STARTBR` still returns `NORMAL`, and the first `READNEXT` or `READPREV` returns `ENDFILE`. IBM CICS returns `NOTFND` at the `STARTBR`.

## READNEXT

### *Function*

Read the next record of a browse.

### *Syntax*

```
EXEC CICS READNEXT FILE(name)
          {INTO(area) | SET(area)}
          [RIDFLD(key)]
          [LENGTH(len)]
          [REQID(id)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**`INTO(area)` or `SET(area)`**

Receives the record.

**`RIDFLD(key)`**

When *key* is a variable, it receives the key of the record returned.

**`LENGTH(len)`**

When *len* is a variable, it receives the length of the record. On a file its input value is not used; on a sequential file in `tmp_dir` it is the size of the receiving area (see "LENGTH on sequential files").

**`REQID(id)`**

The browse, when `STARTBR` named one.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| ENDFILE | 20 | There is no further record (or no further record with the generic key). |
| INVREQ | 16 | No browse of this file (and REQID) is open. |
| LENGERR | 22 | Sequential file only: the record was longer than LENGTH and was truncated. |
| IOERR | 17 | The database reported an error. |

**Figure 14-7. READNEXT and READPREV conditions**

***Examples***

```
EXEC CICS STARTBR FILE('CUSTOMERS') END-EXEC
DO FOREVER
  EXEC CICS READNEXT FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
  IF RC = 20 THEN LEAVE
  SAY K ':' REC
END
EXEC CICS ENDBR FILE('CUSTOMERS') END-EXEC
```

***Notes***

A COBOL or PL/I browse loop must code RESP on READNEXT and test for ENDFILE; otherwise the end of the file abends the task AEIT.

## READPREV

***Function***

Read the previous record of a browse.

***Syntax***

```
EXEC CICS READPREV FILE(name)
          {INTO(area) | SET(area)}
          [RIDFLD(key)]
          [LENGTH(len)]
          [REQID(id)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

The options and conditions are those of READNEXT; ENDFILE means there is no earlier record. A READPREV issued right after STARTBR returns the last record whose key is equal to or less than RIDFLD (the last record of the file when there is no RIDFLD).

## RESETBR

### *Function*

Reposition an open browse without ending it. The browse keeps the view of the file it had at STARTBR.

### *Syntax*

```
EXEC CICS RESETBR FILE(name)
          RIDFLD(key)
          [GTEQ | EQUAL]
          [GENERIC KEYLENGTH(n)]
          [REQID(id)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

The options are those of STARTBR. The condition is INVREQ (16) when no browse of the file is open. As for STARTBR, a key with no qualifying record is reported as ENDFILE by the next read, not as NOTFND.

```
EXEC CICS RESETBR FILE('CUSTOMERS') RIDFLD('00500') END-EXEC
```

## ENDBR

### *Function*

End a browse.

### *Syntax*

```
EXEC CICS ENDBR FILE(name)
          [REQID(id)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Conditions*

None. An ENDBR for a browse that is not open returns NORMAL.

### *Notes*

Every browse still open is ended when the task ends, but a program should end its browses itself.

***Differences from IBM CICS***

- A browse survives `SYNCPOINT`. IBM CICS ends the browses of a recoverable file at a syncpoint. A bricks browse is a read-only snapshot of the file and needs no lock to be released.
- `STARTBR` and `RESETBR` do not return `NOTFND`; the next read returns `ENDFILE`.
- `ENDBR` of a browse that is not open is not an error. IBM CICS returns `INVREQ`.
- `LENGTH` on `READNEXT` and `READPREV` of a file is an output only.

### Sequential Files in tmp_dir

The browse commands and `READ` also work on the sequential text files of the `tmp_dir` directory: the files that `READQ TD` and `WRITEQ TD` use (see "The tmp_dir Directory" in Chapter 15) and that REXX reads and writes with `LINEIN` and `LINEOUT`. Use them when a program needs more than reading a file from the front: reading by byte offset, reading backward, or returning to a known position. `READQ TD` remains the simplest way to read a file once from beginning to end.

The choice is made by name. When a file of that name exists in `tmp_dir`, the command works on that file; otherwise it works on the key-sequenced file. The name is used exactly as written (sequential file names are case-sensitive). When a name exists in both places the sequential file is used, and bricks writes a warning to the log at start-up.

***Records and relative byte addresses***

A record is one line of the file. Lines end with a single line feed (`X'0A'`), which is not part of the record. A last line without a line feed is returned once.

A record is identified by its relative byte address (RBA): the offset of its first byte in the file, counted from 0 and written as decimal digits. In a file whose three lines are 5 bytes long, the records are at RBA `0`, `6` and `12`. `RIDFLD` gives an RBA on input and receives one on output. An RBA that falls inside a record is rounded back to the start of that record, so the record that contains the byte is returned.

### *Commands*

| Command | On a sequential file |
|---|---|
| STARTBR | Supported. Without RIDFLD the browse starts at RBA 0. EQUAL, GENERIC and KEYLENGTH give INVREQ. |
| READNEXT | Returns the next line; RIDFLD receives its RBA. |
| READPREV | Returns the previous line. |
| RESETBR | RIDFLD is an RBA, rounded as for STARTBR. |
| ENDBR | Ends the browse and closes the file. |
| READ | RIDFLD is an RBA; reads one line. |
| UNLOCK | Accepted; does nothing. |
| WRITE, REWRITE | INVREQ; use WRITEQ TD. |
| DELETE | INVREQ; use DELETEQ TD. |

**Figure 14-8. File control commands on sequential files**

The encoding of the file (bricks SBCS records stored as UTF-8) is described under "The tmp_dir Directory" in Chapter 15. The RBA is an offset into the file as stored.

### *LENGTH on sequential files*

On a sequential file, LENGTH(len) on READ, READNEXT and READPREV works as in IBM CICS: on input it is the size of the receiving area, and on output the length of the record. A record longer than the area is truncated to fit, the command returns LENGERR (22), and *len* receives the full length of the record, so that the program can tell how much was lost.

```
LEN = 80
EXEC CICS READNEXT FILE('orders.txt') INTO(REC) RIDFLD(K) LENGTH(LEN) END-EXEC
IF RC = 22 THEN SAY 'Record was' LEN 'bytes; only 80 kept'
```

*Conditions on sequential files*

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | READ: the RBA is exactly the end of the file, so there is no record there. |
| ENDFILE | 20 | READNEXT past the last line, or READPREV before the first. |
| LENGERR | 22 | The record did not fit in LENGTH and was truncated. |
| INVREQ | 16 | EQUAL, GENERIC or KEYLENGTH coded; an RBA that is not a non-negative number, or that lies beyond the end of the file; WRITE, REWRITE or DELETE of a sequential file; no browse open. |
| FILENOTFOUND | 12 | The file was removed between the check for its name and its opening. |
| IOERR | 17 | The file system reported an error. |

**Figure 14-9. Conditions on sequential files**

*Lines added during a browse*

When another task appends lines to the file with WRITEQ TD while a browse is open, a READNEXT that reaches the old end of the file sees the new lines. A report transaction can therefore follow a file that a feeder task is still writing. IBM CICS does not show records added after STARTBR.

*Differences from IBM CICS*

- A sequential file is a text file of lines, not an ESDS of fixed-length or RDW-prefixed records.
- The RBA is decimal text, not a fullword binary number, so REXX and COBOL PIC X fields can hold it and compute with it.
- STARTBR without RIDFLD starts at the beginning of the file. IBM CICS requires RIDFLD.
- A browse survives SYNCPOINT, as for a key-sequenced file.
- Lines added during a browse are seen, as described above.

Chapter 15

# Temporary Storage and Transient Data Commands

This chapter describes the two queue facilities of bricks: temporary storage, whose queues are ordered sets of numbered items held in the region's database, and transient data, whose queues are sequential text files in the `tmp_dir` directory.

## The Two Kinds of Queue

| | Temporary storage (TS) | Transient data (TD) |
|---|---|---|
| Where it is kept | A sub-bucket of `data/files.boltdb` | A text file `tmp_dir/`*name* |
| Contents | Items numbered 1, 2, 3 ..., each an opaque string of bytes | Lines of bricks SBCS text, each ended by a line feed |
| Access | Read any item by number, or the next one; rewrite an item | Read the next line; append a line |
| Names | 1-64 characters `A–Z a–z 0–9 _ –`, folded to uppercase | 1-255 characters `A–Z a–z 0–9 . _ –`, case-sensitive |
| Recoverable | Yes: backed out with the unit of work | No |
| Also used by | CEMT INQUIRE TS | REXX `LINEIN`, `LINEOUT`, `CHARIN`, `STREAM`; the browse commands of "Sequential Files in tmp_dir" in Chapter 14 |

**Figure 15-1. Temporary storage and transient data compared**

Use temporary storage for scratch data that a transaction keeps between tasks, for paging lists, and for passing data from one task to another. Use transient data to load text files into the region and to write files for other programs to read.

Every TS update (`WRITEQ TS`, `DELETEQ TS`) is written at once and recorded in the task's unit of work, so `SYNCPOINT ROLLBACK` or an abend that is not handled restores the queue (see "Backout Points" in Chapter 16). A line written to a TD queue is not backed out.

The keyword `QNAME` may be used instead of `QUEUE` on every command in this chapter.

## READQ TS

### *Function*

Read an item from a temporary storage queue.

### *Syntax*

```
EXEC CICS READQ TS QUEUE(name)
          INTO(area)
          [ITEM(n) | NEXT [ITEM(n)]]
          [LENGTH(len)]
          [NUMITEMS(count)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**`QUEUE(name)`**

The queue. Required.

**`INTO(area)`**

The variable that receives the item. Required.

**`ITEM(n)` without `NEXT`**

Read item *n*. The value is input only and must be a positive integer; otherwise the command returns `INVREQ`. The read position is not changed.

**`NEXT`, or neither option**

Read the next item. The task has one read position per queue; the first such read returns item 1. With `NEXT ITEM(n)`, the variable *n* receives the number of the item read.

**`LENGTH(len)`**

On input, a positive value is the most bytes to return; a longer item is truncated. On output *len* receives the number of bytes returned.

**`NUMITEMS(count)`**

Receives the number of items in the queue.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| ITEMERR | 26 | The item does not exist: ITEM(n) is beyond the end, or the next read passed the last item. |
| QIDERR | 44 | The queue does not exist. |
| INVREQ | 16 | A required option is missing, the name is not valid, or ITEM without NEXT does not hold a positive integer. |
| IOERR | 17 | The database reported an error. |

**Figure 15-2. READQ TS conditions**

***Examples***

```
DO FOREVER
  EXEC CICS READQ TS QUEUE(QNM) INTO(REC) NEXT END-EXEC
  IF RC = 26 THEN LEAVE                    /* end of queue */
  SAY REC
END
```

***Notes***

The read position belongs to the task and is discarded when the task ends; the next task that reads the queue starts again at item 1. The position is not reset by SYNCPOINT.

In a read loop, do not code ITEM(n) without NEXT and let the command update *n*: without NEXT the option is input only, and every call reads the same item.

***Differences from IBM CICS***

- The read position survives SYNCPOINT. IBM CICS resets it for a recoverable queue.
- An item longer than LENGTH is truncated without LENGERR.

## WRITEQ TS

***Function***

Add an item to the end of a temporary storage queue, creating the queue if necessary, or replace an existing item.

***Syntax***

```
EXEC CICS WRITEQ TS QUEUE(name)
          FROM(data)
          [ITEM(n) [REWRITE]]
          [NUMITEMS(count)]
          [MAIN | AUXILIARY]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

***Options***

**`QUEUE(name)`**

The queue. Required.

**`FROM(data)`**

The item to write. Required.

**`ITEM(n)`**

Without `REWRITE`: when *n* is a variable, it receives the number of the new item. With `REWRITE`: the number of the item to replace.

**`REWRITE`**

Replace item `ITEM(n)` instead of adding an item. `REWRITE` requires `ITEM`.

**`NUMITEMS(count)`**

Receives the number of items in the queue after the write.

**`MAIN`, `AUXILIARY`**

Accepted for compatibility; all queues are kept in the database.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| ITEMERR | 26 | `REWRITE`: the item, or the queue, does not exist. |
| INVREQ | 16 | A required option is missing, the name is not valid, or `ITEM` on a `REWRITE` is not numeric. |
| IOERR | 17 | The database reported an error. |

**Figure 15-3. WRITEQ TS conditions**

***Examples***

```
EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(MSG) ITEM(I) END-EXEC
SAY 'Wrote item' I

EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(FIXED) ITEM(3) REWRITE END-EXEC
```

***Notes***

Each write is committed to the database before the command returns. A backout deletes an added item, or restores a replaced item.

***Differences from IBM CICS***

- `REWRITE` of a queue that does not exist returns `ITEMERR`. IBM CICS returns `QIDERR`.
- There is no limit on the number of items and no `NOSPACE` condition.

## DELETEQ TS

***Function***

Delete a temporary storage queue and all its items.

***Syntax***

```
EXEC CICS DELETEQ TS QUEUE(name)
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

***Conditions***

`QIDERR` (44) when the queue does not exist; `INVREQ` (16) when the name is missing or not valid; `IOERR` (17) for a database error.

***Notes***

A later `WRITEQ TS` creates the queue again, starting at item 1. A backout re-creates the deleted queue with all its items.

## READQ TD

***Function***

Read the next line of a sequential file in `tmp_dir`.

### *Syntax*

```
EXEC CICS READQ TD QUEUE(name)
          INTO(area)
          [LENGTH(len)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Options*

**`QUEUE(name)`**

The name of the file in `tmp_dir`. Required.

**`INTO(area)`**

The variable that receives the line, without its line feed. Required.

**`LENGTH(len)`**

When *len* is a variable, it receives the length of the line.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| QZERO | 23 | There are no more lines. INTO is not changed. |
| QIDERR | 44 | The file does not exist. |
| INVREQ | 16 | INTO is missing or is not a variable, or the name is not valid. |
| IOERR | 17 | The file system reported an error. |

**Figure 15-4. READQ TD conditions**

### *Examples*

```
        PERFORM IMPORT-ONE UNTIL DONE-FLAG = 'Y'.
        ...
        IMPORT-ONE.
            EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC)
                      RESP(WS-RESP) END-EXEC.
            EVALUATE WS-RESP
               WHEN DFHRESP(NORMAL) PERFORM HANDLE-RECORD
               WHEN DFHRESP(QZERO)  MOVE 'Y' TO DONE-FLAG
               WHEN OTHER           PERFORM IMPORT-ERROR
            END-EVALUATE.
```

***Notes***

The file is opened on the first `READQ TD` of the task and read line by line. At the end of the file the command returns `QZERO` and closes the file, so a further `READQ TD` starts again at the first line. When the task has written to the same queue, the first `READQ TD` closes the file for writing and reads it from the start. Every file still open is closed when the task ends.

A COBOL or PL/I program must code `RESP` (as above) or handle `QZERO`; otherwise reaching the end of the file abends the task `AEIW`.

To read a file backward, by position, or more than once without starting over, use the browse commands on the same name; see "Sequential Files in tmp_dir" in Chapter 14.

## WRITEQ TD

***Function***

Add a line to the end of a sequential file in `tmp_dir`, creating the file if it does not exist.

***Syntax***

```
EXEC CICS WRITEQ TD QUEUE(name)
          FROM(data)
          [LENGTH(len)]
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

***Options***

**`QUEUE(name)`**

The name of the file in `tmp_dir`. Required.

**`FROM(data)`**

The line to write. Trailing blanks are removed first, so a COBOL `PIC X(n)` record is written without its padding. A line feed is added. Required.

**`LENGTH(len)`**

Accepted for compatibility; the line written is the whole of `FROM` without its trailing blanks.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| INVREQ | 16 | FROM is missing; the name is not valid; or the data contains a control character other than tab, such as a carriage return. The message gives the offset and value of the first such byte. |
| IOERR | 17 | The file system reported an error. |

**Figure 15-5. WRITEQ TD conditions**

### *Examples*

```
DO I = 1 TO REC.0
   EXEC CICS WRITEQ TD QUEUE('export.txt') FROM(REC.I) END-EXEC
END
```

## DELETEQ TD

### *Function*

Delete a sequential file in `tmp_dir`.

### *Syntax*

```
EXEC CICS DELETEQ TD QUEUE(name)
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

### *Conditions*

`INVREQ` (16) when the name is missing or not valid; `IOERR` (17) for a file system error. Deleting a file that does not exist is `NORMAL`.

### *Notes*

Any file the task has open under that name is closed first. Use `DELETEQ TD` to empty an export file before writing a new one.

***Differences from IBM CICS***

- A TD queue is an extrapartition file named by the program. There are no intrapartition queues, no destination definitions and no trigger levels.
- `DELETEQ TD` deletes the file; it does not return `QIDERR` for a file that does not exist.

### The tmp_dir Directory

The directory that holds transient data files is set in `bricks.cnf`:

```
tmp_dir = runtime/tmp
```

When the line is omitted the directory is `runtime/tmp` under the runtime directory. It is created at start-up if it does not exist. See "The runtime Directory" in Appendix D.

***Names***

A queue name is a file name directly in `tmp_dir`: 1 through 255 characters from `A–Z`, `a–z`, `0–9`, period, underscore and hyphen. It may not begin with a period or contain two periods together, so it cannot name a subdirectory or a file outside `tmp_dir`. bricks also checks every resolved path against the directory, so a symbolic link cannot lead out of it. A name that breaks these rules gives `INVREQ`.

***Encoding***

A program reads and writes bricks SBCS characters; the file on disk is UTF-8. On write, the bytes tab, `X'20'-X'7E'` and `X'80'-X'FF'` are accepted and stored as UTF-8; any other control byte, including carriage return, is rejected with `INVREQ`. Files are written with a line feed alone at the end of each line.

On read, each line is converted back. A line that is not valid UTF-8 (a Latin-1 file written by another tool) is taken byte for byte. A carriage return in a line read is kept as it is: bricks does not remove it. Remove the carriage returns from a file that comes from Windows before using it, for example:

```
tr –d '\r' < windows.csv > runtime/tmp/clean.csv
```

REXX `CHARIN` reads characters the same way: a byte that is not valid UTF-8 is kept as its Latin-1 character, and only a character outside the code page becomes `?`. `CHARS()` counts bytes of the file, so it is an upper bound on the characters left.

***Sharing files between languages***

COBOL and PL/I (`READQ TD`, `WRITEQ TD`), REXX (`LINEIN`, `LINEOUT`, `STREAM`) and the file control browse commands all use the same files. A file written by one is read by the others; only the layout of the fields within a line must be agreed (the supplied samples separate fields with `|`).

***End of task***

Every file a task has open is closed when the task ends, so a program that does not close its files does not leave them open.

Chapter 16

# Recovery and Condition Handling

This chapter describes how bricks keeps the changes a task makes consistent, and how a program deals with the exceptional outcomes of its commands. It covers:

- units of work, the points at which they are committed or backed out, and the locks that a unit of work holds;
- the `SYNCPOINT` command;
- conditions, and the order in which `RESP`, `NOHANDLE`, `HANDLE CONDITION`, `IGNORE CONDITION` and `HANDLE AID` apply;
- abends, `HANDLE ABEND`, the abend messages and dynamic backout;
- runaway tasks and abend `AICA`.

The abend codes themselves are listed in Appendix C. The response codes are listed in Chapter 18.

## Units of Work

A **unit of work** is the set of changes a task makes to recoverable resources between two syncpoints. Either all of them are kept (the unit of work is **committed**) or none of them is (it is **backed out**). The recoverable resources of bricks are:

- files: `WRITE`, `REWRITE` and `DELETE` (Chapter 14);
- temporary storage queues: `WRITEQ TS` and `DELETEQ TS` (Chapter 15);
- the database: the SQL transaction of the task's `EXEC SQL` statements ("Units of Work and Commit Points" in Chapter 21).

Transient data lines, terminal output, `START` requests and console messages are not recoverable: a backout does not undo them.

File and TS changes are written to the database as each command runs. bricks also records, for each change, how to undo it (the before-image of a replaced record, the items of a deleted queue). A commit discards these records; a backout applies them in reverse order.

A task has **one** unit of work, however many programs it runs. Every program that the task reaches by `LINK` or `XCTL` works in the same unit of work, so a `SYNCPOINT` issued by any of them commits the changes of all of them, and a backout backs out the changes of all of them.

The changes of a unit of work that has not been committed are visible to other tasks: a READ by another task returns the new record. A unit of work protects the records it has changed by locking them (see "Locks and the Unit of Work"), not by hiding them.

### Commit Points

The unit of work is committed, and a new one begins, at each of the following points:

| Point | Notes |
|---|---|
| SYNCPOINT | Explicit commit by any program of the task. |
| RETURN | From the top-level program only (the program at the highest logical level). RETURN from a program reached by LINK goes back to the caller and is not a commit point. |
| Clean end of task | The top-level program ends without RETURN (COBOL GOBACK or STOP RUN, PL/I end of MAIN, REXX EXIT or end of program). |
| XCTL that cannot carry the unit of work | The target is bound to a different database, is a supplied transaction, or runs in another region (MRO). bricks commits at the XCTL. See "XCTL and the Unit of Work". |

**Figure 16-1. Commit points**

The following are **not** commit points: LINK, RETURN from a LINK level, an XCTL whose target continues the unit of work, and EXEC SQL COMMIT or ROLLBACK, which settle only the database transaction (see "COMMIT and ROLLBACK" in Chapter 21).

The database is committed first, then the file and TS changes. When the database refuses the commit (a deferred constraint, a serialization failure, a lost connection), the file and TS changes are backed out too, and the task abends ASP3, ASP7 or ASPR. No HANDLE ABEND exit is driven for these abends. The abend detail begins with the point that failed: SYNCPOINT:, RETURN:, TASK END: or XCTL:. See "Other Transaction Abend Codes" in Appendix C.

### Backout Points

The unit of work is backed out at each of the following points:

| Point | Notes |
|---|---|
| SYNCPOINT ROLLBACK | Explicit backout by any program of the task. A new unit of work begins and the task continues. |
| Abend not handled | An abend that no HANDLE ABEND exit catches, at any LINK level (dynamic backout). See "How an Abend Ends a Task". |
| Refused commit | Abends ASP3, ASP7 and ASPR (see above). |
| XCTL target not started | A target that passed the checks at the XCTL but could not be loaded. The carried unit of work is backed out and the task ends. |
| Task ended by an error | A task that ends because of an error that is not an abend: an unsupported command, or a COBOL runtime error such as a subscript out of range (see "Runtime Errors" in Chapter 24). |

**Figure 16-2. Backout points**

An abend that a HANDLE ABEND LABEL exit catches backs out nothing. The exit decides whether to issue SYNCPOINT or SYNCPOINT ROLLBACK. A PROGRAM exit runs, and when it returns the abend completes with backout.

### Locks and the Unit of Work

A task holds two kinds of lock:

- **Record locks**, taken by file control ("Record Locks" in Chapter 14). A record that the task has changed with WRITE, REWRITE or DELETE stays locked until the unit of work ends: SYNCPOINT, SYNCPOINT ROLLBACK, the backout of an abend, or the end of the task. A record read for update but not changed is released by UNLOCK, by a READ ... UPDATE of another key in the same file, or at task end.
- **Enqueues**, taken by ENQ ("ENQ and DEQ" in Chapter 13). An enqueue taken without HOLD is released at the next SYNCPOINT or SYNCPOINT ROLLBACK; one taken with HOLD is kept until DEQ or the end of the task.

Both belong to the task, not to the program:

- They are kept across `LINK`. A program reached by `LINK` can `DEQ` an enqueue its caller took, and the caller keeps the locks the linked program took after it returns.
- They are carried across `XCTL` to the target, which holds them as its own.
- Every lock still held is released when the task ends, however it ends.

### XCTL and the Unit of Work

`XCTL` passes control to another program in the same task, and the target continues the unit of work: the open SQL transaction, the undo records of the file and TS changes, the enqueues and the record locks are all handed to it. Its `SYNCPOINT`, its `RETURN` or its abend settles the work of both programs. In summary:

- A target that is not defined or not authorized raises `PGMIDERR` or `NOTAUTH` at the `XCTL`, and nothing is backed out.
- A target that passes these checks but cannot be loaded is not started; the carried unit of work is backed out and the task ends.
- A target bound to a different database, a supplied transaction, or a target routed to another region causes an implicit syncpoint at the `XCTL`. A commit refused there abends the issuing program `ASP3`, `ASP7` or `ASPR`.
- An `XCTL` issued at a `LINK` level replaces the program at that level.

The complete rules are given in "XCTL" in Chapter 12.

**Note:**

In IBM CICS a unit of work can include several resource managers. The commit at an `XCTL` to a program on another database is a bricks deviation.

### SYNCPOINT

***Function***

Commit the unit of work and begin a new one.

***Syntax***

```
EXEC CICS SYNCPOINT
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

***Description***

The SQL transaction is committed, then the undo records of the file and TS changes are discarded: every change made since the last syncpoint (or the start of the task) becomes permanent. Enqueues taken without `HOLD` are released, and so are the record locks of the changed records.

A program that runs to its end never needs to issue `SYNCPOINT`: `RETURN` and the clean end of the task commit. Use it in a long task that wants to keep the work done so far, or to separate work that must be kept from work that may still be backed out.

***Conditions***

None. A commit the database refuses is not a condition: the file and TS changes are backed out and the task abends `ASP3`, `ASP7` or `ASPR`, without any `HANDLE ABEND` exit. `SYNCPOINT` never returns `ROLLEDBACK`.

***Examples***

```
      *  Phase 1: keep the audit record even if phase 2 fails.
           EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(WS-LOG) END-EXEC.
           EXEC CICS SYNCPOINT END-EXEC.
      *  Phase 2: backed out if validation fails.
           EXEC CICS REWRITE FILE('CUSTOMERS') FROM(WS-CUST) END-EXEC.
           IF WS-INVALID
              EXEC CICS SYNCPOINT ROLLBACK END-EXEC
              EXEC CICS ABEND ABCODE('VAL1') END-EXEC
           END-IF.
```

***Notes***

A browse, the read position of a TS queue, the open TD files, web sessions and documents are not affected by `SYNCPOINT`.

A deferred constraint or a serialization conflict is reported only when the database commits. To have it reported while the program can still react, issue `EXEC SQL SET CONSTRAINTS ALL IMMEDIATE END-EXEC` early in the unit of work and test `SQLCODE` after each update. See "Refused Commits" in Chapter 21.

***Differences from IBM CICS***

- bricks has no two-phase commit with other regions and no in-doubt unit of work. A database connection lost during the commit is abend `ASPR`, with the local changes backed out; IBM CICS would keep the unit of work in doubt until the connection returned.
- Browses and TS read positions survive the syncpoint.

## SYNCPOINT ROLLBACK

***Function***

Back out the unit of work and begin a new one.

***Syntax***

```
EXEC CICS SYNCPOINT ROLLBACK
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

***Description***

The SQL transaction is rolled back. The file and TS changes are then undone in the reverse of the order in which they were made:

| Change | How it is undone |
|---|---|
| `WRITE` | The new record is deleted. |
| `REWRITE` | The before-image is written back. |
| `DELETE` | The deleted record is written back. |
| `WRITEQ TS` | The added item is deleted. |
| `WRITEQ TS REWRITE` | The before-image of the item is written back. |
| `DELETEQ TS` | The queue is re-created with all its items. |

**Figure 16-3. How SYNCPOINT ROLLBACK undoes each change**

The record locks of the changed records and the enqueues taken without `HOLD` are then released, and the task continues with a new unit of work.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| ROLLEDBACK | 82 | The backout of the file and TS changes failed for at least one change (the database error is logged). The undo records are discarded anyway, so some changes may remain. Unhandled in COBOL or PL/I: abend AEXJ. |

**Figure 16-4. SYNCPOINT ROLLBACK conditions**

A database rollback that fails (the connection is gone) is logged but does not raise ROLLEDBACK: the database discards an unfinished transaction whose connection is lost, so the outcome is a rollback either way.

### *Examples*

```
EXEC CICS WRITE FILE('ACCT') RIDFLD(DEBIT_ID) FROM(DEBIT_REC) END-EXEC
EXEC CICS WRITE FILE('ACCT') RIDFLD(CREDIT_ID) FROM(CREDIT_REC) END-EXEC
IF DEBIT_TOTAL <> CREDIT_TOTAL THEN DO
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC        /* both writes undone */
   EXEC CICS ABEND ABCODE('IMBL') END-EXEC
END
EXEC CICS SYNCPOINT END-EXEC                    /* commit both writes */
```

### *Differences from IBM CICS*

In IBM CICS, ROLLEDBACK on SYNCPOINT means that a resource manager voted against the commit and the unit of work was backed out. In bricks that case is an abend (ASP3, ASP7, ASPR), and ROLLEDBACK is returned only by SYNCPOINT ROLLBACK when the local backout itself failed.

## Conditions

Every EXEC CICS command ends with a response. The response NORMAL (0) means the command did what was asked; any other response is a **condition**, such as NOTFND or MAPFAIL. After every command EIBRESP and EIBRESP2 are set (and, in REXX and COBOL, RC); see Chapter 17 and Chapter 18.

A program can deal with a condition in four ways. They are applied in this order; the first that applies decides:

| Order | Mechanism | Effect |
|---|---|---|
| 1 | RESP(var) or NOHANDLE on the command | The response is stored and the program continues with the next statement. No HANDLE AID or HANDLE CONDITION branch is taken for this command, and the condition never abends the task. Applies to this one command only. |
| 2 | HANDLE AID | When the command received an attention identifier that has a label, control passes to the label. |
| 3 | HANDLE CONDITION or IGNORE CONDITION for this condition | A label: control passes to it. Ignored: the response is changed to NORMAL and the program continues. |
| 4 | HANDLE CONDITION ERROR or IGNORE CONDITION ERROR | As 3, for any condition not named in 3. |
| 5 | None of these | The condition is unhandled: the default action for the language applies. |

**Figure 16-5. Order in which condition handling applies**

## Unhandled Conditions

A condition is **unhandled** when the command codes neither RESP nor NOHANDLE, no HANDLE CONDITION names it or ERROR, and no IGNORE CONDITION covers it. What happens then depends on the language of the program that issued the command, as in IBM CICS:

**COBOL and PL/I**

The task **abends** with the IBM abend code of the condition: `NOTFND` is `AEIM`, `MAPFAIL` is `AEI9`, `INVREQ` is `AEIP`, and so on ("Condition Abend Codes" in Appendix C). No statement after the command runs. The abend is an ordinary abend: a transaction dump message is logged, a `HANDLE ABEND` exit at this level or a caller's can catch it, and otherwise the unit of work is backed out and the operator sees `DFHAC2206`. At the exit, `EIBRESP` still holds the condition and `ASSIGN ABCODE` returns the abend code.

**REXX**

The task **never** abends on a condition. `RC` and `EIBRESP` hold the response and the program continues with the next statement. `SIGNAL ON ERROR` traps the non-zero `RC` (see "SIGNAL ON and SIGNAL OFF" in Chapter 39), and `HANDLE CONDITION` still branches.

Some conditions never abend, in any language, because IBM's default action for them is to continue: `EOC`, `ENQBUSY`, `QBUSY`, `SYSBUSY` and the others listed in "Conditions That Never Abend" in Appendix C.

**Note:**

In COBOL and PL/I, a command whose response the program tests must code `RESP`. Testing `EIBRESP` after a command without `RESP` works only for `NORMAL`: any other outcome abends the task before the test runs.

## HANDLE CONDITION

### *Function*

Name the labels to which control passes when later commands end with given conditions.

### *Syntax*

```
EXEC CICS HANDLE CONDITION
          condition[(label)] [condition[(label)] ...]
          [ERROR(label)]
END-EXEC
```

### *Options*

***condition*(*label*)**

When a later command ends with *condition*, pass control to *label*. A condition already named is given the new label. Any condition of "Responses" in Chapter 18 may be named.

***condition* without a label**

Cancel the label for *condition*; the condition is again unhandled (or handled by `ERROR`). This also cancels an `IGNORE CONDITION` for it.

**`ERROR`(*label*)**

Pass control to *label* for any condition that has no label of its own and is not ignored.

`HANDLE CONDITION` with no operands cancels every label and every `IGNORE CONDITION`. One command may name up to any number of conditions.

### *Description*

The labels stay in effect for every later command of the program until they are changed; they are not canceled when a branch is taken. When control passes to a label, `EIBRESP` holds the condition. In REXX control passes as if by `SIGNAL` *label*; in COBOL as if by `GO TO` *label*. PL/I has no statement labels: the label names a parameterless internal procedure of `MAIN`, and the task ends when that procedure returns (see "HANDLE ABEND in PL/I" in Chapter 33).

A command that codes `RESP` or `NOHANDLE` never takes a `HANDLE CONDITION` branch; the labels remain in effect for the other commands. The two styles can be mixed in one program.

### *Conditions*

None.

### *Examples*

```
EXEC CICS HANDLE CONDITION NOTFND(NOREC) DUPREC(DUP) ERROR(OOPS) END-EXEC
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
SAY 'Found:' REC
EXIT
NOREC: SAY 'No customer with key' CKEY; EXIT 4
DUP:   SAY 'Duplicate key on insert'; EXIT 8
OOPS:  SAY 'Other CICS error, RC=' EIBRESP; EXIT 12
```

***Differences from IBM CICS***

- The condition names are not checked. A name that is not a condition is accepted and never matches.
- IBM CICS limits one command to 16 conditions; bricks has no limit.
- `PUSH HANDLE` and `POP HANDLE` are not supported ("Commands Not Implemented" in Chapter 19).

## IGNORE CONDITION

***Function***

Specify conditions that are to be ignored.

***Syntax***

```
EXEC CICS IGNORE CONDITION condition [condition ...]
END-EXEC
```

***Description***

When a later command ends with an ignored condition, no branch is taken, the task does not abend, and the program continues with the next statement. The condition stays ignored until a `HANDLE CONDITION` names it again (with or without a label).

An ignored condition is different from one that has no label. A condition with no label still goes to `ERROR(`*label*`)` when that is set; an ignored one does not. `IGNORE CONDITION ERROR` ignores every condition that has no label of its own.

**The response of an ignored condition reads as NORMAL.** bricks sets `EIBRESP` (and `RC`) to 0 after a command whose condition is ignored, so the program cannot tell from `EIBRESP` that the condition occurred. A program that must tell the outcome apart codes `RESP` on that command instead.

***Conditions***

None.

***Examples***

```
           EXEC CICS HANDLE CONDITION ERROR(BADIO) END-EXEC.
      *    MAPFAIL on an empty screen is expected: carry on.
           EXEC CICS IGNORE CONDITION MAPFAIL END-EXEC.
           EXEC CICS RECEIVE MAP('CUST1') END-EXEC.
      *    NOTFND must be told apart: code RESP on that command.
           EXEC CICS READ FILE('CUSTOMERS') INTO(WS-REC)
                     RIDFLD(WS-KEY) RESP(WS-RESP) END-EXEC.
           IF WS-RESP = DFHRESP(NOTFND)
              PERFORM CREATE-NEW-CUSTOMER
           END-IF.
```

***Differences from IBM CICS***

- In IBM CICS, control returns after an ignored condition to the statement that follows the command, and the EIB is set, so the program can test `EIBRESP` for the condition that was ignored. In bricks, `EIBRESP`, `EIBRESP2` and, in REXX and COBOL, `RC` all read 0 (`NORMAL`) after an ignored condition. A program that must know whether the condition occurred codes `RESP` on the command.
- The condition names are not checked.

## HANDLE AID

***Function***

Name the labels to which control passes when the operator presses given attention keys.

***Syntax***

```
EXEC CICS HANDLE AID
          key[(label)] [key[(label)] ...]
END-EXEC
```

***Options***

***key*(*label*)**

*key* is one of `ENTER`, `CLEAR`, `PA1-PA3` and `PF1-PF24`. When a later command receives that key from the terminal, control passes to *label*.

***key* without a label**

Cancel the label for *key*.

`HANDLE AID` with no operands cancels every key.

### *Description*

The branch is taken after a command that set `EIBAID` (`SEND MAP` and `SEND TEXT`, which wait for the operator, `RECEIVE MAP` and `CONVERSE`) and ended `NORMAL`. It is taken once for each key press: a later command that does not read the terminal again does not branch again. It is checked before `HANDLE CONDITION`. A command that codes `RESP` or `NOHANDLE` takes no branch.

**`PA1`** is the operator's break-out key in bricks: it ends the running transaction whatever the program has armed. A label for `PA1` is therefore not reached.

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| INVREQ | 16 | A key name is not one of those listed. |

**Figure 16-6. HANDLE AID conditions**

### *Examples*

```
EXEC CICS HANDLE AID PF3(QUIT) PF12(QUIT) CLEAR(QUIT) END-EXEC
DO FOREVER
   EXEC CICS RECEIVE MAP('CUST1') INTO(SCR.) END-EXEC
   /* process the input */
   EXEC CICS SEND MAP('CUST1') FROM(SCR.) ERASE END-EXEC
END
QUIT:
   EXEC CICS RETURN END-EXEC
```

### *Differences from IBM CICS*

- `ANYKEY`, `OPERID`, `LIGHTPEN`, `TRIGGER` and `CLRPARTN` are not supported and give `INVREQ`.
- **`PA1`** cannot be handled (see above).

## HANDLE ABEND

### *Function*

Activate, cancel or reactivate an exit that receives control when the task abends.

***Syntax***

```
EXEC CICS HANDLE ABEND
          {LABEL(label) | PROGRAM(name) | CANCEL | RESET}
          [RESP(resp) [RESP2(resp2)] | NOHANDLE]
END-EXEC
```

***Options***

**`LABEL(label)`**

When the task abends, pass control to *label* in this program. Not available in PL/I (a compile-time error, as in IBM CICS).

**`PROGRAM(name)`**

When the task abends, `LINK` to program *name* (a literal or a variable), with no COMMAREA. When it returns, the abend continues and the task ends with the original code. When both a label and a program are active, the label is used.

**`CANCEL`**

Deactivate the exit of this program.

**`RESET`**

Reactivate the exit that was deactivated most recently: the exit that is now running (exits are deactivated on entry), or else the exit that `CANCEL` deactivated. With no such exit, `RESET` does nothing.

***Description***

An exit receives control for any abend of the task: `EXEC CICS ABEND`, an unhandled condition in COBOL or PL/I (`AEIM`, `AEI9` ...), a runaway task (`AICA`), a program check (`ASRA`), the abends that CICS raises itself (`ABM0`, `ABMB`, `AEXZ`, the first `ATNI`, `ADBU`, `AD2S`, `AD2P`), and an abend in a program this one reached by `LINK`. The runtime abends of PL/I itself, such as `APLI`, are not offered to the exit of the PL/I program that failed, only to the exits of its callers. It does not receive control for `ABEND CANCEL`, `APCT`, `ASP3`, `ASP7`, `ASPR` or a second `ATNI`; see "Other Transaction Abend Codes" in Appendix C.

The following rules apply, as in IBM CICS:

- **Nothing is backed out** when a LABEL exit catches the abend. The exit decides whether to issue SYNCPOINT or SYNCPOINT ROLLBACK. A PROGRAM exit runs, and when it returns the abend completes with backout.
- **The exit is deactivated on entry**, label and program alike, so an exit that abends (or loops into a second AICA) ends the task instead of entering itself again. HANDLE ABEND RESET in the exit reactivates it.
- **The search starts at the current LINK level and moves up.** An abend in a program reached by LINK that has no active exit of its own is offered to its caller's exit, then to that caller's caller, and so on. An abend is never a condition in the caller: RESP on the LINK, HANDLE CONDITION PGMIDERR and ERROR do not see it.
- **EIBABCODE and ASSIGN ABCODE** give the abend code when the exit receives control (AAAA for an EXEC CICS ABEND without ABCODE). After an unhandled condition, EIBRESP still holds the condition: save it before the next command, which sets it again.
- **After AICA** the runaway interval starts again, so the exit has a full 2 seconds of its own.

***Conditions***

INVREQ (16) when none of LABEL, PROGRAM, CANCEL and RESET is coded.

***Examples***

A REXX exit that backs out the work and ends the task:

```
EXEC CICS HANDLE ABEND LABEL(CLEANUP) END-EXEC
EXEC CICS WRITE FILE('LEDGER') RIDFLD(K) FROM(REC) END-EXEC
EXEC CICS WRITE FILE('AUDIT') RIDFLD(K) FROM(REC) END-EXEC
/* a later command may abend */
EXIT
CLEANUP:
   SAY 'Caught abend' EIBABCODE
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC        /* undo both writes */
   EXEC CICS RETURN END-EXEC
```

A COBOL exit that catches the AEIM abend of an unhandled NOTFND (the supplied sample transaction ABNH):

```
       PROCEDURE DIVISION.
       MAIN.
           EXEC CICS HANDLE ABEND LABEL(RECOVER) END-EXEC.
           EXEC CICS READ FILE('customers') INTO(REC)
                     RIDFLD('NOSUCHKEY') END-EXEC.
           EXEC CICS RETURN END-EXEC.
       RECOVER.
           MOVE EIBRESP TO SCR-RESP.
           EXEC CICS ASSIGN ABCODE(WS-CODE) END-EXEC.
           MOVE WS-CODE TO SCR-CODE.
           EXEC CICS SEND TEXT FROM(SCR) ERASE END-EXEC.
           EXEC CICS RETURN END-EXEC.
```

Figure 16-7 shows the result. The exit has received control instead of the DFHAC2206 message of Figure 16-8: `ASSIGN ABCODE` returns `AEIM`, and `EIBRESP` still holds 13, the response of the `NOTFND` condition that caused the abend. The task then ends normally with `RETURN`.




**Figure 16-7. ABNH: a HANDLE ABEND LABEL exit catches abend AEIM** (Model 2, 24 x 80; capture screens/abnh-caught-aeim.txt)

***Notes***

In PL/I the exit is always a program: `HANDLE ABEND PROGRAM(name)`. See "HANDLE ABEND in PL/I" in Chapter 33.

A PROGRAM exit that cannot be loaded (not defined, not authorized, source missing or not compilable) is not an error when it is activated. When an abend needs it, the abend ends as APCT instead, and no further exit runs. An exit program that starts and then fails is logged, and the original abend continues.

***Differences from IBM CICS***

- The exit program receives no COMMAREA; it can obtain the code with ASSIGN ABCODE.

### Abends

A task abends (ends abnormally) when:

- the program issues EXEC CICS ABEND ("ABEND" in Chapter 12);
- a COBOL or PL/I program leaves a condition unhandled (AEIx, AEYx, AEXx);
- CICS cannot carry out a command or a commit (ABM0, ABMB, AEXZ, ATNI, APCT, ASP3, ASP7, ASPR, ADBU, AD2S, AD2P); these end REXX tasks too;
- the task runs too long without giving control back (AICA, see "Runaway Tasks");
- the program fails in its own language: a COBOL division by zero without ON SIZE ERROR, a COBOL addressing exception or an internal failure of the interpreter (ASRA), a PL/I condition such as ZERODIVIDE ("Runtime Abends" in Chapter 33).

Each code is described in Appendix C.

Most abends also write a transaction dump to abend.log, which the transaction CAID analyses (see "Transaction Dumps" and "CAID: The Abend Analyser").

### How an Abend Ends a Task

An abend that no HANDLE ABEND exit catches, at any LINK level, ends the task as IBM CICS ends it:

1. **Dynamic backout.** The unit of work of the whole task is backed out: the file and TS changes made since the last syncpoint by every program of the task, and the SQL transaction.
2. **Log.** Message `DFHAC2236` is written to the log. It names the transaction the task was started with (even when a program reached by `LINK` abended) and, separately, the program that abended:

   ```
   DFHAC2236 10/22/26 13:15:03 BRICKS01 Transaction ABNC
     transaction number 12 abend AEIM in program
     abnc.cob term T0001. Updates to local recoverable
     resources will be backed out.
   ```

3. **Terminal.** The screen is erased and message `DFHAC2206` is written at the top, with the keyboard unlocked. bricks adds a detail row: the failing command, the message of the condition, or the program and line of a runaway. Press **`CLEAR`** (or any key) and type the next transaction identifier. Figure 16-8 shows the screen.
4. **No hand-off.** A `RETURN TRANSID` or `COMMAREA` that the task set before the abend is discarded: there is no next transaction identifier, and the next transaction the operator types starts with `EIBCALEN` 0.

Figure 16-8 shows the terminal after the supplied sample ABNC has ended with an unhandled `NOTFND`. Row 1 gives the message number `DFHAC2206`, the time, the applid of the region, the transaction identifier and the abend code. Row 2 states that the unit of work was backed out. The `Detail:` row, which IBM CICS does not show, names the command that failed and the condition it raised. The rest of the screen is empty, and the keyboard is unlocked.

```
DFHAC2206 19:07:41 BRICKS01 Transaction ABNC failed with abend AEIM.
  Updates to local recoverable resources backed out.

Detail: READ FILE('customers') NOTFND
```

**Figure 16-8. DFHAC2206 abend message: ABNC ends with abend AEIM** (Model 2, 24 x 80; capture screens/abend-aeim.txt)

For `ADBU` and `AD2S` the screen adds a line that asks the operator to have an administrator check the database. A web task (Chapter 20) receives the same text with HTTP status 500.

***Tasks started by START***

A task started by `EXEC CICS START` that abends while another task owns the terminal writes `DFHAC2236` to the log at once. When the `START` named `TERMID` (a terminal task), its `DFHAC2206` screen is shown as soon as the terminal is free: when the task in the foreground ends, or at once if the terminal is idle. A `START` without `TERMID` is a non-terminal task; as in IBM CICS, its abend is reported in the log only.

***Program checks***

A COBOL division by zero that no `ON SIZE ERROR` covers (IBM's decimal-divide exception, `S0CB`), a COBOL addressing exception, and an internal failure of the REXX or COBOL interpreter (a bricks error) are program checks: the task abends `ASRA`, a transaction dump is taken, and only that task ends; the region keeps running. For an internal failure the Go stack trace also goes to the log. The PL/I interpreter reports its internal failures as `APLI`. As IBM CICS lets an exit catch `ASRA`, bricks offers the program check of a REXX or COBOL program to `HANDLE ABEND` first. The runtime abends of a PL/I program (`APLI`, and the PL/I conditions of "Runtime Abends" in Chapter 33) are not offered to the PL/I program's own exit; the exit of a caller up the `LINK` chain can still catch them. A failure inside the implementation of an `EXEC CICS` command is not the program's fault: it is abend `AEXZ`, handled the same way.

***Lost terminal***

When the terminal connection fails during `SEND MAP`, `SEND TEXT` or `SEND CONTROL`, including the wait for the operator's key (the operator closed the emulator, the network dropped), the task abends `ATNI`. There is no screen left, so step 3 above is skipped: the log has the dump message, an `ATNI detail:` line that names the terminal, program and command, and `DFHAC2236`. EDF is switched off for the rest of the task. `HANDLE ABEND` may catch the first `ATNI`; any terminal command after it ends the task at once with a final `ATNI` that no exit catches, so an exit cannot loop on the dead connection.

***EDF***

Under CEDF the AN ABEND HAS OCCURRED display is shown before the exit is searched, except for `ATNI`. For a command that will leave a condition unhandled, the COMMAND EXECUTION COMPLETE display already forecasts the outcome: `NOT HANDLED – ABEND AEIM` in COBOL and PL/I, `NOT HANDLED – RC SET` in REXX. See "Before and After a Command" in Chapter 8.

### Transaction Dumps

When a task abends, bricks takes a **transaction dump**, as IBM CICS does: a record of the task at the moment of the abend. IBM CICS writes the dump to a dump data set (`DFHDMPA` or `DFHDMPB`) and prints it with the utility `DFHDU`. bricks writes the printed form directly, as text, to the file `abend.log` in the log directory named by `log_location` in `bricks.cnf` (`log/abend.log` by default). The abend analyser CAID reads it ("CAID: The Abend Analyser").

Every dump is also announced in the log by the IBM message DFHDU0203I. Log lines are wrapped to fit the 78-column console:

```
DFHDU0203I 10/23/26 11:16:56 BRICKS01 A transaction
  dump was taken for dumpcode: AEIM, Tranid: ABNC,
  Tranum: 6, Program: abnc.cob.
```

The Tranum is the task number. It is the number to type in CAID, and it is the same number that DFHAC2236 gives as the transaction number and that a program reads in EIBTASKN.

***When a dump is taken***

The dump is taken when the abend is raised, before HANDLE ABEND is searched, so a task whose exit catches the abend has a dump all the same. An abend that travels up the LINK chain is dumped once, where it was raised.

| Abend | Dump |
|---|---|
| EXEC CICS ABEND ABCODE(*code*) | Yes, unless NODUMP is coded. |
| EXEC CICS ABEND without ABCODE (AAAA) | No. |
| An unhandled condition in COBOL or PL/I (AEIx, AEYx, AEXx) | Yes. |
| A program check (ASRA): a COBOL division by zero or addressing exception, an internal failure of the REXX or COBOL interpreter | Yes. |
| A PL/I runtime condition (ABZD, ABSZ, ABCV and the others of "Runtime Abends" in Chapter 33) and APLI | Yes. |
| A runaway task (AICA) | Yes. |
| ABM0, ABMB, AEXZ, APCT, ADBU, AD2S, AD2P | Yes. |
| ATNI | The first one of a task only. |
| A refused commit (ASP3, ASP7, ASPR) | No. |
| An abend requested with **F12** in CEDF | No. |

**Figure 16-9. Abends that take a transaction dump**

One task can have several dumps: an abend caught by a `HANDLE ABEND` exit followed by a second abend, or an abend whose `HANDLE ABEND PROGRAM` exit cannot be loaded (the original abend, then `APCT`).

***The dump data set***

- Each dump is appended to `abend.log` in one piece, so the dumps of two tasks that abend at the same time never interleave. The directory is created when the first dump is written.
- When `abend.log` has reached 32 MiB, it is renamed `abend.log.1` (replacing an older one) before the next dump is written, and a new `abend.log` is started. The data set therefore holds between 32 and 64 MiB of the most recent dumps. CAID reads both files.
- The file is created with permissions `0640`, because it holds program data: the COMMAREA, working storage and the operator's input. For the same reason CAID requires the `admin` group.
- With `log_location=off` no `abend.log` is written; the `DFHDU0203I` message is still logged.
- If a dump cannot be written (a full disk, a directory without write permission), the abend carries on unchanged and the error is logged, at most once a minute.
- Under MRO the dump is written by the region that ran the program. The dump of a program routed to another region is in that region's `abend.log`, and CAID must be run there.

***Record format***

`abend.log` is plain ASCII text. Every line begins with a 13-column prefix: the transaction identifier, left-justified in 4 columns, a blank, the task number as 7 digits with leading zeros, and a blank. The dumps of one task can therefore be found with `grep` as well as with CAID:

```
grep '^ABNC 0000006 ' log/abend.log
```

After the prefix comes a 4-character record tag, a blank and the data. A dump begins with a `DUMP BEGIN` line and ends with a `DUMP END` line:

| Tag | Contents |
|---|---|
| DUMP | BEGIN DUMPID=*nnnn* RUN=*yyyymmdd-hhmmss*: the dump's number, counted from 0001 in each run of the region, and the time the region was started. END LINES=*n*: the number of lines between. |
| HDR | One *keyword value* line each, in this order, an empty value being left out: CODE (the abend code), TRANID, TASKNO, TERMID, USERID, APPLID, DATE, TIME, ABSTIME, PROGRAM, LANG (REXX, COBOL, PLI), LEVEL (the LINK level, 0 for the first program), FILE and LINE (where the program failed), PROC and STMT (PL/I: the procedure and the statement), DETAIL (the text of the Detail: row), MSG (more detail, such as the PL/I condition), and for AICA ICVR and RAN (the limit and the time the task ran, in milliseconds). |
| EIB | The 30 fields of the EIB, one per line, formatted as the CEDF EIB display shows them ("The Auxiliary Displays" in Chapter 8). |
| CMD | The command that failed, if any: the verb, the options in alphabetical order, then the keywords. A variable argument of an option such as FILE, RIDFLD, QUEUE or LENGTH is shown with its value: RIDFLD(WS-KEY='C9999'). A failing EXEC SQL statement is shown as EXEC SQL and its text. |

| Tag | Contents |
|---|---|
| LINK | The LINK chain, one line per level from level 00: the program identifier (the transaction identifier at level 00) and the source file. * marks the level that abended. |
| SRC | FILE *name* LINES *m-n*, then the source lines around the failing statement, numbered, with the lines of the statement marked >: 5 lines before it (25 for AICA, to include the head of the loop) and 3 after. |
| VAR | The program's variables: nesting depth, level number (-- for a REXX variable), name, attributes, and the value as text in quotes and in hexadecimal. A group or structure has no value; an item with no storage (a LINKAGE item whose address is not set) shows (NO STORAGE). |
| STG | Storage areas in the IBM dump layout: an AREA *name* LEN=*n* line, then 16 bytes a line: the offset, four groups of four bytes in hexadecimal, and the characters between asterisks. The COMMAREA is always shown (LEN=0 when there is none), followed by the TWA when the task has one and the program's GETMAIN areas (COBOL). |

| Tag | Contents |
|---|---|
| TIOA | The terminal input/output area: the map last sent to the terminal (MAP *mapset.map*, with the AID and the cursor position of the operator's last reply) and the value of each field of that reply; NONE when no map has been sent. The map may have been sent by an earlier task at the terminal, as in a pseudo-conversation. |
| NOTE | What the dump leaves out: SOURCE NOT AVAILABLE, VARIABLES NOT AVAILABLE, MORE GETMAIN AREAS NOT SHOWN, DUMP TRUNCATED AT 6000 LINES, and so on. |

**Figure 16-10. Records of a transaction dump**

The following is part of the dump of the PL/I sample ABZP, which divides by a variable that is zero; most of the EIB lines are left out. ABZP sends no map; the TIOA line shows the map that the transaction run before it at the same terminal had sent:

```
ABZP 0000005 DUMP BEGIN DUMPID=0001 RUN=20261023-111645
ABZP 0000005 HDR  CODE     ABZD
ABZP 0000005 HDR  TRANID   ABZP
ABZP 0000005 HDR  TASKNO   0000005
ABZP 0000005 HDR  TERMID   T0001
ABZP 0000005 HDR  USERID   DEMO
ABZP 0000005 HDR  APPLID   BRICKS01
ABZP 0000005 HDR  DATE     2026-10-23
ABZP 0000005 HDR  TIME     11:16:54
ABZP 0000005 HDR  ABSTIME  004001735814450
ABZP 0000005 HDR  PROGRAM  abzp.pli
ABZP 0000005 HDR  LANG     PLI
ABZP 0000005 HDR  LEVEL    0
ABZP 0000005 HDR  FILE     abzp.pli
ABZP 0000005 HDR  LINE     10
ABZP 0000005 HDR  PROC     AVERAGE
ABZP 0000005 HDR  STMT     AVG = TOTAL / COUNT ;
ABZP 0000005 HDR  DETAIL   ABZP.AVERAGE L10
ABZP 0000005 HDR  MSG      ZERODIVIDE
ABZP 0000005 EIB  EIBTIME  091654
ABZP 0000005 EIB  EIBDATE  0126296
ABZP 0000005 EIB  EIBTRNID 'ABZP'
ABZP 0000005 EIB  EIBTASKN 5
   ...
ABZP 0000005 EIB  EIBRESP  0 NORMAL
ABZP 0000005 EIB  EIBRESP2 0
ABZP 0000005 EIB  EIBRLDBK X'00'
ABZP 0000005 LINK 00 ABZP abzp.pli *
ABZP 0000005 SRC  FILE abzp.pli LINES 5-12
ABZP 0000005 SRC  00005      DCL AVG    FIXED DEC(7,2) INIT(0);
ABZP 0000005 SRC  00006      CALL AVERAGE;
ABZP 0000005 SRC  00007      EXEC CICS RETURN;
ABZP 0000005 SRC  00008   AVERAGE: PROC;
ABZP 0000005 SRC  00009      DCL I FIXED BIN(15) INIT(3);
ABZP 0000005 SRC  00010 >    AVG = TOTAL / COUNT;
ABZP 0000005 SRC  00011   END AVERAGE;
ABZP 0000005 SRC  00012   END ABZP;
ABZP 0000005 VAR  0 01 I BIN(15) = '3' X'0003'
ABZP 0000005 VAR  0 01 TOTAL DEC(7,2) = '250.00' X'0025000C'
ABZP 0000005 VAR  0 01 COUNT BIN(31) = '0' X'00000000'
ABZP 0000005 VAR  0 01 AVG DEC(7,2) = '0.00' X'0000000C'
ABZP 0000005 STG  AREA COMMAREA LEN=0
ABZP 0000005 TIOA MAP NQUE1.NQUE1 AID=ENTER CURSOR=0
ABZP 0000005 DUMP END LINES=64
```

A storage area with data is shown 16 bytes a line, as in this COMMAREA of 110 bytes. A run of lines equal to the line before them is collapsed into one `SAME AS ABOVE` line that gives the offsets of the first and last byte of the run. (The lines are shown here without their 13-column prefix, as the CAID dump display shows them.)

```
STG  AREA COMMAREA LEN=110
STG  00000000  43313030 31202020 41434D45 20544F4F  *C1001   ACME TOO*
STG  00000010  4C53204C 54442020 20202020 20202020  *LS LTD          *
STG  00000020  20202020 20202020 20202020 20202020  *                *
STG  00000030  LINES 00000030-0000005F SAME AS ABOVE
STG  00000060  30303031 32353030 30304F50 454E      *00012500000PEN  *
```

***Dates, times and task numbers***

- `HDR DATE` and `TIME` are the local time of the host, like the time stamps of the log. `EIBDATE` and `EIBTIME` are the values the program saw, which follow `time_zone` in `bricks.cnf`; the two times differ when the host is not in that zone (above, the host is at UTC+2 and `time_zone=Z`). `ABSTIME` is in milliseconds since 1900-01-01 UTC, as `ASKTIME ABSTIME` returns it.
- Task numbers run from 1 to 9999999 and then start again at 1, because `EIBTASKN` is a 7-digit field (`PIC S9(7)` in IBM CICS). They also start again at 1 when the region is restarted, so a task number can have several dumps in `abend.log`; `RUN` and `DUMPID` tell them apart.

***Limits***

A dump is bounded however large the program is: at most 2000 variables, 50 elements of each REXX stem, 60 characters and 32 bytes of hexadecimal per value, 4096 bytes per storage area, 8 `GETMAIN` areas, 100 terminal input fields, 200 characters per source line, statement or detail, 6 marked source lines per statement, and 6000 lines in all. A value that was cut is followed by `+`; a storage area that was cut ends with `TRUNCATED` *n* `BYTES NOT SHOWN`. Characters outside the printable ASCII range are shown as a period.

***Differences from IBM CICS***

- The dump is written as text, already formatted; there is no `DFHDU` step and no dump data set switching (`CEMT SET DUMPDS`). The dump holds the program's variables by name and its source, which an IBM dump does not, and no CICS control blocks or registers.
- There is no transaction dump table: dumping cannot be switched off for a single code, except by `NODUMP` on `EXEC CICS ABEND`.
- `EXEC CICS DUMP TRANSACTION` is not supported (Chapter 19).

### Runaway Tasks

Like IBM CICS with the system initialization parameter `ICVR=2000`, bricks abends a task with code `AICA` when it runs for **2 seconds** without giving control back to the region. The interval is fixed: there is no `bricks.cnf` setting and no way to switch the check off. It applies to REXX, COBOL and PL/I, and to every kind of task: terminal, `LINK`ed, `START`ed, routed to another region, and web.

#### *What gives control back*

A command that **yields** restarts the 2-second interval when it is issued and again when it returns, so the time spent inside it is never counted. Every `EXEC CICS` command yields except those listed below: terminal I/O (including the wait for the operator), `DELAY`, file, TS and TD commands, `LINK` and `XCTL`, `SYNCPOINT`, `START`, `RETRIEVE`, `CANCEL`, and an `ENQ` that had to wait. Every `EXEC SQL` statement also yields (a bricks decision: the statement waits on the database). The interval also starts again just before each program starts, so loading the program is not counted.

As in IBM CICS, the following commands do **not** yield, so a loop that issues only them is still a runaway: `HANDLE` (`CONDITION`, `AID`, `ABEND`), `IGNORE CONDITION`, `ASKTIME`, `FORMATTIME`, `ASSIGN`, `ADDRESS`, `GETMAIN`, `FREEMAIN`, `DEQ`, and an `ENQ` that obtained the resource at once.

```
DO FOREVER                       /* abended AICA after 2 seconds   */
  EXEC CICS ASKTIME END-EXEC     /* does not yield                 */
END

DO FOREVER                       /* runs until stopped, never AICA */
  EXEC CICS DELAY FOR MILLISECS(100) END-EXEC
END
```

***When the interval expires***

- A transaction dump is taken ("Transaction Dumps"); its source window reaches back 25 lines to show the head of the loop. The log has the dump message and a detail line that gives the terminal, program, source line, runtime and (PL/I) the statement.
- `EIBABCODE` and `ASSIGN ABCODE` become `AICA`, and `HANDLE ABEND` is searched as for any abend. A `LABEL` exit receives control with nothing backed out and a new 2-second interval; a `PROGRAM` exit is linked to and the abend then completes. REXX `SIGNAL ON SYNTAX` does not catch `AICA`.
- Unhandled, the task ends as described in "How an Abend Ends a Task". The detail row reads, for example, `Detail: RUNAWAY TASK – loop.rexx LINE 4`.

A program reached by `LINK` that loops makes the task a runaway: the abend reaches the caller (and its `HANDLE ABEND`), never a normal return from the `LINK`. A started task that loops is ended the same way; until then it delays only later started tasks for the same terminal, not other terminals.

***Differences from IBM CICS***

IBM CICS measures the processor time of the task. bricks cannot measure one task's processor time, so it measures elapsed time between yielding commands. The two agree for a loop that computes; they differ only when the host is so busy that a task waits for a processor, which bricks counts and IBM CICS does not.

**Note:**

Operator commands such as those of CECI have their own limit of 7 seconds for one command. A command typed in CECI is not a program; a program it starts is abended `AICA` after 2 seconds, well within the 7.

## CAID: The Abend Analyser

The supplied transaction `CAID` finds the dumps of an abended task in `abend.log` ("Transaction Dumps"), works out the probable cause of the abend from the dump, and shows it with the values that prove it: the variable that held zero, the key that was not on file, the loop that never ended. When it cannot name the cause, it lists the typical causes of the abend code. It can also show the dump itself.

CAID requires the `admin` group, because a dump holds program data. Any other user is refused with `TRANSID "CAID": access denied -- requires admin group` (Figure 16-11). The same rule applies to a `LINK` or `XCTL` to CAID.

```
BRICKS Transaction Server                                TERM=T0002


TRANSID "CAID": access denied -- requires admin group









Press ENTER to continue.
```

**Figure 16-11. CAID refused to a user who is not an administrator** (Model 2, 24 x 80; capture screens/caid-denied.txt)

### *Starting CAID*

```
CAID [tasknumber]
```

Type `CAID` at the blank screen to display the input screen. Type `CAID` followed by a task number, for example `CAID 6`, to go straight to the analysis of that task's newest dump; **`F3`** then returns to the input screen. The task number is the `Tranum` of message `DFHDU0203I` and the transaction number of `DFHAC2236`, 1 to 7 digits; leading zeros may be omitted.

### *The input screen*

Figure 16-12 shows the input screen after the samples ABZP, ABNC and ABUR have abended.

```
CAID  Task number ===>                                  TERM=T0001  11:24:25
S TASK     TRAN  CODE  PROGRAM       DATE        TIME      DUMPID
  0000007  ABUR  USR1  abur.rexx     2026-10-23  11:24:24  0003
  0000006  ABNC  AEIM  abnc.cob      2026-10-23  11:24:22  0002
  0000005  ABZP  ABZD  abzp.pli      2026-10-23  11:24:20  0001









ENTER=Analyse  PF3=Exit
```

**Figure 16-12. The CAID input screen with the most recent dumps** (Model 2, 24 x 80; capture screens/caid-input.txt)

**Row 1**

The title `CAID`, the input field `Task number ===>` (7 digits, the cursor is placed in it), the terminal identifier and the time.

**Row 2**

The headings of the list of recent dumps.

**Rows 3-22**

The most recent dumps in `abend.log` and `abend.log.1`, newest first, as many as the screen holds (20 on a Model 2): the task number, the transaction identifier, the abend code, the program, the date and time of the dump, and its `DUMPID`. In front of each row, in column `S`, is a one-character selector field. `No dumps in abend.log yet.` is shown when there are none.

**Row 23**

A message (see "Messages").

**Row 24**

The key legend.

There are two ways to choose a dump. Type a task number and press **ENTER** to analyse the newest dump of that task. Or type any character in the selector field in front of a row of the list and press **ENTER** to analyse exactly that dump; this is the way to reach an older dump of a task number that has been used again since the region was restarted. A selection takes precedence over a typed task number. If several rows are selected, the first is opened and the analysis screen says `Several dumps selected – showing the first.` In Figure 16-13 the dump of task 6 is selected; **ENTER** displays the analysis of Figure 16-15.

```
 CAID  Task number ===>                                    TERM=T0001  11:24:30
 S TASK     TRAN  CODE  PROGRAM       DATE        TIME      DUMPID
   0000007  ABUR  USR1  abur.rexx     2026-10-23  11:24:24  0003
 x 0000006  ABNC  AEIM  abnc.cob      2026-10-23  11:24:22  0002
   0000005  ABZP  ABZD  abzp.pli      2026-10-23  11:24:20  0001









 ENTER=Analyse  PF3=Exit
```

**Figure 16-13. A dump selected in the list of recent dumps** (Model 2, 24 x 80; capture screens/caid-select.txt)

**F3**, **CLEAR** and the PA keys end CAID. Any other key displays the screen again.

***The analysis screen***

Figure 16-14 shows the analysis of the dump of the PL/I sample ABZP, which divides by a variable that is zero. The diagnosis names the procedure, the line and the divisor, and the values show why: `COUNT` is 0. The EIB follows; **F8** shows the rest of it and the COMMAREA.

```
CAID ANALYSIS                                              TERM=T0001  11:24:26
Abend ABZD  Tran ABZP  Task 0000005  Program abzp.pli (PLI)  Dump 0001
Taken 2026-10-23 11:24:20  Term T0001  User DEMO  Match 1 of 1
Probable cause:
ZERODIVIDE in procedure AVERAGE at line 10 of abzp.pli:
    AVG = TOTAL / COUNT;
The divisor COUNT is 0 when the statement runs.
Test COUNT for 0 before the division, or make sure the program sets it first.
Values on the failing line:
  AVG              DEC(7,2)      0.00
  TOTAL            DEC(7,2)      250.00
  COUNT            BIN(31)       0
Link chain: ABZP abzp.pli (level 0)
EIB:
EIBTIME  092420           EIBDATE  0126296          EIBTRNID 'ABZP'
EIBTASKN 5                EIBTRMID 'T000'           EIBCPOSN 0
EIBCALEN 0                EIBAID   X'7D' ENTER      EIBFN    X'0000'
EIBRCODE X'000000000000'  EIBDS    '        '       EIBREQID '        '
EIBRSRCE '        '       EIBSYNC  X'00'            EIBFREE  X'00'
EIBRECV  X'00'            EIBSEND  X'00'            EIBATT   X'00'
EIBEOC   X'00'            EIBFMH   X'00'            EIBCOMPL X'00'
EIBSIG   X'00'            EIBCONF  X'00'            EIBERR   X'00'
Lines 1-19 of 22
PF3=Back  PF5=Dump  PF7=Up  PF8=Down  PF10=Older  PF11=Newer  PF12=Exit
```

**Figure 16-14. CAID analysis of a ZERODIVIDE abend (ABZP)** (Model 2, 24 x 80; capture screens/caid-zerodivide.txt)

**Row 1**

The title `CAID ANALYSIS`, the terminal identifier and the time.

**Row 2**

The abend code, the transaction identifier, the task number, the program and its language, and the `DUMPID`.

**Row 3**

When the dump was taken, the terminal and user of the task, and `Match` *k* `of` *n*: this is dump *k*, counted from the oldest, of the *n* dumps found for the task number. `(incomplete)` follows when the dump has no `DUMP END` line, for example because the region stopped while writing it.

**Rows 4-22**

The analysis, which scrolls with **`F7`** and **`F8`**:

- `Probable cause:` and the diagnosis, with the failing statement, indented, as it is in the source;
- `Values on the failing line:` the name, attributes and value of each variable named in the failing statement (at most 12), or `none`;
- `Link chain:` each `LINK` level with its program and source file;
- `EIB:` every EIB field of the dump, three to a row;
- `COMMAREA:` the COMMAREA in hexadecimal and characters, as in the dump, or `COMMAREA: none (EIBCALEN 0)`.

**Row 23**

`Lines` *m*-*n* `of` *t*: the lines of the analysis on the screen, or a message.

**Row 24**

The key legend.

Figure 16-15 shows the analysis of the COBOL sample ABNC, which reads a key that is not on file without coding `RESP`. The diagnosis names the command, the key and the file, gives `EIBRESP` and `EIBFN`, shows the command as it is written in the program, and says how to avoid the abend.

```
CAID ANALYSIS                                              TERM=T0001  11:24:28
Abend AEIM  Tran ABNC  Task 0000006  Program abnc.cob (COBOL)  Dump 0002
Taken 2026-10-23 11:24:22  Term T0001  User DEMO  Match 1 of 1
Probable cause:
READ found no record with key 'NOSUCHKEY' in file customers.
EIBRESP 13 NOTFND, EIBFN X'0602' READ, at line 17 of abnc.cob:
    EXEC CICS READ FILE('customers') INTO(REC)
              RIDFLD('NOSUCHKEY') END-EXEC.
The command coded neither RESP nor NOHANDLE and no HANDLE CONDITION covered
NOTFND, so CICS abended the task with AEIM.
Code RESP(var) and test it for DFHRESP(NOTFND), or check that the key exists
before the READ.
Values on the failing line:
  REC             X(80)         ''
Link chain: ABNC abnc.cob (level 0)
EIB:
EIBTIME  092422           EIBDATE  0126296          EIBTRNID 'ABNC'
EIBTASKN 6                EIBTRMID 'T000'           EIBCPOSN 0
EIBCALEN 0                EIBAID   X'7D' ENTER      EIBFN    X'0602' READ
EIBRCODE X'000000000000'  EIBDS    'customer'       EIBREQID '        '
EIBRSRCE 'customer'       EIBSYNC  X'00'            EIBFREE  X'00'
EIBRECV  X'00'            EIBSEND  X'00'            EIBATT   X'00'
Lines 1-19 of 24
PF3=Back  PF5=Dump  PF7=Up  PF8=Down  PF10=Older  PF11=Newer  PF12=Exit
```

**Figure 16-15. CAID analysis of an unhandled NOTFND (ABNC, abend AEIM)** (Model 2, 24 x 80; capture screens/caid-aeim.txt)

When no rule of the analyser applies, CAID says `Unable to analyse the cause, but these are typical causes:` and lists the typical causes of the abend code, followed by `Where it failed:` with the source line and statement. Figure 16-16 shows this for the REXX sample ABUR, which ends itself with `EXEC CICS ABEND ABCODE('USR1')`: an abend code that the application defines, whose meaning no rule can know.

```
CAID ANALYSIS                                              TERM=T0001  11:24:30
Abend USR1  Tran ABUR  Task 0000007  Program abur.rexx (REXX)  Dump 0003
Taken 2026-10-23 11:24:24  Term T0001  User DEMO  Match 1 of 1
Unable to analyse the cause, but these are typical causes:
  - USR1 is not a CICS abend code. The program ended itself with EXEC CICS
    ABEND ABCODE('USR1'); its meaning is defined by the application. Look for
    that ABCODE in the program source.
  - The program detected an error it could not recover from (a bad response
    code, invalid input, a missing record) and abended on purpose so that the
    unit of work is backed out.
  - HANDLE ABEND was not active, or the exit itself issued the ABEND.
Where it failed: line 12 of abur.rexx
    EXEC CICS ABEND ABCODE('USR1') END-EXEC
Values on the failing line: none
Link chain: ABUR abur.rexx (level 0)
EIB:
EIBTIME  092424           EIBDATE  0126296          EIBTRNID 'ABUR'
EIBTASKN 7                EIBTRMID 'T000'           EIBCPOSN 0
EIBCALEN 0                EIBAID   X'7D' ENTER      EIBFN    X'0E0C' ABEND
EIBRCODE X'000000000000'  EIBDS    '        '       EIBREQID '        '
EIBRSRCE '        '       EIBSYNC  X'00'            EIBFREE  X'00'
EIBRECV  X'00'            EIBSEND  X'00'            EIBATT   X'00'
Lines 1-19 of 24
PF3=Back  PF5=Dump  PF7=Up  PF8=Down  PF10=Older  PF11=Newer  PF12=Exit
```

**Figure 16-16. CAID cannot name the cause of a user abend code (ABUR, abend USR1)** (Model 2, 24 x 80; capture screens/caid-fallback.txt)

### *The dump display*

**F5** on the analysis screen displays the dump itself, as it is in `abend.log` without the 13-column prefix, under the title `CAID DUMP` (Figure 16-17). **F7** and **F8** scroll it; **F5** or **F3** returns to the analysis.

```
CAID DUMP                                                  TERM=T0001  11:24:27
Abend ABZD  Tran ABZP  Task 0000005  Program abzp.pli (PLI)  Dump 0001
HDR  CODE     ABZD
HDR  TRANID   ABZP
HDR  TASKNO   0000005
HDR  TERMID   T0001
HDR  USERID   DEMO
HDR  APPLID   BRICKS01
HDR  DATE     2026-10-23
HDR  TIME     11:24:20
HDR  ABSTIME  004001736260484
HDR  PROGRAM  abzp.pli
HDR  LANG     PLI
HDR  LEVEL    0
HDR  FILE     abzp.pli
HDR  LINE     10
HDR  PROC     AVERAGE
HDR  STMT     AVG = TOTAL / COUNT ;
HDR  DETAIL   ABZP.AVERAGE L10
HDR  MSG      ZERODIVIDE
EIB  EIBTIME  092420
EIB  EIBDATE  0126296
Lines 1-20 of 64
PF3=Back  PF5=Diagnosis  PF7=Up  PF8=Down  PF12=Exit
```

**Figure 16-17. The dump of ABZP displayed by CAID** (Model 2, 24 x 80; capture screens/caid-dump.txt)

### *Function keys*

| Key | Screen | Action |
|---|---|---|
| ENTER | Input | Analyse the selected dump, or else the newest dump of the task number typed. |
| F3 | Input | End CAID. |
| F3 | Analysis | Return to the input screen. |
| F3 | Dump | Return to the analysis. |
| F5 | Analysis, dump | Switch between the analysis and the dump. |
| F7, F8 | Analysis, dump | Scroll back or forward one screen; the last line of a screen is the first of the next. |
| F10, F11 | Analysis | Show the next older or newer dump of the same task number. |
| F12 | Analysis, dump | End CAID. |
| CLEAR, PA1–PA3 | Any | End CAID. |

**Figure 16-18. CAID function keys**

***Messages***

Figure 16-19 shows the input screen after a task number that has no dump was typed.

```
CAID  Task number ===> 9999999                          TERM=T0001  11:24:32
S TASK     TRAN  CODE  PROGRAM       DATE        TIME      DUMPID
  0000007  ABUR  USR1  abur.rexx     2026-10-23  11:24:24  0003
  0000006  ABNC  AEIM  abnc.cob      2026-10-23  11:24:22  0002
  0000005  ABZP  ABZD  abzp.pli      2026-10-23  11:24:20  0001









No dump for task 9999999 in abend.log.
ENTER=Analyse  PF3=Exit
```

**Figure 16-19. CAID: no dump for the task number typed** (Model 2, 24 x 80; capture screens/caid-notfound.txt)

| Message | Meaning |
|---|---|
| `No dump for task` *nnnnnnn* `in abend.log.` | Neither `abend.log` nor `abend.log.1` holds a dump of that task: the task did not abend, its abend took no dump, or the dump has been rotated out. |
| `Task number must be 1 to 7 digits.` | The task number is empty or not a number. |
| `No abend.log yet: no abend has been dumped.` | Neither `abend.log` nor `abend.log.1` exists in the `log_location` directory: no abend has been dumped there yet. |
| `Abend dumps are not written: log_location is off.` | `log_location=off` in `bricks.cnf`. CAID has nothing to read. |
| `Search stopped after 6 seconds: abend.log is too large.` | All reading of `abend.log` for one press of a key is limited to 6 seconds, so that CAID answers within the 7 seconds allowed for an operator command. |
| `abend.log could not be read.` | The file exists but cannot be opened or read (for example, its permissions). |
| `No older dump for task` *nnnnnnn*`. No newer dump for task` *nnnnnnn*`.` | **`F10`** or **`F11`** pressed on the oldest or newest dump of the task. |
| `Several dumps selected – showing the first.` | More than one row of the list was selected. |

**Figure 16-20. CAID messages**

### *What CAID diagnoses*

The analyser has a rule for each abend code below. A rule names the cause only when the dump holds what it needs (for example, the source and the variables); otherwise CAID shows the typical causes of the code.

| Abend | What the diagnosis says |
|---|---|
| ABZD | PL/I ZERODIVIDE: the procedure, the statement, and which divisor in it is zero, with the values of the variables in that divisor. |
| ASRA | COBOL division by zero (IBM S0CB): the divisor of the DIVIDE or COMPUTE that is zero, and the advice to code ON SIZE ERROR. Addressing exception: DFHCOMMAREA used with EIBCALEN 0 (the first run of a pseudo-conversational transaction), or the LINKAGE item whose address was never set. Internal error: a bricks failure, with the time to look for in the region log. |
| AEIx, AEYx, AEXx | The condition from EIBRESP, the command and its resource: for NOTFND the key and the file, for DUPREC the key, for QIDERR and ITEMERR the queue and item, for PGMIDERR and TRANSIDERR the program or transaction, for NOTAUTH the user and the resource, and for ENDFILE the browse loop that does not test for it. It then explains that no RESP, NOHANDLE or HANDLE CONDITION covered the condition. |
| AICA | How long the task ran against the limit, the statement it was at, the loop it is probably in (a DO, PERFORM ... UNTIL or GO TO up to 25 lines before), and the values of the variables in the loop's condition. |
| ABSR | The subscript, the array and its bounds; or a SUBSTR outside its string. |

| Abend | What the diagnosis says |
|---|---|
| ABSZ | The target that the value does not fit, with its attributes; or the DO control variable that overflowed. |
| ABCV | The character variable whose value is not a valid number. |
| ABFO | The FIXEDOVERFLOW detail and the statement. |
| ABSL | The SELECT expression and its value, which no WHEN matched. |
| ABEN, ABHB, ABST, APLI | A procedure entered again that is not RECURSIVE, a function that reached its END without RETURN, the task storage or call depth exhausted, or a program or entry that was not compiled. |
| ABMB | The cursor position and the last position of the screen. |
| ABM0 | The map and the mapset that does not contain it. |
| APCT | The HANDLE ABEND PROGRAM exit that could not be loaded, and the original abend code. |
| ATNI | The command during which the terminal was lost; not a program error. |
| ADBU, AD2S, AD2P | The database failure, what to check in bricks.cnf, and the SQL statement. |
| AEXZ | The command inside which bricks failed; the Go stack is in the region log. |

**Figure 16-21. Abends that CAID diagnoses**

For every other code, and in particular for a code that the application chose with EXEC CICS ABEND ABCODE, CAID shows the typical causes. For a code that is not a CICS abend code it says so, and suggests looking for the ABCODE in the program source (Figure 16-16).

***Differences from IBM CICS***

CAID is a bricks transaction. IBM CICS has no supplied transaction that analyses a transaction dump; the IBM tools that do (such as the IBM Fault Analyzer) are separate products.

Chapter 17

# The Execute Interface Block

The execute interface block (EIB) holds information about the task and about the most recent `EXEC CICS` command. bricks fills it in when a program starts and updates it after every command. A program reads it; it does not change it.

### How Each Language Sees the EIB

**COBOL**

The EIB is the group `DFHEIB`, with the IBM `DFHEIBLK` fields in the IBM order and formats (see "The EIB in COBOL"). The compiler supplies it when the program declares neither `DFHEIB` nor any of its field names. It also supplies `RC`, a copy of `EIBRESP`.

**PL/I**

The EIB is the predeclared structure `DFHEIBLK`, with fixed PL/I types ("The EIB in PL/I"). PL/I has no automatic `RC`; a program declares its own variable for `RESP`.

**REXX**

Each EIB field is an ordinary variable with the field's name. Numeric fields hold decimal digits; `EIBAID` holds one character, tested with `C2X(EIBAID)`. `RC` is a copy of `EIBRESP`.

`ASSIGN` returns several EIB values as well (`EIBTRMID`, `EIBAID`, `EIBCPOSN`, `EIBCALEN`, `EIBTASKN`), and also the full terminal identifier and the abend code; see "ASSIGN" in Chapter 13.

## EIB Fields

| Field | Set | Contents |
|---|---|---|
| EIBAID | SEND MAP, SEND TEXT, RECEIVE MAP, CONVERSE | The attention identifier of the operator's most recent key: one byte, such as X'7D' for **ENTER** or X'F3' for **F3**. Compare with the names in DFHAID. |
| EIBCPOSN | as EIBAID | The cursor position when the key was pressed, as a buffer offset counted from 0: row × columns + column, with rows and columns counted from 0. On a 24 × 80 screen the range is 0-1919. It is on the same scale as CURSOR(n). |
| EIBCALEN | program start | The length of the COMMAREA passed to the program; 0 when none was passed. |
| EIBTRMID | program start | The terminal identifier. REXX receives the full identifier (T0001); the COBOL and PL/I field is 4 characters, as in IBM CICS, and holds T000. ASSIGN TERMID into an 8-character area returns the full identifier. |
| EIBTRNID | program start | The transaction identifier the task was started with. A program reached by LINK sees its caller's. |
| EIBTASKN | program start | The task number. |
| EIBDATE | program start, ASKTIME | The date in the form 0CYYDDD: century (0 for 19xx, 1 for 20xx), year and day of the year. |
| EIBTIME | program start, ASKTIME | The time of day in the form HHMMSS. |
| EIBFN | every command | A two-byte code that identifies the most recent command, as in IBM CICS. |
| EIBDS | file commands | The file named by the most recent file control command (first 8 characters). |
| EIBRSRCE | commands that name a resource | The file, queue, program, map, transaction or ENQ resource named by the most recent command that named one (first 8 characters). |

| Field | Set | Contents |
|---|---|---|
| EIBREQID | commands with REQID | The request identifier of the most recent command that coded REQID. |
| EIBRESP | every command | The response of the most recent command (Chapter 18). |
| EIBRESP2 | every command | The secondary response of the most recent command. It is 0 except where a command documents a value. |
| EIBABCODE | abend | REXX and PL/I: the current abend code, read in a HANDLE ABEND exit. COBOL has no such field; use ASSIGN ABCODE. |
| RC | every command | REXX and COBOL: a copy of EIBRESP. |

**Figure 17-1. EIB fields that bricks sets**

The other fields of the IBM EIB (EIBRCODE, EIBSYNC, EIBFREE, EIBRECV, EIBATT, EIBEOC, EIBFMH, EIBCOMPL, EIBSIG, EIBCONF, EIBERR, EIBERRCD, EIBSYNRB, EIBNODAT, EIBRLDBK) exist in the COBOL EIB so that programs that refer to them compile, but bricks leaves them at zero or blank.

Because every command sets EIBRESP again, a program that needs the response of a command after issuing another one must save it first (or code RESP). This matters in a HANDLE ABEND exit, where even ASSIGN replaces the response of the command that failed.

## The EIB in COBOL

The compiler supplies the following layout, the IBM DFHEIBLK. A program that walks the EIB by address sees the IBM offsets.

```
       01  DFHEIB.
           02  EIBTIME     PIC S9(7)  COMP-3.
           02  EIBDATE     PIC S9(7)  COMP-3.
           02  EIBTRNID    PIC X(4).
           02  EIBTASKN    PIC S9(7)  COMP-3.
           02  EIBTRMID    PIC X(4).
           02  EIBFIL01    PIC X(2).
           02  EIBCPOSN    PIC S9(4)  COMP.
           02  EIBCALEN    PIC S9(4)  COMP.
           02  EIBAID      PIC X.
           02  EIBFN       PIC X(2).
           02  EIBRCODE    PIC X(6).
           02  EIBDS       PIC X(8).
           02  EIBREQID    PIC X(8).
           02  EIBRSRCE    PIC X(8).
           02  EIBSYNC     PIC X.
           02  EIBFREE     PIC X.
           02  EIBRECV     PIC X.
           02  EIBSEND     PIC X.
           02  EIBATT      PIC X.
           02  EIBEOC      PIC X.
           02  EIBFMH      PIC X.
           02  EIBCOMPL    PIC X.
           02  EIBSIG      PIC X.
           02  EIBCONF     PIC X.
           02  EIBERR      PIC X.
           02  EIBERRCD    PIC X(4).
           02  EIBSYNRB    PIC X.
           02  EIBNODAT    PIC X.
           02  EIBRESP     PIC S9(8)  COMP.
           02  EIBRESP2    PIC S9(8)  COMP.
           02  EIBRLDBK    PIC X.
```

The compiler also supplies `RC PIC 9(8)` and, when the program does not declare it, `DFHCOMMAREA` as a 2000-byte area (see "The COMMAREA" in Chapter 25).

```
           IF EIBAID = DFHPF3
              EXEC CICS RETURN END-EXEC
           END-IF.
```

### The EIB in PL/I

Every PL/I program has the following structure predeclared. It holds the fields a PL/I program can use; `EIBABCODE` is a bricks addition.

```
DCL 1 DFHEIBLK,
      2 EIBTIME   FIXED DEC(7),
      2 EIBDATE   FIXED DEC(7),
      2 EIBTRNID  CHAR(4),
      2 EIBTASKN  FIXED DEC(7),
      2 EIBTRMID  CHAR(4),
      2 EIBCPOSN  FIXED BIN(15),
      2 EIBCALEN  FIXED BIN(15),
      2 EIBAID    CHAR(1),
      2 EIBFN     CHAR(2),
      2 EIBRESP   FIXED BIN(31),
      2 EIBRESP2  FIXED BIN(31),
      2 EIBABCODE CHAR(4);
```

A PL/I program tests responses with its own variable:

```
DCL RC FIXED BIN(31);
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) RESP(RC);
SELECT (RC);
   WHEN (DFHRESP(NORMAL)) ;
   WHEN (DFHRESP(NOTFND)) MSG = 'Customer not found';
   OTHERWISE              MSG = 'I/O error, RESP=' || CHAR(RC);
END;
```

See "EXEC CICS" in Chapter 33 for EXEC CICS in PL/I.

### The EIB in REXX

A REXX program tests the EIB variables like any other variable. All fields of the table above are set before the program's first statement runs, except those set only by a later command.

```
IF EIBRESP <> 0 THEN SAY 'Command failed, RC=' EIBRESP
IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
ROW = EIBCPOSN % 80 + 1                /* 1-based row on 80 columns */
```

`SIGNAL ON ERROR` is an alternative to testing `RC` after each command; see "SIGNAL ON and SIGNAL OFF" in Chapter 39.

### Differences from IBM CICS

- `EIBTRMID` is 4 characters in COBOL and PL/I, so a bricks terminal identifier such as `T0001` is truncated there. Use `ASSIGN TERMID` for the full identifier.
- The communication fields (`EIBSYNC` and the others listed above) are never set.
- REXX sees the EIB as variables rather than as a storage area, and has the additional variables `RC` and `EIBABCODE`.

Chapter 18

# Response Codes and Conditions

This chapter lists the responses of `EXEC CICS` commands and shows how a program tests them in each language. How conditions are handled, and what happens to one that is not, is described in "Conditions" in Chapter 16.

## Testing the Response

After every command, bricks sets `EIBRESP` to the response and `EIBRESP2` to the secondary response. A program can also name its own variables:

```
EXEC CICS command ... RESP(resp) [RESP2(resp2)] END-EXEC
```

`RESP(resp)` stores the response in *resp* and tells bricks that the program tests it itself: the command takes no `HANDLE CONDITION` or `HANDLE AID` branch and never abends on a condition. `NOHANDLE` has the same effect without naming a variable; the response is then in `EIBRESP`. In COBOL, *resp* is a fullword (`PIC S9(8) COMP`); in PL/I, `FIXED BIN(31)`.

The names of the responses are available as constants: `DFHRESP(`*name*`)` in COBOL and PL/I, which the compiler replaces by the number. A REXX program compares with the number. The copybook `DFHRESP` and the include member `DFHRESP` also define every response by name (see "DFHRESP" in Chapter 26).

### *REXX*

A REXX program never abends on a condition, so it can test `RC` (or `EIBRESP`) after any command:

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
SELECT
   WHEN RC = 0  THEN NOP
   WHEN RC = 13 THEN MSG = 'Customer' K 'not found'
   OTHERWISE         MSG = 'I/O error, RC=' RC
END
```

Or it can trap every non-zero response in one place:

```
SIGNAL ON ERROR NAME CICSERR
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
...
EXIT
CICSERR:
   SAY 'CICS error at line' SIGL ', RC=' RC
   EXIT
```

### *COBOL and PL/I*

A command whose response is tested must code RESP. Without it, any response other than NORMAL abends the task before the test runs (NOTFND abends AEIM).

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY)
          RESP(WS-RESP) END-EXEC.
EVALUATE WS-RESP
   WHEN DFHRESP(NORMAL) CONTINUE
   WHEN DFHRESP(NOTFND) MOVE 'Customer not found' TO MSG
   WHEN OTHER           MOVE 'I/O error' TO MSG
END-EVALUATE.
```

EIBRESP holds the same value, so a program that tests EIBRESP works once RESP is coded.

## Responses

The table lists the responses that bricks returns. The **Abend** column gives the abend code of a COBOL or PL/I task that leaves the condition unhandled; *none* means the condition never abends. REXX never abends on a condition.

| Name | RESP | Abend | Meaning in bricks |
|---|---|---|---|
| NORMAL | 0 | - | The command completed. |
| ERROR | 1 | AEIA | The command could not be processed at all, for example because it could not be parsed at runtime. |
| EOC | 6 | *none* | A second RECEIVE in the same task: the input has already been read. |
| FILENOTFOUND | 12 | AEIL | A sequential file in `tmp_dir` could not be opened for a browse. |
| NOTFND | 13 | AEIM | The record, key, start request, document, HTTP header or form field was not found. |
| DUPREC | 14 | AEIN | WRITE of a key that already exists. |
| INVREQ | 16 | AEIP | The request is not valid: a required option is missing, options conflict, a name is not valid, the command is not valid in the current state, or the LINK nesting limit (100 levels) was reached. |
| IOERR | 17 | AEIQ | The database or the file system reported an error. |
| ENDFILE | 20 | AEIT | A browse read past the last (or before the first) record. |
| ILLOGIC | 21 | AEIU | INQUIRE ... NEXT with no STARTBROWSE active. |
| LENGERR | 22 | AEIV | A length is not valid, or data did not fit and was truncated. |
| QZERO | 23 | AEIW | READQ TD reached the end of the file. |
| ITEMERR | 26 | AEIZ | The TS item does not exist. |
| PGMIDERR | 27 | AEI0 | LINK or XCTL names a program that is not defined, or that LINK cannot load or the user may not run; INQUIRE PROGRAM of an unknown program. |
| TRANSIDERR | 28 | AEI1 | INQUIRE TRANSACTION of an unknown transaction. |

| Name | RESP | Abend | Meaning in bricks |
|---|---|---|---|
| ENDDATA | 29 | AEI2 | RETRIEVE found no (more) data from START. |
| MAPFAIL | 36 | AEI9 | A map could not be found, sent or received: map not installed or not parsable, no matching SEND MAP before RECEIVE MAP. |
| QIDERR | 44 | AEYH | The TS queue or TD file does not exist. |
| ENQBUSY | 55 | *none* | The resource or record is held by another task (ENQ NOSUSPEND, READ UPDATE, WRITE, DELETE), or an ENQ waited 5 minutes. |
| NOTAUTH | 70 | AEY7 | The user may not run the XCTL target; VERIFY PASSWORD failed. |
| ROLLEDBACK | 82 | AEXJ | SYNCPOINT ROLLBACK could not back out every file or TS change. |
| END | 83 | AEXK | INQUIRE ... NEXT has returned every resource. |

**Figure 18-1. Responses that bricks returns**

Each command in Chapters 11 through 15 and 20 lists the conditions it returns.

### Responses Defined but Not Returned

The following responses are defined, with their IBM values, so that programs that test them compile and DFHRESP resolves them. bricks does not return them. Their abend codes are listed in "Condition Abend Codes" in Appendix C.

| Name | RESP | Name | RESP | Name | RESP |
|---|---|---|---|---|---|
| DUPKEY | 15 | INVLDC | 41 | SYSBUSY | 59 |
| NOSPACE | 18 | NOSTG | 42 | SESSBUSY | 60 |
| NOTOPEN | 19 | JIDERR | 43 | NOTALLOC | 61 |
| SIGNAL | 24 | NOJBUFSP | 45 | CBIDERR | 62 |
| QBUSY | 25 | DSSTAT | 46 | INVEXITREQ | 63 |
| INVTSREQ | 30 | SELNERR | 47 | INVPARTNSET | 64 |
| EXPIRED | 31 | FUNCERR | 48 | INVPARTN | 65 |
| RETPAGE | 32 | UNEXPIN | 49 | PARTNFAIL | 66 |
| RTEFAIL | 33 | NOPASSBKRD | 50 | | |
| RTESOME | 34 | NOPASSBKWR | 51 | | |
| TSIOERR | 35 | SYSIDERR | 53 | | |
| INVMPSZ | 38 | ISCINVREQ | 54 | | |
| IGREQID | 39 | ENVDEFERR | 56 | | |
| OVERFLOW | 40 | IGREQCD | 57 | | |
| SESSIONERR | 58 | | | | |

**Figure 18-2. Responses defined for compatibility**

## Secondary Responses

EIBRESP2 (and RESP2(var)) is 0 unless the command documents a value. The values bricks returns are:

| Command | RESP | RESP2 | Meaning |
|---|---|---|---|
| RETURN | INVREQ | 1 | TRANSID in a task that has no terminal, or IMMEDIATE without a transaction identifier. |
| RETURN | INVREQ | 2 | COMMAREA, CHANNEL or IMMEDIATE from a program that is not at the highest logical level. |
| INQUIRE PROGRAM | PGMIDERR | 1 | The program is not defined. |
| INQUIRE TRANSACTION | TRANSIDERR | 1 | The transaction is not defined. |
| QUERY SECURITY, VERIFY PASSWORD, INQUIRE SYSTEM | INVREQ | 1-3 | The values are listed with each command in Chapter 13. |

**Figure 18-3. Secondary responses**

When `IGNORE CONDITION` turns a response into `NORMAL`, `EIBRESP` reads 0 and `EIBRESP2` is set to 0 as well. This differs from IBM CICS; see "IGNORE CONDITION" in Chapter 16.

Chapter 19

# Commands Not Implemented

This chapter lists the IBM CICS commands, and the forms and options of supported commands, that bricks does not implement, and describes what happens when a program uses one.

### What Happens When a Program Uses One

A PL/I program that names a command bricks does not implement is rejected when it is compiled (`UNKNOWN CICS COMMAND`). In COBOL and REXX the command is found when it is issued:

- The task ends. The operator sees a message that names the transaction and the command, for example `unsupported EXEC CICS verb "LOAD"`, instead of an abend message.
- The unit of work is backed out.
- This is not a condition. `RESP`, `NOHANDLE`, `HANDLE CONDITION` and `IGNORE CONDITION` do not apply to it, and REXX does not continue with the next statement.

The same applies to the unsupported forms and options listed in "Unsupported Forms of Supported Commands".

## Commands Not Implemented

| Command | Remarks |
|---|---|
| LOAD, RELEASE | bricks loads and keeps a program when it is first run. Use a LINK to share code. |
| PUSH HANDLE, POP HANDLE | Save the HANDLE settings in a variable of the program and issue HANDLE CONDITION again. |
| WAIT EVENT, WAIT EXTERNAL, POST, SUSPEND | There are no event control blocks. Use DELAY, or ENQ and DEQ to wait for another task. |
| ENTER TRACENUM, TRACE | There is no CICS trace. CEDF (Chapter 8) shows the commands of a task. |
| DUMP TRANSACTION | A program cannot request a dump. An abend writes its transaction dump to abend.log ("Transaction Dumps" in Chapter 16). |
| SIGNON, SIGNOFF | Operators sign on and off with CSSN and CSSF (Chapter 4). VERIFY PASSWORD checks a user identifier and password. |
| SEND PAGE, PURGE MESSAGE, ROUTE | There is no BMS paging or message routing. |
| ALLOCATE, CONNECT PROCESS, ISSUE, FREE | There is no APPC or LU6.2 communication. Use EXEC CICS WEB ("The HTTP Client" in Chapter 20) to reach other systems. |
| GET CONTAINER, PUT CONTAINER | There are no channels; pass data in a COMMAREA. |
| WRITE JOURNALNAME, SPOOLOPEN, SPOOLWRITE | There are no journals and no JES spool. Use WRITEQ TD to write a file. |
| BIF DEEDIT, CHANGE TASK | Not provided. |

**Figure 19-1. IBM CICS commands not implemented**

### Unsupported Forms of Supported Commands

| Command | Not supported |
|---|---|
| SEND | Any form other than SEND MAP, SEND TEXT and SEND CONTROL. A SEND MAP that waits for input, issued by a task that the scheduler runs in the background (use DATAONLY). |
| RECEIVE | Any form other than RECEIVE MAP and RECEIVE INTO. |
| LINK, XCTL | CHANNEL; use COMMAREA. |
| START | SYSID and QUEUE. |
| WRITE OPERATOR | REPLY and its options: bricks has no console from which a reply could come. |
| ADDRESS | Options other than CWA, TWA, EIB, COMMAREA, TCTUA and ACEE. |
| EXTRACT | Any form other than EXTRACT TCPIP. |
| QUERY | Any form other than QUERY SECURITY. |
| VERIFY | Any form other than VERIFY PASSWORD. |
| INQUIRE | An attribute that bricks does not report for the resource class. |
| WEB, DOCUMENT | A subcommand not described in Chapter 20. |
| HANDLE AID | ANYKEY, OPERID, LIGHTPEN, TRIGGER, CLRPARTN. These give the INVREQ condition. |

**Figure 19-2. Unsupported forms and options**

### Options Accepted Without Effect

The following options are accepted, so that programs written for IBM CICS compile and run unchanged, but they have no effect in bricks. Each is described with its command.

| Command | Options |
|---|---|
| WRITE | MASSINSERT |
| UNLOCK | TOKEN |
| WRITEQ TS | MAIN, AUXILIARY |
| WRITEQ TD | LENGTH |
| START | PROTECT |
| ENQ, DEQ | LENGTH (the whole resource name is used) |
| SEND CONTROL | ALARM, PRINT, STRFIELD, LDC; FRSET is always in effect |
| WRITE OPERATOR | ROUTECODES, NUMROUTES, CONSNAME, EVENTUAL, ACTION, CRITICAL, IMMEDIATE |

**Figure 19-3. Options accepted without effect**

### Commands Described Elsewhere

Earlier editions of this publication listed some commands here that bricks now implements. They are described in these chapters:

| Command | Chapter |
|---|---|
| SEND CONTROL, CONVERSE | Chapter 11 |
| START, RETRIEVE, CANCEL | Chapter 12 |
| DELAY, ENQ, DEQ, WRITE OPERATOR, GETMAIN, FREEMAIN, QUERY SECURITY, VERIFY PASSWORD | Chapter 13 |
| WEB, DOCUMENT | Chapter 20 |

**Figure 19-4. Commands formerly listed as not implemented**

The IBM abend codes that bricks never raises are listed in "Codes bricks Does Not Raise" in Appendix C.

Part 4

# Web Services

This part describes the commands with which a bricks program serves HTTP requests and makes requests of other servers, the documents it builds for a response, and the definitions that route a URL to a program.

**Chapter 20. Web Services: EXEC CICS WEB and DOCUMENT** ..... **20-1**

Chapter 20

# Web Services: EXEC CICS WEB and DOCUMENT

This chapter describes the HTTP interface of bricks. It covers the inbound listener that runs a transaction for each matching HTTP request, the outbound client with which a transaction calls a remote HTTP service, the `DOCUMENT` commands that build a response body from fragments, and the `web_routes.conf` file that holds both the inbound routes and the outbound URIMAP definitions.

The `EXEC CICS WEB` commands fall into two families:

- **Server commands** examine the inbound request and build its response. They are valid only in a task that the web listener started. In any other task (for example, a transaction that an operator typed at a 3270 display station) they return `INVREQ`.
- **Client commands** open a session to a remote server, send a request and receive its response. They are valid in any task, including a 3270 task and a web task. They are recognized by the `SESSTOKEN` option, by the `OPEN`, `CLOSE` and `CONVERSE` verbs, and by `WEB SEND URIMAP`.

Several commands, such as `WEB SEND`, `WEB RECEIVE` and `WEB READ HTTPHEADER`, exist in both families. CICS selects the client form when the command names a `SESSTOKEN` (or, for `WEB SEND`, a `URIMAP`), and the server form otherwise. The two forms are described separately in this chapter.

The commands are the same in COBOL, PL/I and REXX. The examples in this chapter are in COBOL unless otherwise stated.

### How an HTTP Request Becomes a Task

When `enable_wapi=yes` is coded in `bricks.cnf`, bricks opens an HTTP listener (and an HTTPS listener when `tlscert` and `tlskey` are set). For each request, the listener performs the following steps:

1. It matches the request method and path against the routes in `web_routes.conf`, in the order in which they appear in the file. The first matching route wins. If no route matches, the request is answered with status 404, or is served from the static document root (see "Static Files on the Web Listener").
2. It authenticates the caller and checks the caller's groups against the groups of the route (see "Authentication and Authorization").
3. It reads the request body, up to `web_request_max_bytes`.
4. It starts the route's transaction, exactly as if the transaction identifier had been typed at a display station. The transaction is looked up in `transactions.conf`, and the groups coded there are checked again. Both checks must allow the caller.
5. When the task ends, it writes the status, headers and body that the program built with `WEB WRITE HTTPHEADER` and `WEB SEND`.

A web task runs without a 3270 device. Its terminal identifier is generated by the listener and begins with the letter `W` and is unique among the requests in progress; `EXEC CICS ASSIGN USERID` returns the authenticated user, or blanks for an anonymous caller. Terminal control commands such as `SEND MAP` are meaningless in such a task. A program that is reachable from both a display station and the web normally uses one family of commands or the other, depending on how it was started.

***The response***

The listener builds the HTTP response from what the program left behind when the task ended:

| Program action | HTTP response |
|---|---|
| The task ends normally after one or more `WEB SEND` commands. | The status and media type of the first `WEB SEND` (default 200), every header written with `WEB WRITE HTTPHEADER`, and the concatenated bodies of all `WEB SEND` commands. `Content-Length` is always set by bricks. |
| The task ends normally without issuing `WEB SEND`. | Status 200 with an empty body. |
| The task abends before any `WEB SEND`. | Status 500 (`500 transaction error`). The abend is logged on the console with the route that started the task. |
| The task abends after a `WEB SEND`. | The response accumulated so far is sent, with the status of the first `WEB SEND`. |

**Figure 20-1. How the listener builds the HTTP response**

Because a response that was already started is still sent after an abend, a program that detects an error after its first `WEB SEND` cannot change the status. Validate the request completely before the first `WEB SEND`.

***Responses produced by the listener***

Some requests never reach a program. The listener answers them itself:

| Status | Cause |
|---|---|
| 400 | The request body could not be read. |
| 401 | The route requires a group and no credentials were sent, or the credentials were not HTTP Basic, could not be decoded, or were rejected. The response carries a `WWW-Authenticate` header. |
| 403 | The caller was authenticated but holds none of the route's groups. |
| 404 | No route matches, and no static file matches either. |
| 413 | The body is longer than `web_request_max_bytes` (default 1 MiB). |
| 431 | The request headers are longer than `web_header_max_bytes` (default 64 KiB). |
| 500 | The transaction could not be started (for example, the route names a transaction identifier that is not in `transactions.conf`), or it abended before sending a response. |

**Figure 20-2. Responses produced by the web listener**

**Note:**

A route row with no groups column denies every request. The listener answers such a request with 401 when it carries no credentials and with 403 when it does, as for any other caller that is not allowed. A route row that was refused when the file was loaded (for example, one with six fields) does not exist, so its requests receive 404.

### Defining Routes: web_routes.conf

The file named by `web_routes_file` (default `runtime/web_routes.conf`) is the bricks equivalent of the CICS URIMAP resource. It holds two kinds of row: a **route**, which maps an inbound request to a transaction, and a **URIMAP** row, which names an outbound endpoint (see "URIMAP Definitions"). Both kinds may be mixed in one file.

The file is reread automatically when it changes, so a new route takes effect without a restart. A row that is not valid is skipped with a warning on the console; the remaining rows still load. A # at the start of a line, or preceded by a blank or a tab, starts a comment.

***Format of a route row***

```
METHOD  PATH-PATTERN  TRANSID  [groups]  [response_timeout]
```

The fields are separated by blanks or tabs. A row has at least three and at most five fields; a row with six or more fields is refused.

| Field | Meaning |
|---|---|
| METHOD | One of GET, POST, PUT, DELETE, PATCH, HEAD or OPTIONS. Case is not significant. |
| PATH-PATTERN | A path that begins with /. A segment written {name} is a placeholder that matches one path segment (any characters except /) and captures its value under name. Two placeholders must be separated by at least one literal character, and a name may appear only once. |
| TRANSID | The 4-character transaction identifier to start. It must be defined in transactions.conf. |
| groups | A comma-separated list of groups, with no blanks, that may call the route; the literal group public admits unauthenticated callers. A – stands for an empty list, so that a timeout can be coded on a route that admits nobody. A row without this field denies every request, and a warning is written when the file is loaded. |
| response_timeout | A Go duration such as 5s, 250ms or 1m. It must not be negative. |

**Figure 20-3. Fields of a route row in web_routes.conf**

```
# method  path-pattern         transid  groups        timeout
GET       /api/customer/{id}   WAPI     public
POST      /api/customer        WAPC     admin
GET       /api/orders/{id}     OAPI     users,admin
DELETE    /admin/cache/{key}   CACR     admin         5s
```

The routes start ordinary transactions, defined in `transactions.conf` in the usual way:

```
WAPI:rexx:wapi.rexx:public,users,admin
WAPC:cobol:wapic.cob:admin
OAPI:rexx:orders.rexx:users,admin
CACR:cobol:cacheclr.cob:admin
```

***Matching rules***

- Routes are tried in file order and the first match wins. Put a specific pattern before a more general one. If two rows have the same method and pattern, the first one is used and the later one is reported as dead at load time.
- The path is matched in its percent-encoded form, so an encoded `/` (`%2F`) inside a segment cannot split that segment and satisfy a one-segment placeholder.
- A captured value is percent-decoded before the program sees it. For the pattern `/api/customer/{id}`, the request `GET /api/customer/Smith%20John` gives `WEB READ QUERYPARM('id')` the value `Smith John`, the same value that the query string `?id=Smith%20John` would give. A capture with a malformed escape sequence does not match, and the request receives 404.
- Path captures and query-string parameters share one name space. When a capture and a query parameter have the same name, the capture wins.

***The groups column***

- The list is a union. `admin,users` admits any user who belongs to either group.
- `public` anywhere in the list makes the whole route unauthenticated: `admin,public` is exactly as open as `public`. There is no way to require "public and admin". bricks writes a warning when a row mixes `public` with other groups.
- A duration in the fourth field, such as `30s` with no groups before it, is read as a group name. Because no user holds group `30S`, the route denies everyone. bricks warns about this at load time; code `-` in the groups column to give a timeout without groups.

***The response_timeout field***

The `response_timeout` field, and the `web_request_timeout` knob in `bricks.cnf`, are validated and recorded, but bricks does not at present interrupt a web task when the time runs out. A web task is bounded in the same way as any other task, by the runaway task detection described in "Runaway Tasks" in Chapter 16.

### Static Files on the Web Listener

When the static file server is enabled (`webserver_port` greater than zero), the web listener also serves the static document root (`webserver_dir`, default `runtime/web`) for any `GET` or `HEAD` request that matches no route. Routes always take precedence. This allows a browser application and the API it calls to share one origin; for example, the to-do sample's browser front end in `runtime/web/todo/` is opened as `http://`*`host`*`:`*`wapi_port`*`/todo/`. A file that does not exist receives 404. No directory listing is ever produced.

### Authentication and Authorization

A web caller identifies itself with HTTP Basic authentication. bricks verifies the user name and password against the same `users.conf` file and bcrypt hash that the `CSSN` sign-on transaction uses on a display station (see "Signing On: CSSN" in Chapter 4). The password is compared exactly, including case.

| Aspect | 3270 (CSSN) | Web listener |
|---|---|---|
| `Credential store` | `users.conf` | `users.conf` (the same file) |
| `Credential carrier` | Password field on the sign-on screen | `Authorization: Basic` header |
| `Duration` | Until `CSSF LOGOFF` | One request. Every request is authenticated again. |
| `Seen by the program` | `ASSIGN USERID` returns the user | `ASSIGN USERID` returns the verified user |

**Figure 20-4. Sign-on at a display station and on the web listener**

The listener decides as follows for each request:

1. If the route has no groups column, the request is refused.
2. If the request carries no `Authorization` header, it is accepted as anonymous when the route's groups include `public`; otherwise it receives 401 with `WWW-Authenticate: Basic realm="bricks"`, so that a browser shows its credential prompt.
3. If the header is present but is not `Basic`, cannot be decoded, or names a user or password that `users.conf` rejects, the request receives 401, even on a `public` route, so that a client with stale credentials is asked again. When the client tried another scheme (for example `Bearer`), the challenge names that scheme.
4. If the credentials are valid and the route is `public`, or the user holds at least one of the route's groups, the transaction is started with the user signed on for the duration of the request. Otherwise the request receives 403.
5. The dispatcher then checks the groups of the transaction in `transactions.conf`, exactly as for a display station. Both checks must allow the caller.

For example, user `alice`, defined in `users.conf` as `alice:`*`hash`*`:users,admin`, may run transaction `OAPI` from both places with the same password:

```
curl -u alice:alice-password http://localhost:8080/api/orders/42
  -> 200, the program sees USERID ALICE
curl -u alice:wrong-password http://localhost:8080/api/orders/42
  -> 401 Unauthorized
curl http://localhost:8080/api/orders/42
  -> 401 Unauthorized (no Authorization header)
```

bricks has no bearer tokens, cookies, OAuth or HTTP sessions. Each request stands alone. A transaction that needs its own credentials in addition (as the `TODO` sample does with its `X-Todo-User` and `X-Todo-Key` headers) reads them with `WEB READ HTTPHEADER`.

***Transport security***

The HTTPS listener uses the region's `tlscert` and `tlskey`. With `enforce_wapi_tls=yes` only the HTTPS listener opens, so that credentials never cross the network in clear text. The HTTPS listener negotiates HTTP/2 with clients that offer it through ALPN and HTTP/1.1 with the others; the plain listener speaks HTTP/1.1 only.

With `web_inbound_client_ca` set, the HTTPS listener requires every client to present a certificate signed by a CA in that PEM file. A client without such a certificate is refused during the TLS handshake and no transaction is started. The program reads the verified certificate with `WEB EXTRACT CERTIFICATE`.

## Code Page Conversion of Bodies

Program data in bricks is single-byte character data (SBCS); HTTP bodies are usually UTF-8. The same conversion rules apply to the server commands (`WEB RECEIVE`, `WEB RETRIEVE`, `WEB SEND`) and to the client commands (the request body of `WEB SEND` and `WEB CONVERSE`, and the response body of `WEB RECEIVE` and `WEB CONVERSE`).

A body is **text-like** when its media type is `text/*`, `application/json`, `application/xml`, `application/javascript`, `application/ecmascript`, `application/x-javascript`, `application/x-www-form-urlencoded`, `application/xhtml+xml`, or any type ending in `+json` or `+xml`. Any other body (an image, `application/octet-stream`, `multipart/*`) passes unchanged.

***Bodies received by the program***

| Content-Type of the received body | The program receives |
|---|---|
| Text-like, `charset=utf-8` | The body converted to SBCS; a character that SBCS cannot represent becomes `?`. |
| Text-like, no charset | The body converted when it is valid UTF-8; otherwise the bytes unchanged, taken as ISO-8859-1. |
| Text-like, `iso-8859-1`, `us-ascii`, `cp819` or `windows-1252` | The bytes unchanged (they are already SBCS). |
| Any other charset; not text-like; no or malformed Content-Type | The bytes unchanged. |
| Conversion switched off (see below) | The bytes unchanged (raw UTF-8). |

**Figure 20-5. Conversion of a body received by the program**

Conversion is switched off by `NOSRVCONVERT` or `SERVERCONV(NOSRVCONVERT)` on a server command, `NOCLICONVERT` or `CLIENTCONV(NOCLICONVERT)` on a client command, and by `INTOCCSID(1208)` or `INTOCODEPAGE('UTF-8')`. On `WEB RECEIVE`, `CHARACTERSET('UTF-8')` and `HOSTCODEPAGE('UTF-8')` also deliver the raw UTF-8 bytes. The `BODYCHARSET` option returns the character set that applied: `utf-8`, `iso-8859-1`, the declared label, or blanks for a body that is not text-like.

***Bodies sent by the program***

The target character set is the value of `CHARACTERSET` or `CLNTCODEPAGE`; otherwise the `charset` parameter of `MEDIATYPE`; otherwise UTF-8.

- A UTF-8 target converts the body from SBCS to UTF-8. An ISO-8859-1 target sends the SBCS bytes as they are.
- When the body is converted and `MEDIATYPE` has no charset parameter, bricks appends one, for example `application/json; charset=utf-8`. A `CHARACTERSET` that contradicts the parameter in `MEDIATYPE` replaces it.
- `NOSRVCONVERT` or `SERVERCONV(NOSRVCONVERT)` (client: `NOCLICONVERT` or `CLIENTCONV(NOCLICONVERT)`) sends the bytes unchanged and leaves `Content-Type` alone.
- Without `MEDIATYPE` (and without a `Content-Type` written by `WEB WRITE HTTPHEADER`), the body is treated as text and converted, and no `Content-Type` header is added.
- `LENGTH` counts SBCS characters and is applied before the conversion.
- `SRVCONVERT`, `CLICONVERT` and `HOSTCODEPAGE` are accepted; they describe the default.

**Note:**

Always code `MEDIATYPE` (for example `MEDIATYPE('image/png')` or `MEDIATYPE('application/octet-stream')`), or `NOSRVCONVERT` or `NOCLICONVERT`, when the body is not text. Without either, the body is converted as text: every byte from `X'80'` upward becomes a two- or three-byte UTF-8 sequence and the binary data is corrupted.

***Headers, parameters and fields***

Header values, query parameters, path captures, form fields, and the results of `WEB EXTRACT` and `WEB PARSE URL` reach the program as SBCS, and their lengths are counted in characters. Names used for a lookup, such as `QUERYPARM('grösse')`, are matched in UTF-8. `WEB WRITE HTTPHEADER` sends its bytes unchanged.

### Server Command Reference

The commands in this section are valid only in a task that the web listener started. In any other task each of them returns `INVREQ`, except `WEB PARSE URL`, `WEB CONVERTTIME` and `WEB EXTRACT URIMAP`, which are valid in every task.

The conditions are raised in the usual way: with `RESP` or `NOHANDLE` the program tests the response; otherwise an active `HANDLE CONDITION` applies; otherwise a COBOL or PL/I program abends with the condition's abend code (for example `AEIP` for `INVREQ`) and a REXX program continues with `RC` and `EIBRESP` set. See "Unhandled Conditions" in Chapter 16 and Chapter 18.

## WEB EXTRACT

### *Function*

Returns information about the inbound request and the listener that received it. The legacy form `EXTRACT TCPIP`, without the `WEB` keyword, is processed by the same code.

### *Syntax*

```
WEB EXTRACT  [HTTPMETHOD(data-area) | METHOD(data-area)]
             [REQUESTTYPE(data-area)]
             [SCHEME(data-area) | SCHEMENAME(data-area)]
             [HOST(data-area) [HOSTLENGTH(data-area)]]
             [PORT(data-area) | PORTNUMBER(data-area)]
             [PATH(data-area) [PATHLENGTH(data-area)]]
             [QUERYSTRING(data-area) [QUERYSTRINGLENGTH(data-area)]]
             [HTTPVERSION(data-area)]
             [CLIENTADDR(data-area)] [SERVERADDR(data-area)]
             [TCPIPSERVICE(data-area)] [AUTHENTICATE(data-area)]

EXTRACT TCPIP [CLIENT(data-area)] [IPADDRESS(data-area)]
              [AUTHENTICATE(data-area)] [TCPIPSERVICE(data-area)]
              [PORTNUMBER(data-area)]
```

### *Options*

| Option | Value returned |
|---|---|
| HTTPMETHOD, METHOD, REQUESTTYPE | The request method, for example GET. |
| SCHEME, SCHEMENAME | http or https. |
| HOST, SERVERADDR, IPADDRESS | The host name from the request's Host header, without the port. |
| PORT, PORTNUMBER | The port from the Host header, or 80 or 443 when the header names none. |
| PATH | The request path. |
| QUERYSTRING | The query string, without the ?, still percent-encoded. |
| HTTPVERSION | For example HTTP/1.1 or HTTP/2.0. |
| CLIENTADDR, CLIENT | The IP address of the caller. |
| TCPIPSERVICE | WAPI for the HTTP listener, WAPI-TLS for the HTTPS listener. |
| AUTHENTICATE | BASIC when the caller was authenticated with HTTP Basic; NO otherwise. |
| *option*LENGTH | Coded after any of the options above (for example HOSTLENGTH), returns the length of that value in characters. |

**Figure 20-6. Options of WEB EXTRACT**

### *Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| INVREQ | 16 | The task was not started by the web listener, or the request details are not available. |

**Figure 20-7. Conditions of WEB EXTRACT**

### *Example*

```
EXEC CICS WEB EXTRACT HTTPMETHOD(WS-METHOD)
                      PATH(WS-PATH) PATHLENGTH(WS-PATHLEN)
                      CLIENTADDR(WS-CLIENT)
                      RESP(WS-RESP) END-EXEC.
```

*Notes*

- Options that bricks does not recognize are ignored.
- With the `URIMAP` option the command examines a URIMAP definition instead (see "WEB EXTRACT URIMAP"); with `SESSTOKEN` it examines a client session (see "WEB EXTRACT SESSTOKEN"); with `CERTIFICATE` it examines the client certificate (next section).

*Differences from IBM CICS*

- bricks has no TCPIPSERVICE resource; the service name is fixed (`WAPI` or `WAPI-TLS`).
- `SERVERADDR` and `IPADDRESS` return the host name the client addressed, not the numeric address of the listener.

## WEB EXTRACT CERTIFICATE

*Function*

Returns fields of the client certificate that was verified during an inbound mutual-TLS handshake (see `web_inbound_client_ca` in "The bricks.cnf File" in Appendix D).

*Syntax*

```
WEB EXTRACT CERTIFICATE
            [COMMONNAME(data-area)]
            [ORGANISATION(data-area) | ORGANIZATION(data-area)]
            [COUNTRY(data-area)]
            [SERIALNUM(data-area)]
            [ISSUER(data-area)] [ISSUERORG(data-area)]
            [optionLENGTH(data-area) ...]
```

*Options*

| Option | Value returned |
|---|---|
| `COMMONNAME` | The common name (CN) of the subject. |
| `ORGANISATION` | The first organization (O) of the subject. |
| `COUNTRY` | The first country (C) of the subject. |
| `SERIALNUM` | The serial number of the certificate, in decimal. |
| `ISSUER` | The common name of the issuer. |
| `ISSUERORG` | The first organization of the issuer. |

**Figure 20-8. Options of WEB EXTRACT CERTIFICATE**

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The request arrived over plain HTTP, or over TLS without a client certificate. |
| INVREQ | 16 | The task was not started by the web listener. |

**Figure 20-9. Conditions of WEB EXTRACT CERTIFICATE**

Because NOTFND is returned for an anonymous caller, one transaction can serve callers with and without a certificate and branch on the response. The copybook DFHWBCC declares suitable fields.

***Example***

```
       COPY DFHWBCC.
       ...
       EXEC CICS WEB EXTRACT CERTIFICATE
                 COMMONNAME(DFH-WB-CN)
                 ORGANISATION(DFH-WB-ORG)
                 SERIALNUM(DFH-WB-SERIAL)
                 ISSUER(DFH-WB-ISSUER)
                 RESP(WS-RESP)
       END-EXEC.
       EVALUATE WS-RESP
           WHEN DFHRESP(NORMAL)
              PERFORM CERTIFIED-CALLER
           WHEN DFHRESP(NOTFND)
              PERFORM ANONYMOUS-CALLER
       END-EVALUATE.
```

## WEB READ HTTPHEADER, QUERYPARM and FORMFIELD

***Function***

Reads one inbound request header, one query parameter or path capture, or one field of a form-encoded body.

***Syntax***

```
WEB READ HTTPHEADER(name) VALUE(data-area) [LENGTH(data-area)]
WEB READ QUERYPARM(name)  VALUE(data-area) [LENGTH(data-area)]
WEB READ FORMFIELD(name)  VALUE(data-area) [LENGTH(data-area)]
```

*Options*

**HTTPHEADER(name)**

The header name. Case is not significant.

**QUERYPARM(name)**

The name of a query-string parameter or of a `{name}` placeholder in the route. The value is percent-decoded. When a name occurs more than once, the first value is returned.

**FORMFIELD(name)**

The name of a field of an `application/x-www-form-urlencoded` body. The body is parsed at the first `FORMFIELD` request of the task.

**VALUE(data-area)**

Receives the value.

**LENGTH(data-area)**

Receives the length of the value in characters.

*Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The header, parameter or field is not present. |
| INVREQ | 16 | The task was not started by the web listener; the name is blank; or (FORMFIELD) the body is not valid form encoding, for example a bad escape such as `%zz` or a `;` separator. A malformed body returns `INVREQ` on every `FORMFIELD` request of the task. |

**Figure 20-10. Conditions of WEB READ**

*Example*

```
EXEC CICS WEB READ FORMFIELD('title') VALUE(WS-TITLE)
          RESP(WS-RESP) END-EXEC.
IF WS-RESP = DFHRESP(INVREQ)
   PERFORM SEND-400
END-IF.
```

***Notes***

A malformed form body is the client's error. Code `RESP` and answer such a request with status 400.

## WEB STARTBROWSE, READNEXT and ENDBROWSE

***Function***

Returns, one at a time, every inbound header, every query parameter and path capture, or every form field.

***Syntax***

```
WEB STARTBROWSE {HTTPHEADER | QUERYPARM | FORMFIELD}
WEB READNEXT    {HTTPHEADER | QUERYPARM | FORMFIELD}
                NAME(data-area) VALUE(data-area)
                [NAMELENGTH(data-area)] [VALUELENGTH(data-area)]
WEB ENDBROWSE   {HTTPHEADER | QUERYPARM | FORMFIELD}
```

***Options***

**`NAME(data-area)`, `VALUE(data-area)`**

Receive the name and value of the next item.

**`NAMELENGTH(data-area)`, `VALUELENGTH(data-area)`**

Receive their lengths in characters.

Items are returned in ascending order of name. An item with several values (a repeated header or parameter) is returned once for each value. A browse of each kind may be open at the same time.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| ENDFILE | 20 | READNEXT: every item has been returned. |
| INVREQ | 16 | The task was not started by the web listener; READNEXT without a STARTBROWSE; or (FORMFIELD) STARTBROWSE on a body that is not valid form encoding. |

**Figure 20-11. Conditions of the WEB browse commands**

***Example***

The copybook `DFHWBHB` declares the browse fields.

```
        COPY DFHWBHB.
        ...
        EXEC CICS WEB STARTBROWSE HTTPHEADER END-EXEC.
        PERFORM UNTIL WS-RESP = DFHRESP(ENDFILE)
            EXEC CICS WEB READNEXT HTTPHEADER
                      NAME(DFH-WB-HDR-NAME)
                      NAMELENGTH(DFH-WB-HDR-NL)
                      VALUE(DFH-WB-HDR-VAL)
                      VALUELENGTH(DFH-WB-HDR-VL)
                      RESP(WS-RESP)
            END-EXEC
            IF WS-RESP = DFHRESP(NORMAL)
                PERFORM SHOW-HEADER
            END-IF
        END-PERFORM.
        EXEC CICS WEB ENDBROWSE HTTPHEADER END-EXEC.
```

The client form, which browses the headers of a response, is described in "Client Request and Response Headers".

### WEB RECEIVE (server)

#### *Function*

Copies the body of the inbound request into the program, converted as described in "Code Page Conversion of Bodies".

#### *Syntax*

```
WEB RECEIVE INTO(data-area)
            [MAXLENGTH(data-value)] [LENGTH(data-area)]
            [MEDIATYPE(data-area)] [TYPE(data-area)]
            [BODYCHARSET(data-area)]
            [SRVCONVERT | NOSRVCONVERT | SERVERCONV(cvda)]
            [CHARACTERSET(data-value) | INTOCCSID(data-value) |
             INTOCODEPAGE(data-value) | HOSTCODEPAGE(data-value)]
            [NOTRUNCATE]
```

*Options*

**INTO(data-area)**

Receives the body. Required.

**MAXLENGTH(data-value)**

The largest number of characters to return. The default is the whole body.

**LENGTH(data-area)**

Receives the number of characters returned.

**MEDIATYPE(data-area), TYPE(data-area)**

Receive the request's `Content-Type` header.

**BODYCHARSET(data-area)**

Receives the character set that applied to the conversion.

**Conversion options**

See "Code Page Conversion of Bodies". `NOTRUNCATE` is accepted and has no effect.

`MAXLENGTH`, `LENGTH` and `LENGERR` count the characters after conversion.

*Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| LENGERR | 22 | The body is longer than MAXLENGTH. The first MAXLENGTH characters are returned. |
| INVREQ | 16 | The task was not started by the web listener, or INTO is missing. |

**Figure 20-12. Conditions of WEB RECEIVE (server)**

*Example*

```
EXEC CICS WEB RECEIVE INTO(WS-BODY) MAXLENGTH(4096)
          LENGTH(WS-BODYLEN) MEDIATYPE(WS-CTYPE)
          RESP(WS-RESP) END-EXEC.
```

### WEB RETRIEVE

***Function***

Creates a document that holds the body of the inbound request and returns its token. The body is converted with the default rules. The program reads it with `DOCUMENT RETRIEVE`, or uses it with any other `DOCUMENT` command.

***Syntax***

```
WEB RETRIEVE DOCTOKEN(data-area)
```

***Conditions***

`INVREQ` (16): the task was not started by the web listener, or `DOCTOKEN` does not name a data area. `IOERR` (17): no token could be generated.

### WEB WRITE HTTPHEADER (server)

***Function***

Adds a header to the response.

***Syntax***

```
WEB WRITE HTTPHEADER(name) VALUE(data-value)
```

***Notes***

- Writing the same name twice produces two header lines.
- The value is sent as it is, without conversion.
- A `Content-Type` written here is used as the media type of the body when `WEB SEND` has no `MEDIATYPE`.
- bricks sets `Content-Length` itself.

***Conditions***

`INVREQ` (16): the task was not started by the web listener, or the header name is blank.

### WEB SEND (server)

***Function***

Sets the status and media type of the response and adds data to its body.

***Syntax***

```
WEB SEND {FROM(data-value) [LENGTH(data-value)] |
          DOCTOKEN(data-value)}
         [STATUSCODE(data-value)] [MEDIATYPE(data-value)]
         [CHARACTERSET(data-value) | CLNTCODEPAGE(data-value)]
         [SRVCONVERT | NOSRVCONVERT | SERVERCONV(cvda)]
         [HOSTCODEPAGE(data-value)]
         [STATUSTEXT(data-value)] [ACTION(cvda)]
         [CLOSESTATUS(cvda)] [CHUNKING(cvda)]
```

***Options***

**`FROM(data-value)`**

The data to add to the body.

**`LENGTH(data-value)`**

The number of characters of `FROM` to send. A smaller value cuts the data before conversion.

**`DOCTOKEN(data-value)`**

The token of a document whose contents are added to the body.

**`STATUSCODE(data-value)`**

The HTTP status. The default is 200.

**`MEDIATYPE(data-value)`**

The `Content-Type` of the response.

**Conversion options**

See "Code Page Conversion of Bodies".

**`STATUSTEXT`, `ACTION`, `CLOSESTATUS`, `CHUNKING`**

Accepted for compatibility and ignored. The listener always sends the complete response when the task ends.

Only the first `WEB SEND` of a task sets the status, the media type and the conversion. Each later `WEB SEND` appends its data to the body.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The DOCTOKEN is not known. |
| INVREQ | 16 | The task was not started by the web listener. |

**Figure 20-13. Conditions of WEB SEND (server)**

***Example***

```
        COPY DFHWBSC.
        ...
        EXEC CICS WEB WRITE HTTPHEADER('Cache-Control')
                  VALUE('no-store') END-EXEC.
        EXEC CICS WEB SEND FROM(WS-JSON) LENGTH(WS-JSONLEN)
                  MEDIATYPE('application/json')
                  STATUSCODE(DFHRESP-WB-OK) END-EXEC.
```

***Differences from IBM CICS***

- The default target character set is UTF-8. IBM CICS uses ISO-8859-1. bricks deviates so that a browser shows national characters without further options.
- The response is not sent until the task ends; there is no chunked transfer.

### WEB PARSE URL

***Function***

Splits a URL into its parts. The command is valid in any task.

***Syntax***

```
WEB PARSE URL(data-value) [URLLENGTH(data-value)]
          [SCHEMENAME(data-area) | SCHEME(data-area)]
          [HOST(data-area) [HOSTLENGTH(data-area)]]
          [PORTNUMBER(data-area) | PORT(data-area)]
          [PATH(data-area) [PATHLENGTH(data-area)]]
          [QUERYSTRING(data-area) [QUERYSTRINGLENGTH(data-area)]]
```

***Options***

`URLLENGTH` limits the number of characters of `URL` examined. The port defaults to 80 for `http` and 443 for `https`, and is blank for any other scheme that names none. Each output option may be followed by a length option formed by adding `LENGTH` (or `LEN`) to its name.

***Conditions***

`INVREQ` (16): `URL` is missing or cannot be parsed.

### WEB CONVERTTIME

***Function***

Converts an HTTP date string to a number of milliseconds. The command is valid in any task.

***Syntax***

```
WEB CONVERTTIME DATESTRING(data-value) ABSTIME(data-area)
```

***Options***

`DATESTRING` is a date in RFC 1123 form (for example `Tue, 15 Nov 1994 08:12:31 GMT`), RFC 1123 with a numeric zone, RFC 850 form, or the C `asctime` form. `ABSTIME` receives the number of milliseconds since 1970-01-01 00:00 UTC.

***Conditions***

`INVREQ` (16): an option is missing, or the date is in none of the accepted forms.

**Note:**

The value that `WEB CONVERTTIME` returns counts from 1970, not from 1900 like the `ABSTIME` of `EXEC CICS ASKTIME` (see "ASKTIME" in Chapter 13). The two values cannot be compared or passed to `FORMATTIME` without adjustment. The difference between the two bases is 2208988800000 milliseconds.

## The HTTP Client

A transaction calls a remote HTTP service with the client commands. No `bricks.cnf` option is needed to enable them. bricks builds one HTTP client when it starts and every task shares it; its behavior is controlled by the `web_client_*` knobs (see "The bricks.cnf File" in Appendix D):

- `web_client_timeout` (default 30 seconds) bounds each request, from sending it to reading the response. A request that takes longer returns `IOERR`.
- HTTPS server certificates are verified against the host system's trust store, or against `web_client_ca_bundle` alone when that is set. `web_client_cert` and `web_client_key` supply a client certificate for servers that require mutual TLS. `web_client_tls_skip_verify=yes` accepts any certificate; it is meant for tests only and logs a warning at startup.
- Idle connections are kept for reuse, up to `web_client_max_idle_conns` for each host. HTTP/2 is used when the server offers it.

***Sessions***

`WEB OPEN` creates a logical session for one endpoint and returns a **session token**, a 32-character hexadecimal string. The program passes the token on every later command for the session in the `SESSTOKEN` option. A task may hold several sessions at once. Sessions that are still open when the task ends are closed automatically.

A session can be used in two ways:

- `WEB CONVERSE` sends a request and receives the response in one command.
- `WEB SEND` prepares a request; the next `WEB RECEIVE` on the same session sends it and receives the response.

`WEB SEND URIMAP` combines `OPEN`, `CONVERSE` and `CLOSE` in one command for a call that needs no session.

***Request defaults***

Every request carries `Accept: */*` and `User-Agent: bricks-cics/1.0` unless the program wrote those headers itself with `WEB WRITE HTTPHEADER SESSTOKEN`. The `Host` header carries the port only when it is not the default for the scheme.

***Status codes***

Any HTTP response, including 404 or 500, completes the command normally. The program tests the status it receives in `STATUSCODE`. `IOERR` means that no response arrived: the host could not be reached, the TLS handshake failed, or the timeout expired.

### URIMAP Definitions

A URIMAP row in `web_routes.conf` gives a name to an outbound endpoint. A program opens a session with `URIMAP(name)` instead of naming the host, so that the endpoint can be moved by changing one row.

```
URIMAP  NAME  scheme://host[:port][/path-prefix]
```

| Field | Meaning |
|---|---|
| NAME | 1 to 8 characters from A–Z, 0–9, $, @, # and _. It is folded to upper case. A name with any other character is refused. When a name is defined twice, the first row is used. |
| scheme | http or https. |
| host | A host name or address. An IPv6 address must be enclosed in brackets. |
| port | Optional; 80 or 443 by default. |
| path-prefix | Optional. When present, it is placed in front of every PATH the program supplies on the session. |

**Figure 20-14. Fields of a URIMAP row**

```
URIMAP   GITHUB    https://api.github.com
URIMAP   WEATHER   https://api.openweathermap.org/data/2.5
URIMAP   INTRANET  https://api.internal:8443/v1
```

With the definitions above, `WEB OPEN URIMAP('WEATHER')` followed by `WEB CONVERSE PATH('/weather') QUERYSTRING('city=NYC')` requests `https://api.openweathermap.org/data/2.5/weather?city=NYC`.

URIMAP rows are loaded even when `enable_wapi=no`, so a region that only calls out needs no inbound listener. The operator displays the definitions with `CEMT INQUIRE URIMAP` and changes them with `CEDA URIMAP` (see "INQUIRE" in Chapter 5 and "URIMAP" in Chapter 6). A change made with `CEDA` is written to `web_routes.conf` and takes effect at once; sessions already open keep the endpoint they were opened with.

## WEB OPEN

### *Function*

Opens a client session for an endpoint and returns its token.

### *Syntax*

```
WEB OPEN {HOST(data-value) | URIMAP(data-value)}
         [PORT(data-value)] [SCHEME(data-value)]
         SESSTOKEN(data-area)
```

*Options*

**HOST(data-value)**

The host name or address.

**URIMAP(data-value)**

The name of a URIMAP definition, which supplies the scheme, host, port and path prefix. HOST, PORT and SCHEME coded on the same command override the corresponding part of the definition.

**SCHEME(data-value)**

HTTP or HTTPS, in either case. The default is http.

**PORT(data-value)**

The port. The default is 80 for HTTP and 443 for HTTPS.

**SESSTOKEN(data-area)**

Receives the 32-character session token. Required.

*Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The URIMAP name is not defined. |
| INVREQ | 16 | Neither HOST nor URIMAP gives a host; SESSTOKEN is missing; or the URIMAP table is not available. |
| IOERR | 17 | No token could be generated. |

**Figure 20-15. Conditions of WEB OPEN**

*Notes*

WEB OPEN does not connect to the server. Connection errors are reported by the command that sends the first request.

## WEB CONVERSE

*Function*

Sends a request on a session and receives the response. CONVERSE WEB is accepted as an alternative spelling of the same command.

***Syntax***

```
WEB CONVERSE SESSTOKEN(data-value)
             [METHOD(data-value)] PATH(data-value)
             [QUERYSTRING(data-value)]
             [FROM(data-value) [LENGTH(data-area)]]
             [MEDIATYPE(data-area)]
             INTO(data-area) [MAXLENGTH(data-value)]
             [STATUSCODE(data-area)] [STATUSTEXT(data-area)]
             [TYPE(data-area)] [BODYCHARSET(data-area)]
             [CLICONVERT | NOCLICONVERT | CLIENTCONV(cvda)]
             [CHARACTERSET(data-value) | CLNTCODEPAGE(data-value)]
             [INTOCCSID(data-value) | INTOCODEPAGE(data-value)]
```

*Options*

**METHOD(data-value)**

The request method. The default is GET. The copybook DFHWBMETH declares the method names.

**PATH(data-value)**

The path. A leading / is added if missing, and the session's URIMAP path prefix, if any, is placed in front. Required.

**QUERYSTRING(data-value)**

The query string, without ?, already percent-encoded.

**FROM(data-value)**

The request body.

**LENGTH(data-area)**

On input, the number of characters of FROM to send; on output, the number of characters returned in INTO.

**MEDIATYPE(data-area)**

On input, the Content-Type of the request body. If it names a data area, that data area receives the Content-Type of the response.

**INTO(data-area)**

Receives the response body, converted as described in "Code Page Conversion of Bodies".

**MAXLENGTH(data-value)**

The largest number of characters to return. The default is 1 MiB.

**STATUSCODE(data-area)**

Receives the HTTP status, for example 200.

**STATUSTEXT(data-area)**

Receives the status line, for example 200 OK.

**TYPE(data-area)**

Receives the Content-Type of the response.

**BODYCHARSET(data-area)**

Receives the character set that applied to the response body.

CHARACTERSET and CLNTCODEPAGE name the character set of the request body. INTOCCSID(1208) or INTOCODEPAGE('UTF-8') returns the response body unconverted.

*Conditions*

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The session token is not known (never opened, or already closed). |
| INVREQ | 16 | SESSTOKEN or PATH is missing, or the method or URL is not valid. |
| IOERR | 17 | No response arrived: network or TLS failure, or web_client_timeout expired. |
| LENGERR | 22 | The response body is longer than MAXLENGTH; the first MAXLENGTH characters are returned. The status and the other outputs are set. |

**Figure 20-16. Conditions of WEB CONVERSE, WEB RECEIVE (client) and WEB SEND URIMAP**

*Example*

```
       COPY DFHRESP.
       COPY DFHWBSC.
       COPY DFHWBMETH.
       COPY DFHWBSI.
       ...
       EXEC CICS WEB OPEN HOST('api.github.com') PORT(443)
                 SCHEME('HTTPS') SESSTOKEN(DFH-WB-SESS)
       END-EXEC.
       EXEC CICS WEB CONVERSE SESSTOKEN(DFH-WB-SESS)
                 METHOD(WB-GET) PATH('/zen')
                 INTO(DFH-WB-BODY) LENGTH(WS-LEN)
                 STATUSCODE(WS-STAT) RESP(WS-RESP)
       END-EXEC.
       EXEC CICS WEB CLOSE SESSTOKEN(DFH-WB-SESS) END-EXEC.
       IF WS-RESP = DFHRESP(NORMAL) AND WS-STAT = DFHRESP-WB-OK
           PERFORM SHOW-BODY
       END-IF.
```

In REXX the same commands name REXX variables. Each `EXEC CICS` command must stay on one line; a REXX comma continuation inside `EXEC CICS ... END-EXEC` is not allowed (see "The EXEC CICS and EXEC SQL Preprocessor" in Chapter 35):

```
EXEC CICS WEB OPEN URIMAP('GITHUB') SESSTOKEN(TOK) END-EXEC
EXEC CICS WEB CONVERSE SESSTOKEN(TOK) PATH('/zen') INTO(B) END-EXEC
EXEC CICS WEB CLOSE SESSTOKEN(TOK) END-EXEC
```

### WEB SEND and WEB RECEIVE (client)

***Function***

`WEB SEND` with `SESSTOKEN` prepares a request on a session without sending it. The next `WEB RECEIVE` on the session sends the request and receives the response.

***Syntax***

```
WEB SEND SESSTOKEN(data-value) [METHOD(data-value)]
         PATH(data-value) [QUERYSTRING(data-value)]
         [FROM(data-value) [LENGTH(data-value)]]
         [MEDIATYPE(data-value)] [conversion options]

WEB RECEIVE SESSTOKEN(data-value) INTO(data-area)
            [MAXLENGTH(data-value)] [LENGTH(data-area)]
            [STATUSCODE(data-area)] [STATUSTEXT(data-area)]
            [MEDIATYPE(data-area)] [TYPE(data-area)]
            [BODYCHARSET(data-area)] [conversion options]
```

***Options***

The options have the meanings given for `WEB CONVERSE`. On `WEB RECEIVE`, `CHARACTERSET('UTF-8')` also returns the response body unconverted.

***Conditions***

`WEB SEND`: `NOTFND` (unknown session) and `INVREQ` (missing option, invalid method or URL). `WEB RECEIVE`: the conditions of `WEB CONVERSE`, and `INVREQ` when no `WEB SEND` has prepared a request on the session.

***Differences from IBM CICS***

The request is always sent when `WEB RECEIVE` is issued, as with `ACTION(EVENTUAL)`; `WEB SEND` itself does no network I/O. Use `WEB CONVERSE` when both steps are wanted together.

### WEB SEND URIMAP

***Function***

Opens a session for a URIMAP definition, sends one request, receives the response and closes the session, all in one command. No session token is involved.

***Syntax***

```
WEB SEND URIMAP(data-value) [METHOD(data-value)]
         PATH(data-value) [QUERYSTRING(data-value)]
         [FROM(data-value)] [MEDIATYPE(data-area)]
         INTO(data-area) [MAXLENGTH(data-value)]
         [LENGTH(data-area)] [STATUSCODE(data-area)]
         [STATUSTEXT(data-area)] [TYPE(data-area)]
```

***Conditions***

The conditions of `WEB CONVERSE`; `NOTFND` means that the URIMAP name is not defined.

***Example***

```
EXEC CICS WEB SEND URIMAP('HOOKS') METHOD('POST')
          PATH('/notify') FROM(WS-EVENT)
          MEDIATYPE('application/json')
          INTO(WS-REPLY) STATUSCODE(WS-STAT)
          RESP(WS-RESP) END-EXEC.
```

## WEB CLOSE

***Function***

Ends a client session and releases its last response.

***Syntax***

```
WEB CLOSE SESSTOKEN(data-value)
```

***Conditions***

`NOTFND` (13): the session token is not known. `INVREQ` (16): `SESSTOKEN` is missing or blank.

## Client Request and Response Headers

***Function***

With `SESSTOKEN`, the header commands work on a client session: `WEB WRITE HTTPHEADER` adds a header to the next request, and `WEB READ HTTPHEADER` and the browse commands examine the headers of the most recent response.

***Syntax***

```
WEB WRITE HTTPHEADER(name) VALUE(data-value)
          SESSTOKEN(data-value)
WEB READ HTTPHEADER(name) VALUE(data-area)
         [LENGTH(data-area)] SESSTOKEN(data-value)
WEB STARTBROWSE HTTPHEADER SESSTOKEN(data-value)
WEB READNEXT HTTPHEADER NAME(data-area) VALUE(data-area)
         [NAMELENGTH(data-area)] [VALUELENGTH(data-area)]
         SESSTOKEN(data-value)
WEB ENDBROWSE HTTPHEADER SESSTOKEN(data-value)
```

***Notes***

- Headers written with `WEB WRITE HTTPHEADER` stay on the session and are sent with every later request on it. Writing the same name twice sends two header lines. A `User-Agent` or `Accept` written here replaces the bricks default.
- The response headers are those of the most recent `WEB CONVERSE` or `WEB RECEIVE` on the session. Names are compared without regard to case. The browse returns the headers in ascending order of name, one entry for each value.
- `STARTBROWSE` may be issued again to restart the browse.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The session token is not known; or (READ) the header is not in the response. |
| ENDFILE | 20 | READNEXT: every header has been returned. |
| INVREQ | 16 | A required option is missing; (READ, STARTBROWSE) no response has been received on the session yet; or (READNEXT) no browse is open. |

**Figure 20-17. Conditions of the client header commands**

***Example***

```
EXEC CICS WEB OPEN URIMAP('GITHUB') SESSTOKEN(WS-TOK) END-EXEC.
EXEC CICS WEB WRITE HTTPHEADER('User-Agent')
          VALUE('bricks-whdr/1.0') SESSTOKEN(WS-TOK) END-EXEC.
EXEC CICS WEB CONVERSE SESSTOKEN(WS-TOK) PATH('/zen')
          INTO(WS-BODY) STATUSCODE(WS-STAT) END-EXEC.
EXEC CICS WEB READ HTTPHEADER('Content-Type') VALUE(WS-CT)
          SESSTOKEN(WS-TOK) RESP(WS-RESP) END-EXEC.
```

## WEB EXTRACT SESSTOKEN

***Function***

Returns the endpoint of an open client session.

***Syntax***

```
WEB EXTRACT SESSTOKEN(data-value)
            [SCHEME(data-area) | SCHEMENAME(data-area)]
            [HOST(data-area) [HOSTLENGTH(data-area)]]
            [PORT(data-area) | PORTNUMBER(data-area)]
            [PATH(data-area) [PATHLENGTH(data-area)]]
```

`PATH` returns the URIMAP path prefix of the session, or blanks. The values are those in effect when the session was opened, even if the URIMAP definition has since been changed.

***Conditions***

`NOTFND` (13): the session token is not known. `INVREQ` (16): `SESSTOKEN` is blank.

## WEB EXTRACT URIMAP

***Function***

Returns the parts of a URIMAP definition without opening a session. The command is valid in any task.

***Syntax***

```
WEB EXTRACT URIMAP(data-value)
            [SCHEME(data-area) | SCHEMENAME(data-area)]
            [HOST(data-area) [HOSTLENGTH(data-area)]]
            [PORT(data-area) | PORTNUMBER(data-area)]
            [PATH(data-area) [PATHLENGTH(data-area)]]
```

The copybook `DFHWBUM` declares suitable fields.

***Conditions***

NOTFND (13): the name is not defined. INVREQ (16): the URIMAP table is not available.

## The DOCUMENT Commands

A document is a buffer, identified by a token, in which a program assembles a body from many fragments; WEB SEND DOCTOKEN then sends it as the response. WEB RETRIEVE also delivers an inbound body as a document.

A document holds SBCS data, like any other program data, and may grow to 4 MiB. A document belongs to the task that created it; any document that the program does not delete is released when the task ends. The copybook DFHDCDOC declares a token field, DFH-DOC-TOK (PIC X(36)), and delimiter constants.

Every DOCUMENT command except CREATE returns NOTFND (13) when the token is not known and INVREQ (16) when DOCTOKEN is missing.

## DOCUMENT CREATE

***Function***

Creates an empty document and returns its token.

***Syntax***

```
DOCUMENT CREATE DOCTOKEN(data-area)
         [SYMBOLLIST(data-value) [LISTLENGTH(data-value)]]
         [DELIMITER(data-value)]
```

***Options***

**DOCTOKEN(data-area)**

Receives the 32-character token. Required.

**SYMBOLLIST(data-value)**

A list of symbols in the form name=value&name=value, which DOCUMENT INSERT SYMBOL substitutes from. Values are taken as written; they are not percent-decoded.

**LISTLENGTH(data-value)**

The number of characters of SYMBOLLIST to use.

**DELIMITER(data-value)**

The separator between symbols. The default is &.

***Conditions***

INVREQ (16): DOCTOKEN does not name a data area. IOERR (17): no token could be generated.

## DOCUMENT INSERT

***Function***

Adds one fragment to a document.

***Syntax***

```
DOCUMENT INSERT DOCTOKEN(data-value)
         {FROM(data-value) [LENGTH(data-value)] |
          TEXT(data-value) | BINARY(data-value) |
          SYMBOL(data-value) | DOCUMENT(data-value)}
         [AT(data-value)]
```

***Options***

**FROM, TEXT, BINARY**

The data to insert. LENGTH limits the characters of FROM that are used. The three options are equivalent in bricks.

**SYMBOL(data-value)**

Inserts the value of a symbol from the document's symbol list.

**DOCUMENT(data-value)**

Inserts a copy of the contents of another document.

**AT(data-value)**

The position, counted from 1, before which the fragment is inserted. Without AT, or with a position beyond the end, the fragment is appended.

***Conditions***

| Condition | RESP | Meaning |
|---|---|---|
| NOTFND | 13 | The token, the symbol, or the document named by DOCUMENT is not known. |
| LENGERR | 22 | The document would exceed 4 MiB. Nothing is inserted. |
| INVREQ | 16 | No source option is coded. |

**Figure 20-18. Conditions of DOCUMENT INSERT**

### DOCUMENT SET

***Function***

Replaces the symbol list of a document.

***Syntax***

```
DOCUMENT SET DOCTOKEN(data-value) SYMBOLLIST(data-value)
         [LISTLENGTH(data-value)] [DELIMITER(data-value)]
         [UNESCAPED]
```

An empty `SYMBOLLIST` clears the list. A `DELIMITER` coded here also becomes the document's delimiter for later `SET` commands. `UNESCAPED` is accepted; values are never unescaped in bricks.

`INVREQ` (16) is returned when `SYMBOLLIST` is missing.

### DOCUMENT RETRIEVE

***Function***

Copies the contents of a document into the program.

***Syntax***

```
DOCUMENT RETRIEVE DOCTOKEN(data-value) INTO(data-area)
         [LENGTH(data-area)] [CHARACTERSET(data-value)]
         [DATAONLY] [HOSTCODEPAGE(data-value)]
```

`LENGTH` receives the number of bytes delivered. `CHARACTERSET('UTF-8')` (or `'1208'`) delivers the contents converted to UTF-8; any other value, and `DATAONLY` and `HOSTCODEPAGE`, are accepted and change nothing. `INVREQ` (16) is returned when `INTO` does not name a data area.

### DOCUMENT DELETE

***Function***

Releases a document.

***Syntax***

```
DOCUMENT DELETE DOCTOKEN(data-value)
```

***Example of a document***

```
        COPY DFHDCDOC.
        ...
        EXEC CICS DOCUMENT CREATE DOCTOKEN(DFH-DOC-TOK) END-EXEC.
        EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                  FROM('<html><body>') END-EXEC.
        PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-ROWS
            PERFORM FORMAT-ROW
            EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                      FROM(WS-ROW-HTML) LENGTH(WS-ROW-LEN)
            END-EXEC
        END-PERFORM.
        EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                  FROM('</body></html>') END-EXEC.
        EXEC CICS WEB SEND DOCTOKEN(DFH-DOC-TOK)
                  MEDIATYPE('text/html') STATUSCODE(200) END-EXEC.
```

***Differences from IBM CICS***

- Document templates (`DOCTEMPLATE`, `TEMPLATE`) and bookmarks are not supported.
- A document holds data in one code page; `TEXT` and `BINARY` behave like `FROM`.
- Symbols are substituted only by `INSERT SYMBOL`, not inside inserted text.

### Copybooks for Web Programs

The following COBOL copybooks are supplied in the copybook directory (`copybook_dir`, default `runtime/cobolcopy`). See "Web and Document Copybooks" in Chapter 26.

| Copybook | Contents |
|---|---|
| DFHWBSC | HTTP status constants, for example DFHRESP–WB–OK (200), DFHRESP–WB–CREATED (201), DFHRESP–WB–BADREQ (400). |
| DFHWBMETH | Method names WB–GET, WB–POST, WB–PUT, WB–DELETE, WB–PATCH, WB–HEAD, WB–OPTIONS. |
| DFHWBMT | Media types, for example WB–MT–JSON, WB–MT–HTML, WB–MT–PLAIN, WB–MT–OCTET. |
| DFHWBUH | Common header names, for example WB–CONTENT–TYPE, WB–AUTHORIZATION, WB–LOCATION. |
| DFHWBHB | Fields for header browsing: DFH–WB–HDR–NAME, DFH–WB–HDR–NL, DFH–WB–HDR–VAL, DFH–WB–HDR–VL. |
| DFHWBSI | Client session fields: DFH–WB–SESS (PIC X(36)), DFH–WB–BODY (PIC X(8192)), DFH–WB–URL (PIC X(2048)). |
| DFHWBUM | URIMAP fields for WEB EXTRACT URIMAP: name, scheme, host, port, path. |
| DFHWBCC | Client certificate fields for WEB EXTRACT CERTIFICATE. |
| DFHDCDOC | Document token DFH–DOC–TOK and delimiter constants. |

**Figure 20-19. Copybooks for web programs**

### Sample Web Transactions

| Transid | Program | Demonstrates |
|---|---|---|
| WAPI | rexx/wapi.rexx | The server commands in REXX; route GET /api/customer/{id}. |
| TODO | cobol/todo.cob | A multi-user to-do REST API: one program serves every /api/todo/... route, choosing the action from the method and the shape of the path. It stores data in three VSAM files, builds JSON with JSON GENERATE, reads JSON bodies with JSON PARSE, validates all input before its first update, issues SYNCPOINT ROLLBACK and answers 500 on an unexpected response, and answers 409 on a record lock collision. Its browser front end is in runtime/web/todo/; the script scripts/todo_api_test.bash tests it end to end. |
| WZEN | cobol/wzen.cob | A 3270 transaction that calls GitHub's /zen endpoint over HTTPS and shows the URL, the status and the body on map WZEN1, with a separate error path for network and HTTP failures. |
| WHDR | cobol/whdr.cob | Client headers: opens URIMAP GITHUB, writes User-Agent and Accept, and reads two response headers that are present and one that is not, to show the NOTFND path. |

**Figure 20-20. Supplied web sample transactions**

The operator watches requests in progress, and the lifetime request and error counts, with CEMT INQUIRE WEB (see "INQUIRE" in Chapter 5).

## Summary of Differences from IBM CICS

| Area | bricks behavior |
|---|---|
| Resource definitions | Routes and URIMAPs are rows of web_routes.conf. There are no TCPIPSERVICE, PIPELINE or WEBSERVICE resources, and no analyzer or converter programs. |
| Authentication | HTTP Basic only, against users.conf; a client certificate can be required with web_inbound_client_ca. |
| Default code page | Outbound bodies default to UTF-8, not ISO-8859-1. |
| Response timing | The response is sent when the task ends. ACTION, CHUNKING and CLOSESTATUS are ignored. |
| Request timeout | response_timeout and web_request_timeout are recorded but not enforced. |
| Client send | WEB SEND with SESSTOKEN never transmits; WEB RECEIVE sends the prepared request. |
| Extensions | WEB SEND URIMAP (one-shot call) and WEB EXTRACT URIMAP are bricks additions. |
| Time values | WEB CONVERTTIME returns milliseconds since 1970, not an ABSTIME based on 1900. |
| Documents | No templates or bookmarks; one code page. |

**Figure 20-21. Web support: differences from IBM CICS**

Part 5

# Database Access

This part describes embedded SQL: the statements a program may issue, host variables, cursors, the unit of work and its commit points, and the abends that a database error can cause.

**Chapter 21. Embedded SQL** ..... **21-1**

Chapter 21

# Embedded SQL

This chapter describes the `EXEC SQL` interface, with which a COBOL, PL/I or REXX program reads and updates a PostgreSQL database. It covers the configuration that binds a transaction to a database, the SQL statements that bricks supports, host variables and null indicators, cursors, the SQL communication area, the way SQL work takes part in the unit of work, and the abends that a database failure causes.

The interface follows the conventions of DB2 for z/OS under CICS: host variables are written with a leading colon, the outcome of each statement is reported in `SQLCODE` and `SQLSTATE`, and SQL updates are committed and backed out together with recoverable file and temporary storage updates. The statements themselves are passed to PostgreSQL, so the SQL dialect (functions, data types, literals) is the PostgreSQL dialect.

Everything in this chapter applies to all three languages unless a language is named. The examples are in COBOL unless otherwise stated.

## Connecting a Transaction to a Database

Database access is configured by the system programmer (see "The databases.conf File" in Appendix D):

- The `db_*` knobs in `bricks.cnf` name the PostgreSQL server, the user and password, and the size of the connection pool. When `db_host` is not set, bricks runs without SQL and every `EXEC SQL` statement returns `SQLCODE` -1.
- `databases.conf` lists the databases on that server that bricks may use, one name per line. The **first** row is the default database.
- The fifth field of a transaction's row in `transactions.conf` binds the transaction to one of those databases.

***Binding a transaction to a database***

```
transid:language:program[:groups[:database[:twasize]]]
```

```
BANK:cobol:bank.cob:public,users,admin:bank
```

- The database name is matched without regard to case; both sides are folded to lower case, as PostgreSQL folds unquoted names, so `:BANK` and `:bank` name the same database. Blanks around the name are ignored.
- When the field is absent or empty, the transaction uses the default database (the first row of `databases.conf`). A program that uses another database must name it here; otherwise its statements reach the default database, where the table is usually missing, and fail with `SQLCODE` -204.
- When the field names a database that `databases.conf` does not list, the mismatch is logged when the task starts and every statement returns `SQLCODE` -1.

A transaction that never issues `EXEC SQL` may leave the field empty. `transactions.conf` is reread when it changes, and `CEDA TRANSACTION` maintains the field (see "TRANSACTION" in Chapter 6).

### Coding EXEC SQL Statements

An SQL statement is written between `EXEC SQL` and its terminator:

| Language | Form |
|---|---|
| `COBOL` | `EXEC SQL` *`statement`* `END-EXEC`, optionally followed by a period. The statement may span several lines. |
| `REXX` | `EXEC SQL` *`statement`* `END-EXEC`. The statement may span several lines; do not use a REXX comma continuation. |
| `PL/I` | `EXEC SQL` *`statement`*`;` (no `END-EXEC`). |

**Figure 21-1. Form of an EXEC SQL statement in each language**

The following statements are supported. Each is described later in this chapter.

| Statement | Purpose |
|---|---|
| `SELECT ... INTO` | Read one row into host variables. |
| `INSERT, UPDATE, DELETE` | Change rows. A positioned `UPDATE` or `DELETE` names a cursor with `WHERE CURRENT OF`. |
| `DECLARE CURSOR, OPEN, FETCH, CLOSE` | Read a result of any number of rows. |
| `COMMIT, ROLLBACK` | End the database transaction. See "COMMIT and ROLLBACK" for why `EXEC CICS SYNCPOINT` is usually the better choice. |
| `CONNECT TO` | Switch the task to another database. |
| `WHENEVER` | Declare an action for SQL errors, warnings or no data. |
| `Other SQL` | Any other statement, for example `SELECT` without `INTO`, `CREATE TABLE` or `CALL`, is passed to PostgreSQL as it is written. Its result rows are not returned to the program. |

**Figure 21-2. Supported SQL statements**

Statements that create or remove databases or database users, `CREATE`, `DROP` or `ALTER` followed by `DATABASE`, `USER`, `ROLE` or `TABLESPACE`, are refused with `SQLCODE` -2 and `SQLSTATE` 42501. These operations belong to the `CEDA DATABASE` transaction, which records each one in the audit log.

In PL/I, `EXEC SQL INCLUDE SQLCA;` and `BEGIN` and `END DECLARE SECTION` are accepted and do nothing. In COBOL and REXX they are not needed and must not be coded: the SQLCA is always present (see "The SQL Communication Area").

### Host Variables

A host variable is a program variable named in an SQL statement with a leading colon, for example `:CUSTID`. In COBOL and PL/I it must be declared in the program; in REXX it is an ordinary REXX variable. Host variable names are not case-sensitive.

- **Input.** In a `WHERE`, `VALUES` or `SET` clause, the current value of the variable is passed to PostgreSQL as a statement parameter. It is never pasted into the SQL text, so a value cannot change the statement. Trailing blanks are removed before the value is passed, so a `PIC X(10)` field that holds `K001` is passed as `'K001'`.
- **Output.** In the `INTO` clause of `SELECT` or `FETCH`, the variable receives a column value.
- A `:name` inside a quoted SQL literal is data, not a host variable. The scan for host variables also skips `--` comments and the PostgreSQL `::` cast operator.
- In PL/I, a structure named as a host variable stands for its members in declaration order, so `INTO :ROW` is the same as `INTO :NAME, :CITY`. An array is not allowed as a host variable.

***Character data***

Program data is single-byte (SBCS); PostgreSQL works in UTF-8. bricks converts input values and the SQL text to UTF-8, and converts result values back to SBCS. A character that SBCS cannot represent is replaced by `?`, and `SQLWARN8` and `SQLWARN0` are set to `W`. A `BYTEA` column is returned without conversion.

***Column values***

Before a column value is stored in a host variable, bricks adjusts three types so that programs see what DB2 would give them:

| Column type | Adjustment |
|---|---|
| `BOOLEAN` | `t` or `true` becomes `1`; `f` or `false` becomes `0`. |
| `NUMERIC(p,s)` | The value is padded to the declared scale, so `1.5` in a column of scale 2 arrives as `1.50` and a `PIC 9(3)V99` target receives `001.50`. |
| `CHAR(n)` | Trailing blanks are removed. `VARCHAR` and `TEXT` values are not changed. |

**Figure 21-3. Adjustment of column values**

All other types (integers, `TEXT`, `DATE`, `TIMESTAMP`, `JSON` and so on) arrive in their PostgreSQL text form. A COBOL `MOVE` to a numeric item handles sign and leading zeros; REXX uses the string directly.

### Null Indicators

A host variable may be followed by a second host variable, its **indicator**, separated from it only by blanks. The indicator reports or requests a null value.

```
EXEC SQL
    SELECT name INTO :NM :NIND
    FROM customers_sql WHERE id = :CUSTID
END-EXEC.
EVALUATE NIND
    WHEN 0   DISPLAY 'Name: ' NM
    WHEN -1  DISPLAY 'Name is NULL'
END-EVALUATE.
```

- **Output** (`SELECT INTO`, `FETCH`): the indicator is set to 0 when the column has a value. When the column is null, the indicator is set to -1 and the value variable is cleared to an empty string, so that a stale value from an earlier statement cannot be read by mistake.
- **Input** (`WHERE`, `VALUES`, `SET`): when the indicator holds -1, a null value is passed, whatever the value variable contains. Any other value, including 0 or an unset indicator, passes the value variable.
- `:NM, :NIND`, with a comma, is two separate host variables, not a pair.
- The form `:NM INDICATOR :NIND` is not supported.

```
MOVE -1 TO NIND.
EXEC SQL
    INSERT INTO customers_sql (id, name) VALUES (:K, :NM :NIND)
END-EXEC.
```

## SELECT INTO

***Function***

Reads at most one row and stores its columns in host variables.

***Syntax***

```
SELECT column [, column ...]
       INTO :hostvar [:indicator] [, :hostvar [:indicator] ...]
       [FROM ... [WHERE ...] ...]
```

The columns are assigned to the host variables in order. The `FROM` clause is optional, so a value can be computed directly, for example `SELECT CURRENT_DATE INTO :D` or `SELECT nextval('ordno') INTO :N`.

***Results***

| Outcome | SQLCODE | SQLSTATE | Host variables |
|---|---|---|---|
| Exactly one row | `0` | 00000 | Set; `SQLERRD3` = 1 |
| No row | `+100` | 02000 | Unchanged |
| More than one row | `-811` | 21000 | Unchanged |

**Figure 21-4. Results of SELECT INTO**

## INSERT, UPDATE and DELETE

***Function***

Add, change or remove rows.

***Syntax***

```
INSERT INTO table [(column, ...)] VALUES (value, ...)
UPDATE table SET column = value [, ...] [WHERE ...]
UPDATE table SET column = value [, ...] WHERE CURRENT OF cursor
DELETE FROM table [WHERE ...]
DELETE FROM table WHERE CURRENT OF cursor
```

Any value may be a host variable, with or without an indicator.

***Results***

- On success, `SQLCODE` is 0 and `SQLERRD3` holds the number of rows inserted, changed or deleted.
- An `UPDATE` or `DELETE` that affects no row also completes with `SQLCODE` 0, and `SQLERRD3` is 0. It does **not** return +100. Test `SQLERRD3` to find out whether a row was affected.
- A constraint violation returns the codes in "SQLCODE Values", for example -803 for a duplicate key.

### *Positioned UPDATE and DELETE*

`WHERE CURRENT OF` *`cursor`* changes the row that the program last fetched through that cursor. The cursor must be declared with `FOR UPDATE`. On an open cursor without `FOR UPDATE`, the statement is refused with `SQLCODE` -510 and `SQLSTATE` 42828, and no row is changed. See "Cursors".

### *Example*

```
EXEC SQL
    UPDATE accounts SET balance = balance - :AMT
    WHERE acct = :ACCT
END-EXEC.
EVALUATE TRUE
    WHEN SQLCODE NOT = 0
        PERFORM SQL-ERROR
    WHEN SQLERRD3 = 0
        PERFORM NO-SUCH-ACCOUNT
END-EVALUATE.
```

### *Differences from DB2*

DB2 returns +100 for a searched `UPDATE` or `DELETE` that finds no row. bricks returns 0 with `SQLERRD3` = 0. A `WHENEVER NOT FOUND` action therefore never fires for these statements.

## Cursors

A cursor reads a result of any number of rows, one row for each `FETCH`. The life of a cursor has four steps:

```
DECLARE cursor CURSOR FOR select-statement
OPEN cursor
FETCH cursor INTO :hostvar [:indicator] [, :hostvar ...]
CLOSE cursor
```

A cursor name begins with a letter and may contain letters, digits and underscores; case is not significant.

**DECLARE**

Records the name and the SELECT statement. Nothing is sent to the database. The SELECT has no INTO clause. Declaring a name again replaces the earlier declaration, unless that cursor is open.

**OPEN**

Creates the cursor on the database server, inside the task's database transaction, as NO SCROLL and WITHOUT HOLD, the DB2 default. The values of the host variables in the SELECT are taken at this moment, wherever the DECLARE appears in the source. The result is fixed when the cursor is opened: later changes by this or other tasks do not alter the rows it returns.

**FETCH**

Returns the next row into the host variables, with SQLCODE 0 and SQLERRD3 = 1. When no row is left, FETCH returns +100 and closes the cursor.

**CLOSE**

Closes the cursor. Closing a cursor that is not open, or a name that was never declared, is not an error.

***Fetching in batches***

The rows stay on the server. FETCH obtains them from the server in batches of 100 rows and hands them to the program one at a time, so a program never holds more than one batch in storage however large the result is. Each batch is one round trip to the database.

A cursor whose SELECT contains FOR UPDATE is fetched one row at a time, so that the server's position is always the row the program last received. This is what makes WHERE CURRENT OF safe. On any other cursor the server may be up to a batch ahead of the program, so a positioned UPDATE or DELETE on it is refused with -510.

***Other statements while a cursor is open***

Any other SQL statement may run while a cursor is open, and several cursors may be open at once. The common loop that reads a row and then issues UPDATE, INSERT or SELECT INTO for it works as it does on DB2.

***When a cursor is closed***

A cursor is closed by `CLOSE`, by the end of its data, and by the end of the unit of work: `EXEC CICS SYNCPOINT`, `SYNCPOINT ROLLBACK`, `EXEC SQL COMMIT` or `ROLLBACK`, and `CONNECT TO`. A closed cursor stays declared, as on DB2: `OPEN` reopens it from the first row without another `DECLARE`, and a `FETCH` before that reopening is refused as not open.

At an `XCTL`, the cursors of the program that issues the `XCTL` are closed and their declarations are discarded, because the target program cannot refer to them. The database transaction itself continues in the target (see "Units of Work and Commit Points"). A cursor never survives the end of the task, and none survives a pseudo-conversational `RETURN TRANSID`.

***Cursor errors***

| Error | SQLCODE | SQLSTATE | DB2 SQLCODE |
|---|---|---|---|
| OPEN or FETCH of a cursor that is not declared | `-100` | 34000 | -504 |
| FETCH of a cursor that is not open | `-100` | 24501 | -501 |
| OPEN, or DECLARE again, of a cursor that is open | `-100` | 24502 | -502 |
| Statement does not match the syntax above | `-100` | 42601 | -104 |
| WHERE CURRENT OF on a cursor without FOR UPDATE | `-510` | 42828 | -510 |

**Figure 21-5. Cursor errors**

bricks reports the cursor-state errors with the general `SQLCODE` -100 and sets `SQLSTATE` to the specific value. Test `SQLSTATE` when the reason matters.

***Example***

```
EXEC SQL DECLARE C1 CURSOR FOR
    SELECT id, name FROM customers_sql ORDER BY id
END-EXEC.
EXEC SQL OPEN C1 END-EXEC.
PERFORM UNTIL SQLCODE NOT = 0
    EXEC SQL FETCH C1 INTO :ID, :NM END-EXEC
    IF SQLCODE = 0
        DISPLAY 'row: ' ID ' ' NM
    END-IF
END-PERFORM.
EXEC SQL CLOSE C1 END-EXEC.
```

The same loop in REXX:

```
EXEC SQL DECLARE C1 CURSOR FOR SELECT id, name FROM customers_sql END-EXEC
EXEC SQL OPEN C1 END-EXEC
DO FOREVER
  EXEC SQL FETCH C1 INTO :ID, :CUSTNM END-EXEC
  IF SQLCODE <> 0 THEN LEAVE
  SAY 'row:' ID CUSTNM
END
EXEC SQL CLOSE C1 END-EXEC
```

***Example of a positioned update***

```
EXEC SQL DECLARE UPD CURSOR FOR
    SELECT acct, balance FROM accounts
    WHERE branch = :BR FOR UPDATE
END-EXEC.
EXEC SQL OPEN UPD END-EXEC.
PERFORM UNTIL SQLCODE NOT = 0
    EXEC SQL FETCH UPD INTO :ACCT, :BAL END-EXEC
    IF SQLCODE = 0
        COMPUTE NEWBAL = BAL * 1.01
        EXEC SQL UPDATE accounts SET balance = :NEWBAL
                 WHERE CURRENT OF UPD END-EXEC
    END-IF
END-PERFORM.
```

***Differences from DB2***

- Only `FETCH` *`cursor`* `INTO ...` is accepted. The keywords `NEXT` and `FROM`, scrollable cursors, `WITH HOLD`, `OPEN ... USING` and rowset (multi-row) fetches are not supported.
- The cursor-state errors are reported as `SQLCODE` -100 with a specific `SQLSTATE`, as shown in "Cursor errors".

## WHENEVER

### *Function*

Declares what the program does after any later SQL statement that ends with an error, a warning or no data, so that the program need not test SQLCODE after every statement.

### *Syntax*

```
WHENEVER {SQLERROR | SQLWARNING | NOT FOUND}
         {CONTINUE | GO TO label | GOTO label}

PL/I:  WHENEVER {SQLERROR | SQLWARNING | NOT FOUND}
                {CONTINUE | DO CALL procedure | GO TO procedure}
```

| Condition | Raised when |
|---|---|
| SQLERROR | SQLCODE is negative. |
| NOT FOUND | SQLCODE is +100. |
| SQLWARNING | SQLWARN0 is W and neither of the other conditions applies. bricks sets it when a result lost characters in the conversion to SBCS (SQLWARN8). |

**Figure 21-6. WHENEVER conditions**

- CONTINUE takes no action. It is the default for a condition that has no WHENEVER.
- In COBOL, GO TO branches to the paragraph, ending any PERFORM in progress as an explicit GO TO would. In REXX it is a SIGNAL to the label.
- In PL/I, the target is a parameterless procedure nested directly in the main procedure; GO TO is treated as DO CALL. The procedure runs as a final handler: when it returns, the task ends and its work is committed. Issue EXEC CICS SYNCPOINT ROLLBACK or ABEND in it when the work must not be kept.
- A later WHENEVER for the same condition replaces the earlier one.

### *Example*

```
EXEC SQL WHENEVER SQLERROR  GO TO SQL-ERROR-EXIT END-EXEC.
EXEC SQL WHENEVER NOT FOUND GO TO NO-MORE-ROWS   END-EXEC.
EXEC SQL SELECT name INTO :NM FROM customers_sql
         WHERE id = :CUSTID END-EXEC.
```

In REXX:

```
EXEC SQL WHENEVER SQLERROR GOTO SQLERR END-EXEC
EXEC SQL SELECT name INTO :CUSTNM FROM customers_sql WHERE id = :CUSTID END-EXEC
SAY 'customer:' CUSTNM
EXIT
SQLERR:
SAY 'SQL failed, SQLCODE' SQLCODE
EXIT
```

***Differences from DB2***

DB2 applies a `WHENEVER` to the statements that follow it in the **source**, because its precompiler inserts the test after each statement. bricks applies the most recently **executed** `WHENEVER`. The two agree for the usual case of `WHENEVER` statements at the start of the program. A program that issues `WHENEVER` inside conditional logic sees the execution-order rule; keep `WHENEVER` statements at the start of a paragraph or procedure.

## CONNECT TO

***Function***

Switches the task to another database listed in `databases.conf`. The binding in `transactions.conf` chooses the database for the first statement of every task; `CONNECT TO` is needed only to use a second database within one task.

***Syntax***

```
CONNECT TO 'name'
```

The name may also be written without quotes or in double quotes. It is matched without regard to case.

***Results***

- Before switching, bricks commits the database transaction that is open on the current database, as DB2 does, and closes its cursors. A program that wants to discard that work issues `ROLLBACK` first. Pending file and temporary storage updates are not affected; they are committed at the next syncpoint.
- An unknown or offline database returns `SQLCODE` -1 and `SQLSTATE` 08001; the task stays on its current database.
- If the implicit commit fails, `SQLCODE` is -100, the task stays on its current database, and the database work of the transaction is lost.

```
EXEC SQL CONNECT TO 'orders' END-EXEC.
EXEC SQL INSERT INTO order_lines VALUES (:ID, :QTY) END-EXEC.
EXEC SQL CONNECT TO 'customers' END-EXEC.
EXEC SQL SELECT email INTO :EM FROM customers
         WHERE id = :CID END-EXEC.
```

### Units of Work and Commit Points

A task's SQL work and its recoverable file and temporary storage updates form **one** unit of work. The unit of work spans every program that the task reaches by `LINK` or `XCTL`. The database transaction begins at the first SQL statement that touches data, on the database the transaction is bound to. See "Units of Work" in Chapter 16 for the unit of work as a whole.

***Commit and backout points***

The commit and backout points of the unit of work are those of the task, listed in "Commit Points" and "Backout Points" in Chapter 16. The following rules concern SQL in particular:

- At a commit point the database is committed first, and then the file and temporary storage updates. A commit that the database refuses backs out the whole unit of work and abends the task (see "Refused Commits").
- An `XCTL` to a program bound to a **different** database, or to a supplied or MRO-routed transaction, is a commit point: one unit of work cannot span two PostgreSQL databases, so bricks takes an implicit syncpoint at the `XCTL`. An `XCTL` to a program on the **same** database carries the open database transaction, the pending file and temporary storage updates, and the locks into the target (see "XCTL" in Chapter 12).
- An `XCTL` whose target passed the checks but could not be loaded backs out the unit of work. A target that is not defined or not authorized raises `PGMIDERR` or `NOTAUTH` at the `XCTL`, and nothing is backed out.
- `EXEC SQL COMMIT` and `ROLLBACK` settle only the database transaction; they are not commit points of the unit of work (see "COMMIT and ROLLBACK").
- A task started by `EXEC CICS START` has a unit of work of its own.

bricks never loses `EXEC SQL` work silently. Every way in which a task ends either commits or backs out its unit of work, and a commit that cannot be made always abends the task. Should a bricks defect ever leave a database transaction open at task end, it is rolled back to release its connection and row locks, and the console shows `ERROR SQL: term=... transid=... program=...: transaction left open at task end, rolled back`.

### COMMIT and ROLLBACK

***Function***

End the task's database transaction.

***Syntax***

```
COMMIT [WORK]
ROLLBACK [WORK]
```

***Notes***

- `EXEC SQL COMMIT` and `ROLLBACK` act on the database transaction **only**. Pending file and temporary storage updates are not committed or backed out; they wait for the next syncpoint. Use `EXEC CICS SYNCPOINT` and `SYNCPOINT ROLLBACK`, which settle the whole unit of work, unless the database work is deliberately to be settled on its own.
- A commit that PostgreSQL refuses is reported to the program as `SQLCODE` -100, not as an abend; the database work of the transaction is lost. The same refusal at `EXEC CICS SYNCPOINT` abends the task (see "Refused Commits").
- Both statements close every open cursor; the declarations remain.
- Issued when no database transaction is open, both complete with `SQLCODE` 0.

***Differences from DB2***

Under CICS, DB2 refuses `EXEC SQL COMMIT` and `ROLLBACK` (`SQLCODE` -925 and -926) and requires `EXEC CICS SYNCPOINT`. bricks accepts them, with the restricted effect described above.

### Refused Commits

At a commit point, bricks commits the database transaction before the file and temporary storage updates. If PostgreSQL refuses the commit, the file and temporary storage updates are backed out too and the task abends. No `HANDLE ABEND` exit is driven, because the unit of work has already been backed out:

| Abend | Meaning |
|---|---|
| ASP3 | The database backed the unit of work out, and no file or temporary storage update was pending. Causes include a deferred constraint, a serialization failure or a deadlock (SQLSTATE 23xxx, 40001, 40P01), and the failed-statement rule below (25P02). |
| ASP7 | The same, but file or temporary storage updates were pending. They are backed out as well. |
| ASPR | The connection broke during the commit, or the commit failed without an answer from the database. The outcome in the database is unknown. Pending file and temporary storage updates are backed out. |

**Figure 21-7. Abends for a refused commit**

The abend detail names the commit point: SYNCPOINT:, RETURN:, TASK END: or XCTL:, followed by the SQLSTATE when the database gave one. A refused commit at an XCTL abends the program that issued the XCTL, and the target never starts.

### A Failed Statement Spoils the Transaction

When one statement in a PostgreSQL transaction fails, PostgreSQL marks the whole transaction as failed. Every later statement in it fails with SQLSTATE 25P02 (SQLCODE -100, "current transaction is aborted"), and its COMMIT becomes a rollback.

- If no statement in the transaction had changed anything, nothing is lost, and bricks treats the commit as clean, as DB2 (which fails only the statement) would.
- If a change had succeeded before the failure, that change is lost. The commit point abends the task ASP3 with SQLSTATE 25P02, or ASP7 when file or temporary storage updates were pending.

The rule holds across `XCTL`: a failed statement in the program that issued the `XCTL` spoils the transaction that the target continues. bricks counts every statement other than `SELECT` as a change. A `SELECT` that calls a function which writes to the database is not counted, so its work is lost without an abend.

**Note:**

Test `SQLCODE` after every change and issue `EXEC CICS SYNCPOINT ROLLBACK` yourself when one fails. A program that uses `WHENEVER SQLERROR CONTINUE` and carries on after an error sees a series of 25P02 failures for every later statement, and a log line for each.

## Time Limits and Automatic Retry

### *Statement time limit*

`db_stmt_timeout` (default 30 seconds) bounds every statement, from sending it to receiving the last result byte. The limit applies to `SELECT INTO`, `INSERT`, `UPDATE` and `DELETE`, to cursor `OPEN`, and to each batch that `FETCH` obtains from the server. bricks also sets the PostgreSQL `statement_timeout` for the transaction. Whichever limit fires first, the statement returns `SQLCODE` -952 and `SQLSTATE` 57014. Because a canceled statement fails the transaction, the 25P02 rule above then applies.

### *Waiting for a connection*

Each task that uses SQL holds one pooled connection from its first data statement until its unit of work ends. When all `db_max_conns` connections are in use, the task waits up to 30 seconds for one to become free and then abends `AD2P`. The 30-second wait bounds only the acquisition of a connection, not the statements that run on it or the length of the transaction.

### *Automatic retry of deadlocks*

When the **first** data statement of a database transaction fails as a deadlock or serialization victim (`SQLCODE` -911), and `db_retry_transient=yes` (the default), bricks rolls the transaction back, waits briefly (50 milliseconds times the attempt number, at most 500 milliseconds) and issues the statement again, up to `db_retry_max` times (default 1). A failure of any later statement, or of any statement after a cursor was opened, is returned to the program, because earlier work would be lost.

### Database Abends

A database that cannot be used at all abends the task, rather than returning an `SQLCODE` that a program might ignore in a loop:

| Code | Raised when |
|---|---|
| `ADBU` | The database is unreachable: PostgreSQL is down, the host is wrong, the network is broken, or the connection was lost during the task (the server ended the session, SQLSTATE class 08 or 57P01 to 57P03). |
| `AD2S` | PostgreSQL refused the sign-on: wrong `db_user` or `db_password`, or the role does not exist (SQLSTATE class 28). |
| `AD2P` | No connection became free: all `db_max_conns` pooled connections stayed in use for 30 seconds. |

**Figure 21-8. Database abend codes**

All three are ordinary abends, in COBOL, PL/I and REXX alike:

- A transaction dump is taken, with the failing SQL statement in its `CMD` record ("Transaction Dumps" in Chapter 16).
- A `HANDLE ABEND` exit at the failing program's `LINK` level, or at any level above it, gets control. `EIBABCODE` and `ASSIGN ABCODE` name the code.
- If no exit handles the abend, the whole unit of work is backed out (the database transaction, file and temporary storage updates, and record locks), including the work of any program that reached the failing one by `XCTL`. Message `DFHAC2236` is logged and the operator sees the `DFHAC2206` screen, which for `ADBU` and `AD2S` says what to check.

`AD2P` usually means that tasks hold their database transaction while they wait for the operator. Issue `EXEC CICS SYNCPOINT` before a `SEND MAP` that waits for input, or ask for a larger `db_max_conns`. See "Other Transaction Abend Codes" in Appendix C for the full list of abend codes.

***Differences from IBM CICS***

- IBM's DB2 attachment with `THREADWAIT(NO)` abends `AD2P` at once. bricks waits up to 30 seconds, like `THREADWAIT(YES)`, and then abends.
- `ADBU` is a bricks code; under IBM CICS a DB2 that is not available gives `SQLCODE` -923 or an `AEY9` abend.
- A connection lost during the task abends `ADBU`; it is not reported as `SQLCODE` -924.

## The SQL Communication Area

After every `EXEC SQL` statement bricks sets the fields of the SQL communication area (SQLCA). Every field is reset first, so no value from an earlier statement remains.

| Field | COBOL | Contents |
|---|---|---|
| `SQLCODE` | `PIC S9(8)` | 0 for success, +100 for no data, negative for an error. |
| `SQLSTATE` | `PIC X(5)` | The 5-character state: `00000` for success, `02000` for no data, otherwise the PostgreSQL state, or `HV000` for a failure that carried none. Blank for -1. |
| `SQLERRMC` | `PIC X(70)` | The error message, converted to SBCS and cut to 70 characters. The full message is written to the bricks log for every negative `SQLCODE`. |
| `SQLERRP` | `PIC X(8)` | `BRICKS`. |
| `SQLERRD1`–`SQLERRD6` | `PIC S9(9)` | `SQLERRD3` holds the rows inserted, changed or deleted, or 1 after a successful `SELECT INTO` or `FETCH`. The others are 0. |
| `SQLWARN0`–`SQLWARN9`, `SQLWARNA` | `PIC X` | Blank, except `SQLWARN8` and `SQLWARN0`, which are `W` when characters were lost in the conversion to SBCS. |

**Figure 21-9. Fields of the SQLCA**

- **COBOL.** The fields are added to every program automatically; they are not declared. `COPY SQLCA` provides named constants for the `SQLCODE` and `SQLSTATE` values (next section).
- **PL/I.** The SQLCA is predeclared: `SQLCODE FIXED BIN(31)`, `SQLSTATE CHAR(5)`, `SQLERRMC CHAR(70) VAR` and `SQLERRD(6) FIXED BIN(31)`.
- **REXX.** The fields are ordinary variables, set after each statement. `RC` is also set to the value of `SQLCODE`, so `SIGNAL ON ERROR` can catch SQL failures. Choose host variable names that are not also used as stem tails (see "Derived Names of Compound Symbols" in Chapter 36).

A PL/I task started from the scheduler, without a terminal, has no database connection; every statement returns `SQLCODE -1`.

### SQLCODE Values

bricks maps PostgreSQL states to DB2-style `SQLCODE` values. A state with no specific mapping gives the general code -100, with the real state in `SQLSTATE`.

| SQLCODE | SQLSTATE | SQLCA name | Meaning |
|---|---|---|---|
| 0 | 00000 | SQL-OK | Success. |
| +100 | 02000 | SQL-NODATA | SELECT INTO or FETCH found no row. |
| −1 | blank or 08001 | SQL-NOCONFIG | No database: SQL not configured, the transaction's database is not in databases.conf, or CONNECT TO named an unknown or offline database. |
| −2 | 42501 | SQL-DDLREJECTED | Database or user DDL refused. |
| −100 | varies | SQL-GENERIC | Any other failure, including 25P02 and the cursor-state errors. |
| −104 | 42601 | SQL-SYNTAX | Syntax error in the statement. |
| −203 | 42702 | SQL-AMBIG-COL | Ambiguous column reference. |
| −204 | 42P01 | SQL-UNDEF-TBL | Table or view does not exist. |
| −206 | 42703 | SQL-UNDEF-COL | Column does not exist. |
| −407 | 23502 | SQL-NOTNULL | Null value for a NOT NULL column. |
| −433 | 22001 | SQL-STRTRUNC | Value too long for its column. |
| −510 | 42828 | SQL-CURSOR-RO | WHERE CURRENT OF on a cursor without FOR UPDATE. |
| −530 | 23503 | SQL-FKVIOL | Foreign key violation. |
| −545 | 23514 | SQL-CHKVIOL | Check constraint violation. |
| −551 | 42501 | SQL-INSUF-PRIV | Not authorized for the object. |
| −802 | 22003 | SQL-NUMOVERFLOW | Numeric value out of range. |
| −803 | 23505 | SQL-DUPKEY | Duplicate key (unique or primary key). |
| −811 | 21000 | SQL-MULTIPLEROWS | SELECT INTO found more than one row. |
| −911 | 40001, 40P01 | SQL-DEADLOCK | Serialization failure or deadlock; the transaction was rolled back. |
| −952 | 57014 | SQL-TIMEOUT | Statement canceled by a time limit. |

**Figure 21-10. SQLCODE values returned by bricks**

Most constants in the SQLCA copybook also have an IBM-style alias, for example DB2−DUP−KEY for SQL−DUPKEY and DB2−READ−ONLY for SQL−CURSOR−RO. The copybook also defines SQL−CONNLOST (-924) for source compatibility; bricks abends ADBU instead of returning it. There is no copybook for PL/I or REXX; compare the numbers directly.

### Examples in Each Language

The samples `SQLD` (COBOL, `cobol/sqld.cob`) and `SQLR` (REXX, `rexx/sqlr.rexx`) read one row of table `customers_sql` by key and show the result and the SQLCA on the screen. The table is created once with `psql`:

```
CREATE TABLE customers_sql (
    id   text PRIMARY KEY,
    name text NOT NULL
);
INSERT INTO customers_sql VALUES
    ('K001', 'Alice'), ('K002', 'Bob'), ('K003', 'Carol');
```

***COBOL***

```
       COPY SQLCA.
       ...
           EXEC SQL
               SELECT name INTO :NM
               FROM customers_sql
               WHERE id = :CUSTID
           END-EXEC.
           EVALUATE SQLCODE
               WHEN SQL-OK       PERFORM SHOW-NAME
               WHEN SQL-NODATA   PERFORM SHOW-NOT-FOUND
               WHEN OTHER        PERFORM SHOW-SQL-ERROR
           END-EVALUATE.
```

***REXX***

```
CUSTID = 'K001'
EXEC SQL
    SELECT name INTO :CUSTNM
    FROM customers_sql
    WHERE id = :CUSTID
END-EXEC
SELECT
  WHEN SQLCODE = 0   THEN SAY 'Customer:' CUSTNM
  WHEN SQLCODE = 100 THEN SAY 'Not found'
  OTHERWISE SAY 'SQL error' SQLCODE':' SQLERRMC
END
```

***PL/I***

```
SQLP: PROC OPTIONS(MAIN);
   DCL CUSTID CHAR(8) INIT('K001');
   DCL CUSTNM CHAR(40) VAR;
   DCL NM_IND FIXED BIN(15);
   DCL 1 ROW, 2 NAME CHAR(40) VAR, 2 CITY CHAR(20) VAR;

   EXEC SQL WHENEVER SQLERROR DO CALL DB_FAILED;
   EXEC SQL SELECT name INTO :CUSTNM :NM_IND
            FROM customers_sql WHERE id = :CUSTID;
   IF SQLCODE = 100 THEN PUT SKIP LIST('not found');
   ELSE IF NM_IND < 0 THEN PUT SKIP LIST('name is NULL');
   ELSE PUT SKIP LIST('customer', CUSTNM);

   EXEC SQL DECLARE C1 CURSOR FOR
            SELECT name, city FROM customers_sql WHERE id >= :CUSTID;
   EXEC SQL OPEN C1;
   DO WHILE ('1'B);
      EXEC SQL FETCH C1 INTO :ROW;
      IF SQLCODE ^= 0 THEN LEAVE;
      PUT SKIP LIST(ROW.NAME, ROW.CITY);
   END;
   EXEC SQL CLOSE C1;

DB_FAILED: PROC;
   PUT SKIP LIST('SQL failed', SQLCODE, SQLSTATE, SQLERRMC);
   EXEC CICS SYNCPOINT ROLLBACK;
   EXEC CICS ABEND ABCODE('SQLF');
END DB_FAILED;
END SQLP;
```

## Restrictions

| Not supported | Alternative |
|---|---|
| Dynamic SQL: PREPARE, EXECUTE, DESCRIBE, SQLDA | Use static statements with host variables. |
| Result rows of CALL or of a SELECT without INTO | Select from a function with SELECT ... INTO or a cursor. |
| Scrollable cursors, WITH HOLD, rowset fetch | Forward-only cursors within one unit of work. |
| A cursor kept across SYNCPOINT or across a pseudo-conversational RETURN TRANSID | Save the key of the last row (for example in the COMMAREA) and reopen the cursor with WHERE key > :LASTKEY. |
| :hostvar INDICATOR :ind | :hostvar :ind. |
| Host variable arrays (PL/I) | One row per statement. |
| CREATE, DROP or ALTER of a database, user, role or tablespace | CEDA DATABASE (see "DATABASE" in Chapter 6). |

**Figure 21-11. Embedded SQL restrictions**

Table definitions (CREATE TABLE, ALTER TABLE) are passed to PostgreSQL like any other statement, but schema changes are better made with psql outside the region.

## Summary of Differences from DB2 under CICS

| Area | bricks behavior |
|---|---|
| SQL dialect | PostgreSQL. No precompile or bind step; statements are checked when they run. |
| UPDATE, DELETE with no row | SQLCODE 0 and SQLERRD3 = 0, not +100. |
| Cursor-state errors | SQLCODE -100 with SQLSTATE 34000, 24501 or 24502, where DB2 returns -504, -501 or -502. |
| Unmapped errors | SQLCODE -100, with the PostgreSQL SQLSTATE. |
| WHENEVER | Execution order, not source order. |
| EXEC SQL COMMIT, ROLLBACK | Accepted; act on the database only. |
| XCTL to another database | Implicit syncpoint at the XCTL. |
| Database unavailable | Abend ADBU; AD2P after a 30-second wait for a connection. |
| Deadlock | The first statement of a transaction is retried automatically (db_retry_transient). |

**Figure 21-12. Embedded SQL: differences from DB2 under CICS**

Part 6

# COBOL

This part describes COBOL as bricks implements it: the source format, the DATA and PROCEDURE divisions, the Execute Interface Block and `EXEC CICS` in a COBOL program, the supplied copybooks, and the restrictions of the implementation.

**Chapter 22. COBOL Source Format** ..... **22-1**
**Chapter 23. COBOL DATA DIVISION** ..... **23-1**
**Chapter 24. COBOL PROCEDURE DIVISION** ..... **24-1**
**Chapter 25. The EIB and EXEC CICS in COBOL** ..... **25-1**
**Chapter 26. COBOL Copybooks** ..... **26-1**
**Chapter 27. COBOL Restrictions** ..... **27-1**

Chapter 22

# COBOL Source Format

bricks includes an interpreter for a subset of IBM Enterprise COBOL. A COBOL program issues the same `EXEC CICS`, `EXEC CICS WEB` and `EXEC SQL` commands, through the same command handler, as a REXX or PL/I program; what each command does is described in Part 3, Part 4 and Part 5. The chapters of Part 6 describe the COBOL language as bricks implements it:

- Chapter 22, the structure of a program, the reference format, literals and the character set
- Chapter 23, the DATA DIVISION: level numbers, PICTURE, USAGE, VALUE, OCCURS, REDEFINES, condition-names and the LINKAGE SECTION
- Chapter 24, the PROCEDURE DIVISION: statements, conditions, reference modification, intrinsic functions and pointers
- Chapter 25, the EXEC interface block and `EXEC CICS` in COBOL, including what happens when a command raises a condition
- Chapter 26, the `COPY` statement and the supplied copybooks
- Chapter 27, restrictions and limits

COBOL programs are kept as `.cob` files in `runtime/cobol/` and are bound to a transaction identifier in `transactions.conf` (see "Transactions and Programs" in Chapter 1). A program is compiled when it is first dispatched and the parsed form is cached; the cache entry is replaced automatically when the program file, or any copybook it copies, changes.

The utility `brickscompile` checks the syntax of a program without running it:

```
brickscompile runtime/cobol/hello.cob
runtime/cobol/hello.cob: OK
       lines      35
  paragraphs       1
  statements      10
   EXEC CICS       3
     elapsed  0s.0ms
```

A directory argument checks every `.cob`, `.cbl` and `.cobol` file below it. `brickscompile` expands `COPY` statements from the `copybook_dir` of the nearest `bricks.cnf`. Its return code is 0 when every file compiled, 1 when any file has an error and 2 for a usage or I/O error.

### Program Structure

A bricks COBOL program consists of the IDENTIFICATION, ENVIRONMENT, DATA and PROCEDURE divisions, in that order. The compiler rejects a program whose divisions are out of order or missing, rather than attempting to recover.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. program-name.
[ENVIRONMENT DIVISION.]
DATA DIVISION.
WORKING-STORAGE SECTION.
   [data-description-entry] ...
[LINKAGE SECTION.
   [data-description-entry] ...]
PROCEDURE DIVISION [USING data-name ...].
paragraph-name.
   [sentence] ...
[paragraph-name.
   [sentence] ...] ...
```

***Syntax rules***

1. `IDENTIFICATION DIVISION` and `PROGRAM-ID` are required. No other IDENTIFICATION DIVISION paragraph is accepted: `AUTHOR`, `DATE-WRITTEN` and the like are compile errors, so that information the program could not use is not silently lost.
2. The ENVIRONMENT DIVISION header is optional. When it is present it must be empty: no CONFIGURATION SECTION or INPUT-OUTPUT SECTION follows it.
3. The DATA DIVISION and the `WORKING-STORAGE SECTION` header are required, even when the section declares no items. The `LINKAGE SECTION` is optional and, when present, follows WORKING-STORAGE. No other DATA DIVISION section (FILE, LOCAL-STORAGE) is accepted.
4. `PROCEDURE DIVISION USING` is accepted and the names in the list are ignored. COMMAREA and EIB addressability is established by the runtime (see "Automatically Declared Items" in Chapter 25).
5. The PROCEDURE DIVISION consists of paragraphs only. Section headers (`name SECTION.`) are not accepted.

***Example***

The supplied sample `HELC` (`runtime/cobol/hello.cob`) is a complete program:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. HELC.

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 USR     PIC X(8).
       01 TRM     PIC X(4).

       01 SCR.
          05 INFOLINE PIC X(78).
          05 GREETING PIC X(20).
          05 FOOTER   PIC X(60).

       PROCEDURE DIVISION.
       MAIN.
           EXEC CICS ASSIGN USERID(USR) TERMID(TRM) END-EXEC.

           MOVE SPACES TO INFOLINE.
           MOVE SPACES TO GREETING.
           MOVE SPACES TO FOOTER.

           MOVE 'HELLO from COBOL!'    TO GREETING.
           MOVE 'COBOL transaction running under bricks' TO INFOLINE.
           MOVE 'ENTER=Continue'       TO FOOTER.

           EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE END-EXEC.

           EXEC CICS RETURN END-EXEC.
           GOBACK.
```

Execution begins with the first paragraph and falls through the paragraphs in source order. The task ends at `EXEC CICS RETURN`, at `STOP RUN`, `GOBACK` or `EXIT PROGRAM`, or when control runs off the end of the last paragraph; see "Ending the Program" in Chapter 25.

### Reference Format

bricks reads COBOL source in free form. There is no sequence area, no indicator area, no Area A or Area B and no column-72 limit: a statement may begin in any column and continue over as many lines as necessary. Source written in the conventional fixed layout (division and paragraph headers in column 8, statements from column 12) is accepted unchanged, as long as columns 1 through 6 are blank. The rules are summarized in the following table.

| Element | Rule |
|---|---|
| Line comment | A line whose first non-blank character is * in columns 1 through 7 is a comment line. |
| Floating comment | *> begins a comment that extends to the end of the line. It may follow code on the same line. |
| Names | A user-defined word begins with a letter or _ and continues with letters, digits, _ and – (`CUST-RECORD`). Names and reserved words are not case-sensitive; the compiler folds them to upper case. |
| Periods | A period is required after every division header, section header, paragraph name and data description entry. In the PROCEDURE DIVISION a period ends a sentence (see "Sentences and Scope Terminators" in Chapter 24). |
| Separators | Operands are separated by spaces. A comma is accepted only between the values of a level-88 entry, between `INSPECT` phrases, between subscripts and inside `EXEC` blocks; elsewhere, such as between the receivers of a `MOVE`, it is a compile error. |
| Characters | Source files are UTF-8; a leading byte order mark is ignored. Names, reserved words and PICTURE strings must be ASCII. Comments may contain any character. Literals are described under "Character Set". |

**Figure 22-1. COBOL reference format rules**

## Literals

### *Alphanumeric literals*

An alphanumeric literal is enclosed in apostrophes or in quotation marks. The delimiting character is written twice to include it in the literal: `'don''t'`, `"say ""hi"""`.

### *Numeric literals*

A numeric literal is a string of digits with an optional decimal point (`42`, `1.5`, `0.050`) and an optional leading sign written immediately against the digits (`-1`, `+5`, `-1.5`). A numeric literal may be used wherever a numeric operand is allowed, including VALUE clauses and conditions. The spacing distinguishes a signed literal from an operator: `-1` is a literal, while `- 1` is the subtraction operator applied to 1. Inside an arithmetic expression both readings give the same result; in a statement that takes a list of operands they differ, so that `DISPLAY A -1` displays two items and `DISPLAY A - 1` is a compile error.

### *Hexadecimal literals*

`X'F3'` and `X"7C"` denote the bytes whose hexadecimal values are given, two digits per byte. Hexadecimal literals are used chiefly to test the attention identifier (`IF EIBAID = X'F3'`); the named constants of the `DFHAID` copybook are clearer (see "DFHAID" in Chapter 26).

### *Figurative constants*

The figurative constants in the following table may be written in the singular or the plural; the forms are equivalent.

| Constant | Value |
|---|---|
| `SPACE, SPACES` | One or more blanks (`X'20'`). |
| `ZERO, ZEROS, ZEROES` | The value zero for a numeric receiver; the character `0` repeated for an alphanumeric receiver. |
| `HIGH-VALUE, HIGH-VALUES` | `X'FF'` repeated. |
| `LOW-VALUE, LOW-VALUES` | `X'00'` repeated. |
| `QUOTE, QUOTES` | The quotation mark `"` repeated. |
| `NULL, NULLS` | The null pointer, for `USAGE POINTER` items (see "SET (Pointers)" in Chapter 24). |

**Figure 22-2. Figurative constants**

When a figurative constant is moved to a group item, it fills the whole length of the group. The `ALL literal` form is not supported.

### Character Set

COBOL program data in bricks is one byte per character, as on a mainframe. Each byte of a `PIC X` item is one character of the bricks single-byte character set: ISO-8859-1 (Latin-1), with a graphic plane in place of the C1 control characters `X'80'` through `X'9F'`, as shown in the following table.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| X'8n' | € | ▉ | ▀ | ▄ | ▌ | ▐ | ▪ | ─ | │ | ┌ | ┐ | └ | ┘ | ├ | ┤ | ┬ |
| X'9n' | ┴ | ┼ | ═ | ‖ | · | ← | → | ↑ | ↓ | ≤ | ≥ | ≠ | ○ | ◊ | ¯ | ≡ |

**Figure 22-3. The bricks graphic plane, X'80' to X'9F'**

UTF-8 is used only at the edges of the system: the terminal, the web interface, SQL, files and the log. The CICS layer converts once, at the edge. Within the program the following rules apply.

- **Literals.** Each character of an alphanumeric literal is converted to its single byte when the program is compiled: `'Grüße'` is 5 bytes long and `'▉'` is `X'81'`. A character outside the table (for example a CJK character, an em dash, an emoji, a C1 control or `U+FFFD`), or invalid UTF-8, is a compile error and is never replaced by `?`.
- **Hexadecimal literals** are raw bytes and are not converted. `X'C4'` is Ä, not the EBCDIC letter `D`. The AID constants keep their IBM values because `EIBAID` holds the raw AID byte.
- **Byte equals character** in `MOVE`, reference modification, `REDEFINES`, group moves, `STRING`, `UNSTRING`, `INSPECT`, the intrinsic functions, COMMAREA offsets and record keys.

- **Case.** `FUNCTION UPPER-CASE` and `LOWER-CASE` fold a-z and the Latin-1 letters (ä and Ä, þ and Þ) and preserve the length; ß, ÿ, µ, ª, º, ×, ÷ and the graphic plane are unchanged. `INSPECT CONVERTING` is a plain byte table, so a program that folds case with it must list the Latin-1 letters explicitly (see the sample `UPPR`, `runtime/cobol/uppr.cob`).
- **Class conditions.** `ALPHABETIC`, `ALPHABETIC-UPPER` and `ALPHABETIC-LOWER` accept the ASCII letters and the space only, as on IBM: `'Müller' IS ALPHABETIC` is false.
- **Collating sequence.** Comparison is in byte order: `'Ä' > 'Z'` is true and `'A' THRU 'Z'` does not include Ä. The EBCDIC collating sequence is not emulated.
- **HIGH-VALUES** is `X'FF'`, which `DISPLAY`, the terminal and JSON show as ÿ. **LOW-VALUES** is `X'00'`.
- **Numeric MOVE of text.** A letter (ASCII or Latin-1) in the sending item is a data exception: the receiver is set to zero and the event is logged. Other symbols, such as €, £, $ or a no-break space, are dropped.
- **FUNCTION TRIM** removes ASCII blanks only; the no-break space `X'A0'` is data. `FUNCTION NUMVAL` rejects any non-numeric character.
- **FUNCTION ORD, CHAR, BYTE-LENGTH.** `ORD('Ä')` is 197 (byte value plus 1), `CHAR(197)` is Ä, and `BYTE-LENGTH` equals `LENGTH`.
- **JSON.** JSON text in a `PIC X` item is single-byte text. `JSON GENERATE` writes every byte from `X'80'` upward unchanged (the web edge sends it as UTF-8) and uses the `\u00hh` escape only for control characters below `X'20'`. `JSON PARSE` stores a `\uXXXX` escape as its single byte, or as `?` when the code point is not in the table.

The compiler reports character errors with the line and the column, counting characters rather than bytes, in ASCII:

```
line 12 col 22: U+4E2D in literal is not a bricks SBCS character
line 13 col 18: invalid UTF-8 byte X'C4' in literal
line 14 col 20: character U+00C4 not allowed outside quotes (ASCII only)
```

### Compile-Time Diagnostics

The compiler stops at the first error. The message gives the source file name (without its directory), the line number in that file and a short description; an error inside a copybook names the copybook and its own line number. Every message is plain ASCII and is short enough to be shown on the terminal. The limits that the compiler enforces are listed in "Limits" in Chapter 27.

### Differences from IBM Enterprise COBOL

- Free-form reference format only; columns 1 through 6 must not hold sequence numbers, and there is no continuation indicator (`-` in column 7) or page-eject indicator (`/`).
- Only `PROGRAM-ID` is accepted in the IDENTIFICATION DIVISION, the ENVIRONMENT DIVISION must be empty, and the PROCEDURE DIVISION has no sections.
- The code page is Latin-1 plus the graphic plane, not an EBCDIC CCSID (037 or 1140). Hexadecimal literals denote Latin-1 bytes, collation is Latin-1 byte order, and JSON text is single-byte, not UTF-8 (CCSID 1208).
- `PIC N`, `USAGE NATIONAL`, `PIC U`, `FUNCTION NATIONAL-OF` and `DISPLAY-OF` are not supported.

Chapter 23

# COBOL DATA DIVISION

The DATA DIVISION describes every data item that the program uses. It contains a `WORKING-STORAGE SECTION`, which is required, and an optional `LINKAGE SECTION`. Each task that runs the program receives its own copy of WORKING-STORAGE, initialized from the VALUE clauses; storage is never shared between tasks.

Several items are declared automatically when the program does not declare them itself: the EXEC interface block (`DFHEIB` and its fields), `DFHCOMMAREA`, `RC`, the SQLCA fields and the JSON special registers. They are listed in "Automatically Declared Items" in Chapter 25.

### Data Description Entry

```
level-number {data-name | FILLER}
    [REDEFINES data-name-2]
    [{PICTURE | PIC} [IS] picture-string]
    [[USAGE [IS]] {DISPLAY | COMP | COMP-4 | COMPUTATIONAL | BINARY
                  | COMP-3 | COMPUTATIONAL-3 | PACKED-DECIMAL
                  | POINTER}]
    [OCCURS integer [TIMES]]
    [VALUE [IS] literal]
    [BASED] .
```

***Syntax rules***

1. REDEFINES, when present, must immediately follow the data name. The other clauses may be written in any order.
2. Every entry ends with a period.
3. An item with subordinate items is a group item. A group item has no PICTURE, USAGE or VALUE clause.
4. An elementary item has a PICTURE clause, except a `USAGE POINTER` item, which has none.
5. Two items with the same parent may not have the same name, except `FILLER`.

## Level Numbers

| Level | Meaning |
|---|---|
| `01` | A record: a top-level item that owns its own storage. |
| `02–49` | A subordinate item. It occupies part of its parent's storage at an offset fixed by the items that precede it. |
| `77` | A stand-alone elementary item, equivalent to an elementary 01. See the note that follows this table. |
| `88` | A condition-name for the preceding data item (see "Condition-Names (Level 88)"). |

**Figure 23-1. Level numbers**

Levels 66 (RENAMES) and 78 (constant) are not supported.

**Note:**

bricks places each item under the nearest preceding item that has a lower level number, and it applies this rule to level 77 as to any other level. A 77 entry written after an 01 entry (or after any of its subordinate items) is therefore taken as subordinate to the entry that precedes it: if that entry has a VALUE clause, the 77 entry is a compile error (`VALUE not allowed on group items`); otherwise it silently becomes part of the record. Code level-77 items before the first 01 entry of the section, or use level 01.

## PICTURE Clause

The PICTURE clause (`PIC` is an abbreviation) gives the category and the length of an elementary item. A symbol followed by `(n)` stands for *n* occurrences of the symbol: `9(3)` and `999` are equivalent.

### *Unedited pictures*

| Picture | Category | Description |
|---|---|---|
| `X(n)` | Alphanumeric | *n* bytes, up to 1 MiB. A MOVE left-justifies the value and pads with spaces. |
| `A(n)` | Alphanumeric | Treated exactly as `X(n)`. The contents are not checked; use the `ALPHABETIC` class condition to test them. |
| `9(n)` | Numeric | *n* decimal digits, unsigned. A MOVE right-justifies the value and pads with zeros. |
| `S9(n)` | Numeric | Signed. In `USAGE DISPLAY` the item occupies *n*+1 bytes (see "Storage of Signed DISPLAY Items"). |
| `9(n)V9(m)`<br>`S9(n)V9(m)` | Numeric | An assumed decimal point. `V` occupies no storage; the item holds *n*+*m* digits, of which *m* are decimal places. |

**Figure 23-2. Unedited PICTURE strings**

A numeric item may hold at most 18 digits, integer and fraction together; `PIC S9(19)` and wider are compile errors. Arithmetic is carried out in fixed-point decimal, so the scale given by `V` is kept through `MOVE`, `COMPUTE` and the arithmetic statements. The symbol `V` must follow at least one `9` (`PIC V99` is not accepted; write `PIC 9V99`), and the scaling symbol `P` is not supported.

### *Numeric-edited pictures*

A numeric-edited item holds the printable form of a number. The editing symbols in the following table are supported.

| Symbol | Effect |
|---|---|
| Z | Zero suppression: a leading zero is replaced by a space. |
| . | Inserts the decimal point at this position. |
| , | Inserts a comma at this position when a significant digit precedes it. |
| $ | A single $ is a fixed currency sign and must be the leftmost symbol. Two or more form a floating string: one $ is printed immediately to the left of the first significant digit. |
| * | Check protection: a leading zero is replaced by *. |
| 9 | A digit position, never suppressed. |

**Figure 23-3. Editing symbols**

The following rules apply to a numeric-edited picture:

- Z, a floating $ string and * are mutually exclusive. A single fixed $ may be combined with Z ($ZZ9.99).
- Z and * may not appear to the right of the decimal point.
- The length of the item is the number of symbols in the picture. At most 18 digit positions are allowed.
- A numeric-edited picture may not begin with S.

The following table shows the result of moving a value to items of several numeric-edited pictures.

| Value | Picture | Result |
|---|---|---|
| 42 | ZZ9 | " 42" |
| 0 | ZZ9 | " 0" |
| 1.5 | ZZZ.99 | " 1.50" |
| 1234.56 | Z,ZZ9.99 | "1,234.56" |
| 12345.67 | $$$,$$9.99 | "$12,345.67" |
| 0.45 | $$$,$$$.99 | " $.45" |
| 12.34 | **,**9.99 | "****12.34" |
| 42 | $9999 | "$0042" |

**Figure 23-4. Numeric editing examples**

A scaled sending item is edited at its own scale: when N is PIC 9(5)V99 VALUE 12345.67 and N-Z is PIC ZZ,ZZ9.99, MOVE N TO N-Z stores 12,345.67. When a numeric-edited item is the sending item of a numeric MOVE, the insertion characters are removed and the digits are read back as a number (de-editing).

The editing symbols +, -, CR, DB, B, 0 and /, and the BLANK WHEN ZERO clause, are not supported.

## USAGE Clause

The USAGE clause gives the internal representation of an item. The word USAGE (and IS) may be omitted: PIC S9(8) COMP and PIC S9(8) USAGE IS COMP are equivalent.

| USAGE | Synonyms | Representation |
|---|---|---|
| DISPLAY | (default) | One byte per digit or character. |
| COMP | COMP-4<br>COMPUTATIONAL<br>BINARY | Big-endian two's-complement binary in 2, 4 or 8 bytes. |
| COMP-3 | COMPUTATIONAL-3<br>PACKED-DECIMAL | Packed decimal: two digits per byte, the sign in the low-order half-byte of the last byte. |
| POINTER | | A 4-byte pointer, written without a PICTURE clause (see "SET (Pointers)" in Chapter 24). |

**Figure 23-5. USAGE clause values**

`COMP` and `COMP-3` require a numeric picture (`9` or `S9`). `COMP-1` and `COMP-2` (floating point) are rejected with the message `not yet supported`.

***Binary items***

The length of a binary item depends on the total number of digits in its picture (integer and fraction), as on IBM:

| Digits | Bytes |
|---|---|
| 1 to 4 | 2 (halfword) |
| 5 to 9 | 4 (fullword) |
| 10 to 18 | 8 (doubleword) |

**Figure 23-6. Length of binary items**

The sign does not change the length: `PIC 9(4) COMP` and `PIC S9(4) COMP` both occupy two bytes, and the value is always held as a signed two's-complement integer. The scale given by `V` is kept when a value moves between DISPLAY and binary items.

***Packed-decimal items***

A packed-decimal item occupies (*digits* + 1) / 2 bytes, rounded up: 1 digit takes 1 byte, 4 digits 3 bytes, 9 digits 5 bytes, 15 digits 8 bytes and 18 digits 10 bytes. The sign half-byte is `X'C'` for a signed positive value, `X'D'` for a negative value and `X'F'` for an unsigned item. When an item is read, `X'B'` and `X'D'` are negative and every other sign value is positive.

`PIC S9(15) COMP-3` is the usual receiver for an absolute time:

```
01  SYS-DT    PIC S9(15) COMP-3.
01  YYYYMMDD  PIC X(8).
    EXEC CICS ASKTIME ABSTIME(SYS-DT) END-EXEC.
    EXEC CICS FORMATTIME ABSTIME(SYS-DT) YYYYMMDD(YYYYMMDD) END-EXEC.
```

### Storage of Signed DISPLAY Items

bricks stores a `PIC S9(n)` DISPLAY item in *n*+1 bytes. Byte 1 is a separate leading sign, a space for a positive value and `-` for a negative value; bytes 2 through *n*+1 hold the digits. In IBM terms the item behaves as `SIGN IS LEADING SEPARATE`, with a space in place of `+`. IBM's default is *n* bytes with the sign held in the zone of the last digit.

The difference is visible through REDEFINES and in record layouts: the alphanumeric view of `PIC S9(4)` is `PIC X(5)` and reads `-0123` for the value -123. A copybook ported from z/OS that redefines a signed DISPLAY field must be widened by one byte per such field. Unsigned DISPLAY, `COMP` and `COMP-3` items have exactly the IBM layout.

### VALUE Clause

The VALUE clause gives the initial contents of an elementary item. It may be a literal or one of the figurative constants `SPACES`, `ZEROS`, `HIGH-VALUES`, `LOW-VALUES` and `QUOTES` (singular forms accepted). An item without a VALUE clause starts as spaces (alphanumeric) or zero (numeric).

- A VALUE clause on a group item, or on any item subordinate to a group that has one, is a compile error. Give each elementary item its own VALUE.
- A **numeric** literal is placed according to the receiving picture, exactly as by `MOVE` and `COMPUTE`:

  ```
  01 A PIC S9(4)V99 VALUE 1.5.      *> 1.50, stored 000150
  01 B PIC S9(4)V99 VALUE 2.        *> 2.00, stored 000200
  01 C PIC S9(4)    VALUE 1.5.      *> 1, truncated toward zero
  ```

  The stored digits are not a scaled value: `PIC S9(4)V99 VALUE 150` is one hundred fifty, not 1.50.
- An **alphanumeric** literal is stored as bytes, so `VALUE '150'` on the same item is 1.50: the digits are laid into the item's bytes. `MOVE '150'` behaves the same way.

### OCCURS Clause

```
OCCURS integer [TIMES]
```

OCCURS defines a table of *integer* occurrences of the item, which may be an elementary item or a group. The limit is 4096 occurrences. A table element is referred to by a subscript in parentheses: a 1-based integer expression, which may be a literal, a data item or arithmetic (`K(I + 1)`). Subscripts are checked against the bounds at runtime. A subscript on a group applies to its subordinate items as well:

```
01 TABLE-AREA.
   05 ROW OCCURS 15 TIMES.
      10 K  PIC X(8).
      10 N  PIC X(28).
01 I  PIC 9(4).
    MOVE 'KEY-A' TO K(1).
    MOVE 'KEY-B' TO K(I).
```

Only one dimension is supported: an OCCURS item may not be subordinate to another OCCURS item. To address a two-dimensional table, declare it with one dimension and compute the subscript (see "PERFORM" in Chapter 24). `OCCURS DEPENDING ON`, `INDEXED BY`, `ASCENDING KEY` and `DESCENDING KEY` are not supported.

### FILLER

`FILLER` names storage that is not referred to. A group may contain any number of FILLER items. A FILLER item can be reached only through its parent, and it cannot be the object of REDEFINES.

### REDEFINES Clause

```
level-number data-name-1 REDEFINES data-name-2 ...
```

REDEFINES describes storage that a preceding item already occupies. No storage is allocated: the two names refer to the same bytes, and a change made through either name is visible through the other at once. The description of the name used in a statement decides how the bytes are interpreted.

***Syntax rules***

1. REDEFINES must immediately follow *data-name-1*.
2. *data-name-1* and *data-name-2* have the same level number (01 through 49, or 77).
3. *data-name-2* is the immediately preceding item at that level with the same parent. Only other redefinitions of the same item (with their subordinate items) may come between them.
4. Several redefinitions of one area all name the original item: `B REDEFINES A` and then `C REDEFINES A`, never `C REDEFINES B`.
5. The redefining item may not be longer than the item it redefines. This applies at level 01 too. A shorter item overlays the leading bytes.
6. Neither item may have an OCCURS clause, and neither may be subordinate to an OCCURS item. Items subordinate to the redefining item may have OCCURS.
7. No item in the redefining entry or below it may have a VALUE clause (level-88 values are allowed). The VALUE of the redefined item is visible through the redefinition.
8. FILLER may redefine an item, but a FILLER item cannot be redefined. The automatically declared items (see "Automatically Declared Items" in Chapter 25) cannot be redefined either.

***Examples***

Two views of one date:

```
01 WS-DATE      PIC 9(8) VALUE 20260611.
01 WS-DATE-R REDEFINES WS-DATE.
   05 WD-YYYY   PIC 9(4).
   05 WD-MM     PIC 9(2).
   05 WD-DD     PIC 9(2).
    DISPLAY WD-MM.              *> 06
    MOVE 12 TO WD-MM.
    DISPLAY WS-DATE.            *> 20261211
```

Rows of a map, named for `SEND MAP` and subscripted for a loop:

```
01 SCR.
   05 INFOLINE  PIC X(78).
   05 ROWS-AREA.
      10 ROW1   PIC X(76).
      10 ROW2   PIC X(76).
      10 ROW3   PIC X(76).
   05 R-TAB REDEFINES ROWS-AREA.
      10 ROWV   PIC X(76) OCCURS 3.
    MOVE WS-LINE TO ROWV(WS-SLOT).
    EXEC CICS SEND MAP('ESDS') FROM(SCR) ERASE END-EXEC.
```

`SEND MAP` finds the fields `ROW1` through `ROW3` by name, and because `ROWV` shares their bytes, whatever the loop stored through `ROWV(n)` is what the map displays.

***Differences from IBM Enterprise COBOL***

IBM allows a level-01 redefining item to be longer than the item it redefines; bricks does not. A signed DISPLAY item has a different length (see "Storage of Signed DISPLAY Items"), and a redefinition shows the bricks layout.

### Condition-Names (Level 88)

```
88 condition-name {VALUE | VALUES} [IS | ARE]
       literal-1 [{THRU | THROUGH} literal-2]
     [ [,] literal-3 [{THRU | THROUGH} literal-4] ] ...
```

A level-88 entry names a condition on the nearest preceding data item that is not itself a level-88 entry. The condition is true when the item's current value equals one of the literals or lies within one of the ranges. Values are separated by spaces or commas, and ranges may be mixed with single values:

```
01  RESP-CODE  PIC 9(3).
    88 RESP-OK        VALUE 0.
    88 RESP-MISSING   VALUE 13.
    88 RESP-RECOVER   VALUES 12, 26, 80.
    88 RESP-FATAL     VALUE 100 THRU 999.
    88 RESP-SOME      VALUES 0, 1 THRU 5, 9.
```

A numeric item is compared numerically; an alphanumeric item is compared byte by byte with trailing spaces ignored. A condition-name is used in a condition by name alone (`IF RESP-OK`) and combines with `NOT`, `AND` and `OR` like any other condition (see "Conditions" in Chapter 24).

Condition-names are kept apart from data names, but a condition-name that is the same as a data name is a compile error. The `SET condition-name TO TRUE` statement is not supported; move the value to the item instead.

### Qualification

The same name may be used for items in different groups. A reference to such a name must then be qualified with `OF` or its synonym `IN`; an unqualified reference to an ambiguous name is a runtime error (`ambiguous reference`).

```
01 SCR.
   05 CUSTNO PIC X(8).
01 DET.
   05 CUSTNO PIC X(8).
    MOVE 'A1234567' TO CUSTNO OF SCR.
    MOVE 'B7654321' TO CUSTNO OF DET.
    DISPLAY CUSTNO OF SCR.
```

`X OF Y` finds `X` anywhere below `Y`; `X OF Y OF Z` follows a chain of nested groups. A name that is unique in the program needs no qualifier. Two items with the same name and the same parent are a compile error, because no qualifier could tell them apart. A subscript may follow the data name or the qualifier: `K(5) OF ROW` and `K OF ROW(5)` are equivalent.

## LINKAGE SECTION

The LINKAGE SECTION describes records that are reached through an address rather than allocated in WORKING-STORAGE. bricks supports three kinds of LINKAGE record.

### *DFHCOMMAREA*

`DFHCOMMAREA` holds the communication area that the program received from `LINK`, `XCTL` or the previous task's `RETURN TRANSID COMMAREA`. A program may describe it in the LINKAGE SECTION with its own layout:

```
       LINKAGE SECTION.
       01 DFHCOMMAREA.
          05 CA-STATE   PIC X.
          05 CA-CUSTNO  PIC X(8).
```

When the program does not declare it, bricks declares `DFHCOMMAREA PIC X(2000)`. In either case it is a based record with a default buffer: `MOVE ... TO DFHCOMMAREA` works with no further set-up, and `SET ADDRESS OF DFHCOMMAREA` may point it at other storage. Only the `EIBCALEN` bytes that the program received are passed back or passed on from the default buffer. The rules for the length of a COMMAREA are described in "The COMMAREA" in Chapter 25.

### *DFHEIB*

The EXEC interface block is the based record `DFHEIB`. Its fields keep their IBM names (`EIBRESP`, `EIBAID`, `EIBCALEN`, ...) and are referred to without qualification. Its layout is given in "The EXEC Interface Block" in Chapter 25.

### *Based records*

An 01 entry with the `BASED` clause has no storage of its own. BASED is valid only at level 01.

```
       LINKAGE SECTION.
       01 CUST-REC BASED.
          05 CUST-NO   PIC X(8).
          05 CUST-NAME PIC X(30).
```

The record is null until `SET ADDRESS OF CUST-REC TO ptr` gives it an address, where `ptr` came from `EXEC CICS GETMAIN`, `EXEC CICS ADDRESS` or `ADDRESS OF` another item. After that, every reference to `CUST-NO` or `CUST-NAME` reads and writes the addressed bytes. The following are runtime failures that end the task without affecting the region:

- A reference through a based record that has no address is a null pointer reference.
- A numeric, `COMP`, `COMP-3` or `POINTER` item whose addressed storage is shorter than the item is an addressing exception: the task abends `ASRA` (`ADDRESSING EXCEPTION - name`), as on IBM. An alphanumeric item may address a shorter area and uses only the bytes that are there.

See "SET (Pointers)" in Chapter 24 for the statements that set addresses.

Chapter 24

# COBOL PROCEDURE DIVISION

The PROCEDURE DIVISION is a sequence of paragraphs. Each paragraph is a paragraph name followed by a period and by one or more sentences. Paragraphs are the targets of `PERFORM` and `GO TO`; bricks has no PROCEDURE DIVISION sections. This chapter describes the structure of the division, the statements in alphabetical order, conditions, reference modification, the intrinsic functions and pointers. `EXEC CICS` is described in Chapter 25 and `EXEC SQL` in Chapter 21.

## Sentences and Scope Terminators

A sentence is one or more statements ended by a period. A period closes every statement that is still open, as in IBM COBOL; a statement inside an explicitly terminated body (`IF ... END-IF`, `EVALUATE ... END-EVALUATE`) has no period of its own. The explicit scope terminators are `END-IF`, `END-EVALUATE`, `END-PERFORM`, `END-STRING`, `END-UNSTRING`, `END-JSON` and, after `ON SIZE ERROR`, `END-COMPUTE`, `END-ADD`, `END-SUBTRACT`, `END-MULTIPLY` and `END-DIVIDE`. An in-line `PERFORM` must be ended by `END-PERFORM`; a period inside its body is a compile error (`in-line PERFORM needs END-PERFORM`).

## Statement Summary

| Category | Statements |
|---|---|
| Arithmetic | `ADD`, `COMPUTE`, `DIVIDE`, `MULTIPLY`, `SUBTRACT` |
| Data movement | `INSPECT`, `MOVE`, `SET` (pointers), `STRING`, `UNSTRING` |
| JSON | `JSON GENERATE`, `JSON PARSE` |
| Decision | `EVALUATE`, `IF` |
| Sequence control | `CONTINUE`, `EXIT`, `GO TO`, `GOBACK`, `PERFORM`, `STOP RUN` |
| Output | `DISPLAY` |
| Interfaces | `EXEC CICS`, `EXEC SQL` |

**Figure 24-1. Supported statements**

Any other verb (for example `ACCEPT`, `ALTER`, `CALL`, `INITIALIZE`, `SEARCH`) is a compile error (`unknown statement`). The complete list is in "PROCEDURE DIVISION" in Chapter 27.

Wherever a statement names a receiving or sending data item, the name may be qualified with `OF` or `IN` and, for a table element, subscripted (see "Qualification" and "OCCURS Clause" in Chapter 23).

### Runtime Errors

An error that the interpreter detects while the program runs (a subscript or reference modification out of range, a `PERFORM` or `GO TO` to a paragraph that does not exist, an ambiguous name) ends the task. The unit of work is backed out and the message is shown to the operator. Such an error is not an abend: it has no abend code, `HANDLE ABEND` does not receive control (at this level or a caller's), and the operator sees the bricks task-error message, not `DFHAC2206`.

Only these runtime failures of a COBOL program are abends. A division by zero that no `ON SIZE ERROR` covers is abend `ASRA` with the detail `DECIMAL-DIVIDE EXCEPTION`, as the decimal-divide exception (`S0CB`) is in IBM Enterprise COBOL. This applies wherever the division is: in `DIVIDE` or `COMPUTE`, and also in an arithmetic expression of an `IF` or `EVALUATE` condition or a `PERFORM VARYING`. An addressing exception is abend `ASRA`; an internal failure of the interpreter is also reported as `ASRA`. `HANDLE ABEND` can catch each of them, and each takes a transaction dump ("Transaction Dumps" in Chapter 16). A program that runs for 2 seconds without giving control back to the region is abended `AICA` (see "Runaway Tasks" in Chapter 16).

### Arithmetic Statements

```
ADD      operand TO       identifier-1 [GIVING identifier-2] [ROUNDED]
SUBTRACT operand FROM     identifier-1 [GIVING identifier-2] [ROUNDED]
MULTIPLY operand BY       identifier-1 [GIVING identifier-2] [ROUNDED]
DIVIDE   operand INTO     identifier-1 [GIVING identifier-2] [ROUNDED]
DIVIDE   operand BY       identifier-1  GIVING identifier-2  [ROUNDED]
    [ON SIZE ERROR imperative-statement ... END-verb]
```

***Syntax rules***

1. *operand* is one numeric literal, data item or intrinsic function. A list of operands (`ADD A B TO C`) and an arithmetic expression as the operand are not accepted; use `COMPUTE`.
2. There is one receiving item, and at most one `GIVING` item.
3. `DIVIDE ... BY` requires `GIVING`. `REMAINDER` is not supported.
4. When `ON SIZE ERROR` is coded, the matching scope terminator (`END-ADD`, `END-SUBTRACT`, `END-MULTIPLY`, `END-DIVIDE`) is required. `NOT ON SIZE ERROR` is not supported.

***General rules***

1. Without `GIVING` the result replaces *identifier-1*: `ADD A TO B` stores B+A, `SUBTRACT A FROM B` stores B-A, `MULTIPLY A BY B` stores B*A and `DIVIDE A INTO B` stores B/A. With `GIVING` the result is stored in *identifier-2* and *identifier-1* is unchanged. `DIVIDE A BY B GIVING C` stores A/B in C.
2. The result is truncated to the scale of the receiving item, or rounded when `ROUNDED` is coded (see "ROUNDED and ON SIZE ERROR").

## COMPUTE

```
COMPUTE identifier [ROUNDED] = arithmetic-expression
    [ON SIZE ERROR imperative-statement ... END-COMPUTE]
```

An arithmetic expression uses the operators `+`, `-`, `*`, `/` and `**`, unary signs and parentheses, with the usual precedence. Operands are numeric literals, numeric data items and numeric intrinsic functions. There is one receiving item; the symbol `=` is required (`EQUAL` is not accepted). As with the other arithmetic statements, `END-COMPUTE` is required after `ON SIZE ERROR`.

```
COMPUTE WS-TOTAL ROUNDED = (WS-PRICE * WS-QTY) * 1.19
    ON SIZE ERROR MOVE 'TOTAL TOO LARGE' TO WS-MSG
END-COMPUTE.
```

## ROUNDED and ON SIZE ERROR

Arithmetic is carried out in fixed-point decimal of up to 18 digits. Without `ROUNDED` a result is truncated toward zero to the scale of the receiving item; with `ROUNDED` it is rounded half away from zero (`COMPUTE A ROUNDED = 2 / 3` gives 0.7 in a `PIC 9V9` item, and 0.6 without `ROUNDED`).

A size error occurs when the integer part of the result does not fit the receiving item, when an intermediate result exceeds 18 digits, or when a divisor is zero. With `ON SIZE ERROR` coded, the receiving item is left unchanged and the imperative statements run. Without it, high-order digits are lost (adding 1 to a `PIC 99` item holding 99 gives 00), and division by zero abends the task `ASRA` (see "Runtime Errors"). A division by zero inside a condition, which has no `ON SIZE ERROR`, always abends `ASRA`:

```
DIVIDE WS-COUNT INTO WS-TOTAL GIVING WS-AVG
    ON SIZE ERROR MOVE 0 TO WS-AVG
END-DIVIDE
IF WS-TOTAL / WS-COUNT > 10 PERFORM BIG-AVERAGE END-IF
```

With `WS-COUNT` zero, the `DIVIDE` runs its `ON SIZE ERROR` phrase, which sets `WS-AVG` to zero, and the program continues; the `IF` abends the task `ASRA`. Test the divisor first.

### CONTINUE

`CONTINUE` does nothing. It is used where a statement is required but no action is wanted, for example in an empty `WHEN` phrase.

### DISPLAY

```
DISPLAY {identifier | literal} ... [UPON mnemonic-name]
```

`DISPLAY` joins its operands, with no separator, into one line. In a transaction the line has no destination and is discarded: it reaches neither the terminal nor the bricks log. Use `EXEC CICS SEND` or `SEND TEXT` to show data on the terminal, and a transient data queue to keep a trace. A numeric DISPLAY item appears as its stored characters, including the sign byte of a signed item and without a decimal point. `UPON` and its operand are accepted and ignored. `WITH NO ADVANCING` is not supported.

### EVALUATE

```
EVALUATE subject-1 [ALSO subject-2] ...
  {{WHEN object-1 [ALSO object-2] ...} ...
       imperative-statement ...} ...
  [WHEN OTHER imperative-statement ...]
[END-EVALUATE]

subject: identifier | literal | arithmetic-expression | condition
         | TRUE | FALSE
object:  ANY | condition | TRUE | FALSE
         | [NOT] value [{THRU | THROUGH} value]
```

***Syntax rules***

1. Each `WHEN` lists exactly one object per subject, separated by `ALSO`. A mismatch is a compile error (`WHEN has 1 object(s) but EVALUATE has 2 subject(s)`).
2. A subject that is a data item, literal or arithmetic expression takes value objects, ranges and `ANY`. A subject that is `TRUE`, `FALSE` or a condition takes conditions (including condition-names and combined conditions), `TRUE`, `FALSE` and `ANY`.
3. A value object is one value. `WHEN 'A' OR 'B'` is rejected (`WHEN takes one value – write alternatives as separate WHEN lines`); write two `WHEN` phrases that share one body. In a condition object, `OR` is part of the condition and is accepted (`WHEN A = 1 OR B = 2`, `WHEN A = 1 OR 3`).
4. `END-EVALUATE` may be omitted when a period ends the statement.

***General rules***

1. Each subject is evaluated once. The `WHEN` phrases are tested in order, and the statements of the first phrase whose objects all match are run; there is no fall-through. Consecutive `WHEN` phrases share the statements that follow the last of them. `WHEN OTHER` runs when no phrase matched.
2. `v1 THRU v2` matches a subject that is not less than *v1* and not greater than *v2*, compared by the rules of the relation condition.

***Example***

```
EVALUATE TRUE
    WHEN WS-QTY = 0          MOVE 'EMPTY' TO WS-STATE
    WHEN WS-QTY < 10
    WHEN RUSH-ORDER          MOVE 'LOW'   TO WS-STATE
    WHEN OTHER               MOVE 'OK'    TO WS-STATE
END-EVALUATE

EVALUATE WS-ACTION ALSO WS-LEVEL
    WHEN 'A' ALSO 1 THRU 3   PERFORM ADD-BASIC
    WHEN 'A' ALSO ANY        PERFORM ADD-FULL
    WHEN NOT 'Q' ALSO ANY    PERFORM OTHER-ACTION
END-EVALUATE
```

## EXIT

```
EXIT
EXIT PROGRAM
EXIT PERFORM [CYCLE]
EXIT PARAGRAPH
```

- `EXIT` alone does nothing; it is used as the only statement of a paragraph that ends a `PERFORM ... THRU` range.
- `EXIT PROGRAM` is equivalent to `GOBACK`.
- `EXIT PERFORM` ends the innermost in-line `PERFORM` that contains it. `EXIT PERFORM CYCLE` ends only the current iteration; a `VARYING` loop still steps its index. Both must be written inside an in-line `PERFORM`; in a performed paragraph they are a runtime error (`EXIT PERFORM outside an in-line PERFORM`). In a `VARYING ... AFTER` loop, `EXIT PERFORM` leaves every dimension.
- `EXIT PARAGRAPH` ends the current paragraph. Control goes where the end of the paragraph would have taken it: back to the `PERFORM` (or on to its next iteration), on to the next paragraph of a `THRU` range, or on to the next paragraph in sequence. It may be written at any depth, inside `IF`, `EVALUATE` or an in-line `PERFORM`; inside an in-line `PERFORM` it leaves the whole paragraph, not only the loop.
- `EXIT SECTION` is a compile error (`EXIT SECTION needs SECTIONs, not supported in bricks COBOL`).

```
VALIDATE-INPUT.
    IF WS-NAME = SPACES
        MOVE 'NAME REQUIRED' TO WS-MSG
        EXIT PARAGRAPH
    END-IF
    PERFORM CHECK-DUPLICATE.
```

### GO TO

```
GO TO paragraph-name
```

Control passes to the named paragraph and continues from there in source order. A `GO TO` to a paragraph inside the range of an active `PERFORM ... THRU` is an ordinary transfer within the range, and the range still returns when its last paragraph ends. Any other `GO TO` abandons every active `PERFORM`. The target is checked when the statement runs: an unknown paragraph is a runtime error. `GO TO ... DEPENDING ON` is not supported.

### GOBACK and STOP RUN

```
GOBACK
STOP [RUN]
```

`GOBACK` returns control to the program that linked to this one, or ends the task when this is the first program. `STOP RUN` ends the task; `STOP` without `RUN` is accepted. Both run the end-of-task syncpoint. In a CICS program, end with `EXEC CICS RETURN` (see "Ending the Program" in Chapter 25).

### IF

```
IF condition [THEN]
    statement ...
[ELSE
    statement ...]
[END-IF]
```

When `END-IF` is omitted, the period that ends the sentence ends the `IF`. `NEXT SENTENCE` is not supported; use `CONTINUE`. An `ELSE IF` chain needs no nested terminators when it ends with a period:

```
IF A = 1 DISPLAY 'ONE'
ELSE IF A = 2 DISPLAY 'TWO'
ELSE DISPLAY 'OTHER'.
```

### INSPECT

```
INSPECT identifier-1 TALLYING
    {identifier-2 FOR {ALL | LEADING} operand
                    | CHARACTERS}
        [{BEFORE | AFTER} INITIAL operand]} [,] ...

INSPECT identifier-1 REPLACING
    {{ALL | LEADING | FIRST} operand BY operand
     | CHARACTERS BY operand}
        [{BEFORE | AFTER} INITIAL operand] [,] ...

INSPECT identifier-1 CONVERTING operand TO operand
        [{BEFORE | AFTER} INITIAL operand]
```

***General rules***

1. INSPECT examines the whole length of *identifier-1*, trailing spaces included: `INSPECT A TALLYING C FOR ALL ' '` on a `PIC X(10)` item holding `AB` counts 8.
2. `ALL` acts on every occurrence, `LEADING` on consecutive occurrences at the start of the region, `FIRST` (REPLACING only) on the first occurrence, and `CHARACTERS` on every character of the region; `CHARACTERS BY` uses the first character of its operand.
3. `AFTER INITIAL` starts the region after the first occurrence of its operand; `BEFORE INITIAL` ends it at the first occurrence. Both may be coded on one phrase.
4. TALLYING adds to the counter; it does not reset it first.
5. Several phrases may be coded, with or without commas. REPLACING phrases are applied in order, so a later phrase sees the result of an earlier one.
6. CONVERTING translates each character of the region found in the first operand to the character in the same position of the second operand. When a character appears more than once in the first operand, its leftmost position counts. When the operands differ in length, only the overlapping positions are used. Only one CONVERTING phrase may be coded.

```
INSPECT REC REPLACING ALL ',' BY '|' ALL ';' BY '/'.
INSPECT REC TALLYING C FOR LEADING '0', ALL 'X' AFTER INITIAL ' '.
INSPECT VMUSR CONVERTING 'abcdefghijklmnopqrstuvwxyz'
                      TO 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'.
```

### JSON GENERATE

```
JSON GENERATE identifier-1 FROM identifier-2
   [COUNT [IN] identifier-3]
   [NAME [OF] {identifier-4 [IS] {literal | OMITTED}} ...]
   [SUPPRESS {identifier-5 [WHEN fig [OR fig] ...]
             | EVERY [NUMERIC | NONNUMERIC] WHEN fig [OR fig] ...} ...]
   [CONVERTING {identifier-6 TO JSON BOOLEAN USING
                     {condition-name | literal}
              | identifier-6 TO JSON NULL USING fig} ...]
   [[ON] EXCEPTION imperative-statement ...]
   [NOT [ON] EXCEPTION imperative-statement ...]
[END-JSON]
```

*fig* is `ZERO`, `SPACE`, `LOW-VALUE` or `HIGH-VALUE` (singular or plural). The phrases must be coded in the order shown. Entries of `SUPPRESS` and `CONVERTING` may be separated by `ALSO`. The JSON statements work on alphanumeric items, the form used by `EXEC CICS WEB RECEIVE INTO` and `WEB SEND FROM` (see "WEB RECEIVE (server)" in Chapter 20).

***General rules***

1. **Shape.** The text is `{"REC":{...}}`, named after *identifier-2*. `NAME OF rec IS OMITTED` drops the outer name. *identifier-2* may be a group, an elementary item, a table element or a whole table; with `OMITTED`, a table becomes an anonymous array and an elementary item just its value.
2. **Names.** A member is named after its data name, in upper case; `NAME OF item IS 'lit'` replaces it. Use `NAME` to produce lower-case names.
3. **Structure.** Groups become objects and OCCURS items arrays. FILLER items, REDEFINES items (with everything below them) and pointers are left out. A group whose members are all left out disappears, except the outermost item, which is always written, even as `{"REC":{}}`.
4. **Numbers** are written without leading zeros, with a `-` sign and the decimal places given by the picture: `PIC 9V99` holding 1.5 gives `1.50`, and zero gives `0`. Binary and packed items are treated the same. A numeric-edited item is written as a string, trimmed of leading and trailing spaces.
5. **Text** loses its trailing spaces; leading spaces are kept. An item of spaces only is written as `" "`. `"` and `\` are escaped, as are control characters.
6. **SUPPRESS** *item* leaves the item (or group) out. With `WHEN`, or `EVERY ... WHEN`, an elementary item is left out when its value matches: ZERO matches a number equal to zero and text of `0` characters only; the others match DISPLAY items only. Elements of an elementary table are never suppressed one by one.
7. **CONVERTING** ... `TO JSON BOOLEAN` writes `true` when the item satisfies the condition-name (an 88 of that item) or equals the literal, and `false` otherwise. `TO JSON NULL USING fig` writes `null` when the item consists of *fig*.
8. **COUNT IN** receives the number of characters generated; the rest of the receiver is filled with spaces. The receiver may be subscripted or reference-modified; only that part is then written.

1. **Exception.** When the receiver is too small, `JSON-CODE` is set to 1, the receiver holds what fitted, COUNT IN holds the receiver's length and `ON EXCEPTION` runs (without it the program continues). Otherwise `JSON-CODE` is 0 and `NOT ON EXCEPTION` runs.

***Example***

```
01 LST-REC.
   05 LST-KEY.
      10 LST-USER   PIC X(16).
      10 LST-ID     PIC 9(6).
   05 LST-NEXTITEM  PIC 9(8).
   05 LST-COUNT     PIC 9(5).
   05 LST-NAME      PIC X(80).
01 JSON-BUF         PIC X(4000).
01 JS-LEN           PIC 9(5).

    JSON GENERATE JSON-BUF FROM LST-REC COUNT IN JS-LEN
        NAME OF LST-REC IS OMITTED LST-ID IS 'id' LST-NAME IS 'name'
        SUPPRESS LST-USER LST-NEXTITEM
        ON EXCEPTION PERFORM FAIL-JSON
    END-JSON
    EXEC CICS WEB SEND FROM(JSON-BUF) LENGTH(JS-LEN) END-EXEC
```

The text generated is `{"LST-KEY":{"id":12},"LST-COUNT":3,"name":"Buy milk"}`.

## JSON PARSE

```
JSON PARSE identifier-1 INTO identifier-2
   [WITH DETAIL]
   [NAME [OF] {identifier-3 [IS] {literal | OMITTED}} ...]
   [SUPPRESS identifier-4 ...]
   [CONVERTING {identifier-5 FROM JSON BOOLEAN USING
       {condition-name-1 [AND condition-name-2]
        | literal-1 [AND literal-2]}} ...]
   [[ON] EXCEPTION imperative-statement ...]
   [NOT [ON] EXCEPTION imperative-statement ...]
[END-JSON]
```

***General rules***

1. *identifier-1* holds the JSON text and may be reference-modified; trailing spaces and LOW-VALUES are ignored.
2. **Matching.** Without `OMITTED` the text must be `{"IN-REC":{...}}`. With `NAME OF IN-REC IS OMITTED`, the members of the outermost object map onto the subordinate items of IN-REC (for a whole table, the outermost array maps onto its occurrences). Data names match without regard to case; a `NAME` literal replaces the data name and matches with regard to case. FILLER and REDEFINES items never match, and SUPPRESS items are skipped.
3. **Values** are stored by the rules of `MOVE`. A number, or a string holding a number, goes to a numeric or numeric-edited item, aligned on the decimal point (extra decimal places are truncated). Strings and numbers go to alphanumeric items. Objects go to groups and arrays to tables. `true` and `false` need a CONVERTING entry, which stores literal-1 (or the value of condition-name-1) for true and literal-2 (or condition-name-2) for false. `null` leaves the item unchanged. A `\uXXXX` escape is stored as its single byte, or `?` when there is none.
4. Members with no matching item are ignored and items with no member keep their contents. `JSON-STATUS` reports such events as the sum of the codes in the table that follows.
5. An exception stops the statement, sets `JSON-CODE` and runs `ON EXCEPTION`. The whole text is checked before anything is stored, so invalid text (codes 100 to 102) leaves the record unchanged; a later exception (103, 104, 107) leaves the members before it stored.
6. `WITH DETAIL` writes one log line per statement with `JSON-CODE`, `JSON-STATUS` and the reason.

| JSON-STATUS | Meaning |
|---|---|
| 1 | An item had no matching name and was not changed. |
| 2 | A JSON name matched no item. |
| 4 | An item had duplicate names with the same value. |
| 8 | A table had more elements than the JSON array. |
| 16 | A JSON array had more values than the table; the extra values were ignored. |
| 32 | An item was not changed because the value was `null`. |
| 64 | A table element was not changed because the value was `null`. |
| 128 | Size error in a numeric assignment; the value was stored with high-order digits lost. |
| 256 | Text was truncated in an alphanumeric assignment. |

**Figure 24-2. JSON PARSE status codes**

| JSON-CODE | Exception |
|---|---|
| 100 | The JSON text is not valid (including nesting deeper than 64 levels and a top level that is not an object). |
| 101 | The JSON text is empty or contains only white space. |
| 102 | Characters follow the outermost value. |
| 103 | Duplicate names with different values; the leftmost was kept. |
| 104 | A value does not fit its item (an object into an elementary item, text into a numeric item, ...). |
| 106 | No JSON name matched any item. In the `{"REC":{...}}` form the outer name counts as a match, so `{"REC":{"foo":1}}` gives code 0 and JSON-STATUS 3. |
| 107 | `true` or `false` for an item without CONVERTING. |
| 108 | A top-level array without `NAME ... OMITTED` on a table. |

**Figure 24-3. JSON PARSE exception codes**

```
01 IN-REC.
   05 IN-TITLE PIC X(80).
   05 IN-PRIO  PIC X.
   05 IN-DONE  PIC X.
*> body: {"title":"Buy milk","priority":"H","done":true}
    JSON PARSE WS-BODY INTO IN-REC
        NAME OF IN-REC IS OMITTED IN-TITLE IS 'title'
             IN-PRIO IS 'priority' IN-DONE IS 'done'
        CONVERTING IN-DONE FROM JSON BOOLEAN USING 'Y' AND 'N'
        ON EXCEPTION PERFORM BAD-REQUEST
    END-JSON
```

***Differences from IBM Enterprise COBOL***

Generated names are upper case. The single condition-name form `USING condition-name-1` cannot store false, because bricks has no `88 ... WHEN SET TO FALSE`; JSON `false` then gives code 104, so code both condition-names. Invalid text leaves the record unchanged, where IBM may already have changed part of it. The phrases `ENCODING`, `IGNORING`, `INDICATING` and `CONVERTING ... FROM JSON NULL`, and tables with `OCCURS DEPENDING ON`, are not supported.

## MOVE

```
MOVE {identifier-1 | literal | function} TO identifier-2 ...
```

The value is stored in each receiving item in turn. Receivers are separated by spaces (not commas). The result depends on the categories of the two items, as the following table shows.

| Sending | Receiving | Result |
|---|---|---|
| Alphanumeric | Alphanumeric | Copied from the left; padded with spaces or truncated on the right. |
| Numeric | Numeric | Aligned on the decimal point. Excess decimal places are truncated; excess high-order digits are lost. The sign is dropped for an unsigned receiver. |
| Numeric | Numeric-edited | Edited by the receiving picture, at the scale of the sending item. |
| Numeric-edited | Numeric | De-edited: insertion characters are removed and the digits are read as a number. |
| Alphanumeric | Numeric | The digits are read as a number after blanks are removed. A letter is a data exception: the receiver is set to zero and the event is logged. Other symbols are dropped. |
| Numeric | Alphanumeric | The stored digits are copied as characters. |
| Any | Group | The value is stored in each elementary item of the group in turn. A figurative constant fills the whole group. |

**Figure 24-4. MOVE rules**

A numeric literal follows the numeric-to-numeric rule: `MOVE 2` to a `PIC S9(4)V99` item stores 2.00 and `MOVE 1.5` to `PIC S9(4)` stores 1. An alphanumeric literal follows the alphanumeric rule: `MOVE '150'` lays those digits into the item's bytes. A numeric `VALUE` clause and `COMPUTE` treat a literal in the same way as `MOVE`. `MOVE CORRESPONDING` and `MOVE ALL literal` are not supported.

### PERFORM

```
Out-of-line:
PERFORM paragraph-1 [{THRU | THROUGH} paragraph-2] [loop-phrase]

In-line:
PERFORM [loop-phrase] imperative-statement ... END-PERFORM

loop-phrase:
  integer-or-identifier TIMES
| [WITH TEST {BEFORE | AFTER}] UNTIL condition
| [WITH TEST {BEFORE | AFTER}]
       VARYING identifier FROM operand BY operand UNTIL condition
      [AFTER identifier FROM operand BY operand UNTIL condition] ...
```

***General rules***

1. Without a loop phrase the paragraph, range or in-line body runs once. `THRU` runs every paragraph from *paragraph-1* to *paragraph-2* in source order; a range that runs backwards is a runtime error.
2. `n TIMES` runs the body *n* times; *n* is a literal or a numeric item.
3. `UNTIL` tests the condition before each execution (`WITH TEST BEFORE`, the default), so a condition that is already true runs the body zero times. `WITH TEST AFTER` tests after each execution, so the body runs at least once. `WITH` may be omitted. `WITH TEST` is accepted and has no effect on `TIMES` and on a `PERFORM` with no loop phrase.
4. `VARYING` sets the index to the `FROM` value, runs the body while the condition is false and adds the `BY` value after each execution. With `TEST AFTER` the order is body, test, step, so the index is left at its last value within the range.
5. Each `AFTER` adds an inner dimension, whose index is reset to its `FROM` value on every iteration of the dimension outside it.
6. In-line and out-of-line PERFORM statements may be nested in any combination, to a depth of 1024 per task.

***Examples***

```
PERFORM FILL-ROW 15 TIMES.
PERFORM FILL-ROW VARYING I FROM 1 BY 1 UNTIL I > 15.

PERFORM WITH TEST AFTER UNTIL WS-RESP = DFHRESP(NORMAL)
    EXEC CICS READ FILE('CUSTOMER') INTO(WS-REC)
              RIDFLD(WS-KEY) RESP(WS-RESP) END-EXEC
END-PERFORM.

PERFORM VARYING I FROM 1 BY 1 UNTIL I > 100
    IF TBL-KEY(I) = WS-WANTED
        MOVE I TO WS-HIT
        EXIT PERFORM
    END-IF
    IF TBL-KEY(I) = SPACES
        EXIT PERFORM CYCLE
    END-IF
    ADD 1 TO WS-SCANNED
END-PERFORM.
```

A two-dimensional walk addresses a one-dimensional table with a computed subscript, because nested OCCURS is not supported. The following loop runs its body 12 times:

```
PERFORM VARYING I FROM 1 BY 1 UNTIL I > 4
          AFTER J FROM 1 BY 1 UNTIL J > 3
    COMPUTE SLOT((I - 1) * 3 + J) = I * J
END-PERFORM.
```

`PERFORM ... THRU` with an exit paragraph gives an early return:

```
    PERFORM CHECK-IT THRU CHECK-EXIT.
    ...
CHECK-IT.
    IF WS-AMT = 0
        GO TO CHECK-EXIT
    END-IF.
    COMPUTE WS-FEE = WS-AMT * 2.
CHECK-EXIT.
    EXIT.
```

### SET (Pointers)

```
SET identifier-1 TO {ADDRESS OF identifier-2 | NULL | identifier-3}
SET ADDRESS OF identifier-4 TO {identifier-3 | ADDRESS OF identifier-2
                               | NULL}
SET identifier-1 {UP | DOWN} BY arithmetic-expression
```

*identifier-1* and *identifier-3* are `USAGE POINTER` items; *identifier-4* is a based LINKAGE 01 (or `DFHCOMMAREA` or `DFHEIB`). `SET` supports only these pointer forms: `SET condition-name TO TRUE` and `SET` for indexes are compile errors.

A pointer is an opaque 4-byte handle, not a machine address; it is null when declared. Its value comes from `EXEC CICS GETMAIN SET(ptr)`, from `EXEC CICS ADDRESS`, from `ADDRESS OF` an item or from another pointer. A pointer may be moved, compared for equality with another pointer, with `ADDRESS OF` an item or with `NULL` (synonym `NULLS`). `ADDRESS OF` the same item always compares equal.

`SET ADDRESS OF L TO ptr` points the based record `L` at the addressed storage; every reference to an item of `L` then reads and writes those bytes. `SET ADDRESS OF L TO NULL` removes the address. A reference through a null based record, or through a pointer whose GETMAIN storage has been released by `FREEMAIN`, ends the task with a null pointer error.

```
WORKING-STORAGE SECTION.
01 WS-PTR  USAGE POINTER.
LINKAGE SECTION.
01 GM-AREA BASED.
   05 GM-TEXT PIC X(16).
PROCEDURE DIVISION.
MAIN.
    EXEC CICS GETMAIN SET(WS-PTR) FLENGTH(16) END-EXEC.
    SET ADDRESS OF GM-AREA TO WS-PTR.
    MOVE 'POINTER-OK' TO GM-TEXT.
```

`SET ptr UP BY n` moves a pointer *n* bytes forward within the storage it addresses; `DOWN BY` moves it back. *n* must evaluate to a non-negative whole number. A pointer may be moved past the end of its storage, but a reference through it there ends the task; moving it before the start, or moving a null pointer, is an error at the `SET`. A pointer cannot be moved from one area into another.

```
01 WS-REC PIC X(10) VALUE 'AAAAAAAAAA'.
01 P      USAGE POINTER.
LINKAGE SECTION.
01 L BASED.
   05 L-FLD PIC X(6).
    SET P TO ADDRESS OF WS-REC.
    SET P UP BY 4.
    SET ADDRESS OF L TO P.
    MOVE 'XXXXXX' TO L-FLD.        *> WS-REC is now AAAAXXXXXX
```

The sample `GMPT` (`runtime/cobol/gmpt.cob`) is a complete example. `GETMAIN`, `FREEMAIN` and `ADDRESS` are described in "GETMAIN and FREEMAIN" in Chapter 13.

### STRING

```
STRING {{identifier-1 | literal-1} ...
          DELIMITED [BY] {SIZE | identifier-2 | literal-2}} ...
    INTO identifier-3
    [[WITH] POINTER identifier-4]
    [[ON] OVERFLOW imperative-statement ...]
    [NOT [ON] OVERFLOW imperative-statement ...]
[END-STRING]
```

***General rules***

1. `DELIMITED BY SIZE` transfers the whole sending item: a data item at its full length, trailing spaces included, or a literal as written. `DELIMITED BY x` stops at the first *x*. To drop the padding of a data item, use `DELIMITED BY SPACE`.
2. Without `POINTER`, the receiver is filled with spaces before the data is stored.
3. With `POINTER`, storing begins at the position in *identifier-4*, which is increased by one for each character stored; the rest of the receiver is unchanged.
4. Overflow occurs when the data does not fit, or when the pointer is less than 1 or beyond the receiver at the start (nothing is then stored). `ON OVERFLOW` runs on overflow and `NOT ON OVERFLOW` otherwise.
5. The receiver may not be reference-modified (`STRING INTO receiver must not be reference-modified`).

```
01 WS-NAME  PIC X(10) VALUE 'JOE SMITH'.
01 WS-LINE  PIC X(12) VALUE '............'.
01 WS-PTR   PIC 99.
    MOVE 1 TO WS-PTR
    STRING 'Name: ' DELIMITED BY SIZE
           WS-NAME  DELIMITED BY SPACE
        INTO WS-LINE WITH POINTER WS-PTR
        ON OVERFLOW MOVE 'Y' TO WS-TRUNC
    END-STRING
```

`WS-LINE` becomes `Name: JOE...` and `WS-PTR` is 10, the next free position.

***Difference from IBM Enterprise COBOL***

Without `POINTER`, IBM leaves the positions that receive no data unchanged; bricks fills them with spaces.

### UNSTRING

```
UNSTRING identifier-1
    [DELIMITED [BY] [ALL] operand [OR [ALL] operand] ...]
    INTO {identifier-2 [DELIMITER [IN] identifier-3]
                       [COUNT [IN] identifier-4]} ...
    [[WITH] POINTER identifier-5]
    [TALLYING [IN] identifier-6]
    [[ON] OVERFLOW imperative-statement ...]
    [NOT [ON] OVERFLOW imperative-statement ...]
[END-UNSTRING]
```

***General rules***

1. *identifier-1* must be a data item; a literal, figurative constant or function is rejected (`UNSTRING source must be a data item`). It is examined from the pointer position (default 1) over its whole length.
2. Each receiver gets the characters up to the next delimiter. When several delimiters are coded, the first found wins, and on a tie the first coded. `ALL` treats a run of the same delimiter as one. Two adjacent delimiters (without `ALL`) give an empty field: spaces, or zero for a numeric receiver.
3. `DELIMITER IN` receives the delimiter that ended the field (spaces at the end of the data); `COUNT IN` receives the length of the field. Both require `DELIMITED BY`.
4. Without `DELIMITED BY`, each receiver takes as many characters as its own length.
5. `TALLYING IN` is increased by the number of receivers filled. The pointer ends one position past the last character examined. Receivers that are not reached keep their contents.
6. Overflow occurs when the pointer is less than 1 or beyond the sending item at the start, or when characters remain after every receiver is filled.

```
01 WS-REC    PIC X(20) VALUE 'AB   CD,EF'.
    UNSTRING WS-REC DELIMITED BY ALL SPACES OR ','
        INTO WS-F1 DELIMITER IN WS-D1 COUNT IN WS-N1
             WS-F2
        WITH POINTER WS-PTR TALLYING IN WS-FIELDS
        ON OVERFLOW DISPLAY 'MORE DATA'
    END-UNSTRING
```

With `PIC X(5)` receivers and the pointer at 1, WS-F1 is `AB`, WS-D1 a space, WS-N1 2, WS-F2 `CD`, the pointer 9 and the tally 2. `EF` and the padding are left over, so `ON OVERFLOW` runs.

## Conditions

Conditions are used in `IF`, `EVALUATE` and `PERFORM UNTIL`. A simple condition is a relation condition, a class condition, a sign condition or a condition-name. Simple conditions combine with `NOT`, `AND` and `OR`, in that order of precedence, and may be grouped with parentheses. `NOT` must be followed by a condition (`NOT NOT A = 1` is rejected).

### *Relation conditions*

| Relation | Symbol | Words |
|---|---|---|
| Equal | = | [IS] EQUAL [TO] |
| Not equal | <> NOT = | [IS] NOT EQUAL [TO] |
| Greater | > | [IS] GREATER [THAN] |
| Not greater | NOT > | [IS] NOT GREATER [THAN] |
| Less | < | [IS] LESS [THAN] |
| Not less | NOT < | [IS] NOT LESS [THAN] |
| Greater or equal | >= | [IS] GREATER [THAN] OR EQUAL [TO] |
| Less or equal | <= | [IS] LESS [THAN] OR EQUAL [TO] |

**Figure 24-5. Relational operators**

Numeric operands are compared by value. Other operands are compared byte by byte, the shorter padded with spaces. A reference-modified item is always alphanumeric (see "Reference Modification").

When a relation contains a division, the quotient keeps the larger of (*d2* - *d1*) and *dmax* decimal places and is truncated, by IBM's rule for intermediate results: *d1* and *d2* are the decimal places of the divisor and the dividend, and *dmax* the most decimal places of any operand in the relation, divisors and exponents excluded. Thus `IF 5 / 2 > 2` is false, as on z/OS, while `IF B / 10 = 4.5` is true for B equal to 45 because the literal 4.5 makes *dmax* 1. The subject and each object of `EVALUATE` are evaluated separately, so `EVALUATE B / 10 WHEN 4.5` does not match for B equal to 45. `COMPUTE` is not affected.

### *Abbreviated combined relation conditions*

After a relation, later relations in the same condition may omit the subject, or the subject and the operator; the last stated subject and operator are used, by the IBM rules. A `NOT` immediately before a relational operator belongs to the operator; any other `NOT` is the logical NOT.

| Written | Means |
|---|---|
| `A = 1 OR 2 OR 3` | `A = 1 OR A = 2 OR A = 3` |
| `A > 1 AND < 9` | `A > 1 AND A < 9` |
| `A NOT = 1 AND 2` | `A NOT = 1 AND A NOT = 2` |
| `A > 5 AND NOT 9` | `A > 5 AND NOT (A > 9)` |
| `NOT A = B OR C` | `NOT (A = B) OR A = C` |

**Figure 24-6. Abbreviated combined relation conditions**

A condition-name after `AND` or `OR` is a condition, not an omitted object. An abbreviated relation may not open a parenthesized group (`A = B AND (< C OR D)` is rejected), but a group that begins with a full relation may use abbreviations inside it: `IF B = 0 AND (A = 4 OR 5 OR 6)`.

### *Class conditions*

```
identifier IS [NOT] {NUMERIC | ALPHABETIC | ALPHABETIC-UPPER
                     | ALPHABETIC-LOWER}
```

A class condition tests the current contents of the item, not its picture. `NUMERIC` is true when every byte is a digit; for a signed DISPLAY item the sign byte may be a space, `+` or `-`. `ALPHABETIC` accepts the ASCII letters and the space; `ALPHABETIC-UPPER` and `ALPHABETIC-LOWER` accept one case of letter and the space.

### *Sign conditions*

```
arithmetic-expression IS [NOT] {POSITIVE | NEGATIVE | ZERO}
```

The operand must be numeric; a sign condition on an alphanumeric item is a runtime error. Zero is neither positive nor negative, and an unsigned item is never negative.

### *Condition-name conditions*

A level-88 condition-name used as a condition is true when its data item holds one of the condition's values (see "Condition-Names (Level 88)" in Chapter 23):

```
    EXEC CICS READ FILE('CUSTOMER') INTO(REC) RIDFLD(KEY)
              RESP(RESP-CODE) END-EXEC.
    IF RESP-OK
       CONTINUE
    ELSE IF RESP-RECOVER
       PERFORM RETRY
    ELSE IF RESP-FATAL
       PERFORM GIVE-UP.
```

### Reference Modification

```
identifier (start : [length])
```

Reference modification refers to *length* bytes of a data item beginning at byte *start*, counting from 1. Both operands may be arithmetic expressions. When *length* is omitted the reference extends to the end of the item. It may be used for a sending or a receiving item:

```
01 WS-LINE  PIC X(80) VALUE 'HELLO WORLD'.
01 WS-PART  PIC X(5).
01 WS-POS   PIC 9(2)  VALUE 7.
    MOVE WS-LINE(1:5)      TO WS-PART        *> 'HELLO'
    MOVE WS-LINE(WS-POS:5) TO WS-PART        *> 'WORLD'
    MOVE '*'               TO WS-LINE(6:1)   *> 'HELLO*WORLD'
```

- A store changes only the bytes referred to.
- A reference-modified item is alphanumeric whatever its picture: `IF NUM-FLD(1:1) = '1'` is the correct test, and a value stored in it is left-justified and padded with spaces.
- A `COMP` or `COMP-3` item is reference- modified through its decimal digits, not its bytes: `PIC 9(4) COMP VALUE 1234` with `(1:2)` gives `12`.
- A start below 1 or beyond the item, or a length that runs past the end, is a runtime error on reads as well as stores.
- Reference modification may not be combined with a subscript (`TBL(I)(1:2)`) or written on a qualifier (`K(1:2) OF ROW`); both are compile errors.

### Intrinsic Functions

```
FUNCTION function-name (argument [, argument])
```

The intrinsic functions in the following table are supported. Any other function name is a runtime error (`unsupported FUNCTION`).

| Function | Arguments | Result |
|---|---|---|
| UPPER-CASE(s) | 1 alphanumeric | *s* in upper case: a-z and the Latin-1 letters. |
| LOWER-CASE(s) | 1 alphanumeric | *s* in lower case. |
| TRIM(s) | 1 alphanumeric | *s* with leading and trailing blanks removed. The LEADING and TRAILING forms are not supported. |
| REVERSE(s) | 1 alphanumeric | *s* in reverse order. |
| LENGTH(s) | 1 any | The length of the argument in characters: 8 for a PIC X(8) item. |
| BYTE-LENGTH(s) | 1 any | The same as LENGTH. |
| ORD(c) | 1 character | The ordinal position of *c*: its byte value plus 1 (ORD('A') is 66). The argument must be one character long. |
| CHAR(n) | 1 integer, 1 to 256 | The character whose ordinal position is *n*. |
| NUMVAL(s) | 1 alphanumeric | The numeric value of *s* after leading and trailing blanks are removed. Non-numeric content is a runtime error, not zero. |
| POS(needle, hay) | 2 alphanumeric | The position (from 1) of the first occurrence of *needle* in *hay*, or 0. An empty needle gives 0. This function is a bricks extension, matching the REXX POS function. |

**Figure 24-7. Intrinsic functions**

LENGTH, BYTE-LENGTH, NUMVAL, POS and ORD are numeric and may be used in arithmetic and in numeric comparisons (IF FUNCTION POS(NEEDLE, HAY) > 0). The other functions return alphanumeric values. The sample GUSL (runtime/cobol/gusl.cob) uses POS to filter the customers file.

### Differences from IBM Enterprise COBOL

- Paragraphs only; no sections, `EXIT SECTION` or `GO TO ... DEPENDING ON`.
- A `GO TO` out of a performed paragraph abandons all active PERFORM statements.
- The arithmetic statements take one operand and one receiver, `END-verb` is required with `ON SIZE ERROR`, and `NOT ON SIZE ERROR` and `REMAINDER` are not supported.
- `DISPLAY` output is discarded.
- `STRING` without `POINTER` clears the receiver first.
- `SET` supports pointers only.
- Ten intrinsic functions are supported, one of them (`POS`) a bricks extension.

Chapter 25

# The EIB and EXEC CICS in COBOL

This chapter describes how a COBOL program issues CICS commands, how its data items are passed to them, what happens when a command raises a condition, and the items that bricks declares for every program: the EXEC interface block, `DFHCOMMAREA`, `RC`, the SQLCA and the JSON special registers. The commands themselves are described in Part 3; their behavior is the same in COBOL, REXX and PL/I.

## The EXEC CICS Statement

```
EXEC CICS command [option[(argument)]] ... END-EXEC
```

The compiler collects the words between `EXEC CICS` and `END-EXEC` and passes the command, unchanged, to the same command parser that REXX and PL/I programs use. The statement may extend over any number of lines, and a period after `END-EXEC` is optional inside a sentence. Within the parentheses of an option:

- A data name refers to a data item. For an input option the program's current value is passed; for an output option (`INTO`, `SET`, `RESP`, `LENGTH` on a read, ...) the result is stored in the item.
- A literal (`'CUSTOMER'`, `80`) is passed as written.
- `DFHRESP(condition)` is replaced by the condition's numeric value when the program is compiled. It may also be used outside `EXEC CICS`, in any expression: `IF WS-RESP = DFHRESP(NOTFND)`. An unknown name is a compile error (`unknown DFHRESP code ... (see DFHRESP.cpy for the list)`). The values are the same in every language.

### *Data areas*

A data area named in `FROM`, `INTO` or `COMMAREA` is passed with its full declared length, trailing spaces included, unless `LENGTH` is coded. A binary or packed item is passed as its stored bytes, not as decimal text. An unsubscripted table is passed whole.

### *Maps*

`SEND MAP ... FROM(group)` and `RECEIVE MAP ... INTO(group)` match the fields of the map with the subordinate items of the group by name, so the group must contain an item spelled exactly as each map field it is to send or receive. A `REDEFINES` view of those items shares their bytes and may be used to fill them. The color of a field may be overridden at runtime with an item named *field*`-C` in the same group, holding one of the color names of the `DFHCOLOR` copybook:

```
       COPY DFHCOLOR.
       01 SCR.
          05 STATUS    PIC X(10).
          05 STATUS-C  PIC X(9) VALUE SPACES.
           IF BALANCE < 0
               MOVE DFHRED   TO STATUS-C
           ELSE
               MOVE DFHGREEN TO STATUS-C
           END-IF.
           EXEC CICS SEND MAP('BALC') FROM(SCR) ERASE END-EXEC.
```

The utility `bricksconvert` can produce such a group from a BMS map source: `bricksconvert -o out.map -copy out.cpy source.bms` writes the bricks map and a copybook whose 01 groups mirror the named fields of each map. Maps are described in Chapter 3.

### *SEND TEXT*

`SEND TEXT FROM(area)` treats *area* as a flat screen image: each group of *cols* bytes (80 on a Model 2) is one row. A 3270 has no line-feed character, so build the area as a group of `PIC X(80)` items, one per row, and fill the rows individually.

### *Terminal input at task start*

When an operator types a transaction identifier followed by data at the blank screen, the first program of the task can read the whole line, transaction identifier included, with `RECEIVE INTO`; a second `RECEIVE` in the same task returns `EOC`. The sample `EXAM` (`runtime/cobol/exam.cob`):

```
           EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC.

           IF EIBRESP = RESP-EOC THEN
               MOVE '(no input -- chained dispatch)' TO WS-INPUT
           END-IF.

           MOVE WS-INPUT TO RAW.
           MOVE WS-LEN   TO BUFLEN.

           UNSTRING WS-INPUT DELIMITED BY ' '
               INTO TRANSID A B C
           END-UNSTRING.
```

`EOC` is one of the conditions whose default action is to continue (see the next section), so this program may test `EIBRESP` without coding `RESP`.

### Condition Handling

A command that does not complete normally raises a condition (`NOTFND`, `MAPFAIL`, `INVREQ`, ...). A COBOL program handles conditions as on IBM CICS:

- **RESP and NOHANDLE.** When the command codes `RESP(data-name)` or `NOHANDLE`, a condition never ends the task. The response is stored in `EIBRESP` and in the RESP item (and the secondary response in `EIBRESP2` and any `RESP2` item), any `HANDLE CONDITION` or `HANDLE AID` is bypassed for that command, and the program continues with the next statement.
- **HANDLE CONDITION.** Otherwise, when a `HANDLE CONDITION` is active for the condition (or for `ERROR`), control passes to the named paragraph, as by `GO TO`. `IGNORE CONDITION` makes the command return normally with `EIBRESP` set to 0 (`NORMAL`).
- **Unhandled conditions.** Otherwise the condition is unhandled, and the task **abends** with the condition's IBM abend code. No statement after the command runs.

The abend is an ordinary abend: `HANDLE ABEND` at this `LINK` level or a higher one can catch it, and otherwise the unit of work is backed out and the operator receives message `DFHAC2206`. A few conditions have no abend code and continue by default. The rules are given in "Unhandled Conditions" in Chapter 16; the response values are listed in "Responses" in Chapter 18, and the abend code of each condition in "Condition Abend Codes" in Appendix C.

**Note:**

A command whose response the program tests must code `RESP`. Testing `EIBRESP` after a command that codes neither `RESP` nor `NOHANDLE` works only for the normal response and for the continue-by-default conditions, because any other condition abends the task before the test runs. The supplied samples declare `01 WS-CICS-RESP PIC S9(8) COMP` and code `RESP(WS-CICS-RESP)` on every command whose `EIBRESP` they test.

The following excerpt from the sample `ORDR` (`runtime/cobol/ordr.cob`) reads a transient data queue until it is empty:

```
           EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC)
                     RESP(WS-CICS-RESP) END-EXEC.
           IF EIBRESP = RESP-QZERO THEN
               MOVE 'Y' TO DONE-FLAG
           END-IF.
           IF EIBRESP = RESP-NORMAL THEN
               COMPUTE N-READ = N-READ + 1
```

The RESP item may also be tested directly:

```
       01 WS-RESP PIC S9(8) COMP.
           EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY)
                     RESP(WS-RESP) END-EXEC
           EVALUATE WS-RESP
              WHEN DFHRESP(NORMAL) CONTINUE
              WHEN DFHRESP(NOTFND) MOVE 'Customer not found' TO MSG
              WHEN OTHER           MOVE 'I/O error' TO MSG
           END-EVALUATE.
```

***Catching an abend***

The sample `ABNH` (`runtime/cobol/abnh.cob`) lets a `NOTFND` go unhandled and catches the resulting `AEIM` abend with `HANDLE ABEND LABEL`:

```
       PROCEDURE DIVISION.
       MAIN.
           EXEC CICS HANDLE ABEND LABEL(RECOVER) END-EXEC.
           EXEC CICS READ FILE('customers') INTO(REC)
                     RIDFLD('NOSUCHKEY') END-EXEC.
           EXEC CICS RETURN END-EXEC.
           GOBACK.

       RECOVER.
           MOVE EIBRESP TO SCR-RESP.
           EXEC CICS ASSIGN ABCODE(WS-CODE) END-EXEC.
           MOVE WS-CODE TO SCR-CODE.
           EXEC CICS SEND TEXT FROM(SCR) ERASE END-EXEC.
           EXEC CICS RETURN END-EXEC.
           GOBACK.
```

The screen shows `CAUGHT AEIM EIBRESP=0013`, as in Figure 16-7. `EIBRESP` still holds the condition when the exit receives control, but every command, `ASSIGN` included, sets it again, so the program saves it first. The COBOL EIB has no `EIBABCODE` field; the program obtains the abend code with `ASSIGN ABCODE`. A COBOL program has no equivalent of the REXX `SIGNAL ON ERROR`; it uses `RESP`, `HANDLE CONDITION` and `HANDLE ABEND`.

### The EXEC Interface Block

Every COBOL program has the EXEC interface block as the based record `DFHEIB`, laid out as the IBM `DFHEIBLK`. Its fields are referred to by their IBM names without qualification, and `EXEC CICS ADDRESS EIB(ptr)` returns its address.

| Field | Picture | Contents |
|---|---|---|
| EIBTIME | S9(7) COMP-3 | Time the program started, refreshed by ASKTIME. |
| EIBDATE | S9(7) COMP-3 | Date the program started, refreshed by ASKTIME. |
| EIBTRNID | X(4) | Transaction identifier of the task. |
| EIBTASKN | S9(7) COMP-3 | Task number. |
| EIBTRMID | X(4) | The first four characters of the terminal identifier (T000 for T0001); ASSIGN TERMID into an 8-byte item gives the full identifier. |
| EIBCPOSN | S9(4) COMP | Cursor position at the last input, zero-based. |
| EIBCALEN | S9(4) COMP | Length of the COMMAREA received; 0 when none. |
| EIBAID | X | Attention identifier of the last input. |
| EIBFN | X(2) | Function code of the last command. |
| EIBRCODE | X(6) | Response code bytes. |
| EIBDS | X(8) | Last file name used. |
| EIBREQID | X(8) | Request identifier of the last interval control command. |
| EIBRSRCE | X(8) | Last resource used. |
| EIBRESP | S9(8) COMP | Response of the last command. |
| EIBRESP2 | S9(8) COMP | Secondary response of the last command. |

**Figure 25-1. Fields of DFHEIB**

DFHEIB also contains the IBM single-byte fields EIBSYNC, EIBFREE, EIBRECV, EIBSEND, EIBATT, EIBEOC, EIBFMH, EIBCOMPL, EIBSIG, EIBCONF, EIBERR, EIBSYNRB, EIBNODAT and EIBRLDBK, the four-byte EIBERRCD and two filler bytes, so that the offsets of all fields match IBM. bricks does not set these fields. Field meanings are described in "EIB Fields" in Chapter 17.

**Note:**

If a program declares DFHEIB or any one of the EIB field names itself, bricks does not supply the EIB at all, and the program's own declaration is used. Do not use EIB field names for other purposes.

### Automatically Declared Items

The items in the following table are declared for every program that does not declare them itself. They need no `COPY` statement.

| Item | Picture | Contents |
|---|---|---|
| `DFHEIB` | group | The EXEC interface block (see above). |
| `DFHCOMMAREA` | `X(2000)` | The COMMAREA (see "The COMMAREA"). |
| `RC` | `9(8)` | The response of the last command, as `EIBRESP`, for programs written in the REXX style. |
| `SQLCODE` | `S9(8)` | SQL return code: 0, +100 or a negative error. |
| `SQLSTATE` | `X(5)` | SQL state. |
| `SQLERRMC` | `X(70)` | Short error message. |
| `SQLERRP` | `X(8)` | Product tag. |
| `SQLERRD1 – SQLERRD6` | `S9(9)` | Diagnostic counts; `SQLERRD3` holds the number of rows affected. |
| `SQLWARN0 – SQLWARN9, SQLWARNA` | `X` | SQL warning flags. |
| `JSON-CODE` | `S9(9) BINARY` | Exception code of the last JSON statement. |
| `JSON-STATUS` | `9(9) BINARY` | Status of the last `JSON PARSE`. |

**Figure 25-2. Automatically declared items**

`EXEC SQL` and the SQLCA are described in "The SQL Communication Area" in Chapter 21; the named SQLCODE and SQLSTATE constants in "SQLCA" in Chapter 26. In `EXEC SQL` a COBOL data item is written as a host variable with a leading colon (`:CUSTID`).

### The COMMAREA

`DFHCOMMAREA` holds the COMMAREA passed to the program by `LINK`, `XCTL` or the previous task's `RETURN TRANSID ... COMMAREA`; `EIBCALEN` gives its length. A program may declare `01 DFHCOMMAREA` in the LINKAGE SECTION with its own layout. Otherwise bricks supplies `DFHCOMMAREA PIC X(2000)`, and the following rules apply:

- The supplied area passes on only the EIBCALEN bytes that were received, so RETURN COMMAREA(DFHCOMMAREA) gives the next task the same length. A program that must pass on more than it received declares its own area (a WORKING-STORAGE record, or a LINKAGE 01 DFHCOMMAREA of the full size).
- On a first entry (EIBCALEN 0), whatever was moved to the supplied area is passed with trailing spaces removed.

On LINK, as on z/OS, the called program cannot change the length of the COMMAREA: the caller always receives back exactly the number of bytes it passed (or the LENGTH it coded), with the called program's view of DFHCOMMAREA laid over them. Trailing spaces and X'00' bytes are kept, so a binary field at the end of the area survives the round trip. A COBOL group and a PL/I structure with the same byte layout exchange packed and binary fields correctly. The complete rules are given under LINK, XCTL and RETURN in Chapter 12.

### Ending the Program

EXEC CICS RETURN is the normal way to end a CICS program, and the only one that passes a TRANSID and COMMAREA to the next task of a pseudo-conversation. A program that ends by STOP RUN, GOBACK, EXIT PROGRAM or by running off the end of the PROCEDURE DIVISION also ends cleanly, and bricks takes the implicit end-of-task syncpoint for these paths too, so EXEC SQL changes are committed. In a linked program, GOBACK returns to the caller. Code GOBACK after the RETURN as the statement that is never reached; do not use STOP RUN in CICS programs.

```
           EXEC CICS RETURN TRANSID('GUST') COMMAREA(WS-STATE)
                     END-EXEC.
           GOBACK.
```

### Differences from IBM CICS

- No translator step is needed: `EXEC CICS` is interpreted directly, and `DFHEIBLK` and `DFHCOMMAREA` are supplied without `PROCEDURE DIVISION USING`.
- Symbolic map copybooks are not generated by the map compiler; write the group by hand or use `bricksconvert -copy`.
- `EIBTRMID` holds the first four characters of a five-character bricks terminal identifier.
- `DISPLAY` output is discarded (see "DISPLAY" in Chapter 24).

Chapter 26

# COBOL Copybooks

A copybook is a member of COBOL source, usually data descriptions, kept in a file of its own and included in a program by the `COPY` statement. bricks expands every `COPY` statement in the source text before the program is compiled, so the compiler sees one continuous program. This chapter describes the `COPY` statement and the copybooks supplied with bricks.

## COPY Statement

```
COPY text-name.
```

***Syntax rules***

1. The statement must be alone on its line. Leading blanks and a floating comment after the period are allowed (`COPY DFHAID. *> 3270 AID bytes`); any other text on the line is an error.
2. The period is required.
3. *text-name* is the name of a copybook file without its extension. It may contain only letters, digits, `-`, `_` and `.`; a name containing a path separator or `..`, or beginning with a period, is rejected before any file is opened.
4. The word `COPY` and *text-name* are not case-sensitive.
5. `REPLACING`, `OF` or `IN` *library-name*, and `SUPPRESS` are not supported.

***General rules***

1. bricks looks for *text-name*`.cpy`, then *text-name*`.cbl`, then *text-name* itself, in the copybook directory, without regard to case.
2. The copybook directory is set by `copybook_dir=` in `bricks.cnf` (see "The bricks.cnf File" in Appendix D). The default is `runtime/cobolcopy/` under the runtime directory, where the supplied copybooks are installed.
3. A copybook may itself contain `COPY` statements, to a depth of 5. A copybook that copies itself, directly or indirectly, is reported as an error.
4. The program, with every copybook expanded, may not exceed 4 MiB.
5. A compile error inside a copybook names the copybook and the line number within it.
6. When a copybook changes, every program that copies it is compiled again at its next use; there is no need to touch the programs or to clear the program cache. (A copybook that contains only comments is not tracked.)

***Example***

```
       WORKING-STORAGE SECTION.
       COPY DFHAID.
       COPY DFHRESP.
       01 OWN-COUNTER PIC 9(4).
```

### The Dual-Name Convention

Every constant in the supplied copybooks is declared twice, under a short bricks name and under the traditional IBM name, with the same value: `PF03` and `DFHPF3` are both `X'F3'`, and `RESP-NOTFND` and `DFHRESP-NOTFND` are both 13. A program may use either name, or both. Programs ported from z/OS keep their IBM names; the short names read more easily in new programs.

### Supplied Copybooks

| Copybook | Contents |
|---|---|
| DFHAID | 3270 attention identifier values, for `EIBAID`. |
| DFHRESP | Response values, for `EIBRESP` and RESP items. |
| DFHCOLOR | Color names for the *field*`-C` color override of `SEND MAP`. |
| DFHVALUE | Access values returned by `QUERY SECURITY`. |
| SQLCA | Named SQLCODE and SQLSTATE values. |
| DFHWBSC | HTTP status codes for `EXEC CICS WEB`. |
| DFHWBMT | Media types for `WEB SEND` and `WEB RECEIVE`. |
| DFHWBMETH | HTTP method names. |
| DFHWBUH | HTTP header names. |
| DFHWBHB | Work areas for browsing HTTP headers. |
| DFHWBSI | Session token and buffers for the outbound web client. |
| DFHWBCC | Work areas for `WEB EXTRACT CERTIFICATE`. |
| DFHWBUM | Work areas for `WEB EXTRACT URIMAP`. |
| DFHDCDOC | Document token and delimiters for `EXEC CICS DOCUMENT`. |

**Figure 26-1. Copybooks supplied in runtime/cobolcopy**

## DFHAID

`COPY DFHAID` declares one `PIC X` item for each attention identifier. The bricks names use two digits (`PF01` to `PF24`); the IBM names do not (`DFHPF1` to `DFHPF24`).

| bricks | IBM | Value | bricks | IBM | Value |
|---|---|---|---|---|---|
| ENTER | DFHENTER | X'7D' | PF10 | DFHPF10 | X'7A' |
| CLEAR | DFHCLEAR | X'6D' | PF11 | DFHPF11 | X'7B' |
| PA1 | DFHPA1 | X'6C' | PF12 | DFHPF12 | X'7C' |
| PA2 | DFHPA2 | X'6E' | PF13 | DFHPF13 | X'C1' |
| PA3 | DFHPA3 | X'6B' | PF14 | DFHPF14 | X'C2' |
| PF01 | DFHPF1 | X'F1' | PF15 | DFHPF15 | X'C3' |
| PF02 | DFHPF2 | X'F2' | PF16 | DFHPF16 | X'C4' |
| PF03 | DFHPF3 | X'F3' | PF17 | DFHPF17 | X'C5' |
| PF04 | DFHPF4 | X'F4' | PF18 | DFHPF18 | X'C6' |
| PF05 | DFHPF5 | X'F5' | PF19 | DFHPF19 | X'C7' |
| PF06 | DFHPF6 | X'F6' | PF20 | DFHPF20 | X'C8' |
| PF07 | DFHPF7 | X'F7' | PF21 | DFHPF21 | X'C9' |
| PF08 | DFHPF8 | X'F8' | PF22 | DFHPF22 | X'4A' |
| PF09 | DFHPF9 | X'F9' | PF23 | DFHPF23 | X'4B' |
| | | | PF24 | DFHPF24 | X'4C' |

**Figure 26-2. DFHAID constants**

## DFHRESP

`COPY DFHRESP` declares one `PIC S9(8)` item for each response value, named `RESP–`*condition* and `DFHRESP–`*condition* (`RESP–NOTFND`, `DFHRESP–NOTFND`). The values are those that bricks returns, the same as `DFHRESP(`*condition*`)`. The table also gives the abend code with which a COBOL or PL/I task ends when the condition is unhandled ("Condition Handling" in Chapter 25); "continue" means the default action is to continue.

| Condition | Value | Unhandled | Condition | Value | Unhandled |
|---|---|---|---|---|---|
| NORMAL | 0 | - | IGREQID | 39 | AEYC |
| ERROR | 1 | AEIA | OVERFLOW | 40 | continue |
| EOC | 6 | continue | INVLDC | 41 | AEYE |
| FILENOTFOUND | 12 | AEIL | NOSTG | 42 | continue |
| NOTFND | 13 | AEIM | JIDERR | 43 | AEYG |
| DUPREC | 14 | AEIN | QIDERR | 44 | AEYH |
| DUPKEY | 15 | AEIO | NOJBUFSP | 45 | continue |
| INVREQ | 16 | AEIP | DSSTAT | 46 | AEYJ |
| IOERR | 17 | AEIQ | SELNERR | 47 | AEYK |
| NOSPACE | 18 | AEIR | FUNCERR | 48 | AEYL |
| NOTOPEN | 19 | AEIS | UNEXPIN | 49 | AEYM |
| ENDFILE | 20 | AEIT | NOPASSBKRD | 50 | AEYN |
| ILLOGIC | 21 | AEIU | NOPASSBKWR | 51 | AEYO |
| LENGERR | 22 | AEIV | SYSIDERR | 53 | AEYQ |
| QZERO | 23 | AEIW | ISCINVREQ | 54 | AEYR |
| SIGNAL | 24 | continue | ENQBUSY | 55 | continue |
| QBUSY | 25 | continue | ENVDEFERR | 56 | AEYT |
| ITEMERR | 26 | AEIZ | IGREQCD | 57 | AEYU |
| PGMIDERR | 27 | AEI0 | SESSIONERR | 58 | AEYV |
| TRANSIDERR | 28 | AEI1 | SYSBUSY | 59 | continue |
| ENDDATA | 29 | AEI2 | SESSBUSY | 60 | continue |
| INVTSREQ | 30 | AEI3 | NOTALLOC | 61 | AEYY |
| EXPIRED | 31 | AEI4 | CBIDERR | 62 | AEYZ |
| RETPAGE | 32 | continue | INVEXITREQ | 63 | AEY0 |
| RTEFAIL | 33 | continue | INVPARTNSET | 64 | AEY1 |
| RTESOME | 34 | continue | INVPARTN | 65 | AEY2 |
| TSIOERR | 35 | AEI8 | PARTNFAIL | 66 | AEY3 |
| MAPFAIL | 36 | AEI9 | NOTAUTH | 70 | AEY7 |

| Condition | Value | Unhandled | Condition | Value | Unhandled |
|---|---|---|---|---|---|
| INVMPSZ | 38 | AEYB | ROLLEDBACK | 82 | AEXJ |
| | | | END | 83 | AEXK |

**Figure 26-3. DFHRESP constants**

In bricks, `ROLLEDBACK` is returned only by `SYNCPOINT ROLLBACK` when the local backout itself fails. The meaning of each condition for each command is given with the command in Part 3 and summarized in Chapter 18.

### SQLCA

The SQLCA fields themselves (`SQLCODE`, `SQLSTATE`, `SQLERRMC`, ...) are declared for every program (see "Automatically Declared Items" in Chapter 25). `COPY SQLCA` adds named constants to compare them with. The SQLCODE constants are `PIC S9(8)`; where an IBM DB2 name exists it is declared as well.

| bricks | DB2 | SQLCODE | Meaning |
|---|---|---|---|
| SQL-OK | DB2-SUCCESS | 0 | Successful. |
| SQL-NODATA<br>SQL-NOTFND | DB2-NOTFOUND | +100 | No row, or end of cursor. |
| SQL-NOCONFIG | | -1 | No database configured. |
| SQL-DDLREJECTED | | -2 | Data definition statement rejected. |
| SQL-GENERIC | | -100 | Other database error; see SQLERRMC and the log. |
| SQL-SYNTAX | | -104 | SQL syntax error. |
| SQL-AMBIG-COL | | -203 | Ambiguous column name. |
| SQL-UNDEF-TBL | DB2-UNDEF-TABLE | -204 | Table does not exist. |
| SQL-UNDEF-COL | | -206 | Column does not exist. |
| SQL-NOTNULL | | -407 | Null value for a NOT NULL column. |
| SQL-STRTRUNC | | -433 | String too long for its column. |
| SQL-CURSOR-RO | DB2-READ-ONLY | -510 | Cursor cannot be updated. |
| SQL-FKVIOL | DB2-FOREIGN-KEY | -530 | Foreign key violation. |
| SQL-CHKVIOL | | -545 | Check constraint violation. |
| SQL-INSUF-PRIV | | -551 | Not authorized. |
| SQL-NUMOVERFLOW | | -802 | Numeric overflow. |
| SQL-DUPKEY | DB2-DUP-KEY | -803 | Duplicate key. |
| SQL-MULTIPLEROWS | DB2-DUPLICATE | -811 | SELECT INTO returned more than one row. |
| SQL-DEADLOCK | DB2-DEADLOCK | -911 | Deadlock; the unit of work was rolled back. |
| SQL-CONNLOST | | -924 | Connection to the database lost. |
| SQL-TIMEOUT | | -952 | Statement timed out. |

**Figure 26-4. SQLCA SQLCODE constants**

The SQLSTATE constants are `PIC X(5)`: `SQLSTATE-OK` (00000), `-NODATA` (02000), `-WARNING` (01000), `-NOT-NULL` (23502), `-FK-VIOL` (23503), `-UQ-VIOL` and `-DUP-KEY` (23505), `-CHK-VIOL` (23514), `-CONN-FAIL` (08001), `-CONN-LOST` (08006), `-INV-AUTH` (28000), `-INV-PWD` (28P01), `-SYNTAX` (42601), `-UNDEF-TBL` (42P01), `-UNDEF-COL` (42703), `-UNDEF-FN` (42883), `-AMBIG-COL` (42702), `-INSUF-PRIV` (42501), `-STRTRUNC` (22001), `-NUMOVF` (22003), `-SERIAL` (40001), `-DEADLOCK` (40P01) and `-TIMEOUT` (57014), each prefixed `SQLSTATE`.

```
       COPY SQLCA.
           EXEC SQL INSERT INTO orders VALUES (:ID, :CUST) END-EXEC.
           EVALUATE SQLCODE
               WHEN SQL-OK         MOVE 'Inserted.' TO MSG
               WHEN SQL-DUPKEY     MOVE 'Order exists.' TO MSG
               WHEN SQL-FKVIOL     MOVE 'Unknown customer.' TO MSG
               WHEN OTHER          MOVE 'See bricks log.' TO MSG
           END-EVALUATE.
```

bricks writes the full database error text of every negative SQLCODE to the log; the program shows only a short message on the terminal.

## DFHCOLOR

`COPY DFHCOLOR` declares `PIC X(9)` color names for the *field*`-C` color override (see "Maps" in Chapter 25). There is no `DFHWHITE` in IBM CICS; white is `DFHNEUTR`.

| IBM | bricks | Value |
|---|---|---|
| DFHDFT | DEFAULT | DEFAULT |
| DFHBLUE | BLUE | BLUE |
| DFHRED | RED | RED |
| DFHPINK | PINK | PINK |
| DFHGREEN | GREEN | GREEN |
| DFHTURQ | TURQUOISE | TURQUOISE |
| DFHYELLO | YELLOW | YELLOW |
| DFHNEUTR | NEUTRAL | NEUTRAL |

**Figure 26-5. DFHCOLOR constants**

### DFHVALUE

`COPY DFHVALUE` declares the access values that `QUERY SECURITY` returns: `READABLE` (50), `NOTREADABLE` (51), `UPDATABLE` (52), `NOTUPDATABLE` (53), `CTRLABLE` (54), `NOTCTRLABLE` (55), `ALTERABLE` (56) and `NOTALTERABLE` (57), each also as `DFHVALUE–`*name*. bricks declares them as `PIC S9(8)` DISPLAY items, where IBM uses binary. Declare the items that receive the `QUERY SECURITY` results as `PIC S9(8)` DISPLAY too, so that the comparison is made between items of the same representation. `QUERY SECURITY` is described in "QUERY SECURITY" in Chapter 13.

## Web and Document Copybooks

| Copybook | Items |
|---|---|
| DFHWBSC | PIC 9(4) status codes: DFHRESP–WB–OK (200), –CREATED (201), –NOCONTENT (204), –MOVED (301), –FOUND (302), –NOTMOD (304), –BADREQ (400), –UNAUTH (401), –FORBID (403), –NOTFND (404), –METHOD (405), –CONFLICT (409), –LENGREQ (411), –TOOLARGE (413), –INTERNAL (500), –NOTIMPL (501), –BADGW (502), –UNAVAIL (503), –GWTIME (504). |
| DFHWBMT | PIC X(32) media types: WB–MT–JSON, –XML, –HTML, –PLAIN, –FORM, –OCTET, –CSV, –PDF. |
| DFHWBMETH | PIC X(7) methods: WB–GET, WB–POST, WB–PUT, WB–DELETE, WB–PATCH, WB–HEAD, WB–OPTIONS. |
| DFHWBUH | PIC X(20) header names: WB–CONTENT–TYPE, WB–CONTENT–LENGTH, WB–ACCEPT, WB–AUTHORIZATION, WB–LOCATION, WB–CACHE–CONTROL, WB–USER–AGENT, WB–HOST, WB–REFERER, WB–DATE, WB–EXPIRES, WB–IFMODSINCE, WB–IFNONEMATCH, WB–ETAG. |
| DFHWBHB | DFH–WB–HDR–NAME X(64), DFH–WB–HDR–NL 9(4) COMP, DFH–WB–HDR–VAL X(256), DFH–WB–HDR–VL 9(4) COMP. |
| DFHWBSI | DFH–WB–SESS X(36), DFH–WB–BODY X(8192), DFH–WB–URL X(2048). |
| DFHWBCC | DFH–WB–CN, DFH–WB–ORG, DFH–WB–ISSUER, DFH–WB–ISSUERORG X(64), DFH–WB–CO X(2), DFH–WB–SERIAL X(40). |

| Copybook | Items |
|---|---|
| DFHWBUM | DFH-WB-UM-NAME X(8), -SCHEME X(8), -HOST X(64), -PORT X(5), -PATH X(128). |
| DFHDCDOC | DFH-DOC-TOK X(36); delimiters DFH-DOC-DEL-AMP, -COMMA, -SEMI; DFH-DOC-DATAONLY. |

**Figure 26-6. Web and document copybooks**

The web commands are described in Chapter 20.

### Using Named Constants

Named constants make a program easier to read and remove the risk of a mistyped hexadecimal or numeric value. Instead of

```
           IF EIBAID = X'F3' THEN
               MOVE 'Y' TO EXIT-FLAG
           END-IF.
           IF EIBRESP = 13 THEN
               MOVE 'Customer not found.' TO MSG
           END-IF.
```

write

```
       WORKING-STORAGE SECTION.
       COPY DFHAID.
       COPY DFHRESP.
       01 WS-CICS-RESP PIC S9(8) COMP.
       ...
           IF EIBAID = PF03 THEN          *> or DFHPF3
               MOVE 'Y' TO EXIT-FLAG
           END-IF.
           EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY)
                     RESP(WS-CICS-RESP) END-EXEC.
           IF EIBRESP = RESP-NOTFND THEN
               MOVE 'Customer not found.' TO MSG
           END-IF.
```

The RESP option is required for the test of EIBRESP to be reached when the record is not found (see "Condition Handling" in Chapter 25). The samples QAGC (qagc.cob, the **F3** cancel branch), GUSL (gusl.cob, EVALUATE EIBAID with PF03, PF07, PF08 and PF12), ORDR (ordr.cob, RESP-QZERO and RESP-DUPREC) and EXAM (exam.cob, RESP-EOC), all in runtime/cobol/, use the copybooks.

### Differences from IBM Enterprise COBOL

- `COPY ... REPLACING`, library qualifiers (`OF`, `IN`) and `SUPPRESS` are not supported. bricks searches one copybook directory and produces no listing.
- A `COPY` statement must be alone on its line.
- The supplied copybooks declare each constant under both a bricks name and the IBM name; `DFHVALUE` constants are DISPLAY rather than binary.

Chapter 27

# COBOL Restrictions

This chapter lists the elements of IBM Enterprise COBOL that bricks does not accept, and the limits that it enforces. An element that is not listed here and is described in Chapters 22 through 26 is supported. Every restriction is detected when the program is compiled or, for limits that depend on data, when it runs; none of them can affect the region. The restrictions for PL/I are listed in Chapter 34.

## Program Structure

| Element | Status |
|---|---|
| Fixed reference format: sequence numbers in columns 1-6, continuation (–) and page-eject (/) indicators | Not supported; source is free form. |
| IDENTIFICATION DIVISION paragraphs other than `PROGRAM-ID` | Compile error. |
| ENVIRONMENT DIVISION contents | Compile error; the division header alone is accepted. |
| FILE SECTION, LOCAL-STORAGE SECTION | Compile error. |
| PROCEDURE DIVISION sections, `EXIT SECTION` | Compile error. |
| Nested and contained programs, `END PROGRAM` | Not supported. |

**Figure 27-1. Unsupported program structure**

## DATA DIVISION

| Element | Status |
|---|---|
| Levels 66 (RENAMES) and 78 | Compile error. |
| Level 77 after an 01 entry | Taken as subordinate to the preceding entry; code 77 items first (see "Level Numbers" in Chapter 23). |
| USAGE COMP-1, COMP-2 | Compile error (not yet supported). |
| USAGE NATIONAL, PIC N, PIC U, USAGE INDEX | Not supported. |
| SYNCHRONIZED, JUSTIFIED, SIGN, BLANK WHEN ZERO, EXTERNAL, GLOBAL | Compile error (unexpected ... in data item). |
| OCCURS DEPENDING ON, INDEXED BY, ASCENDING/DESCENDING KEY | Compile error. |
| Nested OCCURS (tables of more than one dimension) | Compile error; use one dimension and a computed subscript. |
| Editing symbols +, −, CR, DB, B, 0, /; signed numeric-edited pictures | Compile error. |
| PICTURE symbol P; V as the first symbol | Compile error. |
| VALUE ALL literal; VALUE on a group item | Compile error. |
| A level-01 redefining item longer than the item it redefines | Compile error (IBM allows it). |

**Figure 27-2. Unsupported DATA DIVISION elements**

## PROCEDURE DIVISION

| Element | Status |
|---|---|
| ACCEPT, ALTER, CALL, CANCEL, INITIALIZE, SEARCH, SEARCH ALL, SORT, MERGE, OPEN, READ, WRITE, CLOSE, INVOKE, XML GENERATE, XML PARSE | Compile error (unknown statement). Use EXEC CICS RECEIVE for terminal input, EXEC CICS LINK for another program, and the file control commands for files. |
| GO TO ... DEPENDING ON | Compile error. |
| NEXT SENTENCE | Not supported; use CONTINUE. |
| Several operands or receivers in ADD, SUBTRACT, MULTIPLY, DIVIDE; more than one COMPUTE receiver | Compile error; use COMPUTE or several statements. |
| DIVIDE ... REMAINDER | Compile error; compute the remainder with COMPUTE. |
| NOT ON SIZE ERROR; ON SIZE ERROR without END–*verb* | Compile error. |
| COMPUTE ... EQUAL | Compile error; write =. |
| MOVE CORRESPONDING, ADD CORRESPONDING, MOVE ALL literal, commas between MOVE receivers | Compile error. |
| SET condition-name TO TRUE, SET for index names | Compile error; SET supports pointers only. |
| DISPLAY ... WITH NO ADVANCING | Runtime error. |
| Reference modification combined with a subscript, or written on a qualifier | Compile error. |
| EVALUATE ... WHEN a OR b for value objects | Compile error; code two WHEN phrases. |

| Element | Status |
|---|---|
| Intrinsic functions other than UPPER-CASE, LOWER-CASE, TRIM, REVERSE, LENGTH, BYTE-LENGTH, ORD, CHAR, NUMVAL, POS (for example CURRENT-DATE, MOD, NUMVAL-C, the date, statistical and national functions); TRIM with LEADING or TRAILING | Error. Use EXEC CICS ASKTIME and FORMATTIME for the date and time. |
| JSON phrases ENCODING, IGNORING, INDICATING, CONVERTING ... FROM JSON NULL | Not supported. |
| COPY ... REPLACING, COPY ... OF or IN *library*, COPY ... SUPPRESS, REPLACE | Not supported. |

**Figure 27-3. Unsupported PROCEDURE DIVISION elements**

The following forms, which IBM also forbids, are rejected with a short message rather than run with a guessed meaning: a literal, figurative constant or function as the UNSTRING source; a reference-modified STRING receiver; DELIMITER IN or COUNT IN without DELIMITED BY; an EVALUATE whose WHEN has a different number of objects from the subjects; NOT NOT; and an abbreviated relation that opens a parenthesized group.

## Limits

| Limit | Value |
|---|---|
| Source size, after COPY expansion | 4 MiB (source exceeds the 4 MiB limit). |
| COPY nesting | 5 levels; cycles are detected. |
| Nesting of parentheses, **, conditions and statements within IF, EVALUATE and in-line PERFORM | 512 levels (nesting too deep (limit 512)). |
| Storage per program: WORKING-STORAGE and LINKAGE default buffers, tables included | 128 MiB (working storage exceeds the 128 MiB limit). |
| Length of an alphanumeric item | 1 MiB. |
| Digits in a numeric item or numeric-edited picture | 18. |
| OCCURS | 4096 occurrences; one dimension. |
| Nested PERFORM at runtime | 1024 per task, linked COBOL programs included. |
| Nesting of JSON text | 64 levels in JSON PARSE input (exception code 100). |
| Reference modification | Must lie within the item, for reads as well as stores. |
| Numeric or binary item based on shorter storage | Abend ASRA at the reference. |
| Division by zero without ON SIZE ERROR, in any statement or condition | Abend ASRA (DECIMAL-DIVIDE EXCEPTION), as IBM's S0CB. |
| Runtime without giving control back to the region | 2 seconds, then abend AICA (see "Runaway Tasks" in Chapter 16). |

**Figure 27-4. COBOL limits**

## Summary of Behavioral Differences

The following bricks behaviors differ from IBM Enterprise COBOL and are described where the element is defined.

- The character set is Latin-1 with a graphic plane, not EBCDIC ("Character Set" in Chapter 22).
- A signed DISPLAY item has a separate leading sign byte ("Storage of Signed DISPLAY Items" in Chapter 23).
- A `GO TO` out of a performed paragraph abandons every active `PERFORM`, and `STRING` without `POINTER` clears the receiver ("Differences from IBM Enterprise COBOL" in Chapter 24).
- `DISPLAY` output is discarded ("DISPLAY" in Chapter 24).
- JSON member names are generated in upper case ("JSON GENERATE" in Chapter 24).
- `EIBTRMID` holds four characters of a five-character terminal identifier, and the COBOL EIB has no `EIBABCODE` ("The EXEC Interface Block" in Chapter 25).
- `DFHVALUE` constants are DISPLAY items ("DFHVALUE" in Chapter 26).

Part 7

# PL/I

This part describes PL/I as bricks implements it: program structure, data declarations, expressions and decimal arithmetic, statements, built-in functions, `EXEC CICS` in a PL/I program, and the restrictions and deviations of the implementation.

**Chapter 28. PL/I Program Structure** ..... **28-1**
**Chapter 29. PL/I Data Declarations** ..... **29-1**
**Chapter 30. PL/I Expressions, Conversions and Decimal Arithmetic** ..... **30-1**
**Chapter 31. PL/I Statements and Control Flow** ..... **31-1**
**Chapter 32. PL/I Built-in Functions** ..... **32-1**
**Chapter 33. EXEC CICS in PL/I: Runtime Abends, the Unit of Work and %INCLUDE** ..... **33-1**
**Chapter 34. PL/I Restrictions and Deviations** ..... **34-1**

Chapter 28

# PL/I Program Structure

bricks includes an interpreter for a business subset of IBM Enterprise PL/I. A PL/I program issues the same `EXEC CICS` and `EXEC SQL` commands, through the same command handler, as a REXX or COBOL program; only the statement terminator differs (`EXEC CICS ... ;`, with no `END-EXEC`). This chapter and the six that follow describe the PL/I language as bricks implements it:

- Chapter 28, program structure, source rules and where programs are kept
- Chapter 29, data declarations and the byte layout of every data type
- Chapter 30, expressions, conversions and fixed-decimal arithmetic
- Chapter 31, statements and control flow
- Chapter 32, built-in functions
- Chapter 33, runtime abends, the unit of work, the CICS interface and `%INCLUDE`
- Chapter 34, restrictions, deviations and limits

The subset is defined so that a mainframe PL/I programmer can read any bricks PL/I program without a manual. Where Enterprise PL/I has a behavior that is hazardous in business code (implicit FLOAT declarations, SIZE disabled by default, intermediate decimal overflow, undefined uninitialized storage), bricks takes the safer behavior. Each such case is listed in "Deviations from Enterprise PL/I" in Chapter 34. Command-level behavior (what each `EXEC CICS` command does) is described in Part 3, `EXEC SQL` in Chapter 21.

The supplied sample `CUSP` (`runtime/pli/cusp.pli`) is a one-program PL/I port of the REXX `CUST` application; see "PL/I Samples" in Chapter 41 and "Example 7. Porting CUST to PL/I" in Chapter 42. The samples `ABNP` (`abnp.pli`) and `ABMP` (`abmp.pli`) are also PL/I programs.

### How to Read the Syntax Diagrams

The syntax of each statement in Part 7 is shown in a box in the following notation:

| Notation | Meaning |
|---|---|
| `UPPERCASE` | A keyword, entered as shown. Case is not significant in a program. |
| `lowercase` | A value supplied by the programmer. |
| `[ item ]` | The item is optional. |
| `{ a \| b }` | Exactly one of the items is chosen. |
| `item ...` | The item may be repeated. |

**Figure 28-1. Syntax notation used in the PL/I chapters**

## The PL/I Program

### *Syntax*

```
name: { PROCEDURE | PROC } OPTIONS(MAIN) ;
   declarations, statements and internal procedures
END [ name ] ;
```

### *Description*

A bricks PL/I program is one source file that contains exactly one external procedure. A program that bricks dispatches as a transaction, or that is reached through `EXEC CICS LINK` or `XCTL`, has `OPTIONS(MAIN)`. An `OPTIONS(MAIN)` procedure takes no parameters (compile error `MAIN PROC HAS NO PARAMETERS`); input arrives through the COMMAREA, `RECEIVE` or `RETRIEVE`.

The task ends normally, and its unit of work is committed, when the MAIN procedure executes `RETURN`, reaches its `END`, or issues `EXEC CICS RETURN`. In a program that was the target of `LINK`, the same three actions return control to the LINKing program. `EXEC CICS XCTL` ends the program but not the task: the unit of work is carried into the target program (see "The Unit of Work" in Chapter 33).

***Example***

```
/* HELP - minimum PL/I transaction: paint HELO1 and end. */
HELLOP: PROC OPTIONS(MAIN);
   DCL 1 SCR,
         2 INFOLINE CHAR(78),
         2 GREETING CHAR(20),
         2 FOOTER   CHAR(60);
   DCL UID CHAR(8);
   DCL RC  FIXED BIN(31);

   EXEC CICS ASSIGN USERID(UID) RESP(RC);
   SCR = '';
   SCR.GREETING = 'Hello, ' || TRIM(UID);
   SCR.FOOTER   = 'Press any key.';
   EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE RESP(RC);
   EXEC CICS RETURN;
END HELLOP;
```

An external procedure *without* `OPTIONS(MAIN)` is a callable subprogram. It is reached by `CALL` through an `ENTRY ... EXTERNAL` declaration, not through `LINK`; see "External Entries" in Chapter 31.

### Source Rules

PL/I source is free form. The rules are summarized in the following table.

| Item | Rule |
|---|---|
| Source form | Free form. There are no margins; each statement ends with `;`. |
| Case | Keywords and identifiers are not case-sensitive. |
| Identifiers | 1 to 31 characters from `A–Z 0–9 _ # @ $`, beginning with a letter or one of `_ # @ $`. A longer name is the compile error `NAME TOO LONG`. |
| Comments | `/* ... */`. A comment may span lines. Comments do not nest. |
| Character constants | `'O''BRIEN'` (a doubled quote stands for one quote). Double-quoted constants (`"ABC"`) are also accepted. |
| Bit constants | `'1'B`, `'0101'B` |
| Hexadecimal constants | `'C1C2'X` (character), `'00FF'XN` (numeric) |
| Constant suffixes | Only `B`, `X` or `XN` written directly after the closing quote is a suffix. Any other word in that position is the next token, so `IF S = 'A'THEN` is valid, as in IBM PL/I. |
| Numeric constants | `123`, `123.45`, `.5`. There is no exponent form, because there is no FLOAT data. |
| Abbreviations | `DCL PROC CHAR DEC BIN PIC VAR INIT AUTO EXT OTHER` |

**Figure 28-2. PL/I source rules**

### *Reserved words*

Enterprise PL/I has no reserved words. bricks reserves the statement keywords, so that the parser never has to guess:

```
IF  DO  END  SELECT  WHEN  OTHERWISE  OTHER  CALL  RETURN  LEAVE
ITERATE  PUT  EXEC  DCL  DECLARE  PROC  PROCEDURE  ELSE
```

None of these words may name a variable, a procedure or a label. `DCL SELECT CHAR(1);` fails with `RESERVED NAME SELECT`; a reserved word used as a label fails with `RESERVED WORD`. Every other keyword (`THEN`, `TO`, `BY`, `WHILE`, the attribute names) is recognized by context. Names of the form `_`*digits*`_` (`_1_`, `_42_LEAF`) are reserved for the aliases that the compiler generates for `EXEC CICS` arguments ("Options Are PL/I Expressions" in Chapter 33).

***Built-in function names***

The name of a built-in function (Chapter 32) cannot be declared again as a level-1 variable or as a procedure: `DCL INDEX FIXED BIN(31);` and `DCL DAYS(12) ...` are compile errors (`BUILTIN NAME DAYS`). A structure member at level 2 or deeper may have the name of a built-in function. This allows a map structure to contain a field called `SEARCH` ("Maps as Structures" in Chapter 33). `DCL` *name* `BUILTIN;` is accepted and documents that *name* refers to the built-in function.

## The PL/I Character Set

Program data is held one byte per character in the bricks single-byte character set (SBCS), as for REXX and COBOL. The source file is UTF-8; every quoted constant is converted to SBCS once, when the program is compiled.

- Inside quotes, ASCII, Latin-1 and the bricks graphic characters are allowed. Any other character is the compile error `CHARACTER U+4E2D NOT IN CODE PAGE` (with the code point of the character).
- Comments may contain any valid UTF-8 text.
- Outside quotes and comments only ASCII is allowed, plus the PL/I NOT sign (¬, U+00AC). Any other character is `BAD CHARACTER U+`*xxxx*.

**The NOT sign.** The native PL/I NOT operator is ¬. It is EBCDIC code page 037 `X'5F'`, a key that a 3270 keyboard has, so a program typed in the ISPF editor on a 3270 uses ¬, ¬=, ¬< and ¬>. Because an ASCII keyboard usually lacks the ¬ key, bricks accepts ^ as the same operator (^=, ^<, ^>), and <> for not-equal. OR is written | or !; concatenation is || or !!. The supplied `cusp.pli` uses <>, so that it reads the same in every editor.

```
IF A ¬= 2 THEN PUT SKIP LIST('not-sign');
IF A ^= 2 THEN PUT SKIP LIST('caret');
IF A <> 2 THEN PUT SKIP LIST('angle');
IF ¬F & (A = B ! A > B) THEN PUT SKIP LIST('or-bang');
```

**Note:**

^ and | are not EBCDIC 037 printable characters. Use ^ only in source edited outside bricks, and never put ^ or | in a character constant that is displayed on a 3270 screen.

### Procedures

***Syntax***

```
name: { PROCEDURE | PROC } [ ( parameter , ... ) ]
      [ RETURNS ( attributes ) ] [ RECURSIVE ] [ OPTIONS(MAIN) ] ;
   ...
END [ name ] ;
```

***Description***

- **Internal procedures** may be nested, and see the variables of their parents through lexical scope. Names are resolved over the whole enclosing block, so a procedure may be called before its `PROC` statement, and a variable may be declared anywhere in its block. The usual layout, used by `cusp.pli`, is declarations first, then the main line, then the internal procedures.
- **Parameters** are passed by reference. `CHAR(*)` (length taken from the argument) is allowed on parameters only. Argument passing and dummy arguments are described in "CALL" in Chapter 31.
- **Function procedures** declare `RETURNS(`*attributes*`)` and are invoked in expressions. A `CALL` of a function is a compile error (`CALL OF FUNCTION F`). A function that reaches its `END` without executing `RETURN(`*value*`)` abends `ABEN`.
- **RECURSIVE.** A procedure that is already active may call itself, directly or indirectly, only if it is declared `RECURSIVE`; otherwise the call abends `ABEN`.
- **Call depth.** At most 100 procedure calls may be active in one task at the same time, counting the procedures of PL/I programs reached through `EXEC CICS LINK`. The call that would exceed the limit abends `ABEN`.

***Example***

```
ORDPRICE: PROC OPTIONS(MAIN);
   DCL AMT FIXED DEC(13,2) INIT(199.99);
   PUT SKIP LIST('TAX', CALCTAX(AMT));      /* called before its PROC */
   CALL SHOW('done');

   CALCTAX: PROC(A) RETURNS(FIXED DEC(13,2));
      DCL A FIXED DEC(13,2);
      RETURN (ROUND(A * 0.0825, 2));
   END CALCTAX;

   SHOW: PROC(MSG);
      DCL MSG CHAR(*) VAR;
      PUT SKIP LIST(MSG);
   END SHOW;
END ORDPRICE;
```

***Differences from IBM Enterprise PL/I***

There are no `BEGIN` blocks, no statement labels other than on `PROC` and `DO`, no `ENTRY` variables and no generic entries. The call-depth limit of 100 is a bricks limit.

## Where Programs Are Kept

| Item | Location |
|---|---|
| `Programs` | The directory named by `pli_dir` in `bricks.cnf` (default `runtime/pli`), one program per file, extension `.pli` |
| `Include members` | The directory named by `pli_include_dir` (default `runtime/pliinc`), one member per file, `NAME.inc` ("%INCLUDE" in Chapter 33) |
| `Transactions` | `runtime/transactions.conf`, language token `pli` |

**Figure 28-3. Where PL/I programs and members are kept**

```
# bricks.cnf
pli_dir=runtime/pli
pli_include_dir=runtime/pliinc
```

Both keys default to directories under `runtime_dir`, like `rexx_dir`, `cobol_dir` and `copybook_dir`, so a new installation needs neither line (see "The bricks.cnf File" in Appendix D). A transaction names its program with the language token `pli`:

```
CUSP:pli:cusp.pli:PUBLIC
```

The token is exactly `pli` (not case-sensitive). The spellings `PL/I` and `PL1` are rejected, so that `transactions.conf` keeps one form per language. The optional fifth field (database binding) works as for the other languages ("Connecting a Transaction to a Database" in Chapter 21).

### Checking and Editing Programs

- **brickscompile** accepts `.pli` files, and directories that contain them. It resolves `%INCLUDE` through the `pli_include_dir` of the nearest `bricks.cnf`.
- **CEDA TRANSACTION** accepts `PLI` in the LANGUAGE field (the hint reads `(REXX, COBOL or PLI)`). **CEDA PROGRAM** and **CEMT PERFORM PROGRAMS** list the `.pli` files under `pli_dir` with a SYNTAX verdict (`OK` or `ERR L`*n*). Include members are not programs and are not listed.
- **ISPF** option 4 (`Edit PL/I programs`) edits `pli_dir` with PL/I highlighting, and checks the buffer with the same parser, `%INCLUDE` included, before it is saved ("Saving" in Chapter 9).
- **EXEC CICS INQUIRE PROGRAM(***name***) LANGUAGE(***v***)** returns `PLI` for a PL/I program.

#### *brickscompile output*

For a program without errors, `brickscompile` reports counts:

```
$ go run ./cmd/brickscompile runtime/pli/cusp.pli
runtime/pli/cusp.pli: OK
       lines     463
  procedures      23
  statements     188
   EXEC CICS      22
     elapsed  0s.1ms
```

A compile error names the line and a short ASCII message, followed by the source line:

```
t.pli:3: parse error: UNDECLARED QTYY
  source line 3:    QTYY = 1;
```

The compiler's own message form, which CEDA, CEMT and the ISPF save check display, also gives the column: `line 3 col 4: UNDECLARED QTYY`. An error inside an include member is prefixed with the member name (`DFHAID line 3 col 1: ...`).

### Program Caching and Task Storage

The compiled program is cached, as REXX and COBOL programs are. The cache key covers the program file and every include member that the program uses, so editing a member causes every program that includes it to be compiled again on its next dispatch ("%INCLUDE" in Chapter 33). Each task receives its own storage; no mutable state is shared between tasks ("Storage Classes" in Chapter 29).

Chapter 29

# PL/I Data Declarations

Every variable in a bricks PL/I program is declared with the `DECLARE` (`DCL`) statement. There are no implicit declarations: in Enterprise PL/I an undeclared name such as `QTYY` silently becomes a FLOAT variable; in bricks it is the compile error `UNDECLARED QTYY`. This chapter describes the `DECLARE` statement, the data types, structures and arrays, initial values, storage classes, and the byte layout of every type.

## DECLARE

### *Syntax*

```
{ DECLARE | DCL } item [ , item ] ... ;

item:
   [ level ] name [ ( bounds ) ] [ attributes ]
 | [ level ] ( name , ... ) [ attributes ]

attributes:
   type [ STATIC | AUTOMATIC ] [ INIT ( value , ... ) ]
 | type VALUE ( expression )
 | LIKE structure-name
 | ENTRY ( descriptor , ... ) [ RETURNS ( attributes ) ] EXTERNAL
 | BUILTIN
```

### *Description*

A declaration names one or more variables and gives their attributes. A level number (1 to 15) introduces a structure; an item without a level number is a level-1 scalar or array. A list of names in parentheses (factored attributes) gives every name the same attributes: `DCL (A, B, C) FIXED DEC(9,2) INIT(1.5);`. A declaration may appear anywhere in its block; names resolve over the whole block.

The attributes `UNALIGNED`, `NONVARYING`, `INTERNAL` and `NONASSIGNABLE` are accepted and have no effect. An attribute outside the subset is the compile error *`attribute`* `NOT SUPPORTED` (see "Features Not Supported" in Chapter 34); an unknown word is `UNKNOWN ATTRIBUTE`.

***Example***

```
DCL PRICE    FIXED DEC(9,2)  INIT(1234.56);
DCL QTY      FIXED BIN(31)   INIT(12);
DCL TXN_ID   FIXED BIN(63)   INIT(9000000000);
DCL BYTES    UNSIGNED FIXED BIN(64);
DCL NAME     CHAR(20)        INIT('ACME');
DCL NOTE     CHAR(100) VAR   INIT('short');
DCL FLAGS    BIT(8)          INIT('10100000'B);
DCL ZIP      PIC '99999'     INIT(1234);
DCL MONEY    PIC '$$$,$$9V.99' INIT(1234.5);
```

`ZIP` holds `01234` and `MONEY` holds `$1,234.50`. `HUGE(QTY)` is `2147483647`; `HUGE(BYTES)` is `18446744073709551615`.

## Data Types

| Type | Syntax | Limits | Use |
|---|---|---|---|
| Fixed decimal | `FIXED DEC(p,q)` | p <= 31, 0 <= q <= p; default (5,0) | Money, quantities, rates. The default for business data. |
| Fixed binary | `FIXED BIN(p)` | Signed 15, 31 or 63; other values are rounded up; default 15 | Counters, subscripts, lengths |
| 64-bit integer | `FIXED BIN(63)`<br>`UNSIGNED FIXED BIN(64)` | +/-9.22x10$^{18}$; 0 to 1.84x10$^{19}$ | Identifiers, time stamps, record counts |
| Character | `CHAR(n)` | 1 <= n <= 32767 | Fixed-length text, padded with blanks |
| Varying character | `CHAR(n) VAR` | 1 <= n <= 32767 | Variable-length text |
| Bit | `BIT(n)` | 1 <= n <= 64 | Flags; `BIT(1)` is the Boolean type |
| Numeric picture | `PIC '...'` | At most 31 digit positions | Edited output, zoned input |
| Character picture | `PIC 'XXX'`<br>`PIC 'AAA'` | | Validated text |

**Figure 29-1. PL/I data types**

- **FIXED is required.** `DEC(7,2)` and `BIN(31)` without `FIXED` mean FLOAT in Enterprise PL/I; in bricks they are the compile error `FIXED REQUIRED`.
- **UNSIGNED** is allowed only with `FIXED BIN` (otherwise `UNSIGNED ONLY WITH FIXED BIN`), with the precisions 8, 16, 32 and 64; other values are rounded up. `SIGNED` is the default and may be written.
- `FIXED BIN(64)` without `UNSIGNED` is accepted as a synonym for `FIXED BIN(63)`.
- A scale on a binary type (`FIXED BIN(31,2)`) is the compile error `BIN SCALE NOT SUPPORTED`.
- **Not available:** `FLOAT`, `COMPLEX`, `REAL`, `POINTER`, `OFFSET`, `AREA`, `LABEL`, `FORMAT`, `FILE`, `HANDLE`, `ORDINAL`, `UNION`, `GRAPHIC`, `WIDECHAR`, `CONDITION`, `TASK`, `EVENT`, `GENERIC`, and the storage attributes `BASED`, `CONTROLLED`, `DEFINED` and `ALIGNED`. `EXTERNAL` is accepted only on an `ENTRY` declaration.

## Picture Characters

| Character | Meaning |
|---|---|
| 9 | Digit |
| Z | Digit; a leading zero is replaced by a blank |
| * | Digit; a leading zero is replaced by * |
| V | Implied decimal point. Aligns the value; it is not displayed and occupies no byte. |
| . , / B | Insertion characters (B inserts a blank); suppressed inside a zero-suppressed field |
| $ S + − | Sign or currency symbol: static when written once, drifting when repeated |
| CR DB | Trailing credit or debit symbol, displayed only for a negative value |
| X A | Any character; alphabetic character or blank (character pictures) |

**Figure 29-2. Picture characters**

- A `.` written without `V` is only an insertion character, as in Enterprise PL/I: `PIC '999.99'` has scale 0. Write `PIC '999V.99'`.
- Repetition factors are accepted: `PIC '(5)9'` is `PIC '99999'`.
- Assigning a negative value to a picture that has no sign position (`S + - CR DB`) abends `ABSZ`; the sign is never dropped silently. A `+` sign position displays a blank for a negative value.

### Structures and Arrays

```
DCL MAX_LINES FIXED BIN(31) VALUE(3);
DCL 1 ORDER,
      2 CUST_ID    CHAR(8)       INIT('C0042'),
      2 LINE_COUNT FIXED BIN(15) INIT(2),
      2 LINE(MAX_LINES),
        3 SKU      CHAR(12),
        3 QTY      PIC '99999',
        3 PRICE    FIXED DEC(9,2);
DCL 1 SAVE_ORDER LIKE ORDER;
DCL MDAYS(12) FIXED BIN(15) INIT(31,28,31,30,31,30,31,31,30,31,30,31);
DCL TABLE(5) FIXED DEC(5,2) INIT((5)0);
DCL GRID(0:2, 1:4) CHAR(4);
DCL (A, B, C) FIXED DEC(9,2) INIT(1.5);
DCL BAR CHAR(6) INIT((3)'AB');
```

- **Levels** run from 1 to 15 (`BAD LEVEL NUMBER` otherwise). The first item of a declaration that uses level numbers must be level 1. Arrays of structures are allowed.
- **Arrays** have at most 3 dimensions, counting dimensions inherited from enclosing structure arrays (`MORE THAN 3 DIMENSIONS`). Bounds are constants or `VALUE` names; the default lower bound is 1, and `GRID(0:2, 1:4)` overrides it.
- **Qualified references.** `ORDER.LINE(1).QTY` is the full form. A subscript may be written at any level, so `ORDER.LINE.QTY(1)` is the same element. Partial qualification is allowed while it remains unambiguous: `ORDER.QTY(2)` is valid, but a bare `QTY(2)` in the example above is the compile error `AMBIGUOUS QTY`, because `SAVE_ORDER` also has a member `QTY`.
- **LIKE** copies the layout of a structure, including every member attribute and `INIT`, but not the storage class. A parameter declared with `LIKE` ignores the copied `INIT`: its storage is that of the argument. A `LIKE` target that is not declared is `LIKE TARGET` *name* `UNDECLARED`.

### INIT, VALUE and Default Values

- `INIT(...)` sets the initial value. A factor in parentheses before a constant is an iteration factor for an array (`INIT((5)0)` is five zeros) and a string repetition for a scalar (`INIT((3)'AB')` is `'ABABAB'`). A repetition longer than the target (n characters for `CHAR(n)`, otherwise 32767) is the compile error `INIT VALUE TOO LONG FOR` *name*; more values than the array has elements is `TOO MANY INIT VALUES`.
- `VALUE(`*expression*`)` declares a named constant. A named constant cannot be assigned; it may be used as an array bound, and inside `EXEC CICS` it is folded into the command text. The supplied members `DFHAID` and `DFHRESP` consist of `VALUE` declarations.
- **Deterministic defaults.** Enterprise PL/I leaves uninitialized storage undefined. bricks gives every variable that has no `INIT` a fixed initial value each time its storage is allocated, as shown in the following table.

| Type | Initial value |
|---|---|
| `CHAR(n), character PIC` | All blanks |
| `FIXED DEC, FIXED BIN, numeric PIC` | 0 |
| `BIT(n)` | All `'0'B` |
| `CHAR(n) VAR` | Length 0 |

**Figure 29-3. Initial values of variables declared without INIT**

### Storage Classes

| Class | Lifetime |
|---|---|
| `AUTOMATIC` | The default; abbreviation `AUTO`. One procedure invocation. `INIT`, or the default value, is applied again on every entry to the procedure. |
| `STATIC` | One task. Initialized the first time the program runs in the task, then shared by every activation of that program in the task, including activations reached by `EXEC CICS LINK` and by a `CALL` to an external entry. Never shared between tasks. |

**Figure 29-4. PL/I storage classes**

```
COUNTER: PROC;
   DCL CALLS FIXED BIN(31) STATIC INIT(0);    /* once per task  */
   DCL LOCAL FIXED BIN(31) INIT(0);           /* every entry    */
   CALLS = CALLS + 1;
   LOCAL = LOCAL + 1;
   PUT SKIP LIST('CALLS', CALLS, 'LOCAL', LOCAL);
END COUNTER;
```

When `COUNTER` is called three times in one task, the `PUT` statements form the lines `CALLS 1 LOCAL 1`, `CALLS 2 LOCAL 1` and `CALLS 3 LOCAL 1`. The next task starts again at `CALLS 1`.

***Storage limits***

bricks runs in a fixed memory budget:

- One program may declare at most 1 MiB of `AUTOMATIC` storage (all procedures together) plus `STATIC` storage. More is the compile error `STORAGE LIMIT EXCEEDED`.
- One task may hold at most 8 MiB of live activations. Every active call holds its `AUTOMATIC` storage until it returns, so deep recursion over a large `AUTOMATIC` area counts each level; a call to an external procedure also counts its `STATIC` storage. The call that would exceed the limit abends `ABST`.

**Note:**

`STATIC` storage belongs to the task, not to the region. Data that must outlive the task, or be seen by other tasks, belongs in a KSDS file, a temporary storage queue or the database, never in program storage.

## Storage Layout and COMMAREA Exchange with COBOL

All data is `UNALIGNED`: there is no padding between structure members, and the byte image of every type is fixed. A structure passed as a COMMAREA, a map area or a file record therefore has the same bytes in PL/I and in COBOL. `STG(`*x*`)` (`STORAGE`) returns the declared size; `CSTG(`*x*`)` (`CURRENTSTORAGE`) returns the current size.

| PL/I declaration | Bytes | Encoding | COBOL equivalent |
|---|---|---|---|
| `CHAR(n)` | n | Characters | `PIC X(n)` |
| `CHAR(n) VAR` | 2+n | 2-byte big-endian length, then n bytes (the unused tail is written as blanks) | `PIC S9(4) COMP` length, then `PIC X(n)` |
| `FIXED DEC(p,q)` | p/2+1 | Packed decimal, sign nibble `C` or `D`; an even p has a leading zero nibble | `PIC S9(p−q)V9(q) COMP-3` |
| `FIXED BIN(15)` | 2 | Big-endian two's complement | `PIC S9(4) COMP` |
| `FIXED BIN(31)` | 4 | Big-endian two's complement | `PIC S9(9) COMP` |
| `FIXED BIN(63)` | 8 | Big-endian two's complement | `PIC S9(18) COMP` |
| `UNSIGNED FIXED BIN(8/16/32/64)` | 1/2/4/8 | Big-endian unsigned | `PIC 9(n) COMP` of the same width |
| `BIT(n)` | ceil(n/8) | Bits from the high-order end; unused low-order bits zero | `PIC X(ceil(n/8))` |
| `PIC '...'` | One per picture character (`V` excluded) | Character image as displayed | The matching `PIC 9...` or edited `PIC` (DISPLAY) |

**Figure 29-5. Storage layout of PL/I data types**

When packed data is read, the sign nibbles `F`, `A`, `E` (plus) and `B` (minus) are also accepted, so that a COBOL unsigned `COMP-3` item is read correctly. The size of a structure is the sum of the sizes of its members; the size of an array is the element size multiplied by the number of elements.

***Example***

```
DCL 1 CUST_AREA,
      2 CA_KEY      CHAR(8),
      2 CA_BALANCE  FIXED DEC(11,2),
      2 CA_COUNT    FIXED BIN(31),
      2 CA_FLAGS    BIT(8),
      2 CA_NOTE     CHAR(20) VAR,
      2 CA_ZONED    PIC '9999';
```

`STG(CUST_AREA)` is 45 (8 + 6 + 4 + 1 + 22 + 4). The COBOL program on the other side of the `LINK` declares the same 45 bytes:

```
       01 CUST-AREA.
          05 CA-KEY      PIC X(8).
          05 CA-BALANCE  PIC S9(9)V99 COMP-3.
          05 CA-COUNT    PIC S9(9) COMP.
          05 CA-FLAGS    PIC X.
          05 CA-NOTE-LEN PIC S9(4) COMP.
          05 CA-NOTE     PIC X(20).
          05 CA-ZONED    PIC 9(4).
```

**Note:**

A PL/I numeric picture is held as its character image, one byte per picture character: `PIC '9999'` holding 42 is the four characters `0042`, the same bytes as a COBOL `PIC 9(4)` DISPLAY item. A signed picture (`PIC 'S9999'`) holds the sign in its own leading byte (`+0042`, five bytes), which is not the COBOL zoned overpunch. Carry signed values as `FIXED DEC` / `COMP-3`.

### Differences from IBM Enterprise PL/I

- Every variable must be declared; there are no implicit or contextual declarations.
- `FIXED` is required with `DEC` and `BIN`.
- Variables without `INIT` have defined initial values.
- `STATIC` lasts for one task, not for one program load.
- All storage is `UNALIGNED`; `ALIGNED` is not accepted.
- `FIXED BIN(64)` without `UNSIGNED` is accepted as `FIXED BIN(63)`.
- The storage limits of 1 MiB per program and 8 MiB per task are bricks limits.

Chapter 30

# PL/I Expressions, Conversions and Decimal Arithmetic

This chapter describes the PL/I operators and their precedence, the rules that determine the precision of every fixed-point result, mixed binary and decimal arithmetic, assignment and rounding, and the conversions between data types.

## Operators and Precedence

The operators are listed below from the highest precedence to the lowest.

| Level | Operators | Associativity |
|---|---|---|
| 1 | ** prefix + – ¬ | Right |
| 2 | * / | Left |
| 3 | + – | Left |
| 4 | \|\| | Left |
| 5 | = ¬= < > <= >= ¬< ¬> | Non-chaining |
| 6 | & | Left, short-circuit |
| 7 | \| | Left, short-circuit |

**Figure 30-1. PL/I operator precedence**

- **= is both assignment and comparison.** At statement level the first = assigns, so `F = I = 0;` stores the result of the comparison `I = 0` (a `BIT(1)` value) in `F`.
- **Comparisons do not chain.** `A < B < C` is the compile error `CHAINED COMPARISON`; write `A < B & B < C`.
- **Conditions** (`IF`, `WHILE`, `UNTIL`, the Boolean form of `WHEN`) require a bit expression (`BIT EXPRESSION EXPECTED`); a bit string that is not all zeros is true.
- **Short-circuit evaluation.** When both operands of `&` or `|` are `BIT(1)`, the right operand is evaluated only if it is needed. Otherwise both operands are evaluated and the operation is bitwise.
- **Exponentiation.** The exponent of `**` must be an integer constant from 0 to 64 (`EXPONENT MUST BE A SMALL INTEGER CONSTANT`); the result is computed by repeated multiplication. Prefix minus has the same precedence as `**`, but `**` is applied first: `-2 ** 2` is `-4`.
- A bit string in an arithmetic operation is the compile error `BIT STRING IN ARITHMETIC`; an array or structure in an expression is `AGGREGATE IN EXPRESSION`.

```
IF I > 0 & TAB(I) = 5 THEN PUT SKIP LIST('hit');
ELSE PUT SKIP LIST('no hit, no ABSR');
```

With `I = 0` the subscript `TAB(I)` is never evaluated, so the statement does not abend `ABSR`.

### Fixed-Decimal Precision

Every arithmetic result has a precision (p,q) that is computed when the program is compiled, from the precisions (p1,q1) and (p2,q2) of its operands, with a maximum precision N of 31:

| Operator | Integer digits | Scale q |
|---|---|---|
| + − | max(p1-q1, p2-q2) + 1 | max(q1, q2) |
| * | (p1-q1) + (p2-q2) + 1 | q1 + q2 |
| / | (p1-q1) + q2 | 31 - integer digits |

**Figure 30-2. Precision of fixed-decimal results**

The result precision is then limited as follows:

```
int' = min(int, 31)
q'   = min(q, 31 - int')
p'   = int' + q'
```

The exact value is truncated toward zero to the scale q'. Only if the integer part still needs more than int' digits is the `ABFO` (FIXEDOVERFLOW) abend raised.

**The bricks rule.** When a result would need more than 31 digits, digits of the scale are dropped to make room; an intermediate result never overflows on the right. This removes the Enterprise PL/I trap in which `25 + 1/3` raises FIXEDOVERFLOW:

- `1/3`: both constants are `DEC(1,0)`. Integer digits are (1-0)+0 = 1 and the scale is 31-1 = 30, so the result is `DEC(31,30)`, `0.333...3`.
- `25 + 1/3`: integer digits are max(2, 1)+1 = 3 and the scale is max(0, 30) = 30; the limit gives q' = min(30, 31-3) = 28. The result is `25.3333333333333333333333333333` (28 decimal places). There is no overflow.

***Example: an invoice line***

```
DCL PRICE FIXED DEC(7,2) INIT(19.99);
DCL QTY   FIXED DEC(5,0) INIT(3);
DCL RATE  FIXED DEC(5,4) INIT(0.0825);
DCL TOTAL FIXED DEC(9,2);
DCL TAX   FIXED DEC(9,2);

TOTAL = PRICE * QTY;              /* DEC(13,2): 59.97            */
TAX   = TOTAL * RATE;             /* 4.947525 truncated to 4.94  */
TAX   = ROUND(TOTAL * RATE, 2);   /* 4.95                        */
```

- `PRICE * QTY`: integer digits (7-2)+(5-0)+1 = 11, scale 2+0 = 2, so `DEC(13,2)`; the value 59.97 fits `TOTAL`.
- `TOTAL * RATE`: integer digits (9-2)+(5-4)+1 = 9, scale 2+4 = 6, so `DEC(15,6)` = 4.947525. Assigning it to a `DEC(9,2)` target truncates it to 4.94.
- `ROUND(`*x*`, 2)` rounds half away from zero, giving 4.95. Use `ROUND` wherever money is rounded.

### Division

Division is always decimal; `/` never produces a binary result. `FIXED BIN` operands of `/` are converted to `FIXED DEC` first, so `7/2` is `3.5` (displayed with its full scale, `3.50000000000000000000000000000000`), not `3`. To drop the fraction, use `TRUNC`, an integer target, or `DIVIDE` with an explicit precision:

```
DCL N     FIXED BIN(31)  INIT(7);
DCL R     FIXED DEC(9,2);
R = N / 2;
PUT SKIP LIST(R, TRUNC(N / 2), DIVIDE(N, 2, 5, 1));
```

The `PUT` statement forms the line `3.50 3 3.5`. `ADD(x, y, p, q)`, `MULTIPLY(x, y, p, q)` and `DIVIDE(x, y, p, q)` compute the result at exactly the precision (p,q). Division by zero abends `ABZD`.

### Mixed Binary and Decimal Operands

- **Binary with decimal.** The binary operand is converted to decimal, not the other way round: `BIN(15)` becomes `DEC(5,0)`, `BIN(31)` `DEC(10,0)`, `BIN(63)` `DEC(19,0)`, and `UNSIGNED BIN(64)` `DEC(20,0)`. Decimal constants (`1`, `0.5`) are decimal, so `N + 1` with a `FIXED BIN(31)` variable `N` is a decimal addition.
- **Binary with binary.** +, − and ∗ are computed as `BIN(63)` (64-bit integers); an overflow abends `ABFO` and never wraps around. An `UNSIGNED` operand makes the operation unsigned 64-bit; a result below 0 or above $2^{64}$-1 abends `ABFO`.
- **Pictures** are computed as the `FIXED DEC(p,q)` that their digit positions and `V` imply.
- A `CHAR` operand of an arithmetic operator is converted to `FIXED DEC(31,8)`. A value with more integer digits than that precision allows abends `ABFO`.

### Truncation and ROUND

Assignment between arithmetic types truncates the scale to the scale of the target; it never rounds. If the integer part does not fit the target, the task abends `ABSZ`; the SIZE condition is always enabled.

```
DCL X  FIXED DEC(7,3) INIT(-2.675);
DCL Y  FIXED DEC(7,2);
DCL I  FIXED BIN(31);
Y = X;
I = X;
PUT SKIP LIST(Y, ROUND(X, 2), I, CEIL(X), FLOOR(X), TRUNC(X));
```

The PUT statement forms the line `-2.67 -2.68 -2 -2 -3 -2`. Assigning `12345.67` to a `FIXED DEC(5,2)` target abends ABSZ, instead of keeping `345.67`.

## Conversions

| From and to | Rule |
|---|---|
| arithmetic to arithmetic | The scale is truncated to that of the target; ABSZ if the integer part does not fit. |
| CHAR to arithmetic | Optional blanks, an optional sign, digits, an optional `.`. An all-blank string is 0. Anything else abends ABCV. |
| arithmetic to CHAR | Minimal form: no leading blanks, – only when negative, every scale digit kept (`12.50`, `-3.00`). Use EDIT or a PIC variable for formatted output. |
| CHAR to CHAR | Padded with blanks or truncated on the right; truncation is silent. |
| CHAR to and from BIT | Only the characters `0` and `1`; anything else abends ABCV. |
| arithmetic to BIT | Compile error (ARITHMETIC TO BIT). Use the `BIT(x, n)` built-in function. |

**Figure 30-3. Conversion rules**

```
AMT = '  -12.5 ';                /* -12.50                    */
N   = '';                        /* 0                         */
S   = AMT;                       /* '-12.50'                  */
C   = 'ABCDEFG';                 /* CHAR(5): 'ABCDE'          */
PUT SKIP LIST(CHAR(42), EDIT(AMT, 'ZZZ9V.99-'));
```

`CHAR(42)` is `'42'` (never a right-justified field), and the EDIT result is `'   12.50-'`.

- **Comparisons.** Two `CHAR` operands are compared after the shorter has been padded with blanks, in SBCS byte order (`'AB' = 'AB  '` is true). A `CHAR` operand compared with a number is converted to a number exactly (`'42' = 42.0` is true). Comparing a `BIT` operand with a number is the compile error `COMPARE NUMBER WITH BIT`; a `BIT` operand compared with a `CHAR` operand is compared as characters.
- **Pictures** yield their character image where a string is expected (concatenation, a `CHAR` target, `PUT LIST`) and their value where a number is expected. Picture data that does not match its picture abends `ABCV` only when it is used as a number.
- **Concatenation.** `||` of two bit strings with at most 64 bits in total is a bit string; every other concatenation is character. A character result longer than 32767 characters abends `ABSR`.

### Validating Data Before Conversion

There is no `ON CONVERSION` unit. A program checks data before converting it:

```
IF VERIFY(TXT, '0123456789') = 0 THEN QTY = TXT;
ELSE PUT SKIP LIST('not numeric at', VERIFY(TXT, '0123456789'));
IF VALIDDATE('20260230', 'YYYYMMDD') THEN PUT SKIP LIST('date ok');
ELSE PUT SKIP LIST('bad date');
```

`VALID(`*variable*`)` tests a picture or `FIXED DEC` variable. It is the way to check map input received into a numeric `PIC` member ("Maps as Structures" in Chapter 33).

## Differences from IBM Enterprise PL/I

| Area | bricks | Enterprise PL/I |
|---|---|---|
| Decimal precision | Scale digits are dropped to fit 31 digits; overflow only on the integer part | FIXEDOVERFLOW on intermediate results (`25 + 1/3`) |
| `/` | Always decimal; binary operands are converted to decimal | Binary division for binary operands |
| Binary with decimal | The binary operand is converted to decimal | The decimal operand is converted to binary |
| Comparisons | Non-chaining | Chaining allowed |
| `&` and `\|` | Short-circuit for `BIT(1)` operands | Order of evaluation not guaranteed |
| Arithmetic to `CHAR` | Minimal form (`'42'`, `'-3.00'`) | Right-justified with leading blanks |
| SIZE | Always enabled: `ABSZ` | Disabled by default |
| `**` | Integer constant exponent 0 to 64 only | Any exponent; FLOAT results |

**Figure 30-4. Expression differences from Enterprise PL/I**

Chapter 31

# PL/I Statements and Control Flow

This chapter describes the PL/I statements, other than `DECLARE` ("DECLARE" in Chapter 29), `EXEC CICS`, `EXEC SQL` and `%INCLUDE` (Chapter 33). The statements are presented in alphabetical order of their purpose: assignment, `CALL`, `DO`, external entries, `IF`, `LEAVE` and `ITERATE`, the null statement, `PUT`, `RETURN` and `SELECT`.

A label is allowed only on a `PROC` or a `DO` statement. It names the target of `LEAVE`, `ITERATE` and `END` *name*. There are no other statement labels and no `GO TO`.

## Assignment

### *Syntax*

```
target [ , target ] ... = expression [ , BY NAME ] ;

SUBSTR ( string-variable , start [ , length ] ) = expression ;
```

### *Description*

- Several targets may be given; each receives the value, with its own conversion.
- A scalar value assigned to an array or a structure is assigned to every element, each with its own conversion. `REC = '';` is the usual way to clear a whole map structure: `''` sets every `CHAR` member to blanks and converts to 0 for every numeric member.
- An aggregate assigned to an aggregate requires the same shape (`AGGREGATE SHAPES DIFFER` otherwise). An aggregate cannot be assigned to a scalar (`AGGREGATE TO SCALAR`).
- `BY NAME` copies only the members whose names match, whatever their order, converting each. Members without a partner are left unchanged. Both sides must be structures (`BY NAME NEEDS STRUCTURES`).
- The `SUBSTR` pseudovariable replaces *length* characters of a character variable, starting at position *start*. A position outside the string abends `ABSR`.
- A `VALUE` name cannot be assigned (`NOT ASSIGNABLE`).

### *Examples*

```
A, B = 1.5;                     /* several targets             */
MONTHLY = 0;                    /* scalar to every element     */
OUT_REC = '';                   /* every CHAR blank, numbers 0 */
OUT_REC = IN_REC, BY NAME;      /* members with matching names */
SUBSTR(LINE, 5, 5) = 'TOTAL';   /* SUBSTR pseudovariable       */
```

## CALL

### *Syntax*

```
CALL procedure-name [ ( argument , ... ) ] ;
CALL ABEND ( code ) ;
```

### *Description*

`CALL` invokes an internal procedure, or a PL/I subprogram declared with `ENTRY ... EXTERNAL` ("External Entries"). A procedure that has `RETURNS` is a function and cannot be called (`CALL OF FUNCTION` *name*). `CALL ABEND` ends the task with an abend; see "CALL ABEND" in Chapter 33.

Arguments are passed by reference. If an argument is an expression or a constant, or its attributes differ from those of the parameter, a *dummy argument* (a temporary copy) is passed instead, and the changes that the procedure makes to it are not seen by the caller. A parameter declared `CHAR(*)` or `CHAR(*) VAR` accepts strings of any length by reference, as the procedure `VALIDATE_KEY` in `cusp.pli` does to normalize the caller's key in place.

### *Example*

```
DCL N  FIXED BIN(31)  INIT(1);
DCL D  FIXED DEC(5,0) INIT(1);
CALL BUMP(N);              /* same attributes: by reference   */
CALL BUMP(D);              /* DEC vs BIN: a dummy is passed    */
CALL BUMP(N + 0);          /* an expression: a dummy           */
PUT SKIP LIST(N, D);       /* 11 1                             */

BUMP: PROC(X);
   DCL X FIXED BIN(31);
   X = X + 10;
END BUMP;
```

### *Differences from IBM Enterprise PL/I*

`CALL` reaches PL/I procedures only. To invoke a REXX or COBOL program, or a PL/I `OPTIONS(MAIN)` program, use `EXEC CICS LINK`. There are no `ENTRY` variables and no generic entries. At most 100 calls may be active in a task (`ABEN` beyond that).

## DO

***Syntax***

```
[ label: ] DO ;
[ label: ] DO variable = start [ TO limit ] [ BY step ]
              [ WHILE ( condition ) ] [ UNTIL ( condition ) ] ;
[ label: ] DO [ WHILE ( condition ) ] [ UNTIL ( condition ) ] ;
[ label: ] DO { FOREVER | LOOP } ;
   statements
END [ label ] ;
```

***Description***

- `DO; ... END;` is both the block construct and the loop construct. There are no `BEGIN` blocks.
- `TO` and `BY` are evaluated once, on entry to the loop. `BY` may be written before `TO`. `DO I = X;`, with no `TO`, `BY`, `WHILE` or `UNTIL`, runs once.
- The control variable must be a numeric variable (`DO CONTROL MUST BE A NUMERIC VARIABLE`). It is reloaded on every iteration; do not change it in the body. An overflow while it is incremented abends `ABSZ`.
- `WHILE` is tested before each iteration, `UNTIL` after the body. The two may be written in either order. `END` *name* must match the label of the `DO` (`END` *name* `DOES NOT MATCH`).

***Examples***

```
DO I = 1 TO N;                ... END;
DO I = N TO 1 BY -1;          ... END;
DO I = 1 TO N WHILE(¬FOUND); ... END;
DO WHILE(MORE);               ... END;
DO UNTIL(DONE);               ... END;   /* tested after the body */
DO FOREVER;                   ... END;   /* DO LOOP also accepted */
```

A loop that runs for 2 seconds without giving control back to the region is ended by the runaway task check with abend `AICA` ("Runaway Tasks" in Chapter 16).

## External Entries

***Syntax***

```
DCL name ENTRY [ ( descriptor , ... ) ]
         [ RETURNS ( attributes ) ] EXTERNAL ;
```

***Description***

A call to a separately stored PL/I subprogram is declared with `ENTRY ... EXTERNAL` (an `ENTRY` without `EXTERNAL` is `ENTRY` *name* `NEEDS EXTERNAL`). The subprogram is an external procedure without `OPTIONS(MAIN)`.

- The entry name is resolved as the member *name*`.pli` in `pli_dir` (not case-sensitive), through the program cache. The subprogram needs no `transactions.conf` entry.
- The subprogram runs in the caller's task and unit of work, and shares the task's `STATIC` storage.
- If the member does not exist, is an `OPTIONS(MAIN)` program, or has a parameter list or `RETURNS` attribute that does not match the `ENTRY` declaration, the call abends `ABEN`.

***Example***

```
ORDTOT: PROC OPTIONS(MAIN);
   DCL CALCDISC ENTRY(FIXED DEC(13,2), CHAR(8))
               RETURNS(FIXED DEC(13,2)) EXTERNAL;
   DCL GROSS FIXED DEC(13,2) INIT(250.00);
   DCL CUST  CHAR(8)         INIT('C0042');
   GROSS = GROSS - CALCDISC(GROSS, CUST);
   PUT SKIP LIST('NET', GROSS);
END ORDTOT;
```

```
/* CALCDISC - external subprogram (no OPTIONS(MAIN)) */
CALCDISC: PROC(AMT, CUST) RETURNS(FIXED DEC(13,2));
   DCL AMT  FIXED DEC(13,2);
   DCL CUST CHAR(8);
   IF SUBSTR(CUST, 1, 1) = 'C' THEN RETURN(ROUND(AMT * 0.10, 2));
   RETURN(0);
END CALCDISC;
```

## IF

***Syntax***

```
IF condition THEN statement [ ELSE statement ]
```

***Description***

The condition must be a bit expression; a bit string that is not all zeros is true. Either unit may be a `DO` group. An `ELSE` belongs to the nearest unmatched `IF`.

***Example***

```
IF BAL < 0 THEN FLAG = '1'B;
ELSE DO;
   ...
END;
```

### LEAVE and ITERATE

***Syntax***

```
LEAVE [ label ] ;
ITERATE [ label ] ;
```

***Description***

`LEAVE` ends a `DO` group or loop; `ITERATE` ends the current iteration of a loop.

- Without a label, both statements apply to the innermost *iterative* `DO` (one with a control variable, `WHILE`, `UNTIL`, `FOREVER` or `LOOP`), never to an enclosing simple `DO;` group. Thus `IF X THEN DO; ...; LEAVE; END;` inside a loop leaves the loop. Outside any loop the statement is the compile error `LEAVE OUTSIDE LOOP` (or `ITERATE OUTSIDE LOOP`).
- A labeled `LEAVE` may leave a simple group. A labeled `ITERATE` must name a loop (`ITERATE OF NON-LOOP` *label*). A label that does not name an enclosing `DO` is `LEAVE TARGET` *label* `NOT FOUND`.

***Example***

```
OUTER: DO I = 1 TO 3;
   DO J = 1 TO 3;
      IF J = 2 THEN ITERATE;
      IF I = 3 THEN LEAVE OUTER;
      S = S || CHAR(I) || CHAR(J) || ' ';
   END;
END OUTER;
```

`S` ends as `'11 13 21 23 '`.

### The Null Statement

A semicolon by itself is a null statement. It is used where a statement is required and no action is intended, for example `OTHERWISE;`.

## PUT LIST

### *Syntax*

```
PUT [ SKIP [ ( n ) ] ] [ PAGE ] LIST ( expression , ... ) ;
```

### *Description*

`PUT LIST` is accepted and its items are evaluated, but the line is discarded, in the same way as REXX `SAY` and COBOL `DISPLAY` in a running transaction: it reaches neither the terminal nor the bricks log. (At high transaction rates every line would otherwise reach the operator console.) The line that the statement forms separates items by a blank, and shows strings without quotes, numbers in minimal form and bits as `0` and `1`:

```
PUT SKIP LIST('READ', KEY, 'BAL', BAL, OK);
```

would form `READ C0042 BAL -12.50 1`. The examples in Part 7 quote `PUT LIST` lines in this way, to show values. To see values while debugging a transaction, use CEDF (Chapter 8), display them with `EXEC CICS SEND TEXT`, or write them to a temporary storage queue (`EXEC CICS WRITEQ TS QUEUE('DEBUG') FROM(LINE)`) and browse it with CECI.

### *Differences from IBM Enterprise PL/I*

Stream I/O (`GET`, `PUT EDIT`, `PUT DATA`, `FORMAT`) and record I/O are not available. `PUT EDIT`, `PUT DATA`, `PUT FILE` and `PUT STRING` are compile errors (`PUT EDIT NOT SUPPORTED`). The terminal is reached through `EXEC CICS SEND` and files through `EXEC CICS READ` and `WRITE`.

## RETURN

### *Syntax*

```
RETURN [ ( expression ) ] ;
```

### *Description*

`RETURN` without a value returns from a subroutine. In the `OPTIONS(MAIN)` procedure it ends the program: the transaction ends normally and the unit of work is committed, or, in a program that was the target of `LINK`, control returns to the LINKing program. `RETURN(`*value*`)` returns from a function procedure; the value is converted to the `RETURNS` attributes. A value in a subroutine is the compile error `RETURN VALUE IN SUBROUTINE`; a function without a value is `RETURN VALUE REQUIRED`.

***Examples***

```
CALL VALIDATE(ORDER, RC);
RETURN;                     /* from a subroutine, or from MAIN  */
RETURN (TOTAL);             /* from a function                  */
```

## SELECT

***Syntax***

```
SELECT [ ( expression ) ] ;
   WHEN ( value [ , value ] ... ) statement
   ...
   [ { OTHERWISE | OTHER } statement ]
END ;
```

***Description***

With an expression, each `WHEN` compares the expression with its values; without one, each `WHEN` value is a condition. The first matching `WHEN` is executed. If no `WHEN` matches and there is no `OTHERWISE`, the task abends `ABSL`. Write `OTHERWISE;` when no action is intended.

***Example***

```
SELECT (SHIP);
   WHEN ('EXP')        DAYS_ = 1;
   WHEN ('STD', 'GRD') DAYS_ = 5;
   OTHERWISE           DAYS_ = 0;
END;
SELECT;
   WHEN (DAYS_ > 3) PUT SKIP LIST('slow', DAYS_);
   WHEN (DAYS_ > 0) PUT SKIP LIST('fast', DAYS_);
   OTHERWISE;
END;
```

(`DAYS` is the name of a built-in function and cannot name a level-1 variable, hence `DAYS_`.)

### Differences from IBM Enterprise PL/I

- No `GO TO`, no statement labels other than on `PROC` and `DO`, no label variables.
- No `BEGIN` blocks; use `DO` groups and internal procedures.
- No `ON`, `SIGNAL` or `REVERT` (`ON NOT SUPPORTED`); see "Runtime Abends" in Chapter 33.
- No `ALLOCATE`, `FREE` or `GET` statements.

**Chapter 32**

# PL/I Built-in Functions

The built-in functions described in this chapter are available without a declaration. Arguments in brackets are optional. A built-in function name cannot be declared as a level-1 variable or as a procedure (`BUILTIN NAME`), but may name a structure member at level 2 or deeper; see "Source Rules" in Chapter 28. A wrong number of arguments is the compile error `WRONG NUMBER OF ARGUMENTS FOR` *name*.

The spellings `CHARACTER`, `DECIMAL`, `BINARY`, `STORAGE` and `CURRENTSTORAGE` are accepted for `CHAR`, `DEC`, `BIN`, `STG` and `CSTG`. Functions that do not depend on the clock or on the contents of storage are evaluated when the program is compiled if their arguments are constants.

## String Functions

| Function | Result |
|---|---|
| SUBSTR(s, i [, n]) | Substring of *n* characters starting at position *i* (to the end if *n* is omitted). Also an assignment target (pseudovariable). |
| INDEX(s, t [, start]) | Position of *t* in *s*, or 0 |
| LENGTH(s) | Current length |
| MAXLENGTH(s) | Declared length of a VAR string |
| TRIM(s [, lead [, trail]]) | Blanks removed from both ends. TRIM(s, lead) removes the characters of *lead* on the left and blanks on the right; TRIM(s, lead, trail) uses both sets. |
| LEFT(s, n [, pad]) | *s* left-justified in *n* characters |
| RIGHT(s, n [, pad]) | *s* right-justified in *n* characters |
| COPY(s, n) | *s* repeated *n* times |
| TRANSLATE(s, to [, from]) | Character mapping. The first occurrence of a character in *from* wins; *to* is padded with blanks. Without *from*, the 256 byte values in order are mapped. |
| VERIFY(s, set [, start]) | Position of the first character of *s* that is not in *set*, or 0 |
| SEARCH(s, set [, start]) | Position of the first character of *s* that is in *set*, or 0 |
| UPPERCASE(s)<br>LOWERCASE(s) | Changes only a–z or A–Z, as in Enterprise PL/I |
| REVERSE(s) | The characters in reverse order |

**Figure 32-1. String built-in functions**

### *Usage notes*

- A SUBSTR position or length outside 1 .. LENGTH(s)+1 abends ABSR. The check cannot overflow: SUBSTR(s, 2, HUGE(n)) also abends ABSR. SUBSTR(s, LENGTH(s)+1) and SUBSTR(s, LENGTH(s)+1, 0) are ''.
- INDEX, VERIFY and SEARCH accept a start position from 1 to LENGTH(s)+1.
- A COPY result longer than 32767 characters abends ABSR.

***Example***

```
T2 = TRIM(S);                        /* 'Hello, World'          */
PUT SKIP LIST(INDEX(T2, 'o'), INDEX(T2, 'o', 6));    /* 5 9    */
PUT SKIP LIST(LEFT('AB', 5, '*'), RIGHT('AB', 5, '0'));
PUT SKIP LIST(TRANSLATE('A-B-C', ' ', '-'), TRIM('000120', '0'));
```

The last two statements form the lines `AB*** 000AB` and `A B C 120`.

## Arithmetic Functions

| Function | Result |
|---|---|
| `ABS(x)` | Absolute value |
| `SIGN(x)` | -1, 0 or 1 |
| `MOD(x, y)` | Remainder, always >= 0 (`MOD(-7, 3)` = 2) |
| `REM(x, y)` | Remainder with the sign of *x* (`REM(-7, 3)` = -1); exact for decimal operands: `x - y * TRUNC(x / y)` |
| `MAX(x, y, ...)`<br>`MIN(x, y, ...)` | Largest or smallest argument |
| `ROUND(x, n)` | Rounded half away from zero to *n* decimal places |
| `TRUNC(x)`<br>`CEIL(x)`<br>`FLOOR(x)` | Integer toward zero, upward, downward |
| `ADD(x, y, p, q)`<br>`MULTIPLY(x, y, p, q)`<br>`DIVIDE(x, y, p, q)` | Result computed at the explicit precision (p,q) |
| `HUGE(x)`<br>`TINY(x)` | Largest or smallest value of the type of *x* (`TINY` is 0 for `UNSIGNED`) |

**Figure 32-2. Arithmetic built-in functions**

`MOD` and `REM` with a zero divisor abend `ABZD`. Precision and truncation rules are described in "Fixed-Decimal Precision" in Chapter 30.

## Conversion and Validation Functions

| Function | Result |
|---|---|
| `CHAR(x [, n])` | Minimal character form of a number, for every arithmetic type, `FIXED BIN` included (`CHAR(42)` is `'42'`, never `'  42'`). *n* pads or truncates the result to *n* characters. |
| `DEC(x, p [, q])`<br>`FIXED(x, p [, q])` | Fixed decimal at precision (p,q) |
| `BIN(x, p)` | Fixed binary. `BIN(b)` of a `BIT(64)` value reads it as two's complement; a shorter bit string is read as unsigned. |
| `BIT(x [, n])` | Bit string. Of a number: the rightmost *n* bits of the 64-bit two's-complement image of its integer part, so `BIN(BIT(x, 64), 63) = x`. Of a character variable without *n*: the declared length of the variable (at most 64). |
| `EDIT(x, 'picture')` | *x* formatted through a picture, as `CHAR` |
| `VALID(variable)` | `'1'B` if a picture or `FIXED DEC` variable holds data that is valid for its declaration |
| `STRING(structure)` | The `CHAR` and `PIC` members of the structure, concatenated |

**Figure 32-3. Conversion and validation built-in functions**

## Aggregate and Storage Functions

| Function | Result |
|---|---|
| `HBOUND(a [, d])`<br>`LBOUND(a [, d])`<br>`DIM(a [, d])` | Upper bound, lower bound and extent of dimension *d* (default 1) |
| `SUM(a)` | Sum of all elements of a numeric array |
| `STORAGE(x)`<br>`STG(x)` | Declared size in bytes (see "Storage Layout and COMMAREA Exchange with COBOL" in Chapter 29) |
| `CURRENTSTORAGE(x)`<br>`CSTG(x)` | Current size in bytes; for a VAR string, 2 + current length |

**Figure 32-4. Aggregate and storage built-in functions**

## Date and Time Functions

Date and time patterns are built from the fields `YYYY`, `YY`, `MM`, `DD`, `DDD`, `HH`, `MI`, `SS` and `999` (milliseconds), with literal separators between them. `YY` uses the century window 1950-2049; `DDD` is the day of the year.

| Function | Result |
|---|---|
| `DATETIME([pat])` | The current time, formatted by *pat*; default pattern `'YYYYMMDDHHMISS999'` |
| `DATE()` | `'YYMMDD'`. Kept for compatibility; use `DATETIME`. |
| `TIME()` | `'HHMISS999'` |
| `DAYS([d [, pat]])` | Date *d* as a Lilian day number (day 1 is 15 October 1582); default pattern `'YYYYMMDDHHMISS999'`. Without arguments, the day number of the current date. |
| `DAYSTODATE(n [, pat])` | Day number *n* as a date; default pattern `'YYYYMMDD'` |
| `VALIDDATE(d [, pat])` | `'1'B` if *d* is a real calendar date; default pattern `'YYYYMMDDHHMISS999'` |

**Figure 32-5. Date and time built-in functions**

- Day numbers run from 1 (15 October 1582) to 3074324 (31 December 9999). A day number outside that range, or a malformed date string, abends `ABCV`.
- `DDD` runs from 1 to 365 (366 in a leap year); `000` is not a date, so `VALIDDATE('2026000', 'YYYYDDD')` is `'0'B`.
- A letter or digit in a pattern that is not part of a field makes the pattern invalid: `ABCV` for `DATETIME`, `DAYS` and `DAYSTODATE`, `'0'B` for `VALIDDATE`.
- The clock is the region's NTP-corrected clock, the same one that `EXEC CICS ASKTIME` reads.

Date arithmetic is done with day numbers:

```
D = DAYSTODATE(DAYS('20260131', 'YYYYMMDD') + 30, 'YYYYMMDD');
```

gives `'20260302'`.

### CICS Constants and the ABEND Subroutine

| Name | Result |
|---|---|
| `DFHRESP(cond)` | The `EIBRESP` value of a condition, as a constant computed at compile time: `DFHRESP(NOTFND)` is 13. An unknown name is a compile error (`UNKNOWN CONDITION NOTFOUND`). |
| `DFHVALUE(cvda)` | A CVDA value, as a compile-time constant: `OPEN` 18, `CLOSED` 19, `ENABLED` 23, `DISABLED` 24, `READABLE` 50, `NOTREADABLE` 51, `UPDATABLE` 52, `NOTUPDATABLE` 53, `CTRLABLE` 54, `NOTCTRLABLE` 55, `ALTERABLE` 56, `NOTALTERABLE` 57. Any other name is `UNKNOWN CVDA`. |
| `CALL ABEND('code')` | Abends the task with the given code of 1 to 4 characters; see "CALL ABEND" in Chapter 33. `ABEND` is a subroutine and cannot be used in an expression (`ABEND IS A SUBROUTINE`). |

**Figure 32-6. CICS constant functions and the ABEND subroutine**

`DFHRESP` uses the same single table as the COBOL `DFHRESP(`*name*`)` function and the supplied members `DFHRESP.cpy` and `DFHRESP.inc`, so the numbers cannot differ between languages. The response codes are listed in Chapter 18.

### Differences from IBM Enterprise PL/I

- `CHAR(x)` of an arithmetic value is the minimal form, not a right-justified field.
- The Enterprise PL/I functions for FLOAT data, pointers and storage addresses (`ADDR`, `UNSPEC`, `ALLOCATION` and similar), conditions (`ONCODE`, `ONSOURCE`) and the many string and mathematical functions not listed in this chapter are not available. A name that is not a built-in function and is not declared is the compile error `UNDECLARED`.
- `DFHRESP` and `DFHVALUE` are built-in functions; no preprocessor is involved.

Chapter 33

# EXEC CICS in PL/I: Runtime Abends, the Unit of Work and %INCLUDE

This chapter describes how a PL/I program ends abnormally and how its unit of work is settled, how the program issues `EXEC CICS` and `EXEC SQL` commands, the names that every PL/I program has without declaring them (the EIB, the SQLCA and `DFHCOMMAREA`), and the `%INCLUDE` statement with the supplied members `DFHAID` and `DFHRESP`.

## Runtime Abends

Enterprise PL/I handles runtime conditions with ON-units (`ON CONVERSION`, `ON SIZE`, `SIGNAL`, `REVERT`). bricks PL/I has none: `ON` is the compile error `ON NOT SUPPORTED`. Instead, every runtime condition is deterministic and ends the task. When a condition occurs, bricks:

1. Abends the transaction with a four-character code.
2. Ends the program abnormally. The abend is not offered to this program's own `HANDLE ABEND` exit, but an exit of a caller up the `LINK` chain can catch it; if none does, the unit of work of the task is backed out.
3. Writes the program, procedure, file, source line and statement text to the bricks log.
4. Takes a transaction dump, which holds the procedure, the statement and the values of the variables ("Transaction Dumps" in Chapter 16). CAID ("CAID: The Abend Analyser") explains most of these abends from the dump; for `ZERODIVIDE` it names the divisor that was zero. The sample ABZP (`runtime/pli/abzp.pli`) abends `ABZD` this way.

The terminal operator receives the IBM message DFHAC2206, with the program, procedure and line on a separate detail row. For a statement in the MAIN procedure itself the procedure name is omitted (`CUSP L42`).

```
DFHAC2206 14:02:11 BRICKS01 Transaction CUSP failed with abend ABSZ.
  Updates to local recoverable resources backed out.

Detail: CUSP.ADD_CUSTOMER L42
```

The bricks log holds the full detail on one line (shown here on two):

```
term=T0001 transid=CALC: PL/I ABEND ABZD (ZERODIVIDE) PROGRAM CALCP
  PROC TOTALS FILE calcp.pli LINE 6: ZERODIVIDE | X = 10 / Y ;
```

### *Abend codes*

The following table lists the abend codes that the PL/I interpreter itself raises, followed by the codes that CICS raises for a PL/I task. All transaction abend codes are listed in Appendix C.

| Code | Condition | Raised when |
|---|---|---|
| ABCV | CONVERSION | A character string that is not valid is converted to a number or a bit string; a bad date or date pattern |
| ABSZ | SIZE | The integer part does not fit the target of an assignment; a negative value is assigned to an UNSIGNED target or a picture without a sign; a DO control variable overflows. Always enabled. |
| ABF0 | FIXEDOVERFLOW | An intermediate result needs more than 31 integer digits; a 64-bit binary operation overflows |
| ABZD | ZERODIVIDE | Division, MOD or REM by zero |
| ABSR | SUBSCRIPTRANGE, STRINGRANGE | A subscript or SUBSTR position is out of bounds; a concatenation or COPY result is longer than 32767 characters. Always checked. |
| ABSL | SELECT | No WHEN matched and there is no OTHERWISE |
| ABEN | ENTRY | An external entry is missing or does not match its ENTRY declaration; a procedure that is not RECURSIVE is entered again; more than 100 calls are active in the task (procedures of LINKed PL/I programs included); a function ends without RETURN(*value*); CALL ABEND with a variable that holds a code that is not valid |
| ABHB | HANDLER | A CICS condition, AID or abend handler fired while a handler procedure was already running, or named a procedure that is not a handler of this program |
| ABST | STORAGE | The live activations of the task would hold more than 8 MiB |

| Code | Condition | Raised when |
|---|---|---|
| AEY9 | CICS NOT AVAILABLE | EXEC CICS was executed with no CICS handler attached (offline test harness only) |
| APLI | INTERNAL | The interpreter caught an internal failure. The operator receives the code; the detail is written to the bricks log only. |
| AICA | RUNAWAY | The task ran for 2 seconds without giving control back to the region ("Runaway Tasks" in Chapter 16) |
| AEI*x*, AEY*x*, AEX*x* | Unhandled CICS condition | A command returned a condition (NOTFND: AEIM, MAPFAIL: AEI9, ...) and the program had no RESP, NOHANDLE, HANDLE CONDITION or IGNORE CONDITION for it ("Unhandled Conditions") |
| ABM0, ABMB, AEXZ, ATNI, APCT, ASP3, ASP7, ASPR, ADBU, AD2S, AD2P | Raised by CICS | CICS could not carry out a command or a syncpoint: map not in mapset, cursor beyond the screen, internal failure in a command, terminal lost, abend exit program not loadable, commit refused by the database, database error in a statement, database sign-on failed, no database connection free |
| *any* | Program-raised | CALL ABEND('CU01') or EXEC CICS ABEND ABCODE('CU01'): the given code |

**Figure 33-1. Abend codes of a PL/I task**

## CALL ABEND

### *Syntax*

```
CALL ABEND ( code ) ;
```

***Description***

`CALL ABEND` abends the task with exactly the given code, for a deliberate business-rule abort. The unit of work is backed out. The code is 1 to 4 printable ASCII characters that also exist in EBCDIC code page 037 (no brackets, braces, tilde, backslash, grave accent, vertical bar or caret). A constant that breaks the rule is a compile error (`BAD ABEND CODE`, or `ABEND CODE MUST BE 1 TO 4 CHARACTERS`); a variable that holds such a code abends `ABEN`.

***Example***

```
POST: PROC(B);
   DCL B FIXED DEC(9,2);
   IF B < 0 THEN CALL ABEND('CU01');
END POST;
```

This ends the task with abend `CU01` and the detail `T.POST L7` (program `T`, procedure `POST`, line 7).

## Errors Detected at Compile Time

Some errors that Enterprise PL/I reports late, or not at all, are compile errors in bricks, so that they are reported by `brickscompile`, the SYNTAX column of CEDA and CEMT, and the ISPF save check:

| Message | Cause |
|---|---|
| `STORAGE LIMIT EXCEEDED` | The program declares more than 1 MiB of `AUTOMATIC` (all procedures together) plus `STATIC` storage |
| `NESTING TOO DEEP` | More than 256 nested parentheses, prefix operators, statements (`IF`, `DO`, `SELECT`) or procedures, or an expression tree deeper than 4096 (long chains of `+`, `\|\|` or `&` count) |
| `UNKNOWN CICS COMMAND` *x* | `EXEC CICS` with a verb that bricks does not dispatch, for example the misspelling `SENDD`; reported at compile time instead of `INVREQ` at runtime |
| `BAD ABEND CODE` | A `CALL ABEND` constant that is not EBCDIC 037 printable |
| `INIT VALUE TOO LONG FOR` *x* | A string repetition longer than the target (`DCL R CHAR(4) INIT((3)'AB')`) |
| `TOO MANY INIT VALUES` | `INIT` values (iteration factors included) for more elements than the array has |

**Figure 33-2. Errors that bricks detects at compile time**

A `CHAR` value assigned to a shorter target is truncated silently, as in Enterprise PL/I with STRINGSIZE disabled; this is not an error.

## The Unit of Work

The unit of work of a PL/I task contains its database updates, its recoverable file and temporary storage updates, its enqueues and its record locks. It is settled as shown in the following table.

| Task end | Unit of work |
|---|---|
| RETURN from MAIN, or reaching its END | Committed |
| EXEC CICS RETURN, with or without TRANSID | Committed |
| EXEC CICS XCTL | Carried into the target program: the database transaction, the file and temporary storage journal, the enqueues and the record locks. It is committed when the target ends normally and backed out when it abends. If the target is bound to a different database, the unit of work is committed at the XCTL (an implicit syncpoint); the same happens when the target is a supplied transaction or is routed to another region, and the locks are then released. |
| A handler procedure that returns ("HANDLE CONDITION, HANDLE AID and HANDLE ABEND") | Committed |
| Any abend: a runtime condition, an unhandled CICS condition, CALL ABEND, EXEC CICS ABEND, a failure of the interpreter | Backed out |

**Figure 33-3. How the unit of work of a PL/I task is settled**

EXEC CICS SYNCPOINT and SYNCPOINT ROLLBACK work as in the other languages. Record locks are described in "Record Locks" in Chapter 14. A commit that the database refuses, at a syncpoint, RETURN, XCTL or task end, abends the task ASP3, ASP7 or ASPR. See Chapter 16 for units of work and recovery, and Chapter 12 for XCTL and RETURN.

EIBABCODE (CHAR(4)) holds the abend code for ASSIGN ABCODE and for a HANDLE ABEND PROGRAM exit.

## HANDLE ABEND in PL/I

The abend exit of a PL/I program is EXEC CICS HANDLE ABEND PROGRAM(*name*). When the task abends through CICS (EXEC CICS ABEND, a runaway AICA, an unhandled CICS condition such as AEIM, an abend in a LINKed program), the named program is LINKed to, and when it returns the abend completes.

`HANDLE ABEND LABEL` is a compile error in bricks PL/I (`HANDLE ABEND LABEL NOT SUPPORTED IN PL/I`); use `PROGRAM`. The PL/I runtime conditions of "Runtime Abends" (`ABSZ`, `ABZD` and the others) do not drive the program's own `HANDLE ABEND` exit; a COBOL, REXX or PL/I program that LINKed to it can catch them with its own exit (see "HANDLE ABEND" in Chapter 16).

### Preventing Conditions Instead of Handling Them

ON-units make every statement a possible jump target, which is hard to read and easy to get wrong in money code (`ON SIZE`, disabled by default, loses digits silently). bricks programs prevent conditions instead of catching them:

| Instead of | Write |
|---|---|
| `ON CONVERSION` | `VERIFY(s, '0123456789')`, `VALID(picvar)`, `VALIDDATE(d)` before converting |
| `ON SIZE` | A target wide enough, or `ROUND`, `TRUNC` or `DIVIDE(x, y, p, q)` to the intended precision |
| `ON ZERODIVIDE` | `IF Y ¬= 0 THEN ...` |
| `ON SUBSCRIPTRANGE` | `HBOUND`, `LBOUND` or `LENGTH` checks |
| `ON ERROR around EXEC CICS` | `RESP(RC)` on the command, and a test of `RC` ("RESP, RESP2 and DFHRESP") |

**Figure 33-4. Replacements for ON-units**

## EXEC CICS

### *Syntax*

```
EXEC CICS command [ option [ ( argument ) ] ] ... ;
```

### *Description*

PL/I uses the IBM Enterprise PL/I form: `EXEC CICS`, the command, and a semicolon. There is no `END-EXEC`.

- Every command, option and clause that IBM documents is accepted with the same spelling and meaning as in COBOL and REXX. The command set is described in Part 3 and Part 4; options that bricks cannot act on are accepted and ignored.
- The statement extends to the first semicolon at parenthesis depth 0 outside quotes, so a command may span several lines (unlike the one-line rule for REXX).
- Arguments are plain PL/I references with no `:` prefix: `INTO(ORDER_IN)`, `RIDFLD(CUST.KEY)`, `FROM(LST.ROW(3))`.
- A verb that bricks does not dispatch is a compile error: `EXEC CICS SENDD MAP('X');` fails with `UNKNOWN CICS COMMAND SENDD`.

***Examples***

```
EXEC CICS RECEIVE MAP('ORDMAP') MAPSET('ORDSET') INTO(ORDER_IN)
          RESP(RC);
EXEC CICS SEND MAP('ORDMAP') FROM(ORDER_OUT) ERASE;
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(KEY) RESP(RC);
EXEC CICS LINK PROGRAM('CALC') COMMAREA(CALC_AREA);
EXEC CICS RETURN TRANSID('ORDR') COMMAREA(STATE);
```

### Options Are PL/I Expressions

Option arguments are PL/I expressions: `RIDFLD(KEY)`, `LENGTH(STG(REC))`, `ITEM(I + 1)`, `MAP('CUST' || SUFFIX)`. The compiler rewrites each command once, when the program is compiled:

- An argument that folds to a constant is placed in the command text: `MAP('CU' || 'ST1')` reaches CICS as `MAP('CUST1')`, and `LENGTH(STG(REC))` as `LENGTH(25)`. `DFHRESP`, `DFHVALUE` and `VALUE` names are folded too.
- Every other reference becomes a compiler-generated alias that is bound to its storage each time the command is executed: `READ FILE('F') INTO(_1_REC) RIDFLD(_2_KEY) RESP(_3_RC)`. A computed argument is bound as *_n_*. These names appear in error messages and in CEDF; they are the reason that names of the form *_digits_* are reserved.
- Qualified names (with the usual partial qualification) and subscripted elements are allowed wherever an area is expected. The area must be contiguous storage (`NONCONTIGUOUS ARGUMENT FOR` *option* otherwise).
- The CICS handler reads a fixed-length `CHAR` value without its trailing blanks, as for COBOL. Data areas (`COMMAREA`, `FROM` of `LINK`) are passed with all their bytes, packed and binary fields included.
- Data moved into a structure or array (`INTO`, `COMMAREA`) overlays its first bytes. When the data is shorter (a `READ ... INTO` of a short record), the bytes beyond its end keep their previous value, as in IBM CICS. Clear the area first (`REC = '';`) when stale bytes matter.

### RESP, RESP2 and DFHRESP

- `RESP(`*var*`)` and `RESP2(`*var*`)` are accepted on every command. After the command, *var* holds the response; `FIXED BIN(31)` is the natural type. PL/I has no predeclared `RC`: declare the variable that you name.
- `DFHRESP(`*cond*`)` is a compile-time constant and needs no include. The `DFHRESP` member adds the same values as named constants (`RESP_NOTFND` and `DFHRESP_NOTFND`; see "%INCLUDE").
- Coding `RESP` or `NOHANDLE` makes the conditions of the command non-fatal, and suppresses `HANDLE CONDITION` and `HANDLE AID` for that one command, as in IBM CICS.
- What the runtime writes to `EIBRESP`, `EIBRESP2` or a `RESP` target never changes any other program variable.

***Example***

```
T: PROC OPTIONS(MAIN);
   %INCLUDE DFHRESP;
   DCL REC  CHAR(200) VAR;
   DCL KEY  CHAR(8) INIT('C0042');
   DCL MSG  CHAR(60) VAR;
   DCL (RC, RC2) FIXED BIN(31);

   EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(KEY)
             RESP(RC) RESP2(RC2);
   SELECT (RC);
      WHEN (DFHRESP(NORMAL)) MSG = 'Found.';
      WHEN (DFHRESP(NOTFND)) MSG = 'Customer ' || KEY || ' not found.';
      OTHERWISE MSG = 'READ failed RESP=' || CHAR(RC)
                    || ' RESP2=' || CHAR(RC2);
   END;
   IF RC = RESP_NOTFND | RC = DFHRESP_NOTFND THEN PUT SKIP LIST(MSG);

   EXEC CICS DELETEQ TS QUEUE('SCRATCH') NOHANDLE;
   IF EIBRESP ¬= DFHRESP(NORMAL) THEN PUT SKIP LIST('EIBRESP', EIBRESP);
END T;
```

### Unhandled Conditions

An unhandled CICS condition abends a PL/I task, as on IBM CICS. A response other than `NORMAL` (`NOTFND`, `DUPREC`, `MAPFAIL`, `PGMIDERR`, ...) on a command that has no `RESP` and no `NOHANDLE`, for which no `HANDLE CONDITION` (or `HANDLE CONDITION ERROR`) and no `IGNORE CONDITION` is in effect, abends the task with the IBM abend code of the condition: `NOTFND` gives `AEIM`, `MAPFAIL` gives `AEI9`, `PGMIDERR` gives `AEI0`. A `HANDLE ABEND PROGRAM` exit, if one is active (here or in a caller), is driven first; when it returns the abend completes and the unit of work is backed out. The general rules are given in "Unhandled Conditions" in Chapter 16. The conditions whose IBM default action is to continue (`ENQBUSY`, `EOC`, `QBUSY` and a few others) never abend; `EIBRESP` is set and the program continues. The complete list of codes is in Appendix C.

Code `RESP(RC)` on every command whose response the program tests, as `cusp.pli` does. The supplied sample `ABNP` (`runtime/pli/abnp.pli`) issues a `READ` for a key that is not on file, with no `RESP`, and abends `AEIM`.

Some outcomes are not CICS conditions. They end the program, or the task, whatever the program codes; `RESP` and `NOHANDLE` do not suppress them:

- The terminal connection is lost during SEND or RECEIVE (abend ATNI).
- EXEC CICS RETURN, XCTL and ABEND.
- A HANDLE CONDITION or HANDLE AID branch that fires (when the command codes neither RESP nor NOHANDLE).
- A LINKed program that fails while it runs (an abend or a runtime error). A missing LINK target is PGMIDERR, an ordinary condition.
- A blocking SEND or RECEIVE in a background task started by the scheduler.

**Note:**

COBOL behaves in the same way. REXX differs, as IBM CICS REXX does: an unhandled condition never abends a REXX task; RC and EIBRESP are set and the program continues ("Unhandled CICS Conditions" in Chapter 39).

### The EIB and Other Predeclared Names

Every PL/I program has the following names without declaring them.

**DFHEIBLK, the EIB.** bricks PL/I declares the following subset of the IBM DFHEIBLK fields. The fields are referenced without qualification.

| Field | PL/I type | Meaning |
|---|---|---|
| EIBTIME | FIXED DEC(7) | Task start time, 0HHMMSS |
| EIBDATE | FIXED DEC(7) | Task start date, 0CYYDDD |
| EIBTRNID | CHAR(4) | Transaction identifier |
| EIBTASKN | FIXED DEC(7) | Task number |
| EIBTRMID | CHAR(4) | Terminal identifier, 4 characters as in IBM DFHEIBLK and the COBOL EIB. bricks terminal identifiers have 5 characters (T0001), so EIBTRMID holds T000; use EXEC CICS ASSIGN TERMID(*x*) into a CHAR(8) variable for the full identifier. |
| EIBCPOSN | FIXED BIN(15) | Cursor offset (zero-based) at the last terminal input |
| EIBCALEN | FIXED BIN(15) | Length of the COMMAREA passed to the program |
| EIBAID | CHAR(1) | AID byte of the last terminal input; compare it with the DFHAID names |
| EIBFN | CHAR(2) | Function code of the last command |
| EIBRESP | FIXED BIN(31) | Response of the last EXEC CICS command |
| EIBRESP2 | FIXED BIN(31) | Response detail (RESP2) |
| EIBABCODE | CHAR(4) | Abend code (HANDLE ABEND, ASSIGN ABCODE) |

**Figure 33-5. EIB fields declared in a PL/I program**

The EIB is a real data structure in PL/I: EIBTRNID, EIBTASKN, EIBDATE, EIBTIME, EIBTRMID and EIBCALEN are set when the program starts, not only by ASSIGN. Other IBM EIB fields (EIBDS, EIBRSRCE, EIBRCODE and the rest) are not declared; a reference to one is UNDECLARED. The meaning of every field is described in "EIB Fields" in Chapter 17.

**SQLCA**, the SQL communication area: SQLCODE FIXED BIN(31), SQLERRMC CHAR(70) VAR, SQLERRP CHAR(8), SQLERRD(6) FIXED BIN(31) (SQLERRD(3) is the number of rows affected), the flags SQLWARN0 to SQLWARN9 and SQLWARNA (CHAR(1)), and SQLSTATE CHAR(5), as in the COBOL SQLCA ("The SQL Communication Area" in Chapter 21).

**DFHCOMMAREA**; see "COMMAREA and DFHCOMMAREA".

### Maps as Structures

`SEND MAP ... FROM(`*structure*`)` and `RECEIVE MAP ... INTO(`*structure*`)` move map fields to and from the members of a structure by name, in the same way as the COBOL group and the REXX stem (Chapter 3). Declare one member for each map field, with the length of the field. A map field name is looked up inside the structure in the following order:

1. An exact member path (`INFOLINE`, or a deeper qualified path).
2. A unique member of that name anywhere below the structure (not through an array).
3. A field named `ROW`*n* (as the fields `ROW1` to `ROW15` of map `CUSTL`) maps to element *n* of a unique one-dimensional array member `ROW`. The PL/I program declares `2 ROW(15) CHAR(71)` instead of fifteen scalars. The 35 rows of `CUSTLL` fit `2 ROW(35)`, and a map with fewer rows ignores the extra elements.

- **Color overrides** use the REXX spelling: a member *field*`_C` set to a color name (`MSG_C = 'YELLOW'`) changes the color of *field* for that `SEND` (see "Changing a Field's Color at Runtime" in Chapter 3).
- **Built-in names as members.** A member may be called `SEARCH`, `INDEX`, `LENGTH` and so on, because map fields often are. `MENU_IN.SEARCH` (or an unambiguous bare `SEARCH`) is the member; `SEARCH(s, set)` with arguments is still the built-in function.
- **Separate input and output copies.** `cusp.pli` receives into `MENU_IN LIKE MENU`, so that the next `SEND` of `MENU` never displays again what the operator typed.
- **Numeric input.** `RECEIVE MAP` into a numeric `PIC` member stores the characters as typed (zoned input), so that `VALID` can reject bad data. Into a `FIXED DEC` or `FIXED BIN` member the text is converted, and input that is not numeric abends `ABCV`. Receive numbers into `PIC` or `CHAR` members and validate them.
- **Maps for several screen sizes.** Choose the map name at runtime (`MAP(M_MENU)`, with `M_MENU = 'CUST1' || SUFFIX`), and fall back to another map when `SEND` returns `MAPFAIL` ("Adapting to the Terminal Size" in Chapter 3).

```
CUSPDEMO: PROC OPTIONS(MAIN);
   %INCLUDE DFHAID;
   DCL 1 MENU,                       /* map CUST1                 */
         2 INFOLINE CHAR(78),
         2 ACTION   CHAR(1),
         2 CUSTNO   CHAR(8),
         2 SEARCH   CHAR(30),        /* a builtin name: legal here */
         2 MSG      CHAR(60),
         2 MSG_C    CHAR(9);         /* runtime color override     */
   DCL 1 MENU_IN LIKE MENU;
   DCL 1 LST,                        /* map CUSTL: ROW1 .. ROW15   */
         2 INFOLINE CHAR(78),
         2 ROW(15)  CHAR(71);
   DCL RC FIXED BIN(31);
   DCL P  FIXED BIN(31);

   MENU = '';
   MENU.MSG   = 'Type an action and press ENTER.';
   MENU.MSG_C = 'YELLOW';
   EXEC CICS SEND MAP('CUST1') FROM(MENU) ERASE RESP(RC);
   MENU_IN = '';
   EXEC CICS RECEIVE MAP('CUST1') INTO(MENU_IN) RESP(RC);
   IF EIBAID = PF03 | RC = DFHRESP(MAPFAIL) THEN EXEC CICS RETURN;

   P = SEARCH(MENU_IN.CUSTNO, ' ');  /* with arguments: builtin    */
   LST = '';
   LST.INFOLINE = 'Search: ' || TRIM(MENU_IN.SEARCH);
   LST.ROW(1)   = 'first row';
   EXEC CICS SEND MAP('CUSTL') FROM(LST) ERASE RESP(RC);
   EXEC CICS RETURN;
END CUSPDEMO;
```

### COMMAREA and DFHCOMMAREA

`DFHCOMMAREA` holds the COMMAREA passed to the program by `RETURN TRANSID ... COMMAREA`, `XCTL` or `LINK`; `EIBCALEN` is its length.

- **Declared in MAIN.** If the MAIN procedure declares a variable named `DFHCOMMAREA` (usually a structure), the passed bytes are mapped onto it with the layout described in "Storage Layout and COMMAREA Exchange with COBOL" in Chapter 29, packed and binary members included. A shorter structure leaves the rest of a LINKing program's area unchanged.
- **Default.** Otherwise `DFHCOMMAREA` is predeclared as `CHAR(32767) VAR`, with a current length of `EIBCALEN`. Assigning to it in a LINKed program replaces the reply: a shorter value is padded with blanks, a longer one is truncated.

The reply to a LINK always has exactly the length of the caller's COMMAREA. XCTL ... COMMAREA(DFHCOMMAREA) passes on the area that the program received. A pseudo-conversational state block:

```
ORDC: PROC OPTIONS(MAIN);
   DCL 1 DFHCOMMAREA,
         2 STEP     CHAR(1),
         2 CUST_ID  CHAR(8),
         2 TOTAL    FIXED DEC(11,2);
   DCL RC FIXED BIN(31);

   IF EIBCALEN = 0 THEN DO;          /* first entry               */
      DFHCOMMAREA = '';
      DFHCOMMAREA.STEP = '1';
   END;
   ELSE DO;
      DFHCOMMAREA.TOTAL = DFHCOMMAREA.TOTAL + 10;
      DFHCOMMAREA.STEP  = '2';
   END;
   EXEC CICS RETURN TRANSID('ORDC') COMMAREA(DFHCOMMAREA)
             LENGTH(STG(DFHCOMMAREA)) RESP(RC);
END ORDC;
```

A LINK target that works on the default area:

```
CUSVP: PROC OPTIONS(MAIN);
   DCL KEY CHAR(8) VAR;
   IF EIBCALEN = 0 THEN RETURN;
   KEY = UPPERCASE(TRIM(SUBSTR(DFHCOMMAREA, 1, MIN(EIBCALEN, 8))));
   DFHCOMMAREA = KEY;                /* blank-padded to EIBCALEN  */
END CUSVP;
```

### LINK, XCTL and RETURN Across Languages

PL/I, REXX and COBOL programs LINK and XCTL to each other freely; the COMMAREA is a string of bytes. When a PL/I structure is on one side and a COBOL group on the other, give both the same byte layout ("Storage Layout and COMMAREA Exchange with COBOL" in Chapter 29):

```
DCL 1 CALC_AREA,
      2 CA_KEY      CHAR(8),
      2 CA_BALANCE  FIXED DEC(11,2),
      2 CA_COUNT    FIXED BIN(31),
      2 CA_RC       CHAR(2);
DCL RC FIXED BIN(31);
CALC_AREA = '';
CALC_AREA.CA_KEY = 'C0042';
EXEC CICS LINK PROGRAM('GUSV') COMMAREA(CALC_AREA)
          LENGTH(STG(CALC_AREA)) RESP(RC);
IF RC = DFHRESP(PGMIDERR) THEN PUT SKIP LIST('GUSV not available');
```

- A LINKed PL/I program runs in the caller's task and unit of work, and shares the task's `STATIC` storage with every other PL/I program of the task.
- A REXX target receives the COMMAREA as a string; a COBOL target maps it onto its `DFHCOMMAREA`. Changes are returned to the caller.
- An `XCTL` from a LINKed program, in any language, replaces the program at that `LINK` level; the `RETURN` of the target goes back to the LINKing program. An `XCTL` to a program that does not exist returns `PGMIDERR` at any level.
- Use `EXEC CICS LINK` to reach REXX, COBOL and other `OPTIONS(MAIN)` PL/I programs; use a `CALL` through `ENTRY ... EXTERNAL` for PL/I subprograms ("External Entries" in Chapter 31).
- PL/I transactions also run on an MRO peer region, as REXX and COBOL transactions do.

The rules for `RETURN TRANSID` at a `LINK` level and for `COMMAREA` at a `LINK` level are those of Chapter 12.

### START and RETRIEVE

`EXEC CICS START` schedules a PL/I transaction like any other. When the interval expires while the current task of the terminal is waiting for input, bricks runs the started task as a *background task* (REXX or PL/I programs only), independently of the scheduler and of other terminals (see `SEND MAP DATAONLY` in "SEND MAP" in Chapter 11). A background PL/I task:

- Has no COMMAREA (`EIBCALEN` is 0) and reads the `START` data with `RETRIEVE`.
- May use only commands that do not wait for the terminal (`SEND MAP ... DATAONLY`, file and temporary storage commands, `ASSIGN`, `START`).
- Has no SQL connection (`SQLCODE` -1).
- Commits its unit of work when it ends normally and backs it out when it abends, like a terminal task. The same holds for REXX background tasks.

```
TICK: PROC OPTIONS(MAIN);
   DCL BUF CHAR(4);
   DCL LEN FIXED BIN(15) INIT(4);
   DCL RC  FIXED BIN(31);
   DCL 1 CLK, 2 CLOCK CHAR(8);

   EXEC CICS RETRIEVE INTO(BUF) LENGTH(LEN) RESP(RC);
   IF RC = DFHRESP(NORMAL) & BUF = 'TICK' THEN DO;
      /* scheduler-fired: commands that do not wait only */
      CLK.CLOCK = DATETIME('HH:MI:SS');
      EXEC CICS SEND MAP('CLOCK1') FROM(CLK) DATAONLY RESP(RC);
   END;
   EXEC CICS START TRANSID('TICK') AFTER SECONDS(5) FROM('TICK')
             LENGTH(4) RESP(RC);
END TICK;
```

See "START" in Chapter 12 for `START`, `RETRIEVE` and `CANCEL`.

### HANDLE CONDITION, HANDLE AID and HANDLE ABEND

```
EXEC CICS HANDLE AID PF3(EXIT_PROC) CLEAR(EXIT_PROC);
EXEC CICS HANDLE CONDITION MAPFAIL(NO_INPUT);
EXEC CICS HANDLE ABEND PROGRAM('CLEANUP');
...
EXIT_PROC: PROC;
   EXEC CICS SEND TEXT FROM('Goodbye.') ERASE;
   EXEC CICS RETURN;
END EXIT_PROC;
```

The subset has no statement labels and no `GO TO`, so the target of a `HANDLE CONDITION` or `HANDLE AID` option is a procedure:

- It must be a parameterless internal procedure, without `RETURNS`, directly nested in MAIN. Anything else is a compile error (`HANDLER EXIT_PROC MUST BE A PARAMETERLESS PROC IN T`); an argument that is not a plain name is `HANDLER PROCEDURE NAME EXPECTED`.
- **Terminal-handler semantics.** When the handler fires, the current call stack is discarded and the handler procedure runs. When it returns, the task ends as if MAIN had executed `RETURN`, and the unit of work is committed. The handler may itself issue `EXEC CICS RETURN TRANSID(...)`, `XCTL` or `ABEND`, which then ends the task in that way.
- A handler that is triggered again while a handler procedure is running abends `ABHB`.
- `IGNORE CONDITION` and `HANDLE CONDITION` *cond* with no target (to cancel it) are supported. After an ignored condition, `EIBRESP` and `EIBRESP2` read 0, which differs from IBM CICS (see "IGNORE CONDITION" in Chapter 16). `PUSH HANDLE` and `POP HANDLE` are not supported ("Commands Not Implemented" in Chapter 19).
- `RESP` or `NOHANDLE` on a command suppresses `HANDLE CONDITION` and `HANDLE AID` for that command.
- `HANDLE ABEND` takes `PROGRAM(`*name*`)`, `CANCEL` or `RESET`; `LABEL` is a compile error ("HANDLE ABEND in PL/I").

**Note:**

Prefer `RESP(RC)` tests to `HANDLE CONDITION`. Because a handler ends the task, it suits paths that display a message and end (**`F3`**, **`CLEAR`**, a fatal `MAPFAIL`), not retry loops.

### Commands That Return Addresses

There are no pointers in the subset, so the CICS commands that return addresses are compile errors: `ADDRESS` (`ADDRESS NOT SUPPORTED (NO POINTERS)`), `GETMAIN`, `GETMAIN64`, `FREEMAIN`, `FREEMAIN64`, and every `SET(`*ptr*`)` option (`SET OPTION NOT SUPPORTED (NO POINTERS)`). Use the `INTO(`*area*`)` form of each command instead (`READ ... INTO`, `RECEIVE ... INTO`, `READQ TS ... INTO`).

## EXEC SQL in PL/I

***Syntax***

```
EXEC SQL sql-statement ;
```

***Description***

`EXEC SQL` is written like `EXEC CICS`, with the IBM PL/I `:` prefix on host variables.

```
EXEC SQL INCLUDE SQLCA;
EXEC SQL WHENEVER SQLERROR DO CALL DB_FAILED;

EXEC SQL SELECT name INTO :CUSTNM :NM_IND
         FROM customers_sql WHERE id = :CUSTID;
IF SQLCODE = 100 THEN PUT SKIP LIST('not found');
ELSE IF NM_IND < 0 THEN PUT SKIP LIST('name is NULL');
ELSE PUT SKIP LIST('customer', CUSTNM);
```

- The SQLCA is predeclared; `EXEC SQL INCLUDE SQLCA;` and `BEGIN DECLARE SECTION` / `END DECLARE SECTION` are accepted and do nothing.
- A structure host variable is expanded to its members: `FETCH C1 INTO :ROW` binds `:NAME, :CITY`. A host array is a compile error (`ARRAY HOST VARIABLE`).
- The host-variable scan skips quoted strings, `--` comments and PostgreSQL `::` casts, so `'a:b'` and `x::text` are not host variables. `OPEN` *c* binds the host variables of `DECLARE` *c* `CURSOR` wherever the `DECLARE` is in the program.
- `WHENEVER SQLERROR`, `SQLWARNING` or `NOT FOUND` accepts `CONTINUE`, `DO CALL` *proc* and `GO TO` *proc* (which is treated as `DO CALL`). The procedure follows the handler rules of "HANDLE CONDITION, HANDLE AID and HANDLE ABEND": parameterless, nested in MAIN, and the task ends when it returns.
- The statements, cursors, null indicators, `CONNECT TO`, the database abends and the `SQLCODE` values are those of Chapter 21.

## %INCLUDE

***Syntax***

```
%INCLUDE member-name ;
```

***Description***

`%INCLUDE` is the only preprocessor statement. It inserts the member *member-name*`.inc` from `pli_include_dir` (default `runtime/pliinc`; the file name is matched without regard to case) at token level, before the program is parsed.

- Member names are plain identifiers: no paths, no extension, no directory escape (`%INCLUDE ../DFHAID;` is `INCLUDE NAME EXPECTED`). A missing member is `INCLUDE` *name* `NOT FOUND`.
- Members may include other members up to 8 levels deep; a ninth level, or an include cycle, is a compile error.
- A compile error inside a member names the member and its own line: `DFHAID line 3 col 1: ...`.
- **Cache invalidation.** Every member that a program includes is a dependency of the program's cache entry. Editing or deleting a member causes every program that includes it to be compiled again on its next dispatch. (A COBOL program, by contrast, must itself be changed before an edited copybook takes effect; see "COPY Statement" in Chapter 26.)

***Example***

```
CUSTX: PROC OPTIONS(MAIN);
   %INCLUDE DFHAID;
   %INCLUDE DFHRESP;
   %INCLUDE CUSTREC;
```

A user member is ordinary PL/I text, typically declarations shared by the programs of one application:

```
/* CUSTREC - customer COMMAREA, shared by every CU* program. */
DCL 1 CUST_AREA,
      2 CA_KEY      CHAR(8),
      2 CA_BALANCE  FIXED DEC(11,2),
      2 CA_STATUS   CHAR(1);
```

***Differences from IBM Enterprise PL/I***

The macro preprocessor (`%IF`, `%DO`, `%DCL`, `%REPLACE`) is not available. Members are files in one directory, not members of a partitioned data set.

### Supplied Members: DFHAID and DFHRESP

Two members are supplied in `runtime/pliinc`. Like the COBOL copybooks, each declares every name twice with the same `VALUE`: a bricks mnemonic and the IBM-traditional alias. Either style may be used, or both.

```
DCL PF03     CHAR(1) VALUE('F3'X);           /* in DFHAID.inc  */
DCL DFHPF3   CHAR(1) VALUE('F3'X);
DCL RESP_NOTFND    FIXED BIN(31) VALUE(13);  /* in DFHRESP.inc */
DCL DFHRESP_NOTFND FIXED BIN(31) VALUE(13);
```

| Member | bricks mnemonic | IBM alias | Contents |
|---|---|---|---|
| DFHAID | ENTER, CLEAR, PA1 to PA3, PF01 to PF24 | DFHENTER, DFHCLEAR, DFHPA1 to DFHPA3, DFHPF1 to DFHPF24 | CHAR(1) AID bytes, compared with EIBAID |
| DFHRESP | RESP_NORMAL, RESP_NOTFND, ... | DFHRESP_NORMAL, DFHRESP_NOTFND, ... | FIXED BIN(31) response codes, compared with EIBRESP or a RESP variable |

**Figure 33-6. Supplied PL/I include members**

The byte values are those of the COBOL DFHAID copybook ("DFHAID" in Chapter 26); the response codes are those of DFHRESP.cpy, with _ in the PL/I names where COBOL has -. The DFHRESP(*cond*) built-in function gives the same numbers with no include at all.

**Note:**

ENTER, CLEAR, PF03 and the other mnemonics become names in the program once DFHAID is included. Do not declare variables with those names in the same program.

Chapter 34

# PL/I Restrictions and Deviations

This chapter summarizes where bricks PL/I differs from IBM Enterprise PL/I, the features that are not supported, the errors that bricks detects at compile time, and the limits of the implementation.

## Deviations from Enterprise PL/I

Each of the following is a deliberate choice that makes business code safer.

| Area | bricks behavior | Enterprise PL/I |
|---|---|---|
| Declarations | Every variable must be declared (`UNDECLARED` *name*) | Implicit declaration, usually FLOAT |
| `DEC`, `BIN` | `FIXED` is required | `DEC` or `BIN` alone means FLOAT |
| Reserved words | Statement keywords are reserved | No reserved words |
| Initial values | Every variable starts as blanks, 0, `'0'B` or empty | Undefined unless `INIT` |
| `STATIC` | Per task, shared by the activations of the task | Per program load |
| Storage | Always `UNALIGNED`, fixed byte layout | `ALIGNED` by default for some types |
| Decimal precision | Scale digits are dropped to fit 31 digits; overflow only on the integer part | FIXEDOVERFLOW on intermediate results (`25 + 1/3`) |
| `/` | Always decimal; binary operands are converted to decimal | Binary division for binary operands |
| Binary with decimal | The binary operand is converted to decimal | The decimal operand is converted to binary |
| Comparisons | Non-chaining | Chaining allowed |
| `&`, `\|` | Short-circuit for `BIT(1)` operands | Order of evaluation not guaranteed |
| `CHAR(x)` of a number | Minimal form (`'42'`, `'-3.00'`) | Right-justified with leading blanks |
| SIZE | Always enabled: `ABSZ` | Disabled by default |
| Runtime conditions | Deterministic abend and backout; no ON-units | ON-units, `SIGNAL`, `REVERT` |
| CICS handler targets | Parameterless procedures nested in MAIN; the task ends when the handler returns | Labels, with `GO TO` semantics |
| `HANDLE ABEND` | `PROGRAM` only; `LABEL` is a compile error | `PROGRAM` or `LABEL` |
| `PUT LIST` | Accepted; the output is discarded | Written to `SYSPRINT` |
| `FIXED BIN(64)` | Accepted as `FIXED BIN(63)` | An error without `UNSIGNED` |

**Figure 34-1. Deviations from Enterprise PL/I**

An unhandled CICS condition is *not* a deviation: as in IBM CICS, it abends the PL/I task with the IBM code of the condition (`AEIM` for `NOTFND`, and so on); see "Unhandled Conditions" in Chapter 33.

### Features Not Supported

| Feature | Reason, or what to use instead |
|---|---|
| `ON, SIGNAL, REVERT, ONCODE, ONSOURCE` | Validate data first; abends are deterministic ("Runtime Abends" in Chapter 33) |
| `GO TO, statement labels, label variables` | `LEAVE`, `ITERATE`, `RETURN`; handler targets are procedures |
| `BEGIN blocks` | `DO` groups and internal procedures |
| `FLOAT, COMPLEX, REAL` | Not suitable for money; use `FIXED DEC` |
| `GRAPHIC, WIDECHAR` | Program data is single-byte (bricks SBCS) |
| `POINTER, BASED, ALLOCATE, FREE, AREA, OFFSET, ADDR, UNSPEC` | No raw storage access; tasks remain isolated. The CICS commands `ADDRESS`, `GETMAIN`, `FREEMAIN` and `SET( )` options are compile errors. |
| `ALIGNED` | One byte layout for every type |
| `CONTROLLED, DEFINED, EXTERNAL variables` | Overlays and shared mutable state; use files, temporary storage queues or SQL |
| `ORDINAL, UNION, HANDLE, LABEL, FORMAT, FILE, CONDITION, TASK, EVENT` | Not needed in the transaction model |
| `Stream and record I/O (GET, PUT EDIT, PUT DATA, FORMAT, READ, WRITE, OPEN)` | `EXEC CICS` and `EXEC SQL`; `PUT LIST` is accepted and discarded, like REXX `SAY` |
| `%IF, %DO, %DCL, %REPLACE` | Only `%INCLUDE` |
| `Multitasking, ENTRY variables, GENERIC` | Not needed in the transaction model |
| `SQL host arrays` | One row per `FETCH` |
| `HANDLE ABEND LABEL` | `HANDLE ABEND PROGRAM` |

**Figure 34-2. PL/I features not supported**

An attribute that is not supported is the compile error *`attribute`* `NOT SUPPORTED`; an excluded statement (`ON`, `SIGNAL`, `REVERT`, `GO TO`, `BEGIN`, `ALLOCATE`, `FREE`, `GET`) is *`keyword`* `NOT SUPPORTED`.

### Errors Detected at Compile Time

`STORAGE LIMIT EXCEEDED`, `NESTING TOO DEEP`, `UNKNOWN CICS COMMAND` *x*, `BAD ABEND CODE`, `INIT VALUE TOO LONG` and `TOO MANY INIT VALUES` report at compile time what Enterprise PL/I reports late or not at all; see "Errors Detected at Compile Time" in Chapter 33. `HANDLE ABEND LABEL NOT SUPPORTED IN PL/I` and the `NO POINTERS` messages are also compile errors.

### Limits

| Item | Limit |
|---|---|
| Identifier length | 31 characters (`NAME TOO LONG`) |
| `FIXED DEC` precision | 31 digits |
| `FIXED BIN` precision | 63 bits signed, 64 bits unsigned |
| `CHAR` and `CHAR VAR` length | 32767 |
| `BIT` length | 64 |
| Numeric picture | 31 digit positions |
| Structure levels | 15 |
| Array dimensions per reference | 3, including inherited dimensions |
| `%INCLUDE` nesting | 8 levels |
| Active calls | 100 per task, procedures of LINKed PL/I programs included (then `ABEN`) |
| Concatenation and `COPY` result | 32767 characters (then `ABSR`) |
| Exponent of ** | Integer constant 0 to 64 |
| Storage per program (`AUTOMATIC` of all procedures plus `STATIC`) | 1 MiB (`STORAGE LIMIT EXCEEDED`) |
| Live activation storage per task | 8 MiB (then `ABST`) |
| Nesting of parentheses, prefix operators, statements and procedures | 256 (`NESTING TOO DEEP`) |
| Expression tree depth | 4096 (`NESTING TOO DEEP`) |
| Runtime without giving control back to the region | 2 seconds (then `AICA`; see "Runaway Tasks" in Chapter 16) |

**Figure 34-3. Limits of bricks PL/I**

Part 8

# REXX

This part describes REXX as bricks implements it: program structure, variables and stems, control flow, PARSE templates, conditions and condition traps, and operators and built-in functions.

**Chapter 35. REXX Program Structure** ..... **35-1**
**Chapter 36. REXX Variables and Stems** ..... **36-1**
**Chapter 37. REXX Control Flow** ..... **37-1**
**Chapter 38. REXX Parsing** ..... **38-1**
**Chapter 39. REXX Conditions and Condition Traps** ..... **39-1**
**Chapter 40. REXX Operators and Built-in Functions** ..... **40-1**

Chapter 35

# REXX Program Structure

This chapter describes the form of a bricks REXX program: its clauses, comments, literal strings and character set, the keyword instructions that bricks accepts, internal routines, the `ADDRESS` instruction, and the preprocessor that turns `EXEC CICS` and `EXEC SQL` blocks into commands. The chapters that follow describe variables (Chapter 36), control flow (Chapter 37), parsing (Chapter 38), conditions (Chapter 39) and built-in functions (Chapter 40).

Every command described in Part 3 (`EXEC CICS`), Part 4 (`EXEC CICS WEB` and `DOCUMENT`) and Part 5 (`EXEC SQL`) is available to a REXX program with the same syntax and the same results as from COBOL and PL/I. The one difference is the default action for an unhandled condition, which is described in "Unhandled CICS Conditions" in Chapter 39.

The bricks REXX interpreter follows the IBM TSO/E REXX Reference and *The REXX Language* (TRL) where it implements a feature. Where it differs, the difference is stated in the section concerned. bricks is a member of the REXX Language Association.

## Form of a Program

A REXX program is a source file in the directory named by `rexx_dir` (normally `runtime/rexx`), defined to bricks by a line in `runtime/transactions.conf` whose language field is `rexx`. The program is a sequence of clauses, with optional labels and internal routines. Execution starts at the first clause and ends when control reaches the end of the main program, an `EXIT` instruction, a `RETURN` instruction at the main level, or an `EXEC CICS RETURN`, `XCTL` or `ABEND` command. There is no column discipline: blanks and line ends separate tokens, and indentation is free.

A program conventionally starts with a comment that describes it. The smallest useful program sends a map and returns:

```
/* HELO -- minimum REXX program */
ADDRESS CICS
EXEC CICS SEND MAP('HELO1') ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

The supplied HELO transaction (`runtime/rexx/hello.rexx`) extends this program. It asks for the screen size with `EXEC CICS ASSIGN SCREENHT SCREENWD` and sends `HELO1` (Model 2), `HELO1M` (Model 3) or `HELO1L` (Model 4), falling back to `HELO1` on `MAPFAIL`. See "REXX Samples" in Chapter 41.

The output of the `SAY` instruction is discarded in a transaction. Use `EXEC CICS SEND TEXT`, a temporary storage queue, or CEDF (Chapter 8) to look at values while a program is being debugged.

### Clauses, Labels and Continuation

A clause ends at the end of a line or at a semicolon (`;`). Several clauses may therefore be written on one line. A comma that is the last token of a line continues the clause on the next line; the comma counts as one blank. A comma inside an `EXEC CICS` or `EXEC SQL` block is not a continuation (see "The EXEC CICS and EXEC SQL Preprocessor").

A label is a symbol followed by a colon (`NAME:`). It may stand on a line of its own or precede a clause on the same line. Labels are the targets of `SIGNAL`, of `SIGNAL ON` traps, and of `HANDLE CONDITION`, `HANDLE AID` and `HANDLE ABEND LABEL`. A label followed by the keyword `PROCEDURE` begins an internal routine (see "Internal Routines").

### Comments

A comment begins with `/*` and ends with the matching `*/`. A comment may span lines and may hold any character of the UTF-8 source file. Comments nest, as in IBM REXX: a `/*` inside a comment opens an inner comment, which must be closed by its own `*/` before the outer comment ends.

**Note:**

Because comments nest, a stray `/*` inside a comment, for example in the text `/* reads W/* records */`, leaves the comment open. Everything up to the next unmatched `*/`, including whole internal routines, becomes part of the comment. The program still loads without an error, and the loss shows only at runtime, typically as `unknown function` for a routine that was swallowed. Do not write `/*` in comment text.

As a bricks extension, two hyphens (`--`) start a comment that ends at the end of the line.

### Literal Strings

A literal string is enclosed in single or double quotation marks. A quotation mark of the enclosing kind is written twice inside the string: `'it''s'`. A string immediately followed by the letter `X` or `B`, with no blank between, is a hexadecimal or binary string whose value is the decoded bytes:

| Literal | Value |
|---|---|
| `'F3'X` | One byte, X'F3'. |
| `'F1F2'X` | Two bytes, X'F1' X'F2'. |
| `'1'X` | X'01'. An odd number of digits is padded on the left with `0`. |
| `'0110'B` | X'06'. Binary digits are padded on the left to a whole byte. |
| `''X, ''B` | The null string. |

**Figure 35-1. Hexadecimal and binary strings**

The letter is recognized only when it stands alone: `'03'XY` is the string `03` abutted to the symbol `XY`. A hexadecimal or binary string with an invalid digit (`'GG'X`, `'2'B`) raises the SYNTAX condition. The value is an ordinary string: `C2X('F3'X)` is `F3`.

## Character Set

Every REXX value is a string of bricks single-byte (SBCS) characters, one byte per character, as on a mainframe. This applies to variables, stem tails, literals, function arguments and results, and the text of `EXEC CICS` and `EXEC SQL` commands. The code page is ISO-8859-1 (Latin-1), with a graphic plane in positions X'80' through X'9F' that holds the euro sign, block and box-drawing characters, arrows, and a few mathematical symbols. Therefore `LENGTH('ÄÖÅ')` is 3, `C2X('Ä')` is `C4` and `D2C(196)` is `Ä`; positions used by `SUBSTR`, `POS` and `PARSE` count characters.

Program files are UTF-8. Each literal string is converted to SBCS once, when the program is loaded:

| Where | Allowed |
|---|---|
| Inside a quoted string | ASCII, Latin-1 U+00A0 through U+00FF, and the 32 characters of the graphic plane. |
| Hexadecimal or binary string | Any bytes; never converted. `'C4'X` is one character, `'C384'X` is two. |
| Comments | Any character (the file must be valid UTF-8). |
| Symbols and operators | ASCII only. |

**Figure 35-2. Characters allowed in REXX source**

A UTF-8 byte order mark at the start of the file is ignored. Any other violation is a load error that names the line, the column (a byte offset in the UTF-8 file, starting at 1) and the character, for example:

```
line 1 col 6: character U+4E2D in string literal is not in the bricks
  single-byte character set
line 2 col 2: character U+00C4 is only allowed inside a string literal
  or comment
```

An unterminated string is reported at the column of its opening quotation mark. Typographic dashes and quotation marks, emoji and CJK characters are therefore allowed in comments only.

Conversion between SBCS and UTF-8 happens only where data leaves or enters the program. `SAY` writes UTF-8. `JSONGET` and `JSONCOUNT` convert string results back to SBCS, and a character outside the code page becomes `?`. `INTERPRET` runs a value that is already SBCS and does not decode it again. Values that the CICS layer stores in variables (`RECEIVE MAP`, `READQ`, the COMMAREA, `EIBAID`) are SBCS; the terminal, PostgreSQL and HTTP connections convert. `LINEOUT` and `CHAROUT` store the file as UTF-8 on disk.

**Case.** The bricks extensions `UPPER` and `LOWER` fold the whole Latin-1 alphabet (`UPPER('äöå')` is `ÄÖÅ`); the sharp s, y with diaeresis, micro sign, the division and multiplication signs and the graphic plane are unchanged. Every other folding folds `a` through `z` only: `PARSE UPPER`, one-argument `TRANSLATE`, symbols, and compound-symbol tails.

**Blanks.** Word splitting, the blank stripping of the = comparison and number recognition treat only ASCII blanks as blanks; the no-break space X'A0' is an ordinary character. `DATATYPE(s,'A')` accepts ASCII letters and digits only. Error messages that quote program data are pure ASCII, with other characters shown as escapes.

**Differences from z/OS.** The code page is Latin-1, not EBCDIC: `C2X('A')` is `41` (z/OS: `C1`), and comparisons and `XRANGE` follow Latin-1 order. `UPPER` and `LOWER` are not IBM functions.

### Keyword Instructions

The following keyword instructions are supported. Each is described in the chapter shown.

| Instruction | Function | See |
|---|---|---|
| `ADDRESS` | Set the command environment. | Chapter 35 |
| `CALL` | Call an internal routine or a built-in function. | Chapter 37 |
| `DO ... END` | Group clauses; repeat them. | Chapter 37 |
| `DROP` | Drop variables. | Chapter 36 |
| `EXIT` | End the program. | Chapter 37 |
| `IF` | Choose between two clauses. | Chapter 37 |
| `INTERPRET` | Run a string as REXX. | Chapter 37 |
| `ITERATE` | Start the next pass of a loop. | Chapter 37 |
| `LEAVE` | Leave a loop. | Chapter 37 |
| `NOP` | Do nothing. | Chapter 37 |
| `NUMERIC` | Set DIGITS, FUZZ or FORM. | Chapter 37 |
| `PARSE` | Split a string into variables. | Chapter 38 |
| `PROCEDURE` | Begin an internal routine. | Chapter 35 |
| `RETURN` | Return from a routine. | Chapter 37 |
| `SAY` | Write a line (discarded in a transaction). | Chapter 35 |
| `SELECT` | Choose one of several clauses. | Chapter 37 |
| `SIGNAL` | Transfer control; set condition traps. | Chapter 39 |

**Figure 35-3. Supported keyword instructions**

A clause that is not one of these instructions, not an assignment (`symbol = expression`) and not a label is a command if it is a string expression (see "ADDRESS and Commands"), and an error otherwise.

The following IBM instructions and forms are **not** supported. A program that uses one is rejected when it is loaded, with the error `unexpected token` or `PARSE expects VAR/VALUE/ARG/PULL`, except where the table says otherwise.

| Instruction or form | Use instead |
|---|---|
| `ARG` | `PARSE UPPER ARG`, or `PARSE ARG`. |
| `PULL, PUSH, QUEUE` | `EXEC CICS RECEIVE` for terminal input; a TS queue for a stack of data. `PARSE PULL` is accepted and always yields the null string. |
| `UPPER` | `X = UPPER(X)`. |
| `TRACE` | CEDF (Chapter 8). |
| `OPTIONS` | None. |
| `PARSE SOURCE, VERSION, LINEIN, LOWER, NUMERIC` | The functions `ADDRESS()`, `DIGITS()`, `LOWER()`, `LINEIN()` with `PARSE VALUE`. |
| `CALL ON condition` | `SIGNAL ON`. A `CALL ON` clause loads, but runs as a call of a routine named `ON` and fails with `unknown function`. |
| `DO ... FOR n` | A counter and `LEAVE`, or `TO`. |
| `DROP (list), EXPOSE (list)` | Name the variables explicitly. |

**Figure 35-4. IBM instructions not supported**

### Internal Routines

An internal routine is a label followed by the `PROCEDURE` instruction:

```
name: PROCEDURE [EXPOSE name1 [name2 ...]]
```

The routine has its own variable pool. Variables of the caller are not visible unless they are named on `EXPOSE`; a name ending in a period (`LANG.`) exposes the whole stem (see "PROCEDURE EXPOSE" in Chapter 36). The routine is invoked by a `CALL` instruction or as a function in an expression, and it returns with `RETURN [expression]` or by reaching its end.

```
SAY DOUBLE(21)                  /* 42: routine called as a function */
CALL GREET 'Alice'
EXIT

DOUBLE: PROCEDURE
   PARSE ARG N
   RETURN N * 2

GREET: PROCEDURE EXPOSE LANG.
   PARSE ARG NAME
   SAY LANG.HELLO NAME
   RETURN
```

The following rules of the bricks dialect differ from IBM REXX and must be observed:

- **Only a PROCEDURE label can be called.** `CALL name` and `name(...)` search the program's `PROCEDURE` labels and then the built-in functions. A plain label is a target for `SIGNAL` only; calling it fails with `unknown function`. Write `name: PROCEDURE`, with `EXPOSE` for the variables that the routine shares with its caller.
- **A routine ends at the next label.** The body of a `PROCEDURE` runs from the `PROCEDURE` instruction to the next label of any kind. A label written inside a routine therefore ends the routine; the clauses after it belong to the main program. Do not put labels inside a routine.
- **Routines are not part of the main program.** Control never falls into a routine. The clauses of the main program that follow a routine (after a later plain label) are part of the main sequence. End the main program with `EXIT` or `EXEC CICS RETURN` before the first routine, as usual.

When a routine is called by `CALL`, `RETURN expression` sets the special variable `RESULT` in the caller; a `RETURN` without an expression, or reaching the end of the routine, drops `RESULT`. When a routine is called as a function it must return a value: a `RETURN` without an expression raises the SYNTAX condition with `function did not return data`. `RETURN ''` counts as a value.

The `NUMERIC` settings, the `ADDRESS` environment and the `SIGNAL ON` traps are saved when a routine is entered and restored when it returns. A routine that has the same name as a built-in function replaces the built-in function for the program; see "Calling a Function" in Chapter 40.

Routines may call themselves. A task may nest at most 1000 routine calls, `CALL` and function calls together, counted across every REXX program that the task has `LINK`ed to. The next call raises the SYNTAX condition `control stack full` (IBM REXX error 11).

### ADDRESS and Commands

```
ADDRESS environment
ADDRESS expression
```

`ADDRESS` names the environment to which commands are sent. A command is a clause that consists of a string expression, for example a quoted string. bricks provides one environment, `CICS`, which is the initial environment of every transaction. Any other name is accepted by `ADDRESS`, but the next command then fails with `ADDRESS handler not registered for` the name. `ADDRESS` with no operand has no effect in bricks (IBM REXX switches back to the previous environment). The function `ADDRESS()` returns the current environment; see "Information Functions" in Chapter 40.

```
ADDRESS CICS
"SEND MAP('CUST1') FROM(SCR.) ERASE"
"RETURN"
```

After each command the special variable `RC` holds the return code, which for a CICS command is the `EIBRESP` value. A non-zero `RC` raises the ERROR condition if `SIGNAL ON ERROR` is in effect ("SIGNAL ON and SIGNAL OFF" in Chapter 39).

### Special Variables

| Variable | Set by | Remains until |
|---|---|---|
| RC | Every command, EXEC CICS and EXEC SQL. Holds EIBRESP for a CICS command. Set to 0 when a NOVALUE or SYNTAX trap is taken. | The next command. |
| RESULT | CALL of a routine that returns a value, or of a built-in function. | A CALL whose routine returns no value, or DROP RESULT. |
| SIGL | SIGNAL, a condition trap, and a branch by HANDLE CONDITION, HANDLE AID or HANDLE ABEND: the line number of the main-program clause that was running (Chapter 39). | The next transfer. |
| SQLCODE, SQLSTATE, SQLERRMC | Every EXEC SQL statement ("The SQL Communication Area" in Chapter 21). | The next EXEC SQL. |

**Figure 35-5. Special variables**

The EIB fields (EIBRESP, EIBAID, EIBCALEN and the others), the COMMAREA (DFHCOMMAREA) and the stem MAP. that RECEIVE MAP fills when INTO is omitted are ordinary variables in the program's pool; see "The EIB in REXX" in Chapter 17. These names, and the special variables, can be assigned by the program, but should not be.

### The EXEC CICS and EXEC SQL Preprocessor

```
EXEC CICS command [option ...] END-EXEC
EXEC SQL  statement END-EXEC
```

Before a program is parsed, bricks rewrites each EXEC CICS ... END-EXEC block into a command string at the same place, so that it runs exactly as the same text would run as a command under ADDRESS CICS. EXEC SQL blocks are rewritten in the same way and sent to the SQL executor ("Coding EXEC SQL Statements" in Chapter 21), whatever the current ADDRESS. The two forms below are therefore equivalent:

```
EXEC CICS SEND MAP('HELO1') ERASE END-EXEC

ADDRESS CICS
"SEND MAP('HELO1') ERASE"
```

The EXEC form is preferred, because it does not depend on the current ADDRESS environment. The keywords EXEC, CICS, SQL and END-EXEC may be in any case. Text inside comments and quoted strings is not examined for them.

A block may be written on several lines. The line ends are replaced by blanks and the line numbering of the program is kept, so that error messages and CEDF show the right line:

```
EXEC CICS ASSIGN USERID(USR) TERMID(TRM) CONNECTED(CT)
                 SCREENHT(SCRH) SCREENWD(SCRW) END-EXEC
```

The text between EXEC CICS and END-EXEC is passed to the command parser as written. Therefore:

- Do not end a line of a block with a comma. A REXX continuation comma is not removed inside a block; the command fails with unexpected token ",".
- Do not put a comment inside a block. The comment text reaches the command parser, and the command fails with unexpected character "/".
- A block with no END-EXEC is a load error (EXEC CICS at line n: missing END-EXEC).

In an option value ("Arguments" in Chapter 2), a quoted string is a constant, and a symbol names a variable that supplies or receives a value. A stem name with its period (FROM(SCR.)) names a whole stem.

Chapter 36

# REXX Variables and Stems

This chapter describes REXX symbols and variables: simple variables, stems and compound symbols, the derivation of compound names, `DROP`, and the sharing of variables with `PROCEDURE EXPOSE`.

## Symbols

A symbol consists of the ASCII letters `A-Z` and `a-z`, the digits, and the characters `_ ! ? @ # $` and the period. A symbol that begins with a letter or one of the special characters can name a variable; a symbol that begins with a digit or a period (`77`, `.5`, `3D`) is a constant symbol and can never be assigned. Symbols are not case-sensitive: the name of a variable is the symbol in uppercase.

REXX has one data type, the character string. An assignment `symbol = expression` evaluates the expression once and stores the resulting string. Numbers are strings that happen to be valid numbers.

## Simple Variables and NOVALUE

A simple variable that has not been assigned a value, or has been dropped, has as its value its own name in uppercase: if `X` has never been assigned, `SAY X` writes `X`. Programs use this to test for a first pass, for example `IF QNM = 'QNM' THEN QNM = ''` in `cons.rexx`. If `SIGNAL ON NOVALUE` is in effect, a reference to an unassigned variable raises the NOVALUE condition instead ("Conditions" in Chapter 39).

The variables that a transaction finds already set (the EIB fields, `DFHCOMMAREA` and so on) are described in "Special Variables" in Chapter 35 and in "The EIB in REXX" in Chapter 17.

## Stems

A stem is a symbol that ends in a period, such as `SCR.`. A compound symbol is a stem followed by one or more tail segments separated by periods, such as `SCR.MSG` or `GRID.I.J`. Assigning a value to the stem itself sets the default value of every compound variable with that stem that has no value of its own:

```
STEM. = 'unset'
STEM.42 = 'forty-two'
SAY STEM.1                      /* unset     -- the stem default   */
SAY STEM.42                     /* forty-two -- an explicit value  */
```

A stem assignment sets only the default. Compound variables that already have a value keep it. To remove them, use `DROP stem.` (see "DROP").

Stems are the REXX form of a BMS map: `SEND MAP ... FROM(SCR.)` takes each field from the compound variable whose tail is the field name, and `RECEIVE MAP ... INTO(SCR.)` sets them. See Chapter 3 and "RECEIVE MAP" in Chapter 11.

### Derived Names of Compound Symbols

Each tail segment that is a symbol is replaced by the value of that symbol before the variable is looked up, as in IBM REXX. With `J = 3`, `A.J` refers to `A.3`. The rules are:

| Tail segment | Contributes to the derived name |
|---|---|
| `STEM.42` | A constant symbol: the characters `42`. |
| `STEM.NAME, NAME has a value` | The value of `NAME`. |
| `STEM.NAME, NAME has no value` | The name `NAME` itself, because that is the value of an unset variable. |
| `STEM.I.J` | Each segment separately: with `I = 1` and `J = 2`, the name is `STEM.1.2`. |

**Figure 36-1. Derivation of compound names**

The value of a tail segment is folded to uppercase (`a-z` only) before it is used, so with `K = 'ab'`, `S.K` refers to `S.AB`. IBM REXX uses the value as it is, so that `S.ab` and `S.AB` are different variables there.

**Note:**

Because a tail symbol is replaced by its value, a program must not assign a simple variable whose name is also used as a tail. If a program sets `BIRTH = '19700101'` and then assigns `OUT.BIRTH = 'x'`, the assignment sets `OUT.19700101`, and the map field `BIRTH` stays empty. The supplied programs give local variables names that differ from the map field names: `AKT` for the field `ACTION`, `CKEY` for `CUSTNO`, `NM` for `NAME`. A quoted tail (`SCR.'MSG'`) is not valid syntax. See also "Compound symbols and map fields" in Appendix A.

To build a variable name at runtime, use the `VALUE` function: `CALL VALUE 'SCR.ROW' || J, LINE` sets the compound variable `SCR.ROW1`, `SCR.ROW2` and so on, as `cusl.rexx` does for the rows of its list map.

### DROP

```
DROP name [name ...]
```

`DROP` removes each named variable from the current variable pool, so that it has no value again. A stem name with its period drops the stem default and every compound variable of the stem. A compound symbol drops that one variable. The names are listed explicitly; the IBM form `DROP (list)`, which takes the names from a variable, is not supported.

```
DROP RECS.                      /* the whole stem                */
DROP A B C                      /* three simple variables        */
DROP STEM.42                    /* one compound variable         */
```

Drop an accumulating stem at the start of each pass of a paging loop, so that values from a longer earlier page do not remain.

### PROCEDURE EXPOSE

Inside an internal routine that begins with `PROCEDURE`, the variables of the caller are not visible, except those named on `EXPOSE`:

```
GREET: PROCEDURE EXPOSE LANG. SUFFIX
```

`LANG.` exposes the caller's entire `LANG.` stem, default value and all compound variables; `SUFFIX` exposes one simple variable; a compound symbol such as `A.B` exposes that one compound variable. A change that the routine makes to an exposed variable is made in the caller's pool. Exposure is transitive: when a routine exposes a name that its caller in turn exposed from its own caller, all three refer to the same variable.

The names on `EXPOSE` are listed explicitly; the IBM form `EXPOSE (list)` is not supported. All other variables of the routine are local and disappear when it returns.

Chapter 37

# REXX Control Flow

This chapter describes the instructions that control the order in which clauses run: `IF`, `SELECT`, the `DO` family with `LEAVE` and `ITERATE`, `CALL`, `RETURN`, `EXIT`, `SIGNAL` and `INTERPRET`, together with `NUMERIC`, `NOP` and commands. Condition traps are described in Chapter 39.

## IF

```
IF expression THEN clause [ELSE clause]
```

The keyword `THEN` is required. It may stand on the same line as the condition or on the next line, and so may `ELSE`. Each clause is a single instruction or a `DO ... END` group. The expression must evaluate to `0` or `1`; any other value raises the SYNTAX condition (`logical value must be 0 or 1`). An `ELSE` belongs to the nearest `IF` that has no `ELSE`, which allows `ELSE IF` chains:

```
IF EIBRESP = 13 THEN SCR.MSG = 'Customer' CKEY 'not found.'
ELSE IF EIBRESP \= 0 THEN SCR.MSG = 'READ failed RESP=' || EIBRESP
ELSE DO
   PARSE VAR REC NM '|' AD '|' CY '|' PH
   SCR.MSG = 'Query of' CKEY 'complete.'
END
```

## SELECT

```
SELECT
   WHEN expression THEN clause
   [WHEN expression THEN clause] ...
   [OTHERWISE [clause ...]]
END
```

The `WHEN` expressions are evaluated in order, and the clause of the first one that is `1` runs; the others are skipped. If no `WHEN` is true and there is no `OTHERWISE`, the SYNTAX condition is raised (`SELECT: no WHEN matched and no OTHERWISE`), as in IBM REXX. Write `OTHERWISE NOP` when nothing is to be done.

```
AID = C2X(EIBAID)
SELECT
   WHEN AID = 'F3' THEN EXIT_LIST = 1
   WHEN AID = 'F7' THEN DO
      IF PAGE > 1 THEN PAGE = PAGE - 1
   END
   WHEN AID = 'F8' THEN DO
      IF PAGE < NPAGES THEN PAGE = PAGE + 1
   END
   OTHERWISE NOP
END
```

## DO

```
DO [repetitor] [conditional]
   [clause ...]
END [name]

repetitor:   name = start [TO limit] [BY step]
           | count
           | FOREVER
           | name OVER stem.
conditional: WHILE expression | UNTIL expression
```

| Form | Behavior |
|---|---|
| `DO ... END` | A group: the clauses run once. Used as the clause of `IF`, `WHEN` and `OTHERWISE`. |
| `DO count` | Runs the clauses *count* times. *count* must begin with a number or a symbol (`DO 3`, `DO N`). |
| `DO name = start TO limit BY step` | Counted loop. `BY` defaults to 1 and may be negative. `TO` must come before `BY`. Without `TO`, the loop runs until it is left. `BY 0` with `TO` raises the SYNTAX condition, because the loop would never end. After the loop, the control variable holds the first value that failed the test. |
| `DO WHILE expression` | Tests before each pass; runs while the expression is 1. |
| `DO UNTIL expression` | Tests after each pass; runs at least once, until the expression is 1. |
| `DO name = ... WHILE / UNTIL` | A counted loop with an additional test. |
| `DO FOREVER` | Runs until `LEAVE`, `RETURN`, `EXIT`, `SIGNAL` or an `EXEC CICS RETURN` ends it. |
| `DO name OVER stem.` | A bricks and ooRexx extension. Sets *name* to each tail of *stem.* that has a value: numeric tails first in numeric order, then the others in collating order. The stem default is not visited. |

**Figure 37-1. Forms of the DO instruction**

The name after `END`, if given, is accepted. The IBM forms `DO count WHILE` and `DO name = ... FOR n` are not supported. A label may not appear inside a `DO` group; it is a load error.

Every loop is subject to the runaway task limit: a task that runs for 2 seconds without giving control back to the region is abended `AICA`. Commands such as `ASKTIME`, `ASSIGN` and `HANDLE` do not give control back. See "Runaway Tasks" in Chapter 16.

### LEAVE and ITERATE

```
LEAVE [name]
ITERATE [name]
```

`LEAVE` ends a loop and continues after its `END`. `ITERATE` ends the current pass and starts the next one. With *name*, they act on the loop whose control variable is *name*, leaving any inner loops; without *name*, they act on the innermost `DO`.

**Note:**

In bricks, a plain `DO ... END` group counts as the innermost `DO`. A `LEAVE` or `ITERATE` without a name inside `IF ... THEN DO ... END` therefore ends only the group: control continues with the clause after the group's `END`, inside the same pass of the loop. In IBM REXX it would leave or iterate the enclosing loop. Use one of these forms instead:

```
DO I = 1 TO N
   IF KEYS.I = '' THEN LEAVE         /* bare clause: leaves the loop */
   IF DONE THEN DO
      SCR.MSG = 'Stopped at' I
      LEAVE I                        /* named: leaves the loop       */
   END
END
```

A `DO FOREVER` or `DO WHILE` loop has no control variable to name. Inside such a loop, set a flag in the group and test it with a bare `IF ... THEN LEAVE` after the group, or structure the loop as `DO WHILE DONE = 0`.

### CALL

```
CALL name [expression [, expression] ...]
```

`CALL` invokes an internal routine (a label followed by `PROCEDURE`) or, if there is none of that name, a built-in function. The arguments are evaluated from left to right and are available to the routine through `PARSE ARG` and `ARG()`. A value that the routine returns is assigned to `RESULT`; a built-in function's value is always assigned to `RESULT`. A plain label cannot be called; see "Internal Routines" in Chapter 35. A routine can also be invoked as a function, `X = NAME(args)`.

### RETURN and EXIT

```
RETURN [expression]
EXIT [expression]
```

`RETURN` in an internal routine returns to the caller; see "Internal Routines" in Chapter 35. At the main level it ends the program. `EXIT` ends the program from any level. The value of the expression, if any, is not used by the transaction. These instructions end the REXX program only; to end the task and return control to CICS in the documented way, issue `EXEC CICS RETURN` ("RETURN" in Chapter 12).

### SIGNAL

```
SIGNAL label
SIGNAL ON condition [NAME label]
SIGNAL OFF condition
```

`SIGNAL label` transfers control to a label of the main program. Active `DO` and `SELECT` instructions and internal routines are ended first, so control does not return. `SIGL` is set to the line number of the `SIGNAL`, or, when the `SIGNAL` is inside a `DO`, `IF`, `SELECT` or an internal routine, of the outermost enclosing clause of the main program. A label that does not exist is an error (`SIGNAL: unknown label`). The `ON` and `OFF` forms set and clear condition traps; see "SIGNAL ON and SIGNAL OFF" in Chapter 39. The CICS commands `HANDLE CONDITION`, `HANDLE AID` and `HANDLE ABEND LABEL` transfer control in the same way as `SIGNAL`.

### INTERPRET

```
INTERPRET expression
```

The expression is evaluated and the resulting string is run as REXX clauses in the current variable pool. Labels and routines inside the string are known only while the string runs.

## NUMERIC

| Form | Effect |
|---|---|
| `NUMERIC DIGITS [n]` | Sets the number of significant digits to which arithmetic results are rounded. Default 9. |
| `NUMERIC FUZZ [n]` | Sets the number of digits ignored in numeric comparisons. Default 0; must be less than DIGITS. |
| `NUMERIC FORM SCIENTIFIC \| ENGINEERING` | Accepted; has no effect. Results are always shown in plain decimal notation. |

**Figure 37-2. NUMERIC settings**

Arithmetic is carried out in binary floating point (64-bit) and the result is rounded to `NUMERIC DIGITS` significant digits. Integer division (`%`) and remainder (`//`) are not rounded. Very large results are shown in full rather than in exponential notation: `1E10 * 10` is `100000000000`.

## NOP and Commands

`NOP` does nothing; it is used where a clause is required, as in `OTHERWISE NOP`. A clause that is a string expression is a command for the current `ADDRESS` environment (see "ADDRESS and Commands" in Chapter 35).

Chapter 38

# REXX Parsing

This chapter describes the PARSE instruction, which splits a string into variables under the control of a template. It is the usual way for a REXX program to take apart a record read from a file, the operator's command line, or a COMMAREA.

## PARSE

```
PARSE [UPPER] VAR name template
PARSE [UPPER] VALUE expression WITH template
PARSE [UPPER] ARG template [, template] ...
PARSE [UPPER] PULL template
```

The source keyword says where the string comes from; the template says how it is split and which variables receive the pieces. UPPER translates the source string to uppercase (a-z only) before it is parsed.

### *Sources*

| Source | String parsed |
|---|---|
| VAR name | The value of the variable. A compound name is derived in the usual way: PARSE VAR A.I ... with I = 2 parses A.2. |
| VALUE expression WITH | The value of the expression. WITH ends the expression. |
| ARG | The arguments of the program or internal routine. Each template separated by a comma parses one argument, in order: PARSE ARG H, W parses ARG(1) into H and ARG(2) into W. A comma is valid only with ARG. |
| PULL | Always the null string, because bricks has no data stack. Use EXEC CICS RECEIVE to read terminal input. |

**Figure 38-1. PARSE sources**

PARSE SOURCE, PARSE VERSION, PARSE LINEIN, PARSE NUMERIC, PARSE EXTERNAL and PARSE LOWER are not supported; a program that uses one is rejected when it is loaded (PARSE expects VAR/VALUE/ARG/PULL). To parse a string in lowercase, use PARSE VALUE LOWER(s) WITH ....

*Template Elements*

| Element | Effect |
|---|---|
| `name` | A variable that receives a piece of the string. See "Parsing into Words". A compound name has its tail derived when the value is assigned. |
| `.` | A placeholder: it takes a piece like a variable, and the piece is discarded. |
| `'string'` | A literal pattern. The string is searched for from the current position; the text before it goes to the preceding variables, and parsing continues after it. If the string is not found, the preceding variables take the rest of the source. |
| `n` | An absolute column (1 is the first character). The preceding variables take the text up to column *n*-1, and parsing continues at column *n*. |
| `+n, -n` | A relative column, *n* characters forward or back from the start of the previous pattern. |

**Figure 38-2. Template elements**

A column at or before the current position does not raise an error. As TRL defines it, the variables before it take the rest of the string from the current position, and parsing restarts at the new column. This is the usual way to parse the same string twice: `PARSE VAR REC 5 KEY 9 1 WHOLE` sets `KEY` to columns 5 to 8 and `WHOLE` to the whole record.

The IBM patterns `=n` and `(variable)` are not supported; they are load errors (`unexpected "=" in PARSE template`). Build a variable pattern with `POS` and `SUBSTR` instead.

## Parsing into Words

Between two patterns (or the ends of the string), the piece of text is shared among the variables and placeholders that stand there:

- Each variable except the last takes one blank-delimited word, with leading blanks skipped. Only the blank character delimits words.
- The last variable takes the rest of the piece. Leading blanks are removed; blanks between the words and trailing blanks are kept. IBM REXX removes only the one blank that ended the previous word, so in IBM REXX the rest of the piece can begin with blanks.
- A single variable that stands alone between two patterns, or alone in the template, takes the piece exactly as it is, leading blanks included.

```
LINE = 'CMD   arg1  arg2'
PARSE VAR LINE VERB REST        /* VERB='CMD', REST='arg1  arg2'   */
PARSE VAR LINE VERB .           /* VERB='CMD', the rest discarded  */
```

### Examples

```
/* The operator typed "GETC 100": transaction id, then the key.    */
PARSE VAR BUF TID CKEY .

/* A record whose fields are separated by '|'.                      */
PARSE VAR REC NM '|' AD '|' CY '|' PH

/* Fixed columns.                                                   */
PARSE VAR ROW 1 NAME 21 ADDR 51 PHONE

/* Two arguments of an internal routine.                            */
GREET: PROCEDURE
   PARSE ARG NAME, GREETING
   SAY GREETING NAME
   RETURN

/* A command word in any case.                                      */
PARSE UPPER VAR LINE VERB REST
SELECT
   WHEN VERB = 'LIST' THEN CALL LIST_HANDLER REST
   WHEN VERB = 'ADD'  THEN CALL ADD_HANDLER REST
   OTHERWISE SCR.MSG = 'Unknown command' VERB
END

/* A compound source: the tail is derived from I.                   */
I = 2
A.2 = 'hello world'
PARSE VAR A.I W1 W2             /* W1='hello', W2='world'          */
```

For a record with fewer fields than the template has variables, the extra variables are set to the null string. For example, `PARSE VAR R A '|' B '|' C '|' D` with `R = 'n|a|c'` sets `D` to the null string.

**Note:**

Choose the names of the target variables with care when they are also used as tails of a map stem. `PARSE VAR REC NAME ...` followed by `SCR.NAME = NAME` sets the compound variable whose tail is the customer's name, not the map field `NAME`. See "Derived Names of Compound Symbols" in Chapter 36.

Chapter 39

# REXX Conditions and Condition Traps

This chapter describes the REXX conditions that a program can trap with `SIGNAL ON`, and how a REXX program deals with the conditions that CICS commands raise. The CICS commands for condition handling (`HANDLE CONDITION`, `IGNORE CONDITION`, `HANDLE AID`, `HANDLE ABEND`) are described in Chapter 16; this chapter describes how they behave in REXX.

## SIGNAL ON and SIGNAL OFF

```
SIGNAL ON condition [NAME label]
SIGNAL OFF condition

condition: ERROR | NOVALUE | SYNTAX | HALT
```

`SIGNAL ON` sets a trap for the condition. When the condition is raised, control passes to *label*, or, without `NAME`, to the label that has the name of the condition (`SIGNAL ON ERROR` goes to `ERROR:`). `SIGNAL OFF` removes the trap. The label must be a label of the main program; if it does not exist, the condition is treated as if no trap were set.

Any other condition name, such as `FAILURE`, `NOTREADY` or `LOSTDIGITS`, is accepted when the program is loaded, but the `SIGNAL ON` instruction fails when it runs (`SIGNAL ON: unknown condition`), which raises the SYNTAX condition.

When a trap is taken:

- `SIGL` is set to the line number of the main-program clause that was running. When the condition is raised inside a `DO`, `IF` or `SELECT`, or inside an internal routine, that is the line on which the outermost enclosing clause of the main program begins, not the line of the failing clause itself (IBM REXX gives the failing clause).
- For ERROR, `RC` keeps the return code of the command that failed; for a CICS command that is the `EIBRESP` value, for example 13 for `NOTFND`. For the other conditions, `RC` is set to 0.
- Active `DO` and `SELECT` instructions and internal routines are ended, as for `SIGNAL label`.
- The trap remains set. In IBM REXX a trap is removed when it is taken; in bricks it stays in effect until `SIGNAL OFF`. A handler that itself raises the same condition is therefore entered again, and loops until the task is abended `AICA`. A handler that may fail should begin with `SIGNAL OFF` for its condition.

Traps are saved when an internal routine is entered and restored when it returns, so a routine can set its own traps without affecting its caller.

## Conditions

| Condition | Raised when |
|---|---|
| ERROR | A command, including an EXEC CICS command, ends with a non-zero RC. This includes a command that codes RESP or NOHANDLE. It does not include a condition that HANDLE CONDITION branches for, or that IGNORE CONDITION ignores (bricks then sets RC and EIBRESP to 0). EXEC SQL statements do not raise ERROR: they set RC to SQLCODE; use EXEC SQL WHENEVER or test SQLCODE ("WHENEVER" in Chapter 21). |
| NOVALUE | A simple or compound variable that has no value, and is not covered by a stem default, is referenced. |
| SYNTAX | Any other error detected while the program runs: a value that is not a number, division by zero, an unknown function, an incorrect argument to a built-in function, a logical value that is not 0 or 1, control stack full, a failed SIGNAL ON. |
| HALT | Accepted, but never raised: bricks has no external halt request. |

**Figure 39-1. REXX conditions**

A condition that is not trapped ends the program, and the transaction ends with an error. A runaway task (AICA) and the abends listed in "Abends That End a REXX Task" cannot be trapped by SIGNAL ON.

***Example***

```
SIGNAL ON ERROR  NAME CICSERR
SIGNAL ON SYNTAX NAME OOPS
ADDRESS CICS
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) END-EXEC
...
EXEC CICS RETURN END-EXEC

CICSERR:
   SIGNAL OFF ERROR
   MSG = 'EXEC CICS failed at line' SIGL', RC=' RC
   EXEC CICS SEND TEXT FROM(MSG) ERASE END-EXEC
   EXEC CICS RETURN END-EXEC

OOPS:
   SIGNAL OFF SYNTAX
   MSG = 'REXX error at line' SIGL
   EXEC CICS SEND TEXT FROM(MSG) ERASE END-EXEC
   EXEC CICS RETURN END-EXEC
```

### Unhandled CICS Conditions

A CICS command that does not complete normally raises a CICS condition, such as `NOTFND` or `MAPFAIL`. As in COBOL and PL/I, the condition is dealt with by the first of these that applies:

1. `RESP(var)` or `NOHANDLE` on the command: the command returns, and `EIBRESP`, `RC` and the `RESP` variable hold the response. No `HANDLE CONDITION` branch is taken; an ERROR trap set by `SIGNAL ON ERROR` still is.
2. `IGNORE CONDITION` for the condition: the command returns, and bricks sets `EIBRESP` and `RC` to 0 (`NORMAL`).
3. `HANDLE CONDITION` for the condition (or for `ERROR`): control passes to the label, in the same way as `SIGNAL label`. Control does not return to the clause after the command.
4. Otherwise the condition is unhandled.

The default action for an unhandled condition depends on the language, as on IBM CICS. A COBOL or PL/I task is abended with the condition's abend code (`AEIM` for `NOTFND`; see "Condition Abend Codes" in Appendix C). A REXX task is **not** abended: the command returns with `EIBRESP` and `RC` set to the response, and the program continues with the next clause. If `SIGNAL ON ERROR` is in effect, the ERROR trap is taken, with `RC` holding the real response.

A REXX program can therefore test the response after any command, with or without RESP:

```
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) END-EXEC
IF EIBRESP = 13 THEN SCR.MSG = 'Customer' CKEY 'not found.'
ELSE IF EIBRESP \= 0 THEN SCR.MSG = 'READ failed RESP=' || EIBRESP
```

RESP remains useful in REXX: it keeps a response that a later command would overwrite, and it stops a HANDLE CONDITION branch for that command. When a CEDF or CEDG session watches a REXX task, the forecast for an unhandled condition reads NOT HANDLED – RC SET ("Before and After a Command" in Chapter 8).

***HANDLE CONDITION in REXX***

```
EXEC CICS HANDLE CONDITION NOTFND(NOCUST) END-EXEC
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
...
EXEC CICS RETURN END-EXEC

NOCUST:
   MSG = 'Customer' CKEY 'not found.'
   EXEC CICS SEND TEXT FROM(MSG) ERASE END-EXEC
   EXEC CICS RETURN END-EXEC
```

The label of HANDLE CONDITION, HANDLE AID and HANDLE ABEND LABEL is a label of the main program. SIGL is set as for SIGNAL (see "SIGNAL ON and SIGNAL OFF"). A HANDLE AID trap fires once for each AID received (see "HANDLE AID" in Chapter 16).

### Abends That End a REXX Task

Some failures are abends, not conditions. RESP, NOHANDLE, HANDLE CONDITION, IGNORE CONDITION and SIGNAL ON do not apply to them, and they end a REXX task as they end a COBOL or PL/I task. They include ABM0 (RECEIVE MAP of a map that the mapset does not contain), ABMB (a cursor position beyond the screen), AICA (runaway task), ASP3, ASP7 and ASPR (a commit refused by the database), AD2S and AD2P (database sign-on or connection), ATNI (terminal lost), AEXZ (internal failure in a command) and an EXEC CICS ABEND issued by the program. A HANDLE ABEND exit, where IBM drives one, is driven for a REXX task too. The supplied samples ABMR and SPDB show ABM0 and ASP3 ("The Abend Samples" in Chapter 41). All abend codes are listed in Appendix C.

Chapter 40

# REXX Operators and Built-in Functions

This chapter describes the operators of REXX expressions, the rules for calling a function, and the built-in functions that bricks provides, grouped by purpose. Functions marked as bricks extensions are not in IBM TSO/E REXX.

## Operators

The operators are listed from the lowest priority to the highest. Operators of equal priority are evaluated from left to right, except **.

| Priority | Operators | Meaning |
|---|---|---|
| 1 (lowest) | `\|` | Logical OR. Both operands must be 0 or 1. The right operand is not evaluated when the left one decides the result. |
| 2 | `&` | Logical AND, with the same rules. |
| 3 | `= \= <> >< == \== < > <= >=` | Comparison. == and \== compare exactly. The others compare numerically when both operands are numbers (subject to `NUMERIC FUZZ`), and otherwise compare the strings with leading and trailing blanks removed. \=, <> and >< all mean "not equal". |
| 4 | `\|\| (blank) (abuttal)` | Concatenation; see "Concatenation". |
| 5 | `+ -` | Addition, subtraction. |
| 6 | `* / % //` | Multiplication, division, integer division (truncates toward zero), remainder (has the sign of the dividend). |
| 7 | `prefix + - \` | Sign and logical NOT. |
| 8 (highest) | `**` | Power. Evaluated from right to left: `2**3**2` is 512. Whole-number exponents are exact; others use floating point. |

**Figure 40-1. Operators in priority order**

Comparison operators have one priority, so `A < B = C` compares the result of `A < B` with `C`.

**Differences from IBM REXX.** The exclusive OR operator && and the strict ordering comparisons `>> << >>= <<= \< \>` are not supported (load errors). IBM REXX evaluates ** from left to right and applies prefix operators before **, so `-2**2` is 4 there and -4 in bricks; use parentheses. The character ¬ is not accepted as NOT; use \.

### Concatenation

Two terms written next to each other with no operator are concatenated. What separates them decides the result:

| Written | Result | Rule |
|---|---|---|
| `'a' 'b'` | a b | One or more blanks give exactly one blank. |
| `'a'"b", 'LEN='LENGTH(X)` | ab, LEN=3 | Abuttal (nothing between) gives no blank. |
| `A/*c*/D` | value of A then D | A comment alone between terms is abuttal. |
| `'a' /*c*/'b'` | a b | A blank anywhere between gives one blank. |
| `'a', (line end) 'b'` | a b | A continuation comma counts as a blank. |
| `'41'X'42'X, (1+2)'a'` | AB, 3a | Every kind of term abuts in the same way. |

**Figure 40-2. Forms of concatenation**

Arithmetic binds more tightly than concatenation: `1+2'a'` is `3a`. A string followed at once by the symbol `X` or `B` is a hexadecimal or binary string, so `'['X` is not a concatenation (see "Literal Strings" in Chapter 35).

### Calling a Function

A function is called by writing its name immediately followed by a parenthesized argument list, in an expression, or with the `CALL` instruction, which places the value in `RESULT`. Function names are not case-sensitive.

- A symbol followed by `(` calls the internal routine of that name, if the program has one (a `PROCEDURE` label), and otherwise the built-in function. An internal routine therefore replaces a built-in function of the same name, as in IBM REXX.
- A literal string followed at once by `(` calls the built-in function whose name is exactly the string, skipping internal routines: `'LENGTH'('abc')` is 3. The name is not translated to uppercase, so `'length'('abc')` fails with `unknown function`.
- In bricks, a symbol followed by a blank and then `(` is also a function call: `LENGTH (S)` is the same as `LENGTH(S)`. IBM REXX treats it as a concatenation with a blank. To concatenate in that position, write `X || ' ' || (Y)`.

**Omitted arguments.** An argument cannot be omitted by writing two commas; `STRIP(S,,'0')` is a load error (`unexpected token ","`). Pass the null string instead, which every built-in function treats as an omitted argument: `STRIP(S,'','0')`, `FORMAT(N,'',2)`.

**Errors.** An incorrect call raises the SYNTAX condition, as IBM REXX error 40 ("incorrect call to routine") does: too few arguments, a negative length, count or position (for example `SUBWORD: length must be non-negative, got -1`), a non-numeric argument where a number is required, or `DATATYPE(s,'')`. No string value may be longer than 16 MiB (16,777,216 bytes); a concatenation or a function result (`COPIES`, `SPACE`, `CHANGESTR`, `FORMAT`, `D2C`, `LEFT`, `RIGHT`, `CENTER` and the others) that would be longer raises the SYNTAX condition.

## String Functions

| Function | Returns |
|---|---|
| `ABBREV(s, prefix [,len])` | 1 if *prefix* is a leading part of *s* at least *len* characters long, else 0. |
| `CENTER(s, n [,pad])`<br>`CENTRE(...)` | *s* centered in *n* characters; truncated equally at both ends if longer. |
| `CHANGESTR(needle, s, new [,count])` | *s* with every occurrence (or the first *count*) of *needle* replaced. |
| `COMPARE(s1, s2 [,pad])` | 0 if equal after padding the shorter; otherwise the position of the first difference. |
| `COPIES(s, n)` | *n* copies of *s*. |
| `COUNTSTR(needle, s)` | Number of non-overlapping occurrences. |
| `DELSTR(s, n [,len])` | *s* with *len* characters (default: the rest) deleted from position *n*. |
| `INSERT(new, s [,n [,len [,pad]]])` | *new*, padded to *len*, inserted after position *n* (default 0) of *s*. |
| `JUSTIFY(s, n [,pad])` | The words of *s* spread to fill exactly *n* characters; extra blanks go to the leftmost gaps first. Truncated if too long. |
| `LASTPOS(needle, s [,start])` | Position of the last occurrence at or before *start*, or 0. |
| `LEFT(s, n [,pad])` | The leftmost *n* characters, padded on the right. |
| `LENGTH(s)` | Number of characters. |
| `LOWER(s)` | Lowercase, full Latin-1 (bricks extension). |
| `OVERLAY(new, s [,n [,len [,pad]]])` | *s* with *new* written over it from position *n*, extended if needed. |
| `POS(needle, s [,start])` | Position of the first occurrence at or after *start*, or 0. |
| `REVERSE(s)` | *s* reversed. |

| Function | Returns |
|---|---|
| RIGHT(s, n [,pad]) | The rightmost *n* characters, padded on the left. |
| SPACE(s [,n [,pad]]) | The words of *s* separated by *n* (default 1) pad characters; 0 removes the blanks. |
| STRIP(s [,opt [,char]]) | *s* without leading (L), trailing (T) or both (B, default) occurrences of *char* (default blank). |
| SUBSTR(s, n [,len [,pad]]) | The substring at position *n*, padded to *len*. |
| TRANSLATE(s [,out [,in [,pad]]]) | With one argument, *s* in uppercase (a-z). Otherwise each character of *s* found in *in* (default XRANGE()) is replaced by the character at the same position of *out*, or by *pad*. |
| UPPER(s) | Uppercase, full Latin-1 (bricks extension). |
| VERIFY(s, ref [,opt [,start]]) | Position of the first character of *s* not in *ref* (N, default) or in *ref* (M); 0 if none. |
| XRANGE([from [,to]]) | All characters from *from* (default X'00') to *to* (default X'FF'). |

**Figure 40-3. String functions**

## Word Functions

| Function | Returns |
|---|---|
| `DELWORD(s, n [,len])` | *s* with *len* words (default: the rest) deleted from word *n*. |
| `SUBWORD(s, n [,len])` | Words *n* through *n*+*len*-1 (default: the rest), separated by single blanks. |
| `WORD(s, n)` | Word *n*, or the null string. |
| `WORDINDEX(s, n)` | Position of the first character of word *n*, or 0. |
| `WORDLENGTH(s, n)` | Length of word *n*, or 0. |
| `WORDPOS(phrase, s [,start])` | Word number at which *phrase* first occurs, or 0. |
| `WORDS(s)` | Number of words. |

**Figure 40-4. Word functions**

## Conversion and Bit Functions

| Function | Returns |
|---|---|
| `B2X(bits)` | The binary string as hexadecimal. |
| `BITAND(s1 [,s2 [,pad]])`<br>`BITOR(...)`<br>`BITXOR(...)` | Bitwise AND, OR, exclusive OR; the shorter string is extended with *pad*. |
| `C2D(s [,n])` | The bytes of *s* (the last *n* bytes, if given) as an unsigned decimal number. |
| `C2X(s)` | The bytes of *s* as uppercase hexadecimal. The usual way to test the AID: `IF C2X(EIBAID) = 'F3' THEN` (the F3 key). |
| `D2C(n [,len])` | The whole number as bytes, *len* bytes wide; a negative number is in two's complement. |
| `D2X(n [,len])` | The whole number as uppercase hexadecimal, padded on the left with zeros to *len*. |
| `X2B(hex)` | The hexadecimal string as binary digits. |
| `X2C(hex)` | The hexadecimal string as bytes. An odd number of digits is padded on the left with 0; blanks are allowed between byte pairs. |
| `X2D(hex [,n])` | The hexadecimal string (its last *n* digits, if given) as an unsigned decimal number. |

**Figure 40-5. Conversion and bit functions**

**Difference from IBM REXX.** `C2D(s, n)` and `X2D(hex, n)` do not treat the value as a signed number: `X2D('FFFF', 4)` is 65535, not -1, and `C2D('FF'X, 1)` is 255.

## Arithmetic Functions

| Function | Returns |
|---|---|
| `ABS(n)` | Absolute value. |
| `DIGITS(), FUZZ(), FORM()` | The current `NUMERIC` settings. |
| `FORMAT(n [,before [,after]])` | *n* rounded to *after* decimal places and padded on the left to *before* integer positions. The exponent arguments are accepted and ignored. |
| `INT(n)` | Integer part (bricks extension). |
| `MAX(n, ...), MIN(n, ...)` | Largest, smallest argument. |
| `MOD(a, b)` | The same as `a // b` (bricks extension). |
| `RANDOM([min [,max]])` | A random whole number from *min* to *max* (default 0 to 999); the range may not exceed 100000. |
| `SIGN(n)` | -1, 0 or 1. |
| `TRUNC(n [,places])` | *n* truncated to *places* decimal places (default 0). |

**Figure 40-6. Arithmetic functions**

## Information Functions

| Function | Returns |
|---|---|
| ADDRESS() | The current command environment, normally CICS. Written alone as a clause, ADDRESS() is the ADDRESS instruction; use it in an expression. |
| ARG()<br>ARG(n) | The number of arguments of the program or routine; argument *n*, or the null string. |
| DATATYPE(s) | NUM if *s* is a number, else CHAR. |
| DATATYPE(s, type) | 1 or 0. The types are N (number), W (whole number) and A (ASCII letters and digits only). Any other type returns 0. |
| DATE([opt [,date, 'S']]) | See the table of options below. |
| ERRORTEXT(n) | The string Error followed by *n*. bricks has no table of REXX error messages. |
| SYMBOL(name) | VAR if *name* is a variable that has a value, LIT if it is a valid symbol without a value (or a constant symbol), BAD if it is not a valid symbol. The argument is evaluated once: with J = 3, SYMBOL('J') tests J and SYMBOL(J) tests 3. A compound name is derived as usual. |
| TIME([opt]) | See the table below. |
| VALUE(name [,new]) | The value of the variable whose name is the string *name*; with *new*, also sets it. CALL VALUE 'SCR.ROW' \|\| J, LINE sets a map row whose name is built at runtime. |

**Figure 40-7. Information functions**

| Call | Result (example) |
|---|---|
| `DATE(), DATE('N')` | `22 OCT 2026`: day without a leading zero, month in uppercase. IBM REXX gives `22 Oct 2026`. |
| `DATE('S')` | `20261022` |
| `DATE('E')` | `22/10/26` |
| `DATE('U')` | `10/22/26` |
| `DATE('O')` | `26/10/22` |
| `DATE('B')` | Days since 1 January 0001. |
| `DATE('B', 'yyyymmdd', 'S')` | The base date of the given date; subtract two base dates for the days between them. Only input format `S` is supported. |
| `TIME(), TIME('N')` | `14:05:09` |
| `TIME('L')` | `14:05:09.123456` |
| `TIME('S')` | Seconds since midnight. |

**Figure 40-8. DATE and TIME options**

Any other option (`DATE('W')`, `DATE('D')`, `TIME('E')`, `TIME('H')` and so on) returns the same result as the default option. The date and time are those of the bricks server.

### Stream Functions

The stream functions read and write text files in the sequential-file directory `tmp_dir`, the same files that `READQ TD`, `WRITEQ TD` and `DELETEQ TD` use ("The tmp_dir Directory" in Chapter 15). A stream name is a file name in that directory; a name with a path separator, `..` or a leading period is refused. Lines end with a line feed and may hold printable bricks SBCS characters only; files are stored as UTF-8.

| Function | Action and result |
|---|---|
| LINEIN(name [,1]) | Opens the stream for reading on first use and returns the next line, or the null string at end of file. With 1, starts again at the first line. |
| LINEOUT(name) | Closes the stream. Returns 0. |
| LINEOUT(name, line [,1]) | Opens the stream for appending on first use and writes the line. With 1, the file is first opened for writing from the start. Returns 0. |
| LINES(name) | 1 if data remains to be read (for a stream not yet open: if the file is not empty), else 0. |
| CHARIN(name [,start [,n]]) | The next *n* characters (default 1). A *start* of 1 starts again at the beginning; other values are ignored. |
| CHAROUT(name [,s [,1]]) | Appends *s*; without *s*, closes the stream. Returns 0. |
| CHARS(name) | Bytes of the UTF-8 file not yet read: an upper bound on the characters left. |
| STREAM(name [,'S']) | READY (open), NOTREADY (exists, not open), UNKNOWN (no such file) or ERROR. |
| STREAM(name, 'D') | OK, file not found, or the reason for an error. |
| STREAM(name, 'C', cmd) | OPEN READ, OPEN WRITE, OPEN APPEND, CLOSE and DELETE return READY: or ERROR: and a reason. QUERY EXISTS returns 1 or 0; QUERY SIZE returns the size in bytes. |

**Figure 40-9. Stream functions**

A stream that cannot be opened, read or written (including a refused name) raises the SYNTAX condition, with a message such as LINEIN("x"): ...; it does not raise NOTREADY. Every stream that a program opens is closed when the task ends.

```
/* Append a row, then read every row back.                        */
CALL LINEOUT 'export.txt', 'C0000099|WIDGET-Z|7|9.95'
CALL LINEOUT 'export.txt'                /* close the writer      */
DO WHILE LINES('export.txt') > 0
   ROW = LINEIN('export.txt')
   ...
END
CALL STREAM 'export.txt', 'C', 'CLOSE'
```

### JSON Functions (bricks Extension)

Two functions take values out of a JSON document, typically the body of an HTTP response received with `EXEC CICS WEB CONVERSE` ("WEB CONVERSE" in Chapter 20). The document is parsed on each call. There is no function that builds JSON; build output by concatenation, as the WAPI sample does.

| Function | Returns |
|---|---|
| `JSONGET(json, path)` | The scalar at *path*, a list of keys separated by periods in which a whole number indexes an array from 0. Numbers are formatted as REXX numbers (`671`, not `671.0`); `true` and `false` become 1 and 0. A missing key, an index out of range, `null`, an object or array, or a parse error gives the null string. |
| `JSONCOUNT(json, path)` | The number of elements of the array at *path* (the null path is the whole document), or 0. |

**Figure 40-10. JSON functions**

```
N = JSONCOUNT(RESP, 'data')                /* RESP: the HTTP body */
DO I = 0 TO N - 1
   P   = 'data.' || I
   FLT = JSONGET(RESP, P || '.flight.iata')
   DEP = JSONGET(RESP, P || '.departure.iata')
END
EMSG = JSONGET(RESP, 'error.message')      /* null if no error    */
```

A key that contains a period cannot be reached, array indexes cannot be negative, and whole numbers larger than 2**53 lose precision. The SABR (BOOK) sample uses these functions to read live flight data.

Part 9

# Sample Programs

This part describes the sample transactions supplied with bricks, and works through complete programs that combine the facilities described in the earlier parts.

**Chapter 41. Supplied Sample Transactions** ..... **41-1**
**Chapter 42. Worked Examples** ..... **42-1**

Chapter 41

# Supplied Sample Transactions

bricks is shipped with sample programs in REXX, COBOL and PL/I. Each sample is defined in `runtime/transactions.conf` and can be run by typing its transaction identifier at the blank screen after signing on with CSSN ("The Blank Screen" in Chapter 4). Together the samples exercise every family of commands described in this book. The source is the best guide to the programming model: each program begins with a comment that explains what it shows.

This chapter lists the samples by language. The supplied transactions (CSSN, CEMT, CEDA, CECI, CEDF, ISPF, IDCA and the others) are not samples; they are described in Part 2.

## Sample Definitions

A line of `transactions.conf` has the form

```
TRANSID:language:source-file[:groups[:database[:twasize]]]
```

The language is `rexx`, `cobol` or `pli`; the source file is in `rexx_dir`, `cobol_dir` or `pli_dir` (`runtime/rexx`, `runtime/cobol`, `runtime/pli`). The groups field lists the security groups that may run the transaction; most samples are `PUBLIC`. The optional fifth field names the database that the transaction's `EXEC SQL` statements use ("Connecting a Transaction to a Database" in Chapter 21): BANK, BALC, BALR and BRDS name `bank`, so that their SQL, present or future, goes to the retail-banking database; TOPX names `test3270` and STAR names `blank`. The optional sixth field is the size of the task's TWA in bytes. When a transaction id is defined twice, the first definition is used. The same program may be defined under two identifiers: BOOK and SABR both run `book.rexx`, and BALC (COBOL) and BALR (REXX) are twins. The file `runtime/aliases.conf` adds long names that the operator can type instead: `sabre` for SABR, `customers` for CUST and `custpli` for CUSP. For the configuration files, see "The transactions.conf File" in Appendix D and Chapter 6.

## REXX Samples

| Transid | Source | What it shows |
|---|---|---|
| HELO | hello.rexx | Smallest complete program. ASSIGN of the user, terminal and screen size; sends HELO1 (Model 2), HELO1M (Model 3) or HELO1L (Model 4, with extra panes), falling back to HELO1 on MAPFAIL. |
| CUST | cust.rexx | Customer file maintenance: A=Add, Q=Query, U=Update, D=Delete, L=List, S=Search. READ, WRITE, READ UPDATE and REWRITE, DELETE; LINK to CUSV and CUSL with a COMMAREA. Uses the CUST1L map family on a Model 4. |
| CUSV | cusv.rexx | LINK target: normalizes the customer number in DFHCOMMAREA. |
| CUSL | cusl.rexx | LINK target: lists or searches the customers file with STARTBR, READNEXT, ENDBR; pages with F7/F8 (15 rows, or 35 on a Model 4); returns the match count in the COMMAREA. |
| QAGE, QAGR | qage.rexx, qagr.rexx | A pseudo-conversational pair: QAGE prompts for a birth date and returns with TRANSID('QAGR'); QAGR receives the map, computes the age with DATE('B'), and redisplays the prompt on an error. |
| PROD, CONS | prod.rexx, cons.rexx | A temporary storage producer and consumer: WRITEQ TS; READQ TS NEXT, DELETEQ TS (F4), rewind by RETURN TRANSID('CONS') (F5). |
| GETC | getc.rexx | RECEIVE of the command line (GETC 100), READ, and SEND TEXT without a map. |
| EXAR | exam.rexx | Echoes the arguments typed after the transaction id with PARSE ARG and SEND TEXT. |

| Transid | Source | What it shows |
|---|---|---|
| INQR | inqr.rexx | INQUIRE TASK, and a STARTBROWSE FILE / INQUIRE FILE NEXT / ENDBROWSE loop over every file. |
| TIMR | timr.rexx | START with INTERVAL and FROM; RETRIEVE tells a first entry (ENDDATA) from a scheduled one. Twin of TIMC. |
| SCHD | schd.rexx | START ... AFTER SECONDS(n), ADDRESS() and SYMBOL(), one PASS/FAIL line each. |
| FUNC | func.rexx | PASS/FAIL checks of hexadecimal and binary strings, function calls of internal routines, compound PARSE VAR sources, and blank-preserving PARSE. |
| CHAT | chat.rexx | Multi-user chat that refreshes itself every 2 seconds with START INTERVAL(000002) and SEND MAP DATAONLY, so the operator's typing is kept. Messages are kept in the file CHATLOG. Model 2 and Model 4 layouts. |
| SQLR | sqlr.rexx | EXEC SQL SELECT INTO with a null indicator, WHENEVER, and a SELECT over SQLCODE values. Twin of SQLD. |
| ESDR | esdr.rexx | Browse of a sequential file (audit.log in tmp_dir) by relative byte address, forward and backward. Twin of ESDC. |
| WAPI | wapi.rexx | A REST service for GET /api/customer/{id} (web_routes.conf): WEB READ QUERYPARM, a JSON reply built by concatenation, WEB SEND. |

| Transid | Source | What it shows |
|---|---|---|
| BOOK, SABR | book.rexx | A SABRE-style airline reservation system: a pseudo-conversational command interpreter that keeps its state in the COMMAREA, VSAM files, and live flight data read with WEB CONVERSE and JSONGET. |
| BALR | balc.rexx | Overrides the color of one map field at runtime (SCR.STATUS_C). Twin of BALC. |
| BRDS | brds.rexx | A general browser for any key-sequenced file, with options for the field separator and start key. |
| CSGM | csgm.rexx | INQUIRE SYSTEM GMMTEXT: shows the good-morning text. |
| CSSR | cssr.rexx | VERIFY PASSWORD: asks the signed-on user for the password again. |
| HELP | help.rexx | A help screen for all users. |
| TIME | time.rexx | ASKTIME and FORMATTIME. |
| QUEN | quen.rexx | Counts the solutions of the N-queens problem for QUEN n; RECEIVE and SEND TEXT only. |
| MNDL, MNDU | mndl.rexx, mndu.rexx | Draws the Mandelbrot set in text (CONVERSE, SEND TEXT); MNDU exports, imports and deletes saved settings. |
| FSPF, ESPF | fspf.rexx, espf.rexx | A menu in the style of ISPF with a clock kept current by a background START; ESPF (ADMIN) edits the menu. |
| ABMR | abmr.rexx | Abend ABM0: RECEIVE MAP('NOPE') MAPSET('HELO1') names a map that the mapset does not contain. The abend is not a condition: RESP does not prevent it, and it ends a REXX task. |

| Transid | Source | What it shows |
|---|---|---|
| SPDB | spdb.rexx | Abend ASP3: two INSERTs of one key into a table whose unique key PostgreSQL checks only at commit. The SYNCPOINT is refused; with nothing else pending, the database alone backs out, and the HANDLE ABEND exit is not driven. |
| ABUR | abur.rexx | A user abend: EXEC CICS ABEND ABCODE('USR1'). The abend takes a transaction dump; CAID cannot know what an application code means and lists the typical causes. |

**Figure 41-1. REXX sample transactions**

The REXX samples test EIBRESP after each command without coding RESP, which is valid in REXX because an unhandled condition does not abend a REXX task ("Unhandled CICS Conditions" in Chapter 39). The file runtime/rexx/test2.rexx is empty and is not defined as a transaction.

## COBOL Samples

| Transid | Source | What it shows |
|---|---|---|
| HELC | hello.cob | COBOL twin of HELO: ASSIGN into WORKING-STORAGE, SEND MAP('HELO1') FROM(SCR), RETURN. |
| GUST | gust.cob | COBOL twin of CUST. Codes RESP(WS-CICS-RESP) on the file commands and tests RESP-NOTFND, RESP-DUPREC; QUERY SECURITY refuses D=Delete to users who may not update GUST; LINK to GUSV and GUSL. |
| GUSV | gusv.cob | LINK target: normalizes the customer number in the COMMAREA. |
| GUSL | gusl.cob | LINK target: list and search with STARTBR/READNEXT/ENDBR, RESP(WS-CICS-RESP), F7/F8 paging by restarting the browse; returns the match count in the COMMAREA. |
| QAGC | qagc.cob | The QAGE/QAGR age calculation in one conversational program. |
| EXAM | exam.cob | RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) of the command line and UNSTRING into arguments; RESP-EOC when there is nothing to read. |
| ORDR | ordr.cob | Imports orders.sample.txt from tmp_dir with READQ TD and WRITE FILE('ORDERS'), both with RESP(WS-CICS-RESP). See "Example 5. Sequential Import with READQ TD and WRITE" in Chapter 42. |
| TIMC | timc.cob | CONVERSE, START INTERVAL FROM and RETRIEVE (with RESP(WS-CICS-RESP)). Twin of TIMR. |
| SQLD | sqld.cob | EXEC SQL SELECT INTO with a null indicator, WHENEVER, an EVALUATE over SQLCODE, COPY SQLCA. Needs the customers_sql table (Chapter 21). |

| Transid | Source | What it shows |
|---|---|---|
| GMPT | gmpt.cob | GETMAIN SET, SET ADDRESS OF a based item, FREEMAIN, ADDRESS COMMAREA and ADDRESS EIB. |
| ESDC | esdc.cob | Sequential-file browse by relative byte address with RESP(WS-CICS-RESP). Twin of ESDR. |
| WZEN | wzen.cob | HTTP client: WEB OPEN, WEB CONVERSE (GET /zen from GitHub), WEB CLOSE, each with RESP(WS-CICS-RESP). |
| WHDR | whdr.cob | WEB WRITE HTTPHEADER on a request and WEB READ HTTPHEADER on the response, including a header that is not there, with RESP(WS-CICS-RESP). |
| TODO | todo.cob | A multi-user to-do REST service under /api/todo/ on three VSAM files: JSON GENERATE, JSON PARSE, EVALUATE ... ALSO, SYNCPOINT ROLLBACK. |
| BANK | bank.cob | A pseudo-conversational retail-banking application that keeps its state in the COMMAREA; database bank. |
| BALC | balc.cob | Runtime field color through the STATUS-C field. Twin of BALR. |
| TOPX | topx.cob | A read-only browser of the 3270BBS topics database (test3270) with EXEC SQL cursors, for Model 2 and Model 4. |
| UPPR | uppr.cob | INSPECT ... CONVERTING for Latin-1 case folding. |
| STAR | star.cob | A Space Invaders game: a real-time loop built on SEND MAP ... TIMEOUT(100). |
| CHEL | help.cob | A COBOL help screen painted on the CUST1 map. |

| Transid | Source | What it shows |
|---|---|---|
| ABNC | abnc.cob | An unhandled condition: READ of a missing key with no RESP, no NOHANDLE and no HANDLE CONDITION. The task abends AEIM and the terminal shows DFHAC2206. |
| ABNH | abnh.cob | The same READ under HANDLE ABEND LABEL(RECOVER). The paragraph saves EIBRESP (13) before ASSIGN ABCODE resets it, and shows CAUGHT AEIM. |
| APCH | apch.cob | HANDLE ABEND PROGRAM('NOPGM') names a program that is not defined. ABEND ABCODE('TEST') then ends the task APCT; the log keeps the original code. |
| SPLC | splc.cob | Abend ASP7: the refused commit of SPDB, with a WRITE to the customers file pending as well. The file update is backed out and no HANDLE ABEND exit is driven. |

**Figure 41-2. COBOL sample transactions**

The COBOL samples copy DFHAID and DFHRESP ("Supplied Copybooks" in Chapter 26) instead of coding hexadecimal AID values and numeric responses. A COBOL program that tests a response must code RESP on the command, because an unhandled condition abends a COBOL task before the test is reached; GUST, GUSL, ORDR, ESDC, TIMC, WZEN and WHDR declare 01 WS-CICS-RESP PIC S9(8) COMP for this purpose. The sources compt.cob (a COMP-3 and ABSTIME example) and star_original.cob have no transaction definition.

## PL/I Samples

| Transid | Source | What it shows |
|---|---|---|
| CUSP | cusp.pli | CUST, CUSV and CUSL in one PL/I program on the same maps and file. Validation and listing are internal procedures; every command codes RESP(RC) and tests DFHRESP(...); %INCLUDE DFHAID. Alias custpli. See "Example 7. Porting CUST to PL/I" in Chapter 42. |
| ABNP | abnp.pli | The unhandled NOTFND of ABNC in PL/I: abend AEIM. |
| ABMP | abmp.pli | Abend ABMB: SEND TEXT ... CURSOR(9999), beyond the end of any screen. Cursor positions count from 0. |
| ABZP | abzp.pli | Abend ABZD (ZERODIVIDE) in the internal procedure AVERAGE, whose divisor COUNT is 0. CAID names the divisor. |
| NQUE | nqueen.pli | The N-queens puzzle for boards of 5 to 11, pseudo-conversational with the state in the COMMAREA. It paints Solving ..., please wait. with SEND MAP ... DATAONLY, which leaves the keyboard locked while it searches, and draws the solutions with MARK=Q:YELLOW. |

**Figure 41-3. PL/I sample transactions**

PL/I sources are in pli_dir (runtime/pli) and are edited in ISPF area 4 ("The ISPF Menu" in Chapter 9); the include members DFHAID and DFHRESP are in pli_include_dir (runtime/pliinc).

## The Abend Samples

The samples ABNC, ABNH, ABNP, ABMR, APCH, ABMP, ABZP, ABUR, SPDB and SPLC each end in an abend on purpose; a message such as ABNC: READ RETURNED – NOT EXPECTED appears only if the abend did not occur. They are listed together here as a test of the abend behavior described in "Abends" in Chapter 16 and Appendix C, and of the transaction dumps that CAID analyses ("CAID: The Abend Analyser" in Chapter 16):

| Transid | Language | Abend | Cause |
|---|---|---|---|
| ABNC | COBOL | AEIM | Unhandled NOTFND. |
| ABNP | PL/I | AEIM | Unhandled NOTFND. |
| ABNH | COBOL | AEIM | Caught by HANDLE ABEND LABEL; the task ends normally. |
| APCH | COBOL | APCT | HANDLE ABEND PROGRAM exit not loadable. |
| ABMR | REXX | ABM0 | Map not in mapset. |
| ABMP | PL/I | ABMB | Cursor beyond the screen. |
| ABZP | PL/I | ABZD | Division by zero. |
| ABUR | REXX | USR1 | EXEC CICS ABEND ABCODE('USR1'). |
| SPDB | REXX | ASP3 | Commit refused, database only. |
| SPLC | COBOL | ASP7 | Commit refused with a file update pending. |

**Figure 41-4. Abend samples**

SPDB and SPLC need a database connection ("Refused Commits" in Chapter 21); they create the table `spdb_demo` if it does not exist.

### Sample Screens

The figures in this section show some of the samples as they appear on a terminal, signed on as the user `DEMO`. HELO sends the map `HEL01` on a Model 2 terminal and `HEL01L` on a Model 4 terminal; both screens are shown in Figure 3-3 and Figure 3-4 in Chapter 3. On a Model 3 terminal it sends `HEL01M`.

CUST (REXX), GUST (COBOL) and CUSP (PL/I) share one customers file and one set of maps. Figure 41-5 shows the menu, here as painted by CUST; Figure 41-6 and Figure 41-7 show the same menu under GUST and CUSP. In the GUST menu the terminal id is `T000`, because GUST receives it with `ASSIGN TERMID` into a four-character field, `TRM PIC X(4)`. An eight-character field gives the full identifier (see "COBOL and PL/I" in Appendix A).

```
User: DEMO                            Term: T0019

  Customer Database          CUST Transaction

       (A=Add Q=Query U=Update D=Delete L=List S=Search)
  Customer #:
  Search:









 ENTER=Continue   F12=Exit
```

**Figure 41-5. The CUST menu (REXX)** (Model 2, 24 x 80; capture screens/cust-menu.txt)

```
User: DEMO                                Term: T000

  Customer Database          CUST Transaction

       (A=Add Q=Query U=Update D=Delete L=List S=Search)
  Customer #:
  Search:









 ENTER=Continue   F12=Exit
```

**Figure 41-6. The GUST menu (COBOL)** (Model 2, 24 x 80; capture screens/gust-menu.txt)

```
User: DEMO                            Term: T0021

  Customer Database          CUST Transaction

       (A=Add Q=Query U=Update D=Delete L=List S=Search)
  Customer #:
  Search:









 ENTER=Continue   F12=Exit
```

**Figure 41-7. The CUSP menu (PL/I)** (Model 2, 24 x 80; capture screens/cusp-menu.txt)

```
  Customer Detail
  Customer #:               C1003

  Name:                     Maria Garcia
  Address:                  7 Harbor Road
  City:                     Portsmouth
  Phone:                    555-0107

              Mode: QUERY -- press ENTER to return to the menu.


 ENTER=Save   PF3=Cancel
```

**Figure 41-8. GUST customer detail in QUERY mode** (Model 2, 24 x 80; capture screens/gust-detail.txt)

The list action (L) links to CUSL from CUST, or to GUSL from GUST. Figure 41-9 shows the first page of the REXX list and Figure 41-10 the COBOL list; **F8** shows the next page.

```
CUSL -- customer list   page 1 of 2    total: 18

  Customers

  CUST#    NAME                       CITY              PHONE
  C1001    Alice Smith                Springfield       555-0100
  C1002    Robert Jones               Shelbyville       555-0142
  C1003    Maria Garcia               Portsmouth        555-0107
  C1004    Chen Wei                   Riverside         555-0118
  C1005    Fatima Khan                Lakewood          555-0125
  C1006    John O'Brien               Greenville        555-0109
  C1007    Sofia Rossi                Fairview          555-0133
  C1008    Hans Mueller               Franklin          555-0105
  C1009    Aiko Tanaka                Madison           555-0120
  C1010    Liam Murphy                Clinton           555-0161
  C1011    Olivia Brown               Springfield       555-0114
  C1012    Noah Wilson                Georgetown        555-0188
  C1013    Emma Davis                 Salem             555-0103
  C1014    Lucas Martin               Ashland           555-0127
  C1015    Ava Thompson               Dover             555-0140


  PF7=PageUp  PF8=PageDown  PF3=Exit
```

**Figure 41-9. CUSL customer list, page 1** (Model 2, 24 x 80; capture screens/cusl-list.txt)

```
GUSL -- customers (PF7=Prev PF8=Next PF3=Exit).

  Customers

  CUST#    NAME                       CITY              PHONE
  C1001    Alice Smith                Springfield       555-0100
  C1002    Robert Jones               Shelbyville       555-0142
  C1003    Maria Garcia               Portsmouth        555-0107
  C1004    Chen Wei                   Riverside         555-0118
  C1005    Fatima Khan                Lakewood          555-0125
  C1006    John O'Brien               Greenville        555-0109
  C1007    Sofia Rossi                Fairview          555-0133
  C1008    Hans Mueller               Franklin          555-0105
  C1009    Aiko Tanaka                Madison           555-0120
  C1010    Liam Murphy                Clinton           555-0161
  C1011    Olivia Brown               Springfield       555-0114
  C1012    Noah Wilson                Georgetown        555-0188
  C1013    Emma Davis                 Salem             555-0103
  C1014    Lucas Martin               Ashland           555-0127
  C1015    Ava Thompson               Dover             555-0140


  PF7=PageUp  PF8=PageDown  PF3=Exit
```

**Figure 41-10. GUSL customer list, page 1** (Model 2, 24 x 80; capture screens/gusl-list.txt)

The abend samples end with the DFHAC2206 screen, which names the transaction, the abend code and the command that failed. Figure 16-8 shows ABNC, whose unhandled `NOTFND` abends the task `AEIM`. Figure 16-7 shows ABNH, where the `HANDLE ABEND` exit catches the same abend; `EIBRESP` still holds 13. Figure 41-11 and Figure 41-12 show the REXX samples ABMR and SPDB: REXX tasks end with these abends too, because they are not conditions.

```
DFHAC2206 19:07:49 BRICKS01 Transaction ABMR failed with abend ABM0.
  Updates to local recoverable resources backed out.

Detail: RECEIVE MAP: map "NOPE" not in mapset "HELO1"
```

**Figure 41-11. ABMR ends with abend ABM0** (Model 2, 24 x 80; capture screens/abend-abm0.txt)

```
DFHAC2206 19:08:11 BRICKS01 Transaction SPDB failed with abend ASP3.
  Updates to local recoverable resources backed out.

Detail: SYNCPOINT: DATABASE BACKED OUT UOW, SQLSTATE 23505
```

**Figure 41-12. SPDB ends with abend ASP3** (Model 2, 24 x 80; capture screens/abend-asp3.txt)

Chapter 42

# Worked Examples

This chapter takes the supplied samples apart one technique at a time. Each example is an excerpt of a program in `runtime/`, shortened where `...` appears. The complete programs are listed in Chapter 41.

### Example 1. Producer and Consumer on a TS Queue

PROD (`prod.rexx`) writes one item to a temporary storage queue each time the operator presses **`ENTER`**. CONS (`cons.rexx`) reads the queue one item at a time. Both programs are conversational: each loops on `SEND MAP` and `RECEIVE MAP` until **`F3`**.

```
/* PROD: write one item per ENTER                                  */
DO FOREVER
  ...
  EXEC CICS SEND MAP('PROD1') FROM(SCR.) ERASE END-EXEC
  EXEC CICS RECEIVE MAP('PROD1') END-EXEC
  IF C2X(EIBAID) = 'F3' THEN DO
    EXEC CICS RETURN END-EXEC
  END
  QNM = STRIP(MAP.QNAME)
  PAY = STRIP(MAP.PAYLOAD)
  ...
  EXEC CICS WRITEQ TS QUEUE(QNM) FROM(PAY) ITEM(ITEMNO) END-EXEC
  ...
END
```

`RECEIVE MAP` without `INTO` places the fields in the stem `MAP.`. `ITEM(ITEMNO)` on `WRITEQ TS` receives the number of the item written.

```
/* CONS: read the next item with the browse cursor                 */
EXEC CICS READQ TS QUEUE(QNM) INTO(REC) NEXT ITEM(GOTI) END-EXEC
SELECT
  WHEN EIBRESP = 0 THEN DO
    LITM = GOTI
    LPAY = LEFT(REC, 60)
    NRD  = NRD + 1
  END
  WHEN EIBRESP = 26 THEN DO        /* ITEMERR -- queue exhausted */
    LITM = '(end)'
    LPAY = ''
  END
  WHEN EIBRESP = 44 THEN DO        /* QIDERR -- queue absent     */
    LITM = '(qiderr)'
    LPAY = ''
  END
  OTHERWISE NOP
END
```

`NEXT` reads the item after the last one that this task read; `ITEM(GOTI)` receives its number. The read position belongs to the task, so **`F5`** in CONS starts again at item 1 by ending the task with `EXEC CICS RETURN TRANSID('CONS')`. **`F4`** deletes the queue with `DELETEQ TS`. The local variables (`LITM`, `LPAY`, `NRD`) are named differently from the map fields (`LASTITEM`, `LASTPAY`, `NREAD`); see "Derived Names of Compound Symbols" in Chapter 36.

### Example 2. SEND TEXT and RECEIVE Without a Map

GETC (`getc.rexx`) takes a customer number from the command line (`GETC 100`), reads the customers file, and shows the record as text. The first `RECEIVE` of a task started from the blank screen returns the line the operator typed, transaction id included. Because a 3270 has no line-feed character, the program builds one buffer of 80-character rows:

```
EXEC CICS RECEIVE INTO(BUF) END-EXEC
IF EIBRESP = 6 THEN DO                      /* EOC: nothing typed   */
  ...
END
PARSE VAR BUF TID CKEY .
CKEY = STRIP(CKEY)
...
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) END-EXEC
IF EIBRESP = 13 THEN DO                     /* NOTFND               */
  ...
END
PARSE VAR REC NM '|' AD '|' CY '|' PH
TXT = LEFT('Customer details for #' || CKEY, 80)
TXT = TXT || LEFT('', 80)
TXT = TXT || LEFT('Name:    ' || NM, 80)
TXT = TXT || LEFT('Address: ' || AD, 80)
...
EXEC CICS SEND TEXT FROM(TXT) ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

The COBOL sample EXAM (`exam.cob`) receives the command line in the same way and splits it with `UNSTRING`:

```
           EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC.

           IF EIBRESP = RESP-EOC THEN
               MOVE '(no input -- chained dispatch)' TO WS-INPUT
           END-IF.
           ...
           UNSTRING WS-INPUT DELIMITED BY ' '
               INTO TRANSID A B C
           END-UNSTRING.
```

No RESP is needed for EOC, because the IBM default action for EOC is to continue.

### Example 3. Browsing a File

CUSL (cusl.rexx) reads every record of the customers file in key order and keeps the ones that contain the search term:

```
KEYS.0 = 0
EXEC CICS STARTBR FILE('customers') END-EXEC
DONE = 0
DO WHILE DONE = 0
  EXEC CICS READNEXT FILE('customers') INTO(REC) RIDFLD(KEY) END-EXEC
  IF EIBRESP \= 0 THEN DONE = 1
  ELSE DO
    HAY = UPPER(KEY || ' ' || REC)
    IF FILTER = '' | POS(FILTER, HAY) > 0 THEN DO
      N = KEYS.0 + 1
      KEYS.0 = N
      KEYS.N = KEY
      RECS.N = REC
    END
  END
END
EXEC CICS ENDBR FILE('customers') END-EXEC
```

The loop ends on ENDFILE (20) or any other response by a flag, not by LEAVE inside the ELSE DO group, which would end only the group ("LEAVE and ITERATE" in Chapter 37). To browse only the keys that begin with a prefix, start the browse with a generic key; READNEXT then returns ENDFILE as soon as the prefix no longer matches:

```
KL = LENGTH(FILTER)
EXEC CICS STARTBR FILE('customers') RIDFLD(FILTER) GENERIC
          KEYLENGTH(KL) END-EXEC
```

An option value is a constant or a variable name, not an expression: compute the key length into a variable first. (KEYLENGTH(LENGTH(FILTER)), as written in a comment in cusl.rexx, would pass the text LENGTH (FILTER ) as a constant.)

Each page of the list is then built with VALUE, because the map field names ROW1 to ROW15 are computed:

```
DO J = 1 TO ROWS_PER_PAGE
  IDX = START + J - 1
  LINE = ''
  IF IDX <= TOTAL THEN DO
    K = KEYS.IDX
    R = RECS.IDX
    PARSE VAR R NM '|' AD '|' CY '|' PH
    LINE = LEFT(K,8) LEFT(NM,28) LEFT(CY,18) LEFT(PH,14)
  END
  CALL VALUE 'SCR.ROW' || J, LINE
END
```

### Example 4. LINK With a COMMAREA

CUST (`cust.rexx`) passes the customer number to CUSV for validation. CUSV changes `DFHCOMMAREA`, and the caller's variable receives the changed value when CUSV returns:

```
/* CUST */
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(CKEY) END-EXEC

/* CUSV */
DFHCOMMAREA = UPPER(STRIP(DFHCOMMAREA))
EXEC CICS RETURN END-EXEC
```

A linked program cannot make the COMMAREA longer than the caller passed it. CUST therefore pads the variable in which CUSL returns the match count to the width of the search field:

```
CMSG = LEFT(SRCH, 30)
EXEC CICS LINK PROGRAM('CUSL') COMMAREA(CMSG) END-EXEC
CMSG = STRIP(CMSG)
IF SRCH = '' THEN DO
  ...
END
ELSE IF CMSG = '0' THEN SCR.MSG = 'No matching records found.'
ELSE SCR.MSG = 'Search:' SRCH '(' || CMSG || ' matches)'
```

The COBOL twins GUST, GUSV and GUSL use the same protocol with `PIC X` fields; see "LINK" in Chapter 12 for `LINK`.

### Example 5. Sequential Import with READQ TD and WRITE

ORDR (`ordr.cob`) loads the file `orders.sample.txt` from `tmp_dir` into the key-sequenced file `ORDERS`, keyed on the customer identifier. The input lines look like this:

```
C0000001|WIDGET-A|10|19.95
C0000002|WIDGET-B|2|249.00
...
```

After a confirmation screen, the program runs `PERFORM IMPORT-ONE UNTIL DONE-FLAG = 'Y'`. Both commands code `RESP(WS-CICS-RESP)`, declared after `COPY DFHRESP` as `01 WS-CICS-RESP PIC S9(8) COMP`, so that end of file and duplicate keys do not abend the task:

```
       IMPORT-ONE.
           MOVE SPACES TO REC.
           EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC)
                     RESP(WS-CICS-RESP) END-EXEC.
           IF EIBRESP = RESP-QZERO THEN
               MOVE 'Y' TO DONE-FLAG
           END-IF.
           IF EIBRESP = RESP-NORMAL THEN
               COMPUTE N-READ = N-READ + 1
               MOVE SPACES TO CUST-ID
               MOVE SPACES TO PRODUCT
               MOVE SPACES TO QTY
               MOVE SPACES TO PRICE
               UNSTRING REC DELIMITED BY '|'
                   INTO CUST-ID PRODUCT QTY PRICE
               END-UNSTRING
               PERFORM WRITE-ORDER
           END-IF.
           IF EIBRESP NOT = RESP-NORMAL
              AND EIBRESP NOT = RESP-QZERO THEN
               MOVE 'Y' TO DONE-FLAG
           END-IF.

       WRITE-ORDER.
           MOVE SPACES TO OREC.
           STRING PRODUCT DELIMITED BY SIZE
                  '|' DELIMITED BY SIZE
                  QTY DELIMITED BY SIZE
                  '|' DELIMITED BY SIZE
                  PRICE DELIMITED BY SIZE
               INTO OREC
           END-STRING.
           EXEC CICS WRITE FILE('ORDERS') FROM(OREC) RIDFLD(CUST-ID)
                     RESP(WS-CICS-RESP) END-EXEC.
           IF EIBRESP = RESP-NORMAL THEN
               COMPUTE N-WRITE = N-WRITE + 1
           END-IF.
           IF EIBRESP = RESP-DUPREC THEN
               COMPUTE N-DUP = N-DUP + 1
           END-IF.
```

- `READQ TD` opens the file on its first use and returns `QZERO` (`EIBRESP` 23) at end of file. The file is closed when the task ends.
- Any response other than `NORMAL` or `QZERO`, such as `IOERR`, also stops the loop, so that a failing file does not make the loop spin.
- `DUPREC` (14) is counted, not treated as an error. A `WRITE` of an existing key changes nothing, so running ORDR again on the same data adds no records.
- ORDR is defined `PUBLIC` so that the sample runs at once. A transaction that imports real data should be limited to a group, for example `ORDR:cobol:ordr.cob:ADMIN`.
- A REXX program can read the same file with `LINEIN` ("Stream Functions" in Chapter 40) and write with `EXEC CICS WRITE`; both use the same `tmp_dir` files.

### Example 6. Menu Actions Controlled by QUERY SECURITY

GUST (`gust.cob`) offers A=Add, Q=Query, U=Update, D=Delete, L=List and S=Search. Delete should be allowed only to users who may update the GUST resource. Instead of testing the user identifier in the program, GUST asks the security manager with `QUERY SECURITY`:

```
       WORKING-STORAGE SECTION.
       COPY DFHAID.
       COPY DFHRESP.
       01 WS-CICS-RESP PIC S9(8) COMP.  *> RESP(...) target
       COPY DFHVALUE.
       ...
       01 WS-CACR-UP PIC S9(8) VALUE 0.
       ...
           IF VALID-ACTION = 'Y' AND ACTION = 'D' THEN
               EXEC CICS QUERY SECURITY RESOURCE('GUST')
                                        UPDATE(WS-CACR-UP) END-EXEC
               IF WS-CACR-UP NOT = DFHVALUE-UPDATABLE THEN
                   MOVE 'Delete not permitted for this user.' TO MSG
                   MOVE 'N' TO VALID-ACTION
               END-IF
           END-IF.
```

The test runs before the action is dispatched. A user who is not allowed sees the message on the next menu; other users go on to the delete. The same four lines protect any destructive action of any transaction: replace `'GUST'` with the transaction id. The constants are in `runtime/cobolcopy/DFHVALUE.cpy`; `QUERY SECURITY` and the way bricks decides the access are described in "QUERY SECURITY" in Chapter 13.

### Example 7. Porting CUST to PL/I

The REXX application CUST is three programs: `cust.rexx` (menu and single-record actions), `cusv.rexx` (key validation, reached by `LINK`) and `cusl.rexx` (list and search, reached by `LINK`). CUSP (`runtime/pli/cusp.pli`) ports all three into one PL/I program. It shows the same `CUST1`, `CUST2` and `CUSTL` screens and uses the same customers file, so that a record added under one transaction is found under the other.

```
CUSP:pli:cusp.pli:PUBLIC        (runtime/transactions.conf)
custpli:CUSP                    (runtime/aliases.conf)
```

**Program shape.** Declarations (map structures, then request and list state), a conversational main loop, then internal procedures:

```
   DO FOREVER;
      CALL SEND_MENU;
      MENU_IN = '';
      EXEC CICS RECEIVE MAP(M_MENU) INTO(MENU_IN) RESP(RC);

      /* PF12 from the menu ends the transaction.                    */
      IF EIBAID = PF12 THEN EXEC CICS RETURN;

      AKT  = UPPERCASE(TRIM(MENU_IN.ACTION));
      CKEY = UPPERCASE(TRIM(MENU_IN.CUSTNO));
      SRCH = TRIM(MENU_IN.SEARCH);
      CALL DISPATCH;
   END;
```

**LINK becomes an internal CALL.** CUSV only normalized the key in the COMMAREA. In PL/I that is a procedure with a `CHAR(*) VAR` parameter, passed by reference:

```
   VALIDATE_KEY: PROC(K);
      DCL K CHAR(*) VAR;
      K = UPPERCASE(TRIM(K));
      PUT SKIP LIST('CUSV: validated customer number =', K);
   END VALIDATE_KEY;
```

CUSL becomes `LIST_CUSTOMERS(TERM, COUNT)`, which returns the match count in its second parameter.

**RESP on every command.** `cust.rexx` tests `EIBRESP` against numbers, which is valid in REXX. In PL/I an unhandled `NOTFND` abends the task, so CUSP codes `RESP(RC)` and compares with `DFHRESP`:

```
QUERY_CUSTOMER: PROC;
   EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) RESP(RC);
   IF RC = DFHRESP(NOTFND) THEN
      MENU.MSG = 'Customer ' || CKEY || ' not found.';
   ELSE IF RC <> DFHRESP(NORMAL) THEN
      MENU.MSG = 'READ failed RESP=' || CHAR(RC);
   ...
```

**Stems become structures and arrays.** The REXX list builds field names at runtime (`CALL VALUE 'SCR.ROW' || J, LINE`). The PL/I list structure declares `2 ROW(35) CHAR(71)` once, and the map's `ROW1` to `ROW15` (35 rows on `CUSTLL`) are taken from its elements ("Maps as Structures" in Chapter 33). `cusl.rexx` reads every record into the stems `KEYS.` and `RECS.`; CUSP keeps its storage fixed by counting matches in one browse and building each page with a new `STARTBR`, `READNEXT`, `ENDBR` that skips the earlier pages.

**PARSE becomes INDEX and SUBSTR.** The record is `name|address|city|phone`. The REXX template `PARSE VAR R NM '|' AD '|' CY '|' PH` becomes a field cutter called four times:

```
CUT_FIELD: PROC(S, F);
   DCL (S, F) CHAR(*) VAR;
   DCL P FIXED BIN(31);
   P = INDEX(S, '|');
   IF P = 0 THEN DO;
      F = S;
      S = '';
   END;
   ELSE DO;
      F = SUBSTR(S, 1, P - 1);
      S = SUBSTR(S, P + 1);
   END;
END CUT_FIELD;
```

**Map fallback and the CUSM abend.** Each send tries the map for the screen size first and falls back to the 24x80 map on `MAPFAIL`. If that also fails, the map definitions are broken, and CUSP abends `CUSM` rather than loop on a screen it cannot show:

```
SEND_MENU: PROC;
   EXEC CICS SEND MAP(M_MENU) FROM(MENU) ERASE RESP(RC);
   IF RC = DFHRESP(MAPFAIL) & M_MENU <> 'CUST1' THEN DO;
      M_MENU = 'CUST1';
      EXEC CICS SEND MAP(M_MENU) FROM(MENU) ERASE RESP(RC);
   END;
   IF RC <> DFHRESP(NORMAL) THEN EXEC CICS ABEND ABCODE('CUSM');
END SEND_MENU;
```

Run the program by typing `CUSP` or `custpli` at the blank screen. Edit it in ISPF area 4 and check it with `go run ./cmd/brickscompile runtime/pli/cusp.pli`.

***REXX and PL/I Idioms Compared***

| REXX (CUST) | PL/I (CUSP) |
|---|---|
| `PARSE VAR R NM '\|' AD '\|' ...` | `P = INDEX(S, '\|'); F = SUBSTR(S, 1, P − 1);` |
| `POS(FILTER, HAY) > 0` | `INDEX(UPPERCASE(K \|\| ' ' \|\| R), FILTER) > 0` |
| `UPPER(STRIP(X))` | `UPPERCASE(TRIM(X))` |
| `LEFT(K, 8) LEFT(NM, 28)` | `LEFT(K, 8) \|\| ' ' \|\| LEFT(FNM, 28)` (no blank concatenation) |
| `SCR. = ''; SCR.MSG = ...` | `DCL 1 MENU, 2 MSG CHAR(60), ...;` then `MENU = '';` |
| `CALL VALUE 'SCR.ROW' \|\| J, LINE` | `LST.ROW(J) = LINE;` |
| `KEYS.IDX, RECS.IDX` | A new browse for each page, fixed storage. |
| `SAY 'n =' N` | `PUT SKIP LIST('n =', N);` Both are discarded in a transaction. |
| `LINK PROGRAM('CUSV') COMMAREA(CKEY)` | `CALL VALIDATE_KEY(CKEY);` |
| `LINK PROGRAM('CUSL') COMMAREA(CMSG)` | `CALL LIST_CUSTOMERS(SRCH, N);` |
| `... NOHANDLE; IF EIBRESP = 36` | `... RESP(RC);` and `IF RC = DFHRESP(MAPFAIL)` |
| `IF EIBRESP = 13` | `IF RC = DFHRESP(NOTFND)` |
| `IF C2X(EIBAID) = '7C'` | `%INCLUDE DFHAID;` then `IF EIBAID = PF12` |
| `'Customer' CKEY 'added.'` | `'Customer ' \|\| CKEY \|\| ' added.'` |
| `NPAGES = (TOTAL + ROWS − 1) % ROWS` | `NPAGES = (TOTAL + ROWS − 1) / ROWS;` into a `FIXED BIN` target, which drops the fraction |

**Figure 42-1. REXX and PL/I idioms compared**

**Appendix A**

# Pitfalls and Idioms

This appendix collects the mistakes that programmers most often make when they write bricks programs, and the idiom that avoids each one. The items are grouped by language. Each refers to the chapter that gives the full rules.

## All Languages

### *A 3270 has no line feed*

`SEND TEXT` lays its data out as one buffer, row after row: each group of *cols* characters starts a new row. A line-feed character (`'0A'X`, `'15'X`) does not start a new line. To show several lines, pad each one to the screen width and concatenate them: `LEFT(s, 80)` in REXX, an 80-character `PIC X(80)` item for each line in COBOL, `CHAR(80)` in PL/I. See `getc.rexx` and "SEND TEXT" in Chapter 11. A screen field or text must not reach beyond column 79; text that does wraps to the next row.

### *Cursor positions count from 0*

`CURSOR(n)` on `SEND MAP`, `SEND TEXT` and `SEND CONTROL`, and `EIBCPOSN`, count from 0, as on IBM CICS: `CURSOR(0)` is row 1, column 1, and on an 80-column screen row *r*, column *c* (both from 1) is `(r-1)*80 + c-1`. A position beyond the screen abends the task `ABMB` (sample ABMP).

### *Comparing EIBAID*

In REXX, compare the hexadecimal form: `IF C2X(EIBAID) = 'F3' THEN ...` (the F3 key). In COBOL, `COPY DFHAID` and compare with the name (`IF EIBAID = PF03`), or with a hexadecimal literal (`IF EIBAID = X'F3'`). In PL/I, `%INCLUDE DFHAID;` and compare with the name (`IF EIBAID = PF03 THEN ...`), or with `'F3'X`.

## REXX

### *Compound symbols and map fields*

A tail symbol that has a value is replaced by its value: `SCR.NAME` refers to `SCR.Smith` after `NAME = 'Smith'`. A program that uses the same name for a local variable and a map field therefore writes the wrong compound variable, and the field stays empty. Give local variables names that differ from the map field names, as the samples do (`AKT` for `ACTION`, `CKEY` for `CUSTNO`, `LITM` for `LASTITEM`). See "Derived Names of Compound Symbols" in Chapter 36.

***LEAVE and ITERATE inside IF ... THEN DO***

A `LEAVE` or `ITERATE` without a name inside a `DO ... END` group ends only the group, not the loop around it. Write it as the bare clause of the `IF` (`IF X THEN LEAVE`), name the control variable (`LEAVE I`), or use a flag that the loop condition tests. See "LEAVE and ITERATE" in Chapter 37.

***Only PROCEDURE labels can be called***

`CALL name` and `name()` find only labels followed by `PROCEDURE`; a plain label gives `unknown function`. Expose the variables the routine needs with `PROCEDURE EXPOSE`. A routine ends at the next label, so do not put labels inside a routine. See "Internal Routines" in Chapter 35.

***Comments nest***

A `/*` inside a comment opens a nested comment. Text such as `/* W/* files */` leaves the comment open and silently swallows the code that follows, often whole routines; the program still loads. Do not write `/*` in comment text. See "Comments" in Chapter 35.

***EXEC CICS blocks: no commas, no comments***

An `EXEC CICS ... END-EXEC` block may span lines without any continuation character. A continuation comma at the end of a line and a comment inside the block are both passed to the command parser, which rejects the command. See "The EXEC CICS and EXEC SQL Preprocessor" in Chapter 35.

***Option values are names, not expressions***

In an `EXEC CICS` option, a quoted string is a constant and a symbol is a variable name. `KEYLENGTH(LENGTH(K))` does not call `LENGTH`; assign the value to a variable first.

***Omitted function arguments***

`STRIP(S,,'0')` is a load error. Pass a null string for an argument that is not wanted: `STRIP(S,'','0')`. See "Calling a Function" in Chapter 40.

***Unhandled conditions do not abend REXX***

After every `EXEC CICS`, a REXX program must test `EIBRESP` (or `RC`, or a `RESP` variable) itself, or set a `SIGNAL ON ERROR` trap. Nothing stops a REXX program that ignores a `NOTFND`. See "Unhandled CICS Conditions" in Chapter 39.

***SIGNAL ON traps stay set***

A trap is not removed when it is taken. If the handler can raise the same condition again, start it with `SIGNAL OFF`, or the handler loops until the task abends `AICA`.

***Reset stems between pages***

`SCR. = ''` sets only the default; compound variables set earlier keep their values. A stem that is reused from page to page keeps rows from a longer previous page unless it is dropped (`DROP RECS.`) or every row is assigned again, as `cusl.rexx` does.

***SAY output is discarded***

In a transaction, `SAY` writes nowhere. Show values with `SEND TEXT`, write them to a TS queue, or watch the program with CEDF (Chapter 8).

## COBOL and PL/I

***Code RESP before testing the response***

An unhandled condition such as `NOTFND` or `MAPFAIL` abends a COBOL or PL/I task (`AEIM`, `AEI9`), as on IBM CICS. A test of `EIBRESP` after a command that codes no `RESP` therefore never sees anything but `NORMAL`: any other outcome has already abended the task. Code `RESP(WS-CICS-RESP)` (COBOL) or `RESP(RC)` (PL/I) on every command whose response is tested, and compare with the `DFHRESP` names. Samples ABNC and ABNP show the abend, GUST and CUSP the cure. See "Preventing a Condition Abend" in Appendix C.

***EIBTRMID holds four characters***

bricks terminal identifiers have five characters (`T0001`), but `EIBTRMID` is a four-character field in the COBOL and PL/I EIB, as in IBM's `DFHEIBLK`. It therefore holds `T000`, and two terminals can look alike. For the full identifier, use `EXEC CICS ASSIGN TERMID(TID)` with an eight-character target (`PIC X(8)`, `DCL TID CHAR(8);`), as `cusp.pli` does. In REXX, `EIBTRMID` holds the full identifier.

## COBOL

### *Data names used in several groups*

A child name may be used in more than one group. A reference that matches more than one declaration must be qualified with `OF` or `IN` (`MOVE X OF DET TO Y OF SCR`); a name that is unique in the program needs no qualification. Two children with the same name in one group are rejected, because no qualifier could tell them apart.

### *Conditions follow IBM rules*

- `IF A / 2 > 2` with whole-number operands truncates the quotient: for `A = 5` it compares `2 > 2` and is false. Give the relation a decimal place outside the divisor: a dividend with `V` in its picture, or `IF A / 2 > 2.0`. A decimal divisor does not help: `IF A / 2.0 > 2` is still false.
- `IF A NOT = 1 AND 2` means `A NOT = 1 AND A NOT = 2`: the `NOT` carries to the omitted operator.
- `STRING ... DELIMITED BY SIZE` moves an item at its full width, trailing spaces included. Use `DELIMITED BY SPACE` to stop at the first blank.

See "STRING" in Chapter 24.

## PL/I

### *Declare everything; mind built-in names*

Every variable must be declared (`UNDECLARED QTYY`), and `DEC` and `BIN` need `FIXED`. A built-in function name cannot be a level-1 variable: `DCL DAYS(12) FIXED BIN(15);` fails with `BUILTIN NAME DAYS`, and so do `INDEX`, `LENGTH`, `DATE`, `TIME`, `SUM` and the others. Structure members may use built-in names, which is why a map structure can have a `SEARCH` field. After `%INCLUDE DFHAID;` the names `ENTER`, `CLEAR` and `PF01` to `PF24` are taken too.

### *Assignment truncates; ROUND rounds*

Assigning 4.947525 to a `FIXED DEC(9,2)` gives 4.94. Write `TAX = ROUND(TOTAL * RATE, 2);` wherever money is rounded. A value too large for its target abends `ABSZ` instead of losing high-order digits.

### *Division is always decimal*

`7 / 2` is 3.5, also for `FIXED BIN` operands. Assign the result to an integer target, or use `TRUNC` or `DIVIDE(x, y, p, q)`, for the integer quotient. `CHAR(N / 2)` keeps the full scale (`3.5000000000000000000`).

***CHAR of a number is minimal***

`CHAR(42)` is `'42'` and `CHAR(-3)` is `'-3'`, so `'Total: ' || CHAR(N)` needs no `TRIM`. For aligned columns, use `EDIT(x, 'ZZZ9V.99')` or a `PIC` variable.

***Handlers end the task***

A `HANDLE CONDITION`, `HANDLE AID` or `WHENEVER` target procedure does not return to the failing statement: when it returns, the task ends and commits. Use handlers for exit paths only, and `SYNCPOINT ROLLBACK` in a handler whose work must not be kept.

***Receive numbers into PIC or CHAR members***

`RECEIVE MAP` into a `FIXED DEC` or `FIXED BIN` member converts the typed text, and an operator who types `12A` abends the task `ABCV`. Receive into a numeric `PIC` member (then test it with `VALID`) or a `CHAR` member (then `VERIFY`), and convert after the check.

***An unlabeled LEAVE leaves the loop***

In PL/I, `IF X THEN DO; ...; LEAVE; END;` inside a loop leaves the **loop**: an unlabeled `LEAVE` or `ITERATE` acts on the innermost iterative `DO`, never on a plain `DO;` group. This is the opposite of the bricks REXX rule above.

See Part 7 (Chapter 28 to Chapter 34) for the PL/I rules.

**Appendix B**

# Quick Reference

This appendix summarizes the commands and language forms described in this book. It shows the common options only; see the chapter named in each table for the complete syntax, the conditions, and the differences from IBM CICS. In the tables, *v* is a variable that receives a value, *d* is a constant or a variable that supplies one, and brackets enclose optional items.

## Terminal Control (Chapter 11)

| Command | Purpose |
|---|---|
| `SEND MAP(m) [MAPSET(s)] [FROM(d)] [ERASE] [DATAONLY] [CURSOR(n)] [FREEKB]` | Show a map; wait for an AID (DATAONLY does not wait, and leaves the keyboard locked unless FREEKB is coded or the map has FREEKB). |
| `RECEIVE MAP(m) [MAPSET(s)] [INTO(v)]` | Read the operator's input into the map fields. |
| `CONVERSE MAP(m) FROM(d) INTO(v) [ERASE]` | SEND MAP and RECEIVE MAP in one command. |
| `SEND TEXT FROM(d) [LENGTH(n)] [ERASE] [CURSOR(n)]` | Show text laid out row by row. |
| `SEND CONTROL [ERASE] [FREEKB] [ALARM] [CURSOR(n)]` | Control the screen without data; FREEKB unlocks the keyboard. |
| `RECEIVE INTO(v) [LENGTH(v)]` | Read the command line typed at the blank screen (`EOC` if none). |

**Figure B-1. Terminal control commands**

## Program Control (Chapter 12)

| Command | Purpose |
|---|---|
| `LINK PROGRAM(p) [COMMAREA(v)]` | Call a program and return. |
| `XCTL PROGRAM(p) [COMMAREA(d)]` | Transfer control at the same level; the unit of work continues. |
| `RETURN [TRANSID(t)] [COMMAREA(d)] [IMMEDIATE]` | End the program; optionally name the next transaction. |
| `ABEND [ABCODE(c)] [CANCEL] [NODUMP]` | End the task abnormally; with ABCODE a dump is taken unless NODUMP. |
| `START TRANSID(t) [INTERVAL(hhmmss) \| TIME(hhmmss) \| AFTER ... \| AT ...] [FROM(d)] [TERMID(t)] [REQID(r)]` | Start a task later. |
| `RETRIEVE INTO(v) [LENGTH(v)]` | Read the data passed by START (ENDDATA if none). |
| `CANCEL REQID(r) [TRANSID(t)]` | Cancel a START that has not yet run. |

**Figure B-2. Program control commands**

## System Services (Chapter 13)

| Command | Purpose |
|---|---|
| `ASSIGN option(v) ...` | USERID, TERMID, SCREENHT, SCREENWD, ABCODE and others. |
| `ASKTIME [ABSTIME(v)]` | Refresh EIBDATE and EIBTIME; milliseconds since 1 January 1900. |
| `FORMATTIME ABSTIME(d) [YYYYMMDD(v)] [TIME(v)] [DATESEP] [TIMESEP] ...` | Format an ABSTIME. |
| `DELAY [INTERVAL(hhmmss) \| FOR SECONDS(n) MILLISECS(n) \| UNTIL ...]` | Wait. |
| `QUERY SECURITY RESOURCE(r) [READ(v)] [UPDATE(v)] ...` | Ask whether the user may use a resource. |
| `VERIFY PASSWORD(d) USERID(d)` | Check a password. |
| `INQUIRE SYSTEM GMMTEXT(v)` | The good-morning text. |
| `INQUIRE / SET FILE, TASK, TRANSACTION, PROGRAM, TERMINAL; STARTBROWSE / INQUIRE ... NEXT / ENDBROWSE` | Inquire on and change resources. |
| `ENQ RESOURCE(d) [LENGTH(n)] [NOSUSPEND]; DEQ RESOURCE(d)` | Serialize on a resource. |
| `GETMAIN SET(v) FLENGTH(n) [INITIMG(x)]; FREEMAIN` | Task storage. |
| `ADDRESS COMMAREA(v) EIB(v) TWA(v) CWA(v)` | Pointers to system areas. |

**Figure B-3. System service commands**

## File Control (Chapter 14)

| Command | Purpose |
|---|---|
| `READ FILE(f) INTO(v) RIDFLD(d) [UPDATE] [LENGTH(v)]` | Read by key; UPDATE locks the record. |
| `WRITE FILE(f) FROM(d) RIDFLD(d)` | Add a record (DUPREC if the key exists). |
| `REWRITE FILE(f) FROM(d)` | Replace the record read for update. |
| `DELETE FILE(f) [RIDFLD(d) [GENERIC KEYLENGTH(n) NUMREC(v)]]` | Delete a record, the one read for update, or every record with a key prefix. |
| `UNLOCK FILE(f)` | Release a record read for update. |
| `STARTBR FILE(f) [RIDFLD(d)] [GTEQ \| EQUAL] [GENERIC KEYLENGTH(n)]` | Start a browse. |
| `READNEXT / READPREV FILE(f) INTO(v) RIDFLD(v)` | Next or previous record (ENDFILE at the end). |
| `RESETBR FILE(f) RIDFLD(d)` | Reposition the browse. |
| `ENDBR FILE(f)` | End the browse. |

**Figure B-4. File control commands**

## Queues (Chapter 15)

| Command | Purpose |
|---|---|
| `WRITEQ TS QUEUE(q) FROM(d) [ITEM(v)] [REWRITE]` | Add or replace a TS item. |
| `READQ TS QUEUE(q) INTO(v) [ITEM(d) \| NEXT [ITEM(v)]]` | Read a TS item (ITEMERR past the end). |
| `DELETEQ TS QUEUE(q)` | Delete a TS queue. |
| `WRITEQ TD QUEUE(q) FROM(d)` | Append a line to the file *q* in `tmp_dir`. |
| `READQ TD QUEUE(q) INTO(v)` | Read the next line (QZERO, 23, at the end). |
| `DELETEQ TD QUEUE(q)` | Delete the file. |

**Figure B-5. Queue commands**

### Recovery and Condition Handling (Chapter 16)

| Command | Purpose |
|---|---|
| `SYNCPOINT [ROLLBACK]` | Commit or back out the unit of work. |
| `HANDLE CONDITION cond(label) ...` | Branch on a condition. |
| `IGNORE CONDITION cond ...` | Ignore a condition. |
| `HANDLE AID key(label) ...` | Branch on an AID. |
| `HANDLE ABEND LABEL(l) \| PROGRAM(p) \| CANCEL \| RESET` | Set or cancel an abend exit. |
| `RESP(v) [RESP2(v)] \| NOHANDLE` | On any command: return the response instead of raising the condition. |

**Figure B-6. Recovery commands**

An unhandled condition abends a COBOL or PL/I task with its IBM code (`NOTFND` gives `AEIM`); a REXX task continues with `RC` and `EIBRESP` set. See "Condition Abends" in Appendix C.

Most abends write a transaction dump to `<log_location>/abend.log`. An administrator types `CAID` *tasknumber* (the `Tranum` of message `DFHDU0203I`) to see the probable cause, or `CAID` alone for a list of the recent dumps. See "CAID: The Abend Analyser" in Chapter 16.

## EXEC SQL (Chapter 21)

| Statement | Purpose |
|---|---|
| `SELECT cols INTO :v [:ind], ... FROM ...` | Read one row; +100 if none, -811 if more than one. |
| `INSERT, UPDATE, DELETE` | Change rows; 0 rows is not an error. |
| `DECLARE c CURSOR FOR select [FOR UPDATE]` | Declare a cursor. |
| `OPEN c; FETCH c INTO :v, ...; CLOSE c` | Read rows; +100 at the end. |
| `UPDATE ... WHERE CURRENT OF c` | Change the row last fetched. |
| `COMMIT WORK; ROLLBACK WORK` | Commit or roll back the database transaction only; file and TS changes wait for `SYNCPOINT`. A refused commit gives `SQLCODE` -100. |
| `CONNECT TO 'db'` | Switch database for the task. |
| `WHENEVER SQLERROR \| NOT FOUND \| SQLWARNING CONTINUE \| GO TO label` | Declarative error handling. |

**Figure B-7. EXEC SQL statements**

| Field | Meaning |
|---|---|
| `SQLCODE` | 0 OK, +100 no data, negative: error. |
| `SQLSTATE` | Five-character state code. |
| `SQLERRMC` | Error message text. |
| `SQLERRD3` | Rows affected by INSERT, UPDATE, DELETE. |
| `SQLWARN0, SQLWARN8` | `W` when a character had no bricks SBCS equivalent and became `?`. |

**Figure B-8. SQLCA fields**

## Web Services (Chapter 20)

| Command | Purpose |
|---|---|
| `WEB EXTRACT METHOD(v) PATH(v) QUERYSTRING(v) ...` | Server: describe the request. |
| `WEB READ HTTPHEADER(n) VALUE(v)` | Read a header. |
| `WEB READ QUERYPARM(n) VALUE(v); WEB READ FORMFIELD(n) VALUE(v)` | Read a parameter, path capture or form field. |
| `WEB STARTBROWSE / READNEXT / ENDBROWSE HTTPHEADER \| QUERYPARM \| FORMFIELD` | Walk headers, parameters or fields. |
| `WEB RECEIVE INTO(v) [LENGTH(v)]` | Server: read the body. |
| `WEB WRITE HTTPHEADER(n) VALUE(d) [SESSTOKEN(t)]` | Set a response or request header. |
| `WEB SEND FROM(d) \| DOCTOKEN(t) [MEDIATYPE(d)] [STATUSCODE(n)]` | Server: send the response. |
| `WEB OPEN HOST(d) \| URIMAP(u) SESSTOKEN(v)` | Client: open a session. |
| `WEB CONVERSE SESSTOKEN(t) METHOD(m) PATH(p) [FROM(d)] INTO(v) [STATUSCODE(v)]` | Client: send a request and receive the reply. |
| `WEB CLOSE SESSTOKEN(t)` | Client: close the session. |
| `WEB PARSE URL(d) ...; WEB CONVERTTIME` | Split a URL; convert an HTTP date. |
| `DOCUMENT CREATE / INSERT / SET / RETRIEVE / DELETE DOCTOKEN(t)` | Build a document from text and symbols. |

**Figure B-9. Web and document commands**

## EIB Fields (Chapter 17)

| Field | Meaning |
|---|---|
| EIBAID | The AID of the last terminal input. |
| EIBCPOSN | Cursor position at the last input, counted from 0. |
| EIBCALEN | Length of the COMMAREA (0: none). |
| EIBTRMID | Terminal id; four characters in COBOL and PL/I (use ASSIGN TERMID for all five). |
| EIBTRNID, EIBTASKN | Transaction id, task number. |
| EIBRESP, EIBRESP2 | Response of the last command. |
| EIBDATE, EIBTIME | 0CYYDDD and HHMMSS, set at task start and by ASKTIME. |
| RC (REXX) | Return code of the last command (EIBRESP for EXEC CICS, SQLCODE for EXEC SQL). |

**Figure B-10. EIB fields**

## Copybooks and Include Members (Chapters 26 and 33)

| Member | Contents |
|---|---|
| DFHAID | AID names (ENTER, PF03 ...) and the DFH... aliases. COBOL and PL/I. |
| DFHRESP | Response names (RESP–NOTFND and DFHRESP–NOTFND; PL/I RESP_NOTFND). COBOL and PL/I. |
| DFHVALUE | CVDA values (DFHVALUE–UPDATABLE ...). |
| SQLCA | SQLCODE, SQLSTATE names. |
| DFHWBSC, DFHWBUH, DFHWBMT, DFHWBMETH, DFHWBSI, DFHWBUM, DFHWBHB, DFHWBCC, DFHDCDOC | Web status codes, header names, media types, methods, session templates, URIMAP fields, header browse, certificate fields, document constants. |

**Figure B-11. Copybooks and include members**

In PL/I, DFHRESP(cond) and DFHVALUE(cvda) are built-in and need no include; the EIB and the SQLCA are predeclared.

## REXX (Chapters 35 to 40)

| Form | Example |
|---|---|
| Command | EXEC CICS READ FILE('f') INTO(REC) RIDFLD(K) END-EXEC (may span lines; no commas or comments inside) |
| Assignment, stem | SCR. = '', SCR.MSG = 'Done.' |
| IF | IF RC = 13 THEN ... ELSE ... |
| SELECT | SELECT; WHEN x THEN ...; OTHERWISE NOP; END |
| DO | DO I = 1 TO N BY 2, DO WHILE c, DO UNTIL c, DO FOREVER, DO 5, DO K OVER S. |
| LEAVE, ITERATE | IF c THEN LEAVE; LEAVE I inside a group |
| Routine | R: PROCEDURE EXPOSE A B. ... RETURN x; CALL R 1, 2; Y = R(1) |
| PARSE | PARSE VAR REC NM '\|' AD, PARSE UPPER ARG A B, PARSE VALUE e WITH 5 K 9 |
| Traps | SIGNAL ON ERROR NAME L, SIGNAL OFF ERROR; conditions ERROR, NOVALUE, SYNTAX, HALT |
| AID test | IF C2X(EIBAID) = 'F3' THEN ... |
| Dynamic name | CALL VALUE 'SCR.ROW' \|\| J, LINE |
| Not supported | ARG, PULL, PUSH, QUEUE, UPPER, TRACE, OPTIONS, CALL ON, DO ... FOR, &&, omitted arguments (f(a,,b)) |

**Figure B-12. REXX forms**

## PL/I (Chapters 28 to 34)

| Construct | Form |
|---|---|
| Program | NAME: PROC OPTIONS(MAIN); ... END NAME; |
| Internal procedure | P: PROC(A, B) [RETURNS(attrs)] [RECURSIVE]; ... END P; |
| Declarations | DCL X FIXED DEC(p,q) [INIT(v)] [STATIC]; FIXED BIN(15\|31\|63), CHAR(n) [VAR], BIT(n), PIC '...' |
| Structure | DCL 1 S, 2 A CHAR(8), 2 R(15) CHAR(71); DCL 1 T LIKE S; |
| Named constant | DCL MAX FIXED BIN(31) VALUE(50); |
| External entry | DCL F ENTRY(attrs, ...) RETURNS(attrs) EXTERNAL; |
| Assignment | A, B = 0; T = S, BY NAME; SUBSTR(L, 5, 3) = 'ABC'; |
| Conditional | IF c THEN s; ELSE s; SELECT (x); WHEN (v1, v2) s; OTHERWISE s; END; |
| Loops | DO I = 1 TO N [BY k] [WHILE(c)]; DO WHILE(c); DO UNTIL(c); DO FOREVER; |
| Loop control | LEAVE [label]; ITERATE [label]; |
| Calls | CALL P(args); RETURN; RETURN (v); |
| Abend | CALL ABEND('CU01'); |
| CICS | EXEC CICS READ FILE('f') INTO(REC) RIDFLD(KEY) RESP(RC); |
| SQL | EXEC SQL SELECT c INTO :V FROM t WHERE k = :K; |
| Include | %INCLUDE DFHAID; |

**Figure B-13. PL/I forms**

The abend codes of the PL/I runtime (ABCV, ABSZ, ABFO, ABZD, ABSR, ABSL, ABEN, ABHB, ABST, APLI) and every other abend code are listed in "Other Transaction Abend Codes" in Appendix C.

**Appendix C**

# Abend Codes

This appendix lists the transaction abend codes that bricks raises: the codes of conditions that a COBOL or PL/I program leaves unhandled, and the codes that CICS raises itself when it cannot carry out a command, a commit or the program. For each code it gives the meaning, when the code occurs, how to prevent it, and whether a `HANDLE ABEND` exit can catch it.

How an abend ends a task (dynamic backout, the messages `DFHAC2236` and `DFHAC2206`) is described in "How an Abend Ends a Task" in Chapter 16, and the transaction dump (message `DFHDU0203I` and `abend.log`) in "Transaction Dumps". The transaction CAID analyses a dump and names the probable cause ("CAID: The Abend Analyser").

## Condition Abends

When a COBOL or PL/I program leaves a CICS condition **unhandled**, bricks abends the task with the IBM abend code for that condition, as IBM CICS does. A condition is unhandled when the command codes neither `RESP` nor `NOHANDLE`, no `HANDLE CONDITION` names the condition (or `ERROR`), and no `IGNORE CONDITION` covers it.

The abend is an ordinary abend. A transaction dump is taken; the program's `HANDLE ABEND` exit, or a caller's up the `LINK` chain, receives control; and when nothing catches it, the unit of work is backed out, `DFHAC2236` is logged, and the operator sees `DFHAC2206 ... failed with abend AEIM.` with the failing command on the `Detail:` row.

**Note:**

REXX never abends on a condition, as in IBM CICS REXX. After an unhandled condition `RC` and `EIBRESP` hold the response and the program continues with the next statement. `HANDLE CONDITION` still branches, and `SIGNAL ON ERROR` traps the non-zero `RC`. The codes in "Other Transaction Abend Codes" are not conditions and end REXX tasks too.

## Condition Abend Codes

The tables list every condition that has an abend code, in order of its `RESP` value. The column **Raised** tells whether bricks returns the condition at all; a condition that is never returned can never cause its abend, but the code is reserved with its IBM meaning.

| Code | Condition | RESP | Raised | Meaning |
|---|---|---|---|---|
| AEIA | ERROR | 1 | Yes | The command could not be processed; no more specific condition applies. |
| AEIL | FILENOTFOUND | 12 | Yes | The file named by the command was not found (IBM DSIDERR). |
| AEIM | NOTFND | 13 | Yes | The record, key, item or named resource was not found. |
| AEIN | DUPREC | 14 | Yes | A WRITE named a key that already exists. |
| AEIO | DUPKEY | 15 | No | Duplicate key on an alternate index. |
| AEIP | INVREQ | 16 | Yes | The request is not valid: missing or conflicting options, a name that is not valid, the wrong state (such as READNEXT with no browse), or the LINK nesting limit. |
| AEIQ | IOERR | 17 | Yes | The database or file system reported an I/O error. |
| AEIR | NOSPACE | 18 | No | No space is left in the file or queue. |
| AEIS | NOTOPEN | 19 | No | The file is not open. |
| AEIT | ENDFILE | 20 | Yes | A browse read past the last record. |
| AEIU | ILLOGIC | 21 | Yes | A logic error, such as INQUIRE ... NEXT with no browse. |
| AEIV | LENGERR | 22 | Yes | A length is not valid, or the data did not fit and was truncated. |
| AEIW | QZERO | 23 | Yes | The transient data queue is empty. |
| AEIZ | ITEMERR | 26 | Yes | The TS item does not exist. |
| AEI0 | PGMIDERR | 27 | Yes | The program is not defined or cannot be loaded. |
| AEI1 | TRANSIDERR | 28 | Yes | The transaction is not defined. |
| AEI2 | ENDDATA | 29 | Yes | RETRIEVE found no (more) data. |
| AEI3 | INVTSREQ | 30 | No | The TS request is not supported. |
| AEI4 | EXPIRED | 31 | No | The requested time has already expired. |
| AEI8 | TSIOERR | 35 | No | I/O error on temporary storage. |
| AEI9 | MAPFAIL | 36 | Yes | The map could not be found, sent or received. |

**Figure C-1. Condition abend codes AEIA through AEI9**

| Code | Condition | RESP | Raised | Meaning |
|---|---|---|---|---|
| AEYB | INVMPSZ | 38 | No | The map is too large for the terminal. |
| AEYC | IGREQID | 39 | No | REQID prefix conflict (BMS). |
| AEYE | INVLDC | 41 | No | The logical device code is not valid. |
| AEYG | JIDERR | 43 | No | The journal was not found. |
| AEYH | QIDERR | 44 | Yes | The TS queue or TD file was not found. |
| AEYJ | DSSTAT | 46 | No | Data-stream status change. |
| AEYK | SELNERR | 47 | No | Destination selection error. |
| AEYL | FUNCERR | 48 | No | Function error. |
| AEYM | UNEXPIN | 49 | No | Unexpected input. |
| AEYN | NOPASSBKRD | 50 | No | No passbook on read. |
| AEYO | NOPASSBKWR | 51 | No | No passbook on write. |
| AEYQ | SYSIDERR | 53 | No | The remote system is not available. |
| AEYR | ISCINVREQ | 54 | No | The remote system rejected the request. |
| AEYT | ENVDEFERR | 56 | No | RETRIEVE asked for an option the START did not supply. |
| AEYU | IGREQCD | 57 | No | The request was ignored because a SIGNAL was received. |
| AEYV | SESSIONERR | 58 | No | Session allocation error. |
| AEYY | NOTALLOC | 61 | No | The session is not allocated. |
| AEYZ | CBIDERR | 62 | No | Control block identifier error. |
| AEY0 | INVEXITREQ | 63 | No | The exit request is not valid. |
| AEY1 | INVPARTNSET | 64 | No | The partition set is not valid. |
| AEY2 | INVPARTN | 65 | No | The partition is not valid. |
| AEY3 | PARTNFAIL | 66 | No | Partition failure. |
| AEY7 | NOTAUTH | 70 | Yes | The user is not authorized for the request. |
| AEXJ | ROLLEDBACK | 82 | Yes | SYNCPOINT ROLLBACK could not back out every file or TS change. |
| AEXK | END | 83 | Yes | INQUIRE ... NEXT has returned every resource. |

**Figure C-2. Condition abend codes AEYB through AEXK**

## Conditions That Never Abend

IBM's default action for the following conditions is to continue, so an unhandled one never abends, in any language. EIBRESP (and RESP, if coded) still report it:

EOC (6), SIGNAL (24), QBUSY (25), RETPAGE (32), RTEFAIL (33), RTESOME (34), OVERFLOW (40), NOSTG (42), NOJBUFSP (45), ENQBUSY (55), SYSBUSY (59), SESSBUSY (60).

ENQBUSY is among them, so a record lock conflict or an ENQ NOSUSPEND can always be retried by testing the response.

### When Condition Abends Occur

The commonest causes in bricks programs are:

| Code | Typical cause | Shown by sample |
|---|---|---|
| AEIM | READ or DELETE of a key that is not on file; CANCEL of a request that is not pending; WEB READ HTTPHEADER or FORMFIELD of an absent name. | ABNC (COBOL), ABNP (PL/I) |
| AEIN | WRITE of a key that is already on file. | |
| AEIT | A READNEXT loop that does not test for the end of the file. | |
| AEIW | A READQ TD loop that does not test for the end of the file. | |
| AEYH, AEIZ | READQ TS of a queue that does not exist, or of an item past the end of the queue. | |
| AEI9 | SEND MAP of a map that is not installed or did not parse (the detail names it: SEND MAP: map "NOPE" not found); RECEIVE MAP with no matching SEND MAP. | |
| AEIP | Conflicting options (SEND MAP ... DATAONLY ERASE); a browse command with no browse open; a LINK beyond 100 levels. | |
| AEI0 | LINK or XCTL to a program that is not in transactions.conf. | |
| AEY7 | XCTL to a program the user may not run. | |
| AEI2 | RETRIEVE in a task that was typed at the terminal rather than started. | |

**Figure C-3. Common causes of condition abends**

A condition abend gives the same DFHAC2206 message in COBOL and in PL/I. Figure 16-8 in Chapter 16 shows the COBOL sample ABNC; Figure C-4 shows its PL/I equivalent ABNP, which issues the same READ without RESP. Only the transaction identifier and the time differ: the abend code and the Detail: row, which names the command and the condition, are the same in both languages.

```
DFHAC2206 19:08:02 BRICKS01 Transaction ABNP failed with abend AEIM.
  Updates to local recoverable resources backed out.

Detail: READ FILE('customers') NOTFND
```

**Figure C-4. ABNP (PL/I) ends with abend AEIM** (Model 2, 24 x 80; capture screens/abend-aeim-pli.txt)

### Preventing a Condition Abend

Use one of the following for each command:

1. **RESP** (recommended). Code `RESP(var)` and test it with `DFHRESP`. The command never abends on a condition.

```
           EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(KEY-IN)
                     RESP(WS-RESP) END-EXEC.
           IF WS-RESP = DFHRESP(NOTFND)
              MOVE 'Customer not found.' TO MSG
           END-IF.
```

```
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(KEY) RESP(RC);
IF RC = DFHRESP(NOTFND) THEN MSG = 'Customer not found.';
```

2. **NOHANDLE**. As `RESP`, with the response in `EIBRESP`.
3. **HANDLE CONDITION** *cond*(*label*) or `ERROR(`*label*`)`. Control passes to the label instead of abending. In PL/I the label names a handler procedure; the task ends when it returns.
4. **IGNORE CONDITION** *cond*. The condition is ignored for every later command, and `EIBRESP` reads 0.

### Catching a Condition Abend

A condition abend is caught like any other abend:

- COBOL: `EXEC CICS HANDLE ABEND LABEL(RECOVER) END-EXEC`. At `RECOVER`, `ASSIGN ABCODE` returns `AEIM` and `EIBRESP` still holds the condition (13). Save `EIBRESP` before the next command, which sets it again. The supplied sample `ABNH` shows `CAUGHT AEIM EIBRESP=0013` (see "HANDLE ABEND" in Chapter 16).
- PL/I: `EXEC CICS HANDLE ABEND PROGRAM(name)`. The program is linked to and the abend then completes.
- Under CEDF, the COMMAND EXECUTION COMPLETE display of the command forecasts `NOT HANDLED - ABEND AEIM`, and the AN ABEND HAS OCCURRED display follows ("Abends and Task Termination" in Chapter 8).

***Abends in a program reached by LINK***

A condition abend in a program reached by `LINK` behaves like every other abend there. If that program has no exit of its own, the abend moves up to its caller: the caller's `HANDLE ABEND` is searched, then its caller's, and so on. It is never a condition in the caller: `RESP` on the `LINK`, `HANDLE CONDITION PGMIDERR` and `ERROR` do not absorb it. When no level catches it, the task ends and `DFHAC2236` names the program that abended, not the task's first program.

This holds whatever the caller's language: a REXX program that links to a COBOL program which abends `AEIM` ends with that abend too, unless the REXX program has an active `HANDLE ABEND`.

A failure that is not a CICS condition is never turned into an `AEIx` abend. A lost terminal (`ATNI`) and an internal failure inside a command (`AEXZ`) have codes of their own, and a command that bricks does not support ends the task with a message of its own ("What Happens When a Program Uses One" in Chapter 19). `RESP` absorbs none of them.

### Other Transaction Abend Codes

The following abends are raised by CICS itself when it cannot carry out a command, a commit or the program. They are not conditions: `RESP`, `NOHANDLE`, `HANDLE CONDITION` and `IGNORE CONDITION` never apply to them, `EIBRESP` is left as it was, and they end REXX tasks exactly as they end COBOL and PL/I tasks. When nothing catches them they end the task like every abend: backout, `DFHAC2236` in the log, and `DFHAC2206` on the screen with the detail on the `Detail:` row.

In the table, **Dump** means that a transaction dump is written to `abend.log` and the dump message `DFHDU0203I` is logged, for most codes followed by a line *code* `detail:` that gives the terminal, the program and what failed.

| Code | Meaning | When it occurs in bricks | HANDLE ABEND | Dump |
|---|---|---|---|---|
| `AAAA` | Abend with no code | `EXEC CICS ABEND` without `ABCODE` | Yes, unless `CANCEL` | No |
| *code* | Application abend | `EXEC CICS ABEND ABCODE(`*code*`)` with a code that the program chose | Yes, unless `CANCEL` | Yes, unless `NODUMP` |
| `ABM0` | Map not found | `RECEIVE MAP` names a `MAPSET` that exists but does not contain the map | Yes | Yes |
| `ABMB` | Cursor beyond the screen | `SEND MAP`, `SEND TEXT` or `SEND CONTROL` with `CURSOR(n)`, *n* not less than rows × columns | Yes | Yes |
| `AD2P` | No database thread | Every one of the `db_max_conns` pooled connections stayed in use for 30 seconds | Yes | Yes |
| `AD2S` | Database sign-on failed | The database refused `db_user` or `db_password` (SQLSTATE class `28`) | Yes | Yes |
| `ADBU` | Database unavailable | An `EXEC SQL` statement could not reach the database (server down, connection refused or lost, no answer) | Yes | Yes |
| `AEXZ` | Serious failure in a CICS component | An `EXEC CICS` command failed inside its own implementation | Yes | Yes |
| `AEY9` | CICS not available | PL/I: `EXEC CICS` issued by a program run without the CICS interface (never in a task of the region) | No | No |
| `AICA` | Runaway task | The task ran 2 seconds without giving control back to the region | Yes | Yes |
| `APCT` | Program cannot be loaded | An abend needs the `HANDLE ABEND PROGRAM` exit, and the exit cannot be loaded | No | Yes |
| `APLI` | PL/I internal failure | The PL/I interpreter failed, or ran a program or entry that was not compiled | Callers only | Yes |
| `ASP3` | Unit of work backed out | At a commit point the database refused the commit; no file or TS change was pending | No | No |
| `ASP7` | Resource manager voted No | At a commit point the database refused the commit while file or TS changes were pending | No | No |
| `ASPR` | Link failed during commit | The database connection broke during the commit; the outcome there is unknown | No | No |

| Code | Meaning | When it occurs in bricks | HANDLE ABEND | Dump |
|---|---|---|---|---|
| ASRA | Program check | COBOL division by zero without ON SIZE ERROR, a COBOL addressing exception, or a REXX or COBOL interpreter failure (a bricks error) | Yes | Yes (and the stack trace of an interpreter failure is logged) |
| ATNI | Terminal error | The terminal connection was lost during SEND MAP, SEND TEXT or SEND CONTROL | First one only | Yes |

**Figure C-5. Other transaction abend codes**

The abends of PL/I runtime conditions (ZERODIVIDE, SIZE, CONVERSION and others) are listed in "Runtime Abends" in Chapter 33.

## When Each Code Occurs

**ABM0**

RECEIVE MAP('NOPE') MAPSET('HELO1'): the mapset HELO1 is installed, but the map is not in it. The detail reads RECEIVE MAP: map "NOPE" not in mapset "HELO1". A mapset that does not exist at all, or a map whose file failed to parse, is still the MAPFAIL condition, and so is every failure of SEND MAP. The ABMR sample (REXX) shows this abend (Figure 41-11 in Chapter 41).

**ABMB**

The screen has rows × columns positions, numbered from 0 as in IBM CICS: 0-1919 on a 24 × 80 Model 2, 0-3439 on a 43 × 80 Model 4. CURSOR(3440) on a Model 4 is past the end. The detail reads, for example, SEND TEXT CURSOR(9999) BEYOND 43X80 SCREEN. Nothing is sent to the terminal. The ABMP sample (PL/I) shows it.

Figure C-6 shows the ABMP sample on a Model 2 terminal. The detail row gives the cursor position that the program coded and the size of the screen it was checked against.

```
DFHAC2206 19:07:58 BRICKS01 Transaction ABMP failed with abend ABMB.
  Updates to local recoverable resources backed out.

Detail: SEND TEXT CURSOR(9999) BEYOND 24X80 SCREEN
```

**Figure C-6. ABMP ends with abend ABMB** (Model 2, 24 x 80; capture screens/abend-abmb.txt)

**AEXZ**

A bricks failure inside a command, or the map renderer failing. The detail reads, for example, `ASKTIME: INTERNAL ERROR`; the Go stack trace goes to the log only. A failure of the interpreter itself, outside any command, is `ASRA`.

**ATNI**

The operator closed the emulator, or the network dropped, while the task was painting the screen or waiting for a key. Nothing is shown (there is no screen); the log has the dump message, a line such as `ATNI detail: term=T0001 program=GUST verb=SEND MAP cause=...`, and `DFHAC2236`. EDF is switched off for the rest of the task.

**APCT**

`HANDLE ABEND PROGRAM('NOPGM')` names a program that is not defined in `transactions.conf`, that the user may not run, or whose source cannot be read or compiled. Activating the exit is not an error; the abend comes when an abend needs the exit. The abend being handled then ends as `APCT`, and the log line `APCT detail: ... exit=NOPGM original=TEST` keeps the original code. An exit that starts and then fails is not `APCT`: the original abend continues. The `APCH` sample (COBOL) shows it.

In Figure C-7 the APCH sample has activated `HANDLE ABEND PROGRAM(NOPGM)` and then abended. The message gives `APCT`, not the code of the original abend, and the detail row names the exit program that could not be loaded.

```
DFHAC2206 19:07:53 BRICKS01 Transaction APCH failed with abend APCT.
  Updates to local recoverable resources backed out.

Detail: HANDLE ABEND PROGRAM(NOPGM) cannot be loaded
```

**Figure C-7. APCH ends with abend APCT** (Model 2, 24 x 80; capture screens/abend-apct.txt)

**`ASP3`**

The database refused the commit of `SYNCPOINT`, of the implicit syncpoint of `RETURN`, of the end of a program that ends without `RETURN`, or of an `XCTL` to a program on another database: a deferred unique or foreign key (SQLSTATE `23xxx`), a serialization failure (`40001`), a deadlock (`40P01`), or a transaction already aborted by an earlier error (`25P02`). The detail reads, for example, `SYNCPOINT: DATABASE BACKED OUT UOW, SQLSTATE 23505`; it begins `RETURN:`, `TASK END:` or `XCTL:` for the implicit commits. The `SPDB` sample (REXX) shows it (Figure 41-12 in Chapter 41).

**`ASP7`**

The same refusal while file or TS changes were pending; they are backed out too. Detail: `SYNCPOINT: DATABASE VOTED NO, SQLSTATE 23505`. The `SPLC` sample (COBOL) shows it.

Figure C-8 shows the SPLC sample. Its unit of work held a file update as well as the database insert that the database refused at `SYNCPOINT`, so the code is `ASP7` rather than the `ASP3` of Figure 41-12, and the file update was backed out with it. The detail row gives the point that failed and the SQLSTATE that the database returned.

```
DFHAC2206 19:08:16 BRICKS01 Transaction SPLC failed with abend ASP7.
  Updates to local recoverable resources backed out.

Detail: SYNCPOINT: DATABASE VOTED NO, SQLSTATE 23505
```

**Figure C-8. SPLC ends with abend ASP7** (Model 2, 24 x 80; capture screens/abend-asp7.txt)

**ASPR**

The connection to the database broke during the commit, or the commit failed with no answer from the database. Detail: `SYNCPOINT: DATABASE LOST DURING COMMIT, OUTCOME UNKNOWN`. IBM CICS would keep the unit of work in doubt until the connection returned; bricks cannot, so pending file and TS changes are backed out at once and the log says `SQL OUTCOME UNKNOWN, LOCAL UPDATES BACKED OUT`. Check the database for the outcome there.

**AD2S**

The first `EXEC SQL` statement that needs a connection finds that the database refuses the sign-on. Detail: `DATABASE SIGN-ON FAILED FOR bank, SQLSTATE 28P01`; the screen adds `Have an admin check the database user and password.`

**AD2P**

The task's first data statement waited 30 seconds for one of the `db_max_conns` pooled connections and none became free. Detail: `NO DATABASE CONNECTION FREE FOR bank (db_max_conns=8)`. The 30-second wait bounds only the acquisition of a connection. IBM's DB2 attachment with `THREADWAIT(NO)` abends at once; bricks waits first, like `THREADWAIT(YES)`. A wait that runs out while a connection is free means the database does not answer: `ADBU`.

**ADBU**

The database cannot be reached: the server is down, the connection is refused or lost, or the database does not answer. It is a normal abend with full backout; the screen adds a line asking for the database host in `bricks.cnf` to be checked. See "Database Abends" in Chapter 21.

**AICA**

See "Runaway Tasks" in Chapter 16. The detail reads `RUNAWAY TASK –` *program* `LINE` *n*.

**ASRA**

A COBOL division by zero that no `ON SIZE ERROR` covers, in any statement including an `IF` or `EVALUATE` condition (detail `DECIMAL-DIVIDE EXCEPTION`, IBM's `S0CB`); a COBOL addressing exception (the detail names the item); or an internal failure of the REXX or COBOL interpreter (detail `INTERNAL ERROR`, with the Go stack trace in the log). Only the failing task ends. CAID names the divisor that was zero, or the item that had no address.

**APLI, AEY9**

PL/I only. `APLI` is an internal failure of the PL/I interpreter or a call of something that was not compiled. `AEY9` is issued when a PL/I program runs without the CICS interface; it does not occur in a task of the region.

### Preventing the Other Abends

- **ABM0**: code the `MAPSET` the map actually belongs to (a map without a `MAPSET` line belongs to the mapset named after its file), or leave `MAPSET` out.
- **ABMB**: keep `CURSOR` within the smallest screen the program can run on (0-1919 on a Model 2), or compute it from `ASSIGN SCREENHT` and `SCREENWD`.
- **AD2S**: have an administrator check `db_user` and `db_password` in `bricks.cnf` and the role in the database.
- **AD2P**: do not hold a database transaction across a terminal wait. Issue `SYNCPOINT` before a `SEND MAP` that waits for the operator, so the connection goes back to the pool; or raise `db_max_conns`.
- **ASP3, ASP7, ASPR**: deferred constraints and serialization conflicts only appear at the commit. To see them while the program can still react, issue `EXEC SQL SET CONSTRAINTS ALL IMMEDIATE END-EXEC` before the `SYNCPOINT`: a violation then comes back as `SQLCODE` on that statement, and the program can issue `SYNCPOINT ROLLBACK`. Test `SQLCODE` after every update.
- **AICA**: issue a yielding command (such as `DELAY`) in a long loop, or split the work.
- **APCT**: define the exit program before activating it.
- **ATNI**, **AEXZ**, **ADBU**, **ASRA** and **APLI** are not program errors to prevent.

### Catching the Other Abends

HANDLE ABEND LABEL and HANDLE ABEND PROGRAM catch ABM0, ABMB, AEXZ, AICA, ADBU, AD2S, AD2P, ASRA and the first ATNI like any abend, at this LINK level or a caller's; ASSIGN ABCODE (or EIBABCODE) names the code. Under CEDF the AN ABEND HAS OCCURRED display comes first, except for ATNI. They cannot be caught in the following cases:

- APCT: the exit itself is what failed; no exit at any level runs.
- ASP3, ASP7, ASPR: everything has been backed out before the abend, and IBM CICS drives no exit for them, at any LINK level.
- A second ATNI: once the terminal is lost, any terminal command (typically a SEND in the exit) ends the task at once, so an exit cannot loop on the dead connection.
- ABEND CANCEL: whatever the code, no exit runs at any level.
- APLI and the other PL/I runtime abends: not offered to the exit of the PL/I program that failed; a caller's exit up the LINK chain can catch them.

### Codes bricks Does Not Raise

| Code | Reason |
|---|---|
| ASRB | bricks has no unrecoverable abend. |
| AFCF | A record lock conflict returns ENQBUSY at once, so two tasks cannot deadlock on records. |
| AEYD, AEYF, ABMU | There are no storage addresses to check. |
| AEZD | There is no EXECKEY. |
| ATCV, AZTS | There is no APPC. |
| ADLA | There is no DL/I. |
| ADPL | There is no distributed program link. |
| AD2V | IBM CICS does not abend in the corresponding case either. |
| AD2Z | The DB2 attachment failure it reports does not arise. |
| ASP1, ASP2, ASPF | There is no syncpoint across systems. |
| AFCA | Not in CICS TS 6; a disabled file is AEXL. |
| AKCT, AKCS, AEXY, AKC3, AEXL | Terminal read time-out, CEMT SET TASK PURGE and SET FILE DISABLED are not provided. |

**Figure C-9. IBM abend codes that bricks does not raise**

**Appendix D**

# Configuration for the Application Programmer

This appendix describes the parts of the bricks configuration that affect application programs: where bricks looks for programs, maps, copybooks and data; the `bricks.cnf` knobs that change how commands behave; and the resource definition files in the `runtime/` directory. Knobs that concern only the running of the region (listener ports for terminals, logging, metrics, caches, NTP and so on) are described in the operations documentation, `README.md`.

## The runtime Directory

The following layout is the default. Each directory and file can be moved with the knob shown. When `runtime_dir` is set to another directory, every directory in the table that is not set explicitly moves with it.

| Default path | Knob | Contents |
|---|---|---|
| runtime/ | runtime_dir | Root of the runtime tree. |
| runtime/cobol/ | cobol_dir | COBOL programs (*.cob). |
| runtime/pli/ | pli_dir | PL/I programs (*.pli). PL/I external entries resolve to *name*.pli here. |
| runtime/rexx/ | rexx_dir | REXX programs (*.rexx). |
| runtime/map/ | maps_dir | Map definitions (*.map). See "Map Files" in Chapter 3. |
| runtime/cobolcopy/ | copybook_dir | COBOL copybooks for COPY: DFHAID, DFHRESP, SQLCA and the web copybooks. See "Supplied Copybooks" in Chapter 26. |
| runtime/pliinc/ | pli_include_dir | PL/I %INCLUDE members (*name*.inc), including DFHAID and DFHRESP. See "%INCLUDE" in Chapter 33. |
| runtime/tmp/ | tmp_dir | Sandbox for sequential text files: REXX LINEIN, LINEOUT and STREAM, and transient data queues. See "Sequential Files in tmp_dir" in Chapter 14 and "The tmp_dir Directory" in Chapter 15. |
| runtime/web/ | webserver_dir | Document root of the static file server. Must lie under runtime_dir. |
| runtime/transactions.conf | transactions_file | Transaction definitions. |
| runtime/users.conf | users_file | User identifiers, password hashes and groups. |
| runtime/aliases.conf | aliases_file | Optional long names for transactions. |
| runtime/databases.conf | databases_file | PostgreSQL databases that programs may use. |
| runtime/web_routes.conf | web_routes_file | Web routes and URIMAP definitions. |
| runtime/mro.conf | mro_file | Peer regions for multiregion operation. |
| data/ | data_dir | The file store files.boltdb: VSAM files and temporary storage queues. Not under runtime_dir. |

**Figure D-1. Default layout of the runtime directory**

The program name in transactions.conf is relative to the directory of the program's language and may include subdirectories, for example billing/inv01.cob. Programs, maps, copybooks and include members are reloaded when their files change; a changed copybook or include member recompiles every program that uses it at its next use.

### The bricks.cnf File

`bricks.cnf` holds one `key=value` setting per line. The rules that matter when reading or changing it are:

- Keys are not case-sensitive; values are.
- An unknown key, or a line without `=`, stops bricks at startup with the line number.
- When a key appears more than once, the **last** line wins, and a warning names every line. Every other bricks configuration file uses the **first** definition instead.
- A `#` at the start of a line, or preceded by a blank or a tab, starts a comment. A `#` with no blank before it is data, so `db_password=pass#word` is kept whole.
- A value enclosed in a pair of quotes is taken verbatim between them, including any `#`.
- A switch accepts `yes`, `true`, `on` or `1`, and `no`, `false`, `off` or `0`. Any other value stops bricks at startup.
- A time value is a Go duration (`5s`, `250ms`, `1m30s`) or a whole number of seconds; `0` or `off` means no limit.

bricks reads `bricks.cnf` only at startup. The tables that follow list the knobs that affect application programs, with their defaults.

*Terminals, sign-on and the region*

| Knob | Default | Effect on programs |
|---|---|---|
| logon_transid | CSSN | The sign-on transaction. secure_login_transaction is accepted as another name. |
| enforce_secure_login | no | yes refuses every transaction except the sign-on transaction until the terminal is signed on. |
| start_transaction | none | A transaction started automatically after a successful sign-on. |
| gmtext | Welcome to bricks | The good-morning text, up to 256 characters, returned by EXEC CICS INQUIRE SYSTEM GMMTEXT. |
| wrkarea | 0 | Length in bytes (0 to 3584) of the common work area returned by EXEC CICS ADDRESS CWA; ASSIGN CWALENG returns it. With 0, there is no CWA and the address is null. |
| time_zone | Z | Military time-zone letter (Z = UTC, A-M = UTC+1 to +12, N-Y = UTC-1 to -12) for EIBDATE, EIBTIME, the FORMATTIME date and time, and ASSIGN date values. ABSTIME is always UTC milliseconds since 1900-01-01. |
| ntp_server | time.google.com | Corrects the clock that ASKTIME and the EIB use. off uses the host clock. |
| log_location | log | The directory of the region log files and of abend.log, the transaction dumps that CAID analyses ("Transaction Dumps" in Chapter 16). A relative name is taken from the directory bricks was started in. off logs to the console only and writes no dumps. |

**Figure D-2. bricks.cnf knobs: terminals, sign-on and the region**

The directory and file knobs are listed in "The runtime Directory".

*Database*

| Knob | Default | Effect on programs |
|---|---|---|
| db_host | none | The PostgreSQL server. When it is not set, SQL is not configured and every EXEC SQL statement returns SQLCODE -1. |
| db_port | 5432 | The server port. |
| db_user, db_password | none | The PostgreSQL sign-on used by every task. A refused sign-on abends the task AD2S. |
| db_sslmode | disable | The PostgreSQL SSL mode (require, verify-full and so on). |
| db_max_conns | 8 | Connections in the pool of each database (1 to 1024). A task holds one from its first data statement until its unit of work ends; when none is free for 30 seconds the task abends AD2P. |
| db_stmt_timeout | 30s | Time limit for each statement, each cursor OPEN and each FETCH batch. When it expires, SQLCODE is -952. |
| db_retry_transient | yes | Retries the first statement of a database transaction after a deadlock or serialization failure (-911). |
| db_retry_max | 1 | Number of such retries (0 disables them). |
| databases_file | runtime/databases.conf | The list of databases; see "The databases.conf File". |

**Figure D-3. bricks.cnf knobs: database**

See "Connecting a Transaction to a Database" in Chapter 21 for the use of these values.

*Web listener (EXEC CICS WEB server commands)*

| Knob | Default | Effect on programs |
|---|---|---|
| enable_wapi | no | yes opens the web listener that starts transactions for HTTP requests. |
| wapi_port | 8080 | HTTP port. web_port is accepted as another name. |
| wapi_tls_port | 443 | HTTPS port, opened when tlscert and tlskey are set. |
| enforce_wapi_tls | no | yes opens the HTTPS port only. |
| web_routes_file | runtime/web_routes.conf | Routes and URIMAP definitions. |
| web_request_max_bytes | 1048576 | Largest request body; a longer one receives 413 and starts no transaction. |
| web_header_max_bytes | 65536 | Largest total size of the request headers. |
| web_request_timeout | 30s | Accepted and recorded; not enforced at present. |
| web_inbound_client_ca | none | PEM file of CAs. When set, the HTTPS listener requires a client certificate, which WEB EXTRACT CERTIFICATE returns. |
| webserver_port | 0 | Port of the static file server. When it is greater than 0, the web listener also serves webserver_dir for requests that match no route. |

**Figure D-4. bricks.cnf knobs: web listener**

*HTTP client (EXEC CICS WEB client commands)*

| Knob | Default | Effect on programs |
|---|---|---|
| `web_client_timeout` | `30s` | Time limit for each outbound request. When it expires the command returns `IOERR`. |
| `web_client_max_idle_conns` | `32` | Idle connections kept for reuse, for each host. |
| `web_client_ca_bundle` | none | PEM file of CAs used **instead of** the system trust store to verify servers. |
| `web_client_cert, web_client_key` | none | Client certificate and key for servers that require mutual TLS. Both must be set together. |
| `web_client_tls_skip_verify` | `no` | `yes` accepts any server certificate. For tests only. |

**Figure D-5. bricks.cnf knobs: HTTP client**

See "The HTTP Client" in Chapter 20 for the web commands.

### The Resource Definition Files

The files in `runtime/` share these rules:

- A line whose first non-blank character is `#` is a comment; a `#` preceded by a blank or a tab starts a trailing comment; a `#` with no blank before it is data. Blank lines are ignored.
- When a key is defined twice, the **first** definition is used and the later one is logged and ignored. A correction appended at the end of the file therefore has no effect; change the original row.
- A row that is not valid is logged with its line number and skipped; the rest of the file still loads.
- A byte-order mark at the start of the file is ignored. A row is limited to 1 MiB.
- `transactions.conf`, `users.conf`, `aliases.conf` and `web_routes.conf` are reread automatically when they change, so an edit takes effect without a restart.

The operator maintains most of these files with `CEDA` (see Chapter 6), which validates each change and rewrites the file safely. A file may also be edited by hand.

### The transactions.conf File

Each row defines one transaction:

```
TRANSID:LANG:PROGRAM[:GROUPS[:DATABASE[:TWASIZE]]][:[MRONAME]]
```

| Field | Meaning |
|---|---|
| TRANSID | Exactly four characters from A–Z and 0–9. The name of a supplied transaction (such as CEMT) cannot be defined. |
| LANG | cobol, pli or rexx, in either case. |
| PROGRAM | The program file, relative to the language's directory. |
| GROUPS | A comma-separated list of the groups that may run the transaction. PUBLIC admits everyone, signed on or not; * admits any signed-on user; any other name admits users who hold that group. An empty field admits everyone. A list that names only groups that no user holds denies the transaction to everyone. Group names use A–Z, 0–9, _ and –. |
| DATABASE | The database the transaction uses for EXEC SQL, a name from databases.conf, matched without regard to case. Empty means the default database. See "Connecting a Transaction to a Database" in Chapter 21. |
| TWASIZE | The size of the transaction work area in bytes, 0 to 32767, returned by EXEC CICS ADDRESS TWA. Empty means no TWA. |
| [MRONAME] | Optional, in brackets, as the last field: the transaction runs in the named peer region from mro.conf, which owns the program. |

**Figure D-6. Fields of a transactions.conf row**

```
SQLD:cobol:sqld.cob:PUBLIC,USERS,ADMIN
BANK:cobol:bank.cob:PUBLIC,USERS,ADMIN:bank
RPT1:pli:rpt1.pli:ADMIN::512
CSGM:rexx:csgm.rexx:PUBLIC,USERS,ADMIN:[dbreg1]
```

An empty field is written as two adjacent colons, as in the `RPT1` row, to reach a later field. A row with more than six fields (not counting the routing field) is refused, because a stray colon would otherwise shift every later field. The group check applies to every way a transaction is started: from a terminal, by `START`, and from the web listener.

### The users.conf File

```
userid:bcrypt-hash[:group1,group2,...]
```

The user's groups are what `GROUPS` in `transactions.conf`, the groups column of `web_routes.conf`, and `EXEC CICS QUERY SECURITY` test. The same entries authenticate a terminal user at `CSSN`, a web caller using HTTP Basic, and `EXEC CICS VERIFY PASSWORD`. Users are maintained with `CEDA USER`, which computes the hash.

### The aliases.conf File

```
alias:TRANSID
```

An alias is a name of 2 to 8 letters and digits that an operator may type instead of a 4-character transaction identifier, at the sign-on prompt or on a blank screen. Arguments typed after the alias are passed on unchanged, so `sabre JFK LAX` runs as `SABR JFK LAX`. A real transaction identifier always takes precedence over an alias, and an alias may not be the name of a supplied transaction. The file is optional; a missing file means that there are no aliases.

Aliases are a convenience for people at a terminal only. A program always names the real transaction identifier, in `RETURN TRANSID`, `START` and elsewhere.

### The databases.conf File

```
name[:description]
```

Each row names one PostgreSQL database on the server given by `db_host`. The name follows the PostgreSQL rules for an unquoted identifier (letters, digits and underscore, starting with a letter or underscore, at most 63 bytes) and is stored in lower case. The description is shown by `CEDA DATABASE`.

```
bricks:default application data
bank:retail banking demo
```

- The **first** row is the default database, used by every transaction whose `transactions.conf` row names none. `CEDA DATABASE` marks it `(def)` and does not allow it to be deleted.
- bricks opens a connection pool for every row at startup. It does not create or drop the databases themselves.
- Use `CEDA DATABASE` to change the list while bricks is running.

### The web_routes.conf File

The file holds two kinds of row, described fully in "Defining Routes: web_routes.conf" in Chapter 20:

```
METHOD  PATH-PATTERN  TRANSID  [groups]  [response_timeout]
URIMAP  NAME  scheme://host[:port][/path-prefix]
```

```
GET     /api/customer/{id}   WAPI   public
POST    /api/todo/lists      TODO   public
URIMAP  GITHUB               https://api.github.com
```

### The mro.conf File

`mro.conf` lists the peer regions to which transactions can be routed, one row per peer: `name:host:port:token[:pin]`. A program does not refer to this file; a transaction is routed by the bracketed field of its `transactions.conf` row. The setup of multiregion operation is described in `MRO.md`.

## Readers' Comments

**BRICKS Transaction Server**
**Application Programming Guide and Reference**
**BX26-1500-02**

Use this form to tell us what you think of this publication. Comments on its accuracy, organization, completeness and clarity are welcome: an error in a syntax block or an example, a condition that is not listed, a description that does not match what the program does.

Give the chapter and the page (the folio at the foot of the page, such as `11-4`) of each comment. Comments are most useful when they include the version of bricks in use and the source of a short program that shows the behavior in question.

Comments:

Name _____ Installation _____

Address or e-mail _____

Return this form to the administrator of your bricks region, or open an issue at `https://github.com/moshix/BRICKS_TS`. Comments may be used without obligation to the person who submits them.

**Copyright 2025, 2026 by moshix. All rights reserved.**